Covers Windows for Work Groups

Network Programming in Windows NT™

Alok K. Sinha



Network Programming in Windows NT™

Alok K. Sinha
Microsoft Corporation



ADDISON-WESLEY PUBLISHING COMPANY

Reading, Massachusetts • Menlo Park, California • New York
Don Mills, Ontario • Wokingham, England • Amsterdam • Bonn
Sydney • Singapore • Tokyo • Madrid • San Juan • Milan • Paris

This book is in the **Addison-Wesley UNIX and Open Systems Series.**
Series Editors: Marshall Kirk McKusick and John S. Quarterman

Senior Acquisitions Editor: Thomas E. Stone
Associate Editor: Deborah Lafferty
Savannah Consultants Editor: Bonnie Mae Savage
Production Supervisor: Nancy H. Fenton
Cover Designer: Barbara Atkinson
Production: Editorial Services of New England
Project Manager: Bonnie Jo Collins
Text Designers: Pat Nieshoff/Nieshoff Design, Mark Heffernan
Text Figures: George Nichols
Copy Editors: Sandra Sizer Moore, Beverly Miller
Senior Manufacturing Manager: Roy Logan

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and Addison-Wesley was aware of a trademark claim, the designations have been printed in initial caps or all caps.

The programs and applications presented in this book have been included for their instructional value. They have been tested with care, but they are not guaranteed for any particular purpose. The publisher does not offer any warranties or representations, nor does it accept any liabilities with respect to the programs or applications.

Figures 2-1, 2-2, 2-3, 2-4, 2-6 and 2-7 in Chapter 2 are from H. Custer, *Inside Windows NT™*, © 1993. Microsoft Press, Redmond, WA. Reprinted with permission.

Access the latest information about Addison-Wesley books from our Internet gopher site or visit our World Wide Web page:
`gopher aw.com`
`http://www.aw.com/cseng/`

Library of Congress Cataloging-in-Publication Data
Sinha, Alok.
Network programming in Windows NT / Alok Sinha.
p. cm.
Includes bibliographical references and index.
ISBN 0-201-59056-5
1. Microsoft Windows NT. 2. Computer networks. I. Title.
QA76.76.063S567 1996
005.2-—dc20 95-31136
CIP

Copyright © 1996 by Addison-Wesley Publishing Company

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of the publisher.

Printed in the United States of America

1 2 3 4 5 6 7 8 9 10—MA—99 98 97 96 95

Dedication

To Bonnie Mae Savage

Contents

Preface *ix*

Acknowledgments *x*

One: Introduction *1*
1.1 Historical Perspective 4
1.2 Impact of Windows NT Design on Users 5
1.3 Impact of Windows NT Design on Developers 6
1.4 Client/Server Computing with Windows and Windows NT 8
1.4.1 Communication Methods in Windows and Windows NT 11
1.5 Summary 20

Two: Understanding Windows NT Architecture *21*
2.1 Salient Windows NT Architecture Features 21
2.1.1 Windows NT Executive 23
2.1.2 Windows NT Protected Subsystems 24
2.2 Networking in Windows NT 27
2.2.1 Windows NT Networking Components 27
2.2.2 Accessing Network Resources 30
2.2.3 Accessing Network Resources from Applications 31
2.2.4 Multiple Network Providers 32
2.2.5 Windows NT Server and Domain Controller 34
2.2.6 Access Control and Network Security 37
2.2.7 Interprocess Communication in Windows NT 40
2.2.8 Networking Within VDM and Windows on Win32 42
2.3 Salient Win32 Programming Features 42
2.3.1 Error Handling in Windows NT 43
2.3.2 Structured Exception Handling 44
2.3.3 Threads 46
2.3.4 Interprocess and Intraprocess Synchronization 51
2.3.5 Overlapped I/O 64
2.3.6 Memory-Mapped File I/O and Sharing Memory 72
2.3.7 Control C Handlers 79
2.4 Creating an Echo Server 81
2.5 Summary 97

Three: Understanding Windows Architecture 99
3.1 Windows Operating Modes 100
3.2 Networks in Windows for Workgroups 105
3.2.1 Enhanced-Mode Windows for Workgroups 106
3.2.2 Standard-Mode Windows for Workgroups 109
3.3.3 Accessing Network Resources 109
3.3 Summary 114

Four: RPC Programming in Windows NT 115
4.1 Historical Perspective 116
4.2 Microsoft RPC Concepts 118
4.3 Writing a Simple RPC-Based Client and Server 121
4.4 Designing an Efficient RPC Client and Server 124
4.4.1 Base Types 126
4.4.2 Arrays 127
4.4.3 Strings 129
4.4.4 Structures 131
4.4.5 Pointers 132
4.4.6 Nested Arrays 137
4.4.7 Unions 137
4.4.8 Using transmit_as 139
4.5 Designing Communication Infrastructure 141
4.5.1 Mechanics of Connecting Client and Server 145
4.5.2 Using Name Service 160
4.5.3 About Binding Handles 170
4.6 Using Callback Routines 178
4.7 RPC Management 182
4.8 RPC Error and Exception Handling 184
4.9 Secure RPC 188
4.10 Echo Server—RPC-Based Win32 Server 191
4.11 RPC Programming in Windows 196
4.12 Summary 198

Five: Windows Sockets in Windows NT 199
5.1 Introduction to Sockets Programming 201
5.1.1 Creating a Socket 203
5.1.2 Associating Address with a Socket or Binding 206
5.1.3 Server Accepting Calls 219
5.1.4 Client Connecting to Server 224
5.1.5 Address Resolution in TCP/IP Environment 229
5.1.6 Sending and Receiving Data on Stream Socket 232
5.1.7 Sending and Receiving Data on SOCK_DGRAM Socket 238
5.1.8 Terminating a Connection 246
5.1.9 Asynchronous Operations 247
5.1.10 Out-of-Band Data Processing 254

5.2 Windows Sockets Extensions 255
5.2.1 Startup and Cleanup Functions 256
5.2.2 Error-Handling Functions 259
5.2.3 Functions for Handling Blocked Sockets 260
5.2.4 Asynchronous Database Functions 262
5.2.5 Asynchronous Select 267
5.3 Interoperability with IPX/SPX-Based Applications 277
5.4 Name Service Provider APIs 280
5.5 A Windows-Socket-Based Win32 Service 282
5.6 Guidelines for 16-bit Windows Sockets Applications 298
5.7 Summary 299

Six: Pipes in Windows NT 301
6.1 Named Pipes in Windows NT 301
6.1.1 Platforms Supported 303
6.1.2 Simple Named Pipe Client and Server 305
6.1.3 Creating a Named Pipe 306
6.1.4 Accepting Incoming Open Requests from Clients 309
6.1.5 Opening a Pipe 317
6.1.6 Reading/Writing Named Pipe 322
6.1.7 "Peeking" into a Named Pipe 328
6.1.8 Transaction-Oriented Client/Server Applications 333
6.1.9 Dynamically Getting and Setting Pipe Mode 344
6.1.10 Using Information Available on Named Pipe 346
6.1.11 Using Events and Overlapped I/O 347
6.1.12 Using Asynchronous Pipes 356
6.1.13 Multiple Instance Management 357
6.1.14 Security Considerations with Named Pipes 364
6.2 Anonymous Pipes 366
6.2.1 Creating an Anonymous Pipe 367
6.2.2 Reading/Writing Anonymous Pipe 368
6.3 Win32 Service Using Named Pipes 373
6.4 Named Pipes in Windows 385
6.5 Comparison with UNIX Pipes 389
6.6 Summary 390

Seven: Using Mailslot in Windows NT 393
7.1 Architecture of Mailslot 395
7.2 Programming Mailslot 397
7.2.1 Creating a Mailslot Server 397
7.2.2 Creating a Mailslot Client 401
7.2.3 Efficient Use of Mailslot 405
7.3 A Win32 Service Using Mailslot 408
7.4 Mailslots in Windows for Workgroups 423
7.5 Summary 424

Eight: NetBIOS Programming in Windows NT 425
8.1 Overview of NetBIOS Programming 425
8.1.1 Elements of an NCB 431
8.2 NetBIOS Support in Windows NT 435
8.3 NetBIOS Programming in Windows NT 437
8.3.1 Setting the NetBIOS Application Environment 437
8.3.2 Name Management Services 440
8.3.3 Exploiting Session Service 449
8.3.4 Designing Programs Using Asynchronous Commands 462
8.3.5 Handling Multiple Clients and Large Data Transfers 480
8.3.6 Communication Using Datagram Services 487
8.4 A NetBIOS-Based Win32 Service 496
8.5 NetBIOS Programming in Windows 507
8.6 Summary 510

Nine: SPX/IPX Programming in Windows NT 511
9.1 An Overview of Novell NetWare 512
9.2 NetWare Services 515
9.3 IPX Programming 517
9.3.1 IPX Programming APIs 518
9.3.2 Data Structures Relevant to IPX Programming 520
9.3.3 Asynchronous Processing of ECBs 524
9.3.4 Initializing and Deinitializing an IPX Application 528
9.3.5 Opening and Closing IPX Sockets 529
9.3.6 Receiving IPX Packets 531
9.3.7 Locating a Specific Server Receiving IPX Packets 532
9.3.8 Sending Datagram Packets 536
9.3.9 IPX Sample Program 538
9.4 SPX Programming 554
9.4.1 Data Structures and Asynchronous Processing 557
9.4.2 Initializing SPX Applications 559
9.4.3 Opening and Closing SPX Sockets 560
9.4.4 Listen For Connections 560
9.4.5 Establishing a Connection 563
9.4.6 Breaking a Connection 564
9.4.7 Send and Receive Data 565
9.4.8 SPX Sample Program 567
9.5 Summary 582

Appendix A: Determining NetBIOS System Characteristics 583

Appendix B: Multicasting with Windows Sockets 593

Bibliography 603

Index 605

Preface

I was not always interested in networking, or computer science for that matter. In fact, I received a master of science degree in mining engineering from University of Alaska, Fairbanks. After graduating, I was looking for something interesting to do while my would-be wife completed her graduate degree in civil engineering, so I signed up to get a graduate degree in computer science. The first year's courses were full of complexity theory and algorithms, both of which I enjoyed but they didn't thrill me. In the second year, I took the Networking Architecture class from an unorthodox professor, Dr. Peter Knoke, who had recently come to academia after decades of industry work. Instead of following the prescribed book, he narrated the latest technology developments straight out of trade magazines and newspapers (the *Wall Street Journal* was his favorite). Our individual projects involved research in the latest areas of networking, summarized in in-depth presentations. I picked local area networks as my topic and scoured all the trade magazines for information. When I hit an unknown term, such as "router," I would pore over the books and research papers to understand the technology and architecture. By the end of the semester, I had a broad knowledge not only of the theory of networking but also of the latest technology. Needless to add, I was hooked on networking and have spent most of my professional career in and around networking—from Microsoft LAN Manager to Directory Service to interactive TV applications. Most important, however, Dr. Knoke succeeded in showing his students how to learn on their own in a rapidly changing industry—a vital skill for surviving in the computer industry.

The Audience

This book is targeted toward software developers and project managers who want to learn about using the interprocess communication (IPC) mechanisms offered by Windows and Windows NT for creating client/server applications. Actually, students will also find this book useful in developing a new generation of networking applications in their laboratories. I assume that the reader is familiar with Microsoft Windows and Windows NT programming, has working knowledge of C programming, and understands the basics of networking.

The first three chapters of the book present the background for network programming in Windows and Windows NT environments. These chapters discuss the networking architecture of Windows for Workgroups™ and Windows NT as well as certain advanced programming features such as Win32 services. The other six chapters explain the concepts and use of these IPC mechanisms in Windows and Window NT environments: Remote Procedure Call (RPC), Windows Sockets, Named Pipe, Mailslot, Network Basic Input/Output Specification (NetBIOS), Internet Packet Exchange™/ Sequenced Packet Exchange™ (IPX/SPX).

Within Chapters 4 through 9, I first present the basic concepts and a primary-use model of an IPC mechanism in the Windows NT environment and then proceed to convey the details of using the given IPC mechanism by explaining the necessary system calls, showing sample programs illustrating programming details, and unfolding intricate details wherever pertinent. I then present any special programming details necessary to use the IPC mechanism from an application in the Windows environment.

Network programming is like any other discipline of computer science wherein one optimizes every line of code, tweaks every algorithm, to maximize efficiency and performance. But it differs from other disciplines in that a significant amount of effort needs to be spent on anticipating the inevitable network and software failures. Such upfront concern and understanding allow you to build software so robust that your applications continue to function despite such failures.

Acknowledgments

This book takes its current form due to the constructive critiques of Bonnie Mae Savage, the local editor and program manager at Savannah in Redmond, Washington, who shaped the focus and the content of the book. Also formative were the constructive critiques of the reviewers and the editorial talents at Editorial Services of New England. Together, they have all molded a raw technical blurb into a coherent, extremely readable, and I hope a downright enjoyable networking book. I thank their persistence, patience, tenacity, and every other human virtue I forgot to mention! In addition, I must thank Tom Stone at Addison-Wesley for accepting my half-page e-mail as the table of contents for almost half a year! I also thank all individuals who have directly and indirectly helped this book: Deborah Lafferty from Addison-Wesley for her patience and good counsel; Roger Soles from Oracle Corporation; and James "J" Allard, Manny Weiser, David Treadwell, Ryszard Kott, Margaret Johnson, and Jim Allchin from Microsoft Corporation.

Alok Sinha
August 1995

Chapter One

INTRODUCTION

Microsoft® Windows™ operating system is the predominant **personal computer (PC)** operating system, and one of the most popular desktop environments in use today. Windows users can run their productivity tools, such as word processors and spreadsheets, in a consistent, simple graphical user environment. However, the original Windows system can only be used on PCs containing Intel® x86-based processors. Further, Windows is not suitable for computing-intensive applications, such as **computer-aided design (CAD)** applications, nor is it robust enough to handle "mission-critical" applications such as accounting and payroll. Although such applications are not appropriate for the original Windows operating system, they can be hosted on the Microsoft Windows NT™ operating system.

Whether they are running the Windows or Windows NT desktop operating system, networked users need to access file servers to store information and print servers to print documents. To do either of these tasks, most users connect their PCs to a **local area network (LAN)**. Today's LAN-based file servers and print servers provide access to information storage and printers, and also to many other networked resources. There is a growing demand for desktop access to such new services as database servers, client/server computing, and the Internet. For these reasons, software developers are now focusing on a new breed of applications. In this next level of computing, applications running on the desktop interact with other applications that might reside on one or more remote machines. This defines network computing, in which applications on one machine communicate with their counterparts on a remote machine using an **interprocess communication (IPC)** method.

This book, *Network Programming in Windows NT*, introduces software developers, network administrators, and others to the methods of building client/server systems on Windows and Windows NT IPC interfaces. It also provides insights into the network architectures of Windows and Windows NT. By understanding the basic architectures of these environments and by following the explanations of the IPC mechanisms, you should be able to design and implement efficient Windows and Windows NT applications that can harness the power of network computing.

Network Programming is intended primarily for people in the software development community who are (or who plan to be) designing client/server software for Windows and Windows NT environments: software developers writing new applications for Windows and Windows NT environments; engineers porting applications from UNIX® and OS/2™ environments to the Windows NT environment; project managers involved in developing Windows and Windows NT-based applications; software consultants; and students who are using Windows and Windows NT environments to learn the principles of networking. To benefit the most from this book, you should be familiar with Windows and Windows NT programming, have a working knowledge of the C programming language, and a basic understanding of networking. Some advanced sections of the book may be easier to grasp if you are familiar with the concepts of object-oriented programming. All sample programs are written in C and have been compiled in Microsoft Visual C™ 2.0 environment for Windows and Windows NT. All sample programs can be downloaded from **ftp://ftp.aw.com/cseng/sinha/windowsnt** via anonymous **file transfer protocol (FTP)** on the Internet.

The book is organized to help you understand each IPC mechanism quickly. The first three chapters present the background for network programming in Windows and Windows NT environments. They cover the networking architecture of **Windows for Workgroups™ (WFW)** and Windows NT and certain advanced programming features. Readers already familiar with these concepts may want to skip these chapters or use them as a reference. However, the remaining chapters assume the knowledge of these materials.

Chapters 4 through 10 explain the concepts and use of IPC mechanisms in Windows and Windows NT environments: **remote procedure call (RPC)**, Windows Sockets, Named Pipe, Mailslot, **Network Basic Input/Output Specification (NetBIOS)**, and **Internet Packet Exchange™/Sequenced Packet Exchange™ (IPX/SPX)**. Each chapter begins with the basic concepts and a primary-use model of the IPC mechanism in the Windows NT environment. It examines the details of using the IPC mechanism, explains the necessary system calls, shows sample programs illustrating programming details, and unfolds intricate details wherever pertinent. It presents any special programming details necessary for using the mechanism in an application in the Windows environment and ends with a summary of the IPC mechanism's highlights.

Each chapter discusses issues that can affect the reliability or the performance[1] of an application when you use a feature, and suggests ways you can improve reliability and performance. Whenever pertinent, I present comparisons

1. Performance of a network-aware application is often measured in terms of a number of matrices, such as the time delay involved in network I/O, the resource overhead per user, the total data throughput per second, and so on.

among features of the different IPC mechanisms, but I do not compare the performance of one IPC mechanism with another in Windows or Windows NT environments. (Given the wide range of possible test scenarios, performance matrices, and software upgrade packages for Windows and Windows NT, performance comparisons between one IPC with another are outside the scope of this book.) I also briefly address Windows NT security concepts, which can be used to design secure software systems; security is an important consideration in any network-aware software system.

Throughout this book, the term "communication methods" refers to IPC over a network by some physical medium such as Ethernet. (For the purposes here, other communication methods, such as serial communication by **communications (COM)** ports and modems, are irrelevant.) Whenever specific differentiation is not necessary, the versions of MS-DOS-based Windows (Windows 3.0,™ Windows 3.1,™ Windows 3.11,™ and Windows for Workgroups 3.11,™ etc.) are referred to as Windows. Similarly, unless explicitly stated otherwise, Windows NT refers to all versions on Windows NT platforms (Windows NT 3.1, Windows NT 3.5 Workstation, Windows NT 3.5 Advanced Server, and so on). The evolution of the Windows and Windows NT operating systems is illustrated in Figure 1-1. Finally, the term Windows NT Workstation is used except when the distinction between Windows NT Workstation and **Windows NT Advanced Server (NTAS)** products is important for the discussion at hand.

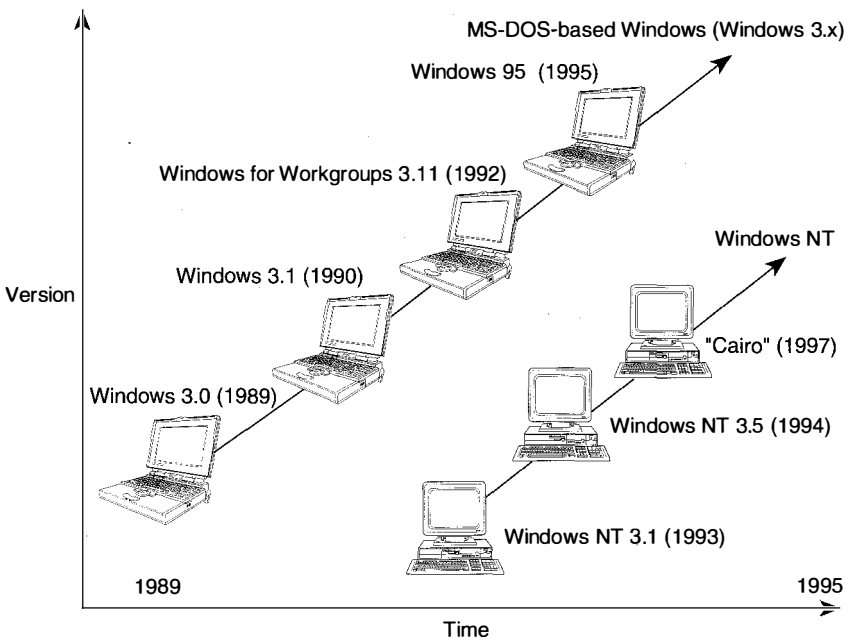


Figure 1-1. *Evolution of Microsoft Windows and Windows NT Operating Systems*

1.1 HISTORICAL PERSPECTIVE

The arrival of the Windows 3.0 operating system in 1989 caused a major shift in the way people used IBM®-PC compatibles, a shift no less important than the introduction of the PC in the early 1980s. Before the widespread use of Windows, PCs were operated by a simple **Microsoft disk operating system (MS-DOS®)**, consisting of a command shell with a series of nonintuitive commands and utilities. Most users found MS-DOS unwieldy and nonfriendly. Furthermore, the MS-DOS environment did not specify any user interface guidelines. Thus, each MS-DOS–based application had its own user interface, which usually bore little resemblance to other applications. Today, most software houses and **independent software vendors (ISVs)** are delivering their second or third generation of Windows-based applications. As you walk down the aisles of your neighborhood stores, you can easily spot rows of computers running Windows. What promoted such popular acceptance of Windows 3.0 was its simple, user-friendly **graphical user interface (GUI)** and consistent use of Windows **application programming interfaces (APIs)**.

The Windows operating system simplified and standardized the look and feel of all Windows-based applications. Although it was not always stable, the Windows 3.0 operating environment was complete enough that users did not have to face the MS-DOS prompt ever again—unless they wanted to. Windows, in establishing a consistent look and feel for application GUI and by standardizing APIs, made it possible for users to adapt quickly to Windows-based programs. This shift should not surprise anyone, given the success of Apple Macintosh® (starting in 1984) and the proven work on user interfaces done at Xerox® Labs by Alan Kay and his group. GUIs just took time to arrive in the PC market because of the sheer lack of standards, the shortage of sophisticated software and hardware, and the inability of the major players to forge agreements.

Another reason for the success of Windows has been the immense number of application programming interfaces developed for it, about 700 in Windows 3.0 and about 1,100 in Windows 3.1. These standardized interfaces have allowed the applications to use common ways to access resources and system services. They have also reduced the time needed to develop applications.

Windows does have some limitations as an operating system. It is based on MS-DOS and allows only nonpreemptive multitasking. Thus, an errant application can refuse to relinquish control of the **central processing unit (CPU)** while starving other applications of computing cycles. This problem has been resolved in newer versions of Windows (Windows 95) as well as in Windows NT. Another issue was that networking capabilities were not tightly integrated within Windows 3.0. To solve networking woes with Windows, Microsoft offered Windows for Workgroups 3.11 in 1992. This package not only made networking easier, but it also allowed peer-to-peer communication, file sharing and printing, and mail among members of a workgroup.

In 1985, nearly parallel to the Windows evolution, Microsoft and IBM began developing a true multitasking operating system with a GUI and based on a new architecture, not MS-DOS. This work led to early versions of IBM OS/2.™ IBM's version did not attract a major following for many reasons. In 1989, rather than fixing OS/2, Microsoft started designing a new operating system. The new system is known today as Windows NT.[2] This multitasking operating system is designed to:

- Port to varying hardware, including the **reduced instruction set computing (RISC)** processors
- Scale to multiple processor systems
- Integrate with networking, thus allowing client/server computing capability
- Be POSIX[3] compliant
- Meet C2[4] government security certification

While Windows NT has a completely new architecture, its designers made some major decisions that empower users and application developers alike. First, Windows NT has the same user interface as that of MS-DOS–based Windows. Second, most existing Windows and MS-DOS applications can be executed on Windows NT regardless of the underlying hardware. For example, MS-DOS applications can run in Windows NT on a MIPS® RISC machine. In short, Windows NT is largely backward compatible.

Thus, Microsoft has provided two operating systems that incorporate similar user and application programming interfaces. Because Windows NT has been designed to be the state-of-the-art operating system that is also backward-compatible with MS-DOS–based Windows, users can run Windows on computers with limited resources, and they can use Windows NT on high-end computers as shown in Figure 1-2.

1.2 IMPACT OF WINDOWS NT DESIGN ON USERS

The Windows NT operating system, much like the UNIX operating system, can run on diverse hardware platforms. The user interface remains the same across all Windows systems; users can move easily from one system to another, and quickly learn and adapt to new applications. While businesses use Windows operating systems for running productivity applications, such as word processors and spreadsheet applications, the scientists, engineers, and

2. See Helen Custer's book *Inside Windows NT* for insight into the design of the NT operating system.
3. POSIX is the portable operating system interface based on UNIX, as defined by IEEE standard 1003.1-1988.
4. See Department of Defense Trusted Computer System Evaluation Criteria, DOD 5200.28-STD, for details.

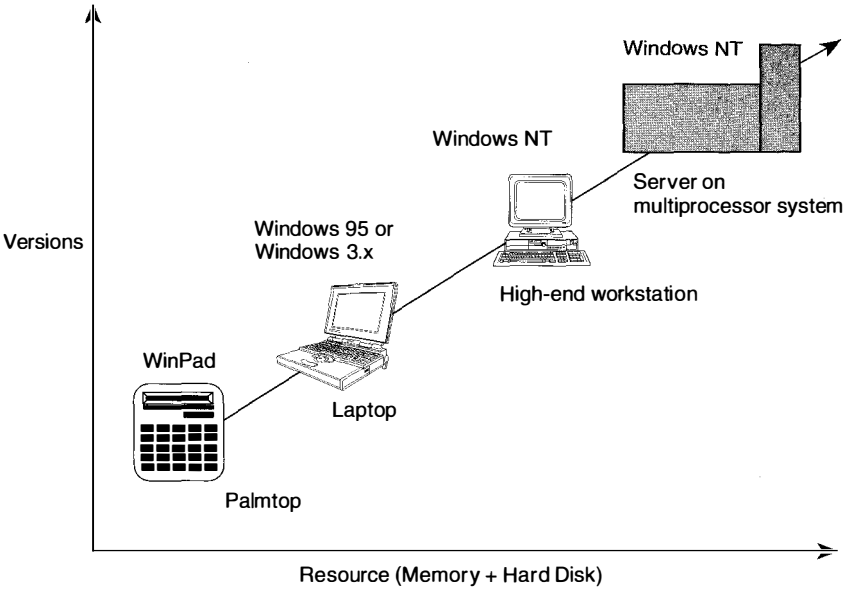


Figure 1-2. *Windows and Window NT on Different Types of Hardware*

power users use Windows NT for their computing-intensive applications, such as real-time process control, stock-market financing and forecasting, and important applications such as file servers and database servers.

1.3 IMPACT OF WINDOWS NT DESIGN ON DEVELOPERS

Windows NT can run a multitude of applications, each of which depends on different interfaces—Win16 API, Win32 API, OS/2 API, MS-DOS API, POSIX API, and Windows NT API. This flexibility is illustrated in Figure 1-3.

Win16 Application Programming Interfaces: These interfaces ensure all Windows applications written to 16-bit Windows APIs can be executed on Windows NT. (Nothing undermines the success of an operating system more than not having good applications for it, so this is a big boost to Windows NT.) This is especially important for the ISVs who currently produce Windows applications, because they can continue to meet their customer needs even if the customers move to the Windows NT platform. Chapter 2, "Understanding Windows NT Architecture," explains in detail the mechanism by which the Win16 interfaces are supported on Windows NT.

Win32 Application Programming Interfaces: Windows NT is a 32-bit operating system. To take full advantage of its power, Windows NT exports 32-bit versions of Win16 interfaces. (Of course, an application written to Win32 APIs will perform better than their Win16 counterparts.) At run time, a process called "thunking" maps Win16 API calls to Win32 calls. To make it

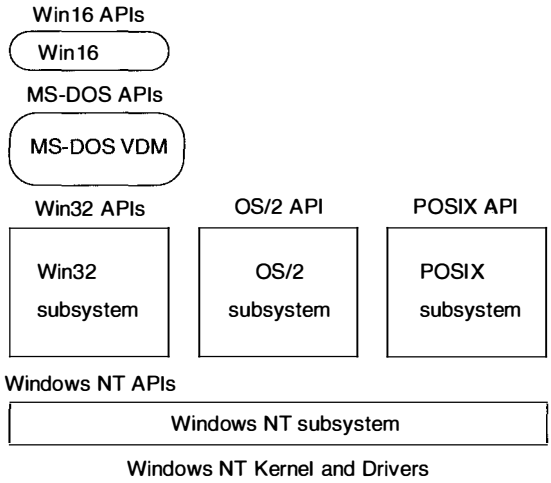


Figure 1-3. *Windows NT Programming Interfaces*

easy to gradually port Win16 applications to a Win32 environment, the arguments and return types of Win32 interfaces are designed to be almost identical to those of the Win16 interfaces, although most arguments and return types are 32-bit quantities in the Win32 environment. It is therefore possible to create a Win32 application from a Win16 application by recompiling the source, provided the original application is a well-written, "well-behaved" Win16 application. In practice, some changes might be necessary, but they will be minimal.

Win32 interfaces also provide advanced operating system features, such as semaphores, events, and threads. Hence, it is prudent to implement enhancements in the application so it uses the advanced features given by Win32 APIs. Chapter 2, "Understanding Windows NT Architecture," has more details.

MS-DOS Application Programming Interfaces: Windows NT supports most MS-DOS calls (for example, INT 21h), so most of the popular MS-DOS applications can be run on Windows NT. Some MS-DOS interfaces are not allowed in Windows NT.[5] These include the interfaces that allow applications to directly interact with the hardware, such as the video or the hard disk.

OS/2 Application Programming Interfaces: Windows NT supports most OS/2 APIs, including console-based **I/O (input/output)** APIs. Currently, it does not support **Presentation Manager™ (PM)** interfaces, but Microsoft expects to make PM subsystems available in the near future.[6]

5. See [MS 93a] for details.
6. See [MS 93a] for details on the OS/2 APIs supported by Windows NT.

POSIX Application Programming Interfaces: Windows NT supports most POSIX standard interfaces, making Windows NT a viable platform for UNIX users who want to migrate to a platform that can run their favorite UNIX applications as well as Windows applications.[7]

Windows NT Programming Interfaces: Although most application writers may stay away from low-level Windows NT APIs, some advanced applications, and typically device-driver writers, will need to use these interfaces, because Windows NT APIs are the foundation blocks for Win32 APIs.[8]

1.4 CLIENT/SERVER COMPUTING WITH WINDOWS AND WINDOWS NT

This section defines the concept of client/server computing and describes some of the key features of Windows NT that make it one of the ideal platforms for client/server systems.

First, let us define a **client/server system (CSS):** *In the client/server computing paradigm, one or more clients and one or more servers, along with the underlying operating system and interprocess communication systems, form a composite that allows distributed computation, analysis, and presentation.*

Figure 1-4 shows the anatomy of a CSS.

In this system, the term "client" refers to a process that interacts with the user and has the following characteristics:

- It presents the user interface, which is often a GUI.
- The client has sufficient local intelligence to present intelligent queries to the server, and it can form queries or commands in a predefined structure for presentation to the server, based on the user input. For example, unlike a terminal emulation program, it generally does not relay every mouse click to the server.
- The client communicates with the server through an IPC method and uses this IPC to transmit queries or commands to the server.
- It performs data analysis on the results sent by the server and presents those results to the user in an application-defined format.

In a CSS, a server is a process, or a set of processes, all of which can exist on a single machine that provides a service to one or more clients. The server has the following characteristics:

- It works for the client. The nature and extent of its service is defined by the business goals of the CSS. For example, an accounting

7. See [MS 93a] for POSIX support on Windows NT and [Quarterman 93] for details on POSIX standards.
8. See [MS 93b] for details on Windows NT interfaces.

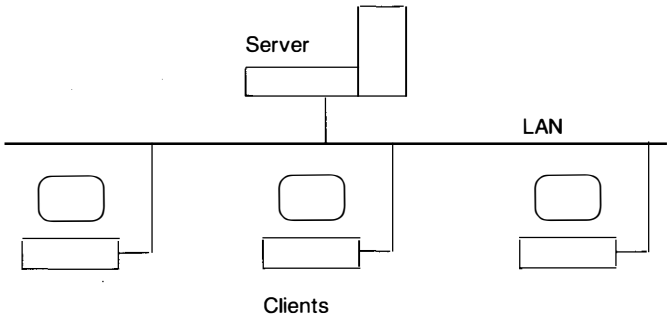


Figure 1-4. *Anatomy of a Client/Server System*

CSS server offers an accounting data retrieval and processing service to its clients.

- It merely responds to the queries or commands from the clients. It does not initiate a conversation with any client, although there can be a dialogue between client and server after the initial contact has been made by the client. A server primarily acts as a repository of data (for example, a file server) or knowledge (for example, a database server), or as a service provider (for example, a print server).
- The ideal server hides the composite CSS from both client and user. A client communicating with a server should be completely unaware of the server's hardware and software platform, as well as the communication technology involved. This allows the server to change or scale as demands change, without affecting the client.[9]

With a well-established GUI and connectivity support to most networks, Windows and Windows NT make good candidates for a client desktop that hosts client/server applications. At the same time, Windows NT can be an excellent server platform because it incorporates these advanced operating system features:

Networking Integration: Networking is tightly integrated with the Windows NT operating system, making an efficient, highly secure server. This makes an ideal platform for a CSS.

Standard Protocol Support: Windows NT provides support for the most common protocols (for example, **Transmission Control Protocol/Internet Protocol [TCP/IP]**, IPX/SPX, **NetBIOS Extended User Interface [NetBEUI]**, AppleTalk®) and the most common network adapters (for example, Token Ring and Ethernet).

9. "Replicated" database servers are becoming available commercially. In such systems, the database tables can reside on multiple physical machines. See [Sinha 92a] for references to papers on distributed database systems.

File Server and Printing Features: The Windows NT server has all the features of a fully functional file and print server. In addition, Windows NT can provide file and print support to remote dial-in clients and MacIntosh-based clients.

Security, Auditing, and Error Logging: Windows NT has been designed with security features built in. These features allow the network administrator to control all resources on the server. With a domain-centric model and a trust relationship between domain controllers, total security is both effective and easy to administer. Windows NT also provides varying levels of auditing and error logging. (See the brief discussion of security in Chapter 2, Section 2.2.6, "Access Control and Network Security.")

Network Management: Windows NT provides a complete system of network management features. Administrators can have total management and inventory control over all the workstations. Administrators can also connect a Windows NT server to a back-end mainframe so the complete LAN can be centrally monitored. Windows NT workstations and servers can be monitored using **Simple Network Management Protocol (SNMP)**.

Backup: An essential part of any CSS is the ability of the system to be restored to its original state, should a disaster occur. Windows NT provides easy-to-use tools to back up and restore data. It also provides a new file system, **Windows NT File System (NTFS)**, which has built-in data recovery features.

Scalability: This is one of Windows NT's major strengths. As the number of clients or the size of the server data grows, it is easy to add higher capacity disks (or smart disk arrays), multiple processors, and additional memory to accomodate higher demand. Windows NT is designed to support up to four gigabits of memory, 17 billion gigabits of disk space, and **symmetric multiprocessing (SMP)**.

Fault Tolerance: This feature provides services that can make a Windows NT machine a fault-tolerant server. These services include disk mirroring support, disk stripping, and **uninterrupted power supply (UPS)**.

Protected Systems: In Windows NT, every Win32 application has its own address space and input queue and is completely independent of other applications. Hence, no application can affect the security or stability of the entire system. If an application crashes, the effects are limited to the resources owned by that application. All Win16 and MS-DOS applications are run in a special subsystem that is itself a Win32 protected system. Hence, ill-behaved Windows or MS-DOS applications cannot compromise other applications executing on the Windows NT workstation.

Advanced Features: These include threads, events, semaphores, asynchronous procedure calls, spin locks for multiprocessor support, and other operating system features that allow programmers to develop powerful client/server applications. These features are discussed in subsequent chapters.

Variety of Standard Communication Methods Available: Designers have several choices among communication methods for a CSS. These methods and their uses in Windows and Windows NT are discussed in later chapters.

1.4.1 Communication Methods in Windows and Windows NT

In the CSS described in Section 1.4, the clients communicate with the server by IPC mechanisms. These IPCs can vary according to which operating system (for instance, MS-DOS, Windows, or UNIX) is hosting the CSS. The underlying network also affects the availability of a communication method. For example, applications in a Novell NetWare® environment can always rely on using the IPX protocol to communicate, whereas NetBIOS is the ubiquitous low-level protocol in an IBM LAN environment. This section first covers general networking terminologies and concepts. (If you are already familiar with terms such as OSI or virtual circuits, proceed to the next subsection.) We then present a brief overview of the most common communication methods available in the Windows and Windows NT environments. (Subsequent chapters will explore the details of each of these methods.)

General Networking Terms and Concepts

Open Systems Interconnection (OSI) is a reference model developed by the **International Standards Organization (ISO)** that provides a layered abstraction from network software and hardware. It is based on two simple concepts.

Peer-to-peer communication. Each layer assumes it is communicating with its peer on the remote machine. Each layer is unaware of any other layer on the remote machine, talking only with its peer in a standard or negotiated protocol known to both parties. Figure 1-5 illustrates this peer-to-peer communication.

Protocol stack. Each layer of the network services its upper layer, exposing a set of interfaces to the upper layer and hiding all implementation details within itself. Together, the layers on a machine form a protocol stack.

Peer-to-peer communication is achieved through data encapsulation. Each layer (the service requester) generally wraps an envelope, or "frame," around the data to be transmitted and passes it to the lower layer (the service provider). The frame contains header information that is understood only by the peer layer on the remote machine. When a layer receives a frame, it reads in the header information and then interprets the header information for its own service controls. Finally, it strips the header out of the frame and passes the rest of the packet to the upper layer, as shown in Figure 1-6.

Each layer can provide either a connection-oriented service or a connectionless service to the upper layer. Connection-oriented service is like a telephone connection in which the two people who are connected are unaware of

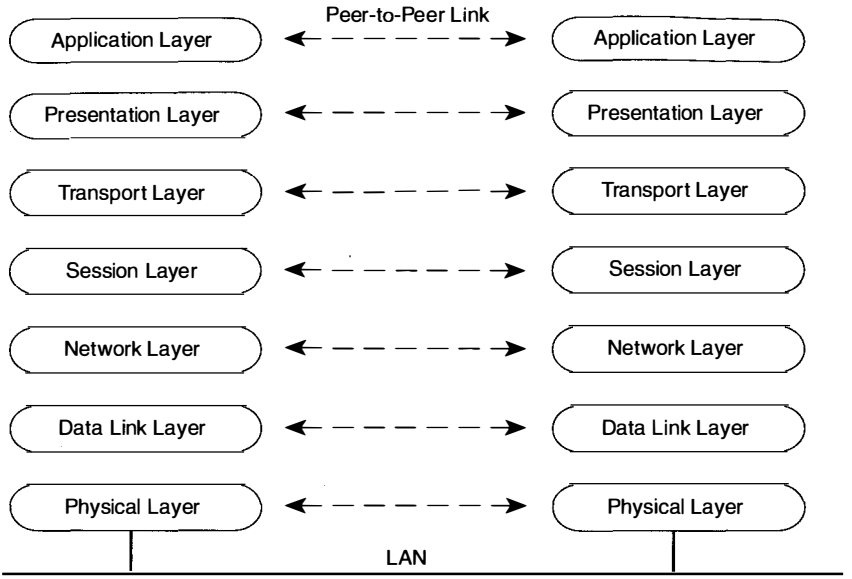


Figure 1-5. *OSI Reference Model*

the underlying telephone network. A connection-oriented service sets up a "virtual" connection between the two parties at the beginning of communication. During the connection's lifetime, messages are passed back and forth in an ordered,[10] error-free[11] manner. The connection ceases when either end wishes to terminate the session. Thus, the connection-oriented service constructs a virtual circuit between the sender and the recipient.

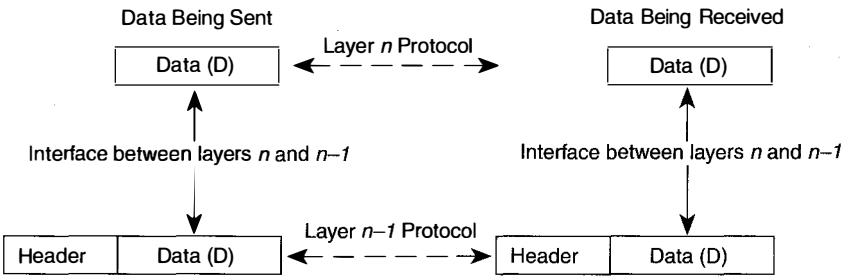


Figure 1-6. *Data Encapsulation Between Layers*

10. In an "ordered" connection, the packets are received in the order in which the sender transmits them. However, this does not mean the packets travel through the network in order.
11. A service provider guarantees error-free transmission by using a number of schemes, such as acknowledging the receipt of messages, checksums, parity bits, and so on.

A connection-oriented service can provide a reliable message stream, in which message boundaries are maintained; a reliable byte stream, in which message boundaries are not maintained; or an unreliable connection, in which messages may not be acknowledged due to time-delay constraints. Traditionally, FTP uses reliable message streams; terminal emulation programs use reliable byte streams; and some voice communication programs use unreliable connections, because waiting for acknowledgment increases the turnaround time.[12]

Connectionless service is more like the postal service that makes its "best effort" to get letters from the senders to the recipients. In this case, each letter routed to the recipient is independent of preceding and following packets. In fact, two letters mailed at the same time and destined for the same recipient may follow completely separate routes and arrive out of order. Thus, the connectionless service provides what is called a datagram service. Unlike virtual circuits, datagrams do not incur the cost of connection setup.

The connectionless service can provide an unreliable datagram service, in which the recipient is not guaranteed to receive the datagram; an acknowledged datagram service, in which the sender can be assured the receiver will get the datagram (much like the registered letter service); or a request-reply service, in which a sender sends a request by a datagram and the reply contains the answer [Tanenbaum 88].

Note that it is easy to build a connection-oriented service on top of a connectionless service. For example, TCP, a connection-oriented protocol, uses the connectionless services provided by the **Internet Protocol (IP)**.

Look again at Figure 1-5. There are several layers in the OSI model:

- The physical layer is responsible for transmitting data over a physical communication medium such as microwave, twisted pair, coaxial cable, and so on. The topology of the medium, such as Ethernet or Token Ring, is also a part of the physical layer. The physical layer usually provides error-prone service.
- The data link layer provides media-level transmission of raw data frames. It can, if necessary, implement error-control techniques, such as packet sequencing and acknowledgment, to provide error-free service.
- The network layer determines the route between any two communicating stations, which could be on the same LAN or across the globe and in a different communications environment.
- The transport layer uses the services of the network layer. It provides an end-to-end or source-to-destination data transmission channel to the session layer. It does so by hiding the network itself. If necessary, it can split a data packet into multiple packets to suit

12. See [Tanenbaum 88].

the network layer limitations, and it can multiplex several data streams over the same physical channel. The IP layer of TCP/IP, for example, corresponds with the network layer.

- The session layer enables a session between two communicating processes (at the presentation layer) first by using some transport layer service to bind to the remote process, and then by managing the session for the upper layer. Thus, while the transport layer provides only connectionless services, the session layer will present connection-oriented service to the upper layer. For example, the NetBIOS protocol provides session layer services.
- The presentation layer is the layer mostly concerned with data representation and transformation. The presentation layer transforms data when the data is going across different platforms, for example, between a little endian machine (for example, Intel 80486) and a big endian machine (say, Motorola 68030) in which the byte ordering is reversed.
- The application layer, the topmost layer, allows application programs (for example, FTP, Telnet) to define their own protocols for interacting with their counterparts.

The OSI reference model provides the framework for sophisticated networks that connect any two or more computers. Most personal computers and UNIX workstations are connected to each other on a LAN. However, most LANs today follow **American National Standards Institute/The Institute of Electrical and Electronics Engineers, Inc. (ANSI/IEEE)** 802-1985 Standard, known simply as "802," which was adopted by ISO as a draft and subsequently approved as ISO Standard 8802:1989. Figure 1-7 compares the 802 standard with the OSI reference model.

The 802 is a set of standards, each of which describes, in detail, the particular parts of the LAN. The first standard, 802.1, introduces all the other 802 standards. The **logical link control (LLC)** defined in the 802.2 standard out-

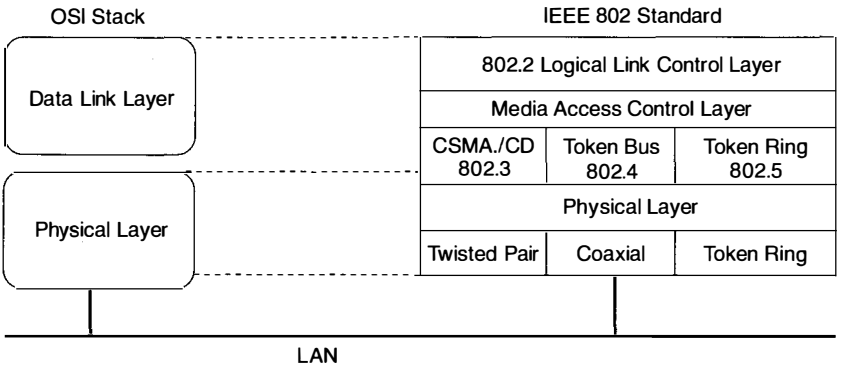


Figure 1-7. *OSI Model Compared with ANSI/IEEE 802 Standards*

lines a low-level protocol for maintaining a logical link between two physical stations on the network. The underlying network topology is defined by the **media access control (MAC)** layer and can be one of the following: **carrier sense multiple access with collision detection (CSMA/CD)**, Token Bus, or Token Ring. These are known as 802.3, 802.4, and 802.5, respectively. Ethernet, a popular media specification, is similar to the 802.3 specification.

ISO has published its counterparts to the IEEE 8802.x standards: ISO 8802-2, 8802-3, 8802-4, and 8802-5. IEEE 802 does not define the layers above the data link layers. The standards for **fiber distributed data interface (FDDI)** LANs are specified by the IEEE 802.6 and ISO 8802-6 standards.[13]

Wide Area Networks

In this book, you will often see the efficiency of an IPC mechanism (for example, datagram-based communication) in a LAN compared with the same mechanism in a WAN. However, there are conflicting definitions of a WAN. You may also have encountered other systems, such as **metropolitan area network (MAN)**. Here, a WAN is *a collection of LANs loosely connected by links that usually have a different data transmission rate than the LANs they connect.*

"Loosely connected," means a WAN connection is more likely to fail, or be unavailable, than is a LAN connection. Consider a WAN consisting of a LAN in Chicago that is connected to another LAN in Australia by leased telephone lines. Assume that due to the high cost of the leased telephone lines, the WAN link is available only between midnight and 2 A.M. In this case, WAN connectivity is guaranteed only for two hours.

Traditionally, the leased telephone lines that form the links between LANs operate between 9600 **bits per second (bps)** to 1.54 **million bits per second (Mbps)**. (Compare these speeds with the typical LAN speeds of 10–16 Mbps.) However, using the new fiber optic networks, which can accommodate data transmission rates of 100 Mbs (and one gigabit per second in the not-so-distant future), the WAN links have become much faster than LAN links.

Figure 1-8 shows a WAN in which several LANs are connected by links with varying data transmission speeds.

Another interesting aspect of the WAN is the topology of the interconnections among the LANs. In Figure 1-8, each individual LAN is connected with the other LANs. Even if one link fails, the other LANs can continue to communicate. However, it is possible to have topologies (for example, hierarchical connections, ring connections, and so on) in which LANs are not completely interconnected. In such cases, WAN connectivity may not be available because one or more of the intermediate links has failed.

To summarize, a WAN is a collection of LANs in which the links connecting the LANs may not be guaranteed to function at all times.

13. See [Sinha 92a] for additional details on the material in Section 1.4.1.

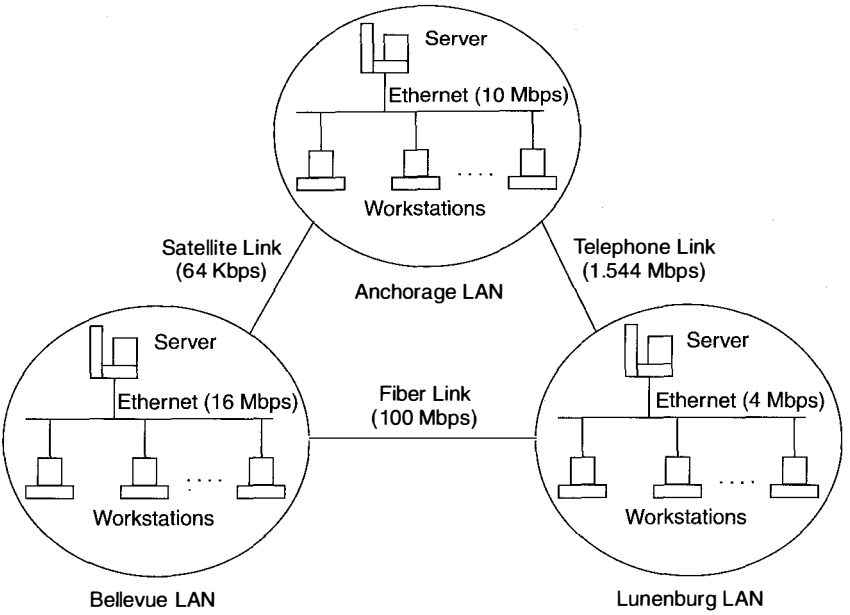


Figure 1-8. *Wide Area Network*

Datagram-Based Communication

Before we look at the details of the communication methods available in Windows and Windows NT, we define the different types of datagrams and discuss the efficiency concerns that dictate choosing one kind of datagram over another. We begin by distinguishing among unicast, multicast, and broadcast datagrams.

Every datagram contains the full network address of the sender and the recipient. This is unlike virtual circuit messages, which usually contain only the circuit identifier with the data. If a datagram is sent to a specific recipient, it is called a unicast, or directed, datagram. If it is sent to a group of recipients, it is called a multicast datagram. (In this case, the recipient address contains a supplementary distinguishing feature. For example, in an Ethernet environment, where each address is 48 bits long, the presence of "one" bit in the 40th place (01000000000000h) makes this a multicast address.) Finally, a broadcast message is a datagram sent to every station on the network. In Ethernet, a recipient address of all ones causes the packet to be broadcast to all listening devices on the Ethernet. (Note: Indiscriminate use of broadcast messages tends to adversely affect the entire network and should therefore be avoided.)

Imagine a network containing multiple computers, each using datagram services. Without getting into the details of the network media type (for example, Token Ring, Ethernet, and others), it is safe to generalize that each computer has a unique network address through which other computers can

reach it; each computer has a hardware device (network adapter) that can send and receive packets from the network; and each computer has the correct software for using the network adapter to send or receive packets. To receive a unicast message, each network adapter needs to "listen" only to those packets on the network that have its computer's address in the recipient field. When such a packet arrives, the network adapter forwards it to the software to process.

Now, suppose all these computers are logically partitioned into separate groups (for example, accounting, engineering, and so on). To participate in group communications, the software on each computer has to "feed" the network adapter a list of group or multicast addresses. To send a message to the entire group, the sender puts the group or multicast address in the recipient field. The network adapters must now detect group addresses in the network packets and forward to the software any multicast packets directed toward a group to which the computer belongs. (However, each multicast datagram packet must be inspected by each network adapter, which increases the computing "overhead" on each computer.)

While multicast datagrams make it possible for computers to participate in group communications, the situation deteriorates when broadcast datagrams are sent on the network. Because the broadcast packets are meant for everybody in the network, each broadcast packet has to be processed first by each network adapter and then by the software on each computer. This affects the CPU cycles on each computer that is listening to the broadcast traffic. In addition, the broadcast messages in certain networks can adversely affect the network bandwidth. Finally, broadcast packets can flood an entire WAN unless special measures are taken to avoid sending the broadcast packets from the originating LAN to the WAN. Because of these concerns, broadcast packets should be used with discretion.

Communication Methods in Windows and Windows NT

The major communication methods available in Windows and Windows NT environments are RPC, Windows Sockets, Named Pipe, Mailslots, NetBIOS, and IPX/SPX.

RPC is a collection of libraries and tools that allows the client/server application writer to focus on the application itself, rather than on the low-level communications code. With this tool, the application writer designates one or more functions in the server code that can be remotely called by one or more clients. RPC provides the "glue" that bonds client/server communications. The client calls some function that looks no different than a local procedure call, except that the remote procedure is executed at the server's end. With the aid of RPC tools and libraries, applications can become independent of transport protocols, and can use the Windows NT security that is built into Microsoft Windows NT RPC. Most important, RPC provides the framework

for handling client/server transactions through a simple, familiar programming model. When the client/server system designer uses any communication method other than RPC, he or she has to design and implement a communication substrate for the CSS.

Named Pipe is another interprocess communication method available in Windows and Windows NT. This mechanism is much closer to **first in, first out (FIFO)** than the "Pipe" mechanism available in UNIX.[14] It allows a bidirectional flow of data between the client and the server, who can be on the same machine or across the network. Typically, the server creates a pipe and waits to read requests sent by the client. The client opens the pipe and sends requests to the server by writing data into the pipe.

The applications that do not need bidirectional communication can use Mailslots. In Windows NT environments, Mailslots provides a unidirectional guaranteed (called "first-class" Mailslots) or nonguaranteed ("second-class") flow of data. However, in the Windows environments, only second-class Mailslots are available. Any process can create a Mailslot, and any other process can send it a message by writing a message into the first process's Mailslot. Mailslots always use broadcast datagrams to send and receive data.

Windows NT can function in Novell NetWare LANs. In this environment, the applications can use IPX/SPX interfaces to communicate with either the NetWare servers or with custom application servers.

IPX/SPX is a set of protocols that also define their own programming interfaces. These interfaces can be used to write client/server applications. The application can also use the same interfaces to achieve peer-to-peer communication. IPX/SPX can be used for both connection-oriented and connectionless communication.

One of the low-level communication methods used in some PC-networking environments is the NetBIOS protocol. NetBIOS exposes a set of interfaces that can be used to write client/server or peer-to-peer applications, and for both connection-oriented and connectionless communication.

Windows and Windows NT support the Windows Socket interface, which, unlike other PC interfaces, allows applications to use transport protocols (for example, TCP/IP, UDP, AppleTalk) directly, without imposing any "on-the-wire" protocol overhead. **WinSock**, the familiar term for **Windows Sockets interface**, is an extension of the Berkeley sockets interface, which has been specially modified to suit the Windows environment. WinSock, like Berkeley sockets, can be used for a bidirectional flow of data between client and server. In addition, on completion of asynchronous I/O, a WinSock interface exports functions that send messages to the application. This extension has been added so sockets can fit the message-driven interface model of Windows.

14. See [Stevens 90] for details.

Windows and Windows NT provide functions called WinNet APIs, which can be used by applications to access the common resources of a file/print server. These vendor-independent APIs ensure that the application writer does not have to write vendor-specific code to access various network resources. Of course, network vendors (for example, Banyan®, Novell) make available providers, which are the underlying supports that allow such vendor-independent access to resources.

While it is beyond the scope of this book to cover all the WinNet APIs, material is available on these APIs in [MS 93a], [Davis 93], and [Baker 93].

Table 1-1 summarizes the IPC methods available in Windows and Windows NT. In this table, "peer-to-peer" means that any two processes hosted either on Windows or Windows NT can communicate without any restriction. However, in some cases (for example, RPC), the server must reside on a Windows NT (as well as on an OS/2) server.

Table 1-1. *Interprocess Communication Methods in Windows and Windows NT*

IPC	*Windows*	*NT*	*Covered in This Book*	*Comments*
RPC	yes	yes	yes	RPC server-only in Windows NT.
Named Pipes	yes	yes	yes	Server on Windows NT only.
Mailslots	yes	yes	yes	Peer-to-peer.
IPX/SPX	yes	yes	yes	Peer-to-peer and client end to **Novell NetWare Loadable Modules (NLMs).**
NetBIOS	yes	yes	yes	Peer-to-peer.
WinSock	yes	yes	yes	Peer-to-peer.
Memory-Mapped Files*	no	yes	yes	Peer-to-peer. Processes must exist on same machine.
Win Net APIs	yes	yes	no	For accessing resources shared by file servers and print servers.

*The memory-mapped file I/O mechanism allows a program to map a file to the virtual memory. The program can then read from, or write into, the file using memory access functions (for example, memopy). Once a program has created this mapping, other applications with the proper access rights can share this file by defining a view onto the file.

1.5 SUMMARY

This chapter unfolded the "road map" for this book and presented a primer on Windows NT as a host for client/server applications. It pointed out that Microsoft has two lines of operating systems—MS-DOS–based Windows and Windows NT—and gave a brief history of the evolution of Windows and Windows NT. While Windows is suitable for desktops and low-resource computers, Windows NT is preferable for servers and high-end workstations. Both operating systems incorporate the same GUI.

This chapter reviewed CSSs and OSI-layered networking architecture and then presented the key features of Windows NT that make it an ideal platform for hosting client/server systems. Finally, we surveyed the communication methods available in Windows and Windows NT that can be used to build client/server systems.

Chapter 2 presents the architecture of Windows NT to help you understand how networking fits into NT's general architecture.

Chapter Two

UNDERSTANDING WINDOWS NT ARCHITECTURE

This chapter presents an overview of the architecture and networking in Windows NT and certain key programming features, such as Win32 services. By learning about the architecture of networking in Windows NT and by knowing about its advanced features, you will be better prepared to understand and use the IPC methods in Windows NT presented in later chapters—and in your own applications.

The chapter begins by illustrating the Windows NT architecture and then examines networking in the Windows NT operating system. We take a close look at how communication interfaces (for example, NetBIOS) are exposed by the system. The chapter tours some of the advanced features of Windows NT, such as threads, semaphores, memory-mapped files, mutex, exception handling, signal handling, and Win32 services. For each of these features, the chapter describes the whys and wherefores and then illustrates the basic usage model by showing sample programs. Additionally, you will build an Echo Server[1] using Win32 services APIs. If you are porting UNIX daemons or NetWare NLMs to Windows NT, you may find the description of a Win32 service particularly interesting. (Later in the book you will modify this Echo Server to use various IPC methods.) The Echo Server example should help you understand the workings of a service that uses a given IPC mechanism. Furthermore, by using a simple and consistent sample server throughout, you will have a tool you can use to compare one IPC mechanism with another.

2.1 SALIENT WINDOWS NT ARCHITECTURE FEATURES

Because of the duality of the requirements presented to them, designers had a tough challenge selecting architectural features for Windows NT. On one hand, the Windows NT architecture had to export the advanced operating system features that supported computing-intensive scientific and business applications. On the other hand, it had to support most of the existing MS-DOS and Windows applications. Windows NT had to live up to its promises to the

1. An Echo Server returns a message to the client that originated it.

software developer community, meeting their needs for advanced features such as 32-bit computing, yet it needed to be backward-compatible with applications written for four diverse operating systems (MS-DOS, Windows, OS/2, and POSIX). In the end, the designers delivered Windows NT, an advanced 32-bit operating system that was scalable, symmetrical, multiprocessing-capable, C2-secure, tightly integrated with networking, and capable of supporting most of the operating systems' applications.[2]

As Figure 2-1 shows, the Windows NT operating system kernel provides system services, while the protected subsystems support their own variant of applications. (For example, the OS/2 subsystem supports OS/2 applications on Windows NT. In other words, the subsystems emulate their given operating

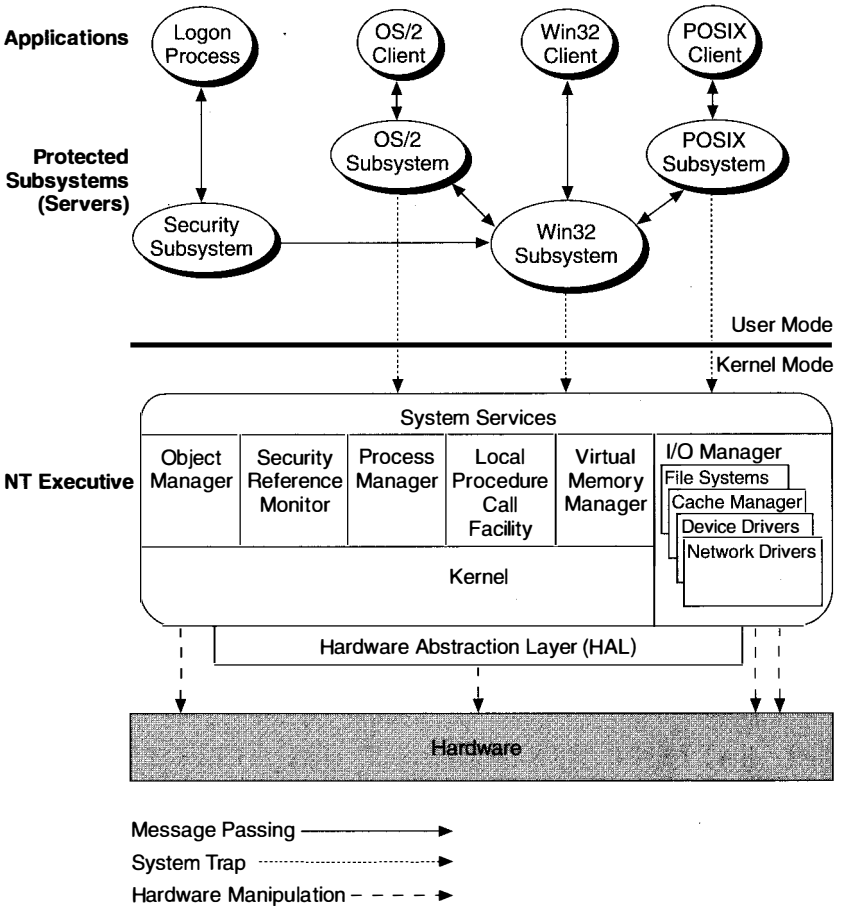


Figure 2-1. *Windows NT Architecture (Source: Custer 93)*

2. See [Custer 93] if you are interested in understanding the creative process behind this complex yet easy-to-use operating system.

system.) The clear dichotomy allows the Windows NT kernel to provide a rich set of system services while leaving the operating-system-specific details to the subsystems. In fact, each subsystem is merely a protected application in Windows NT; errant subsystems or applications running in a subsystem cannot compromise the entire system. Such a protection scheme is essential because MS-DOS and Windows applications are notorious for compromising a system, often leaving no choice but to reboot.

2.1.1 Windows NT Executive

The Windows NT kernel components shown in Figure 2-1 collectively form the Windows NT Executive. The Windows NT Executive is isolated from the hardware by the **hardware abstraction layer (HAL)**, which is hardware-specific code. The separation between the Executive and the hardware allows the rest of the Executive, and most of the I/O subsystem, to be independent of hardware features, such as the type of processor (for example, Intel 486, Digital® Alpha,™ MIPS R4000, Motorola/IBM PowerPC®), I/O interfaces, and multiprocessor communication mechanisms. An immediate benefit of HAL's presence is that Windows NT is portable to diverse hardware platforms.

The system services that the Executive provides can be exploited by using a set of kernel-mode Windows NT APIs. Windows NT APIs are extensively used by the device drivers and subsystems. In general, Microsoft does not expect application writers to use the Windows NT APIs; instead, they should use the Win32 APIs exported by the Win32 subsystem. At present, only the Windows NT subsystems and device drivers (because they are a part of the I/O system) use Windows NT APIs. Hence, these APIs are published in Windows NT device driver development kits.

The Windows NT Executive consists of a number of components: Object Manager, Security Reference Monitor, Process Manager, Local Procedure Call Facility, Virtual Memory Manager, kernel, and the I/O Manager. Let's take a brief look at each of these components:

- *Object Manager:* This component creates, manipulates, and destroys system objects. Windows NT systems represent resources such as files, processes, devices, device drivers, and so on as named objects, and they maintain a name space that distinguishes one object from another.
- *Security Reference Manager:* Every Windows NT object has some access control attributes associated with it. To access any object, the caller must have the required privilege level. The Security Reference Manager enforces access control policies and performs optional audits on such accesses.
- *Process Manager:* This component creates, controls, and terminates processes. In Windows NT, this basic unit of execution is called a

thread. Whenever a process is started, the first, or main, thread begins execution. A process can have more than one thread of execution. Threads are discussed in detail in Section 2.3.3.

- *Local Procedure Call Facility:* The **local procedure call (LPC)** is similar to the RPC, except that it is optimized for local IPC. LPC is used extensively within Windows NT itself.
- *Virtual Memory Manager:* In Windows NT, every process can work with up to four gigabytes of virtual address space. The address space of any process is private to that process and is protected by the operating system from being used by any other processes. Hence, an errant process cannot comprise other processes. The Virtual Memory Manager implements this virtual, protected memory management for all processes and drivers in the system.
- *Kernel:* The kernel implements the Windows NT scheduling policy. It schedules each thread of execution, using a priority-based dynamic scheduling policy. It also handles all hardware interruptions and exceptions and dispatches them to one or more proper drivers via the I/O Manager.
- *I/O Manager:* The I/O Manager controls the I/O devices by using drivers for each kind of device. It provides the basic framework within which these drivers operate. Windows NT has a set of generic drivers for the keyboard, mouse, one or more floppy disks, and hard disk. (Hardware vendors can override the generic drivers by supplying specialized drivers for their hardware.)

In short, the Windows NT Executive provides kernel services to build a modern operating system without bias toward any other operating system or subsystem. It provides process management, built-in security, protected virtual memory, and abstraction from hardware via the HAL and the I/O system.

2.1.2 Windows NT Protected Subsystems

Each subsystem is a Windows NT application that uses the system services provided by the Windows NT Executive. All subsystems are implemented with a client/server model. Through the client/server model, clients of Windows NT subsystems communicate with the server (that is, the subsystem) by LPC. This technique allows "bad parameter" testing to be done on the client side before passing a call to the server. In most cases, each subsystem does this by exporting its public interfaces by one or more **dynamic-link libraries (DLLs)**. (For example, "ntdll.dll" contains most of the Windows NT APIs.) These DLLs can test the parameters of an API before transmitting them to the subsystem by LPC. The separation between the client and server sides of the subsystem ensures that even when the client requests to the subsystem fail, the failure does not compromise the entire subsystem. This design contrasts with Windows 3.1 architecture,

in which applications call directly into the system DLLs, which can cause catastrophic system failure. Thus, subsystems' use of the client/server model gives them, as well as the main system, additional protection and security.

In Windows NT, Win32 is the key subsystem, controlling the GUI as well as the console (the character I/O window). In other words, Win32 subsystem presents a GUI identical to the Windows GUI in a Windows NT desktop, and that controls the keyboard, mouse movements, and display devices. Thus, the Win32 subsystem intercepts a mouse click by the user and can direct it to a Win32 application. (This implies that the OS/2, MS-DOS, and POSIX subsystems must channel their I/O requests through the Win32 subsystem. For example, the OS/2 subsystem must ask the Win32 subsystem to display a text string on the console.)

All interfaces exported by the Win32 subsystem are available through Win32 APIs, which are documented in the Windows NT software development kit. The Win16 applications indirectly use these Win32 APIs as well. How is this done? As you know from Chapter 1, Windows NT supports a Win16 subsystem which thunks the Win16 API calls (at run time) to their Win32 counterparts. This means existing Win16 applications can be run on the Windows NT platform without recompilation or modification.

When a program is launched in Windows NT, the Win32 subsystem determines the operating environment requirements for the application. For non-Win32 applications, Win32 initiates the proper subsystem, which in turn hosts the application. (For example, the Win32 subsystem activates the OS/2 or POSIX subsystems for OS/2 and POSIX applications.) These subsystems, in turn, use Windows NT to emulate the complete environment and the environment-specific programming interfaces, and the application executes as if it were running on the native operating system. For example, POSIX applications can use the fork() system function to launch another process. An OS/2 application can use the DosExecPgm() API to start a new process.

Figure 2-2 illustrates how MS-DOS applications are run within a special console that emulates MS-DOS. Such an environment is called a **Virtual DOS Machine (VDM)**. The VDM allows users to run most of the commercial MS-DOS applications. The VDM restricts access to certain hardware functions, such as direct access to the machine **read-only memory basic input/output system (ROM-BIOS)** or the video driver memory. Thus, certain MS-DOS-based applications cannot be executed on the Windows NT platform. However, although the VDM is a special environment, the user does not perceive any visible difference between the execution of a Win32 program and that of an MS-DOS program. A VDM can be launched for each MS-DOS application, an innovation made possible by the **multiple VDM (MVDM)** feature in Windows NT. (Compare this to normal MS-DOS machines, which allow only one application to execute at a time.) The MVDM feature makes it possible to run many MS-DOS applications simultaneously.

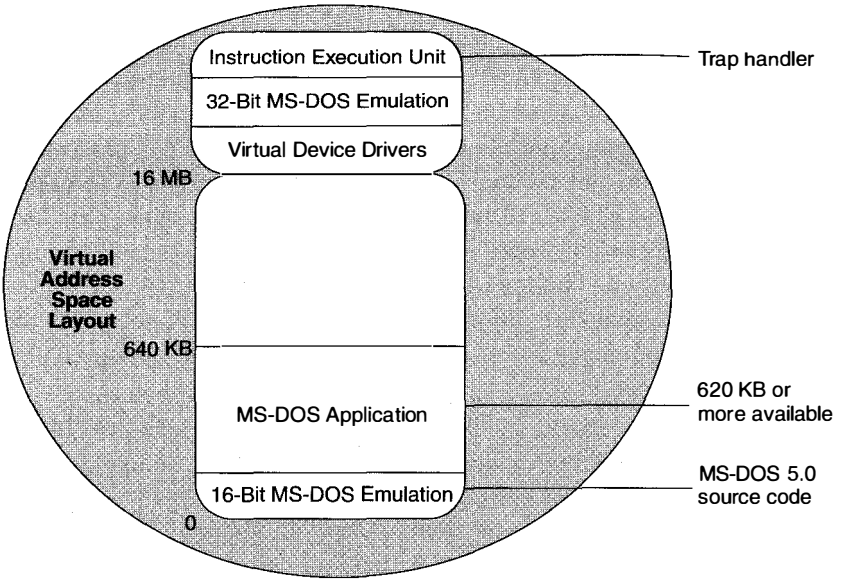


Figure 2-2. *Virtual MS-DOS Machine (VDM) (Source: Custer 93)*

Figure 2-2 shows that each VDM emulates a 16-bit MS-DOS environment and launches an instruction emulation unit on non-Intel platforms. (Recall that MS-DOS application binaries are composed of Intel x86 instructions.) Thus, emulation allows MS-DOS applications to run on non-Intel platforms, even though certain hardware access functions provided in MS-DOS cannot be executed on non-Intel platforms. In other words, users can run many of their favorite MS-DOS programs on, say, a RISC processor-based PC.

In Windows NT, most Windows (that is, Win16 API-based) applications can be executed in either single or separate VDMs. This mechanism is referred to as the **Windows on Win32 (WOW)** subsystem. WOW closely emulates the nonpreemptive multitasking environment of Windows 3.1.[3] By default, all Windows applications are run within a single WOW. This replicates the Windows environment of a single input message queue for all applications, as opposed to the Win32 input message queue model, in which each Win32 process has a distinct input queue filled with keyboard and mouse events only for the given process. This architecture, called a desynchronized input model, allows each application to process its input queue without blocking any other application. Figure 2-3 illustrates this concept. In Windows NT 3.1, all WOW applications are launched within the same VDM. This has the same drawback as a native Windows system (for example, Windows 3.1)—a single message queue for the whole system. Also, an errant Windows application can compromise other Windows applications

3. On Intel 386 (or higher) platforms, WOW emulates enhanced-mode Windows; on non-Intel platforms, it emulates standard-mode Windows.

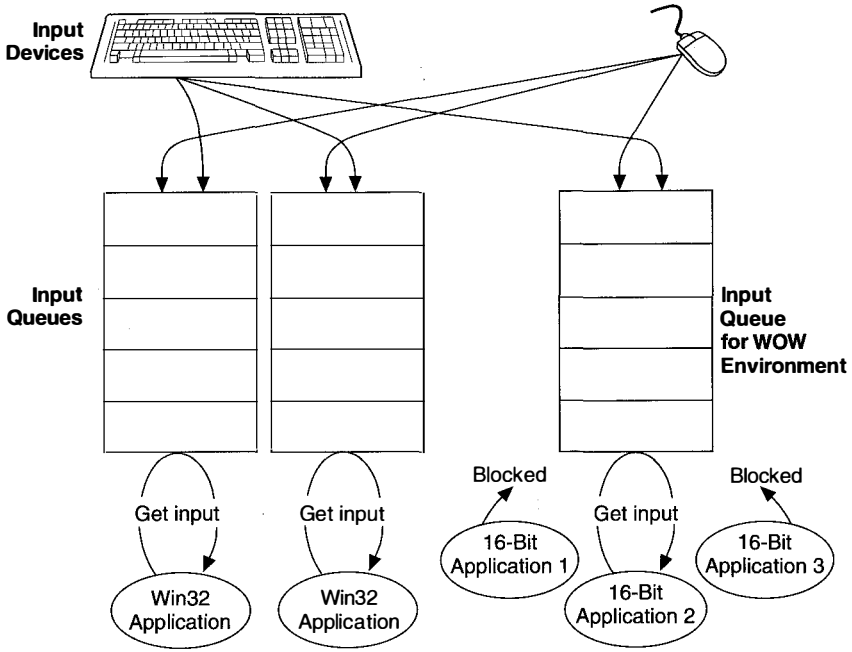


Figure 2-3. *Input Model for Win32 and WOW Applications on Windows NT (Source: Custer 93)*

executing within the WOW subsystem. Thus, starting with Windows NT 3.5, users can launch each Windows application in a separate WOW subsystem. All the distinct WOW subsystems running on Windows NT 3.5 cooperate with each other, so window messages can be passed between Windows applications, just as they would be in a native Windows system. This is also true for most Windows interprocess communication methods, such as **Dynamic Data Exchange (DDE)** and **object linking and embedding (OLE)**. Multiple WOW subsystems allow Windows NT to provide a stable environment for Windows applications without imposing any limitations.

2.2 NETWORKING IN WINDOWS NT

This section delineates the architecture of networking in Windows NT, enabling you to visualize the flow of data from the application to the network through Windows NT networking components. It begins by comparing the network layers of Windows NT with those of the OSI/ISO reference model.

2.2.1 Windows NT Networking Components

Figure 2-4 shows the Windows NT components compared with the OSI/ISO reference model. At the physical layer, Windows NT provides suitable MAC drivers for most networking media such as Ethernet, Token Ring, and FDDI. It also supports remote (that is, telephone) Windows NT server access from a Windows

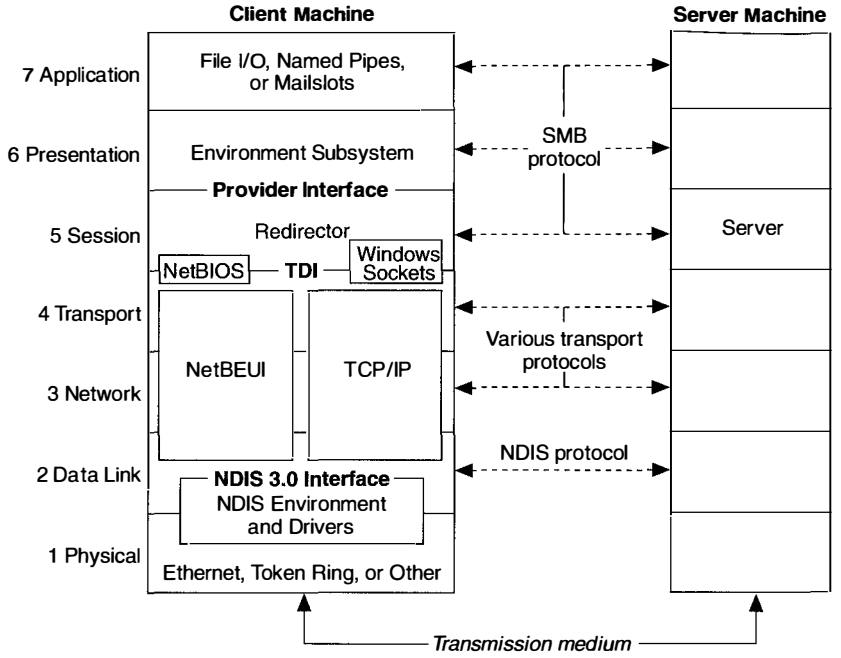


Figure 2-4. *Windows NT Network Components Compared with the OSI Model (Source: Custer 93)*

NT workstation. To do this, it uses the telephone network with a built-in facility known as **remote access service (RAS)**. All the MAC drivers in a Windows NT environment can be accessed by the upper layers through the **Network Driver Interface Specification (NDIS)** 3.0. Windows NT provides drivers for most industry standard protocols, such as NetBEUI, NWLink, (the Microsoft-supplied IPX/SPX-compatible protocol stack), TCP/IP, AppleTalk, OSI/TP4, and DECNet.™ All transport drivers use the **transport driver interface (TDI)** to expose their services to the upper layers. These drivers can also expose other standard interfaces, so applications can directly exploit these drivers. (For example, the NetBIOS interface as well as the TDI interface are exported by the NetBEUI driver.) Windows NT provides built-in drivers, called the redirector and the server, so users can access or share network resources, respectively. The redirector and the file server communicate with each other using the **server message block (SMB)** protocol.

The redirector and the server use the TDI interface to communicate with the transport drivers. A Windows NT workstation can access any Windows NT server (or a "compatible" server) by using the redirector. Similarly, every workstation has the limited capability (up to 5 users) to share its resources, such as files, printers, and communication ports, with other workstations on

4. Transport Protocol. See [Rose 90] for detailed presentation of OSI protocols.

the network. (A licensed Windows NT server allows unlimited access.) As Figure 2-4 shows, the network file I/O services use the redirector to access remote files, while they use the server on the Window NT workstation to share files and other resources. Certain IPC methods, such as Named Pipe and Mailslot, also use the services of the redirector and the server.

The user can take advantage of layered architecture (illustrated in Figure 2-4) to interchange components, as long as each individual component conforms to the specifications. For example, if you replace an Ethernet card (and its corresponding MAC driver) with a Token Ring card (and its corresponding MAC driver), your Windows NT machine can interact on a Token Ring network. Similarly, you can access a NetWare server by replacing the NetBEUI protocol driver with the IPX/SPX protocol driver.

You might ask: "How does a transport driver know whether to use the Ethernet MAC or the Token Ring MAC?" To answer that question, you need to understand the concept of network binding. Network binding is a static relationship between the network layers that collectively provide access to the network. (Future Windows and Windows NT systems will use a technique called "Plug-and-Play" to allow dynamic binding.) The relationship is usually established at Windows NT installation time, and it can be modified at other times by using the Windows Setup program. Whenever a Windows NT machine boots, the different layers establish logical binding to each other based on the predefined relationships. For example, a user may set up the redirector to reach the services of TCP/IP protocol, which in turn may be set up to use an Ethernet MAC driver.

A significant part of establishing the network binding is determining the compatibility between layers. (For example, you cannot bind an Ethernet MAC with a Token Ring card.) The compatibility of the layers being bound is verified by the Windows NT Setup program when the user is installing or modifying, Windows NT network drivers. Each layer negotiates the "service contract" with its upper or lower layer at run time. These negotiations do not change the interrelationship; instead, they fine-tune the exact parameters of service. (For example, at startup time, the MAC driver informs the transport protocol of the maximum size of the packets that can be transmitted.)

In Windows NT, it is possible for a layer to bind to many upper or lower layers. This is called multiple binding. (The exact specifications can be found in NDIS 3.0 and TDI specifications.) For example, an administrator can configure both NetBEUI and TCP/IP protocol drivers to use the Ethernet card. Such a configuration may be set up so the user accesses the LAN-based servers using the NetBEUI protocol and the WAN-based servers through the TCP/IP protocol. Similarly, a Windows NT server may have the TCP/IP protocol driver using two network cards—an Ethernet card connected to the Ethernet network and a Token Ring adapter connected to the Token Ring network. Such a configuration allows the server to service the users on either network. (If you are interested in multiple binding setup and other operational details, see the Windows NT administrator's guide.)

Windows NT provides the freedom to replace the default redirector. This is useful when Windows NT is being used in a non-Microsoft network environment. For example, Microsoft provides a special NetWare redirector as part of the **Client Services for NetWare (CSNW)** package. The redirector can be installed on a Windows NT machine to give access to Novell NetWare file and print resources. (The user can also install the NetWare redirector supplied by Novell.) The Windows NT machine must use the IPX/SPX protocol to communicate with the NetWare servers. Of course, this Windows NT machine can support other protocols (for example, NetBEUI) and also allow the user to access other servers (for example, a Windows NT server that uses the NetBEUI protocol) at the same time.

2.2.2 Accessing Network Resources

The Windows NT redirector communicates with a remote server by using the SMB protocol. Because the SMB protocol is used by a large number of LAN file/print servers—the IBM LAN Server,® Microsoft OS/2 LAN Manager,® Windows for Workgroups 3.11, Digital PathWorks,® and others—a Windows NT workstation is immediately capable of communicating with these servers.

When the redirector tries to locate a remote server, it uses each of the underlying transport protocols in the order in which they were installed, and uses the TDI interface to communicate with the transport drivers. The search ends when a server responds or all transports have been tried. In other words, the redirector and the server can communicate with each other as long as both ends support at least one common transport protocol on one common physical medium. Figure 2-5 shows how this works. For example, a Windows NT workstation can access a Windows NT server by using the TCP/IP protocol, which in turn can use the RAS facilities to connect to the server over the telephone network.

Your next question might be, "When does a redirector set up a connection with a server and start communicating?" With a few exceptions, the connection is set up when you map a local logical device (for example, H:, LPT1, COM1, and others) to the network resource shared by a server (for example, \\FileServer\HardDisk, \\PrintServer\Laserjet, \\FaxServer\Fax, and so on). In a Windows NT environment, you can use the "net" commands, or the File Manager program, to set up a connection. Once the connection is set up, network I/O can begin. For example, in the following commands, you map the logical disk drive ("h:") to the disk space ("HardDisk") shared by the server ("FileServer"). The next command causes a file to be copied to the remote server, and then the connection is torn down.

```
net use h: \\FileServer\HardDisk
copy config.sys h:\config.sys
net use h: /d
```

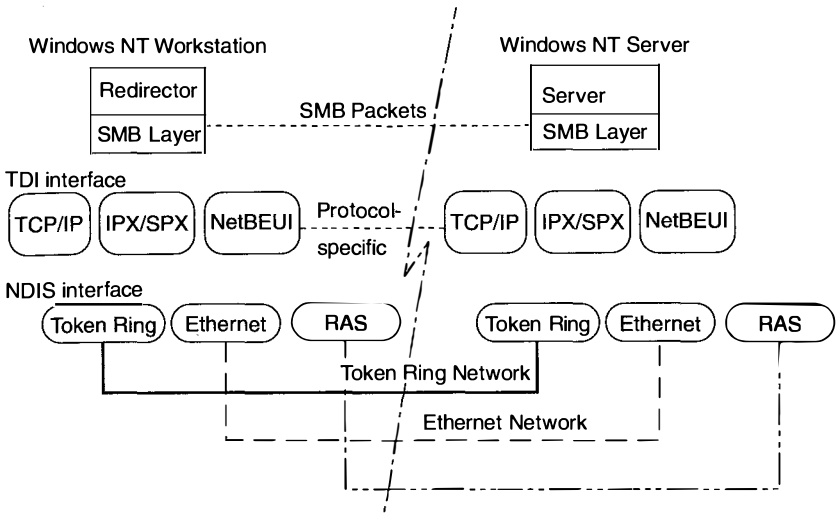


Figure 2-5. *Redirector and Server Using SMB Protocol*

2.2.3 Accessing Network Resources from Applications

This section provides a brief introduction to the programming interfaces available to applications to access the network resources.

In Windows and Windows NT environments, a special set of APIs are available for accessing the network resources (for example, storage, printers) made available by file servers and print servers. These interfaces, called WinNet APIs or WNet APIs, are available on Windows NT as part of the Win32 API set. In the Windows environment, the WinNet APIs are a subset of the Win32 WinNet APIs.

The most important feature of these APIs is that they can be used to access network resources regardless of the type of the underlying network software. For example, a WinNet API used to browse the servers present on the network will function properly whether the workstation is using a Windows NT network or a Novell NetWare network.

Windows NT supports the WinNet APIs by supplying a WinNet driver and a WinNet DLL. These allow applications to access the network through the Windows NT redirector. The WinNet API design is based on a layered architecture, making it easy to replace WinNet drivers and DLLs with others that better match the underlying network. Layered architecture also makes it possible for applications to use the same WinNet APIs to access a set of network resources in a vendor-independent manner. In other words, general-purpose programs (for example, word processors) are functional in all network environments as long as the applications and the network vendors both conform to the WinNet specifications.

The basic WinNet drivers and DLLs are useful in accessing Microsoft-supplied servers. When remote servers (for example, Digital PathWorks) are compatible with Microsoft network protocols, then the programs use the default Windows NT redirector and WinNet DLL/drivers to access them. Windows NT also provides proper redirector and WinNet API support to access Novell NetWare file/printer resources. The collection of drivers, DLLs, and network tools needed to access NetWare servers from Windows NT is the CSNW services referred to earlier. For other networks, the vendor must provide the vendor-specific drivers and DLLs to support the WinNet APIs. Table 2-1 lists the WinNet APIs supported by Windows NT. For a detailed description of using the WinNet APIs, see [MS 93a], [Baker 93], and [Davis 93].

Table 2-1. *WinNet APIs Supported by Windows NT*

WinNet Functions	***Description***
WNetAddConnection()	Redirects a local device to a network resource
WNetAddConnection2()	Redirects a local device to a network resource
WNetCancelConnection()	Breaks an existing network connection
WNetCancelConnection2()	Breaks an existing network connection
WNetCloseEnum()	Ends a network resource list
WNetConnectionDialog()	Starts a network connection dialog box
WNetDisconnectDialog()	Starts a network connection dialog box
WNetEnumResource()	Continues listing network resources
WNetGetconnection()	Gets the name of network resouce
WNetGetLastError()	Returns last error for network functions
WNetGetUser()	Gets the current network user name
WNetOpen Enum()	Starts listing network resources

2.2.4 Multiple Network Providers

Now you might ask, "How can I access the network resources in a multivendor environment? For example, I wish to access a Banyan server as well as a Microsoft Windows NT server from my Windows NT workstation."

In this case, Windows NT must accommodate access to many networks at the same time. The collection of drivers, DLLs (to support WinNet APIs), and tools needed to access a given network (for example, NetWare) is referred to as the network provider. This section discusses multiple network providers.

As you know from the section on program and access, Windows NT has a default provider that supports the use of WinNet APIs over Windows NT (or compatible) networks. The WinNet APIs are used by the file manager and other Window NT utilities for common functions such as connecting to, and

disconnecting from, the file servers and print servers. However, the default WinNet provider only supports Microsoft servers, such as Windows NT servers, Microsoft LAN manager servers, and MS-Net–based servers. In addition, Windows NT's CSNW package allows access to NetWare servers through WinNet APIs. For accessing all other network servers (for example, a Sun® server), individual network vendors must supply their own WinNet provider. Of course, a user should be able to connect to multiple servers at the same time from the same Windows NT workstation. To accommodate such uses, Windows NT has a **multiple provider router (MPR)** driver that routes requests from a program to the correct provider. This process is illustrated in Figure 2-6. Each provider exports the vendor-independent WinNet APIs, at the same time implementing each API in a vendor-specific manner. For example, a NetWare provider uses IPX to communicate with the NetWare servers, while a Banyan provider may use sockets to connect to the Banyan Vines™ server. Each application is completely shielded from the networking details.

Now that I've said you can access multiple vendor-specific servers simultaneously by using WinNet providers, you might wonder how two or more providers get to their servers. They do so by having vendor-specific redirectors that use the transport drivers to communicate with the vendor-specific servers. Figure 2-6 illustrates this and shows an I/O request being directed to the appropriate redirectors.

Multiple redirectors allow you to connect to a Windows NT server as well as to a NetWare server simultaneously by typing these commands:

```
net use H: \\NTServer\Public
map G: = NWServer/Sys:Public
type h:\config.sys
type g:\config.sys
```

With these commands, you can map a logical drive (H:) to the **Windows NT server (NTServer)** and another logical drive (G:) to a **Novell server (NWServer)**.

After a logical drive has been mapped to a server, the files on the distinct servers can be accessed using logical-drive-based names (e.g., H:\config.sys or G:\config.sys). By choosing one logical drive instead of another, you direct the I/O subsystem to forward the I/O request to a particular redirector.

Another convention for addressing network resources in Windows and Windows NT environments is called the **uniform naming convention (UNC)**. A UNC name is expressed as:

```
\\<Computer Name>\<Shared Resource>\<Further details>.
```

The next pair of commands illustrates how UNC conventions are used. Note that when you use a UNC name, you do not need to map a logical drive to a server before accessing its resources.

```
type \\NTServer\Public\config.sys
type \\NWServer\Sys\Public\config.sys
```

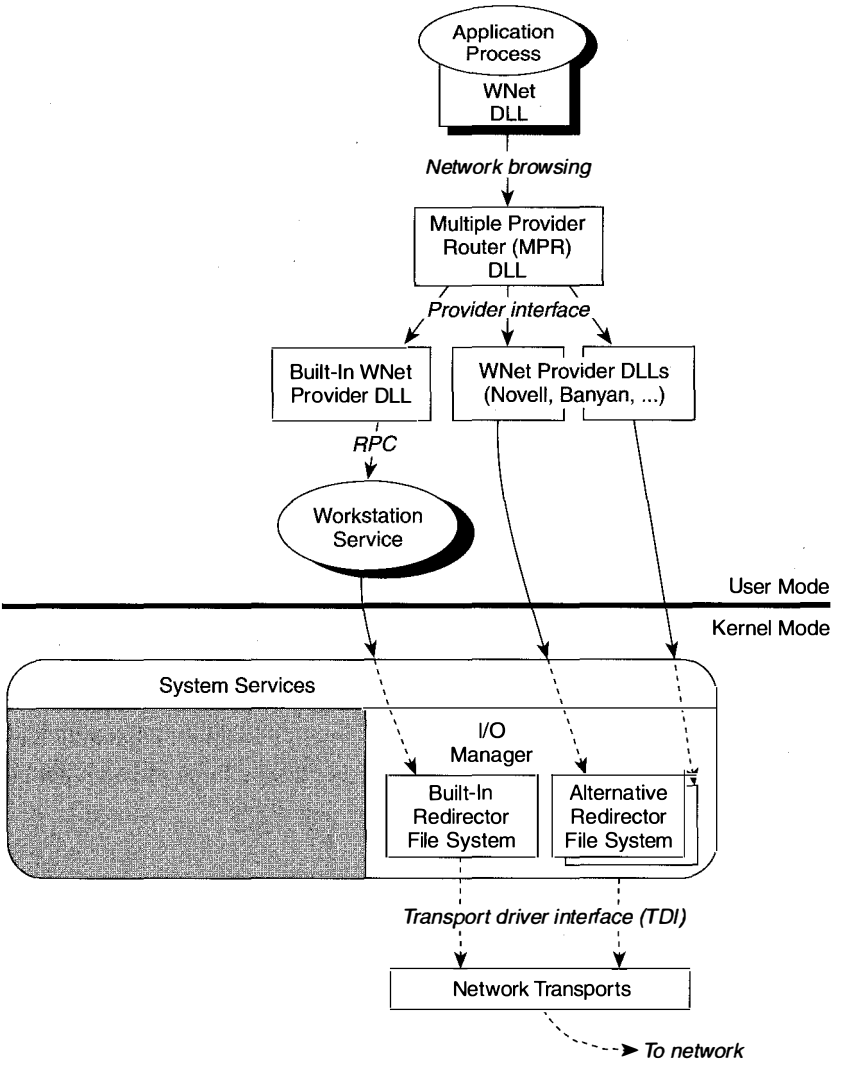


Figure 2-6. *Multiple Provider Router (Source: Custer 93)*

In this case, by providing an arbitrator called **multiple UNC provider (MUP)**, the Windows NT I/O subsystem knows which redirector should receive the I/O request. Figure 2-7 illustrates the MUP. When an I/O request containing a UNC name is given to the MUP driver, MUP calls each redirector with the complete UNC name of the network resource, and the redirector determines if it can provide access to the requested server resource.

2.2.5 Windows NT Server and Domain Controller

While any Windows NT workstation allows you access to the resources on the network, the **Windows NT Advanced Server™ (NTAS)** provides the traditional file server and print server functions. Additionally, NTAS has been

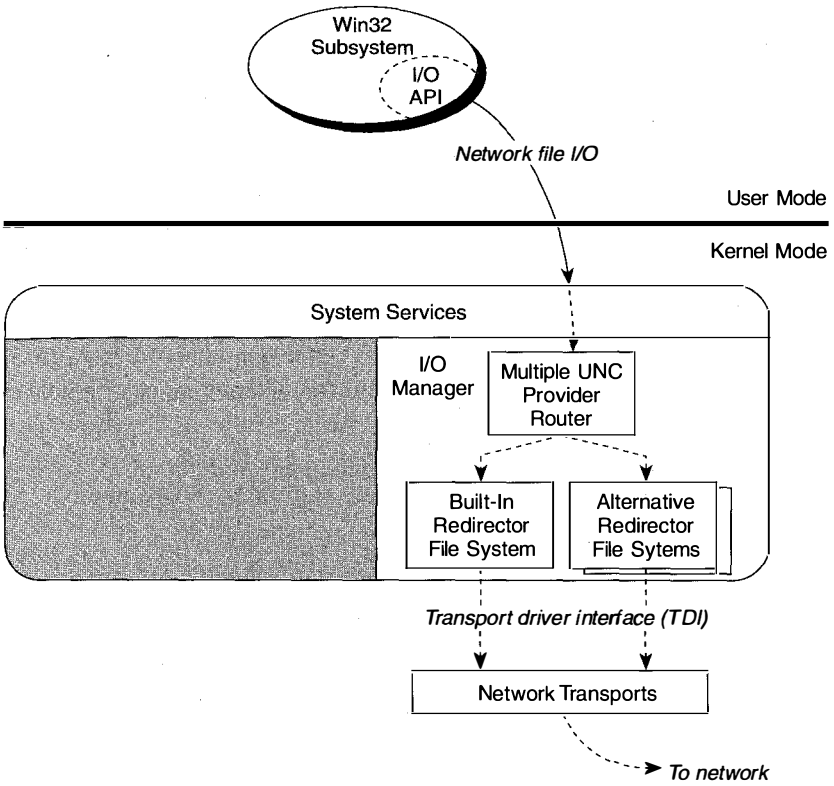


Figure 2-7. *Multiple UNC Provider (MUP) (Source: Custer 93)*

designed specifically to support CSSs. It is an ideal platform to host application servers, such as database servers (for example, the Microsoft SQL Server™), communication servers that allow access to mainframes (for example, the Microsoft SNA Server™), and system management and control servers (for example, the Microsoft SMS Server™). NTAS has enhanced scalability and rigorous security capabilities, maintaining tight user access control while accommodating many users. The next section elaborates on Windows NT security.

For system administrators to have tight control over user access, Windows NT uses domains. Each domain is an environment comprising computers, users, and resources in which one Windows NT server is designated as the **domain controller (DC)**. A domain can have an almost unlimited number[5] of Windows NT servers, Windows NT workstations, Windows workstations (including Windows for Workgroups), and users.

In the domain model, an administrator controls access to all the servers with that one DC, which validates each user's logon and access rights to all resources in the domain. (Each user has a single account for the entire domain. This account is maintained on the DC.) This contrasts with some

5. The actual numbers are usually dictated by the license obtained by the company. Theoritical limits are in millions.

network systems in which a user has an account for each server. In such systems, the administrator must manage user accounts on each individual server. The single account in a domain allows the administrator to manipulate user access by manipulating user account attributes.

An administrator can also set up one or more **backup domain controllers (BDC)**. Each BDC contains a complete replica of the domain's user-account database. The first DC is called the **primary domain controller (PDC)**. The presence of the backup DC has two effects: the user logon and access validation load is distributed among the DCs, which reduces the load on the PDC; and if the PDC should fail, the administrator can quickly bring a BDC online as the PDC. (Note: While a BDC is used to validate user access and domain logon, it cannot be used for modification in the domain user account database. All changes to user accounts must be made on the PDC only.[6]) A domain can span a WAN. However, administrators usually map a Windows NT domain to a geographical area, or sometimes to a LAN. When multiple domains are set up, domain administrators can work together to create a trust relationship between the domains that allows the users from one domain easy access to resources available in the other.

In Figure 2-8, you see that a company has set up three domains across the United States—the Anchorage domain in Alaska, the Bellevue domain in

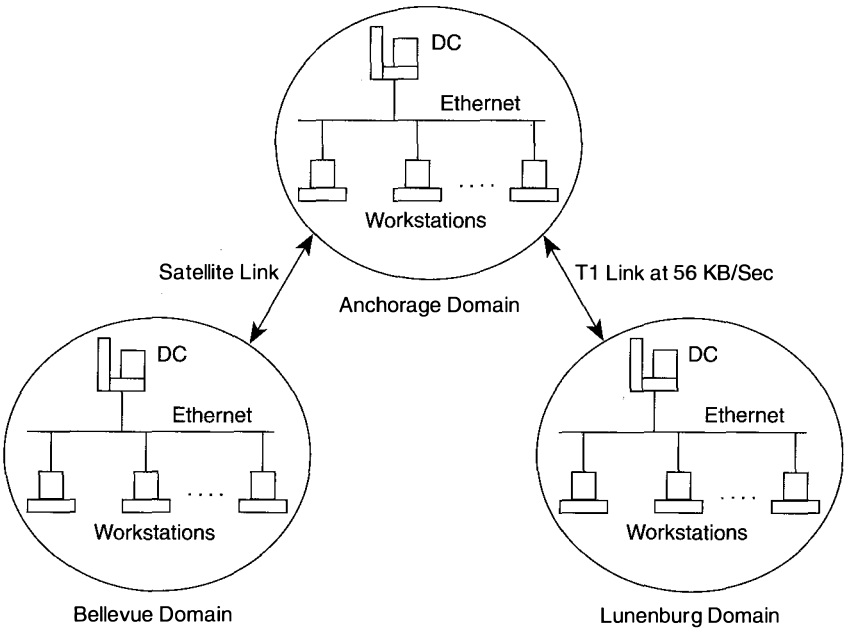


Figure 2-8. *Multiple Domains*

6. This is orthogonal to remote administration of the DC. Usually, administrators use remote administration tools to manage the DC from their workstation.

Washington, and the Lunenburg domain in Vermont. The company has established pair-wise trust relationships between Anchorage and Bellevue and Anchorage and Lunenburg domains, because the users from the Bellevue and Lunenburg offices often access file servers present at the headquarters in Anchorage. (For further details on the Windows NT domain model and the administrative control model, see the NTAS documentation.)

An important point to remember is that NTAS has advanced features for customizing access validation and control, auditing, and user environment control. However, insofar as networking programming is concerned, NTAS provides exactly the same number of programming interfaces as a Windows NT workstation. Hence, for the rest of this book, I will make no distinction between a Windows NT workstation and a Windows NT server.

2.2.6 Access Control and Network Security

Windows NT provides four built-in security features: secure logon facility, discretionary access control, auditing, and memory protection. While there are brief descriptions of each of these features in this section, study the Windows NT administrative guides for complete details.

Every Windows NT workstation is protected by a secure logon facility that controls logon access by means of user accounts. A user account contains credentials, such as the logon name and password. The logon facility allows only those users for whom an account has been established to log into the system.

A user account can only be created by an administrator. For a standalone Windows NT,[7] the user who installs Windows NT and is the administrator, that is, the one who has complete control over the machine.

In a Windows NT domain environment, only the domain administrators can create user accounts. The user who has successfully logged on to a system is assigned specific access rights and privileges. The access rights are defined by the user account and can be changed only by the administrator.

There are three types of user logons—local logon, domain logon, and network logon. A user is said to have logged on locally to a Windows NT workstation when he or she supplies, via the logon prompt presented by Windows NT, the tuple {user name, workstation name, user password}. For example, the user ("Fred") logs on locally to his workstation ("Fred's-machine") by supplying a password associated with his account ("Fred") and defined locally on the workstation ("Fred's-machine"). However, corporations often prefer to control user access rights and logon validations. In such companies, a user may not be able to log on to the Windows NT workstation or to access network resources unless he or she can be authenticated by the DC. The user needs to log on to a domain by supplying the tuple {domain user name, domain name, domain user password}.

7. With respect to security, Windows NT's being installed so as to participate in a workgroup is similar to a standalone machine.

To continue with the "Fred" example, assume Fred has a domain account ("FredTheEmployee") in the domain ("CompanyRulez") with a certain domain password. In this case, Fred performs a domain logon by supplying domain credentials so that his credentials will be authenticated by the DC and not by the local workstation security system. (An administrator can set up a Windows NT workstation so a user can use resources [for example, email] on the workstation by performing a domain logon from that workstation.)

No matter how Fred logged on to the workstation (that is, local or domain), he may need to access the network resources shared by one or more servers. However, before Fred can access a server's resources, he needs to perform a network logon (that is, the user usually needs to supply a {user name, user password} tuple to the server for authentication purposes). You may well grumble, "This is tedious, because I log off at the end of the day and need to reestablish connections with servers after logon every morning!" Fortunately, you do not have to perform explicit network authentication in the Windows NT system; the system promotes the use of a single logon. When you try to connect to any server from a Windows NT workstation, the underlying security system supplies user credentials to the server for you. The server can then check to see if you have a local account set up, or else the server asks the DC to authenticate you. (This implies that a domain administrator can control access to all network resources through the DC by *not* creating local user accounts on the servers.) Whatever the case, once you have been authenticated, you can access the server resources, provided discretionary controls do not restrict access.

Discretionary access controls allow an administrator to protect resources from unauthorized users. These controls are honored whether the access is local or over the network. An access control is explicitly defined by an **access control list (ACL)**, which is a list of **access control entry (ACE)**. Each ACE contains the access rights granted to a specific account. Every object (for example, file, printer, directory, memory) in Windows NT can have an ACL associated with it. The data structure by which ACLs are specified for any Windows NT object is called a security descriptor. Of course, an object without ACLs has no protection after a user has gained entry to the system; by default, the ACL of an object is the same as the access rights of the creator.

Figure 2-9 shows the ACL on a file object that contains three ACEs. The first ACE defines that the user ("Joe") has read(R), write(W), and execute(X) access rights. The second ACE shows that another user ("Guest") has only read access rights. (In Windows NT, the user ["Guest"] is a built-in guest account that defines the access rights of all users who do not have an account on the system.) The third ACE is more interesting. It shows that a user ("Molly") is allowed read/write rights, but she is specifically not allowed to delete(D) the file. Thus, an ACE can contain not only specific permissions, but also specific restrictions.

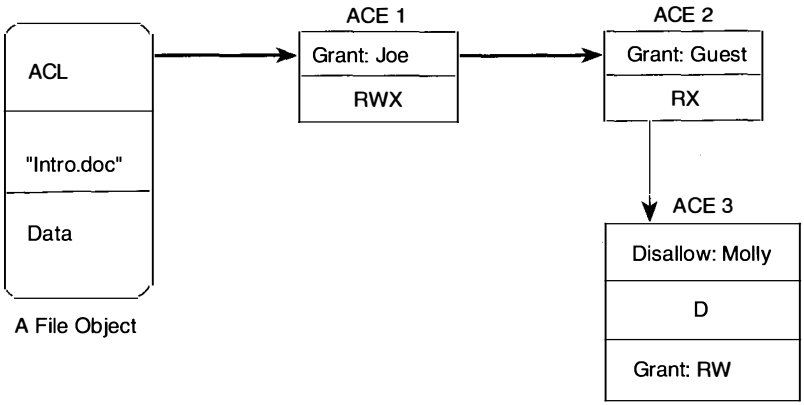


Figure 2-9. *Access Control List*

A system administrator does not need to put an ACL on every file on a server, even if that server has millions of files. Windows NT provides a mechanism by which an object can inherit the ACLs of its parent. Thus, the ACLs placed in a main directory apply to all the files and subdirectories contained in the main directory. A system administrator can simply place the proper ACLs on the root (for example, C:\, D:\, and so on) of a file system, and they will be effective for the entire volume.

The final ACL for any object is obtained by examining the ACLs inherited from the parents and any ACLs placed specifically on that object. Figure 2-10 shows how an administrator can use inheritance to give all users read/execute privileges in a directory. The administrator does this by placing an ACL on the directory itself, so the ACL grants the guest account read/execute privileges. Then, by placing specific ACLs on the file ("Intro.doc"), access to documents is constrained such that the ultimate access rights are exactly those we showed in Figure 2-9.

While inherited ACLs certainly reduce the time that system administrators spend on server access control, there is another concept available in Windows NT that makes administering a server even easier. By studying the scheme of access control in Figure 2-10, can you see a "scaling" problem? Suppose 200 people in the engineering department need access to the file "Intro.doc." In the scheme shown in Figure 2-10, unless the system administrator grants the "guest" account all the access rights, he or she will have to generate an ACE for each engineer. To reduce this task to a minimum, Windows NT uses the group accounts concept. A group can have zero or more users, and each of its individual users inherits the same access rights as those of the group. More important, an administrator can allow (or withdraw) access rights to all the users of a group by granting (or denying) the group account the necessary access rights. Thus, to apply the group concept in Figure 2-10, the system administrator must add a

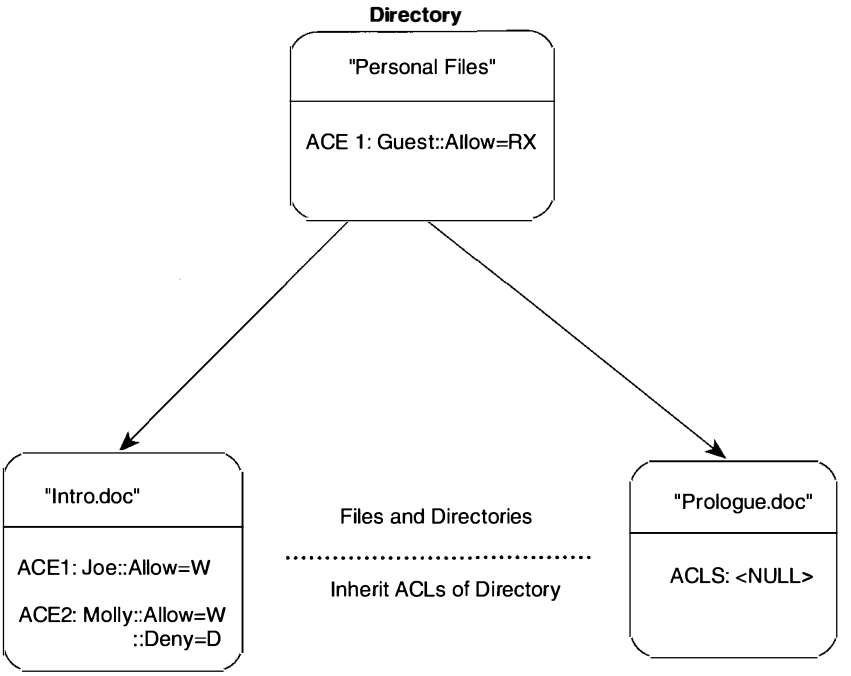


Figure 2-10. *Using ACL Inheritance*

group—called, say, "Engineers"—and grant users in that group read, write, and delete access to the file "Intro.doc" by specifying a single ACE ("Engineers::Allow=W").

Another Windows NT security feature allows auditing of important events, such as usage and access of server resources and user logons. The auditing facility generates audit logs whenever the designated trigger (for example, logon failure due to a bad password) is activated.

Finally, the memory protection security feature of Windows NT is enabled by starting each new process in distinct virtual address space. No process is allowed to access the address space belonging to any other process. (Windows NT provides specific mechanisms by which a process can share its address space in a secure manner. These mechanisms are discussed in Section 2.3.6, "Memory-Mapped File I/O and Sharing Memory.")

2.2.7 Interprocess Communication in Windows NT

In Chapter 1, you were introduced to the major IPC mechanisms available in the Windows NT environment. Figure 2-11 shows the position of each of these communication interfaces as they relate to the OSI/ISO reference model. Note that the NetBIOS and IPX/SPX interfaces are "low-level" transport layer interfaces that can function only when the proper transport stack is present. Thus, for you to use the IPX interface, an IPX transport stack must be

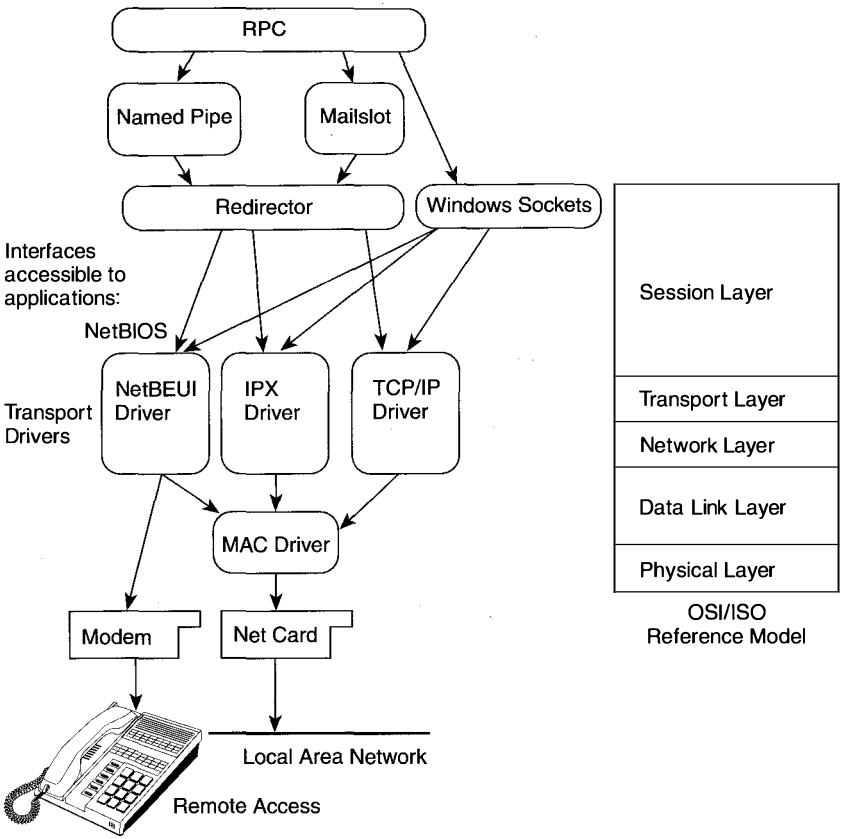


Figure 2-11. *Communication Interfaces in Relation to the OSI/ISO Model*

present on the system. The Windows Socket, Named Pipe, and Mailslot interfaces are higher-level interfaces and provide transport-stack transparency. For example, the Named Pipe interface is functional no matter which interface—the NetBEUI transport stack or the IPX/SPX transport stack—the computer supports. Notice that a redirector must be present for Named Pipes and Mailslots to function.

As Figure 2-11 shows, WinSock can operate without the redirector and can be used as an IPC mechanism regardless of the underlying protocol. For example, WinSock can use the NetBEUI protocol. This is a complete departure from the traditional UNIX and NetWare environments in which sockets are traditionally tied to TCP/IP transports.

Finally, RPC allows applications to be independent of transport stacks and communication interfaces. It does so by exposing its interfaces at the highest level while hiding the transport or interface details from applications. As a general rule, distributed applications should be designed to use the highest communication interface. (For example, given a choice, a designer should use the Windows Sockets interface rather than the IPX/SPX interface.)

2.2.8 Networking Within VDM and Windows on Win32

Now we look at how networking works within each VDM and WOW. Recall that the VDM is the special subsystem of Windows NT that allows most MS-DOS applications to be executed on Windows NT. The Win16 subsystem, WOW, is launched within a VDM, and its main function is to emulate the behavior of its MS-DOS counterpart (for example, Windows 3.1). WOW accomplishes this by thunking each Win16 API to the corresponding Win32 API. Because the Win32 APIs are supported by the Win32 subsystem, WOW is simply a client of the Win32 subsystem.

Networking in WOW is no different from networking in Windows. WOW thunks the networking APIs to their Win32 counterparts and returns the resulting values to the application. The Windows and MS-DOS programming interfaces such as **INT 2A, INT 2F,** and **INT 5C** are mapped to the proper entry points of the Windows NT redirector.[8] Windows networking functions, such as 16-bit WinNet APIs or the NetBIOSCall(), are mapped to their corresponding Win32 functions. The most important difference between the Windows and WOW architectures is the absence of **Windows virtual device drivers (VxD)**, which WOW does not support. (This affects only those vendors who supply VxDs for Windows and who must therefore provide proper drivers for Windows NT. The absence of the VxD drivers does not affect Windows applications as long as the proper Windows NT drivers are installed.)

In other words, you can view networking in WOW (or VDM) as functionally equivalent to networking in Windows (or MS-DOS). Hence, the guidelines described in this book for networking in Windows can be used to understand networking for Win16 (or MS-DOS) applications running on Windows NT.

2.3 SALIENT WIN32 PROGRAMMING FEATURES

This section presents some of the salient programming features of Windows NT, namely error and structured exception handling, threads, synchronization, semaphores, mutex, events, critical sections, overlapped I/O, Memory Mapped File I/O, and control-C handling. For each feature, we will discuss its basic concept. Code fragments illustrate how to use each feature and, when appropriate, its correct use in a distributed application. If you are familiar with these programming features, you may want to skip to Section 2.4, which discusses the design and implementation of a network service using Win32 Service APIs and presents the sample server template. In the rest of the book, this server template will be modified suitably to illustrate use of one IPC mechanism versus another.

8. The generic Microsoft redirector provides access to Microsoft-compatible workstations and servers. The Novell redirector supports accessing NetWare workstations and servers.

Here in Section 2.3 we illustrate the use of the advanced programming features of Windows NT in the context of a real-time, multiple-process control program. Each process handles a different device—such as a boiler, a water pump, or a ventilation duct—by means of some process control hardware. Assume each process can communicate with the device by a combination and software and hardware. (The details of the process-control hardware and software, and their interaction, are not important in the present context.) Also assume each process writes the current status of the device to a single database file at a certain frequency. For example, a pump monitoring program might update the database every five minutes on the status of the water pumps. Each process may contain one or more threads. Assume a "master" process presents the latest statistics and event logs to the user.

2.3.1 Error Handling in Windows NT

Windows NT supports all the Win32 programming interfaces and follows the error-handling techniques set by Win32 programming standards. In Win32, most APIs return a boolean value; a return value of TRUE indicates success, and FALSE may indicate failure or noncompletion of a function. When a Win32 API returns FALSE, applications can call the **GetLastError()** API to determine the cause of the error (or delay). The GetLastError() API returns a DWORD containing an appropriate error code. Applications can then use the **FormatMessage()** API to retrieve a system message associated with the error code, provided a message corresponding with the error code exists and the appropriate message file can be located. The next code fragment illustrates how to use both APIs. It also shows the definition of the function **PrintWin32Error()** used throughout this book.

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
//
// A function to retrieve Win32 error and print a message
//
DWORD PrintWin32Error ( LPSTR pszErrorString)
{
    DWORD dwRc;
    WCHAR wszMsgBuf [ FORMAT_MESSAGE_MAX_WIDTH_MASK + 1 ];
    // Retrieve the error code
    dwRc = GetLastError();
    printf("Error: %ld returned from [%s]\n",
        dwRc, pszErrorString);
    // Search system message tables and resources for an
    // error message associated with the error returned
    if (FormatMessage (     // Format option

        FORMAT_MESSAGE_FROM_SYSTEM,

        FORMAT_MESSAGE_MAX_WIDTH_MASK,
```

```
                                    NULL,       // Message source
                                    dwRc,       // Error Code
                                    0,          // Language Specifier
                                    wszMsgBuf,  // Message Buffer
                                                // Message Buffer Size

        FORMAT_MESSAGE_MAX_WIDTH_MASK,
                                    NULL
                ))
        printf("Error Message: %ws\n", wszMsgBuf);
    return dwRc;
}
int _cdecl main (int argc, char ** argv)
{
    // Call RemoveDirectory() Win32 API with a nonexistent
    // directory name to get an error.
    if (!RemoveDirectory ( L"NonExistentDirectory"))
    {
        PrintWin32Error ("RemoveDirectory");
    }
    return (0);
}
```

2.3.2 Structured Exception Handling

Exceptions are not new phenomena. Windows NT provides system-level support for exception handling. An exception is defined as the occurrence of an unexpected hardware or software error during normal processing of a program. Exception handling allows applications to recover from such unexpected conditions. A software exception can occur in a number of ways, the most common of which happens when poorly designed software tries to access code or data via a null pointer. Other examples are failure of disk while writing to it, or the breakdown of a network connection because of unexpected reasons (such as a remote system crash). Hardware exceptions due to memory failure are also not uncommon. By providing a way to handle these exceptions, Windows NT allows designers to write robust client/server applications.

Of course, when an application makes a normal Win32 API call, the API will always try to return a valid error code. Thus, the application writer does not have to watch for exceptions all the time. However, it is a good idea to safeguard the application against exceptions in areas when they can be disastrous. Some examples of exception-prone areas are listed here:

- Exception due to division by zero.
- Accessing a buffer or pointer supplied by an external agent. For example, a server in a client/server system can receive a buffer from any number of clients. The server must use the client-supplied buffer with caution, because there is no guarantee the client supplied a correctly formatted buffer.

- Using RPC. When RPC is used for IPC, applications should always be prepared for exceptions. (RPC is covered in detail in Chapter 4, "Remote Procedure Call Programming in Windows NT.")
- Memory allocation failure. It is possible a memory allocation failure will not be detected, and that an application may start using the uninitialized memory pointer. This causes a software exception.

Microsoft C compilers provide structured exception capabilities for a C or C++ program by using **try-except** and **try-finally** constructs. These constructs help make exception-safe code, and quite often they allow structured error-handling capability without cluttering the entire code with if-then-else clauses. Essentially, the code is divided into two parts: the safeguarded code and the exception handler or termination handler. Figure 2-12 shows this structured exception handling.

As the Figure 2-12 schematic shows, when an application uses a try-except construct, it installs an exception handler that is called whenever there is an exception in the safeguarded code. The application can choose either to process an exception or to pass it on to a higher-level exception handler. For example, when it is manipulating strings in the safeguarded code, an application can get an access violation resulting from bad pointers. If this happens, it can choose to catch the ACCESS_VIOLATION exceptions and pass all other exceptions, such as hardware failure, to higher handlers. The

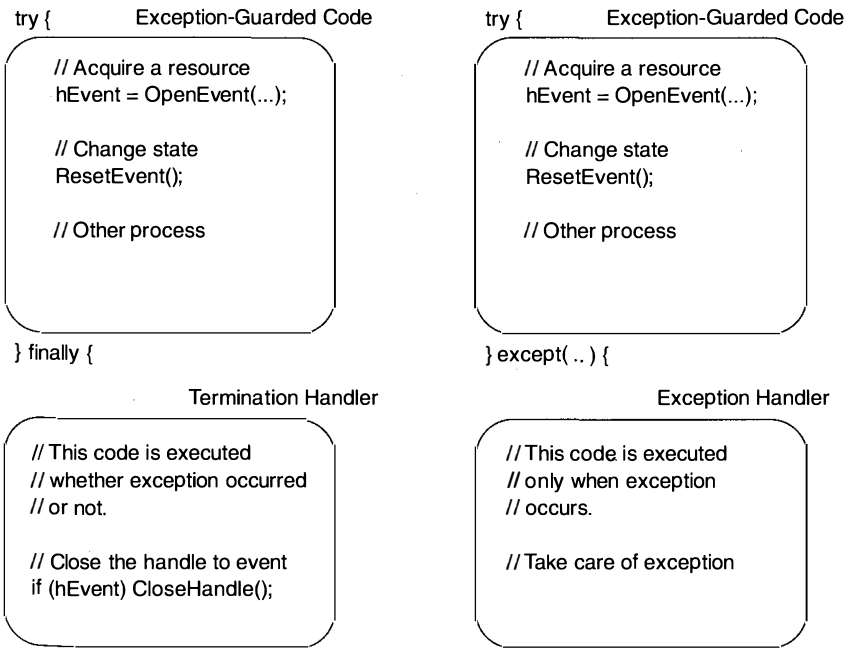


Figure 2-12. *Structured Exception Handling*

next code fragment shows how the program safeguards itself from a potentially bad pointer by accessing it within a try block. The code fragment also shows how a main code path can be simplified by using exception handling. This is accomplished by *not* checking the error from the **malloc()** memory allocator, thereby saving one instruction: if(pchMyString==NULL). Should malloc() fail, an exception will be generated and caught by the exception handler (where the program can check for the cause of error). Note that this exception handler handles only the access violations and passes all other exceptions to upper-layer handlers.

```
//
// The code is using a string pointer given by a third party
// so it may be a bad pointer
//
try
{
        // Allocate a new string. While allocation can fail
        // returning NULL, Exception can occur if supplied
        // string pointer is bad.
        pchMyString = (CHAR *) malloc (strlen(pchSuppliedString) +1);
        // Exception can occur if "malloc" failed above
        strcpy ( pchMyString, pchSuppliedString);
}
except ( GetExceptionCode() ==  // Handle access violations
    EXCEPTION_ACCESS_VIOLATION ? EXCEPTION_EXECUTE_HANDLER:
                                 // Pass the rest
                                 EXCEPTION_CONTINUE_SEARCH
        )
{
        printf("An exception occurred: %ld\n", GetExceptionCode();
}
```

The try-finally construct can also be used to install a termination handler, as Figure 2-12 showed. In this case, the code fragment present in the finally { .. } block is always executed, regardless of whether an exception occurs. This construct is very useful for ensuring that a resource (such as an event, mutex, critical section, and others) acquired in the try { ... } block is always released.

For performance reasons, because a compiler sets up an exception context around the safeguarded code, a safeguarded code block should not contain return statements. A return from this context can cause as many as 100 instructions (depending on the processor) to be executed. This may be all right for exceptions, but not for regular (nonexceptional) processing. By using simple control statements (for example, switch, if-then-else), you can avoid having the code return from within an exception context during normal processing.

2.3.3 Threads

At its simplest level, a program is composed of a sequence of instructions. When a program starts up, the operating system creates a process for the

application and the application starts to execute the instructions in a linear sequence until a jump or call instruction changes the sequence. Each process contains its own private address space, data and code segments, and any resources created or opened by the application. Of course, a process can start one or more child processes. Each child process executes in its own address space and contains its own data and code segments. A child process may share none, some, or all of the resources of its parent process.

In an operating system such as MS-DOS, each process contains a single thread of execution. However, in Windows NT, a process can have multiple threads of execution. Every thread shares the process's global variables and virtual address space and any resources owned by the process. Thus, a file opened by one thread can easily be read by another thread within the same process, provided the second thread can get at the handle of the file in some way (traditionally, through a global variable). However, each thread has its own unique call stack and CPU state and its own **thread local storage (TLS)** in which thread-specific static data is stored.

To benefit from their parallelism, a typical application can use multiple threads. For example, a word processor program can use one thread to respond to user requests and another thread to print a document. Threads are essential to network-aware applications, especially to application servers. Figure 2-13 shows a database server in which each new client is serviced by a new thread in the server, which reduces the wait time for the other clients. Of course, each thread adds to the system overhead for the process, even though the additional overhead per thread is generally lower than the overhead caused by each new child process. Also, with the presence of two or more threads of execution, synchronization and mutual exclusion become issues that application designers need to address.

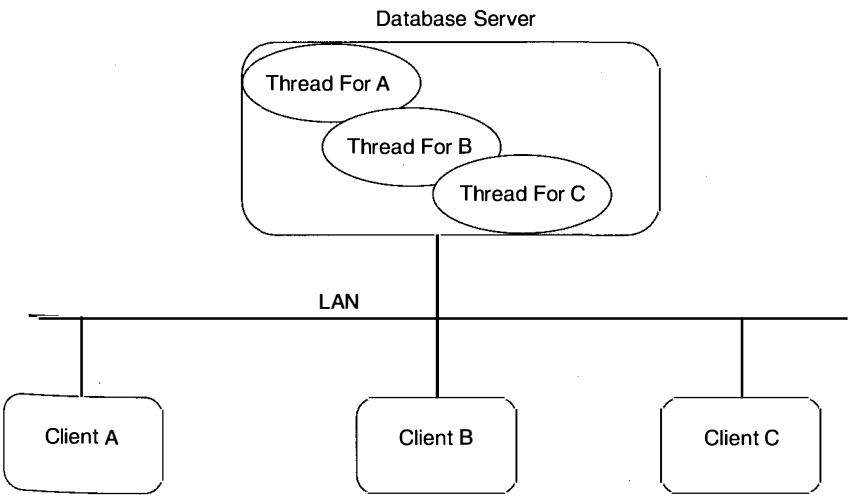


Figure 2-13. *Server with Multiple Threads*

Table 2-2 shows the Win32 APIs available to create, terminate, and manipulate threads in a Win32 application. For comparison, process-related functions are shown as well.

Table 2-2. *Win32 APIs for Manipulating Threads and Processes*

Win32 Function	*Description*
Functions Related to Threads	
AttachThreadInput()	Attaches one thread to another
CreateRemoteThread()	Creates a thread in another process
CreateThread()	Creates a new thread
TerminateThread()	Kills a thread
ExitThread()	Exits the current thread
GetCurrentThread()	Returns a handle to the current thread
GetCurrentThreadId()	Returns the current thread ID
GetExitCodeThread()	Returns thread termination code
GetThreadPriority()	Returns specified thread priority
GetThreadTimes()	Obtains timing information for a thread
ResumeThread()	Restarts a suspended thread
SuspendThread()	Suspends a thread
SetThreadPriority()	Sets thread priority
Sleep()	Suspends a thread for a set time
SleepEx()	Suspends a thread until I/O completes
Functions for Manipulating Thread Local Storage	
TlsAlloc()	Allocates thread local storage index
TlsFree()	Frees thread local storage index
TlsGetValue()	Returns a TLS value
TlsSetValue()	Sets a TLS value
Functions Related to Processes	
CreateProcess()	Creates a new process and thread object
DuplicateHandle()	Duplicates an object handle
ExitProcess()	Exits the current process
GetCommandLine()	Returns a pointer to the command line
GetCurrentProcess()	Returns a handle to the current process
GetCurrentProcessId()	Returns the current process ID
GetEnvironmentStrings()	Returns a pointer to the environment block

(continued)

Table 2-2 continued

Win32 Function	*Description*
GetEnvironmentVariable()	Returns process environment variable
GetExitCodeProcess()	Retrieves process termination code
GetPriorityClass()	Returns priority class for a process
GetProcessShutdownParameters()	Retrieves process shutdown parameters
GetProcessTimes()	Obtains timing information for a process
GetStartupInfo()	Gets startup information for current process
LoadModule()	Loads and executes a program
OpenProcess()	Returns a handle to a process object
SetEnvironmentVariable()	Sets an environment variable
SetPriorityClass()	Sets priority class for a process
SetProcessShutdownParameters()	Sets process shutdown parameters
TerminateProcess()	Kills a process and its threads
WaitForInputIdle()	Waits until a process is idle
WinExec()	Runs a program

The following code fragment shows a method of creating multiple threads in a process. In this example, the main program sets a global variable for the number of threads it starts. Each thread decrements the global count and prints its thread sequence number.

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
// Global Variable Shared by all threads
int nGlobalCount = 0;
// Main function which starts out each thread
int _cdecl main (int argc, char ** argv)
{
        int i, nThreads = 5;
        DWORD dwThreadId;
        // Determine the number of threads to start
        if (argc > 1)
        {
                nThreads = atoi ( argv [1] );
        }
        // Set the global count to number of threads
        nGlobalCount = nThreads;
        // Start the threads
        for ( i = 1; i <= nThreads; i++)
        {
                if (CreateThread ( NULL,       // Choose default security
                                   0,          // Default stack size
                                   ThreadProc, // Routine to execute
```

```
                                        (LPVOID) &I, // Thread parameter
                                        0,     // Immediately run the thread
                                        &dwThreadId // Thread Id
                                        )==NULL)
                {
                        PrintWin32Error("Creating Thread");
                        printf("Error Creating Thread#: %d\n", i);
                        return (1);
                }
        }
        // Wait for all threads to complete
        while (nGlobalCount)
        {
                printf("Global Thread Count: %d\n", nGlobalCount);
                Sleep ( 1 );
        }
        return (0);
}
// Thread Routine
DWORD ThreadProc (LPDWORD lpdwThreadParam )
{
        // Print Thread Number
        printf("Thread #: %d\n", *lpdwThreadParam);
        // Reduce the count
        nGlobalCount--;
        // End of thread
        return (0);
}
```

If you are unfamiliar with threads, I recommend you execute this sample program so you understand the thread creation model. You may also want to use software tools, such as PMon and PView from the Microsoft Software Development Kit, to understand the system overhead associated with threads and processes.

Thread Usage Guidelines for Client/Server Systems

Windows NT-based CSSs can benefit immensely from the careful use of threads. This is especially true for servers, which can efficiently serve multiple clients by using multiple threads. However, there is always the possibility that threads will be underused or overused. While it is impossible to define a quantitative measure for proper thread usage for all applications, here is a list of issues to be considered:

- *Thread Overhead:* Each thread adds to the system overhead of the overall process. Therefore, a process should not have too many threads. As a general rule, a process with more than 10 threads should be very carefully scrutinized.
- *Peak Load:* A server with too few threads may provide unacceptable service at peak load. The designer should determine the peak load for a server and define the upper bound on the number of threads per server process accordingly.

- *Scalability:* A server designed to serve anywhere from a few clients to hundreds requires considerable performance analysis. Starting a new thread for each incoming client request could adversely affect the performance of the server. The server might be designed to use one thread to serve a group of clients.
- *Application Overhead:* An application with multiple threads needs to have good control over the execution of the threads. Multiple threads often need to access global or shared data, which can lead to extensive mutual exclusion code within the application. The complexity of the code increases dramatically with the increase in the number of threads that are accessing common resources. Similarly, multiple threads may need to synchronize their activities occasionally, which means adding synchronization code to the application. Therefore, designers must be aware of the increase of application overhead associated with multiple threads.
- *Debugging Overhead:* This is the overhead incurred during the development and testing of applications that are using multiple threads. With the increase in the number of threads, the debugging becomes increasingly complex. The overhead can be contained by careful design and extensive stepwise testing.[9]

A server specifically designed to handle a large number of clients might use client queuing and thread pools, in which the incoming clients' requests are put in a queue and served by the next available "worker" thread. The server maintains a pool of worker threads, the number of which may be dynamically changed as the load on the server varies.[10] Section 2.4 uses this technique in its sample server design.

By understanding the issues in this list, and by actively monitoring the performance of the applications during their development, the designer should be able to harness the power of threads and create efficient Windows NT CSSs.

2.3.4 Interprocess and Intraprocess Synchronization

In a multiprocessing system such as Windows NT, a process may need to synchronize its activities with other processes (including child processes). Similarly, threads within a process may need to synchronize their activities with each other to maintain resource integrity. These synchronization needs are especially

9. In this case, "stepwise testing" refers to the approach in which the load on an algorithm is incremented in a stepwise fashion. In other words, first test with one thread, then with two threads, and so on, until the algorithm has been tested with a certain maximum number of threads.

10. Readers familiar with queuing theory will realize that this scheme creates a (M/M/1) queue and can provide optimum solution.

important for server applications that contain multiple threads. Each such thread needs to access a set of server resources on behalf of one or more clients and at the same time meet the resource allocation limits and criteria.

Windows NT provides semaphore, mutex, and event facilities to achieve interprocess synchronization. These facilities can also be used by a multithread program to achieve intraprocess synchronization. Windows NT also provides critical section objects, which are better suited for synchronizing threads in a process. Finally, Windows NT provides APIs for synchronized access to 32-bit variables, using multiple threads either in the same or in distinct processes. These APIs make it possible to perform and fine-tune consistent increments or decrements on shared variables and examine the result of such operations. While it is beyond the scope of this book to examine these synchronization mechanisms in detail, a brief introduction is called for.

An important characteristic of all synchronization objects is their "state," which can be either "signaled" or "nonsignaled."[11] The Win32 API set contains several APIs to create, open, or manipulate the synchronization objects. The states are used by the "wait" functions. Typically, a calling thread blocks on a wait function and remains blocked as long as the object remains nonsignaled. Once the object reaches a signaled state, the wait function completes, which tells the caller the object has transitioned to a signaled state. Table 2-3 lists the wait functions used in conjunction with semaphores, mutex, and event objects. Their use is shown in the context of the object being discussed.

Table 2-3. *Wait Functions Used with Synchronization Objects*

Win32 Wait Function	***Description***
WaitForMultipleObjects()	Waits for multiple waitable objects. Can be made to return either when all objects are signaled or when any one of them is signaled.
WaitForMultipleObjectsEx()	Same as above, plus can wait for I/O completion.
WaitForSingleObject()	Waits for single waitable object.
WaitForSingleObjectEx()	Same as above, plus waits for I/O completion.
MsgWaitForMultipleObjects()	Waits for multiple object handles or message input queue. Wait completes when objects are signaled or a message, such as a mouse click, arrives in the input queue.

11. Also referred to as "set" or "reset" in some operating system literature.

Semaphore

A Win32 semaphore is an object that can be shared among multiple processes and can be used for flow-control activities. A semaphore object can be described by the tuple {semaphore value, semaphore state}. A semaphore state can be either signaled or nonsignaled, and its value can be any nonnegative value. When the value of the semaphore is nonzero, it is said to be in a signaled state. Win32 APIs can be used for changing the value of the semaphores, changing their states from signaled to nonsignaled and vice versa. The semaphore is used in conjunction with wait functions to synchronize the activities of two or more threads from either the same or distinct processes. In the process control program example, imagine that the program is used to acquire, use, and release water pumps out of a pool of available water pumps. Further imagine that the program has multiple processes, each of which is vying for the limited resource, namely the water pumps. In this case, a semaphore can be used by these processes to determine which process can acquire a pump from the pool of pumps. Once a process has acquired a pump, it is free to use it until it releases the pump back to the general pool.

A process can create a semaphore by calling the **CreateSemaphore()** API. This API can be used to set the initial value, a maximum value, and the name of the semaphore. The semaphore value is originally set to the initial value, and can be defined by the mathematical expression: zero ≤ initial value ≤ current value < maximum value. A named semaphore can be shared among multiple processes; an unnamed semaphore is used among multiple threads in a process. Once a semaphore has been created, it can be opened by other processes or threads by calling the **OpenSemaphore()** API. The current value of a semaphore dictates the state of the semaphore. A semaphore with a positive current value is said to be signaled. It is nonsignaled when its current value becomes zero.

The current value of a semaphore changes when it is decremented by the functions that are used to block or wait on a semaphore, for example, the **WaitForSingleObject()** API, or incremented by release functions, such as the **ReleaseSemaphore()** API. Whenever a semaphore is in a signaled state, the wait functions complete and decrement the semaphore value by one. When the semaphore value becomes zero, it transitions to a nonsignaled state. Thus, an application will block on the nonsignaled semaphore. The function ReleaseSemaphore() can be used to increment the value of a semaphore by a specified amount so long as it does not increment the value beyond the maximum specified when the semaphore was created.

The following code fragment shows how child processes use semaphores to determine how many pumps can be started in the process control application. The master process makes three pumps available out of a maximum of three. Then each child process opens, and waits on, the semaphore. If the number of available pumps (that is, the current semaphore value) is greater than zero,

the process will complete the WaitForSingleObject() function. Of course, the wait function decrements the semaphore value (that is, the available pump count). Hence, after using the pump, the child process increments the semaphore value by calling ReleaseSemaphore().

```
//
//  Master Process
//
int main ()
{
    HANDLE hSem;
    // Indicate that all 3 pumps are idle at this time
    if((hSem = CreateSemaphore( NULL,            // Default security
                                3,               // Available pumps
                                3,               // Maximum pumps
                                L"PumpSemaphore" // Semaphore name
                                )) == NULL)
    {
            // Error occurred
            PrintWin32Error("CreateSemaphore");
            return (1);
    }
<Rest of master program >
}
//
// Child Processes
//
int main ()
{
    DWORD dwStatus;
                // Request for a pump
    HANDLE hSem;
    if ((hSem = OpenSemaphore(
                              SEMAPHORE_ALL_ACCESS,   // Access mode
                              FALSE,                  // Inherit flag
                              L"PumpSemaphore"
                             )) == NULL)
    {
        // Error occurred
        PrintWin32Error("OpenSemaphore");
        return (1);
    }
        // Wait until a pump becomes available
        dwStatus = WaitForSingleObject ( hSem, WAIT_TIMEOUT_VALUE);
        switch ( dwStatus )
    {
    case WAIT_OBJECT_0:
        // Semaphore is signaled and its value
        // decremented by 1
        <..Use the pump..>
        // Now release the pump back
        if (!ReleaseSemaphore (hSem,
                               1,
                               NULL))
        {   // Error occurred
```

```
                PrintWin32Error("ReleaseSemaphore");
        };
        break;
    case WAIT_ABANDONED:
        // Thread owning the semaphore
        // terminated without releasing it
        < Use the pump and increment value >
        break;
    case WAIT_TIMEOUT:
        // Wait has timed out
        break;
    }
       < Rest of the Child Process Code>
}
```

It is not necessary to establish a static "master-child" relationship between the processes. In fact, all processes can call the CreateSemaphore() API. The function CreateSemaphore() behaves like an OpenSemaphore() if the semaphore has already been created. In this case, GetLastError() returns ERROR_ALREADY_EXISTS, while CreateSemaphore() returns a handle to the existing semaphore.

Mutex

Often, designers need to create a system in which exactly one entity must have complete access to a resource. Thus, ownership of such a resource is mutually exclusive. Windows NT provides mutex for enforcing mutual exclusion between multiple processes (or threads) when they are accessing a shared resource. Only one thread can have complete ownership of a mutex at any time, and the rest of the threads will block, waiting for release of the mutex. In the example of a real-time process control program, you can use a mutex to guard the database file that reflects the complete status.

Imagine that the database file is periodically opened by each process and updated with latest statistics, and that you have one process dedicated to monitoring each operating water pump. While each pump-monitoring process updates distinct portions of the database, all these processes need to update the global variable, the total number of gallons of fluid pumped in a given period. Without mutual exclusion, this variable would corrupt very fast. It makes sense to protect the database file by using a mutex. If only one process can update the fields of the database at any time, you ensure data integrity and consistency.

A mutex is first created by calling the **CreateMutex()** API, which returns a handle to the mutex. The mutex creator thread can take immediate ownership of the mutex at create time. You can specify the name of the mutex by using the CreateMutex() API and create a named mutex that other processes can open. (An unnamed mutex is only used among the threads of a process.) By default, after a mutex has been created, it remains in a signaled state, waiting for any thread to claim ownership by calling any wait function [for

example, WaitForSingleObject()]. Once a thread has acquired the ownership of a mutex (that is, the Wait function completes without an error), the mutex goes into a nonsignaled state, causing other processes or threads to block and wait for the mutex to become available. The owner thread can release the mutex by calling the **ReleaseMutex()** API, which puts the mutex into a signaled state. (The owner thread can keep waiting on the same mutex and it will not block. This is useful so a thread does not deadlock trying to get a mutex it already holds. However, in such a situation, each wait function that was not blocked must be matched by a corresponding ReleaseMutex() function call.)

Once a mutex has been created, other threads or processes can access it by using the **OpenMutex()** API. In situations where it is unclear as to which process will start first, each process can use CreateMutex(); this is possible because CreateMutex() returns a valid handle to the mutex that is behaving like OpenMutex() when the named mutex already exists.

The following code is a modified version of the fragment shown earlier. In this example, each process acquires a pump by using the semaphore, and then after it has the pump, the process uses a mutex to achieve mutual exclusion in updating the total flow of fluid through all the pumps. For simplicity, you should assume the data is kept in a file and modified by a common function, **SetFlowParameter()**.


```
//
// Common functions used to manipulate mutex
//
HANDLE CreateMux ( LPWSTR pwszMutexName, BOOL fInitial)
{
    HANDLE hMutex = NULL;
    if ((hMutex = CreateMutex ( NULL,          // Default Security
                                fInitial,      // initial ownership
                                pwszMutexName  // Mutex Name
                              ))=NULL)
    {
        // Error occurred
        PrintWin32Error("CreateMutex");
    }
    return hMutex;
}
HANDLE OpenMux ( LPWSTR pwszMutexName, BOOL fInherit)
{
    HANDLE hMutex = NULL;
    if ((hMutex = OpenMutex ( SYNCHRONIZE,     // Access Flag
                              fInitial,        // initial ownership
                              pwszMutexName    // Mutex Name
                            ))=NULL)
    {
        // Error occurred
        PrintWin32Error("OpenMutex");
    }
```

```
    return hMutex;
}
DWORD AcquireMux (HANDLE hMutex, DWORD dwTimeOut)
{
    return WaitForSingleObject (hMutex, dwTimeOut);
}
DWORD ReleaseMux (HANDLE hMutex)
{
    DWORD dwRc = 0;
    if (!ReleaseMutex (hMutex))
    {
        dwRc = PrintWin32Error("ReleaseMutex");
    }
    return dwRc;
}
//
// Master Process
//
int main ()
{
    HANDLE hSem, hMutex;
    // Indicate that all 3 pumps are idle at this time
    if((hSem = CreateSemaphore( NULL,            // Default security
                                3,               // Available pumps
                                3,               // Maximum pumps
                                L"PumpSemaphore" // Semaphore name
                              )) == NULL)
    {
        // Error occurred
        PrintWin32Error("CreateSemaphore");
        return (1);
    }
    // Create a mutex to control total flow
    if ((hMutex = CreateMux ( FALSE,       // No initial ownership
                              L"PumpMutex" // Mutex Name
                            ))=NULL)
    {
        // Error occurred
        CloseHandle (hSem);
        return (1);
    }
    // Set the total gallons flowed thru pumps to zero
    dwStatus = SetFlowParameter (hMutex, 0);
    <Rest of master process program >
    // Close handles
    CloseHandle (hMutex);
    CloseHandle (hSem);
}
//
// All processes use the function shown below
// (potentially exported from a DLL)
//
DWORD SetFlowParameter (HANDLE hMutex, DWORD dwFlow)
{
    DWORD dwStatus;
    // Get ownership of the mutex
```

```
    dwStatus = AcquireMux (hMutex, INFINITE);
    if (dwStatus)
        return (dwStatus); // Error in acquiring mutex
    < Update the file containing global statistics >
    // Release ownership of the mutex
    dwStatus = ReleaseMux (hMutex);
    return dwStatus;
}
//
// Child Processes
//
int main ()
{
    DWORD dwStatus, dwFlowByThisPump;
    // Request for a pump
    HANDLE hSem, hMutex;
    if ((hSem = OpenSemaphore(
                                SEMAPHORE_ALL_ACCESS, //Access mode
                                FALSE,                // Inherit flag
                                L"PumpSemaphore"
                                )) == NULL)
    {
        // Error occurred
        PrintWin32Error("OpenSemaphore");
        return (1);
    }
    // Wait until a pump becomes available
    dwStatus = WaitForSingleObject (hSem, WAIT_TIMEOUT_VALUE);
    switch ( dwStatus )
    {
    case WAIT_OBJECT_0:
        // Semaphore is signaled and its value
        // decremented by 1
        <..Use the pump which set value of
        dwFlowByThisPump..>
        // Now release the pump back
        if (!ReleaseSemaphore (hSem,
                                1,
                                NULL))
        {
            PrintWin32Error("ReleaseSemaphore");
        };
        break;
    case WAIT_ABANDONED:
        // Thread owning the semaphore
        // terminated without releasing it
        < Use the pump and increment value >
        break;
    case WAIT_TIMEOUT:
        // Wait timed out
        break;
    }
    if (dwStatus ! = WAIT_OBJECT_0)
        return (1);
    // Obtain a handle to the mutex
```

```
    if ( (hMutex = OpenMux (
                                FALSE,          //Inherit Flag
                                L"PumpMutex"    // Name of the mutex
                                ))!=NULL)
    {
        // Wait to get the mutex and update global flow value
        SetFlowParameter ( hMutex, dwFlowByThisPump );
    }
    CloseHandle (hMutex);
   < Rest of the program >
}
```

Event

Events are extremely useful for notifying other processes of an occurrence of mutual interest, and they provide a simple, efficient, system-wide mechanism for interprocess notification and synchronization. Events cannot be used directly between processes resident on distinct machines.

The event use model is straightforward: A process creates a named event and sets its initial state to either signaled or nonsignaled. After that, other processes can open the named event and synchronize their activities, based on the state of the event. One or more processes can access an event and change its state from signaled to nonsignaled and vice versa. Thus, events can be used in creating a master-child synchronization in which a master process dictates the transitions of child processes from one state to another. Alternatively, events can be used for notification and synchronization between peer processes.

In the continuing example of a real-time process control system (so far, mostly monitoring pumps), you can imagine the master process triggering an emergency event that causes all processes to shut down. (This will be further illustrated by the code fragment you will see shortly.)

A process creates a named or unnamed event using the **CreateEvent()** API. An unnamed event can only be used for synchronization purposes by multiple threads within the same process. While creating the event, the caller can set the state of the event as either signaled or nonsignaled. An event can be set to the signaled state using the **SetEvent()** API. Based on the signaled-to-nonsignaled-state transition behavior, an event can be one of two types: a manual-reset event or an auto-reset event.

By using the **Open-Event()** API, another process (or thread) can access the event. Threads block on a nonsignaled event and are released from waiting as soon as the event becomes signaled.

Once a manual event has been signaled, it remains signaled until it is manually reset by calling the **ResetEvent()** API. Thus, one or more threads complete their wait functions as long the event remains signaled. On the other hand, an auto-reset event allows only one thread to be released when it changes from a nonsignaled to signaled state, and then it automatically changes event state to nonsignaled. When no threads are waiting on an auto-reset

event, the event remains signaled until it releases the first thread that waits on it. (An auto-reset event can be compared to a turnstile that allows only one person through at a time. A manual-reset event is more like a floodgate that lets water through until the gate is closed.)

The following code fragment shows how a manual-reset event is used by the master process to notify all pump-monitoring processes to shut down the pumps. The event is initially set to a nonsignaled state. A time-out value of zero is used to determine the state of the event in the child process. When the master process sets the event to a signaled state, the wait function indicates a transition, in which case the processes shut down the pumps.

```
//
// Master Process Control Program
//
int main ()
{
        HANDLE hEvent;
        // Create a manual-reset event
        if ((hEvent= CreateEvent ( NULL,           // Default security
                                   TRUE,           // Manual-reset event
                                   FALSE,          // Nonsignaled
                                   L"PumpShutdown" // Name of event
                                 ))==NULL)
        {   // Error processing
            PrintWin32Error("CreateEvent");
            return (1);
        }
        < Rest of Program >
        // Inform the pump-monitoring processes to shut down
        if (!SetEvent( hEvent))
        {   // Error Processing
            PrintWin32Error("SetEvent");
        }
}
//
// Pump-Monitoring Child Processes
//
int main ()
{
        HANDLE hEvent;
        DWORD  dwStatus;
        // Open the event - Mere synchronization is sufficient
        if ((hEvent = OpenEvent (SYNCHRONIZE,     // Access Flag
                                 FALSE,           // Inherit Flag
                                 L"PumpShutdown"  // Name of event
                                ))==NULL)
        {
            PrintWin32Error("OpenEvent");         // Error processing
            return (1);
        }
        do
        {
            < General Pump Monitoring activities>
```

```
            // Check the status of the event
            dwStatus = WaitForSingleObject ( hEvent, 0);
            switch (dwStatus)
            {
            case WAIT_OBJECT_O; // Pump Shutdown event is set!
                    printf("Prepare shutdown\n");
                    break;
            default:            // All other cases
            }
    }while ( dwStatus != WAIT_OBJECT_0);
    < Shut down the pump and terminate >
}
```

Critical Section

Critical-section objects are powerful programming devices that guarantee mutual exclusion between multiple threads in a process. Additionally, critical sections are designed to be lightweight, compared to other synchronization constructs (explained shortly). Much like the Win32 mutex, only one thread at a time can claim ownership of a critical section. Unlike a mutex, critical sections cannot be used for interprocess mutual exclusion. Traditionally, critical section objects are used to safeguard global resources shared among the threads in a process.

A critical-section object emerges when the **InitializeCriticalSection()** API is called on a CRITICAL_SECTION data structure. Typically, one thread in a process sets up the global resource, such as a data structure, and then initializes a critical section, which is used for enforcing mutual exclusion between the threads that are accessing the global resource. (Note that unlike other synchronization objects, critical sections are directly handled by the CRITICAL_SECTION data structure, and hence, there is no handle involved. This implies that when you are manipulating a critical section, there is no handle-to-object lookup involved. Object access control checks may also be unnecessary, because all the callers are threads of the same process. This explains why critical sections are lightweight compared to a mutex.)

Any thread that wants to get at the safeguarded resource calls the **EnterCriticalSection()** API. The thread calling EnterCriticalSection() blocks indefinitely until it has ownership of the resource. On successful completion of EnterCriticalSection(), the calling thread can safely proceed to manipulate the resource (for example, a global resource). Afterward, to relinquish ownership of the critical section, it must call the **LeaveCriticalSection()** API. This allows the next waiting thread to claim ownership of the critical section. A thread that already has ownership of a critical section will successfully complete multiple calls of EnterCriticalSection() on the given critical section. However, it must call a LeaveCriticalSection() corresponding to each call to EnterCriticalSection(). Once it has called LeaveCriticalSection() as many times as it has called EnterCriticalSection(), the thread loses ownership of the critical section.

Because EnterCriticalSection() blocks until the owner thread relinquishes control, it is very easy for poorly designed application threads to deadlock. Hence, it is crucial for application designers to check the algorithms carefully to ensure that any thread that has ownership of a critical section releases the critical section, even if the thread encounters errors. Using a try-finally construct around critical sections is highly recommended. In all events, an application should delete an initialized critical section by calling the **DeleteCriticalSection()** API before termination.

Now that you have examined the concept of the critical section, you can practice using it by applying it to the process control program mentioned earlier in this section. Assume the pump-monitoring process has multiple threads, each of which monitors a different set of parameters. For example, one thread could monitor the temperature gauge while another could be watching the water level. Each thread updates a global data structure that contains details of all monitored data. The following code fragment shows how to use a critical section for enforcing mutual exclusion between multiple threads when they access the global data structure.


```
//
// Pump Monitoring Process
//
// Global Data Structure which all threads access and update
struct {
    DWORD       dwWaterLevel;
    DWORD       dwTemparature;
    SYSTEMTIME dwTimeOfLastUpdate;
    ....
} GlobalData;
// Critical section used to enforce mutual exclusion
CRITICAL_SECTION CritSect;
//
// Main thread initializes the critical section and also
// deletes it. It also starts out one or more threads
// which use the critical section.
//
int _cdecl main (int argc, char ** argv)
{
    // Initialize a critical section
    InitializeCriticalSection ( &CritSect );
    // Start multiple threads
    < start threads >
    // Wait forever until it is time to shut down
    <..Loop and monitor all threads until signaled to shut down..>
    // Delete the critical section
    DeleteCriticalSection( &CritSect );
    return (0);
}
//
```

```
//Temperature Monitoring Thread
/
DWORD TempMonitor ( PVOID pvThreadParam)
{
    DWORD dwLocaltemp ;
    do {
        // Garner temparature data
        < .. Monitor Temparature .. >
        // Acquire critical section
        EnterCriticalSection ( &CritSect );
        // Update the global structure
        GetSystemTime (&GlobalData.dwTimeOfLastUpdate);
        GlobalData.dwTemperature = dwLocaltemp;
        // Relinquish control on Critical Section
        LeaveCriticalSection ( &CritSect );
    } while (FOREVER);
}
//
// Water Level Monitoring Thread
//
DWORD WaterMonitor (PVOID pvThreadParam)
{
    DWORD dwLocalWaterLevel;
    do {
        // Garner water level data
        < .. Monitor Water .. >
        // Acquire critical section
        EnterCriticalSection ( &CritSect );
        // Update the global structure
        GetSystemTime (&GlobalData.dwTimeOfLastUpdate);
        GlobalData.dwWaterLevel = dwLocalWaterLevel;
        // Relinquish control on Critical Section
        LeaveCriticalSection ( &CritSect );
    } while (FOREVER);
}
```

In this section, we have discussed the four programming constructs available in the Windows NT environment that can be used for interprocess and intraprocess synchronization and mutual exclusion. These constructs are semaphores, mutex, events, and critical sections. While critical section is a lightweight synchronization construct used among multiple threads in a process, the other three can be used for both intraprocess and interprocess synchronization. The semaphores are traditionally used to control access to a limited resource. When access to a resource must meet mutual exclusion criteria, applications can use the mutex constructs. The events are used to notify other processes of completion of a noteworthy step. Finally, critical sections are used in a process when multiple threads need to access a global data structure or resource. All synchronization objects can cause deadlock unless their usage is not carefully designed. Table 2-4 summarizes the behavior of the four synchronization objects.

Table 2-4. *Comparison of Synchronization Objects*

	Object			
	Semaphore	*Mutex*	*Event*	*Crit. Section*
Choice of initial value when the object is created	[0, n]	[0, 1]	[0, 1]	N/A
Maximum value	n	1	2	N/A
Value of signaled object	>0	1	1	N/A
Can the object be used in interprocess activity?	yes	yes	yes	no
Is the object used for mutual exclusion?	no*	yes	no	yes
Number of threads released when signaled	**	1	all	one
Does application get a handle when object is created/opened?	yes	yes	yes	no
Lightweight	no	no	no	yes
Used for system-wide notification?	no	no	yes	no

*Semaphores can be used for mutual exclusion, but they are usually not.
**Same as value of signaled object.
n: A long number

2.3.5 Overlapped I/O

Overlapped I/O allows Windows NT to host highly sophisticated scientific and I/O-intensive business applications. Simply put, the overlapped I/O feature allows an application to specify a function that is called when an asynchronous I/O call has been completed. It has two immediate gains: Because an asynchronous I/O function is called to begin with, the application does not block and continues to process other I/O or user requests; and, because the system calls the application-specified overlapped function on completion of the asynchronous I/O call, the application does not need to wait before using some synchronization technique. When used properly, the overlapped I/O feature can be used to develop almost-pipelined I/O processing engines with high throughput.

To understand how overlapped I/O helps maximize throughput, think of it in terms of cooking at home. Suppose you want to cook mashed potatoes and an entrée, and bake a pie. The pie and the entrée can be cooked in 30 minutes each; "instant" mashed potatoes take about 5 minutes. Obviously, if you cook each of the three dishes one after another, it will take at least 65 minutes.

This example uses a completely synchronous model, but this is seldom the case in real life. One way to reduce the cooking time would be to start the pie first (say it takes 10 minutes to get it ready). Then, start cooking the entree while checking the pie every so often, and cook the mashed potatoes at the end. This way, you could reduce the time to almost 45 minutes. This case represents an asynchronous model.

But wouldn't it be nice if you did not have to look in the oven (that is, poll) every 5 minutes to see if the pie is done? Of course. Most people use the timer on the oven, which goes off after a predetermined period, indicating the pie is done. This case used the overlapped I/O model, in which you are notified when an asynchronous activity is complete, leaving you free to concentrate on other activities.

The overlapped I/O feature is comparable to asynchronous procedure calls of VAX/VMS™ or 4.3 BSD UNIX. In VAX/VMS, an application can specify a handler for an I/O signal (SIGIO) that is sent from the kernel when an asynchronous I/O completes. The application must also re-enable the asynchronous signal handler after receiving the last SIGIO [Digital 82, Stevens 90].

Asynchronous I/O processing that uses overlapped structures can be handled in two ways: by associating an OVERLAPPED structure with an I/O call and subsequently checking the status of I/O by examining the OVERLAPPED structure, or by using special Win32 I/O APIs that call a completion routine when an I/O completes. These methods, which are covered in the following sections, should not be confused with special IPC-specific mechanisms available for asynchronous programming (for example, certain Windows Sockets APIs that send notification messages to the application at the completion of I/O).

Asynchronous I/O Using the OVERLAPPED Structure

A number of Win32 APIs, such as **ReadFile()**, **ConnectNamed-Pipe()**, **WriteFile()**, **CreateFile()**, **WaitCommEvent()**, and **TransactNamedPipe()**, take as a parameter the OVERLAPPED structure shown here:

```
typedef struct _OVERLAPPED {
    DWORD  Internal;
    DWORD  InternalHigh;
    DWORD  Offset;
    DWORD  OffsetHigh;
    HANDLE hEvent;
} OVERLAPPED;
//
// Prototype of ReadFile() function used to receive input from
// a file, a pipe, a socket, or a communication port
//
BOOL ReadFile(
    HANDLE  hFile,    // handle of file to read
    LPVOID  lpBuffer, // address of buffer that receives data
```

```
    DWORD   nNumberOfBytesToRead, // number of bytes to read
    LPDWORD lpNumberOfBytesRead,  // address of nos. of bytes read
    LPOVERLAPPED lpOverlapped     // address of structure for data
);
```

When the caller does not supply an overlapped structure (NULL in lpOverlapped parameter), the Win32 API blocks until the I/O is complete. However, when the caller specifies a non-NULL OVERLAPPED structure, the Win32 API in question performs asynchronous I/O. The caller associates a Win32 event with an OVERLAPPED structure (through the *hEvent* field of the overlapped structure) and supplies the structure as a parameter to a specific Win32 API, such as ReadFile(), shown above. When the API cannot complete immediately, it returns an error [FALSE in the case of the ReadFile() above] and the **GetLastError()** function returns ERROR_IO_PENDING. The application can use any synchronization function to wait for the event specified in the OVERLAPPED structure until the asynchronous I/O is complete. However, the caller must set the event to a nonsignaled state before making the Win32 call. When the asynchronous Win32 call [ReadFile(), for example] completes, it sets the event to signaled state. Whenever pertinent, the Win32 call returns the number of bytes transferred during the asynchronous call (using the *Offset* and *OffsetHigh* fields of the overlapped structure). For example, the ReadFile() API will return the number of bytes asynchronously read into the user-supplied buffer.

Alternatively, an application can use a special Win32 API, called **GetOverlappedResult()**, to determine the status of the asynchronous call. In other words, the GetOverlappedResult() API is a convenience function that allows an application to determine the status of an overlapped I/O call without using the event lookup mechanism just described. The prototype of the function is shown here:

```
BOOL GetOverlappedResult(
   HANDLE hFile,                 // handle of file, pipe, or comm. device
   LPOVERLAPPED lpoOverlapped,   // address of overlapped structure
   LPDWORD lpcbTransfer,         // address of actual bytes count
   BOOL fWait                    // wait flag
);
```

In cases when the original asynchronous call has not completed, the GetOverlappedResult() API returns FALSE and the GetLastError() API returns ERROR_IO_INCOMPLETE. The caller specifies the handle of object (using *hFile*), which is the target of the asynchronous I/O operation and the OVERLAPPED structure that was passed as a parameter to the original asynchronous call. Completion of an asynchronous call is indicated by GetOverlappedResult() returning TRUE. The caller can specify if GetOverlapped() should block while waiting for asynchronous call to be complete (by setting *fWait* to TRUE). Because GetOverlappedResult() uses the

event handle set in the OVERLAPPED structure, the event should be a manual-reset type. This way, both the calling application and the GetOverlapped() implementation can wait on the event.[12] Note also that the caller should not use the same OVERLAPPED structure and event pair for distinct asynchronous calls between the time an asynchronous operation has started and the end of asynchronous operation. This is necessary because the OVERLAPPED structure contains the "progress report" on a given asynchronous call, and mixing data about one asynchronous I/O call with another is a way to guarantee chaos.

The following code fragment shows the use of the OVERLAPPED structure in conjunction with the WriteFile() in printing a large buffer to a **line printer (LPT)** port. The advantage of using overlapped I/O is obvious: the main application does not block while printing a potentially large block of data.

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define LARGE_BUFFER_SIZE (64 * 1024)
#define SLEEP_TIME        (5L)
int _cdecl main (int argc, char ** argv)
{
    HANDLE hPort, hEvent;
    DWORD  dwRc, i, dwBytesWritten, dwBufSize;
    OVERLAPPED ol;
    argv++;
    argc--;
    if (argc)
    {
        dwBufSize = atoi(*argv);
        if (dwBufSize > LARGE_BUFFER_SIZE)
        {
            dwBufSize = LARGE_BUFFER_SIZE;
        }
   }
    else dwBufSize = LARGE_BUFFER_SIZE;
    // Open the LPT1 port in OVERLAPPED mode so that it
    // main thread can continue to function while
    // printing asynchronously
    if ((hPort  = CreateFile(L"LPT1",              // Open LPT1 port
                             GENERIC_WRITE,        // Access mode
                             0,                    // Share mode (Exclusive)
                             NULL,                 // Default security
                             OPEN_EXISTING,        // Open mode
                             FILE_ATTRIBUTE_NORMAL, // Attribute
                             FILE_FLAG_OVERLAPPED,
```

12. Recall from the earlier discussion that an auto-reset event will revert to a nonsignaled event state as soon as it lets one thread pass. Depending on what thread waited on the event first, the next caller will block forever, because the asynchronous completion mechanism triggers the event only once.

```
                        NULL // No Template
                        ))==NULL)
{
    PrintWin32Error("Create File on LPT1");
    return (1);
}
// Set up a large buffer to be sent to printer
CHAR szBuffer [ LARGE_BUFFER_SIZE ];
for ( i = 0; i < dwBufSize; i += 80)
{
    sprintf(szBuffer + i, "The Count is %ld\n\r", i);
}
// Create a manual-reset event for OVERLAPPED I/O
if ((hEvent = CreateEvent ( NULL, TRUE, FALSE, NULL))
    ==NULL)
{
    PrintWin32Error("CreateEvent");
    goto CleanupAndReturn;
}
// Set up an overlapped structure
memset ((PVOID)&ol, 0x00, sizeof(OVERLAPPED));
ol.hEvent = hEvent;
// Send a large buffer to the port to be
// printed asynchronously
if (WriteFile ( hPort,           // Handle to the LPT port
                szBuffer,        // Buffer to send
                dwBufSize ,      // Bytes to write
                &dwBytesWritten, // Actual bytes written
                &ol              // Overlapped structure
            ))
{
    printf("WriteFile returned TRUE. Wrote: %ld bytes\n",
        dwBytesWritten);
    goto CleanupAndReturn;
}
// WriteFile Returned FALSE. Ensure that I/O is pending
if (GetLastError() != ERROR_IO_PENDING)
{
        PrintWin32Error("WriteFile");
        goto CleanupAndReturn;
}
// Now we poll every so often, waiting for async printing
// to complete
while (!(GetOverlappedResult ( hPort,          // LPT port handle
                               &ol,            // Overlapped structure
                               &dwBytesWritten, // Bytes
                               FALSE           // Do not block
                               )){
    if ((dwRc = GetLastError())!= ERROR_IO_INCOMPLETE)
    {
        PrintWin32Error("GetOverlappedResult");
        break;
    }
    // Complete other activities while buffer is printing
    // In this example, we will sleep
        Sleep ( SLEEP_TIME);
```

```
            printf(".");
        }
        // Done printing
        printf("Bytes Written to LPT port: %ld\n",
            dwBytesWritten);
CleanupAndReturn:
        // Close handles
        CloseHandle (hEvent);
        CloseHandle (hPort);
        return (0);
}
```

Using Asynchronous Completion Routines

The code fragment you've just seen has a disadvantage in that the main application needs to poll every so often to determine the status of the asynchronous I/O call. For heavily I/O-intensive applications, such as a print server or a disk backup program, polling is inefficient. For such applications, Win32 offers extended APIs, such as **ReadFileEx()** and **WriteFileEx()**, which call an application-supplied completion routine when an I/O operation is complete. This frees up the main application to do other operations. A prototype of WriteFileEx() and an asynchronous completion routine is shown here:

```
VOID WINAPI FileIOCompletionRoutine(
    DWORD fdwError,        // completion code
    DWORD cbTransferred,   // number of bytes transferred
    LPOVERLAPPED lpo       // address of structure with I/O
                           // information
);
BOOL WriteFileEx(
    HANDLE       hFile,           // file to write into
    LPCVOID      lpBuffer,        // address of buffer
    DWORD nNumberOfBytesToWrite,  // nos. of bytes to write
    LPOVERLAPPED lpOverlapped,    // address of overlapped structure
                                  // Address of Completion Routine
    LPOVERLAPPED_COMPLETION_ROUTINE lpCompletionRoutine
    );
```

The caller specifies the completion routine address when making an asynchronous I/O call, such as WriteFileEx(). In addition, the caller must supply a valid OVERLAPPED structure. When these tasks are successful, the extended API returns TRUE and causes an asynchronous completion routine to be called when the I/O operation completes.

There is an additional requirement, however: when the asynchronous I/O operation completes, the application must be in the alertable state. An application can put itself in alertable state when it is blocked while waiting on certain Win32 APIs, such as **SleepEx()**, **WaitForSingleObjectEx()**, or **WaitForMultipleObjectEx()**. These wait functions take not only the parameters expected by their nonextended counterparts but also a parameter that

dictates whether the application is to be put in alertable state. When an asynchronous I/O completes, the system does not call the application on its completion routine until the application is in an alertable state. When an application is not alert, calls to the completion routine are queued up. By adding a new state (that is, *alertable*), Win32 APIs put more control in the hands of Win32 application designers, who can pipeline the I/O requests and also tightly control when they wish to receive notification of the completion of an asynchronous call.

The following code fragment shows an example of printing a large buffer using the WriteFileEx() API and an asynchronous completion routine.

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#define LARGE_BUFFER_SIZE (64 * 1024)
#define SLEEP_TIME        (5L)
//
// Asynchronous Routine Called when printing is complete
VOID WINAPI AsyncRoutine (
                          DWORD dwRc,          // Error code
                          DWORD cbTransferred, // Bytes written
                          LPOVERLAPPED lpo     // OL structure
                          )
{
    printf("Async Routine Called: %ld\n", dwRc);
    printf("Bytes printed: %ld\n", cbTransferred);
    printf("Overlapped offset: %ld High Offset: %ld\n",
        lpo->Offset, lpo->OffsetHigh);
}
//
// Main Function
//
int _cdecl main (int argc, char ** argv)
{
    HANDLE hPort, hEvent;
    DWORD  dwRc, i, dwBufSize;
    OVERLAPPED overlap;
    argv++;
    argc--;
    if (argc)
    {
       dwBufSize = atoi(*argv);
       if (dwBufSize > LARGE_BUFFER_SIZE)
       {
           dwBufSize = LARGE_BUFFER_SIZE;
       }
    }
    else
       dwBufSize = LARGE_BUFFER_SIZE;
    // Open the LPT1 port in OVERLAPPED mode so that
    // main thread can continue to function while
    // printing asynchronously
```

```
if ((hPort = CreateFile(L"LPT1",              // Open LPT1 port
                        GENERIC_WRITE,        // Access mode
                        0,                    // Share mode (Exclusive)
                        NULL,                 // Default security
                        OPEN_EXISTING,        // Open mode
                        FILE_ATTRIBUTE_NORMAL< // Attribute
                        FILE_FLAG_OVERLAPPED,
                        NULL                  // No Template
                    ))==NULL)
{
    PrintWin32Error("CreateFile on LPT1");
    return (1);
}
// Set up a large buffer to be sent to printer
CHAR szBuffer [ LARGE_BUFFER_SIZE ];
for (i = 0; i < dwBufSize; i += 80)
{
    sprintf(szBuffer + i, "The Count is %ld\n\r", i);
}
// Create a manual-reset event for OVERLAPPED I/O
if ((hEvent = CreateEvent (NULL, TRUE, FALSE, NULL))
    ==NULL)
{
    PrintWin32Error("CreatEvent: %ld\n");
    goto CleanupAndReturn;
}
// Set up an overlapped structure
memset ((PVOID)&overlap, 0x00, sizeof(OVERLAPPED));
overlap.hEvent = hEvent;
// Send a large buffer to the port to be
// printed asynchronously using WriteFileEx()
if (!WriteFileEx(hPort,       // Handle to the LPT port
                 szBuffer,    // Buffer to send
                 dwBufSize,   // Bytes to write
                 &overlap,    // Overlapped structure
                 AsyncRoutine // Async completion routine
                ))
{
    PrintWin32Error("WriteFileEx");
    goto CleanupAndReturn;
}
// At this point the application can continue to
// process other requests. In this simple
// sample, we use SleepEx() to wait in
// "alertable mode" so that SleepEx completes
// when the asynchronous completion routine is complete
do
{
    // Complete other activities while buffer is printing
    // In this example, we will wait in "alertable"
    // state so that completion routine can be called.
    dwRc = SleepEx ( SLEEP_TIME, TRUE);
    if (dwRc == WAIT_IO_COMPLETION)
            printf("One or more Completion Routines
                    called.\n");
            printf(".");
```

```
    } while (dwRc == 0); // 0 indicates SleepEx timed out
    printf("Async Function was completed\n");
CleanupAndReturn:
    // Close handles
    CloseHandle (hEvent);
    CloseHandle (hPort);
    return (0);
}
```

2.3.6 Memory-Mapped File I/O and Sharing Memory

Memory-mapped file I/O is an advanced feature in Windows NT that allows an application to map a file into its own virtual address space. Once a file has been mapped to this space, it can be accessed by the process by using memory pointers. This process contrasts with the traditional ways to access a file through file handles and a set of file functions [for example, ReadFile(), WriteFile(), and so on]. Thus, the immediate benefit of memory-mapped file I/O is the convenience of not having to call file read/write functions to read data from, or write data into, a file.

However, the primary advantage of using memory-mapped file I/O is the use of the operating system paging file and operating-system-provided file-caching mechanism. An application is free to map a file of any size from one byte to two gigabytes (the upper limit of a process virtual address space) to its address space. Once a file has been mapped to the address space, it is supported by system paging and caching. In other words, the application does not have to implement its own caching to reduce I/O time delay.[13] Normally, a program allocates an array of a certain size in the address space to which the data is read in from a file. Of course, determining the optimum size of such a data array is important for most I/O-intensive applications.

Finally, memory-mapped file I/O allows two or more processes to share memory. To share memory, a process first maps a file to its own address space and then maps a view into the address space. The view can be anywhere from one byte to the full size of the mapped file. The mapping is supported by a system object called a file-mapping object. Once a file-mapping object has been created, other processes can also get a view into the memory space of the first process by using file-mapping functions. When two or more processes share the same view into the address space, they are, effectively, sharing memory.

The concept of file mapping and views is illustrated in Figure 2-14. In this figure, the master process control program creates the file-mapping for the global database file shared by the child processes. In particular, the pump monitor and master process share the "Pump Status" address space by having a

13. Disk access is usually about 1000 times slower than memory access. Unless an application implements a particularly good data-caching mechanism, it is going to pay a severe penalty for accessing data from the disk when data is not cached. Memory-mapped files can also fall out of cache and cause disk access.

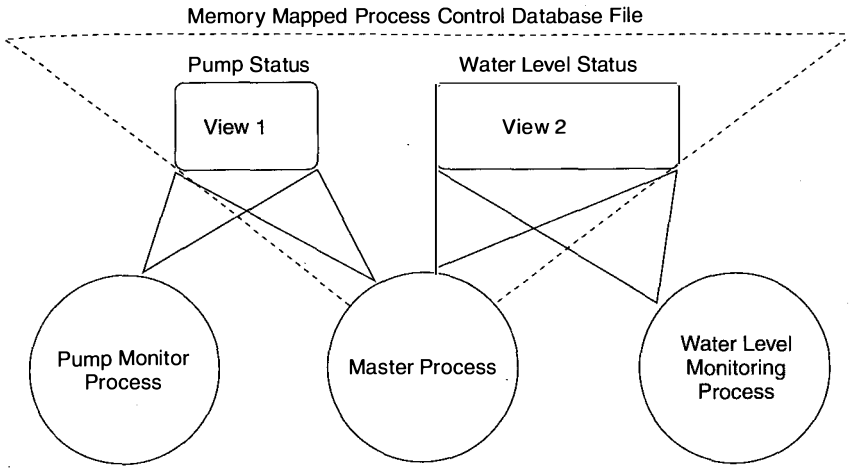


Figure 2-14. *Sharing Memory by Defining Views into Address Space*

common view (view 1), whereas the water-level monitoring process shares the other view (view 2) with the master process.

In CSSs, a server may use the file-mapping techniques shown in Figure 2-14. This is especially true for database servers. In such systems, the server can map the database tables to the memory, then translate the client query definitions to the corresponding views into the database tables. This mechanism allows each client's database view to translate to a file-mapping view on the server end. For example, a client can constrain its query space to all names starting with the letter "c." On the server side, this query space constraint may be translated to certain rows (say, 3 through 24) of the database table. In other words, the server implementation can map the client's logical view into the database into a file view of the database file. Once a view into the database table has been defined, all subsequent client queries are constrained within the file view.

Mapping Files into Address Space

The application first gets a file handle to the file by using the CreateFile() or the OpenFile() API. The application can then pass the file handle to the **CreateFileMapping()** API to create a file-mapping object. The CreateFileMapping() API returns a handle to the file-mapping object. A file-mapping object can be named or unnamed. A named file-mapping object can be opened by any process by using the **OpenFileMapping()** API, provided the process has proper access rights to the object. To open a mapping object created by another process, the caller of the OpenFileMapping() API specifies the name of the file-mapping object. An application usually creates an unnamed file-mapping object when it wants to map the file to its own address space so it can access the file with memory pointers.

Once a file-mapping object has been created, a process gets a view into the virtual address space by using the **MapViewOfFile()** API. The caller specifies the byte offsets for its view boundaries within the address space. For example, the caller could specify a view window starting at the 2400th byte and ending at the 54 Kbyte boundary of the address space. The MapViewOfFile() API returns a pointer to beginning of the view boundary (a pointer to the 2400th byte for our example). The application can freely use this pointer to access the file, using any memory access functions, such as memcpy(), printf(), and so on.

When mapping a file into the memory, the caller can specify a variety of access modes and paging behavior. For example, the caller could specify "write on copy," which indicates that any writes into the memory are committed to the physical disk space, while untouched files are discarded when the mapping is undone. (You may want to read the Win32 API documentation for details.)

An application can force the flushing of a memory-mapped file by calling **FlushViewOfFile()**. The application can eventually relinquish access to the memory space by calling **UnmapView-OfFile()**. The following code fragment shows how an application maps a file to its address space and then accesses the file using simple C-runtime functions such as printf() and sprintf().


```
//
// Mapping a File to Virtual Address Space
//
#include <windows.h>
#include <stdlib.h>
#include <stdio.h>
int _cdecl main (int argc, char ** argv)
{
        WCHAR  wszFileName [MAX_PATH] = L"test.txt";
        HANDLE hFile, hMappedFile;
        LPVOID lpFile;
        // Determine the name of the file
        if (argc > 1)
        {
            mbstowcs (wszFileName, argv[1], strlen(argv[1])+1);
        }
    // First open the file
    if ((hFile = CreateFile ( wszFile,
                              GENERIC_READ<GENERIC_WRITE,
                              0,
                              NULL,
                              OPEN_EXISTING,
                              FILE_ATTRIBUTE_NORMAL,
                              NULL
                             ))==INVALID_HANDLE_VALUE)
    {
```

```
        // Error Processing
        PrintWin32Error("CreateFile");
        return (1);
    }
    // Create a mapped file object
    if ((hMappedFile = CreateFileMapping (
                            hFile,
                            NULL,      // Default security
                            PAGE_READWRITE, // Read/write acces
                            0, // Maximum Mapped file object
                            0,         // = current file size
                            NULL       // Unnamed mapped file
                            ))==NULL)
    {
        // Error Processing
        PrintWin32Error("CreateFileMapping");
        goto CleanupAndReturn;
    }
    // Get a memory pointer address into the mapped file
    if ((lpFile = MapViewOfFile (hMappedFile,
                        FILE_MAP_ALL_ACCESS, // access
                        0,                   // Offset into file
                        0,                   // where mapping begins
                        0                    // map the entire file
                        ))==NULL)
    {
            // Error processing
            PrintWin32Error("MapViewOfFile");
            goto CleanupAndReturn;
    }
    // Now that we have a pointer to a file, we can use
    // simple memory routines to do I/O on file
    // Read from file
    printf("The first line in file: %s\n", lpFile );
    // Write into the file
    sprintf( (PCHAR) lpFile, "This is first line");
    // Flush the contents
    if (!FlushViewOfFile(lpFile, 0))
    {
            // Error Processing
            PrintWin32Error("FlushViewOfFile");
            goto CleanupAndReturn;
    }
    // Unmap the file
    if (!UnmapViewOfFile ( lpFile ))
    {
            // Error Processing
            PrintWin32Error("UnmapViewOfFile");
            goto CleanupAndReturn;
    }
CleanupAndReturn:
    // Close handles
    CloseHandle ( hFileMapping );
    CloseHandle ( hFile );
    return (0);
}
```

Shared Memory Using Memory-Mapped File I/O Functions

Two or more processes can share memory by using memory-mapped file I/O functions. The steps are similar to mapping a file into the process address space by first creating a file-mapping object, and then creating a view into the file. Once a process has created a named mapping object, other processes can get a handle to the file-mapping object by calling the OpenFileMapping() API. Finally, the process can then get a view into the mapped file using the MapView-OfFile() API.

As you learned in the preceding section, if two or more processes are sharing the same view, they are sharing memory. The processes that are sharing memory need a synchronization object to maintain the integrity of that shared memory. When the file mapped into the memory is a local file, the system merges as a shared memory any changes made by the processes accessing the file. In other words, all the involved processes see a single, consistent view of the file. This is not true when the file mapped to memory is a remote-drive-based file.

The following code fragment shows two processes sending messages to each other by shared memory. Assume both processes use a mutex to enforce mutual exclusion when accessing memory.

```
//
// Sharing Memory Using Memory Mapped File I/O
// Process 1 creates file-mapping object; process 2
// opens the object and starts to use the memory
//
#include <windows.h>
#include <stdlib.h>
#include <stdio.h>
//
// Process 1 creates Shared Memory using a temporary file
//
int _cdecl main (int argc, char ** argv)
{
        WCHAR  wszTempFileName [MAX_PATH] = L"test.tmp";
        HANDLE hFile, hMappedFile, hMutex;
        LPVOID lpFile;
        // First create a temporary file
        if ((hFile = CreateFile ( wszTempFileName,
                                  GENERIC_READ<GENERIC_WRITE,
                                  0,
                                  NULL,
                                  CREATE_NEW,
                                  // Keep file in memory for allowing fast
                                  // access
                                  FILE_ATTRIBUTE_TEMPORARY <
                                  // Delete file when all handles are
                                  // closed
                                  FILE_FLAG_DELETE_ON_CLOSE,
                                  NULL
                                ))==INVALID_HANDLE_VALUE)
```

```
{
    // Error Processing
    PrintWin32Error("CreateFile");
    return (1);
}
// Create 4K-sized shared memory
if ((hMappedFile = CreateFileMapping (
                                    hFile,
                                    NULL,     // Default security
                                    PAGE_READWRITE,
                                    0,        // Shared memory
                                              // size = 4k
                                    4*1024,   // Name of shared
                                              // memory
                                    L"MySharedMemory"
                                    ))==NULL)
{
    // Error Processing
    PrintWin32Error("CreateFileMapping");
    goto CleanupAndReturn;
}
// Get a memory pointer address into the mapped file
if ((lpFile = MapViewOfFile (hMappedFile,
                             FILE_MAP_ALL_ACCESS,// access
                             0,              // Offset into file
                             0,              // where mapping begins
                             0               // map the entire file
                            ))==NULL)
{
    // Error processing
    PrintWin32Error("MapViewOfFile");
    goto CleanupAndReturn;
}
// Create a mutex
if ((hMutex = CreateMux (FALSE,        // No initial ownership
                         L"TestMutex" // Mutex Name
                         ))=NULL)
    {
    // Error occurred
    goto CleanupAndReturn;
    }
// Loop reading messages from Process 2
do
{ // Get access to shared memory
    AcquireMux (hMutex, INFINITE );
    // Read message sent by other process
    printf("Message from Process 2: [%s]\n", lpFile + 80);
    // Send Message to Process 2
    sprintf( (PCHAR) lpFile, "This is from Process 1.\n");
    // Release memory
    ReleaseMux ( hMutex );
} while ( FOREVER );
    // Flush the contents
    if (!FlushViewOfFile(lpFile, 0))
    {
    // Error Processing
```

```
            PrintWin32Error("FlushViewOfFile");
            goto CleanupAndReturn;
        }
        // Unmap the file
        if (!UnmapViewOfFile ( lpFile ))
        {
            // Error Processing
            PrintWin32Error("UnmapViewOfFile");
            goto CleanupAndReturn;
        }
CleanupAndReturn:
        // Close handles
        CloseHandle ( hFileMapping );
        CloseHandle ( hFile  );
        CloseHandle ( hMutex );
        return (0);
}
//
// Process 2 - Opens the Shared Memory
//
int _cdecl main (int argc, char ** argv)
{
    HANDLE hMappedFile, hMutex;
    LPVOID lpFile;
    // Open the already created mapped-file object
    if ((hMappedFile = OpenFileMapping (
                                        FILE_MAP_ALL_ACCESS,// Access
                                        0,                  // Inherit flag
                                                            // Mapped File Name
                                        L"MySharedMemory"
                                       ))==NULL)
    {
          // Error Processing
          PrintWin32Error("OpenFileMapping");
          goto CleanupAndReturn;
    }
    // Get a memory pointer address into the mapped file
    if ((lpFile = MapViewOfFile (hMappedFile,
                                 FILE_MAP_ALL_ACCESS, // access
                                 0,                   // Offset into file
                                 0,                   // where mapping begins
                                 0                    // map the entire file
                                ))==NULL)
    {
         // Error processing
         PrintWin32Error("MapViewOfFile");
         goto CleanupAndReturn;
    }
    // Create a mutex
    if ((hMutex = CreateMux (FALSE,   // No initial ownership
                             L"TestMutex" // Mutex Name
                            ))=NULL)
    {
         // Error occurred
         goto CleanupAndReturn;
    }
```

```
    // Loop sending messages to Process 1
    do
    { // Get access to shared memory
            AcquireMux ( hMutex );
            // Send Message to Process 2
            "This is from Process 2.\n");
            sprintf( (PCHAR) lpFile,
            // Read Message sent by Process 1
            printf("Message from Process 1: [%s]\n", lpFile + 80);
            // Release mutex
            ReleaseMux (hMutex);
    } while ( FOREVER );
    // Flush the contents
    if (!FlushViewOfFile(lpFile, 0))
    {
        // Error Processing
        PrintWin32Error("FlushViewOfFile");
        goto CleanupAndReturn;
    }
        // Unmap the file
        if (!UnmapViewOfFile ( lpFile ))
        {
            // Error Processing
            PrintWin32Error("UnmapViewOfFile");
            goto CleanupAndReturn;
        }
CleanupAndReturn:
        // Close handles
        CloseHandle ( hFileMapping );
        CloseHandle ( hMutex );
        return (0);
}
```

2.3.8 Control C Handlers

Recall that Windows NT allows a Win32 application to be either a Windows GUI-based application or a simple console application that uses character mode input/output in a command window. Console applications can be closed abruptly if the user presses a <CTRL><C> key sequence. Other user actions that send a signal to the console application are pressing a <CTRL><Break> key sequence, logging off, and closing an application by using the system menu.

Console applications can safeguard themselves against such abrupt termination by installing a control handler.[14] Win32 provides the **SetConsoleHandler()** API, which can be used to install a control handler for an application. The **SetConsoleCtrlHandler()** function adds or removes an application-defined **HandlerRoutine()** function from the list of handler functions for the calling process. If no handler function is specified, the function sets an inheritable attribute that determines whether the calling process ignores the <CTRL><C> signal.

14. In UNIX environments, control handlers are called "signal handlers."

The following code fragment shows the use of the SetConsoleCtrlHandler() API and a sample control handler that prevents the user from terminating a console application by pressing <CTRL><C>.

```
//
// Control Handler for a Win32 Console Application
//
BOOL CtrlHandler(DWORD fdwCtrlType)
{
    switch (fdwCtrlType)
    {
    // Handle the CTRL+C signal (console app only)
    case CTRL_C_EVENT:
        MessageBox (NULL,
                    L"Illegal attempt to Terminate App",
                    L"Error",
                    MB_OK);
        return TRUE;
    // CTRL+CLOSE: confirm that the user wants to exit.
    case CTRL_CLOSE_EVENT:
    // Cleanup
    return TRUE;
    // Pass other signals to the next handler.
    case CTRL_BREAK_EVENT:
    case CTRL_LOGOFF_EVENT:
    case CTRL_SHUTDOWN_EVENT:
    default:
        return FALSE;
    }
}
//
// An example application that installs a control-c handler so
// that user must use application-provided shutdown procedure.
//
VOID main(void)
{
          // Install a Control handler
          if (!SetConsoleCtrlHandler(
                    (PHANDLER_ROUTINE) CtrlHandler,// handler function
                                       TRUE))      // add to list
          {
               printf("Error installing Control Handler.\n");
               return (1);
          }
               // Now the application is safe from control-c
}
```

In this code fragment, when a <CTRL><C> signal is received, the control handler returns TRUE, indicating it has handled the signal. This prevents other control handlers from being invoked. The control handler also returns TRUE when it receives a CTRL_CLOSE_EVENT signal. This causes the system to display a dialog box giving the user the choice of either terminating the process and closing the console or allowing the process to continue execution. If the user chooses to continue the process, the system closes the console

when the process finally terminates. When it receives a CTRL_BREAK, CTRL_LOGOFF_EVENT, or CTRL_SHUTDOWN_EVENT signal, the control handler returns FALSE. This causes the signal to be passed to the next control-handler function. If no other control handlers have been registered, or if none of the registered handlers returns TRUE, the default handler will be invoked, and the process will be terminated.

2.4 CREATING AN ECHO SERVER

An Echo Server simply echoes back a message sent by the client. The server runs as a Win32 network service and is able to handle multiple Windows and Windows NT clients.

In this section, you will build only the infrastructure of the Echo Server so it can run as a service. It will not, however, be able to communicate with any client. You will add the connectivity pieces into the server in later chapters that deal with a given IPC mechanism. For example, in Chapter 5, "Windows Sockets in Windows NT," you will modify the server so it uses WinSock to communicate with clients. The Echo Server presented in Chapter 5 also uses IPX/SPX to communicate with clients. This book follows a running design and implementation of the Echo Server for these reasons:

- The sample server does incorporate some of the advanced programming features mentioned in this chapter, and thus clearly shows you how to use these advanced features.
- You will become familiar with designing Win32 network services.
- By the end of the book, you will have working Echo Server code that uses each of the IPC mechanisms discussed in this book.
- You will also be able to compare the use of one IPC mechanism with another by comparing the Echo Server codes.

Table 2-5 shows all the Win32 services APIs, some of which will be used to create the Echo Server. Because it is beyond the scope of this book to explain the details of these service APIs, you may want to refer to the Microsoft manual.

In the Windows NT environment, a Win32 service must have an executable binary image (that is, an .EXE file). However, a user starts or stops a service by using the "net" commands shown next or by using the services applet from the control panel. The Windows NT **service controller (SC)** controls all the services and exposes a series of Win32 service APIs. These APIs are used to install, operate, and administer the Win32 services.

```
REM Start the Echo Server
net start echoserver
REM Stop the Echo Server
net stop echoserver
REM list all current running services
net start
```

Table 2-5. Win32 Service Interfaces

Win32 Function	*Description*
Win32 Service Configuration Functions	
ChangeService Config()	Changes service configuration parameters
CloseServiceHandle()	Closes Service Control Manager object
ControlService()	Sends a control to a service
CreateService()	Creates a service object
DeleteService()	Removes service from SC Manager database
EnumDependentServices()	Enumerates services dependent on device
EnumServicesStatus()	Enumerates services in SC Manager database
GetServiceDisplayName()	Obtains a service's display name
GetServiceKeyName()	Obtains a service's registry key name
LockServiceDatabase()	Locks specified SC Manager database
NotifyBootConfigStatus()	Notifies and responds to acceptability of boot configuration
OpenService()	Opens an existing service
OpensSCManager()	Connects to service control manager
QueryServiceConfig()	Gets service configuration parameters
QueryServiceLockStatus()	Gets service database lock status
QueryServiceObjectSecurity()	Gets service object security descriptor
QueryServiceStatus()	Gets service status
StartService()	Starts running a service
UnlockServiceDatabase()	Unlocks specified database
Functions Provided by or Used by Win32 Services	
Handler()	Controls handler function of a service (provided by service)
RegisterServiceCtrlHandler()	Registers service control request handler
ServiceMain()	Main function of a service (provided by service)
SetServiceStatus()	Updates service status to SC Manager
SetServiceObjectSecurity()	Modifies service object security descriptor
StartServiceCtrlDispatcher()	Connects thread as dispatch thread

The next code fragment shows the basic skeleton of the Echo Server as a Win32 service. Once you supply the code for the functions (that is, for **Init-EchoServer()**, **PauseEchoServer()**, and **ResumeEchoServer())**, you will have a fully functional Echo Server. These functions will be provided in each chapter to suit the needs of the particular IPC mechanism under discussion. For example, in Chapter 4 on RPC, you will start a thread in InitEchoServer() that will service the RPC clients of the Echo Server. Note: you can compile and link the following code to get a fully functional Win32 service;[15] however, it will not perform any useful functions, such as communication with clients.

```
////////////////////////////////////////////////////
// File: EchoSrv.c -
//     main program for Echo Service sample.
//
//     The Echo service will respond to the basic
//     service controller functions, i.e., Start,
//     Stop, and Pause.
//
#include "header.h"
#pragma  hdrstop
// this event is signaled when the
// worker thread ends
//
HANDLE                  hServDoneEvent = NULL;
SERVICE_STATUS          ssStatus;//current status of the service
SERVICE_STATUS_HANDLE   sshStatusHandle;
DWORD                   dwGlobalErr;
VOID     service_main(DWORD dwArgc, LPTSTR *lpszArgv);
VOID     service_ctrl(DWORD dwCtrlCode);
BOOL     ReportStatusToSCMgr(DWORD dwCurrentState,
                             DWORD dwWin32ExitCode,
                             DWORD dwCheckPoint,
                             DWORD dwWaitHint);
VOID     StopServer(LPTSTR lpszMsg);
// main() -
//          all main does is call StartServiceCtrlDispatcher
//          to register the main service thread.  When the
//          API returns, the service has stopped, so exit.
//
VOID main()
{
    SERVICE_TABLE_ENTRY dispatchTable[] = {
          {  TEXT("EchoService"), (LPSERVICE_MAIN_FUNCTION)service_main },
          {  NULL, NULL }
    };
```

15. Add the following lines to complete the linking:
BOOL InitEchoServer() { return TRUE;}
BOOL PauseEchoServer() { return TRUE;}
BOOL ResumeEchoServer() { return TRUE;}

```
    if (!StartServiceCtrlDispatcher(dispatchTable))
    {
        StopServer(TEXT("StartServiceCtrlDispatcher failed."));
    }
}
//  service_main() -
//     this function takes care of actually starting the
//     service, informing the service controller at each step
//     along the way. After launching the worker thread, it
//     waits on the event that the worker thread will signal at
//     its termination.
VOID service_main(DWORD dwArgc, LPTSTR *lpszArgv)
{
    DWORD                   dwWait;
    // register our service control handler:
    //
    sshStatusHandle = RegisterServiceCtrlHandler(
                                        TEXT("EchoService"),
                                        (LPHANDLER_FUNCTION)service_ctrl);
    if (!sshStatusHandle)
        goto cleanup;
    // SERVICE_STATUS members that don't change in example
    //
    ssStatus.dwServiceType = SERVICE_WIN32_OWN_PROCESS;
    ssStatus.dwServiceSpecificExitCode = 0;
    // report the status to Service Control Manager.
    //
    if (!ReportStatusToSCMgr(
                            SERVICE_START_PENDING, // service state
                            NO_ERROR,              // exit code
                            1,                     // checkpoint
                            3000))                 // wait hint
                            goto cleanup;
    // create the event object. The control handler function
    // signals this event when it receives the "stop" control
    // code.
    hServDoneEvent = CreateEvent(
                                NULL,    // no security attributes
                                TRUE,    // manual reset event
                                FALSE,   // not-signalled
                                NULL);   // no name
    if (hServDoneEvent == (HANDLE)NULL)
        goto cleanup;
    // report the status to the service control manager.
    //
    if (!ReportStatusToSCMgr(
        SERVICE_START_PENDING, // service state
         NO_ERROR,             // exit code
         2,                    // checkpoint
         3000))                // wait hint
         goto cleanup;
    //
    // start the thread that performs the work of the service.
    //
    if (!InitEchoServer())
        goto cleanup;
```

```
    // report the status to the service control manager.
    //
    if (!ReportStatusToSCMgr(
        SERVICE_RUNNING, // service state
        NO_ERROR,        // exit code
        0,               // checkpoint
        0))              // wait hint
        goto cleanup;
    // wait indefinitely until hServDoneEvent is signaled.
    //
    dwWait = WaitForSingleObject(
        hServDoneEvent,  // event object
        INFINITE);       // wait indefinitely
cleanup:
    if (hServDoneEvent != NULL)
        CloseHandle(hServDoneEvent);
    // try to report the stopped status to the service control
    // manager.
    //
    if (sshStatusHandle != 0)
        (VOID)ReportStatusToSCMgr(
                                    SERVICE_STOPPED,
                                    dwGlobalErr,
                                    0,
                                    0);
    // When SERVICE MAIN FUNCTION returns in a single service
    // process, the StartServiceCtrlDispatcher function in
    // the main thread returns, terminating the process.
    //
    return;
}
//
//  service_ctrl() -
//     this function is called by the Service Controller
//     whenever someone calls ControlService in reference to
//     our service.
//
VOID service_ctrl(DWORD dwCtrlCode)
{
    DWORD  dwState = SERVICE_RUNNING;
    // Handle the requested control code.
    //
    switch(dwCtrlCode) {
        // Pause the service if it is running.
        //
        case SERVICE_CONTROL_PAUSE:
              if (ssStatus.dwCurrentState == SERVICE_RUNNING) {
                   PauseEchoServer();
                   dwState = SERVICE_PAUSED;
              }
              break;
        // Resume the paused service.
        //
        case SERVICE_CONTROL_CONTINUE:
              if (ssStatus.dwCurrentState == SERVICE_PAUSED) {
                  ResumeEchoServer();
```

```
                    dwState = SERVICE_RUNNING;
                }
                break;
        // Stop the service.
        //
        case SERVICE_CONTROL_STOP:
                dwState = SERVICE_STOP_PENDING;
                // Report the status, specifying the checkpoint and
                // waithint, before setting the termination event.
                //
                ReportStatusToSCMgr(
                     SERVICE_STOP_PENDING, // current state
                     NO_ERROR,             // exit code
                     1,                    // checkpoint
                     3000));               // wait hint
                                           // Stop the Server
                StopServer(TEXT("ServiceStop"));
                return;
        // Update the service status.
        //
        case SERVICE_CONTROL_INTERROGATE:
                break;
        // invalid control code
        //
        default:
                break;
    }
    //
    // send a status response.
    //
    ReportStatusToSCMgr(dwState, NO_ERROR, 0, 0);
}
//
// utility functions...
//
// ReportStatusToSCMgr() —
//     This function is called by the ServMainFunc() and
//     ServCtrlHandler() functions to update the service's
//     status to the service control manager.
//
BOOL ReportStatusToSCMgr(DWORD dwCurrentState,
                         DWORD dwWin32ExitCode,
                         DWORD dwCheckPoint,
                         DWORD dwWaitHint)
{
    BOOL fResult;
    // Disable control requests until the service is started.
    //
    if (dwCurrentState == SERVICE_START_PENDING)
        ssStatus.dwControlsAccepted = 0;
    else
        ssStatus.dwControlsAccepted = SERVICE_ACCEPT_STOP 1
            SERVICE_ACCEPT_PAUSE_CONTINUE;
    // These SERVICE_STATUS members are set from parameters.
    //
    ssStatus.dwCurrentState = dwCurrentState;
```

```
    ssStatus.dwWin32ExitCode = dwWin32ExitCode;
    ssStatus.dwCheckPoint = dwCheckPoint;
    ssStatus.dwWaitHint = dwWaitHint;
    // Report the status of the service to the service control
    // manager.
    if (!(fResult = SetServiceStatus(
                    sshStatusHandle,    // service reference handle
                    &ssStatus))) {      // SERVICE_STATUS structure
        // If an error occurs, stop the service.
        //
        StopServer(TEXT("SetServiceStatus"));
    }
    return fResult;
}
// PrintServiceError - Used to log error and information
//                     messages in the event viewer
//
static HANDLE hEventSource = NULL;
void PrintServiceError  (DWORD dwRc, LPTSTR lpwszMsg, LPSTR szExtra,
    WORD wEventType)
{
    TCHAR   wszMsg1  [MAX_PATH],
            wszMsg2  [MAX_PATH],
            wszMsg3  [MAX_PATH];
    LPTSTR alpszStrings[2];
    alpszStrings[0] = wszMsg1;
    alpszStrings[1] = wszMsg2;
    //
    // Use event logging to log the error.
    //
    if (hEventSource == NULL)
        hEventSource = RegisterEventSource(NULL, TEXT("EchoService"));
    if (wEventType==EVENTLOG_INFORMATION_TYPE)
        swprintf(wszMsg1, TEXT("EchoService info: %lx\n"), dwRc);
    else
        swprintf(wszMsg1, TEXT("EchoService error: %lx\n"), dwRc);
    if (lpwszMsg != NULL)
    {
        if (szExtra == NULL)
            swprintf(wszMsg2, lpwszMsg, dwRc);
        else
        { // convert from ASCII to Unicode
          mbstowcs(wszMsg3, szExtra, strlen(szExtra)+1);
          swprintf(wszMsg2, lpwszMsg, wszMsg3);
        }
    }
    if (hEventSource != NULL)
        ReportEvent(hEventSource,          // handle of event source
                    wEventType,            // event type
                    0,                     // event category
                    0,                     // event ID
                    NULL,                  // current user's SID
                    (lpwszMsg==NULL)?1:2 , // strings in lpszStrings
                    0,                     // no bytes of raw data
                    alpszStrings,          // array of error strings
                    NULL);                 // no raw data
```

```
}
//
// The StopServer function can be used by any thread to report
//  an error, or stop the service.
//
VOID StopServer(LPTSTR lpszMsg)
{
    // Report the error
    dwGlobalErr = GetLastError();
    PrintServiceError(dwGlobalErr,lpszMsg, NULL,
                                  EVENTLOG_ERROR_TYPE );
    // Stop the Service, if started
    StopEchoServer();
    // if the event source has been opened, close it
    if (hEventSource != NULL)
    {
     (VOID) DeregisterEventSource(hEventSource);
    }
    // Set a termination event to stop SERVICE MAIN FUNCTION.
    //
    SetEvent(hServDoneEvent);
}
//
// A function to retrieve Win32 error and print a message
//
DWORD PrintWin32Error ( LPSTR pszErrorString)
{
          DWORD dwRc;
          TCHAR wszMsgBuf [ FORMAT_MESSAGE_MAX_WIDTH_MASK + 1 ],
                wszMsg   [ sizeof(wszMsgBuf) *2];
          // Retrieve the error code
          dwRc = GetLastError();
          // Search system message tables and resources for an
          // error message associated with the error returned
          if (FormatMessage (// Format option
                             FORMAT_MESSAGE_FROM_SYSTEM 1
                             FORMAT_MESSAGE_MAX_WIDTH_MASK,
                             NULL,                 // Message source
                             dwRc,                 // Error Code
                             0,                    // Language Specifier
                             wszMsgBuf,            // Message Buffer
                             // Message Buffer Size
                             FORMAT_MESSAGE_MAX_WIDTH_MASK,
                             NULL
                            ))
    {
        swprintf(wszMsg, L"%s error message: %ws",
                         pszErrorString, wszMsgBuf);
        PrintServiceError(dwRc, wszMsg, NULL,
                         EVENTLOG_ERROR_TYPE);
    }
    return dwRc;
}
//
// File: PRINTCMD.h - contains the macros used by Client/Server
// to print messages
```

```
ifndef _PRINT_CMD_H
#define _PRINT_CMD_H
void  PrintServiceError  (DWORD dwRc, LPTSTR lpwszMsg,
                          LPSTR szExtra,  WORD wEventType);
DWORD   PrintWin32Error  (LPSTR pszErrorString);
#if defined(_SERVICE)
#pragma message("Building Service")
#define PrintError( szErrorString) {\
    PrintServiceError (0, TEXT(szErrorString), NULL,\
                       EVENTLOG_ERROR_TYPE);\
}
#define PrintErrorl(szErrorString, dwRc) {\
    PrintServiceError (dwRc, TEXT(szErrorString),NULL, \
                       EVENTLOG_ERROR_TYPE);\
}
#define PrintInfo( szInfoString) {\
    PrintServiceError (0, TEXT(szInfoString),NULL, \
                       EVENTLOG_INFORMATION_TYPE);\
}
#define PrintInfol(szInfoString, dwRc) {\
    PrintServiceError (dwRc, TEXT(szInfoString), NULL, \
                       EVENTLOG_INFORMATION_TYPE);\
}
#define PrintInfoString(szInfoString, str) {\
    PrintServiceError (0, TEXT(szInfoString), str, \
                       EVENTLOG_INFORMATION_TYPE);\
}
#else // !_SERVICE
#define PrintError(szErrorString) {\
    printf(szErrorString); \
}
#define PrintErrorl(szErrorString, dwRc) {\
    printf (szErrorString, dwRc); \
}
#define PrintInfo(szInfoString) {\
    printf(szInfoString); \
}
#define PrintInfol(szInfoString, dwRc) {\
    printf(szInfoString, dwRc); \
}
#define PrintInfoString(szInfoString, str) {\
    printf(szInfoString, str);\
}
#endif
#endif // _PRINT_CMD_H
//
// File: SERVICE.H - Contains prototype of functions
//
#ifndef _SERVICE_H_
#define _SERVICE_H_
#include "resource.h"
// Service Functions
BOOL InitEchoServer();
BOOL StopEchoServer();
BOOL PauseEchoServer();
BOOL ResumeEchoServer();
```

```
BOOL ReportStatusToSCMgr (DWORD dwCurrentState,
                          DWORD dwWin32ExitCode,
                          DWORD dwCheckPoint,
                          DWORD dwWaitHint
                          );
// Queue Functions
DWORD InitializeQueue();
DWORD QueueRequest(VOID * pvData, DWORD dwDataSize);
DWORD GetNextRequest ( OID ** pfBuffer, DWORD *pdwBufferSize);
// Thread Pool Functions
typedef VOID (CDECL *PTHREAD_ROUTINE )(PVOID);
DWORD InitializeThreads (PTHREAD_ROUTINE pfThreadProc);
DWORD GetThreadSyncEvent ();
VOID SetThreadSyncEvent(DWORD dwEventIndex);
VOID CloseThreadSyncEvent(DWORD dwEventIndex);
DWORD WaitForAllThreadsToTerminate();
// Thread State at any given time
#define THREAD_STATE_NONE       0
#define THREAD_STATE_STARTED 1
#define THREAD_STATE_PAUSED  2
// Structure to contain thread specific data
typedef struct _tagThreadData
{
    ULONG  hThread;
    DWORD  dwState;
    DWORD  dwEventIndex;
}ThreadData;
#endif // _SERVICE_H_
//
// File - QUEUE.C - Contains functions used by non-RPC based
//                  services to maintain Client queue
//
#include "header.h"
#pragma  hdrstop
// Client Request Queue
#define MAX_CLIENT_Q_LENGTH 5
// Client Request Queue
typedef struct _tagQueue
{
    // The caller is responsible for allocating/freeing
    // ClientData
    VOID   *ClientData;
    DWORD  dwDataSize;
}Queue;
// Global Data
static Queue ClientQueue [ MAX_CLIENT_Q_LENGTH ];
static DWORD dwFront;
static DWORD dwBack;
static HANDLE hQueueEvent;
static CRITICAL_SECTION QueueCriticalSection;
//
// InitializeQueue - Create a Client Request Queue.
//
DWORD InitializeQueue()
{
    DWORD dwRc = 0;
```

```
    // Initialize the queue
    dwFront = 0;
    dwBack  = 0;
    memset (ClientQueue, 0x00,
            sizeof(Queue) * MAX_CLIENT_Q_LENGTH);
    // Create an event which will be used to signal new incoming
    // requests
    hQueueEvent = CreateEvent (NULL, // security
                               TRUE, // Manual Reset
                               FALSE,// Initial state
                               NULL  // unnamed
                               );
    if (hQueueEvent == NULL)
    {
        dwRc = PrintWin32Error("CreateEvent");
        return(dwRc);
    }
    // Create a Critical Section which will protect the Queue
    // manipulation
    InitializeCriticalSection (&QueueCriticalSection);
    return(dwRc);
}
//
// QueueRequest
//          - This function is called to add a new request to the
//            queue.
//
DWORD QueueRequest(VOID * pvData, DWORD dwDataSize)
{
    DWORD dwRc = 0;
    // Acquire critical section
    EnterCriticalSection ( &QueueCriticalSection );
    // First check if there is room in the queue
    if (((dwBack + 1) % MAX_CLIENT_Q_LENGTH) != dwFront )
    {
        // Add the new request to the queue
        ClientQueue [dwBack].ClientData = pvData;
        ClientQueue [dwBack].dwDataSize = dwDataSize;
        // Move the back pointer
        dwBack = (dwBack + 1) % MAX_CLIENT_Q_LENGTH;
        // Signal the Queue event
        SetEvent ( hQueueEvent );
    }
    else
    {
        PrintError("Dropping InComing Request. Queue Full\n");
        dwRc = (DWORD)-1;
    }
    // Release the critical section
    LeaveCriticalSection( &QueueCriticalSection );
    return(dwRc);
}
//
// GetNextRequest
// - This call is made by the worker threads to get
```

```
// the next Client Request from the Queue
//
DWORD GetNextRequest ( VOID **pfBuffer, DWORD *pdwBufferSize)
{
        DWORD dwRc = 0;
        // All threads wait on an event which remains signaled
        // as long as there is at least one request in the
        // queue
        //
        dwRc = WaitForSingleObject ( hQueueEvent, 3000);
        // Check if the wait was successful
        if ( dwRc == WAIT_OBJECT_0 )
        {
            // Serialize access to the Queue
            EnterCriticalSection ( &QueueCriticalSection );
            if (dwFront != dwBack )
            {
                    // Return the requested data (pointer)
                    *pfBuffer      = ClientQueue [dwFront].ClientData;
                    *pdwBufferSize = ClientQueue [dwFront].dwDataSize;
                    memset (&ClientQueue [dwFront], 0x00, sizeof(Queue));
                    // Advance the Front ptr
                    dwFront = (dwFront + 1) %MAX_CLIENT_Q_LENGTH;
                    // If the queue is empty now, set the event in
                    // non-signaled state
                    if (dwFront == dwBack)
                    {
                         ResetEvent (hQueueEvent);
                    }
                    // Clear the error
            dwRc = 0;
            }
            else
            {
                    // Queue is empty. Treat this as timeout
                    PrintError("Queue Empty. Resume Wait\n");
                    dwRc = WAIT_TIMEOUT;
        }
        // Leave Critical Section
        LeaveCriticalSection (&QueueCriticalSection);
    }
    return(dwRc);
}
//
// File: THREADS.C
//
#include "header.h"
#pragma hdrstop
//
// Thread Pool Size is static in this example. Readers
// can easily make the pool size dynamic by monitoring
// the size of the Client Request Queue
//
#define MAX_THREADS 5
#define MAX_EVENTS  10
// Global Data
```

```
static ThreadData ThreadPool [ MAX_THREADS ];
static DWORD      dwNextThread = 0;
// We maintain one event per thread
// for signaling the threads to quit
static HANDLE   EventArray [ MAX_EVENTS ];
static DWORD  dwEventCount = 0;
//
// Simple Functions for handing out Thread Sync. events
//
DWORD GetThreadSyncEvent ()
{
    HANDLE hEvent  = 0;
    DWORD  dwEvent = (DWORD)-1;
    if (dwEventCount != MAX_EVENTS)
    {
        hEvent = CreateEvent (NULL, TRUE, FALSE, NULL);
        if (hEvent != NULL)
        {
            dwEvent = dwEventCount;
            EventArray [ dwEventCount ] = hEvent;
            dwEventCount++;
        }
    }
    return (dwEvent);
}
//
// Close an event. This is called by a thread which is going to
// terminate. Notice that in this simple example, we do not
// attempt to recycle the event slot emptied by exiting thread
//
VOID CloseThreadSyncEvent(DWORD dwEventIndex)
{
    if (dwEventIndex <= dwEventCount)
    {
        // Signal it in case anyone is waiting on it
        SetEvent(EventArray[dwEventIndex ] );
    }
}
//
// Set an event.
VOID SetThreadSyncEvent(DWORD dwEventIndex)
{
    if (dwEventIndex <= dwEventCount)
    {
        // Signal it in case anyone is waiting on it
        SetEvent(EventArray[dwEventIndex ] );
    }
}
//
// Wait for all threads to terminate
//
DWORD WaitForAllThreadsToTerminate()
{
    DWORD i, k, dwRc = 0, dwCheckPoint = 2;
    // Walk thru the array of event removing all NULL events
    // created due to exiting threads
```

```
    for ( i = 0; i < dwEventCount;)
    {
        if (EventArray[i] !=NULL)
        {
            i++;
        }
        else
        {
            // Shift all event handles to the left by one
            for (k = i; k < dwEventCount -1; k++)
            {
                EventArray[k] = EventArray[k+1];
            }
            dwEventCount—;
        }
    }
    //
    // Wait for all threads to terminate
    //
    if (dwEventCount != 0)
    {
        do
        {
            dwRc = WaitForMultipleObjects (dwEventCount,
                     (const HANDLE *)&EventArray,
                      TRUE,
                      3000
                     );
            if (dwRc ==WAIT_TIMEOUT)
            {
                // Inform the Service Manager that we are
                // still trying to close shop
                ReportStatusToSCMgr (
                                    SERVICE_STOP_PENDING, // current state
                                    NO_ERROR,             // exit code
                                    dwCheckPoint,         // checkpoint
                                    3000);                // waithint
            }
            else if (dwRc != dwEventCount)
            {
                PrintError1("WaitMultiple Error: %lx\n", dwRc);
            }
            // Advance the check point
            dwCheckPoint++;
        } while ( dwRc == WAIT_TIMEOUT);
    }
    return (dwRc);
}
//
// InitializeThreads  - Initialize the worker threads
//
DWORD InitializeThreads (PTHREAD_ROUTINE pfThreadProc)
{
    DWORD i;
    DWORD dwRc = 0;
    DWORD  dwEventIndex = 0;
```

```
    // Now start out worker threads
    for (i = 0; i < MAX_THREADS; i++)
    {
        // Get an event for this thread
        ThreadPool[i].dwEventIndex = GetThreadSyncEvent ();
        if   (dwEventIndex != (DWORD)-1)
        {
            ThreadPool[i].hThread = _beginthread(
                                                pfThreadProc,
                                                4096,  // stack size
                                                (PVOID) &ThreadPool[i]
                                               );  // argument to thread
            if (ThreadPool[i].hThread != -1)
            {
                dwNextThread++;
            }
            else
            {
                PrintErrorl("Error starting Thread #. %d\n", i);
                dwRc = (DWORD)-1;
                    break;
                }
        }
        else
        {
            PrintErrorl("Error Creating Event#. %d\n", i);
                break;
        }
    }
    return (dwRc);
}
```

Notice that the code example includes two files (THREADS.C and QUEUE.C) that are not yet used. The functions defined in THREADS.C and QUEUE.C will be used in subsequent chapters to build a Win32 Echo Server for a particular IPC mechanism. The functions in the thread.c file are used to manage multiple worker threads, with each thread servicing a given client. The functions in the queue.c file are used to maintain client data queues whose actual data format is defined by specific implementations. In the Echo Server example that follows, you will designate one thread to "listen" to the clients. When a new client request comes to the server, the listen thread will put the necessary data in the data queue. The next available worker thread will remove the new client data from the queue and service the client request by echoing back to the client. In subsequent chapters, you will see how these functions are used.

When a service is started, by either the "net start" command or the **StartService()** Win32 APIs, the main() of the service gets called. As the code above shows, the main() of the service provides a service dispatch table to the SC and blocks on the call to **StartServiceCtrlDispatcher()** API. When the service is stopped, the API completes terminating the service. The SC starts a

service by calling the **service-main()** function [supplied earlier by StartServiceCtrlDispatcher() API]. The Service-Main() function communicates the status of the service as it initializes by the ReportStatusToSCMgr() function, which in turn calls the **SetServiceStatus()** API. A service needs to initialize within 30 seconds after service-main() has been called by the SC. Hence, it is best to start a new thread from service-main(), which can initialize the server. In fact, you will do exactly this from the InitEchoServer() function. In service-main(), a Win32 event (*hServDoneEvent*) is created and waited on indefinitely at the end of the service-main() function. This means the SC thread that called the service-main() remains blocked during the lifetime of the service.

Once a service has been properly initialized, the SC communicates with it by using a service control handler. In the next example, you register the **ServCtrlHandler()** function through the **RegisterServiceCtrlHandler()** API. The handler **Service-Ctrl()** will be called to stop, pause, or resume a service. The SC can also call on this handler to interrogate the status of the service. Notice that you will call the PauseEchoServer() and ResumeEchoServer() functions when the SC asks the Echo Server to be paused and resumed, respectively. To stop the service, set the global event (*hServDoneEvent*) created in service-main(). This causes service-main() to complete, which terminates the service.

A service can be installed by using the CreateService() API. Only after a service has been installed can it be started. The code fragment that follows can be used to install the Echo Server.

```
#define ECHO_SERVER L"EchoServer
VOID main()
{
        // Full path of the image file
        LPWSTR lpszBinaryPathName =
               L"%SystemRoot%\\system32\\echosrv.exe";
        SC_HANDLE          schService,schSCManager;
        // First open the SC Manager
        schSCManager = OpenSCManager(
                        NULL,                    // local machine
                        NULL,                    // ServicesActive database
                        SC_MANAGER_ALL_ACCESS); // full access rights
        // Now create a service
        if (schSCManger != NULL)
        {
               schService = CreateService(
                    schSCManager,                // SCManager database
                    ECHO_SERVER,                 // name of service
                    SERVICE_ALL_ACCESS,          // desired access
                    SERVICE_WIN32_OWN_PROCESS,   // service type
                    SERVICE_DEMAND_START,        // start type
                    SERVICE_ERROR_NORMAL,        // error control type
                    lpszBinaryPathName,          // service's binary
```

```
                NULL,                           // no load ordering group
                NULL,                           // no tag identifier
                NULL,                           // no dependencies
                NULL,                           // LocalSystem account
                NULL);                          // no password
        if (schService == NULL)
        printf("CreateService Error: %ld", GetLastError());
        else
                printf("CreateService SUCCESS\n");
    }
    CloseServiceHandle(schService);
    CloseServiceHandle(schSCManager);
}
```

You will use this service skeleton in later chapters to create the Echo Server versions for the given IPC mechanism, supplying code for InitEchoServer(), PauseEchoServer(), and ResumeEchoServer() functions. In addition, you will present the client code that communicates with the Echo Server, using the given IPC mechanism.

2.5 SUMMARY

This chapter gave you an overview of Windows NT architecture, networking in Windows NT, and the advanced Win32 programming interfaces. This presentation was designed to help you understand the underlying system and the means by which you can write efficient and robust applications. The chapter presented the major components of the Windows NT operating system and then delineated the details of networking components, such as the redirector, WinNet APIs, and the IPC mechanism. It also demonstrated the concept and use of the advanced programming features of Windows NT that can be used as the building blocks for client/server applications. In particular, the chapter illustrated programming constructs for error and exception handling, creating and managing threads, synchronization objects (semaphores, mutex, events, and critical sections), and control-c signal handling. Finally, it demonstrated the creation of the Win32 network service (Echo Server) that will be used as a running sample program throughout the book.

With this overview, you are prepared to plunge into the communication interfaces available in Windows NT. If you are primarily interested in networking in Windows, Chapter 3 presents the networking architecture of Windows. If you are only interested in communication methods available in Windows NT, you may want to go to Chapter 4.

Chapter Three

UNDERSTANDING WINDOWS ARCHITECTURE

The majority of computers at corporations host some version of the Windows operating system. While fewer and fewer machines have the Windows 3.0 version, many workstations still host Windows 3.1 or Windows for Workgroups 3.1/3.11 (WFW). We usually refer to these Windows operating system variants as "Windows 3.x," or, simply, "Windows." Now that Windows 95 has been introduced, most PCs are expected to host the Windows 95 operating system.

As you know, Windows 95 is very distinct from Windows 3.x: Windows 95 is a fully multitasking, 32-bit operating system that supports a large subset of the Win32 APIs, making Windows 95 closer to Windows NT than it is to Windows 3.x.

Because there are so many Windows 3.x workstations, a client/server designer will probably need to create Windows 3.x versions of the client until all desktops have migrated to Windows 95 or Windows NT. That is why this chapter presents the networking architecture of Windows. This insight should provide you with the background necessary for designing communication infrastructures that use the IPC mechanisms presented in later chapters of this book.

This chapter first presents the distinctions among the three operating modes of Windows (real mode, standard mode, and enhanced mode), followed by detailed illustrations of the networking architecture of standard-mode and enhanced-mode Windows for Workgroups. Finally, it discusses the WinNet APIs used to access network resources from Windows workstations.

You might ask, "Why are we focusing only on the networking architecture of Windows for Workgroups? Why not cover networking in all variants of Windows 3.x?" The answer is that WFW contains the most mature version of networking in Windows: Windows 3.0 and Windows 3.1 have only individual subsets of all the networking features found in WFW. Windows 3.0 extends the MS-DOS networking model, but it contains only a few programming interfaces. (NetBIOS was the basic Windows networking interface.) Most Windows programmers directly access the programming interfaces (for example, the IPX interface) exposed by using MS-DOS device drivers. Thus, while you could access the network by means of LAN software (for example,

Microsoft LAN Manager or Novell NetWare), Windows itself is not well-integrated with the networking software. Windows 3.1 has improved networking specifications, allowing network drivers better integration with Windows. However, WFW provides the most comprehensive networking features, including workstation access to multiple distinct networks (for example, the Novell and Microsoft networks).

With each variation of Windows 3.x, LAN vendors have provided their own set of network drivers and programming interfaces. Because it is beyond the scope of this book to present a detailed study of these variations, we focus on networking in WFW instead. By understanding the architecture of WFW and the different Windows operating modes, you will gain insight into the other variants of Windows and their concomitant network software.

3.1 WINDOWS OPERATING MODES

This section covers the three operating modes of Windows, which are called real mode, standard mode, and enhanced mode. Windows 3.0 and Windows 3.1 support all three modes of operation, while WFW supports only standard-mode and enhanced-mode Windows.

The three modes are directly related to the Intel 80x86 processor architecture. Real-mode Windows was designed to run on the Intel 8086/8088 processors, standard-mode Windows on 80286, and enhanced-mode Windows on 80386/80486. Thanks to the backward compatibility built into the 80x86 family of processors, each high-end processor (for example, 80386) can emulate the lower-end processors (for example, 8088). In other words, an 80386 (or higher) processor can support all three Windows modes. However, before you look at the details of the Windows operating modes, you need to understand the memory protection schemes of the Intel processors.

The Intel 8086/8088 processors always run in real mode. In this mode, all memory addressable by the processor is accessible by all programs; this configuration is referred to as real-mode memory. Traditional MS-DOS programs, **terminate-and-stay-resident programs (TSRs)**, and device drivers operate in this mode.

The 80286 (and higher) processors are capable of running in the protected mode, which means applications cannot access each other's memory without proper permission. This is referred to as protected-mode memory. In protected mode, the protection is provided by the processor hardware. Usually, an operating system such as Windows or Windows NT switches a processor to protected mode before launching protected-mode applications.

The majority of Windows applications are protected-mode applications. Certain MS-DOS applications can be designed to operate in protected mode, using the **DOS Protected Mode Interface (DPMI).** However, protected mode applications do not include traditional MS-DOS device drivers and TSRs.

Real-mode and protected-mode applications can coexist, provided the processor is 80286 or higher. In fact, in a typical Windows workstation, the MS-DOS TSRs and device drivers operate in real mode and the Windows applications in protected mode. The operating system switches the processor to real mode when it schedules a real-mode application, and then switches the processor to protected mode for a protected-mode application.

Real mode gets its name from the fact that memory is accessed by real, physical addresses [Yao 92], and no translation is required. In real-mode Windows, applications can access up to one megabyte (1MB) of physical memory. Figure 3-1 shows the components of Windows in real mode. Windows itself is started by the WIN.COM program running on MS-DOS. The WIN.COM program starts the real-mode kernel (KERNEL.EXE) that manages the Windows device drivers, such as the video driver (VGA.DRV) or the keyboard driver (KEYBOARD.DRV). The kernel also starts the two key components of Windows, which are USER.EXE and GDI.EXE. While USER.EXE contains most of the Windows operating system functions and exposes them to the applications by several Windows APIs, GDI.EXE manages the graphical objects and interfaces manipulated by Windows and Windows applications. The Windows shell, called the Program Manager (PROGMAN.EXE), executes as a normal Windows application on top of the key Windows components.

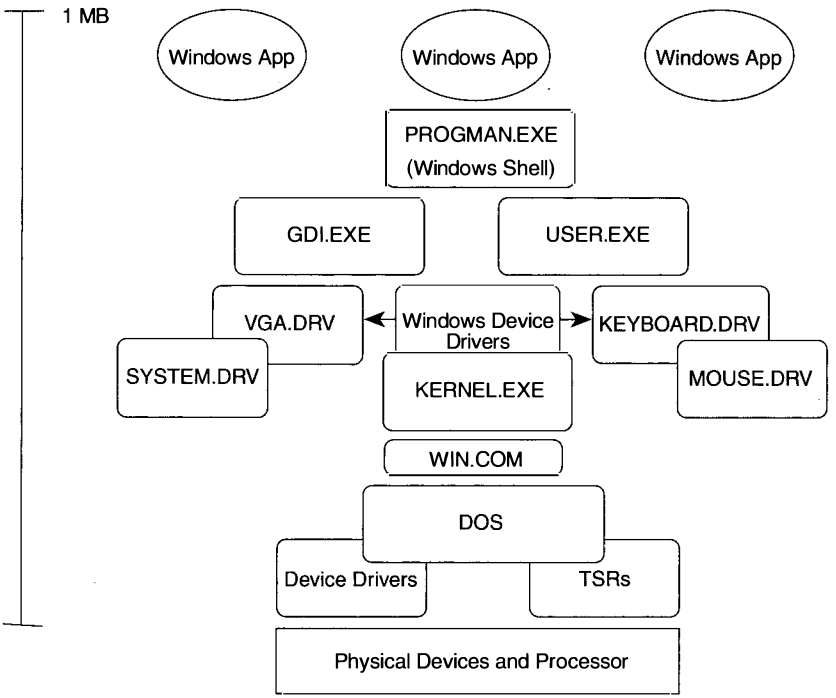


Figure 3-1. *Real-Mode Windows*

Windows runs in standard mode on Intel 80286 processors, and by launching as WIN.COM /s, Windows may be forced to run in standard mode on higher-end processors. When Windows is run in standard mode, the processor can be switched between the real and protected modes (that is, standard-mode Windows allows real-mode and protected-mode applications to coexist). The real-mode applications can access only the first 1MB of memory; the protected-mode applications can access 16MBs of protected memory space. While most MS-DOS applications are real-mode applications, some MS-DOS applications can also access, and use, protected-mode memory through the DPMI interfaces. Most Windows applications run in the protected-mode address space.

Multiple applications can be launched in standard-mode Windows. However, only one application at a time can be active, as Figure 3-2 shows. Thus, true multitasking is not available in standard mode. Note that in standard mode, the WIN.COM program launches KRNL286.EXE, the standard-mode kernel. Additionally, in standard mode, an MS-DOS extender (DOSX.-EXE) is provided to handle switching between real and protected modes.

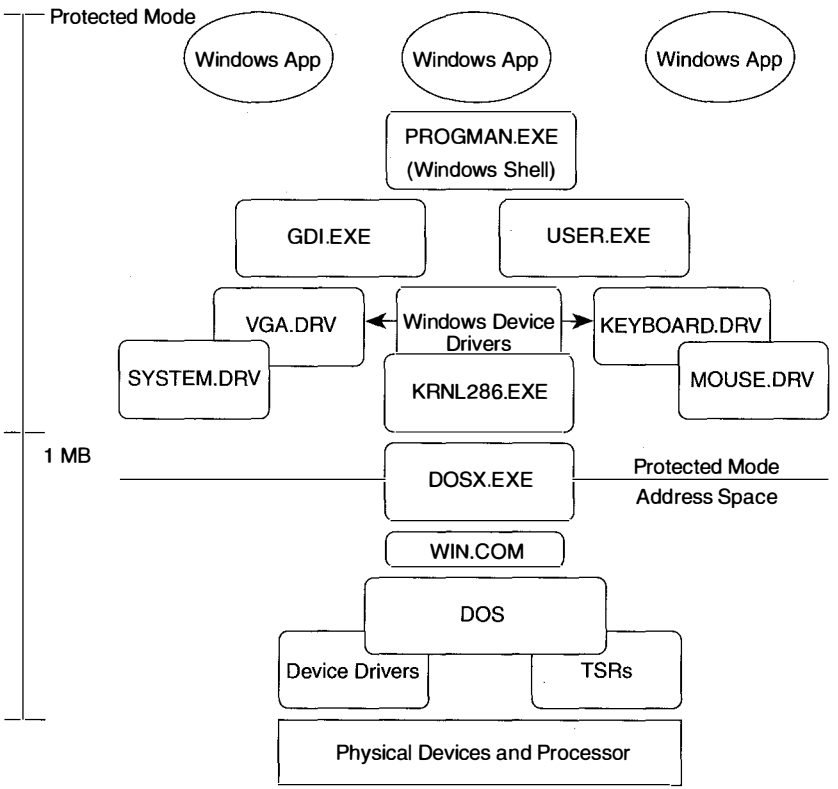


Figure 3-2. *Operation of Standard Mode Windows*

The extender is important for networking, because in standard-mode Windows all network drivers are MS-DOS device drivers operating in the real-mode memory space (<1MB). This poses a problem during the transfer of data buffers between the network drivers and the protected-mode applications: Imagine that a data packet has just been received by the network driver, and that the packet needs to be transferred to a Windows application. Somehow the driver needs to access a protected-mode address, transfer the data from the real-mode memory to the protected-mode memory, and switch the processor to protected mode so that the Windows application can access the newly arrived data. (The reverse is true when the application sends a data packet via the network driver.) Fortunately, DOSX.EXE does most of this work when the Windows (or protected-mode MS-DOS) application uses the NetBIOS interface to access the network drivers. In other words, DOSX.EXE performs the switching between real mode and protected mode (and vice versa). It also handles the transfer for NetBIOS applications of data buffers from real-mode to protected-mode memory. Most network software vendors provide their own MS-DOS extenders so that Windows applications can access their software's network drivers.

Any application can use the **DOS3Call()** function (Windows API) to access the real-mode drivers and TSRs; in standard mode, the DOSX.EXE actually performs the real-to-protected (and vice versa) memory mapping and processor switching for applications that call the DOS3Call() function.

In enhanced mode, Windows takes advantage of the **Virtual 8086 (V86)** mode of the Intel 80386 (or higher) processors. In this mode, instead of one application running at a time, multiple programs can be run simultaneously, in what is called a **virtual machine (VM)**. Every VM runs as a protected V86 with 1MB memory address space and VM-specific I/O ports and interrupts. This configuration is illustrated in Figure 3-3.

On 80386 (or higher) PCs with more than 2MB of **random access memory (RAM)**, the WIN.COM program automatically starts the enhanced-mode Windows.[1] In this mode, a special protected-mode component (WIN386.EXE) manages all the virtual memory, thus providing the VM Manager function. The VM Manager schedules each VM in a round-robin manner, which allows enhanced-mode Windows to be a nonpreemptive, multitasking operating environment. Further, the VM architecture allows users to run multiple Windows and MS-DOS applications simultaneously. In fact, all the Windows system components, including the enhanced-mode kernel (KRNL386.EXE), are launched within the **system virtual machine (SVM)** [Geary 90]. Hence, all Windows applications run within the SVM; other real-mode or protected-mode applications execute within their own VM.

1. Enhanced-mode Windows can be forced by specifying WIN/3 command.

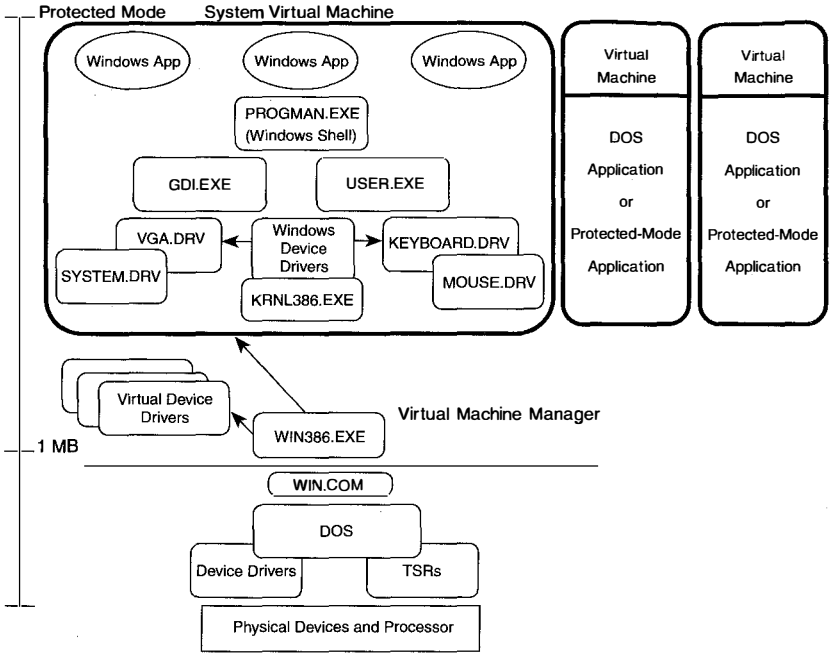


Figure 3-3. *Enhanced-Mode Windows Operation*

In enhanced-mode Windows, applications can use 32-bit instructions and access 32-bit virtual address space. A logical address is first translated to a 32-bit virtual address that is then mapped to a physical address, using the paging hardware built into 80386 (or higher) processors. While the 80386 (or higher) processors have 32 address lines, only 24 are available on the standard IBM-AT style bus. For this reason, Windows limits itself to 16MB of physical memory when it is running on such systems [Yao 92].

As you know, enhanced-mode Windows allows multitasking, which means that at any given time, multiple applications can access the real-mode drivers (and TSRs) or MS-DOS. A typical example of this is when two or more applications are accessing the network through a real-mode network driver. This is a problem for both MS-DOS and the real-mode drivers, because they are written for a single-tasking MS-DOS environment. Enhanced-mode Windows solves this problem with the virtual device drivers discussed earlier.

A VxD essentially takes over the resource and serializes multiple VM access to it. A resource may be a real-mode network driver, the video screen buffers, or a communication port (for example, COM1). Thus, all Windows and non-Windows applications must use the VxD to access the resource. This allows the VxD to ensure that only one VM accesses the resource at a time and maintain caller context so that it can locate and deliver the results to the correct caller on completion of a resource request.

For example, in Windows 3.1, VNETBIOS.386 is a special VxD that serializes access to the NetBIOS interface. When an application makes a synchronous (that is, blocking) NetBIOS call, the nonpreemptive nature of Windows prevents any other application from executing. On the other hand, multiple applications can make asynchronous (that is, nonblocking) calls to the NetBIOS interface. VNETBIOS.386 maintains both the one or more data buffers associated with each NetBIOS call and the identification of each caller. When any asynchronous NetBIOS request completes, VNETBIOS.386 schedules the correct application and delivers the data buffers to it. For further details on network VxD drivers in Windows 3.1, see [Sinha 92a].

In this section, we presented a brief overview of the three modes of the Windows operating system(real mode, standard mode, and enhanced mode). Having familiarized ourselves with the basic operation of Windows, we now proceed to understand the Windows networking architecture.

3.2 NETWORKS IN WINDOWS FOR WORKGROUPS

Windows for Workgroups is an enhancement of Windows 3.1 that brings peer-to-peer file and print sharing capabilities to the desktop. It builds on the architecture of Windows 3.1 by integrating the network capabilities in the base operating system. This makes the WFW workstations robust network clients that can access information from a variety of servers, everything from Novell NetWare to IBM mainframes. Additionally, the peer-to-peer server capabilities built into each workstation allow users to share information within their workgroup. Finally, WFW makes available the most important IPC interfaces: RPC, WinSock, Named Pipe, Mailslot, NetBIOS, and IPX/SPX. These interfaces can be used to build a communication infrastructure to bridge to application servers. A number of operating systems, such as Windows NT, OS/2, and UNIX, can host the servers. As a result, a WFW workstation can host the client side of a CSS.

Figure 3-4 shows a schematic of WFW from a network programming point of view. As you can see, a Windows application can access all the major programming interfaces (RPC, and the lower-layer interfaces WinSock, Named Pipe, Mailslot, NetBIOS, and IPX/SPX). (Note that the RPC layer uses all the available lower layer interfaces.) Named Pipe and Mailslot layers need a redirector to become functional. WFW provides a default redirector that can be replaced by any other redirector (for example, Novell NetWare). On the other hand, the Windows Socket interface is independent of the redirector and can use any of the underlying protocol drivers (such as NetBIOS, TCP/IP, or IPX/SPX drivers) to set up a socket connection. As Figure 3-4 shows, WFW makes available a number of WinNet APIs for Windows applications.

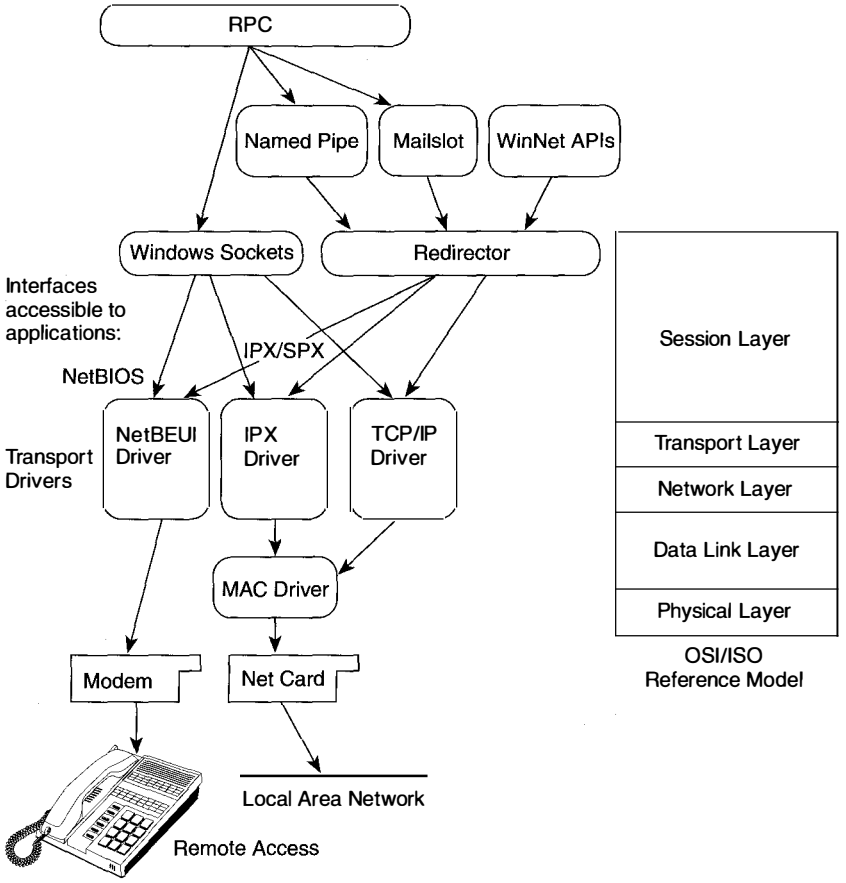


Figure 3-4. *Functional View of Windows for Workgroups*

3.2.1 Enhanced-Mode Windows for Workgroups

In this mode, WFW has the full function of a network client for nearly all popular network servers, such as Novell NetWare and Windows NT. It also contains a peer-to-peer server.

As shown in Figure 3-5, the enhanced-mode (default) redirector (which is VRE-DIR.386) provides access to the network resources. In enhanced mode, the WinNet APIs also use the redirector to provide network access to the applications. VSERVER.386 provides the peer-to-peer server functionality. By default, both the WFW redirector and the server use the services provided by the NetBIOS protocol (implemented in VNB.386) to communicate with their counterparts. The VNETBIOS.386 driver provides an abstraction layer above VNB.386 and vendors can replace the VNB.386 driver with their own implementation of NetBIOS.

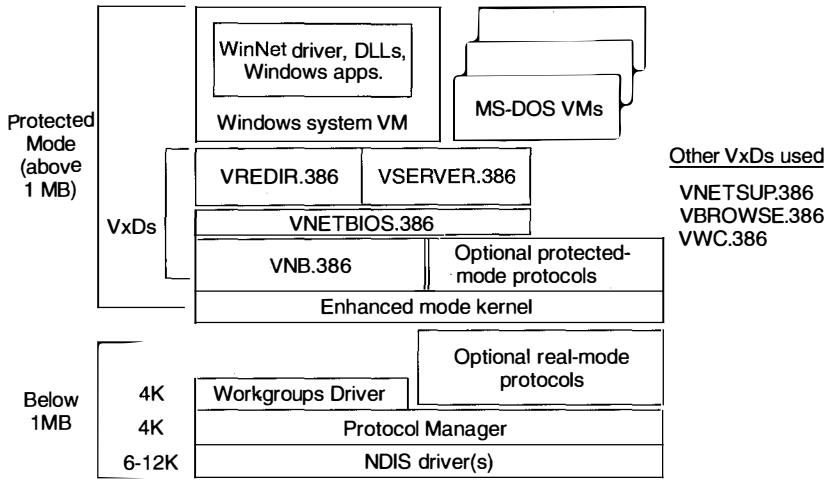


Figure 3-5. *Windows for Workgroups in Enhanced Mode*

The protected-mode protocol drivers (for example, VNB.386) provide network services by communicating with protected-mode or real-mode network adapter drivers. By default, all network adapter drivers must conform to the NDIS to function within the WFW environment.

Multiprotocol Support

Even though WFW uses the NetBIOS protocol by default, it can be configured to use other protocols, such as TCP/IP and IPX/SPX. A WFW workstation can also support multiple protocols simultaneously, which allows the workstation to use different protocols to communicate with remote machines. For example, a user can configure a WFW workstation to support both NetBIOS and IPX/SPX protocols. This configuration allows the user to access a Windows NTAS through the NetBIOS protocol, and a NetWare Server through the IPX/SPX protocol. The system components involved in such a configuration are shown in Figure 3-6.

Novell NetWare Client Support

As Figure 3-6 shows, installing a Microsoft-provided NetWare client on a WFW workstation means adding a NetWare redirector (VNETWARE.386). The NetWare redirector uses the protected-mode IPX/SPX VxD (VIPX.386) that serializes access to the real-mode IPX protocol driver from multiple virtual machines. The real-mode protocol driver (MSIPX.COM) then communicates with the real-mode network adapters, which are written to the NDIS standard. In other words, WFW replaces the traditional NetWare client drivers (NETX.COM and IPX.COM) with another pair of drivers (NETX.COM and MSIPX.COM).

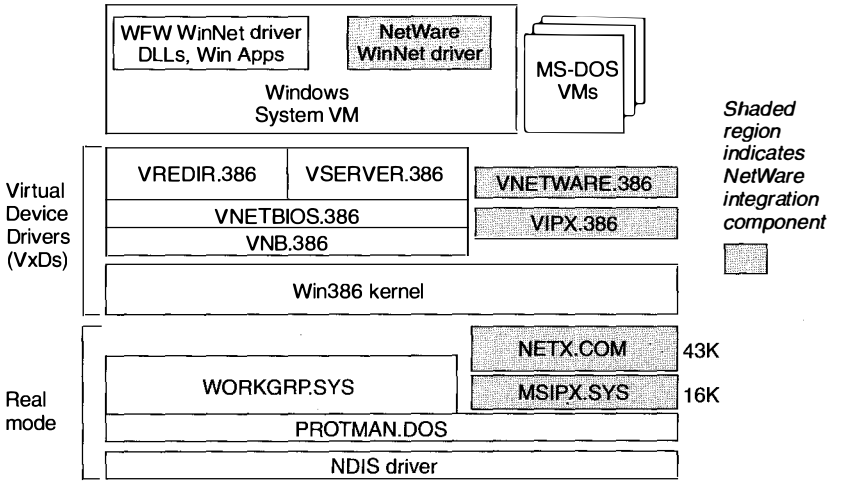


Figure 3-6. *Windows for Workgroups Set Up as a Novell NetWare Client*

If you are familiar with NetWare, you might ask, "Why do we need MSIPX.COM? Where are the NetWare **open datalink interface (ODI)** network adapter drivers?" The answer is that the Novell-supplied IPX protocol driver (IPX.COM) communicates only with the network adapter drivers written to the ODI standard. Because WFW networking architecture is based on the NDIS drivers, Microsoft provides a replacement IPX protocol driver (that is, MSIPX.COM) that provides the full services of an IPX/SPX protocol driver while using the services of the underlying NDIS drivers. See [MS-WFW 93] for further details.

This configuration allows access to Microsoft servers as well as NetWare servers. If you are interested in accessing only the NetWare servers, you can easily remove the default WFW drivers (for example, NetBIOS driver, WFW redirector, or MSIPX.COM) and install only the Novell NetWare client software for WFW.

Peer Server in Windows for Workgroups

The peer-to-peer server (VSERVER.386) provided with WFW communicates with other print servers and file servers that use the SMB protocol. This allows a WFW workstation to share local files, printers, and modems with other workstations within the workgroup. Additionally, the resources shared by the workstation can be accessed by other SMB-protocol-based print clients and file clients, such as IBM LAN server workstations, Microsoft LAN Manager workstations, Digital Pathwork workstations, and so on. The user, of course, can control access by associating passwords with each resource.

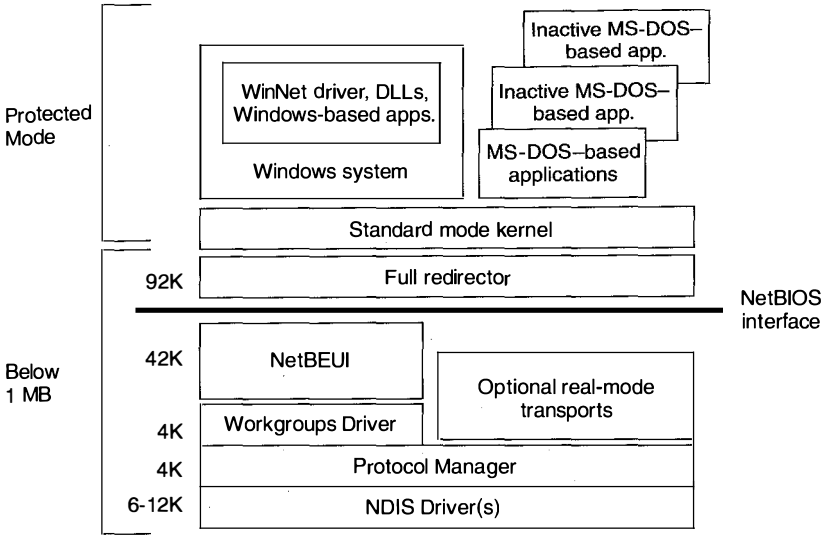


Figure 3-7. *Windows for Workgroups in Standard Mode*

3.2.2 Standard-Mode Windows for Workgroups

In this mode, WFW provides full network access support for many supported protocols. However, a peer-to-peer server is not enabled, which means that a user cannot share standard-mode WFW workstation resources with others in the workgroup. Multitasking is not available in standard-mode WFW.

Figure 3-7 illustrates how applications still use the WinNet drivers and DLLs that allow vendor-independent means of accessing network resources. The WFW redirector uses (by default) the real-mode NetBEUI protocol driver.

Standard-mode WFW supports most network programming interfaces, such as IPX/SPX, NetBIOS, and WinSock. Thus, a standard-mode WFW workstation can communicate with other workstations by traditional IPC methods and can access resources shared by remote print servers and file servers, such as Novell NetWare or Windows NTAS, provided all the necessary components (for example, IPX drivers for NetWare Server access) are loaded. Finally, standard-mode WFW can support multiple protocols.

3.2.3 Accessing Network Resources

In a Windows environment, applications can access most of the common network resources, such as file servers and printers, by using the WinNet APIs (introduced in Chapter 2 and also known as WNet APIs). This section presents a list of the WinNet APIs supported on a WFW workstation.

WinNet APIs are available in all versions of Windows, Windows NT, and Windows 95. The number of WinNet APIs has grown since they were first introduced in Windows 3.0, and it will grow even further. These APIs allow programmers to access network resources from their applications in a consistent, portable manner. See [King 94] to understand how WinNET APIs are integrated within the Windows 95 operating system and [Baker 93] for an overview of them across multiple versions of Windows, Windows NT, and OS/2.

While it is not appropriate to expand here on how the WinNet APIs are used, the following code fragment illustrates their use. In this code fragment, **WNetGetCaps()** API determines the type of network (for example, WFW, NetWare, multiple networks) present on a WFW workstation.

```
////////////////////////////////////////////////////
// Determine the type of network using the WNetGetCaps() API
//
WORD nIndex = 0;
nIndex = WNetGetCaps(WNNC_NET_TYPE); // Ask for type of network
if (nIndex & WNNC_NET_MultiNet)
{          MessageBox(NULL, "Multiple Networks Present", "Net Type",
                         MB_OK);
           if (nIndex & WNNC_SUBNET_NetWare)
                    MessageBox(NULL, "NetWare Networks", "SubNet Type",
                         MB_OK);
           if (nIndex & WNNC_SUBNET_WinWorkgroups)
                    MessageBox(NULL, "WFW is present as well",
                         "SubNet Type", MB_OK);
}
else
           MessageBox( NULL, "Single Network Present", "Net Type",
                         MB_OK);
```

The next program code example shows how an application uses the **WNetAddConnection()** API to connect a local drive (F:) to a remote file server share (\\SERVER\Docs). This code fragment opens a file on the remote server, processes it, and uses the **WNetCancelConnection()** API to break the network connection to the remote server.

```
////////////////////////////////////////////////////////////
// Add a connection, read a file, and close the connection.
//
char  szLocalName [] = {"F:"};
char  szNetPath   [] = {"\\\\Server\\Docs");
char  szFileName  [] = {"F:\\Public.doc"};
LPSTR pszPassword = NULL;
int   fh;
UINT  iRet;
// Create a connection
iRet = WNetAddConnection ( szLocalName, pszPassword,szNetPath );
if (iRet == WN_SUCCESS)
{
          // open a file on the remote drive
          fh = fopen (szFileName, "r");
          // process the file
```

```
        <...>
        fclose ( fh );              // close the file
        // Close the connection.
        iRet = WNetCancelConnection(szLocalName, TRUE);
}
```

Table 3-1 lists some of the most useful WinNet APIs available in WFW for accessing remote printers, disks, modems, and other resources. See the WFW software development kit [MS 92d] or [Baker 93] for sample programs and complete documentation on the WinNet APIs.

Table 3-1. *Frequently Used WinNet and MNet APIs in Windows for Workgroups*

Windows for Workgroups Functions	*Description*
Network Information	
WNetErrorText()	Returns the error message associated with the last WNet API error.
WNetGetCaps()	Returns the capabilities of the underlying network such as type of network (NetWare, Windows for Workgroups), information about multiple networks, and so on.
WNetGetError()	Returns the error code associated with the last WNet API.
WNetGetShareCount()	Returns the number of shared resources (disk drives or printer ports) of this workstation.
WNetGetShareName()	Returns the share name associated with a local drive-based path.
WNetGetSharePath()	Returns the drive-based path associated with a local share name.
Network Resources: Connecting to Network Directories and Printer Queues	
WNetAddConnection()	Redirects the specified local device (either a disk drive or a printer port) to the appropriate shared device or remote server.
WNetCancelConnection()	Disconnects a local device from the shared device or remote server.
WNetGetConnection()	Returns the name of the shared resource associated with the specified redirected local drive or device.
WNetGetLastConnection()	Returns an index identifying the most recent device to be connected to a network resource.
WNetRestoreConnection()	Reconnects one or more permanently connected devices.

(continued)

Table 3-1 *continued*

Windows for Workgroups Functions	*Description*
Network Resources: Dialog Boxes for Connecting to Network Directories and Printer Queues	
WNetBrowseDialog()	Displays one or more dialog boxes that enable the user to select a remote shared resource.
WNetConnectDialog()	Displays driver-specific connection dialog box.
WNetConnectionDialog()	Displays the dialog box used for both connecting and disconnecting devices.
WNetDisconnectDialog()	Displays driver-specific disconnection dialog box.
WNetServerBrowseDialog()	Displays a dialog box containing the names of workgroups and computers. If the user chooses a computer, the function returns the computer name in a caller-supplied buffer.
WNetSetDefaultDrive()	Sets the default drive for the WNetDisconnectDialog functions.
Network Resources: Shared Directories	
WNetShareAsDialog()	Creates a shared directory. It displays a dialog box requesting the name, comment, and one or more passwords for the shared directory.
WNetStopShareDialog()	Removes the given shared directory. It displays a dialog box identifying the shared resource to remove.
Multiple Network Functions	
MNetGetLastTarget()	Returns the multinet handle of the network most recently accessed by a WinNet call.
MNetGetNetInfo()	Retries information about a particular network driver.
MNetGetResourceNet()	Provides the multinet handle for the network associated with a particular resource.
MNetNetworkEnum()	Enumerates installed networks.
MNetSetNextTarget()	Sets target network driver for next WinNet call.

The MNet APIs listed in the last section of Table 3-1 switch between networks when WFW is configured to use multiple networks, such as Microsoft Network and Novell NetWare. Figure 3-7 shows how WFW can be configured to support both the WFW and NetWare drivers, which makes it possible for a user to access Windows NTAS as well as NetWare servers. There is a problem, however, because WFW is limited in its multiple network support. It only allows one active network at a time. When the user needs to use a nonactive network, its drivers have to be activated by some application (for example, the Control Panel, File Manager, or a Windows application). The application calls the MNet APIs to switch one network to another.

Figure 3-8 shows a schematic in which the MNet API is used to change from the (default) WFW driver to the Netware driver so the user can access the NetWare servers from the workstation. Figure 3-8(a) shows the initial state of the workstation in which the WFW drivers are being used, providing access to Windows NTAS. When the user chooses a NetWare server, the drivers may be changed by using the Control Panel and setting it as shown in Figure 3-8(b). Any Windows application can call the **MNetSetNextTarget()** API to change from WFW drivers to NetWare drivers (or vice versa). Once this change has been made, all the network resources being handled by the WNet API are directed toward the NetWare drivers.

Fortunately, both Windows NT and Windows 95 automatically activate the correct network drivers, so Windows applications need not concern themselves with the MNet APIs. See Chapter 2 to understand how multiple networks are handled in Windows NT. [King 94] contains an explanation of MPR support in Windows 95. For further details on MNet APIs and their use in WFW, see the WFW software development kit [MS 92d] or [Baker 93].

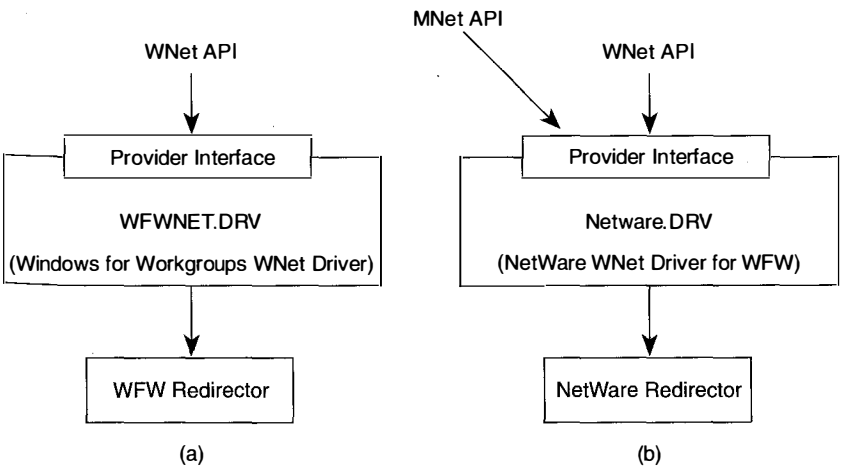


Figure 3-8. *Using MNet API to Change Network in WFW*

3.3 SUMMARY

This chapter presented an overview of the three operating modes of the Windows (version 3.0/3.1) operating system (real mode, standard mode, and enhanced mode) and then introduced the networking architecture of Windows for Workgroups 3.11 as the most network-capable version of 3.x variants of Windows. It illustrated the network components in standard-mode and enhanced-mode Windows for Workgroups. Both modes of WFW have built-in networking components (redirector, protocols, and network adapter drivers, and so on) to allow users easy access to the network resources within their workgroup. Most of the networking components can be replaced by similar components from other vendors so as to allow access to other networks (for example, Novell NetWare). Enhanced-mode WFW includes a peer-to-peer file server, which can be used to share workstation resources (for instance, files, printers, modems) by a workgroup. WFW supports several IPC methods, such as RPC and WinSock, so that client/server applications can be hosted on WFW workstations. Finally, the chapter introduced the WinNet APIs that are available for accessing network resources regardless of the underlying network software.

Chapter Four

RPC PROGRAMMING IN WINDOWS NT

At the heart of the remote procedure call (RPC) is the simple idea of being able to call a procedure that resides in a server process running on a remote machine. Microsoft RPC provides all the necessary tools—from compiler to run-time support—to develop client/server applications in which the clients execute procedures resident on the servers. How does a client call get transferred to a remote server function? Envision a packet of data being sent from the client to the server. The packet contains an ID of the remote function to be executed on the server and the parameters of that function. Undoubtedly, you will need to send a packet, containing the result of the function call, back from the server to the client. RPC provides all the necessary libraries and DLLs to transmit remote procedure calls between client and server. The developer need only specify one or more IPC mechanisms (such as TCP/IP, Named Pipe, IPX/SPX, or Windows Sockets) that are to be used for the transfer of data. RPC allows you to focus on developing the application function rather than on writing networking code.

This chapter illustrates RPC concepts and use in the development of client/server applications in the Windows NT environment. Section 4.11, "RPC Programming in Windows," discusses RPC use in the Windows environment.

This chapter focuses primarily on Microsoft RPC, with extra emphasis on design considerations that can enhance the performance of RPC-based client/server applications. Because Microsoft RPC is designed to be compatible with **Distributed Computing Environment™ (DCE)**, proposed by **Open Software Foundation™ (OSF)**, the chapter occasionally illustrates the differences between Microsoft RPC and DCE. See [Rosenbary 93] for details on interoperability between DCE and Windows NT. [MS 93a] presents a concise list of incompatibilities between DCE RPC and Windows NT RPC. See [Sinha 92b] for a historical perspective of RPC and [Finnegan 94] for a short article on Microsoft RPC use. [Liu 94] provides a discourse on the OSI RPC model.

The chapter begins with a historical perspective on RPC, and then it illustrates Microsoft RPC concepts. Next, it presents a simple and almost

functional RPC-based client/server application to familiarize you with the development of an RPC-based client/server application. The rest of the chapter describes in detail these key RPC concepts:

- Designing efficient client/server applications by using RPC data types such as pointers, nested arrays, and others
- Designing a communication infrastructure with RPC
- Using Name Service from the client/server application
- RPC binding handles
- Using callback routines
- RPC management
- RPC error and exception handling
- Secure RPC

The chapter ends with a fully functional example of the RPC-based Win32 Service (Echo Server), first introduced in Chapter 2.

4.1 HISTORICAL PERSPECTIVE

PCs provide the framework for a distributed processing system in which processing load is spread across heterogeneous processors loosely coupled by networks. RPC provides the mechanism by which a local procedure can be executed on a remote processor, with the results returned to the local procedure, without the application's being aware of the exchange [Sinha 92b].

In an RPC client/server system, when a remote procedure at the client end is called, the arguments of the procedure are packaged into a packet and sent to the server, and the calling procedure at the client end is blocked. At the server end, the packet is unpacked upon delivery, and a procedure local to the server executes with the arguments passed by the client. At the completion of the server procedure, all results are placed in a packet and sent back to the client. The results are unpacked as return values from the remote procedure. At this time, the client's calling procedure resumes execution.

However, there is no single mechanism for implementing RPC. Historically, two distinct (commercial) RPC implementations can be traced to two distributed systems architectures—**Open Network Computing (ONC™)** from Sun Microsystems and **Network Computing Architecture (NCA™)** from Apollo Computers®. Here, briefly, are implementations in each network architecture.

- **RPC in ONC (Sun RPC).** In this implementation, a procedure is executed remotely and its return values are brought back by means of arguments to the call. In this system, the data sent between client and server is packaged into a linear array of bytes in a format known as **external data representation (XDR)**. Both the client and the server provide the XDR functions that perform any data

translation required when data is transferred between heterogeneous computer systems. Figure 4-1(a) illustrates the Sun RPC call, **CallRPC()**, which takes a set of parameters such as the remote server name, the program name, a version number, the procedure number, an XDR routine to convert input arguments into a byte of array in XDR format, the actual arguments, an XDR routine for returned arguments, and the return arguments. In this model, applications need to be aware of the underlying network and protocols.

- **RPC in NCA (Apollo RPC).** In this model, a set of routines, called stubs, is provided at both the client and server ends. Thus, any local calls on the client are routed to the remote server procedure through the client and server stubs, as shown in Figure 4-1(b). Typically, the stubs are generated by an RPC compiler that interacts with the RPC run-time library. The RPC run-time library shields both the server and the client applications from network communication details . For example, the Apollo RPC run-time library provides a connection-oriented data transfer service over user datagram protocol (UDP), which is a connectionless service.

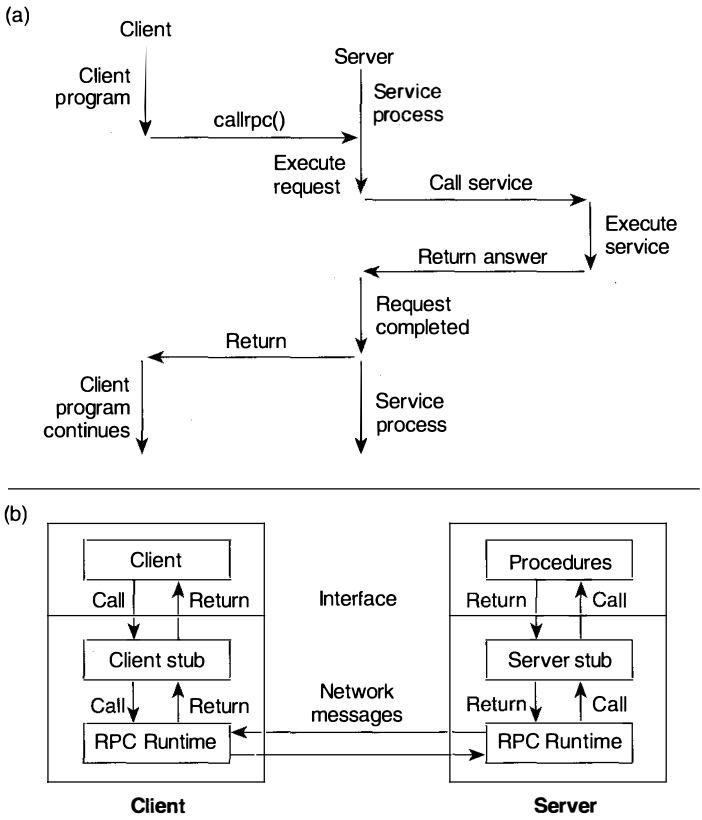


Figure 4-1. *Apollo Compared with Sun RPC (Source: Sinha 92a)*

The DCE RPC, like the Microsoft RPC, is based on the NCA model. On the other hand, the Netwise™ implementation of RPC for Novell NetWare is similar to the ONC mechanism. See [Manson 89] for a detailed comparison between NCA RPC and ONC RPC.

Note that RPC provides a communication channel over the session layer in the OSI reference model. (RPC atually lies between the presentation layer and the application layer.) This gives applications an abstraction layer over any network communications that may have incompatibility problems. In typical PC-based LAN environments, the network software exposes session-layer services to applications. Hence, RPC (using stubs and RPC run time) provides the presentation layer that converts data into a common data format, such as XDR, before sending the data over the network. This allows RPC-based applications to communicate over heterogeneous systems and to be used over any available network protocol (for example, TCP/IP or IPX/SPX), as long as both client and server have at least one common protocol.

4.2 MICROSOFT RPC CONCEPTS

Microsoft RPC for Windows and Windows NT (as well as MS-DOS, Macintosh, and Windows 95) is a package containing APIs, libraries, and binaries that allows developers to write client-server applications without having to write any low-level communication code. The developer chooses one or more protocols (Named Pipe,[1] WinSock, and so on) that the client/server application wishes to support; RPC provides all the necessary code (libraries) and run-time support (Win32 services and DLLs) to enable one or more clients to communicate with the server. Using the RPC run-time support, a client makes a function call that is executed on the server, as illustrated in Figure 4-2.

Like a normal (local) function, the remote function may take parameters. The IN/OUT convention is used to distinguish the parameters supplied by the caller (that is, IN parameters) of a function from the values returned by the callee (that is, OUT parameters). The calling function that is resident in the client application may supply some values by using one or more IN parameters, and the called function that is resident in the server application may return an error code as well as some results by using the OUT parameters.

If the complete transparency between a local procedure and a remote procedure is to work, the designer has to solve two major impediments—connection-oriented network communication over connectionless or connection-oriented protocols, and the marshaling and unmarshaling of data as it is sent across the network.

1. Note that Named Pipe actually runs over all transport protocols supported on Windows and Windows NT machines.

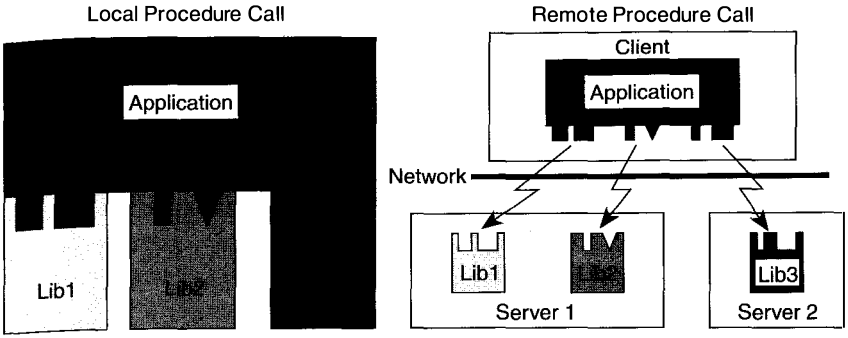


Figure 4-2. *Local Call Compared with Remote Procedure Call*

Typical RPC provides connection-oriented service to applications, which means that after making an RPC call, an application expects the remote application to receive the call and return results. (Starting with Windows NT 3.5, Microsoft RPC provides connectionless service, as well.) When the underlying protocol (for example, TCP or Named Pipe) provides a connection-oriented service, it is easy for RPC to provide guaranteed service. However, the underlying protocol (for example, IPX or Mailslot) may have connectionless service. In this case, RPC must provide guaranteed service using the underlying connectionless service. In either case, RPC must give the application network communication support so that following sequence of operations can complete:

- A session, or virtual circuit, is established between client and server.
- Client-supplied parameters is sent from client to server.
- Wait until server sends back a return value and results via parameters.
- Disconnect the session if the client has completed its dialog with the server.

Microsoft RPC provides seamless support for a range of network communication through a number of libraries and DLLs. RPC provides run-time support for a number of protocols—Named Pipe, NetBIOS, TCP/IP, IPX/SPX, and local RPC (between processes on the same machine).

RPC also marshals and unmarshals data. In distributed applications, the client and the server applications are typically on two different machines. The data representation format can be quite different between the two machines, for example, little endian or big endian machines, in which the byte ordering is reversed. Hence, you must send and receive data in a standard format that dictates how bits are to be laid out in packets, regardless of the machine architecture. To do this, Microsoft RPC converts all the data to be sent and received over the network to a standard format called **Network Data Representation (NDR)**. Then, to maintain transparency between a local procedure call and a remote procedure call, a programming language support is

given. The primary language of choice is C. Of course, there is no point in sending a pointer (a four-byte address) from the client to the server unless you can send the object that the pointer points to. This additional code, provided by RPC, is called the stub.

Figure 4-3 illustrates the relationship between the RPC client/server applications and the RPC run-time support and stubs. In the figure, the sequence numbers (1 through 16) illustrate the transition of the RPC from the client application to the server application, and back. First, the client-side stub converts the client (function) parameters to a buffer in an NDR format, and uses RPC run-time support to send the remote call to the server-side RPC run-time. The server-side stub receives the buffer and converts it to one or more parameters of the target procedure resident in the server application. The server-side calls the server functions and awaits the results. Once the server function completes, the server stub marshals any results returned from the server function into its NDR format and transmits the results back to the client stub. Upon receiving the server response, client run time transfers the result (in NDR format) to the client stub. The client stub translates the server response from the NDR format to values meaningful in the client application address space. At this point, the RPC ends and control returns to the calling function in client code.

The next section presents an almost functional RPC-based client/server application to give you an idea of how RPC is used.

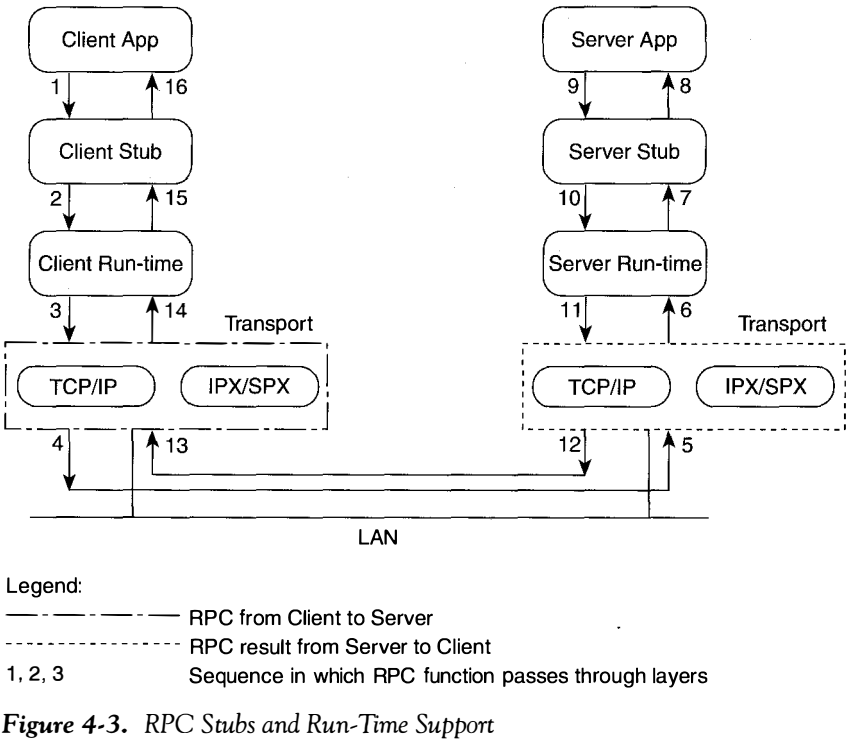


Figure 4-3. *RPC Stubs and Run-Time Support*

4.3 WRITING A SIMPLE RPC-BASED CLIENT AND SERVER

Figure 4-4 is a schematic that shows the work involved in creating an RPC client and server. You write two C programs, two network-related headers files, and then leave the rest to RPC. The client application (Client.c) involves one or more remote procedure calls, and the server application (Server.c) provides support for the remote procedures. Next, you create two files that look like standard C header files. These are the interface definition language (IDL) and the application configuration language (ACF) files, called "Simple.idl" and "Simple.acf" in this example.

With these files, you instruct the **Microsoft RPC IDL compiler (MIDL)** to produce client and server stubs. In Figure 4-4, the client stub generated by MIDL is simple_c.c, while simple_s.c is the server stub. MIDL also generates a common header file (in this case, simple.h) that contains C prototypes for common data structures and remote procedure calls. You then compile and link these stubs and header files with the RPC run-time library to produce client or server executables.

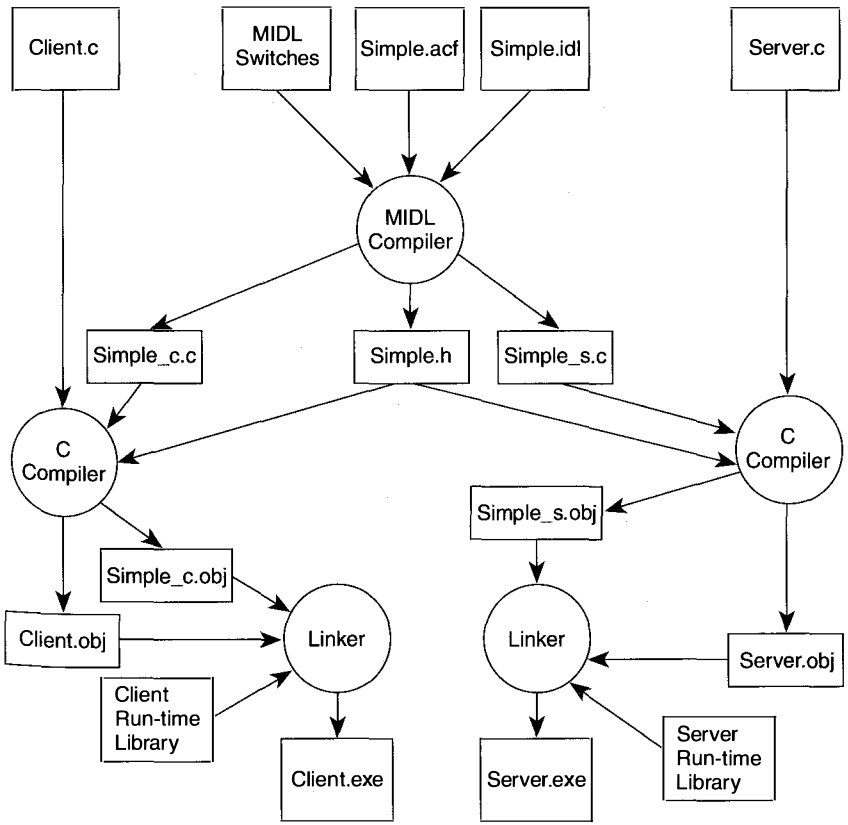


Figure 4-4. RPC Client/Server Sample Production Process

The following code fragment shows an RPC-based client/server application that allows the client to send a message to the server, which prints the message on the server console. In the listing below, the files simple.acf, simple.idl, and client.c are complete and fully functional. However, some RPC calls have been left out of server.c (which the server needs to call to set up its run-time environment). Once the RPC server is set up, the server's main thread calls the RPC function **RpcServerListen()**, which causes the main thread to wait indefinitely. The RpcServerListen() API triggers the RPC run-time to start one or more threads[2] as child threads of the server's main thread (that is, these threads are part of the server process). Each of these threads is always listening for the clients to send messages. When it receives a message from the client, the RPC run time will hand the request to one of the child threads, which will execute the remote procedure **PrintOnServer()** contained in this sample server. Finally, any results (none in this case) are returned to the client.

Simple.IDL File

```
[ uuid (906B0CE0-C70B-1067-B317-00DD010662DA) ]
interface Simple
{
            void PrintOnServer( [in, string] char * pszMessageToServer);
}
```

Simple.ACF File

```
[auto_handle]
interface Simple
{
}
```

Client.C

```
#include <stdio.h>
#include <rpc.h>
#include "simple.h"
void main(void)
{
        // Call a local procedure
          PrintOnClient("This line is printed by client application\n");
        // Call the remote procedure. The client is blocked
        // until the function completes execution on server.
          PrintOnServer("This line is printed by server on the server\n");
          return;
}
void PrintOnClient (char *pszString)
{
            printf("Client: [%s]\n", pszString);
}
```

Server.c

```
#include <stdio.h>
#include <rpc.h>
```

2. The number of threads is configurable at run time, and is discussed later.

```
#include "simple.h"
//
// The main thread of server application waits on RpcServerListen()
// forever until server is stopped.
//
void main(void)
{
            RPC_STATUS status;
            // Some functions that make the server available to the
            // one or more clients.
            < Missing Server Configuration Functions >
            // The main thread is now going to wait forever on
            // following function until server is terminated
            // by some means to be discussed later
            status = RpcServerListen ( .. );
            return;
}
//
// When the client request comes in, RPC starts a thread which executes
// the following procedure for client. Notice that the main thread
// keeps listening to new clients without doing much.
//
void PrintOnServer (char *pszMessageFromClient)
{
            printf("Client Sent: [%s]\n", pszString);
}
```

The IDL file contains one or more interfaces that define the program contract between the server and its clients. Each interface defines the common data structures, function prototypes, input/output/in-out parameters, and return values from the remote procedure functions. Each interface is distinguished by a universally unique 16-byte-long ID, called GUID or UUID, which can be generated by the RPC tool uuidgen. Uuidgen uses the computer time and its network card address to generate a unique ID and a version number. Why does every interface need a UUID? Imagine a network on which hundreds of RPC servers are running, each supporting its set of interfaces. Because the clients on the network need to use the servers, they have to be able to locate a server that services a given interface. An interface's unique ID allows it to be identified among all the many interfaces supported by one or more RPC servers present on the network.

When you study the IDL file, you will notice the use of []. An attribute in square brackets [] strongly types[3] the parameters, which is necessary to RPC. The code example above specifies that the server takes a string parameter by using [in, string]. The server may support one or more interfaces defined in an IDL file. IDL files may contain network-specific details (for example, server machine address) if any. It therefore may define how data is transmitted on the network.

3. This refers to unambiguous references to a data type. For example, VOID* can point to any type of data object just as int can refer to a 16- or 32-bit number. References such as USHORT, LPSTR, BOOL distinguish the data type unambiguously.

The ACF file defines the RPC applications for a specific operating environment. In the example above, by using the keywords (auto_handle), you specify that client and server connect with (or bind to) each other "automatically." For automatic binding, RPC uses internal techniques (Windows NT Name Service, discussed later) to locate the server that services a given interface at run time, and does so without the client having to specify the name of server or the name of the computer hosting the server.

This almost complete RPC client/server program will work regardless of the underlying operating system[4] (for example, Windows 95, WFW, Windows 3.1, Windows NT, MS-DOS), the LAN software type (for example, Windows NT, DEC, Novell NetWare), or network protocol (for example, TCP/IP, IPX/SPX, NetBIOS). The power of RPC lies in hiding the details of communication, the potential heterogeneity of the computer systems, and the operat-ing environments. RPC's most significant contribution to application develop- ment is that it allows application writers to concentrate on the application, rather than on the communication between the client and the server. RPC therefore enables rapid prototyping, and eventual deployment, of client/server applications.

However, RPC is not useful in all circumstances. For example, you cannot use RPC for peer-to-peer networking, or for developing custom application servers for Windows 3.1 and MS-DOS environments. In such cases, you must use lower-level communication mechanisms (such as NetBIOS, IPX) directly.

4.4 DESIGNING AN EFFICIENT RPC CLIENT AND SERVER

As you have seen in the previous sections, in Microsoft RPC, the client communicates with the server by calling a procedure resident in the server. The client does so by means of client/server stubs, RPC run-time support, and the underlying transport support. The client code calls a client-side procedure, and this is transferred to the server. On the server, a server-side procedure is executed, and the results are returned to the client procedure. In this section, we will discuss designing efficient remote procedure calls by focusing on the parameters passed via RPC. We will introduce each of the RPC data types and illustrate their proper usage. The data types presented are base types, arrays, strings, structures, pointers, and nested arrays.

In the next section, "Designing Communication Infrastructure," we will discuss how a client and server initially get connected. In that section, we point out the ways to make the communication between the client and the server more efficient. Thus, in this section, we will assume that client and server have *somehow* become connected and any client-side RPC call causes the server-side procedure to be activated.

4. However, you must recompile and link with proper libraries for each environment. Further, the server applications run only on Windows NT, Windows 95, certain varieties of UNIX, and OS/2 platforms.

Recall that you write an RPC function prototype into an IDL file. This is processed by the MIDL compiler to produce the client and server stubs and a common interface header. The interface header contains the prototype of the RPC(s). MIDL understands strongly typed C and C++ definitions. This is necessary if it is to produce client/server stubs that always work, irrespective of the computer type. For example, it is common to declare a parameter or function return type as **(void*)**. This is not allowed in IDL because (void*) cannot be resolved unambiguously and so arguments cannot be re-created as needed. Similarly, IDL forces **int** to be properly defined as a signed/unsigned 16- or 32-bit number.[5] Furthermore, it defines base types that have the same meaning across platforms, so RPC applications can always work consistently. Finally, beside declaring the type of parameter, like in C, you may have to specify additional attributes in square brackets []. For example, the following application specific function—RemoteFunction()—uses an unsigned 16-bit integer as a parameter:

```
void RemoteFunction(
    [in] unsigned short Number
    );
```

You read earlier about the [in], [out], and [in,out] attributes. These attributes define, respectively, whether a given parameter is passed from the client to the server; from the server back to the client; or from the client to the server and back, which must return a value to the client. The following example shows how to use the [in] and [out] parameters. It is an RPC call that allows a banking-system client to deposit money into, or withdraw money from, an account.

```
void BankTransaction(
    [in] short TransactionType,        // Deposit, Withdrawal etc.
    [in] char * szAccountName,         // Client sends user name
    [in,out] signed long *lMoney,      // Money deposited by client or
                                       // Transaction amount allowed by
                                       // the Server
    [out]  signed long *lBalance       // Server returns a balance via this
                                       // field
    );
```

These attributes allow proper client/server stubs to be generated to deal with each case. Programmers should design prototypes with an eye on the data transfer price tags. Further, note that an [out] parameter remains undefined until the server returns a value in it, yet the pointer must point to a valid memory area in the client's address space prior to the call (see Section 4.4.5, "Pointer," for details).

5. Actually, **void*** and **int** may be used in some specific situations, and MIDL supplies means of specifying the precise meaning of each.

4.4.1 Base Types

Table 4-1 shows RPC-defined base data types that can be used as building blocks for more complex data types, such as structures and unions. Integer-based base types can be modified by **signed** or **unsigned** keywords.

Any RPC client/server application can use the base types to communicate with each other, as shown by the following example:

```
unsigned short SendRequest (
    [in] wchar_t *pwszClientName,           // Unicode-based client name
    [in] short SomeNumber,                  // signed 16-bit integer
    [out] small SequenceNo,                 // signed 8-bit number
    [in] byte bClientDigitalSignature[100], // Bits transported as is
    [in,out]  double *TotalBalance
);
```

In a network environment, connection breakdown is not uncommon. Some examples are power failure, machine crash, or LAN failure. While they are infrequent, such faults can occur while a client is communicating with the server. If this happens, RPC run time and the stubs throw an exception. Client/server applications need to be protected against such exceptions. You can use a C **try/catch** block around every RPC call, but that may lead to cluttered code. Or you can use an RPC-provided primitive error_status_t, which catches these exceptions and returns a valid and documented RPC error code to the caller. (See Section 4.8, "RPC Error and Exception Handling," for strategies for handling RPC errors and exceptions.)

Table 4-1. *RPC Base Types*

Base Type	*Description*
boolean	Can be either TRUE or FALSE
byte	8-bit data transmitted as-is without any conversion
cha	8-bit unsigned data
double	64-bit floating-point number
float	32-bit floating-point number
handle_t	RPC primitive handle indicating client/server binding (connection)
long	32-bit integer
short	16-bit integer
small	8-bit integer
wchar_t	Wide-character type widely used in Windows NT for internationalization purposes*

* See [Hall 91] and [Sheldon 91] for internationalization issues.

The following example shows a function declaration with error_status_t.

```
typedef error_status_t ERRORCODE;     // All RPC functions will use this
                                      // type to catch RPC exceptions
// This function is safeguarded against RPC exceptions
ERRORCODE SafeRemoteFunction (
    [in  ] long SendACodeToServer,    // Client sends an opcode to server
    [out] long *ServerReply           // Server replies suitably
    );
//
// Client pseudo code using SafeRemoteFunction()
//
int main (void) {
    ERRORCODE err;
    long lCode, lReply
    err = SafeRemoteFunction (lCode, &lReply);
    if (err != RPC_S_OK)
    {
        // Handle error
    }
}
```

Note that RPC uses 16-bit signed integers for representing enumerated types. (Use [V1-enum] for getting 32-bit enum.) However, most C compilers implement enum as their base type int. Int's value varies, depending on the operating system (16-bit or 32-bit, signed or unsigned). This leads to overhead in the client/server stubs, in which an enumerated parameter or enumerated members or structures/unions must be converted to NDR format and back. Further, the RPC run time may throw an exception if the value of the enumerated type is outside the range [-32768, 32767]. Thus, functions using enumerated types should be safeguarded by using error_status_t as the return value of the function.

4.4.2 Arrays

RPC provides support for common C data types and structures. This allows applications to use RPC functions as transparently as they do local functions. Now, think about the implications of sending and receiving an array of data through a remote procedure. To send an array of data, the client stub must extract the members of an array and put them in a linear data buffer in NDR format, which is then sent to the server. The server stub disassembles the array parts from the NDR-formatted data buffer, puts them in another array, and passes that array to a server function.

In RPC, multidimensional arrays can be used as parameters of an application-generated remote function. Both single and multidimensional base type arrays can be transported by using several RPC parameter attributes, each of which is provided to enhance efficiency. Just like C, the array parameter(s) in an RPC call can be fixed in size or dynamically determined at run time. A simple static array causes the stub to transmit or receive all the elements of

the array, depending on whether the parameter is marked [in], [out], or [in,out]. The function declaration below shows a client sending a static array of ten *long* numbers to the server:

```
    error_status_t StaticArrayFunction (
    [in] long lStaticArray [ 10 ]      // Number of elements transmitted
                                       // and the size of array is fixed
);
```

While the static arrays are simple, they are not very efficient if the data transmissions vary in length. To determine the size of the array dynamically, use **size_is()** attribute, as shown in the following example. Notice that for dynamic arrays to work, the size must be known to RPC and hence be a parameter of the RPC function. The size of a dynamic array is set by the client for an [in] dynamic array, and by the server for [out] dynamic array.

```
error_staus_t DynamicArrayFunction1   (
    [in] unsigned short usSize,        // This parameter defines the size
                                       // of array. Client sets the value
    [in, size_is(usSize)]
        long lConformantArray   []     // Conformant array
);
error_staus_t DynamicArrayFunction2   (
    [in, out] unsigned short *usSize,  // This parameter defines the size
                                       // of array. Server sets the value
    [out, size_is(*usSize)]
        long lConformantArray []
);
```

Further refinement is possible using the **length_is()** attribute, which allows the sending application to specify the amount of data that actually needs to be transmitted. This can be used in situations in which the size of the array is fixed and large (say, twenty), but the actual number of transmitted elements of the array will vary significantly (say, two to twenty). This helps improve efficiency, as fewer bytes are actually being transmitted over the network. As the following example shows, the client sets the size of the array dynamically and fills it up with data. However, the server's response may be shorter than the size of the array, so it sets the length of the array that defines the actual amount of data (*long* in this case) transmitted across the network.

```
error_staus_t DynamicArrayFunction2 (
    [in]   unsigned short usSize,   // This parameter defines the size
                                    // of array. Client sets it
    [out]  unsigned short *usLength, // The server sets this
    [in,   out, size_is(usSize), length_is(*usLength) ]
           long lConformantAndVaryingArray []
);
```

The attributes listed in Table 4-2 further enhance efficiency of the data transmission per call.

Table 4-2. Attributes That Enhance Data Transmission

Attribute	Description

first_is	Can be used to mark the first datum in the array starting from which transmission should start. For example:
	`void Function1( [in] long size,`
	`[in] small index,`
	`[in, size_is(size), first_is(index)] long Array[] );`

last_is	Can be used to mark the last datum in the array which should be transmitted. For example:
	`void Function2( [in] long size,`
	`[in] small index,`
	`[in, size_is(size), last_is(index)] long Array[] );`

min_is	Defines the lowest valid index of the array. One cannot use negative indices. For example:
	`void Function3( [in] long size,`
	`[in] small index,`
	`[in, size_is(size), min_is(0)] long Array[]);`

max_is	Defines the highest valid index for the dynamic array. For example:
	`void Function4( [in] long size,`
	`[in] small index,`
	`[in, last_is (index), max_is(size)] long Array[] );`

4.4.3 Strings

In a typical program, character strings are one of the most common types of data sent from one function to another, and RPC provides support for the transmission of such strings from the client to the server.

We are all familiar with the ASCII characters, or strings, that most programmers use for English-speaking users, for example:

```
char *pszAString [] = "Hello World"
```

In ASCII code, each character takes one byte (8 bits). However, with the growing need to write software that can be easily ported to work in diverse languages, locales, and countries, we are seeing unicode, the so-called wide characters that use two bytes (16 bits) to represent a character. The encoding scheme recommended by the Unicode Consortium™ makes it possible for all the languages of the world to be represented by double-byte characters. [Unicode 92]. Hence, a wide-character string is an array of wide characters that looks like this:

```
wchar_t *pwszWideString [] = L "Hello World"
```

RPC allows applications to send and receive both ASCII and Unicode characters.

RPC provides several attributes for transporting string parameters. The simplest string attribute is [string], which indicates that the parameter is a null-terminated character array. The length of the array is value returned plus one to account for a null terminator. The following example shows how to use a string attribute for both [in] and [out] parameters and also for char and wchar_t type base type parameters:

```
// Client sends a char based name to the server which converts the
// name to a wide-character string and returns it
//
error_status_t ExchangeNames(
    [in, string]
    char szClientName []        // Client sends ASCII chars to the server
                                // The length of the string sent
                                // = strlen(szClientName) + 1 char
    [in, out, string]
    wchar_t wszServerName[]     // The server returns a
                                // wide-character to client whose length
                                // = wcslen(wszServerName) + 1 wchar
);
```


As the example above shows, an [in] string is sent from the client to the server and the [out] string is returned by the server. The client stubs determine the length of the string for [in] strings, and the server stub determines the length of the string for [out] strings. When a string parameter is marked as [in,out], the server-sent string is returned in the buffer already allocated by the client. Thus, the returned string must not exceed the length of the original string. When a string parameter is marked [out] or [in,out], the calling application must allocate a buffer large enough to hold the data sent from the server. In addition, a client can allocate the memory and a server can simply fill in the memory when a parameter has [in,out,unique] attribute. (We discuss the unique attribute later.)

Another technique is sometimes used by client and servers to handle strings. Strings transport can be optimized by dynamically determining the size of the string array and length of the string itself, as the next example shows. In this example, an application-defined CountedString data structure is used to send and receive strings. The caller sets the size of the buffer, the length of the string, and the pointer to the string. This way, the RPC stub code does not need to compute string length and allocate buffers, which means improved application efficiency—provided the application itself uses an efficient method for detecting the string length.

```
typedef struct _tagCountedString{
    unsigned short ulLength;
    unsigned short ulMaxLength;
    [size_is(ulMaxLength), length_is(ulLength) ] wchar_t * string
} CountedString;
```

```
error_status_t RemoteFunction (
    [in]  CountedString ClientRequest,      // Client request transport is
                                            // now optimized
    [out] CountedString *ServerReply        // Server reply is efficient as
                                            // well
);
```

4.4.4 Structures

You can keep using C structures when you are using RPC. In fact, the string example above shows a C structure being used to transmit a counted string. Note that certain data structure members may need to have RPC attributes attached before they can be used with RPC. For example, in the code fragment above, the wide-character string array in the CountedString structure has the RPC [size_is] and [length_is()] attributes attached. The need for, and rules of, such attachments are generally the same as those for base data types, strings, and arrays.

Even before RPC came along, if you wanted to achieve the best efficiency in data packing in memory, you had to align the members of a structure so that the members began on certain architecture-dependent memory boundaries. For example, some machines require all data to begin on an even byte and others, notably RISC architecture machines, require the structure to begin on a double-word (DWORD). Furthermore, handling of misaligned data can cause system exceptions on RISC-based processors. The following example demonstrates how a badly aligned structure can be properly aligned by adding padding character(s) if the system requires even-byte alignment.[6]

```
typedef struct _badlyAligned {
    char     abc;                   // Starts at byte 0
    unsigned short def;             // Starts at byte 1 and ends on byte 2
} BadlyAligned;
typedef struct _WellAligned {
    char     abc;                   // Starts at byte 0
    unsigned short def;             // Starts at byte 1 and ends on byte 2
    char     padding;               // This causes structure to end on byte 3
} WellAligned;
```

Obviously, the data structures must be properly aligned for their use with RPC. I recommend defining the structures in such a way that all members are naturally aligned, causing the whole structure to be well aligned. This is illustrated by the following code, in which a string array is placed before smaller members of the data structure to ensure even-byte alignment.

```
typedef struct _NaturallyAligned {
    char     string [9];  // Large member.
                          // Start at byte 0 and end on 8
```

6. I assume the application uses no packing of bytes, that is, Zp1 switch. On a different packing/compiler, these "bad alignments" may be fixed by compiler automatically.

```
    char     padding1;     // Starts at byte 9
    unsigned short def;    // Starts at byte 10 and ends on byte 11
    char     abc;          // Starts at byte 12
    char     padding2;     // This causes structure to end on byte 13
} NaturallyAligned;
```

4.4.5 Pointers

We are discussing the data types that can be used with remote functions that are called by a client and serviced by the server. To make RPC functions appear the same as any normal C function, RPC provides support for most common data types (for example, boolean, char) and data structures (for example, arrays, strings). In each case, we present the basic usage and then proceed to point out ways to make data transfer efficient. Here, we illustrate the use of pointers in RPC functions. RPC provides three kinds of pointers: full pointers, reference pointers, and unique pointers. We discuss each type of pointer and note the situations where their use is most appropriate.

Pointers are necessary for most C programs. Therefore, RPC provides support for strongly typed pointers (for instance, void * is not supported). However, pointer support is difficult. Suppose a pointer is pointing to a string. In a self-contained program, a calling function can pass the pointer to the called function, which is free to examine the string and modify its contents, unless the string is declared as unmodifiable (for example, char const * pszString;). However, when the same pointer is sent from a client program to a server program running on a remote computer, the pointer will point to some undefined area. To avoid this, when you transport pointers, you should also transport the object that they point to. This means the object has to be well-defined. Also important: If the server modifies the object (string, in this case), then the server stub must transport back the modified object so that the client stub can modify the local object pointed to by the pointer. This is illustrated in Figure 4-5.

When a pointer parameter is marked [in] or [in, out], the client stub serializes the object pointed to (also called the "pointee") by the pointer into a transport buffer in an NDR format, so that the server can unravel the bits deterministically. Then the client stub, using RPC run-time support, sends the transport buffer to the server stub. This stub deserializes the object from the buffer, creates a copy of the original pointee on the server machine, and then passes a pointer to the server RPC function for this newly created object. The server application may modify its copy of the original object. If the parameter was marked [in], then the server stub does not return the changes made by the server application. However, for [in,out] parameters, the server stub sends back to the client stub. The changes returned by the server overwrite the original object, that is, changes are written over the client's existing storage. The original storage must therefore be large enough to hold the changes made.

The stubs must constantly allocate and free memory on behalf of the client or the server. To do this, they typically call the **MIDL_user_allocate()** and

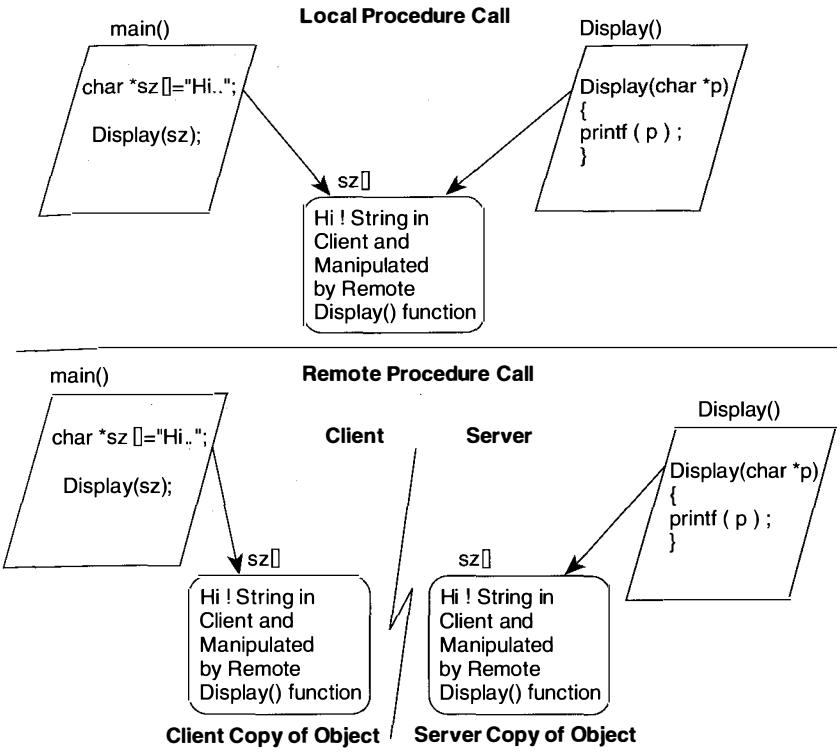


Figure 4-5. *Local Pointer Compared with RPC Pointer Manipulation*

MIDL_user_free() functions, respectively. These functions must be supplied by the RPC applications, which, in their simplest form, can call the C functions malloc() and free() to allocate or free memory. Often, developers implement these application-supplied functions as a check on memory leaks or plot memory usage trends.

For [in] pointer parameters, the server stub allocates memory to make a copy of the original object. Once the RPC call completes, the server stub deallocates the memory before returning the call to the client stub. For [out] or [in, out] pointer parameters, if the original pointer is null and the server application is returning a non-null object to the client, then the following sequence of operations takes place on the return path:

- The server stub copies the object into the transport buffer. (Remember that the server application has already allocated this object.)
- The server stub deallocates the object, allocated by the server application within the context of the RPC, by calling a user-specified deallocator, typically MIDL_user_free(). Note that the server application should use the same memory management as specified to MIDL.

- When it receives the transport buffer, the client stub allocates memory for the new object and passes a pointer to the client application. The client application must free this memory later by calling an appropriate deallocator, like MIDL_user_free().

If the server sets an [out] or [in, out] pointer parameter to null when it was non-null prior to the call, the client stub will reset the pointer to null, leaving it to the client application to free the original memory. Finally, if the server is expected to set a pointer to null, the client application should save a copy of the original pointer before making an RPC call.

To develop efficient client/server applications with minimum overhead, Microsoft RPC provides three pointer options—full pointers, unique pointers, and reference pointers.

Full Pointers

Full pointers are just like C pointers. When using full pointers, both the client and the server can change both the pointer and the contents to which it points. Furthermore, there can be multiple pointers or variables pointing to the same object. As the following example shows, transmitting a doubly linked structure by RPC requires full pointers, because each node in the list is pointed to by two pointers simultaneously.


```
// A structure that can be used for doubly linked list
typedef struct _tagDLINK {
    int          SomeValue;
    [ptr]struct  _tagDLINK *forward;
    [ptr]struct  _tagDLINK *backward;
}DLINK, *PDLINK;
// A function which allows the client to send a doubly linked list to
// the Server
void RemoteFunction (
    [in] DLINK DoubleLink,
    [out] unsigned short *usResult
);
```

While they are very flexible, full pointers cause the most expensive RPC operation, because they allow cyclical references. (Cyclical references are created when more than one pointer points to the same object.) Note that **[ptr]** is a full pointer attribute. In general, full pointers allow for data aliasing whereby different pointers can point to the same object; cyclical references may be thus created. Microsoft RPC supports this functionality. However, there is an overhead related to resolving aliasing. Therefore, if an application does not use aliasing, other pointer types should be used to set a better performance.

Unique Pointers

A pointer is defined as unique by using the **[unique]** attribute, or by redefining the default, using the directive **[pointer_default(unique)]** in the 1DL

file. Unique pointers are almost as flexible as full pointers, but are far less expensive. Unique pointers assume no data aliasing, that is, that only a unique pointer can point to a given object.

The code fragment below shows how we use unique pointers. We assume the client sends a null-terminated string (using the ClientRequest parameter); the length of the string can vary. The caller cannot anticipate the size of the buffer sent by the server, so it marks the server response buffer as a unique pointer and sets the server buffer (using the pszServerResponse parameter). This allows the caller to set the pszserverresponse to null. If no error occurs, the server returns a valid buffer (using pszServerResponse) which the caller must free.

```
long Transaction(
    [in] unsigned short usClientRequestBuffer,
    [in, string, size_is(usClientRequestBuffer)]
        char ClientRequest [],
    [out] unsigned short *usServerBufferLength,
    [out, unique, length_is(*usServerBufferLength)]
        char pszServerResponse[]
);
```

While the general use and memory allocation schemes remain the same as those for full pointers, unique pointers have the following special characteristics:

- Unique pointers can be NULL or non-null. This is very important for designing APIs that may take a number of optional parameters. The example above shows that the server is free to send either a NULL or a valid buffer.
- The value of unique pointers changes from NULL to non-null and vice versa. When the client sends NULL pointers, the server might return a non-null object. In this case, the client must deallocate the memory after use. Similarly, should the server set a non-null pointer to NULL, the client is responsible for freeing the resources.
- A unique pointer's pointee cannot be pointed to by another pointer or variable during an RPC call. Unique pointers cannot be used to send a doubly linked list from the client to the server.

Reference Pointers

Reference pointers, which are more efficient than unique pointers although less flexible, are defined by using the **[ref]** attribute. Reference pointers cannot be NULL. Although the pointee may change, an RPC call does not change the value of a reference pointer; the client must pass a valid pointer. Furthermore, the buffer pointed to by the reference pointer must be large enough to hold any changes sent by the server. With reference pointers, data aliasing is not supported, and using aliased data with reference pointers may result in unspecified values. The following code fragment shows the use of reference pointers.

```
// Remote call prototype
RemoteFunction (
    [in, string]
        char pszClientRequest[],          // By default, ref pointer
    [out, string]
        char *pszServerReply              // By default, ref pointer
);
void main () {
    CHAR *pszClientRequest = "Hello";
    CHAR szServerReply [MAX_REPLY_SIZE] ;
    // Reference pointer proper usage
    RemoteFunction ( pszClientRequest, pszServerReply );
    // Reference pointer improper usage
    RemoteFunction ( pszClientRequest, NULL );
}
```

For reference pointer parameters, the client stub can copy data from the client application object into a transport buffer before calling the server. On the successful completion of an RPC call, the client stub copies the results (change in object) into the existing buffer created by the client application. Similarly, the server stub is now involved only in creating the copy of the object on the server side, and returning any changes for [out] or [in, out] parameters.

Defaults for Pointers

You can specify the default attribute for all pointers except the top-level pointers that appear in parameter lists. You do this by using the **pointer-default** [ptr/ref/unique] attribute in the IDL file. This attribute affects (a) embedded pointers (pointers that appear in arrays, structures, and unions); (b) pointers returned by functions. The default is always overridden whenever a pointer attribute is supplied. In the following example, we show an excerpt of an IDL file for an application that wishes to set all pointers to be of unique pointer type for the given interface and then a function that expects a unique pointer back as a result.

```
//
// ACF File
//     Declare unique pointers by default for the interface
//     'NewInterface'
//
[uuid( <1234-4567-7890-12345678>,
    version ( 4.0),
    pointer_default(unique)]     interface NewInterface
{
}
//
// IDL File
//     - Selective overrides
//
interface NewInterface
{
}
```

```
typedef [unique, string] char *STRING;
error_status_t  GetResponse;
    [in] long *Bytes,                    // reference pointer by default
    [in, out] STRING RequestReply        // selective override from
                                         // reference. to unique
);
```

4.4.6 Nested Arrays

Nested arrays are the arrays as members of structure. When the size of the array changes at run time, it is called a conformant array. Microsoft RPC allows only one conformant array per structure. A nested array might look like this:

```
//
typedef struct _tagPerson {
    [string]
        char szPersonName [ MAX_NAME_LENGTH ]; // Fixed size
    [size_is(MAX_BITMAP_SIZE), length_is(length)]
        byte Picture [];      // Conformant array. Dynamic
                              // size and varying length
    unsigned long length;     // Size of Picture[]
} PERSON;
```

Designers should bear in mind that transmitting arrays and complicated structures is expensive. The utmost care is advised in designing with frequently used nested array structures. To minimize the array elements transmitted across the network, for large arrays, use length_is(). If the variation in the size of arrays is not significant, use static arrays instead of dynamic arrays, because the client/server stub overhead is significantly lower for fixed arrays than arrays of varying sizes.

4.4.7 Unions

C programs often use a union construct to define types capable of containing different kinds of variables in the same memory space. A union allocates the memory space needed by its largest member. Unions are usually encapsulated within a structure. When a union is used by itself, it is called a nonencapsulated union. When you simply examine a nonencapsulated union, you cannot tell which member is set. Thus, typically, a field in the encapsulating strucure is used as a discriminant. In the next C example, DiscriminatingPerson uses utype as a discriminant for the person union. Notice that person union is itself a nonencapsulated union with no discriminant.

```
// Union without a discriminant
typedef union {
    int        Number;
    char*    Name;
    byte*    PersonalPicture;
} Person;
```

```
typedef struct
{
    // Discriminant
    int      utype;
    Person   PersonDetail;
} DiscriminatingPerson;
```

Microsoft RPC supports discriminated unions with remote functions. It supplies two attributes, **[switch_is()]** and **[switch_type()]**, for use with the nonencapsulated union definition. The switch_type() defines the type of the discriminant (for example, switch_type(short), switch_type(long), and so on). The switch_is() attribute is used to point to the discriminant itself. RPC allows a union's discriminant to be declared as a parameter of an RPC call, or as a member of a structure of which the discriminated union is a member.

The following example shows a union's declaration that does not have a specific, self-contained discriminant. This example defines a discriminant, *small* (because it contains only three members). Next, each member of the union is explicitly associated with a type (for example, 0, 1, and so on). Assume that by default, the union contains a pointer to a bitmap containing someone's picture. Because the union itself does not contain a discriminant, an external discriminant must be used and passed in as a parameter to the remote function. Examine MyFunction(), shown below. Notice that MyFunction() takes the union as its first parameter and the external discriminant as its second parameter. Hence, the program that calls MyFunction() must supply a valid value for a discriminant.


```
//
// Declaring discriminated union to be compatible with RPC
//
typedef [switch_type(small)] union
{
    [case (0)]
        int                         Number;
    [case (1)]
        [string] char*              Name;
    [default]
        [size_is(MAX_BITMAP_SIZE)
        BYTE*                       PersonalPicture;
} Person;
//
// Declaring an RPC call which uses the above union as parameter.
// Notice use of an external discriminant.
error_status_t    MyFunction (
    [in, switch_is(Discriminant)]
        Person    SomeValue ,       // Union
    [in] small Discriminant         // Discriminant for the union
);
//
// Usage of the RPC call in an application
//
Person     Billy;
```

```
char        *Name = "Billy";
Billy.Name = Name;              // Put value in the union
//
// call the RPC function. '1' as second
// parameter informs RPC that union contains a name
//
RPCError  = MyFunction ( Billy ,  1);
```

Alternatively, you could create a data structure that contains both a union and the discriminant. Some designers favor this model, because the resulting data structure is self-contained, and function calls need not be aware that a union is being passed as a parameter. It also reduces programming errors. The following example illustrates such a data structure, (DiscriminatingPerson), which contains the union (Person) defined earlier, and a discriminant (Discriminant). Notice that MyFunction() only needs to take one parameter.

```
//Using a structure to contain the discriminant
typedef struct_tag{
    [switch_is(Discriminant)] Person SomeValue;
    small Discriminant;
} DiscriminatingPerson;
//
// Declaring a RPC call that uses the above union as parameter.
// Notice use of an external discriminant.
error_status_t   MyFunction (
    [in] DiscriminatingPerson   SomeValue ,          // Union
);
// Usage of the RPC call
error_status_t RPCError;
DiscriminatingPerson  Sally;
char*  Name = "Sally";
Sally.Discriminant    = 1;
Sally. SomeValue.name = Name;      // Put some value in the union
RPCError  = MyFunction ( Sally );
```

4.4.8 Using transmit_as

The **[transmit_as]** attribute is used for application-defined data structures and objects between the client and the server that cannot be transmitted by using any of the attributes discussed above. Some applications may use transmit_as to take over marshaling/unmarshaling of data from client/server stubs, because they may be able to marshal/unmarshal the data much more efficiently than can the generic client/server stubs.

The basic concept behind transmit_as is that RPC stubs ask the sender to convert the data (masked with transmit_as attribute) from an application defined data type (structure, linked list, C++ objects, and so on) to another data object that is marshaled by stubs. RPC then transmits that object to the recipient. On the receiving end, the receiver stub unmarshals the object of the intermediary type and then calls the application to convert the data from the object to the application-defined data type. Thus,

Table 4-3. Functions that Transmit DLINKS

Function Name	*Description*
DLINK_TYPE_to_xmit()	Called by the DLINK sender stub. Allows the application to marshal a doubly linked list into a byte array. The byte array is allocated by the application and is freed when RPC calls the application at DLINK_TYPE_free_xmit() function.
DLINK_TYPE_from_xmit()	Called by RPC to convert a byte array into a doubly linked list. The list is freed when RPC calls the application at DLINK_TYPE_free_inst() function.
DLINK_TYPE_free_inst()	RPC calls this to free the DLINK
DLINK_TYPE_free_xmit()	RPC calls this to free the byte array

there is an inherent inefficiency in transmit_as because an intermediate object is created and marshaled.

Table 4-3 shows the four functions called by RPC to transmit an application-defined data (DLINK). The functions must be supplied by the client.

The following code fragment shows how to use transmit_as to send a doubly linked list from the client to the server. The server can modify any part of the list, including adding new nodes or removing existing nodes. Notice that the fragment first defines a new data type (DLINK_TYPE), which is to be transmitted using the [transmit_as] attribute.

```
// A structure that can be used for doubly linked list
typedef struct _tagDLINK {
    int              SomeValue;
    struct _tagDLINK *forward;
    struct _tagDLINK *backward;
}DLINK;
typedef struct _DLINK_XMIT_TYPE{
    short &size;
    [size_is(&size)] short a&number[];
}DLINK_XMIT_TYPE;
// Transmit the DLINK as a DLINK_TYPE
typedef [transmit_as(DLINK_XMIT_TYPE)]PDLINK DLINK_TYPE;
//
// Functions which must be supplied by application to convert from
// DLINK to linear array of bytes and vice-versa. Application
// also needs to allocate and free memory
//
void _RPC_API DLINK_TYPE_to_xmit
(
    DLINK_TYPE                  *pDLink,      // Link to transmit
    DLINK_XMIT_TYPE __RPC_FAR*  ppByteArray  // as a Byte Array
);
void __RPC_API DLINK_TYPE_from_xmit
```

```
(
    DLINK_XMIT_TYPE __RPC_FAR *pByteArray,     // Byte array containing a
                                               // DLINK
    DLINK_TYPE __RPC_FAR      *pDLink          // Converted to a DLINK
);
void __RPC_API DLINK_TYPE_free_inst
(
    DLINK_TYPE __RPC_FAR *pDLink                // Free Linked List
);
void __RPC_API DLINK_TYPE_free_xmit
(
    DLINK_XMIT_TYPE __RPC_FAR *pByteArray       // Free byte array
);
// A function that allows the Client to send a doubly linked list to
// the Server
void RemoteFunction (
    [in, out] DLINK_TYPE DoubleLink,
    [out] unsigned short *usResult
);
```

The transmit_as attributes give applications flexibility in declaring RPC functions to accept application-defined data types. Used prudently, the attributes can lead to higher data marshaling/unmarshaling efficiency; however, it can also result in inefficient RPC when data transmitted back and forth is complex and data conversion routines are not efficient. Novice developers should not use the transmit_as attributes until they understand architecture-dependent (that is, Intel, RISC, and so on) data packing/data alignment issues and the impact of memory allocation/deallocation on overall process efficiency.

Let's summarize this section. RPC needs each data parameter of a remote function to be unambiguous. To this end, the developer may use special attributes (for example, [string]) to strongly type the parameters of remote functions. Furthermore, application designers need to take special heed of the costs of marshaling and unmarshaling data as it is transmitted from the client to the server and vice versa. With this in mind, we have provided specific guidelines to define efficient data structures to optimize data transfer between the client and the server. In this section, we introduced the basic data types supported by RPC. We discussed the special handling of arrays, strings, pointers, nested arrays, and union data structures in the context of RPC programming. And we also illustrated how application-defined data structures can be transmitted using the transmit_as attribute.

4.5 DESIGNING COMMUNICATION INFRASTRUCTURE

This section discusses the mechanics of connecting RPC clients to RPC servers. To establish a connection between client and server, you need to be familiar with new concepts, such as registering interfaces, using a name service, establishing endpoints, and establishing a connection with an object.

While RPC provides all the necessary plumbing for connection management and data transmission, you need to be aware of the various services offered by RPC so you can make a good selection among the communication services.

A logical connection between an RPC client and an RPC server results from the successful culmination of a client's search for a server. The client looks for specific characteristics, and servers display them. A successful match occurs when the services sought by the client are offered by the server. (See Figure 4-6.) Note that a server can offer more than one kind of service (for instance, database and document archiving services).

An RPC server is defined by these characteristics: network address, protocols supported, endpoint, interfaces, and objects contained within itself. While the server's network address, protocols supported, and endpoint characteristics allow it to be located by a client, the interfaces and objects—the supported characteristics of a server—define the services it offers.

The network address of the machine on which the server is running becomes the server's network address. Every server is identified by its network address, and multiple servers running on the same machine will have the same network address. If a server and a client reside on the same machine (that is, local RPC), a network address is immaterial.

The transport-specific protocols supported by a server define its protocol characteristics. A server can support a non-null subset of all of the protocols available on the server workstation. If a Windows NT workstation supports TCP/IP, Named Pipe, and IPX/SPX protocols, the RPC server running on the workstation can support all these protocols, or it may support only one or two.

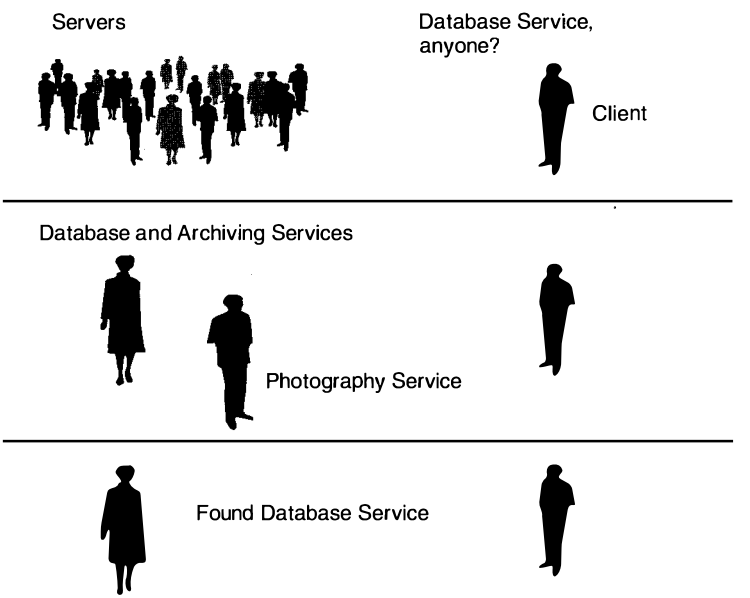


Figure 4-6. *Client Locating a Server by Characteristic*

Each server decides the protocols it will support. A server that does not support a given protocol (say, IPX) will not service clients who request support for that protocol,[7] even if rest of the client-requested characteristics match.

End points are a transport protocol-specific means of locating a server. For example, if the server supports the Named Pipe protocol, it might select \PIPE\MyRPCServer as the end point. This allows clients to connect to this server by looking for a server that supports the Named Pipe protocol and exports an endpoint \PIPE\MyRPCServer. A server can have fixed (well-known) endpoints (for example, \PIPE\MyRPCServer), or dynamic end points. RPC creates a dynamic end point for the server at run time; this dynamic endpoint allows servers and clients to connect without requesting a specific endpoint.

The interfaces exported by a server define its service characteristics. This is the reason for designing RPC servers, because interfaces allow an RPC-based server to be viewed as a repository of functions. Recall that the interfaces are delineated in the IDL file, and that each interface may contain one or more functions (or methods like a C++ class declaration). A client requests a specific interface from the server, and then calls the functions exposed by the interface. For example, a database server can export an ITransaction interface, which might contain functions such as **Get()**, **Put()**, **Commit()**, and so on. Each of these function calls will cause a remote call from the client to the server. Additionally, a server may support multiple related or unrelated interfaces, as Figure 4-7 shows. Recall that each interface is uniquely identified by the universally unique identifier defined in the IDL file and its version number.

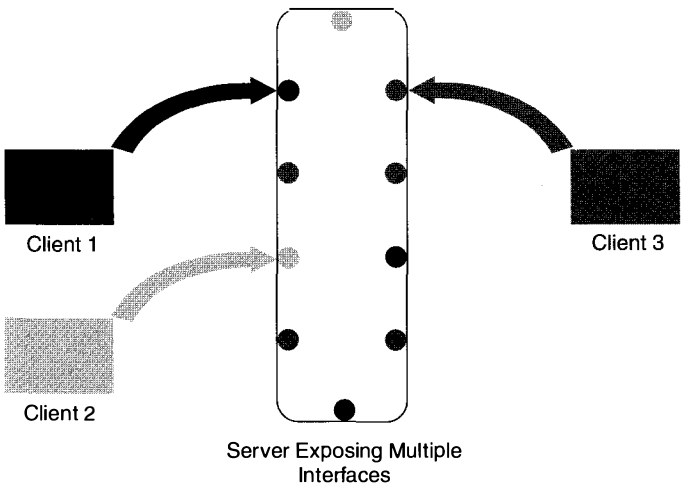


Figure 4-7. *RPC Server Containing Multiple Interfaces and Serving Multiple Clients*

7. In fact, if the client is running on a workstation that only supports a given protocol (for example, NetBEUI) that is not supported by the server, the client will not be able to contact the server at all.

As Figure 4-7 shows, an RPC server may hold many interfaces. It is also a repository for interesting objects. Thus, objects also define a server's characteristics. The server usually "publishes" a list of objects that it contains so that clients can request a specific interface of a specific object. Most simple RPC client/server programs don't have to use these features, in which case the server remains a collection of interfaces.[8]

An RPC server is also a container of one or more objects, which makes RPC immediately useful in designing object-oriented communication, as shown in Figure 4-8. For example, a file/print/fax server can be implemented as an RPC server that supports three different objects: a printer object, a file object, and a fax object. Each client can connect to one of these objects as needed.

A server can support a large number of objects, but the server might be slow to locate a specific object. When a server receives a client request for a specific object, the server must search through an "object table" to determine if it supports such an object. Once the object has been located, the server must then provide the services associated with that object. A server designer might have duplicate code to support each object, which will lead to a large, inefficient code base. To reduce the server's code size, RPC allows objects to have an associated object type. Using object type, similar objects can be collated and serviced by the same code base. Figure 4-9 illustrates the object type. In this example, an information server services questions about three disparate objects (plants, cars, and animals) by implementing three object types. All incoming client requests are channeled to the right objects, which themselves may handle diverse questions. For example, the Plants object contained within the server may handle questions about tulips as well as water lilies.

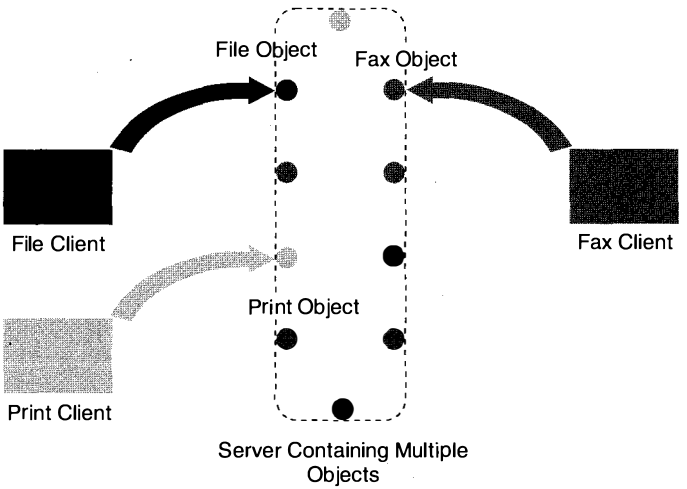


Figure 4-8. *Server Containing Multiple Objects*

8. RPC itself only understands interfaces. We can abstract "objects" around the interfaces.

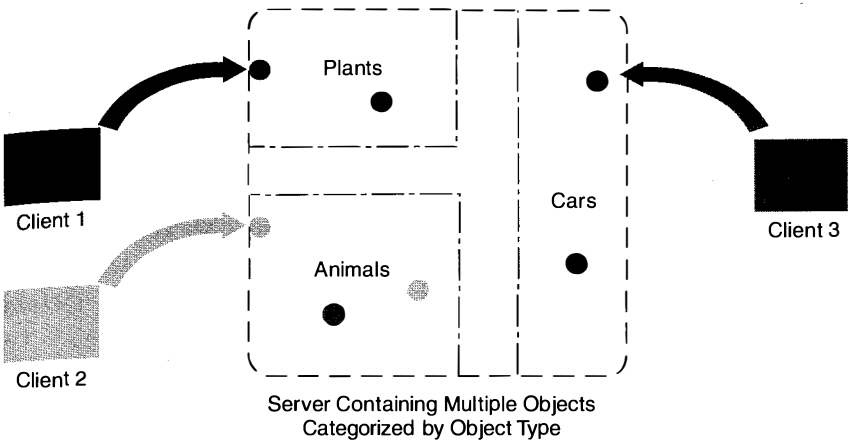


Figure 4-9. *Server Categorizing Contained Objects by Type*

4.5.1 Mechanics of Connecting Client and Server

Searching for a server is also called locating. The process starts with the client preparing a laundry list of attributes that it is looking for in a server. Next, either the client itself can start looking for the server, or it can use the services offered by the Windows NT Name Service, which includes the Windows NT Locator. The name service in Windows NT contains a database of server characteristics, which is queried by the Locator on behalf of the client. The Locator finds a match between the characteristics sought by the client and those published by the servers.[9]

The clients and their servers must agree on the method they will use to locate a server:

- *Client Locating Server Directly.* In this method, the client must provide the RPC run time with sufficient detail about the server. Because there is no container or repository of server characteristics in this model, the client must either know all the details beforehand (that is, hard-coded in the client application), or else it must prompt the user for details, such as the server address.
- *Using Name Service.* In this method, the server exports into a name service the interfaces and objects it supports. The name service records the server's address, protocols supported, and any well-known end points. Thus, the name service offers a flat database to the servers wherein each server entry is a tuple:

```
{Resource Name, Location Details}
```

- The resource name, also referred to as the entry name, identifies the server or its services. The location details—network address, protocols

9. Note that a server may listen and service incoming calls on an entry point that is not public but is otherwise known to the client.

supported, and end point—associate a logical (entry) name with the server's interfaces (also known as services). This allows one or more clients to locate a server by querying the name service for the server's logical name. After the server entry is located, RPC run time uses the server characteristics (for example, server address) to establish a connection between the client and the server.

RPC client/server designers should use the name service in a distributed computing environment, because it makes locating the server easier. Using the name service is very helpful in managing a distributed system, as it allows handling the following changes without affecting the clients (that is, if the client is flexible enough):

- The server address can be changed easily. This usually needs to be done if a computer crashes irrecoverably, in which case, the server application can be installed on a new computer system. When the server starts, it puts its entries in the name service database, which now includes its new network address. Thus, new clients can access the server without being aware of the change in its location. Also, if the client has a rebinding capability, it can find another server on the fly after the first server dies.
- The protocols that the server supports are easy to change or modify. This gives system administrators some flexibility to modify the protocols to suit changing user needs. For example, consider a server that may be used by only those clients who support TCP/IP protocol. To support new clients who use the IPX protocol, the server need only add IPX protocol support to its existing TCP/IP protocol support. The server then publishes its new capability through the name service database so that other IPX-based clients can use its services.
- The interfaces supported by the server can be upgraded or modified easily.
- The endpoint details can be changed without bothering users.

However, sometimes it is necessary or convenient not to use the name service:

- The application developer learning about RPC or rapid prototyping may not use Name Service. In either case, to allow simple applications to be quickly designed and tested, it is best to avoid name service during the early development phases. Once the application has stabilized, it can easily be modified to use the name service.
- For many reasons, including security, the server might not wish to publicize its services. In this case, it is not appropriate to publish its interfaces into the name service.

Client Not Using Name Service

In a simple client/server application, the user supplies the network address and other server-specific details to the client application. When it has the server address information, the client can establish a connection with the server. First, the client creates a string containing the server characteristics, such as the server's address, the interface requested, and so on, and then it calls an RPC function to locate the server.

The client composes the string by using the **RPCStringBindingCompose()** API, which allocates a buffer large enough to hold all the supplied values and create a well-formatted string describing the desired binding. As the function prototype below shows, the client supplies the network address, the endpoint information, the protocols it expects the server to support, and any related object ID (that is, the RPC interface ID). All the parameters except StringBinding are optional, so the client can supply only those parameters that it is aware of. For example, the client may not supply an endpoint, if it knows that the server supports dynamic endpoints.

```
RPC_STATUS RPC_ENTRY
RpcStringBindingCompose(
    unsigned char *     ObjUuid,            // Optional. Object id.
    unsigned char *     ProtSeq,            // Protocols expected to be
                                            // supported by the server
    unsigned char *     NetworkAddr,        // Network Address of server
    unsigned char *     EndPoint,           // Endpoint exported by server
    unsigned char *     Options,            // Any other options
    unsigned char *     StringBinding);     // Composed string is returned
                                            // here which must be freed
```

Once the string has been successfully composed, the client calls the **RpcBindingFromStringBinding()** API. The prototype is shown here.

```
RPC_STATUS RPC_ENTRY
RpcBindingFromStringBinding(
    unsigned char *         StringBinding,  // Created earlier by
                                            // RpcStringBindingCompose()
    RPC_BINDING_HANDLE *    Binding);       // On successful binding,
                                            // a valid handle is
                                            // returned in "Binding"
```

When the client has supplied all the parameters, RPC runtime connects the client with a server that matches all the requested server attributes. The function completes successfully when an appropriate server is located and a logical connection has been established between the client and the server.

The function returns a binding handle to the client. The binding handle is an opaque handle that the client uses for subsequent calls to the server. In the following example, on successful completion of both APIs, the client has found a server on a specific computer (`"\\Disneyland"`)

supporting a Named Pipe protocol ("ncacn_np") with a well-known endpoint ("\PIPE\DisneyLand"). In this example, the client requests the DisneyTour interface from the server by passing in the interface handle (DisneyTour_IfHandle) to the RpcBindingFromStringBinding() API.

```
//
// Interface Definition Language (IDL) File
//
[uuid ( <some uuid generated by uuidgen utility>)]
interface DisneyTour
{
    void MagicMountainTour();
    void MickeyMouseSighting();
    ....
};
//
// Application Configuration File (ACF)
//
[implicit_handle ( handle_t DisneyTour_IfHandle)]
interface DisneyTour
{
}
//
//     Client code showing client-initiated binding to server
//
unsigned char          *pszString = NULL;
// Create a binding string
RPC_STATUS status = RpcStringBindingCompose ( NULL,
                        "ncacn_np",           // Support named Pipe Protocol
                        "\\\\Disneyland",     // server address
                        "\\pipe\\Disneyland", // well-known end point
                         NULL,                // no options
                         &pszStringBinding);
// Bind to the Server
status = RpcBindingFromStringBinding( pszStringBinding,
                                      &DisneyTour_IfHandle);
// Now, we can get a tour of magic mountain
MagicMountainTour();
.....
// Let us disconnect from the server
status = RpcBindingFree( DisneyTour_IfHandle );
// Also, don't forget to release the binding string
status = RpcStringFree ( &pszStringBinding );
```

The example introduces two new APIs, **RpcBindingFree()** and **RpcStringFree()**. The former is called to disconnect from the server, and the latter to release the memory associated with the binding string created by RpcStringBindingCompose().

Server Not Using Name Service

The RPC server that does not use a name service usually performs three distinct duties:

- It must register its characteristics with the RPC run time. These characteristics include interfaces exposed, protocols supported, endpoints, and, optionally, the object contained by the server.
- It must manage client connections. RPC run time provides many services that the server can use to manage clients. For example, using the RPC-provided supports, a server can allow multiple clients to connect at the same time.
- It must provide the implementation of functions contained in the interfaces exported by the server.

Except for the data transport issues, providing implementation of an RPC function is no different from doing any regular function.

The code on the next page shows a simple server that services the client described in the section immediately above. First, the server must give the RPC run time information (on the server side) on the protocol supported and endpoint by calling the **RpcServerUseProtseqEp()** API. This API can be called multiple times when supporting multiple protocols.

```
RpcServerUseProtseqEp(
    unsigned char *    Protseq,          // Protocol supported e.g. named pipe
    unsigned int       MaxCalls,         // Maximum number of concurrent calls
                                         // that Server expects to receive
                                         // from this protocol. Each protocol
                                         // may have its own limit
    unsigned char *    Endpoint,         // Protocol specific end point e.g
                                         // say \pipe\RpcServer for named pipe
    void *             SecurityDescriptor); // Additional security
                                         // restrictions supported on NT
```

On successful completion of the RpcServerUseProtseqEp() API, one or more clients can connect to the server if they specify the server characteristics. However, the server is not yet ready. It must now register with the (server-side) run time support any interfaces that it plans to support. It does this by calling the **RPCServerRegisterIf() API**. The server can optionally register the object that it supports.

```
RpcServerRegisterIf(
    RPC_IF_HANDLE          IfSpec,       // Interface Handle defined in
                                         // ACF file.
    UUID                   MgrTypeUuid,  // Optional. Object type
                                         // should the Server support
                                         // objects of multiple types.
    RPC_MGR_EPV *          MgrEpv);      // Optional. Manager Entry
                                         // Point Vector should
                                         // the Server provide
                                         // multiple implementation
                                         // of same interface.
```

The server must call RpcServerRegisterIf() for every interface that it plans to support. When the interfaces are successfully registered, the server stub prepares to accept client requests for a specific interface. At this point, the server has to let the clients connect to it and execute its remote procedures. The server does this by calling the RpcServerListen() API, which enables the RPC server to "listen" to client calls. On successful completion of the RpcServerListen() API, the client requests are accepted and serviced.

```
RpcServerListen(
    unsigned int     MinimumCallThreads,     // Minimum nos of threads
                                             // started by RPC to Serve
                                             // RPC Clients
    unsigned int     MaxCalls,               // Maximum nos of clients
                                             // who can be concurrently
                                             // be serviced.
    unsigned int     DontWait);              // If set to 0, the
                                             // function blocks until
                                             // RpcServerStopListening
                                             // has been called
```

Each incoming client request causes the RPC run time to start a new thread, which executes the server-supplied remote procedure for the client and returns results to the client. The server application can control the minimum and maximum number of such threads started by RPC (by using the MinimumCallThreads and MaxCalls parameters). Note that the maximum number of incoming client requests can be limited by the capabilities of the underlying transport protocol. The call to the RpcServerListen() API blocks, if DontWait is set to zero, until the **RpcServerStopListening()** API has been called by another server thread. When DontWait is set to non-zero prior to the call, the RpcServerListen() API completes immediately; in this case, the server can call the **RpcMgmtWaitServerListen()** API later.

The following code fragment shows the DisneyLand server corresponding to the client from the preceding section. When a remote client binds to the DisneyTour interface and executes the function MagicMountainTour() and MickeyMouseSighting() on the client machine, the remote procedures are executed on the server (which prints strings on the server console).

```
//
//  A simple Server which declares named pipe as means of transport
//  with "\pipe\Disneyland" as well-known end-point
//
int main ()
{
    // Server informs RPC run-time the protocol and end-point
    // supported
    RPC_STATUS status = RpcServerUseProtoseqEp(
                        "ncacn_np",
                        1,
                        "\\pipe\\Disneyland",
                        NULL);
    // Register the interface
```

```
    status = RpcServerRegisterIf(
                                DisneyTour_IfHandle,
                                0,          // No objects
                                0);         // Use default manager
    // Begin servicing the clients by showing them
    // sights and sounds of Disney Land
    status = RpcServerListen (
                1,                          // Minimum 1 thread (Client)
                                            // else Disney is in loss!
                40,                         // Maximum 40 people on a ride
                0);                         // Block
    // Only when RpcServerStopListening() has been called or
    // an error has occurred, will the program control come here
    return (0);
}
//
// Remote procedures executed by the Server on requests from Client
//
void MagicMountainTour()
{
    printf("Welcome to magic mountain.\n");
    .....
}
void MickeyMouseSighting()
{
    printf("Heeeere is Mickey.\n").
    ....
}
```

In the example above, the server never stops, because it is blocked indefinitely at the RpcServerListen() call.[10] The server must be gracefully shut down. This is accomplished either by having a function that says CloseDisneyTour() as part of the interface that a client can call to stop the server, or by having the server start a thread, prior to calling RpcServerListen(), that accepts administrator commands on the server console.

```
//
// Modified IDL file which allows client (probably an admin client)
// to call server to shut itself down
//
[uuid ( <some uuid generated by uuidgen utility>)]
interface DisneyTour
{
    void MagicMountainTour();
    void MickeyMouseSighting();
    ....
    void CloseDisneyTour( [in, string] CHAR * pszClientName );
}
//
// Client code needs to call CloseDisneyTour() to close the server
//
int main ()
```

10. Or an RpcMgmtWaitServerListen() API, should the server call it after calling a nonblocking version of RpcServerListen() call.

```
{
    <...>
    // Assume the client is bound to the server already
    CloseDisneyTour ("Walt Disney");
}
//
// Following routine needs to be added to the server code to
// service the shut down procedure
//
void CloseDisneyTour( CHAR * pszClientName )
{
    // Verify the authority of client
    if (!stricmp(pszClientName , "Walt Disney"))
    {
        // Stop accepting any more client requests
        RPC_STATUS status;
        status = RpcMgmtStopServerListening(NULL);
        // Further, server must unregister its interface
        // so that new client will not find the server
        // during server location
        status = RpcServerUnregisterIf(
            NULL, // Unregister all the interfaces for this server
            NULL, // Unregister default EPVs
            FALSE // Unregister immediately without
                  // waiting for any calls in progress
                  // to complete
                );
    }
}
```


Well-Known Endpoint Compared with Dynamic Endpoint

After the server selects a particular protocol, it must designate its protocol-specific address. For example, when the server selects the Named Pipe protocol, it can specify a well-known endpoint, such as

```
"\PIPE\MyPipe"
```

This section illustrates the use of a well-known endpoint and then presents the use of dynamic endpoints.

When a server application specifies its endpoints for each of the protocols it supports, these endpoints are called "well-known." The endpoints can be specified in code as in the server example in the preceding section, or they can be specified in the IDL file using the **endpoint** keyword in the IDL file, as shown in the following example. The example specifies two protocols (Named Pipe and IP) and the endpoints for each, using the endpoint keyword in the IDL file. Once the endpoints have been defined in the IDL, the server can export its well-known endpoints by either of two means: it can call the **RpcServerUseProtseqIf()** API for every protocol supported, or it can call the **RpcServerUseAllProtseqIf()** API to register all the protocols, endpoints for each protocol, and interfaces. This code fragment shows the conditional use of both APIs.

```
[uuid  ( <some uuid generated by uuidgen utility>),
// Defining an endpoint for named pipe protocol and another for IP
endpoint("ncacn_np:[\pipe\\Disneyland]", "ncacn_ip_tcp:[234]")]
interface DisneyTour
{
    void MagicMountainTour();
    void MickeyMouseSighting();
    ....
    void CloseDisneyTour( [in, string] CHAR * pszClientName );
}
//
// Server using endpoint and protocol defined in  IDL file
//
int main() {
    RPC_STATUS status;
#ifdef REGISTER_FOR_EACH_PROTOCOL
    // Use a single API to register interface, protocol supported,
    // and well-known (picked from IDL) endpoint file
    // Register named pipe protocol
    status = RpcServerUseProtseqIf(
                "ncacn_np",          // protocol
                1,                   // max concurrent calls
                DisneyTour_IfHandle,// Interface
                NULL);               // security
    // Register IP support
    status = RpcServerUseProtseqIf(
                "ncacn_ip_tcp",
                1,
                DisneyTour_IfHandle,
                NULL);
#else
    // Because we already showed the protocol supported in IDL,
    // we can simply call one API to register support for
    // both protocols.
    status =  RpcServerUseAllProtseqsIf(
        1,                   // MaxCalls,
        DisneyTour_IfHandle, // Interface
        NULL                 //   security
        );
#endif
    // Begin servicing the Clients by showing them
    // sights and sounds of Disney Land
    status = RpcServerListen (
                              1,          // Minimum 1 thread (Client)
                                          // else Disney is in loss
                              40,         // Maximum 40 people on a ride
                              0);         // Block
    // Only when RpcServerStopListening() has been called or
    // an error has occurred will the program control comes here
    return (0);
}
```

Dynamic endpoints are an alternative to well-known endpoints. Dynamic endpoints are created by RPC run time for the server application. Although it may seem convenient to specify a well-known endpoint (for example, `endpoint("ncacn_nb_nb:[MyServer]")`) in the IDL file, well-known

endpoints have their limitations. While it is certainly easy to debug an application that uses well-known endpoints and thus recommend for rapid prototyping of client/server systems, endpoints are limited resources for some transports. This can be critical when errant servers do not release the endpoints to the system upon termination. Additionally, the chances of conflict between endpoints selected by distinct servers are high (for example, two applications using `"\PIPE\MyPipe"` as endpoints). For this reason, RPC makes it possible to create dynamic endpoints for the server. If all the servers use the RPC dynamic endpoint services, there is no conflict and the endpoints are better utilized. Additionally, RPC automatically cleans up the dynamic endpoint when the server terminates, thus reclaiming scarce resources.

To create dynamic endpoints, a server first calls the **RpcServerUseProtseq()** or the **RpcServerUseAllProtseq()** APIs. The prototypes of these functions are shown here.

```
//
// RpcServerUseProtseq() is called once per protocol sequence supported
// but notice that no endpoint information is passed here.
//
RPC_STATUS RpcServerUseProtseq(
    unsigned char   *                   ProtSeq,
    unsigned int                        MaxCalls,
    void   *                            SecurityDescriptor);
//
// RpcServerUseProtseq() is called once only to add support for all
// protocols but notice that no endpoint information is passed here.
//
RPC_STATUS RpcServerUseAllProtseq(
    unsigned int                         MaxCalls,
    void   *                             SecurityDescriptor);
```

Once a dynamic endpoint has been created, the server can call the **RpcServerInqBindings()** API to determine the current binding information. The binding information includes the server-supported protocols and its dynamic endpoints. The binding vector returned from RpcServerInqBindings() is passed on to the **RpcEpRegister()** API to register a dynamic endpoint.

```
typedef struct _RPC_BINDING_VECTOR {
    unsigned long Count;
    RPC_BINDING_HANDLE BindingH[1];
} RPC_BINDING_VECTOR;
RPC_STATUS RpcServerInqBindings(
    RPC_BINDING_VECTOR  *  *     BindingVector);
RPC_STATUS RpcEpRegister(
    RPC_IF_HANDL                IfSpec,         // Interface to be registered
    RPC_BINDING_VECTOR *        BindingVector,  // Containing binding info
                                                // such as protocol seq
    UUID_VECTOR   *             UuidVector,     // Object vector, optional
    unsigned char   *           Annotation);    // Some annotation
```

```
// can be put here although
// it is not used by RPC
// for Client/Server
// binding
```

The dynamic endpoints are created for the servers by the local RPC runtime. All the registered endpoints are maintained in a nonpersistent local database called the endpoint map and accessed by the endpoint mapper service. The endpoint mapper service maintains a record of the endpoints in its database, with each entry in the tuple form:

```
{Interface UUID, Interface version, Object UUID, Protocol Sequence,
                                    Binding Handle}
```

The endpoint information is considered more dynamic and volatile than server entries stored in a name service. While name service entries persist until interfaces are unregistered or the system is rebooted, endpoint map information is changed every time the server starts or stops. Good programming etiquette dictates that before the server terminates, it should call the **RpcEpUnregister()** API to unregister the endpoints and clear the endpoint database. Furthermore, the server should free the binding vector returning by the RpcServerInqBindings() API by calling the **RpcServerBindingFree()** API. Designers should use structured exception handling mechanisms to guarantee that endpoints are always cleared.

A Server That Supports Multiple Protocols

A server can be designed to support any (and all) protocols available on a given system. The server can be unaware and independent of the underlying transports and their protocols, and yet be able to service clients. To do this, the server may call the **RpcServerUseAllProtseqs()** API. Alternatively, a server can be selective and register support only for specific protocols. For example, a server can decide to service only those clients that use IPX/SPX transports. In such a case, the server only needs to support one protocol. Sometimes, the decision to support a single or multiple protocols is based on the performance cost of supporting all the protocols. When a server registers its interface(s) with the RPC run time, the run time may create a thread for each protocol that the server supports. Each thread awaits client requests on a specific protocol. Thus, the server incurs an overhead of at least one RPC thread per protocol supported.

A server can register support for a set of protocols by calling one of the following RPC APIs: **RPCServerUseProtSeq()**, **RpcServerUseProtSeqEp()**, or **RpcServerUseProtUseSeqIf()**. These RPC functions are called once for each protocol supported by the server. (The prototype of these functions is shown in the section above.) Depending on the application-specific details, a designer will choose one API over another. For example, an application can

call RpcServerUseProtUseSeqIf() to register an interface, a protocol, and its end point, all in a single call. Other APIs give varying degrees of flexibility.

Table 4-4 shows the syntax for most common protocols supported by RPC, and their corresponding endpoints. The third column shows examples when the protocol and endpoints are defined in an IDL file.

Table 4-4. The Protocols Supported by RPC and Their Syntax

Address Type	*Endpoint Examples*	*Comments*
Protocol Sequence String: `ncacn_np`		
Named Pipe	`"ncacn_np:\\\\MyServer[\\pipe\\MyPipeName]"`	Reserved key word.
Protocol Sequence String: `ncacn_ip_tcp`		
TCP/IP	`"ncacn_ip_tcp:MyServer[2004]"`	Port number is optional; can be a 16-bit number; values less than 1024 are reserved.
Protocol Sequence String: `ncacn_nb_nb`		
NetBIOS on NetBeui	`"ncacn_nb_nb:[234]"`	Port number is optional; can be any number between 0 and 255; values less than 0x20 are reserved.
Protocol Sequence String: `ncacn_nb_tcp`		
TCP over NetBIOS	`"ncancn_nb_tcp:[234]"`	Port number is optional; can be any number between 0 and 255; values less than 0x20 are reserved.
Protocol Sequence String: `ncacn_dnet_nsp`		
DECnet	`"ncacn_dnet_nsp:MyServer[#234]"`	DECnet object name or number; can be up to 16 characters. May be prefixed with #.
Protocol Sequence String: `ncalrpc`		
Local RPC	`"ncalrpc:[MyProcessName]"`	A legal Windows NT object name without the backslash (\) character.

A server determines the protocols supported on a computer by calling the **RpcNetworkInqProtoseqs()** API. This function returns the protocols supported by both RPC run time and the operating system. This API can only be called on the server. Clients can call the **RpcNetworkIsProtseqValid()** API to verify that a requested protocol is supported by both RPC run time and the operating system.

A Server That Supports Multiple Objects

A server can provide multiple implementations of the same interface. This is usually the case when the server is supporting different objects. Although multiple objects may have the same interface (that is, the same functions with the same signatures), their behavior can be different enough to need separate implementations. For example, imagine a mail server that implements normal mail, urgent mail, and slow mail services as separate objects, with each object exposing the same interface MailExchange, as shown in the IDL file below. In this example, the client requests the MailExchange interface from the Urgent Mail Object, implying that the client needs to send an urgent message.

```
[uuid <uuid created by uuidgen>,
 version 1]
interface MailExchange
{
    // UUIDs for objects Supported by server
    // Values filled in once by running uuidgen utility
    const UUID *URGENT_MAIL_OBJECT = [... ];
    const UUID *NORMAL_MAIL_OBJECT = [... ];
    const UUID *SLOW_MAIL_OBJECT   = [... ];
    // Each object supports the following methods
    RPC_STATUS Send (...);
    RPC_STATUS Receive(...);
}
//
// Client requesting the above interface from a specific object
//
int main()
{
    // Create a binding string  for sending Urgent mail
    RPC_STATUS status;
    status  = RpcStringBindingCompose (
        URGENT_MAIL_OBJECT_UUID,    // Urgent Mail Object uuid
        "ncacn_np",                 // Support named pipe Protocol
        "\\\\Mailserver",           // Server address
        "\\pipe\\MailServer",       // well-known endpoint
         NULL,                      // no options
         &pszStringBinding);
    // Bind to the server
    status = RpcBindingFromStringBinding(
                                        pszStringBinding,
                                        &MailExchange_IfHandle);
    // Now, we can send/receive mail
    status = Send(...);
}
```

As the example shows, the client requesting service from an object contained in the server specifies the object UUID as well as the interface ID. The client code is relatively simple. However, the server's implementation of multiple object support is slightly more complicated. To understand how multiple object support works, you must first understand how a client request arrives at the correct server function.

RPC provides a default dispatch table, also called the **manager entry point vector (EPV)**, for each interface declared in the IDL file. An EPV is an array of function pointers in which each pointer points to a function (or method) supported by the server. If a server provides multiple implementations of the same interface, it must register an EPV for each interface implementation. Thus, if multiple objects supported by the server expose the set of interfaces, the server will have one EPV per object. However, the overhead associated with providing separate EPVs can be very high if the server supports many objects. In such cases, the designer should segregate the objects by an object type or category.

When the objects are categorized by object types, the server only needs to register one EPV per object type. In this design, an RPC server designer can collate similar objects and provide only one EPV for each type. For example, suppose a server supports four types of objects: oranges, apples, TVs, and microwaves. In this case, it may provide only two EPVs, one for the appliance object type and another for fruit object type.

A server uses the RpcServerRegisterIf() API to register an EPV for each interface and object type. The server also needs to associate objects with each object type by calling the **RpcObjectSetType()** function.

```
// Registering a non-default Manager EPV for a given object type
RPC_STAUS RpcServerRegisterIf (
    RPC_IF_HANDLE interface,
    UUID *      MgrTypeUuid,  // Object Type (UUID)
    RPC_MGR_EPV * MgrEpv      // Manager EPV for this object type
);
// Associating an object (UUID) with a given object type (UUID
RPC_STATUS RpcSetObjectType (
    UUID *      ObjUuid,      // Object
    UUID *      ObjType,      // and its type
    );
```

RPC provides a default manager EPV that supports the nil object type. Thus, when a server does not support multiple objects or multiple object types, the default manager EPV will dispatch the incoming client request to the default (and only) implementation of the interface. Even if an application supports only one interface implementation, it may provide its own manager EPV. It does this by supplying NULL as MgrTypeUuid, and an EPV address in MgrEpv when calling RpcServerRegisterIf(). Table 4-5 shows how the server-side RPC run time decides which EPV (and hence which implementation) to call when a client request arrives at the server end.

Table 4-5. Relationships among Client-Requested Object, Object Type, and Manager EPV

Object UUID requested by client	*Server set type for object UUID*	*Server registered type for manager EPV*	*Dispatching action*
Nil	N/A	Yes	Uses manager with nil type UUID.
Nil	N/A	No	Error. RPC call rejected.
Non-nil	Yes	Yes	Use manager with same type UUID.
Non-nil	No	Ignored	Uses manager with nil type UUID (the default manager). If no nil type UUID, reject RPC call.
Non-nil	Yes	No	Error. RPC call rejected.

Now we return to the implementation of the mail server that supports three objects, Urgent Mail, Normal Mail, and Slow Mail. Assume that you want to provide one implementation of the MailExchange interface when the client requests normal or slow mail service, but you want to provide a separate implementation for servicing urgent mail requests. Because normal or slow mail requests are the norm, you associate the default EPV with Normal Mail and Slow Mail object types, and you need to associate a new EPV with the Urgent Mail object type. Based on Table 4-5, you need only set the object type and EPV for the Urgent Mail object. Because you have not specified any EPV for them, the default manager EPV will be called automatically for the Normal and Slow Mail objects. The following code fragment shows the implementation. Note that MailExchange interface has two methods, Send() and Receive(). Thus, EPV for urgent mail should specify the functions to support these methods.

```
// Urgent mail type. Details filled by running uuidgen utility by hand
const UUID * URGENT_MAIL_OBJECT_TYPE = { ..} ; // A uuid
int main()
{
    // Declaring 2nd Manager EPV and setting up the new entry
    // points for MailExchange interface.
    // ("xx_SERVER_EPV" declarations for an interface "xx" are
    // found in the rpc header file generated by MIDL compiler)
    MailExchange_SERVER_EPV UrgentMailEpv;
    UrgentMailEpv.Send = UrgentSend;
    UrgentMailEpv.Receive = UrgentReceive;
    // Declare protocols and endpoints supported
    .....
```

```
    // Declare to use default manager EPV by default. In this
    // case, it will handle normal and slow mail objects.
    status = RpcServerRegisterIf( MailExchange_IfHandle, NULL, NULL);
    // Set the type for urgent mail.
    status = RpcObjectSetType (
        URGENT_MAIL_OBJECT,
        URGENT_MAIL_OBJECT_TYPE
        );
    // Now, register again for handling urgent mail
    status = RpcServerIf(
        MailExchange_IfHandle,
        URGENT_MAIL_OBJECT_TYPE,
        (RPC_MGR_EPV *)&UrgentMailEpv);
    // Ready to accept calls
    status = RpcServerListen( 1, 20, FALSE );
}
// Default methods called for "normal" and "slow" mail
RPC_STATUS Send( ... )
{
    ....
}
// Method called for urgent mail
RPC_STATUS UrgentSend(...)
{
    .....
}
```


Once you understand the manager EPV concepts, you can generate code, similar to the code above, for handling multiple objects. This puts the basic infrastructure in place. After that, you need to fill in the actual functions (or methods) supported by the multiple objects. For example, suppose you want the mail server to handle **facsimiles (FAX)**, and that FAX has higher priority than normal or slow mail. Then, for the example above, you would call the RpcSetObjectType() API one more time to associate the FAX object with the UrgentMail object type. Then, the urgent mail manager EPV will automatically field the RPC requests from the client that wants to send or receive a FAX.

4.5.2 Using Name Service

In a distributed environment, locating a server can be a complex problem. In the Windows NT environment, RPC provides Name Service to solve this problem. Name Service allows clients to determine a server location by specifying a logical name.

Locating a resource—be it a file, a printer, or a service—is a simple task in a centralized system in which you can devise a means of maintaining a global table with each entry of the form {Resource name, Location}. (The central authority sets the rules for syntax and legal values for the location of a resource.) But in the current state of PC-based computing, the problem of locating a resource is immensely complicated, due to absence of any centralized control. This becomes a problem when desktop applications are sent into distributed

applications. Thus, if a company wants to design a client/server application, it must devise some means for the clients to locate the servers. The simplest approach is to ask the user for the network address of the server. However, should another company decide to design a client/server system for their PCs, they will design some other mechanism for connecting the client and the server. This means there are many different mechanisms for locating resources, and therefore great possibilities for confusion.

To reduce the confusion, Windows NT collects groups of machines into domains. Every domain can have one or more PCs and one **domain controller (DC)**, with the administrative authority over all the PCs in its domain. Note that a domain is a logical, not a physical, collection of machines. For example, a machine in Tokyo, Japan, can belong to a domain that contains machines resident in Redmond, Washington, USA.[11] Further, a company can devise a hierarchy of domains to make administration of each domain easier. Figure 4-10 illustrates this.

In each domain, you assume the DC has the most authoritative knowledge of the entire system. For load sharing and fault tolerance, system administrators set up one or more **backup domain controllers (BDCs)**. Windows NT

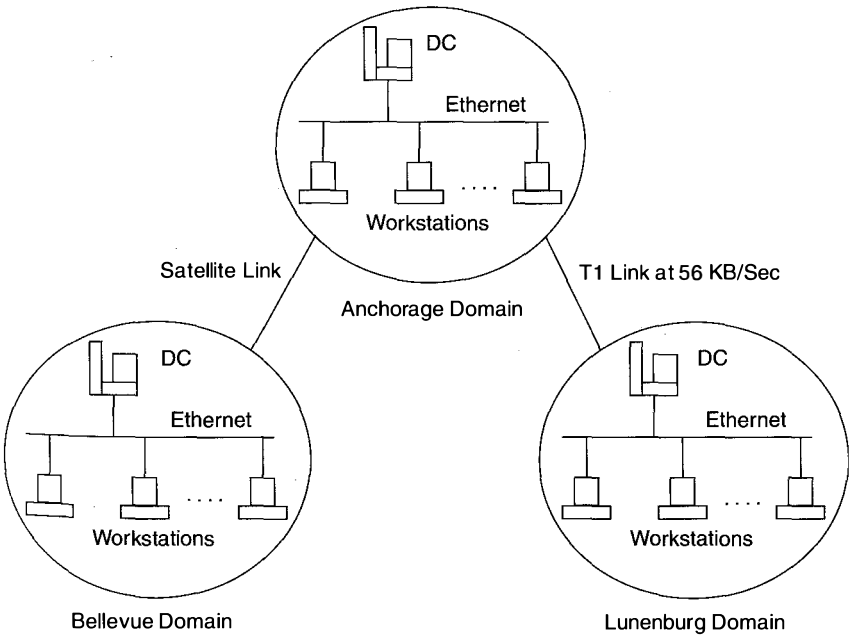


Figure 4-10. *Organizing Windows NT Machines into Domains*

11. Of course, matching physical topology closely with logical connections will be most efficient from network traffic point of view.

replicates crucial system information from each **primary domain controller (PDC)** to the BDCs. Thus, a DC becomes the ideal candidate for storing the resource location table mentioned earlier. This table is easily accessed by all the machines in the domain.

In the Windows NT environment, every DC offers Name Service by running the Windows NT Locator, and maintains a database of RPC servers in the Windows NT domain. The RPC Locator can also be run on any appropriate servers in the Windows NT domain. When a server registers its interface via the Name Service function, the interface data is registered in the local locator database. When a client application calls for binding information, the locator residing on the client computer will try to satisfy the request. If there is nothing in the cache, it will forward the request to the master locator. (The master locator runs on a PDC. When a PDC cannot be located, the client contacts the locator on the BDC.) If the master locator finds nothing in its cache, it sends the request to all computers in the domain using a Mailslot broadcast. The computer running the requested servers(s) will respond back by sending a directed mailslot message. These messages are collated by the locator on the client and given to the client application via Name Service APIs.

As you read earlier in this chapter, Name Service contains a database in which each entry is a tuple: {Resource name, Location Details}. This information is also referred to as the binding information, because it allows the client to bind, or connect, to the server. Each resource name is a logical name and is expressed in one of the two syntaxes:

- /:./name[/name..] Example, /:./DatabaseServer. This format is used when Windows and Windows NT clients/servers need to communicate with a DCE Name Service and vice versa.
- /.../domainname/name[/name..]. Example, /.../Marketing Domain/DatabaseServer. This syntax is used by most Windows and Windows NT applications.

Thus, a database server running in the Redmond domain can be logically referred to as "/.../Redmond/DatabaseServer." The location detail contains the server's network address, plus the interfaces or objects and the protocols supported by the server. An application can access a remote domain by specifying its name, provided the user has proper access privileges. Thus, Name Service locates useful resources and services, whether they are in the local or a distant domain.

Servers export their characteristics into the database so the clients can locate them. In the Windows NT environment, the database is nonpersistent, lasting only as long as the system is running. Once a computer is rebooted, Name Service must recreate its database; servers have to export their details into the Name Service database every time they are started. (Recall that the endpoint information, because it is more dynamic, is kept in the endpoint mapper rather than the Name Service database.)

Microsoft RPC interoperates with the distributed computing environment (DCE) **Cell Directory Service (CDS)**. Currently, Digital Equipment Corporation offers CDS on their platforms, and Microsoft RPC Name Service clients and servers can communicate with CDS-based clients and servers, provided proper drivers and modules have been loaded. Read [Rosenbary 93] for details on CDS.

The differences between Windows NT Name Service and DCE CDS are listed here.

- The DCE CDS provides a persistent Name Service database. In the DCE environment, servers need to export data into the database only when they are being installed.
- In the DCE environment, each Name Service entry can have one of the following attributes: server, group, or profiles. In Windows NT Name Service, each entry can have only one of these attributes, making three distinct types of entries: server entries, group entries, and profile entries. Every server adds an entry for itself, called the server entry. A server can belong to a (logical) group of servers, and a group entry is needed for each. A group contains the name of the server entry and entries for any other nested group. For example, a "Redmond" group may contain the names of all the server entries that belong to the Redmond domain. Finally, when conducting a database lookup, applications use specific profiles for easy navigation of the database. Each profile entry contains the name of other profiles, servers, or groups. For example, accountants use the "accounting" profile, because it contains names of groups and servers that meet the needs of the accountants, and engineers use the "engineering" profile to, say, locate a CAD/CAM server. The Windows NT Name Service only supports one default profile; applications cannot change the default profile or create new profiles.
- DCE CDS can maintain a hierarchy of entries, while in the Windows NT Name Service database, all entries are at the same level. Applications may use backslash-separated string names to create the illusion of a hierarchy.
- The Windows NT Locator cannot set access controls on Name Service entries. DCE CDS can set access controls (read/write) on each entry.
- The DCE CDS uses the environment variable RPC_DEFAULT_ENTRY_SYNTAX to determine the syntax type. The Windows NT Locator does not use such an environment variable, but provides the constant RPC_DEFAULT_ENTRY_SYNTAX. When this constant is supplied as the syntax type, the Locator maps this to the default syntax type (RPC_C_NS_NS_SYNTAXT_DCE).

To facilitate access to the Name Service database, RPC provides several APIs, that can be used to set information in the database and query for specific information from the database. Table 4-6 shows the Name-Service-specific APIs used to manipulate the name service database.

Table 4-6. *Name Service APIs*

Name	*Description*
Category: Exporting into Database	
RpcNsBindingExport() RpcNsBindingUnexport()	These APIs allow a server to export (or unexport) server entry to (from) the database. The server associates a logical entry name with the server characteristics.
Category: Importing	
RpcNsBindingImportBegin() RpcNsBindingImportNext() RpcNsBindingImportDone() RpcNsBindingLookupBegin() RpcNsBindingLookupNext() RpcNsBindingLookupDone() RpcNsBindingSelect() RpcNsBindingInqEntryName()	Client applications call these APIs to determine binding details about a server by searching the database. The clients search by logical entry names, and Name Service returns server details, if entry is found. Import methods return one binding handle at a time. The lookup methods return a vector and the client selects one handle by calling RpcNsBindingSelect(). In either case, the binding handle is partially bound. The client tries to bind to the server using this handle. On success, the handle is converted to a fully bound handle that can be used for all subsequent RPC calls from the client to the server.
Category: Object Query	
RpcNsEntryObjectInqBegin() RpcNsEntryObjectInqNext() RpcNsEntryObjectInqDone()	These routines can be used by the client, the server, and Name Service management applications to seek details about a specific object by object UUID.
Category: Group Entry	
RpcNsGroupMbrAdd() RpcNsGroupMbrRemove() RpcNsGroupMbrInqBegin() RpcNsGroupMbrInqNext() RpcNsGroupMbrInqDone() RpcNsGroupDelete()*	Members in a group can be added by RpcNsGroupMbrAdd() and removed by RpcNsGroupMbrRemove(). If group does not exist, RpcNsGroupMbrAdd() will create the group before adding a member to it. Applications can use RpcNsGroupMbrInqXXX() calls to determine names of group members.

(continued)

Table 4-6 continued

Name	*Description*
Category: Profile Entry	
RpcNsProfileDelete() RpcNsProfileEltAdd() RpcNsProfileEltRemove() RpcNsProfileEltInqBegin() RpcNsProfileEltInqNext() RpcNsProfileEltInqDone()	These APIs allow manipulation of profile entries and the elements in each profile. However, these are not implemented by Microsoft RPC.
Category: Management	
RpcNsMgmtEntryCreate() RpcNsMgmtEntryDelete() RpcNsMgmtInqIfIds() RpcNsMgmtSetExpAge() RpcNsMgmtInqExpAge() RpcNsMgmtBindingUnexport()	RpcNsMgmtEntryCreate() and RpcNsMgmtEntryDelete() can be used to create and delete specific entries (for example, groups or server entries). RpcNsMgmtInqIfIds() can be used to determine interfaces exported by a given server entry. RpcNsMgmtSetExpAge() can be used to control the reduce-or-enhance lifetime of an entry. Finally, RpcNsMgmtBindingUnexport() is useful for cleaning orphan entries, usually created when servers crash without cleaning the database.

* Not supported in Microsoft RPC v 1.0.

It is not appropriate to discuss the details of those APIs in this book. The following sections focus instead on the use of the export and import functions that are used by servers and clients to set and query for information from the name service.

Server Exporting Details into Name Service

A server may use the **RpcNsBindingExport()** API to export its binding information into a name service database, and associate a logical name with itself. **RpcNsBindingExport()** is also used for exporting server-supported objects into Name Service. Before terminating, the server should call the RpcNsBindingUnexport() API to remove its entry from the database.

```
RPC_STATUS RpcNsBindingExport(
    unsigned long        EntryNameSyntax,    // Syntax of logical name
    unsigned char        *EntryName,         // Logical name
    RPC_IF_HANDLE        IfSpec,             // Interface supported
    RPC_BINDING_VECTOR   *BindingVec,        // Binding information
    UUID_VECTOR          *ObjectUuidVec);    // Objects supported
RPC_STATUS RpcNsBindingUnexport(
    unsigned long        EntryNameSyntax,    // Syntax of logical name
```

```
    unsigned char          *EntryName,       // Logical name
    RPC_IF_HANDLE          IfSpec,           // Interface to remove
    UUID_VECTOR            *ObjectUuidVec);  // optionally object to remove
```

The following code fragment shows how to use these APIs. The server logical name is "SomeServer." It is present in the Windows NT domain "NewDomain," and exports a logical entry "\...\NewDomain\SomeServer" into Name Service. The server is also using dynamic endpoints that are registered in the endpoint mapper.

```
//
// ACF file
//
[implicit_handle(handle_t SomeInterface_IfHandle)]
interface SomeInterface
{
}
//
// IDL file for Client/Server using name service
//
[ uuid( <some uuid created by uuidgen utility>), version 1.0]
interface SomeInterface
{
    RPC_STATUS SomeMethod();
    void ShutDownServer();
}
//
// Server exporting its binding information into name service and
// the endpoint-mapper. Assume both client
// and server are running in same domain ("\\NewDomain") and DC
// is functioning.
//
int main ()
{
    RPC_STATUS status;
    RPC_BINDING_VECTOR *pBindingVector;
    // Support all protocols
          status = RpcServerUseAllProtseqs(
                                        1,          // Maximum 1 concurrent
                                                    // coming per protocol
                                        NULL);      //Security
    // Register the interface(s)
    status = RpcServerRegisterIf (
                           SomeInterface_ServerIfHandle,
                           NULL,
                           NULL);
    // Get the binding vector
    status = RpcServerInqBinding ( &pBindingVector);
    // Now, register dynamic endpoints in endpoint-mapper
    // Annotation associated with endpoint is not used by RPC
    // but may be used by applications for display purposes
    status = RpcEpRegister (
                        SomeInterface_ServerIfHandle,
                        pBindingVector,
                        NULL,                // Objects
```

```
                        "Some Interface");  // Annotation
    // We export the server info into name service
    // under the entry "/.../NewDomain/SomeServer"
    status = RpcNsBindingExport (
                            RPC_C_NS_SYNTAX_DEFAULT,
                            "/.../NewDomain/SomeServer",
                            SomeInterface_ServerIfHandle,
                            pBindingVector,
                            NULL); // No Objects to export
    // Start servicing the clients. This call will block
    // until ShutDownServer() has been called
    status = RpcServerListen (
                        1,  // Min. Threads
                        20, // Max Threads aka Max Clients
                        FALSE
                        );
    // Release binding vector
    status = RpcBindingVectorFree ( pBindingVector );
}
//
// Following routine either called remotely or locally shuts down the
// Server gracefully
//
void ShutDownServer()
{
    RPC_STATUS status;
    RPC_BINDING_VECTOR pBindingVector;
    // Stop servicing clients
    status = RpcMgmtStopServerListen (NULL);
    // Get binding vector
    status = RpcServerInqBinding (&pBindingVector);
    // Unregister endpoint from endpoint mapper
    status = RpcEpUnregister (          // Interface
                    SomeInterface_ServerIfHandle,
                    pBindingVector,     // Bindings
                    NULL);              // Objects
    // Remove entry from name service database
    status = RpcNsBindingUnexport ( RPC_C_NS_SYNTAX_DEFAULT,
                            "/.../NewDomain/SomeServer",
                            SomeInterface_ServerIfHandle,
                            NULL);          // Objects
    // Release binding vector
    status = RpcBindingVectorFree (pBindingVector);
    // Unregister all interfaces
    status = RpcServerUnregisterIf (NULL, NULL, FALSE)
}
```

Thus, the server registers its interface and binding information with Name Service. It also uses dynamic endpoints, which are registered in the endpoint mapper.

Locating a Server Using Name Service

One of the great benefits of Name Service is that a client can locate a server by specifying the server's entry name. Let us look at how the client obtains the binding information from Name Service.

On Windows NT, clients' Name-Service-related requests are first serviced by the local Locator, which checks the local database. Next, the database maintained by the primary Locator running on the DC is searched. If the DC cannot find the information, it broadcasts to the domain asking about the interface and fills up its database from the replies sent by the locator from a machine where the server has registered the required interface. [Note that in order to use the Locator on the DC, both client and server must use domain name based entry syntax (that is, \...\DomainName\name[\name]).] When clients use the domain name based entry name syntax, they can query locators on DCs of other domains by changing the domain name contained in the entry name. If they have a logical name or the entry name of the server, clients can use one of two schemes to locate a server. They can import an entry from Name Service, or they can look up an entry.

A client can import one entry at a time from Name Service. This is accomplished by calling the RpcNsBindingImportXXX. APIs, in much the same way as programs call the FindFirstFile(), the FindNextFile(), and the FindClose() Win32 APIs to locate a file. The clients call the RPC name service import APIs

- **RpcNsBindingImportBegin()**
- **RpcNsBindingImportNext()**
- **RpcNsBindingImportDone()**

(in that sequence) until they find a suitable match. Import methods return one partially bound binding handle at a time. A partially bound handle contains the server information, such as its address and the protocols it supports. This information is returned from the database for every server that has a matching interface or object ID. Suppose a client queries Name Service for a server known by the logical name "\...\NewDomain\SomeServer" and requests a specific interface *ISomeInterface*. Further assume that the server is present on six servers. In this case, the client will get six partially bound handles.

Once a client has obtained one or more partially bound handles, it attempts to bind to the server, using the partially bound handle. When such a binding attempt succeeds, a matching server has been found. At this point, the binding handle is converted to a full binding handle, representing the logical connection between the client and the server. However, it is possible that binding cannot be established: Name Service could have stale data, an errant server might not have removed its entries from Name Service prior to termination, or the server does not support the protocols known to the client.

Recall that the Name Service database is replicated among all the locators in the domain. On the other hand, the endpoint mapper on the server machine actually contains the endpoint information. To bind to a server using a partially bound handle, the client-side RPC must communicate with the endpoint mapper present on the server. The endpoint mapper contains logical

records in the form {interface, version, end points, protocols supported}. Thus, the client-side RPC can determine the server end points by matching the client-requested interface with a list of interfaces supported by the server. After the endpoint(s) have been determined, the client attempts to bind to the server. For example, the client may detect that the server supports the Named Pipe ("ncacn_np") protocol and has an end point ("\PIPE\ SomeServer"). If the client supports the Named Pipe protocols, the client can try to connect to the server Named Pipe ("\PIPE\SomeServer").

In the following code example, the client uses import methods to look up a server, using its entry name ("\...\NewDomain\SomeServer").

```
//
// Client using name service to locate the server. Assume both client
// and server are running in same domain ("\\NewDomain") and DC
// is functioning.
//
int main()
{
    RPC_STATUS                status;
    RPC_NS_HANDLE             ImportContext;
    BOOLEAN                   fServerFound = FALSE;
    RPC_BINDING_HANDLE hBinding;
    // Begin importing one name service entry at a time
    // in search for a server exporting ISomeInterface.
    status = RpcNsBindingImportBegin(
                      RPC_C_NS_SYNTAX_DEFAULT,          // Entry syntax
                      "/.../NewDomain/SomeServer",      // Entry Name
                      SomeInterface_IfHandle,           // Interface
                      NULL,                             // Object
                      &ImportContext);                  // Import Context
    if (status)
        return 1;
    do
    {   // Try to get a binding handle
        status = RpcNsBindingImportNext(
                      ImportContext,
                      &hBinding);
        if (status == RPC_S_OK) && (hBinding != NULL)
        {
            // Try to call the server
            status = SomeMethod();
            if (status == RPC_S_OK)
            {
                // We connected to the right server
                // Other calls and operations
                .....
                fServerFound = TRUE;
            }
            // We have to always release the binding handle
            // returned by RpcNsBindingImportNext()
            status = RpcBindingFree (&hBinding);
    } while (!fServerFound || status != RPC_S_NO_MORE_BINDINGS);
```

```
    if (status == RPC_S_NO_MORE_BINDINGS)
    printf("Could not find/connect to Server\n");
    // Success or not, we need to release import context
    status = RpcNsBindingImportDone( &ImportContext);
    return 0
}
```

Another way to locate a server is to use the RPCNsBindingLookupXXX. APIs. To do this, the client calls the **RpcNsBindingLookupBegin()**, **RpcNsBindingLookupNext()**, and **RpcNsBindingLookupDone()** APIs (in that sequence). Each RPC lookup call returns a vector of partially bound handles. A vector may contain partially bound handles to one or more potential servers. Next, the client calls the **RpcNsBindingSelect()** API to establish a firm connection and convert a partially bound handle to a fully bound handle. In the DCE environment, the CDS maintains a persistent database store; in the DCE/CDS environment, lookup methods are likely to be more efficient than import methods. In the Windows NT environment, import methods can be more efficient.

4.5.3 About Binding Handles

A logical handle, also referred to as a binding handle, represents the connection between the client and the server during the lifetime of the connection. A binding handle is opaque, much like a traditional file handle, and is considered "fully bound" when a logical connection exists between the client and the server. After this point, the client can make calls to the server. However, the binding handle can be partially bound. A partially bound handle is usually created when the client queries Name Service for a server by server's logical name. After such a query, the details of bindings (such as server name, address, or the endpoint) may be known, but the server endpoint has yet to be activated. This results in a partially bound handle. If the client now tries to bind to the server by calling one of the methods of the requested interface, it causes the client-side RPC run time to attempt connection with the server-side run time. On successful establishment of a connection, the handle becomes fully bound. The client does not have to call some API explicitly to turn a partially bound handle into a fully bound handle: RPC run time does the work automatically. Note that a partially bound handle can fail to become a fully bound handle. For example, the server may not be reachable (for any number of reasons) by the client even if the client can access the Name Service and extract server information. Be prepared for an occasional failure to connect a partially bound handle.

Client/server applications can choose the type of the handle. The choices range from user-controlled handles to completely automated ("autobinding") handles.

Autobinding Handles

As the name suggests, autobinding handles are managed by RPC and are appropriate for a large number of applications. For instance, when an application

does not mention any binding handle in the IDL or ACF files, the RPC compiler uses the default auto handles. In this case, the client and the server do not have to pass the binding handle as a parameter to any RPC call. The binding is managed by the client stubs, the RPC Locator, and the Name Service. In fact, the client does not have any code that requests an interface, a protocol sequence, or an endpoint. However, the server needs to register its interface(s), register the protocols supported, and declare its intent to use well-known or dynamic endpoint, as usual, and it must also export information about itself into Name Service. The client-end Locator locates the server by querying the Name Service database.

As the following example shows, the application chooses to use auto handles through the ACF file. Once proper ACF and IDL files have been declared, the client can call the remote function. Note the simplicity of the client-side code and compare it with the client-side codes shown earlier in this chapter. The server registers its interfaces with Name Service and exports a logical name for its services.

```
//
// ACF file for auto handle
//
[auto_handle]
interface StockQuote;
{
}
//
// IDL file
//
[uuid( <uuid generated by uuidge.exe>),
version 1.0]
interface StockQuote{
{
    RPC_STATUS Quote (
               [in, string] char *pCompanyTicker,
               [in, out, unique] char  *pStockPrice
               );
}
//
// Client code using auto handles to get latest stock price
// from a StockQuote Server
//
int (int argc, char *argv[])
{
    // We assume that user has supplied ticker symbol
    // as 1st parameter argv[1]
    RPC_STATUS status;
    char szStockPrice [MAX_PATH];
    status = Quote (argv[1], szStockPrice);
    if (!status)
    {
        printf("Stock Price for %s was %s\n", argv[1], szStockPrice);
    }
}
```

```
void __RPC_FAR * __RPC_API midl_user_allocate(size_t len)
{
    return(malloc(len));
}
void __RPC_API midl_user_free (void * ptr)
{
    free(ptr);
}
//
// StockQuote Server using auto handles
//
#define RETURN_ON_ERROR(string) \
    if (status != RPC_S_OK)    \
    {\
        printf("%s error: %ld\n", string, status);\
        return(1);\
    }\
int main ( int argc, char * argv[] )
{
    RPC_STATUS status;
    RPC_BINDING_VECTOR *BindingVector;
    // Register one or more protocols supported
    status =RpcServerUseProtseqEp(
                "ncacn_np",                 // Protocol
                1,                          // MaxCalls,
                "\\pipe\\StockQuote",       //Endpoint,
                NULL                        // SecurityDescriptor
                );
    RETURN_ON_ERROR("RpcServerUseProtseqEp");
        // Register interface(s)
    status = RpcServerRegisterIf (
                                  StockQuote_ServerIfHandle,
        NULL,
        NULL);
    RETURN_ON_ERROR("RpcServerRegisterIf");
    // Get at the binding vector
    status = RpcServerInqBinding ( &BindingVector);
    RETURN_ON_ERROR("RpcServerInqBinding");
    // Now, we export an entry per interface
    status = RpcNsBindingExport(
                RPC_C_NS_SYNTAX_DEFAULT,     // EntryNameSyntax,
                "/.:/StockQuote",            // EntryName,
                StockQuote_ServerIfHandle,   //IfSpec,
                BindingVector,               // Bindings information
                NULL                         // ObjectUuidVec
                );
    RETURN_ON_ERROR("RpcNsBindingExport");
    // Now we are ready to receive client requests
    status = RpcServerListen (1, 20, FALSE);
    // Deregister the interface and remove entry from Name Service
    <..>
    return(0);
}
RPC_STATUS Quote ( PCHAR pchCompany, PCHAR pchQuote)
{
    // Potentially look up some database for stock quote
    // based on company name
```

```
    strcpy(pchQuote, GetPriceFromDatabase(pchCompany));
    return 0;
}
void __RPC_FAR * __RPC_API midl_user_allocate(size_t len)
{
     return(malloc(len));
}
void __RPC_API midl_user_free(void * ptr)
{
     free(ptr);
}
```

Auto handles are recommended when the client is not interested in any one specific server, that is, any server that supports the client request can be used. Generally, the servers that provide general services, such as time services or on-line services, are prime candidates for autobinding handles. A company may set up one or more such servers to increase availability. Fault tolerance is another benefit of maintaining multiple servers. When clients can connect to any of the available servers, they are less likely to be affected by system crashes on one or more servers.

Application-Managed Handles

Application-managed handles can be classified into three categories: primitive binding handles, user-defined binding handles, and context binding handles.

- *Primitive Handles:* RPC defines the basic handle primitive, handle_t. Applications can use this handle directly, without any further embellishment. The handle points to important binding information, such as server address, endpoint, and so on. When a connection is established between client and server, the handle guides subsequent RPC calls to their correct destination. Just like file handles, applications should use the handle as an opaque entity, and only use RPC calls to determine the information the handle may point to. Once the connection has been established, primitive handles can be used in two ways: explicit and implicit.
    - *Explicit:* Each RPC call from the client to the server contains the handle, usually as the first parameter of the call. Recall that the client gets the handle by calling RPC functions such as RpcBinding-FromStringBinding(). Explicit handles are useful when the client communicates with many servers at the same time.

```
//
//
IDL file showing usage of explicit handles for a Music Server
//
[uuid ( <some uuid generated by uuidgen> ),
 version (7.77)]
interface Entertainment
```

```
{
    // The binding handle is passed as a parameter of each RPC call
    RPC_STATUS PlayMusic (
                    [in] handle_t hBindingHandle,
                    [in, string] char *pchComposer,
                    [in, string] char *pchSymphony.
                    [in, string] char *pchDetails
                   );
}
//
// Client code showing usage of explicit handle
//
int main ()
{
    char * pszStringBinding;
    RPC_BINDING_HANDLE hMusicServer;
    // Create binding string
    status = RpcStringBindingCompose(
                              ...,     // Details left out
                              &pszStringBinding);
    // Obtain a binding handle as a result of binding to the
    // Server
    status = RpcBindingFromStringBinding ( pszStringBinding,
                                      &hMusicServer);
          // Now, we can call the Server by using explicit handle
          status = PlayMusic (
                      hMusicServer,   // Notice the RPC handle
                      "Beethoven",
                      NULL, NULL   // pick default
                      );       // Details left out
    ....
}
```

- *Implicit:* Most client applications can communicate with multiple servers, but usually with only one at a time. They bind to a server, use its services, and then disconnect. RPC provides an implicit handle mechanism so that the clients do not have to pass the handle in every RPC call. This simplifies the client code, because the same binding handle is available globally. This is different from auto handles, in which the stubs take care of calling the server via Name Service/Locator. With implicit handles, the client still has to call RPC functions to connect with the server, as shown below. The RPC run time uses the same global handle to communicate with its counterpart on the other end.

```
//
// ACF file for using implicit handle
//
[implicit_handle (handle_t hMusicServer)]
interface Entertainment
{
}
//
// IDL file for implicit handles for a Music Server
```

```
//
[uuid ( <some uuid generated by uuidgen> ),
 version (007)]
interface Entertainment
    // Notice that the binding handle is not passed as a parameter
    RPC_STATUS PlayMusic (
                          [in, string] char *pchComposer,
                          [in, string] char *pchSymphony.
                          [in, string] char *pchDetails
                          );
}
//
// Client code showing usage of implicit handle
//
int main ()
{
    char * pszStringBinding;
    // Create binding string
    status = RpcStringBindingCompose(
                             ...,     // Details left out
                             &pszStringBinding);
    // Obtain a binding handle as a result of binding to the
    // Server. Recall that the handle is a global variable
    status = RpcBindingFromStringBinding (
                          pszStringBinding,
                          &hMusicServer);
    // Now, we can call the Server by using implicit handle
    status = PlayMusic ("Beethoven", NULL, NULL);
    ....
}
```

- *User-Defined Handles:* RPC provides the means for specifying user-defined handles and the user-supplied bind/unbind routines that are called when the caller wishes to connect to, or disconnect from, the callee. Thus, the application has tighter control over RPC-based communication. Typically, this option will be used by the packages (or class libraries) that may want to shield their applications from the underlying RPC-based communication. In this case, when the application calls a package-defined remote function, the package-supplied bind/unbind routines are called, which allows the package to have control over network communication. Just like primitive handles, user-defined handles can be explicitly defined as a parameter of each RPC call, or be used implicitly.
- *Context Handles:* Context handles are useful for applications that need to maintain a context at the server from one RPC call to the next. A typical example is that of remote file manipulation by an RPC, in which the file pointers to the remote file must be maintained at the server from one remote file operation to the next. In this case, you should specify a context handle by specifying the **[context_handle]** attribute on the user-defined context data type. Next, specify the context handle as a parameter to

each method of an RPC interface in which each context needs to be maintained. The server usually has one function that creates the context handle; the rest of the functions use the context handle. A context rundown routine, which must be provided by the server, is called by RPC run time when the connection is broken to clean up the server context. Of course, the client also needs to have methods for closing the handle.

The following example demonstrates an interactive TV golf game in which the server tracks the name of the player, his score, and his progress on the golf course. The client first opens the game, thus creating a context. The context passes from the client to the server at each remote call, so the server can take appropriate action based on the context. For instance, it can direct the user to play hole 9 if he has already played hole 8. The example shows how an interactive TV golf server can keep details of the play as the player progresses through the game. Notice that the data about the client itself is being sent back and forth between the client and the server. This includes the client (player) name. Pause for a moment and think about improving the network efficiency of these contexts.[12]


```
// ACF file. Note we are using implicit primitive handle for
// binding purposes and context handles for maintaining context
// as well as for binding.
//
[implicit_handle (handle_t hGolfByInteractiveTV]
interface GolfByInteractiveTV
{
}
//
// IDL file for an interface using context handle. In this case,
// context can be player's handicap, hole no., last score, etc.
//
interface GolfByInteractiveTV
{
    typedef [context_handle] void * PGOLF_GAME_CONTEXT;
    typedef [ref] PGOLF_GAME_CONTEXT * PPGOLF_GAME_CONTEXT;
    // This method is called once to create a context
    RPC_STATUS StartGolf (
                    [in, string] char *pchPlayerName,
                    [out] PPGOLF_GAME_CONTEXT pphContext
                    );
    // Rest of methods take context as parameters
    RPC_STATUS ResumeGolf (
                    [in] PGOLF_GAME_CONTEXT phContext
                    [in] unsigned short usSwingType,
                    [in] short sHandicap
                    ...
                    );
```

12. Hint: Transmitting strings is expensive, and problematic in multilingual environments. Use a number-based identification, such an sn account number.

```
    RPC_STATUS ChangeClub (
                    [in] PGOLF_GAME_CONTEXT phContext,
                    [in] BOOLEAN fClubType , // Iron or Wood
                    [in] unsigned short usNumber , // Wood 3 or Wood 5
                    );
    RPC_STATUS EndGame ( [in,out] PPGOLF_GAME_CONTEXT pphContext );
}
//
// Interactive TV Client communicating with a Golf Server
//
int main ()
{
    PGOLF_GAME_CONTEXT phContext = NULL;
    PCHAR pszStringBinding;
    // Create a binding string
    status = RpcStringBindingCompose(
                            NULL,                   // Object
                            "nacn_np",              // Protocol
                            NULL,                   // Network address
                            "\\pipe\\GolfSrv",      // Endpoint
                            NULL,                   // options
                            &pszStringBinding);
    // Set the binding handle that will be used to bind to the server.
    status = RpcBindingFromStringBinding(
                                pszStringBinding,
                                &hGolfByInteractiveTV);
    // Start the game which will create a context as well
    StartGolf("John", &phContext);
    // Now, we can close the initial binding because
    // the context handle also manages the binding.
    status = RpcBindingFree(&hGolfByInteractiveTV);
    // Play for a while
    ...
    // go to Restaurant to have lunch
    Sleep(LUNCH_TIME_OUT);
    // Resume the game
    ResumeGolf(phContext, ....);
    // Now, close the game
    EndGame(&phContext);
}
//
// Golf server implementation
//
// First server defines a structure to maintain the state of a
// player across multiple RPC calls
typedef struct {
    CHAR        szPlayerName [ MAX_NAME_LENGTH ];
    BOOLEAN     fSlotTaken;         // Indicates if this structure is active
    int         iPlayerNos;         // player Sequence Number
    short       sHandicap;
    short       sGreenNo;
...                                 // Other parameters
} PLAYER_DATA;
// Assume that the server simply stores all data in global memory
PLAYER_DATA Players[MAX_PLAYERS];
int   iCurrentPlayer = 0;
// StartGolf() method creates a context handle much like OpenFile()
```

```
RPC_STATUS StartGolf (
                CHAR *pchPlayerName,
                PPGOLF_GAME_CONTEXT pphContext)
{
    *pphContext = NULL;
    if (iCurrentPlayer == MAX_PLAYERS)
        return GOLF_COURSE_TOO_BUSY;
    // Find a convenient slot for new player
    while (int i = 0; i < MAX_PLAYERS; i++)
        if (Players [i].fSlotTaken == FALSE) break;
    *pphContext = &Players [i];
    // Set up the slot with proper data
    Players [i].fSlotTaken = TRUE;
    Players [i].iPlayerNos = i;
    ....
    // Bump the current usage count
    iCurrentPlayer +=  1;
    return 0;
}
// End Game method which is much like CloseFile()
RPC_STATUS EndGame ( PPGOLF_GAME_CONTEXT* pphContext )
{
    PLAYER_DATA *pPlayer = (PLAYER_DATA *) *pphContext;
    // Mark the slot in Players array empty
    Players[pPlayer          iPlayerNos].fSlotTaken = FALSE;
    iCurrentPlayer—;
    *pphContext = NULL;
    return 0;
}
// Finally, the server must provide rundown routine called by RPC
// when connection is broken (or lost) with client
void PGOLF_GAME_CONTEXT_rundown ( PGOLF_GAME_CONTEXT phContext )
{
    EndGame (&phConext);
}
```


4.6 USING CALLBACK ROUTINES

RPC is specifically designed and optimized for the client/server computing model:

1. The client calls the server and waits for the server's response,
2. The server receives the request, processes it, and sends back a response,
3. The client receives the server's response and continues operation.

However, sometimes it is convenient for the server to call back the client for some additional detail concerning the client's original call. Microsoft RPC provides callback as an extension to DCE. In such cases, the client can provide a callback routine for the server. These routines are defined within the interface block of the IDL file and are prefixed with the **[callback]** keyword. The implementation is provided by the client; any method on the server that is part of an interface can call this function. The callback functions are called

from the server over the same binding as the one used by the client to connect to the server in the first place. Furthermore, the callbacks can nest to an arbitrary depth (although a designer should be cautious when allowing recursive callbacks).

The two IDL files that follow show how using callback functions makes designing interfaces a little easier. In these files, the interface is exposed by a file backup server. The first file does not use the callback function; it needs to maintain a context handle, because the server needs to know the latest position of file pointer for its own backup file. The client therefore starts by calling the **SaveMyFile()** function, and continues to call the **ContinueSave()** function until all its local file has been read and sent to the server. Finally, it calls the **DoneSave()** function, at which time the server can flush and close its backup file.

In the second file, the interface contains a callback function; the client calls the SaveMyFile() method of the new interface. If the file does not fit in the buffer supplied, then the client sets fMore to TRUE. This prompts the server to use the callback function to get more data from the client. The client can keep returning TRUE in the pfMore parameter of the callback function **GetData()** for as long as there is more data to be archived. When the callback function returns to the server, the server can call the client back again, as long as the client sets the pfMore parameter to TRUE. The server is in control and it does not have to build a context handle.

```
//
//  IDL file NOT using Callback function
//
[uuid (<some uuid generated by uuidgen>), version (1.0),
pointer_default(unique)]
interface BackupFileOnServer
{
    typedef [context_handle] void * PFILE_HANDLE_CONTEXT;
    typedef [ref] PFILE_HANDLE_CONTEXT * PPFILE_HANDLE_CONTEXT;
    RPC_STATUS SaveMyFile (
                    [in, string] PCHAR pchLocalFile,
                    [in, length_is(ulBuflen)] BYTE *bBuffer,
                    [in] ULONG ulBuflen,
                    [out] PPFILE_HANDLE_CONTEXT ppContext
                    );
    RPC_STATUS ContinueSave (
                    [in, length_is(ulBuflen)] BYTE *bBuffer,
                    [in] ULONG ulBuflen,
                    [in] PFILE_HANDLE_CONTEXT ppContext
                    );
    RPC_STATUS DoneSave([in,out] PFILE_HANDLE_CONTEXT ppContext);
}
//
// IDL file using Callback function
//
[uuid (<some uuid generated by uuidgen>), version (2.0),
 pointer_default(unique)]
```

```
interface BackupFileOnServer
{
    RPC_STATUS SaveMyFile (
                    [in, string] PCHAR pchLocalFile,
                    [in, length_is(ulBuflen)] BYTE *bBuffer,
                    [in] ULONG ulBuflen
                    [in] BOOLEAN fMore
                    );
    [callback] RPC_STATUS GetData (
                            [out, length_is(ulBuflen), unique] BYTE *bBuffer,
                            [out ] ULONG *pulBuflen,
                            [out] BOOLEAN *pfMore
                            );
};
//
// Client using Callback mechanism
//
// Global File Pointer Parameter used to manipulate the file
// to be backed up on server
FILE *pFile = NULL;
int main ( int argc, char *argv[] )
{
    BYTE bBuffer [MAX_BUFFER_SIZE];
    BOOLEAN fMore = FALSE;
    // Bind to the server
    ....
    // Now, we have a binding handle. We request the
    // Server to backup the file
    if (pFile = fopen(argv[1], "r") == NULL)
        exit (1);
    else
        fMore = ReadFile (bBuffer, &ulBuflen, MAX_BUFFER_SIZE);
    // Call the server which may call client back if fMore=TRUE
    status = SaveMyFile (
                argv[1] , // Local File Name
                bBuffer,  // Buffer
                ulBuflen, // nos of bytes in buffer
                fMore
                );
    // Close the file
    fClose (pFile);
}
//
// Helper function to read the file
//
BOOLEAN ReadFile ( BYTE *bBuffer, ULONG *pulBuflen, ULONG ulBufSize)
{
    *pulBuflen = fread ( bBuffer,                // Buffer to read into
                         sizeof(BYTE),           // item size
                         ulBufSize/sizeof(BYTE), // max items
                         pFile
                         );
    if (*pulBuflen == -1) // Check for end of file
    {
        *pulBuflen = 0;
        return FALSE;
    }
```

```
    else
        return TRUE;
}
//
// Client implementing Callback function
RPC_STATUS GetData ( BYTE *pBuffer, ULONG *pulBuflen, BOOLEAN *pfMore )
{
    if (ReadFile (pBuffer, pulBuflen, MAX_BUFFER_SIZE)==TRUE)
    {
                pBuffer = MIDL_user_allocate (MAX_BUFFER_SIZE);
                *pfMore - TRUE;
                return RPC_S_OK;
    }
    else
    {
         *pfMore = FALSE;
         *pulBuflen = 0;
         *pBuffer = NULL;
         return RPC_S_OK;
    }
}
//
// Client functions called by RPC to allocate and free memory
//
void __RPC_FAR * __RPC_API MIDL_user_allocate( size_t len )
{
     return(malloc(len));
}
void __RPC_API MIDL_user_free( void __RPC_FAR * ptr )
{
    free(ptr);
}
//
// Server using Callback mechanism
//
int main()
{
    ...
}
//
// Remote procedures implemented on server
//
RPC_STATUS SaveMyFile (
               PCHAR pchLocalFile,
               BYTE *bBuffer,
               ULONG ulBuflen
               BOOLEAN fMore
               )
{
    // Create a local backup and write into it the buffer
    // sent by client
    .....
    // Now, check if client wants to send more data
    while (fMore)
    {          BYTE *pbServerBuffer = NULL;
               ULONG ulBuflen;
               // Call the client so long as he sends back fMore = TRUE
```

```
            if (GetData (
                    pbServerBuffer,
                    &ulBuflen,
                    &fMore      )  ! = RPC_S_OK)
                    break;      // error occurred
        // Write the buffer to file and free the buffer
        ....
        if  (*pbServerBuffer)
              MIDL_user_free (*pbServerBuffer);
    }
    // Flush and close local backup file
    ...
    return RPC_S_OK;
}
```

Thus, callback functions can be used to allow the RPC server to call a predefined client callback function in the context of the original connection between client and server.

4.7 RPC MANAGEMENT

RPC provides a variety of management functions, all of which can be used by the client, the server, or management applications. Applications can use these functions to control the RPC communication components, the name service, the endpoint mapper, and the security aspects of RPC. These APIs are summarized in Table 4-7.

Table 4-7. *RPC Management APIs by Use Category*

API Names	*Description*
Category: Endpoint Management	
RpcMgmtEltInqBegin() RpcMgmtEltNext() removeRpcMgmtEltDone() RpcMgmtEpUnregister()	These allow management applications to check the current end points and any endpoints created by dead servers. These APIs are not supported by MS RPC v1.0.
Category: Server Management	
RpcMgmtIsServerListening() RpcMgmtSetServerStackSize() RpcMgmtStopServerListening() RpcMgmtWaitSeverListen()	RpcMgmtIsServerListening() allows a client to verify the presence of a server. However, in MS RPC v1.0, clients can only use this for detecting presence of the local servers. RpcMgmtSetServerStackSize() can be used to change the stack size for each server thread started by RPC when a new client call comes to the server. The other two APIs are discussed on the next page.

(continued)

Table 4-7 continued

API Names	*Description*
Category: Communication Management	
RpcMgmtSetComTimeout() RpcMgmtInqComTimeout() RpcMgmtSetCancelTimeout()	RpcMgmtSetComTimeout() is used by a client as an indication of timeout options; the timeout values are not in unit any time, but are a relative value compared to the default wait options. RpcMgmtSetCancelTimeout() is not supported by MS RPCv1.0.
RpcMgmtInqStats() RpcMgmtStatsVectorFree()	On the server, RpcMgmtInqStats() returns a vector full of run-time statistics on the number of incoming/outgoing calls and number of incoming/outgoing packets.
Category: Security Management	
RpcMgmtInqServerPrinName() RpcMgmtInqDfltAuthnLevel() RpcMgmtSetAuthorizationFn()	Not implemented by Microsoft RPCv1.0.

Of all these APIs, the **RpcMgmtStopServerListening()** and **RpcMgmtWaitServerListen()**[13] APIs are the two most frequently used. Recall that when a server is ready, it calls the RpcServerListen() API, whose prototype follows:

```
RPC_STATUS
RpcServerListen(
        unsigned int MinimumCallThreads,
        unsigned int MaxCalls,
        unsigned int fDontWait);
RPC_STATUS
RpcMgmtWaitServerListen(void);
```

Usually, the server supplies FALSE in the *fDontWait* parameter, causing the RpcServerListen() API to block. However, the server can supply a nonzero value in the fDontWait parameter, which will cause the RpcServerListen() API to return immediately. In this case, clients are not serviced until the server calls the RpcMgmtWaitServerListen() API, which blocks the server thread calling RpcMgmtWaitServerListen().

Once the server is blocked on the listen call, it cannot stop or unblock on its own. In this state, the RPC run time keeps starting a new thread every time a remote procedure call comes from a client; the number of

13. RpcWaitServerListen() is a Microsoft extension to RPC.

threads spawned depends on the MaxCalls parameter specified in the RpcServerListen() call. To instruct the RPC run time to stop servicing clients and unblock the RpcServerListen() (or the RpcMgmtWaitServerListen) call, use RpcMgmtStopServerListening(). This API comes into play when you want to instruct the RPC run-time to stop servicing clients. The prototype is shown here:

```
RPC_STATUS RpcMgmtStopServerListening( RPC_BINDING_HANDLE  Binding);
```

4.8 RPC ERROR AND EXCEPTION HANDLING

All RPC applications must carefully check any error codes returned by an RPC API and take suitable action. Most RPC APIs use the RPC_STATUS data type to return an error code; the RPC_S_OK value indicates successful completion of an RPC function. Frequently, the sample code fragments presented in this chapter do not check for the error code. This was done to reduce the number of lines of code so that you could focus on the discussion at hand. However, I strongly recommend that all return codes be checked, as shown in the following pseudocode.


```
//
// Bad coding practice. Here the caller is not checking the value
// of return code.
//
RPC_STATUS status;
status = RpcServerRegisterIf(...);
status = RpcServerListen(...);
//
// Good programming practice. While the coding style may vary
// from one programmer to another, note that error checking.
//
if (RpcServerregisterIf(...) == RPC_S_OK)
{
    if (RpcServerListen(...) == RPC_S_OK)
    {
    }
    else
    {
        printf("ServerListen failed\n");
        // appropriate action
    }
}
else
{
    printf("RpcRegisterIf Failed\n");
    // Appropriate action
}
```

Application developers must also be aware of RPC exceptions, which are NT exceptions raised on all platforms by the RPC components. (See Chapter 2,

Section 2.3.2, "Structured Exception Handling.") RPC applications should be able to handle exceptions whenever the clients and the server are communicating with each other. (This doesn't apply to OLE object interfaces because the exceptions are taken care of and proxies return on HRESULT.) If the application does not catch an exception, Windows NT will display an application failure message and then shut down the application. This can result in important information being lost.

Unless noted otherwise, calling an RPC API does not cause an exception. Exceptions are raised for the following reasons:

- Recall that the client or the server data marshaling and unmarshaling stubs convert data, pointers, and objects pointed to by pointers, into an NDR format prior to shipping to the other side and vice versa. These stubs can raise exceptions that are caused by an incorrect IDL file (for example, a reference pointer being NULL) or by run-time errors (such as a buffer for containing incoming data being smaller than the incoming data). Exceptions in this class are usually caused by bugs in the application that can be detected during testing of the application.
- Exception caused by network errors. RPC run time usually throws an exception if an unexpected network error occurs, such as a server crash, a cable failure, and so on. Applications must be safeguarded against such exceptions, which can occur at any time. Indeed, such exceptions may occur only after a client/server connection has been established.
- Exceptions caused by the server. A good server should always catch any exceptions and return a valid return code. This is necessary, because RPC brings the exception back to the client when a remote procedure is being executed on a server that does not catch the exception. (See **comm_status_t** usage.) Still, client applications must be safeguarded against errant servers.

To facilitate safeguarding an application, RPC provides two constructs: the RpcExcept and the RpcFinally.

- The RpcExcept construct is put around the section of code in the server (or in the client callback function) to catch any exceptions, should either server or client fail to catch an error. The following code fragment shows how to use the construct. The **RpcTryExcept()** API and the **RpcEndExpect()** API mark the beginning and the end, respectively, of the exception block. The code fragment contained by RpcTryExcept() is called the "guarded statement"; the code fragment contained by RpcExcept() is the exception-handling code. RpcExcept() takes an expression as an argument (one in this case). When an exception occurs in the

guarded statements, an exception is thrown by RPC on any platform. The exception is caught by the exception-handling code, provided the expression for RpcExcept() evaluates to nonzero. When it does, the exception-handling code is executed. Within the exception-catching block, the **RpcExceptionCode()** API can be called to determine the cause of the exception.

```
//
// IDL file
//
[ uuid(< some uuid generated by uuidgen utility>), version (1.0),
  pointer_default(unique)
]
interface ISomeInterface
{
          void SomeMethod();
}
//
// Client code safeguarded against exceptions
//
int main()
{
          // Bind to server
          .....
          // Now we are ready to call the remote procedure 'SomeMethod'
          unsigned long ulCode = 0;
          RpcTryExcept
          {   // Guarded statements
              // Call the remote procedure
              SomeMethod();
          }
             // Catch all exceptions
          RpcExcept (1)
          {
              // An exception occurred, hence this exception
              // catching block will be executed.
              ulCode = RpcExceptionCode();
              printf("Runtime reported exception 0x%lx\n", ulCode);
          }
          RpcEndExcept;
          // More processing
          if (ulCode)
          {
            // Ready to abort the program after cleanup of resources
              ...
              return (1);
          }
          // No exception. Continue processing
          .....
}
```

- The RpcFinally construct is the second programming method. The code block immediately following **RpcFinally()** is always executed, regardless of whether there was an exception in the

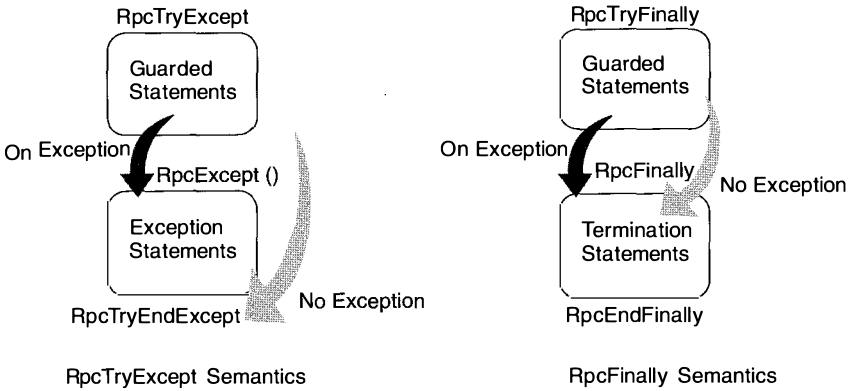


Figure 4-11. *Flow of Control in RpcTryExcept Compared with RpcFinally*

guarded statements. This is illustrated in Figure 4-11. When the RpcFinally construct is used, the termination statements are always executed, but when RpcTryExcept is used, the exception-handling code is only executed when an exception occurs in the guarded statements.

To determine if termination statements are being executed (because of an exception in the guarded statement or because of normal control flow into the termination block), RPC provides the RpcExceptionCode() API, which returns a nonzero value if an exception has occurred in the guarded statements. The following code fragment shows the use of RpcFinally() construct and the RpcExceptionCode() API to safeguard against exceptions.

```
// Client using RpcFinally construct. Assume same IDL file (and Server)
// as for RpcTryExcept example above.
//
int main()
{
    // Bind to server
    .....
    // Now we are ready to call the remote procedure 'SomeMethod'
    unsigned long ulCode = 0;
    RpcTryFinally
    {  // Guarded statements
       // Call the remote procedure
       SomeMethod();
    }
    RpcFinally
       {  // The following block will be whether exception occurred or not
       if (ulCode == RpcExceptionCode())
       {
       printf("Runtime reported exception 0x%lx\n", ulCode);
      // Other chores to be done on exception
       ......
    }
```

```
    else
    {
        // This block is executed if no exceptions have
        // occurred
        ....
    }
    // Common cleanup chores
    .....
}
RpcEndFinally;
{
    // More processing
    if (ulCode)
    {
          // Ready to abort the program after cleanup of resources
                    ...
                    return (1);
    }
          // No exception. Continue processing
          .....
}
```

The RpcFinally construct is especially useful when the program must perform some tasks (such as closing an opened file, freeing memory, and so on) whether an exception occurred or not. Thus, RPCFinally provides a nice framework for writing consolidated cleanup code.

4.9 SECURE RPC

Security and access controls are significant parts of most client/server applications. This is especially true if the server is manipulating sensitive data for the client. (For example, a database server may contain a client's personnel records or payroll information.) The system designer can take one of the two approaches to safeguarding the data: designers can use the access control methods of the database (if they are building a database client/server system), or they can use the security mechanisms of the underlying operating system. In the client/server system, the designer creates special "views" into the database tables and places access controls on the views so that only authorized personnel can access the data. For the underlying operating system, the database access is controlled by restricting the access to the computer that runs the, database server. It is also common to use a combination of the access control's two methods.

As you know from Chapter 2, Windows NT provides an elaborate set of tools and mechanisms to secure access to computers in a Windows NT domain. The basic strategy is to let the client/server system be as accessible as the computer hosting the server. In such systems, the administrator controls the security of the client/server system by allowing only the authorized users (or group of users) to connect with the Windows NT workstation that hosts the server.

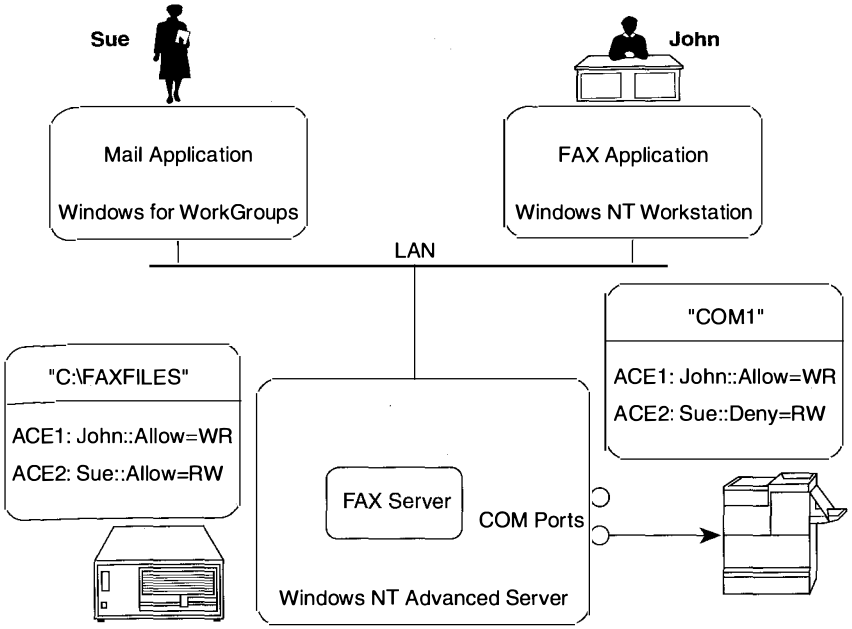


Figure 4-12. *Access Control of Resources Used by a FAX Server*

However, it is possible to gain a finer degree of access control in an RPC-based client/server system by having the server "impersonate" the remote client. That is, the server process mimics to its local system the client's security context. This allows administrators to restrict access to the computer that runs the server, and also to control each individual resource (for example, threads, files, tables, and so on) that the server manipulates on behalf of the client. Figure 4-12 shows how this works. The FAX Server is an RPC-based server that impersonates a client prior to sending or receiving facsimiles. To send and receive faxes, the server must manipulate both the COM ports and the local disk for storing sent/received files. In this example, "John" has proper rights to both the disk and the COM ports, hence the FAX application (the client) will be able to send faxes. On the other hand, "Sue" does not have access controls to the COM ports on the server, and so, the Mail application (another client) will fail.

To function properly, a server running on Windows NT has full administrative access over all the resources on the local system. For example, the FAX server shown above needs administrative rights to initialize the COM ports at startup time. However, as long as it is impersonating the client, its access rights are the same as that of the remote user. An RPC server impersonates the client's security context by having the server thread (that services the client request) call the **RpcImpersonateClient()** API. After the client request has been processed, the server should revert to its own

security context by calling the **RpcRevertToSelf()** API. The prototypes of the APIs are shown here:

```
RPC_STATUS  RpcImpersonateClient(RPC_BINDING_HANDLE  CallHandle);
RPC_STATUS  RpcRevertToSelf (void);
```

The following code fragment shows how these functions are used in a **SendFax()** function of the FaxServer interface. Notice that at the start of the SendFax() function, the thread has full server rights and can initialize the COM ports if necessary. Then it impersonates the client. If the client does not have proper access to the COM ports, the function **InternalSendFax()** (which we assume does the real work of sending the file to the COM port) will fail.

```
//
// Server impersonating the clients
//
// This code is not multi-thread safe
// ACF file
[implicit_handle (handle_t hBindingHandle]
interface SomeInterface
{
}
// IDL file
interface FaxInterface
{
          void SendFax(LPSTR szFileName);
          void ReceiveFax(LPSTR szFileName);
}
//
// Implementation of SendFax() (on Server)
//
static BOOL  fFirstTime = TRUE;
static ULONG ulClientServed = 0;
void SendFax( LPSTR szFileName )
{
          // At this point, the security context is that of the
          // server. Hence, we can initialize COM ports if
          // this is the first call
          if (fFirstTime)
          {
                    InitializeComPorts();
                    fFirstTime = FALSE;
          }
          // Now impersonate as the Client
          RpcImpersonateClient();
          // send the Fax
          if (InternalSendFax(szFileName)==ERROR_ACCESS_DENIED)
          {
                    printf("Client does not proper access.\n");
          }
          // Revert Back to Server's original security context
          RpcRevertToSelf();
```

```
        // Increment the Client served count
        ulClientServed++;
}
```

4.10 ECHO SERVER—RPC-BASED WIN32 SERVER

Here, you see illustrated an RPC-based Win32 service, Echo Server, which was first introduced in Chapter 2. The Echo Server shown here uses RPC to communicate with its clients, echoing back the string sent by the client. This section shows only those parts of the server that relate to RPC.

As the code fragment below shows, the Echo Server supports an implicit binding handle and exports its functions through an interface, Echo, which has a single function, EchoProc(). The **EchoProc()** function takes the client-sent string as the pszString parameter, and returns the server reply through the fixed length character array (szOutput) parameter.

```
// ACF File
implicit_handle(handle_t Echo_IfHandle)]
interface Echo
{
}
// IDL File
[ uuid (906B0CE0-C70B-1067-B317-00DD010662DA),
  version(1.0), pointer_default(unique)
]
interface Echo
{
        #define MAX_OUTPUT_LEN 128
        cpp_quote("#define MAX_OUTPUT_LEN 128")
        void EchoProc([in, string] char * pszString,
                    [out, string ]
                            char szOutput[MAX_OUTPUT_LEN]
                    );
}
//
// FILE: ECHOS.C - Implements EchoServer using RPC
//
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <process.h>
#include <ctype.h>
#include "echo.h"      // header file generated by MIDL compiler
#include <service.h>
#include <printcmd.h>
// The thread which blocks waiting for RPC calls
void ServerWaitThread(VOID *pF);
void ShutDown(void);
//
// Function called by generic service stub developed in chapter 2
// to start a service
BOOL InitEchoServer()
{
```

```
    //
    // The thread created by _beginthread()
    // will automatically get cancelled
    // when ServerWaitThread() completes
    //
    _beginthread(
                    ServerWaitThread,
                    4096,       // stack size
                    NULL);      // argument to thread
    return(TRUE);
}
void ServerWaitThread(VOID *pF)
{
    RPC_STATUS status;
    LPTSTR pszProtocolSequence = L"ncacn_np";
    LPTSTR pszSecurity          = NULL;
    LPTSTR pszEndpoint          = L"\\pipe\\echo";
    unsigned int     cMinCalls  = 1;
    unsigned int     cMaxCalls  = 20;
    unsigned int     fDontWait  = FALSE;
    // Register the protocol sequence, endpoint
    status = RpcServerUseProtseqEp(pszProtocolSequence,
                                   cMaxCalls,
                                   pszEndpoint,
                                   pszSecurity);  // Security descriptor
    if (status != RPC_S_OK)
    {
        PrintError1("RpcServerUseProtseqEp returned 0x%lx\n", status);
        return;
    }
    // Register the interface
    status = RpcServerRegisterIf(
                            Echo_ServerIfHandle, // interface to register
                            NULL,                // MgrTypeUuid
                            NULL);          // null means use default EPV
    if (status != RPC_S_OK)
    {
        PrintError1("RpcServerRegisterIf returned 0x%lx\n", status);
        return ;
    }
    PrintInfo("Calling RpcServerListen\n");
    status = RpcServerListen(cMinCalls,
                            cMaxCalls,
                            fDontWait
                            );
    if (status != RPC_S_OK)
        PrintInfo1("RpcServerListen returned: 0x%lx\n", status);
}
BOOL PauseEchoServer()
{
    PrintInfo("Pausing the RPC Service is not supported\n");
    return(TRUE);
}
BOOL ResumeEchoServer().
{
    PrintInfo("Resuming the RPC Service is not supported\n");
    return(TRUE);
}
```

```
BOOL StopEchoServer()
{
     RPC_STATUS status;
     PrintInfo("Calling RpcMgmtStopServerListening\n");
     status = RpcMgmtStopServerListening(NULL);
     if (status != RPC_S_OK)
     {
          PrintError1("RpcMgmtStopServerListening error 0x%lx\n", status);
          return(FALSE);
     }
     PrintInfo("Calling RpcServerUnregisterIf\n");
     status = RpcServerUnregisterIf(NULL, NULL, FALSE);
     if (status != RPC_S_OK)
     {
          PrintError1("RpcServerUnregisterIf returned 0x%lx\n", status);
          return(FALSE);
     }
     return(TRUE);
}
//
//              MIDL allocate and free on server
//
void __RPC_FAR * __RPC_USER midl_user_allocate(size_t len)
{
     return(malloc(len));
}
void __RPC_USER midl_user_free(void __RPC_FAR * ptr)
{
     if(ptr)
     free(ptr);
}
//
//  File : ECHOP.C - Implements the function EchoProc() on Server
//
#include <stdlib.h>
#include <stdio.h>
#include "echo.h"     // header file generated by MIDL compiler
#include <printcmd.h>
static lPingCount = 0;
void EchoProc(char * pszString, char *pszOutputString)
{
     PrintInfoString("Client Sent: [%s]\n", pszString);
     sprintf(pszOutputString, "Ping#: %d [%s]", lPingCount++, pszString);
}
// FILE: ECHOC.C - Echo Client Implementation
//
#include <stdlib.h>
#include <stdio.h>
#include <ctype.h>
#include "echo.h"     // header file generated by MIDL compiler
#include <service.h>
#include <printcmd.h>
#define MAX_NETWORK_ADDR_LEN 20
void Usage(char *ptr)
{
     printf("%s -n <network address> -s <echo string>\n", ptr);
     exit(1);
}
```

```
void _CRTAPI1 main(int argc, char **argv)
{
     RPC_STATUS status;
     LPTSTR pszUuid                    = NULL;
     LPTSTR pszProtocolSequence        = L"ncacn_np";
     LPTSTR pszNetworkAddress          = NULL;
     WCHAR  wszNetworkAddress          [ MAX_NETWORK_ADDR_LEN];
     LPTSTR pszEndpoint                = L"\\pipe\\echo";
     LPTSTR pszOptions                 = NULL;
     LPTSTR pszStringBinding           = NULL;
     LPSTR  pszString                  = "Ping from Client";
     unsigned long ulCode;
     char szOutputString               [ MAX_OUTPUT_LEN ];
     int i;
     // allow the user to supply a network server address
     // or provide a new string override settings with command line
     // switches
     for (i = 1; i < argc; i++)
     {
          if ((*argv[i] == '-') || (*argv[i] == '/'))
          {
               switch (tolower(*(argv[i]+1)))
               {
               case 'n':  // network address
                    ++i;
                    mbstowcs ( wszNetworkAddress, argv[i],
                                        strlen(argv[i])+1);
                    pszNetworkAddress = &wszNetworkAddress[0];
                    break;
               case 's':
                    pszString = argv[++i];
                    break;
               case 'h':
               case '?':
               default:
                    Usage(argv[0]);
               }
          }
          else
               Usage(argv[0]);
     }
     // Create a string binding.
     status = RpcStringBindingCompose(pszUuid,
                                      pszProtocolSequence,
                                      pszNetworkAddress,
                                      pszEndpoint,
                                      pszOptions,
        &pszStringBinding);
     if (status != RPC_S_OK)
     {
          PrintError1("RpcStringBindingCompose returned 0x%x\n", status);
          exit(status);
     }
     else
          wprintf(L"pszStringBinding = %s\n", pszStrilngBinding);
```

```
    // Set the binding handle that will be used to bind to the server.
    status = RpcBindingFromStringBinding(pszStringBinding,
                        &Echo_IfHandle);
    if (status != RPC_S_OK)
    {
        PrintError1("RpcBindingFromStringBinding returned 0x%x\n",
                        status);
        exit(status);
    }
    PrintInfo("Calling the remote procedure 'EchoProc'\n");
    RpcTryExcept
    {    // make call with user message
        EchoProc(pszString, szOutputString);
        PrintInfoString("Server Returned %s\n", szOutputString);
    }
    RpcExcept(1)
    {
        ulCode = RpcExceptionCode();
        PrintError1("Runtime reported exception 0x%lx\n",  ulCode);
    }
    RpcEndExcept
    //  Free the string and the binding handle
    status = RpcStringFree(&pszStringBinding);
    if (status != RPC_S_OK)
    {
        PrintError1("RpcStringFree returned 0x%x\n", status);
        exit(status);
    }
    // Remote calls done. Unbind the handle
    status = RpcBindingFree(&Echo_IfHandle);
    if (status != RPC_S_OK)
    {
        PrintError1("RpcBindingFree returned 0x%x\n", status);
        exit(status);
    }
    exit(0);
}   // end main()
//
//              MIDL allocate and free on Client
void  __RPC_FAR * __RPC_USER midl_user_allocate(size_t len)
{
    return(malloc(len));
}
void __RPC_USER midl_user_free(void __RPC_FAR * ptr)
{
    free(ptr);
}
```

The Echo Server uses the Named Pipe protocol and a well-known endpoint (`"\PIPE\ECHO"`). It registers only one interface and waits, listening (and responding) to the clients. The client connects to the server by specifying the full server detail (network address, end point, and interface ID). After successful connection is established, the client sends a string to the server by EchoProc(). The server returns the string to the client after adding a count in the string.

4.11 RPC PROGRAMMING IN WINDOWS

This section focuses on any issues that a client/server system designer needs to address when implementing Windows 3.x-based RPC clients.

Recall that the RPC computing model is based on a client/server model. Thus, an RPC-based client calls a server-resident remote function to access certain services offered by the server. It then blocks, waiting for the server response. Only after the server has sent back a reply can the client continue operation. This model of computing does not work well in a Windows 3.x nonpreemptive (or cooperative multi-tasking) environment. In Windows 3.x, applications should never block on any function call unless the waiting time is both deterministic and short. Because the waiting times in networking are never deterministic, the client can "hang" the Windows system. So Microsoft RPC provides the Windows 3.x RPC client implementors with a special API, **RpcWinSetYieldInfo()**, which allows an RPC call to complete asynchronously. The prototype of the function is shown below:

```
RPC_STATUS      RpcWinSetYieldInfo(HWND   hWnd, BOOL  fCustomYield,
WORD wMsg, DWORD dwOtherInfo);
```


If a Windows application does not call this API, all remote functions (for example, client-side stubs, client-side RPC run time, and so on) use the underlying transport protocols in the synchronous mode. This will cause the application (and the entire Windows system) to block until the function is complete. This is handling an appropriate model even when the server is highly available and the network I/O time is short.

However, most Windows applications should use RpcWinSetYieldInfo() API. RpcWinSetYieldInfo() causes the RPC run-time to use the asynchronous services offered by the transport protocols (for example, asynchronous Named Pipes, asynchronous NetBIOS commands, non-blocking Windows Sockets, and so on) to send and receive data. Additionally, the RPC run-time yields control to the rest of the Windows system while it waits for the server's response.

To use the RpcWinSetYieldInfo() API, use the RPC-provided "standard yield" method. Do this by setting an application window handle (using the hWnd parameter) and by setting fCustomYield to FALSE before calling the RpcWinSetYieldInfo() API. This causes RPC to use a standard (or default) yielding technique whenever a remote procedure call is called. Additionally, the RPC run time displays a dialog box explaining that a remote function call is in progress. The dialog box also contains a cancel button so that a user can stop an RPC call in progress. While the dialog box is up, the application continues to process messages, but the user cannot give the application any further keyboard or mouse input. The user can switch to other

Table 4-8. WPARAM and LPARAM Settings

Yield Period	*WPARAM*	*LPARAM*
Beginning	1	unused
Ending	0	unused

applications (because RPC yields for the RPC-based client). An application can supply a message (through the wMsg parameter) that is sent by RPC before and after entering a yield period. The value of the message must be greater than WM_USER. When this message is received by the window whose handle has been supplied to the RpcWinSetYieldInfo() API, the WPARAM and LPARAM parameters are set in Table 4.8.

In addition, when calling the RpcWinSetYieldInfo() API, an application can supply a handle to a dialog box template (using the dwOtherInfo parameter), which is used by the RPC as the second parameter to the DialogBoxIndirect() Windows API. This allows the application to customize the dialog box presented to the user whenever a remote function called by the client does not complete immediately.

Finally, an application may choose to use its own "custom yield" implementation. For this, set the fCustomYield parameter to TRUE and supply a procedure-instance (see MAKEPROCINSTANCE Windows macro) to an application-supplied callback function by using the dwOtherInfo parameter. The RPC calls an application-supplied function whenever a remote operation is in process. Thus, the application-supplied function must yield by retrieving the messages from the application message queue during the yield period. The following code fragment shows a custom yield function that assumes that the client-called **RpcWinSetInfo()** supplied a message of (WM_RPC_CUSTOM-_YIELD). The WIN32 SDK contains a yield sample application (in the \mstools\samples\rpc\yield directory), which demonstrates the use of custom functions.

```
#ifndef WM_RPC_CUSTOM_YIELD
#define WM_RPC_CUSTOM_YIELD (WM_USER + 101)
#endif
BOOL FAR PASCAL __export YieldFunction(void)
{
     MSG msg;
     // Assume a global variable which gets set to TRUE if
     // user hits CANCEL button via a dialog box provided by
     // this application.
     fCancel = FALSE;
     while (TRUE) // message processing
     {
          GetMessage(&msg, (HWND)NULL, 0, 0);
               // Check if the mesage is from RPC, telling us
               // that yielding is over
```

```
            if (msg.message == WM_RPC_CUSTOM_YIELD)
                 return(TRUE);  // RPC operation is complete
                 // Check if user hit CANCEL button. If yes,
                 // tell RPC to abort the remote operation
            if (fCancel == TRUE)
                 return(FALSE);
            TranslateMessage(&msg);
            DispatchMessage(&msg);
       }
       return(TRUE);
}
```

The RpcWinSetInfo() API can be called multiple times. The information supplied by each such call supersedes those supplied by the previous call.

4.12 SUMMARY

This chapter presented in depth the concepts and use of Microsoft RPC in building the communication infrastructure of client/server systems. It introduced the data structures supported by RPC that can be used to transmit integers to linked lists between the client and the server. Then it illustrated how clients connect with a server by supplying information about that server—its network address, end point, and protocol sequence. On successful binding, the client obtains a logical binding handle that can be implicitly or explicitly used to call the server-resident functions. The chapter also presented autobinding handles to application-managed context handles. The chapter discussed how the client can use Windows NT Name Service to locate any server within a Windows NT domain, provided the details about the server were exported into the Name Service. It concluded with a Win32 service (Echo Server) code example that showed how a service could use RPC to communicate with its clients.

Chapter Five

WINDOWS SOCKETS IN WINDOWS NT

This chapter presents the concepts and use of the Windows Sockets interface in Windows NT. We begin with a historical perspective of Windows Sockets and then present its functional architecture. Then, we introduce client/server network programming with Windows Sockets, illustrating salient features via sample programs. Throughout the chapter, we specifically address techniques for designing efficient Windows Socket-based server applications. Additionally, interoperability between Windows Socket-based applications and IPX/SPX-based applications is discussed. We end the chapter with an illustration of a Windows Socket-based Win32 Service (Echo Server) first introduced in Chapter 2. We should point out that a special section has been dedicated to programming considerations for using Windows Sockets in the Windows 3.x environments.

In 1982, Berkeley Software Distribution's 4.1c BSD system for the VAX introduced sockets as an interface for communication between local processes. In 1986, the Berkeley UNIX version 4.3 BSD extended the sockets interface for use over the TCP/IP and XNS protocols for IPC. This extension allowed two processes to communicate with each other, whether they were present on the same computer or connected by the Internet. Soon, most UNIX vendors supported the socket interface, and such wide acceptance made the sockets interface a standard of network computing in the UNIX environment. Today, many applications depend on these interfaces. With the popularity of Windows, various proprietary packages (such as sockets libraries and TCP/IP drivers) were developed so users could run their favorite sockets-based applications (FTP, VTP, and so on). However, because there was no standard socket interface in the Windows environment, there was minimal operation of socket-based Windows applications over different socket and TCP/IP implementations.

A consortium of companies, including Microsoft, proposed an industry-wide socket standard for the Windows 3.x and Windows NT environments. We call this standard the Windows Sockets Interface or WinSock. WinSock standardizes the application programming interfaces and defines a binary interface to which all WinSock providers must adhere. The specification defines

the library calls and associated semantics to which an application developer can program and that a network software vendor can implement. The result is that applications written to the WinSock interfaces will work across software libraries provided by multiple vendors (because of standard programming interfaces) and across protocol suites provided by any WinSock-compliant vendor (due to the binary interface).

Figure 5-1 shows the interrelationship between WinSock and the underlying protocols. As the figure shows, Windows Sockets-based applications use the WinSock interface to access the default Windows NT (or Windows) WinSock implementation (contained in WinSock.DLL) or an alternate implementation (for example, the FTP WinSock.DLL). These WinSock implementations may support a number of underlying protocols. For example, Windows NT (and Windows) WinSock implementations can provide access to TCP/IP, IPX/SPX, (implemented in Microsoft NWLink driver) AppleTalk®, NetBIOS, OSI TP4, and Digital DecNet®, among others. Notice that WinSock can also be used to access a remote system, using the Windows NT **Remote Access Service (RAS)**.

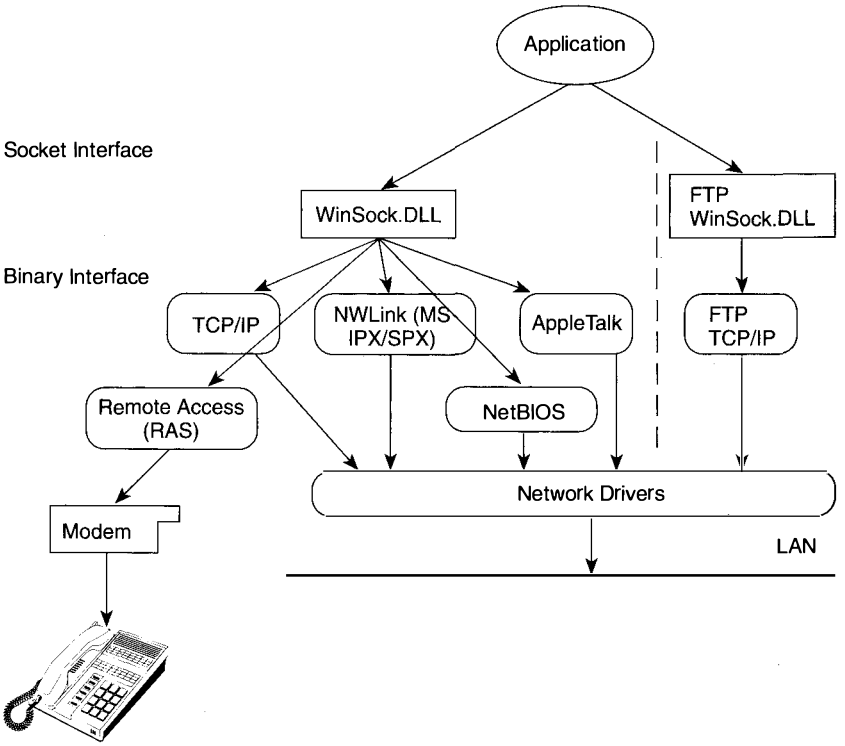


Figure 5-1. *Windows Socket, Protocols, and Applications*

You may wonder why WinSock is available over non-TCP/IP protocols. The reason is that while Windows Sockets was designed to provide a Berkeley-style socket interface over TCP/IP in Windows and Windows NT environments, it has quickly become the standard network programming interface over multiple protocols in those environments. In fact, Microsoft encourages network software developers to write WinSock interfaces instead of accessing individual lower-level interfaces, such as NetBIOS, AppleTalk, or IPX/SPX. This single standardized interface model has two interesting points:

- A client/server application can access any of the underlying protocols through the WinSock interface. Indeed, users may have to make certain special calls for a selected protocol. For example, the applications using WinSock over Microsoft IPX/SPX stack (that is, the NWLink) may need to set IPX header information.
- A client/server designer can develop a single communication library (APIs, C++ classes, and so on) that will work across multiple protocols, such as AppleTalk or IPX/SPX. Contrast this with having to develop a communication library for each protocol. With WinSock, there is definitely a great potential for code size reduction.

5.1 INTRODUCTION TO SOCKETS PROGRAMMING

The term "socket" has several definitions. One defines a socket as the programming interface (that is, a collection of function calls that allow interprocess communication). Another definition calls it a logical construct, much like a file handle, that an application uses for communicating through the socket interface. This chapter refers to the first definition as the socket interface and the second as the socket.

While WinSock is not restricted to TCP/IP transport protocol, discussion of sockets programming often intertwines with the details of TCP/IP. For basic information on TCP/IP see [Comer 91a, 91b] [Stevens 90]. Recall from Chapter 1 that a connection-oriented protocol allows data to be sent over a virtual circuit (a data stream) such that the data is delivered to the receiver in the order in which it was transmitted. Also most connection-oriented protocols guarantee data delivery. Examples of such protocols are TCP and SPX. A connectionless protocol sends each data packet independent of either the preceding or the following packet. Datagram services do not attempt to maintain data order. More often than not, they do not guarantee data delivery. When using connectionless protocols such as UDP and IPX, applications must have their own mechanisms to ensure that data is correctly delivered to the receiver.

A fully connected socket represents a logical communication channel between two processes. The communication channel can be bidirectional and is either stream- or datagram-oriented, depending on whether the underlying protocol provides connection-oriented or connectionless service. A connected socket can be described as follows:

{protocol, local-address, local-process, foreign-address, foreign-process}

Both the local and the foreign process must select a common network protocol (for instance, TCP for connection-oriented IPC). Both processes must have a definitive network address by which they can be uniquely identified within the network. Additionally, they must have unique names (or identifiers) so they can be located among all the other processes on a given machine. To set up such a relationship, each process first creates its own part of the socket infrastructure, called endpoints, which consist of {protocol, local address, local process}. Once the endpoints have been created by both processes, the processes can call one or more socket interfaces to connect the two endpoints, and a connected socket results. After a socket is connected, the connected processes can use socket functions, such as **send()** and **recv()**, to send or receive data. In a Windows NT environment, the socket is a valid file handle upon which system primitives [such as ReadFile()/WriteFile()] can be applied to send or receive data. Finally, both processes disconnect, each closing its end of the socket by calling one or more socket interfaces.

In a client/server paradigm, a server must first create its endpoint by calling the socket() API, as shown in Figure 5-2. Next, it associates, or binds, the socket with the network address of the server by calling the **bind()** API. Binding the socket to an address creates the server endpoint. Any client that attempts to connect to the server endpoint will not succeed until the server sets its socket to a "listening" state. The server does this by calling the function **listen()**, which establishes a queue of incoming client calls. Then the server calls the **accept()** API to begin receiving incoming calls. Before calling the **connect()** function, the client must have created its endpoint by calling the **socket()** API, and may associate its own address with the client endpoint by calling the bind() function. The client can now connect to the server by calling the connect() API. The client must specify the name (or the address) of the server to the connect() function. The connect() API uses the underlying protocols to locate a server that matches the client's requests, and attempt to connect server and client endpoints. When a connection is established, the client and the server can both use functions, such as recv() and send(), to receive and send data to each other.

The most obvious differences between Berkeley and Windows Sockets is that in 4.3BSD, the socket is programmatically defined by an *int* data type, whereas in the Windows Socket Interface, sockets are defined by a SOCKET data type. Programmers should check for SOCKET_ERROR on all functions, rather than looking for a negative one (–1). Finally, an application (or DLL)

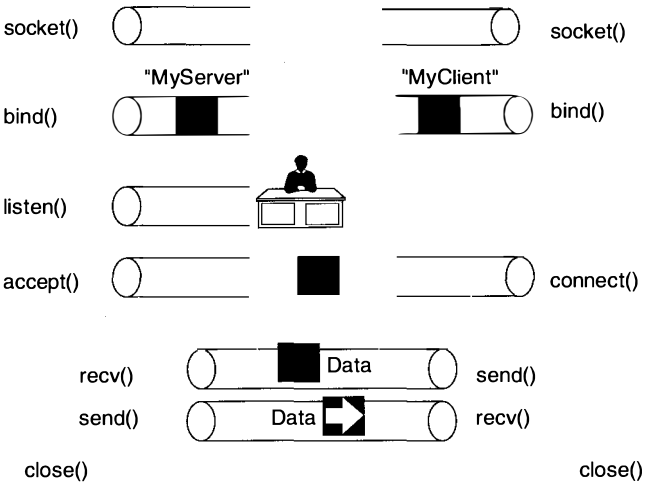


Figure 5-2. *Life Cycle of a Socket*

must call **WSAStartup()** Windows Socket API before calling any WinSock functions. It should also call the **WSACleanup()** API before terminating. The application uses the WSAStartup() API to negotiate the WinSock version required (1.0, 1.1, 2.0, and so on). These APIs are discussed later in this chapter.

5.1.1 Creating a Socket

Developing a socket-based application begins when the application creates a socket by calling the socket() API. On successful completion of this function, the application has established its own endpoint. Here is the socket() call prototype:

```
SOCKET PASCAL FAR socket (int af, int type, int protocol);
```

The application then receives a socket descriptor, which can be used for further activities. At this point, a valid socket endpoint has been created. When the returned socket descriptor value equals INVALID_SOCKET, an error has occurred. The application can determine the cause of the error by calling the **WSAGetLastError()** API; the WSAGetLastError() API maps to the GetLastError() API on Win32 environments.

The user specifies the namespace or address family within which the application operates (using the af parameter). Specify zero (0) to select the default address family. Each address family has its own representation of names and addresses, so it is important to specify the family being used. Once an address family has been selected, the application must present to the socket interface all network addresses in a syntax that complies with that of the selected address family. Some legal values of an address family are shown

below. Note that you can use AF_ and PF_ (protocol family) prefixes interchangeably. The address syntax for some interesting address families is shown below; an address syntax for AF_UNIX also appears for comparison, although it is not supported by WinSock.

```
//
// AF_UNIX - Namespace for local IPC on UNIX
//
struct sockaddr_un {
    short     sun_family;          // 2 bytes
    char      sun_path[108];       // 108 bytes
};
//
// AF_INET - Namespace for IPC in Windows, Windows NT, and Internet
//
struct in_addr {
    u_long S_addr;
};
struct sockaddr_in {
    short   sin_family;            // 2 bytes
    u_short sin_port;              // 2 bytes port number
    struct  in_addr sin_addr;      // 4 bytes net ID, host ID
    char    sin_zero[8];           // 8 bytes unused
};
//
// AF_IPX - Namespace for NetWare and Windows/Windows NT[1] namespace.
//
typedef struct sockaddr_ipx {
    short sa_family;                    // 2 bytes
    char  sa_netnum[4];                 // 4 bytes net ID
    char  sa_nodenum[6];                // 6 bytes node (host) ID
    unsigned short sa_socket;           // 2 bytes socket number
} SOCKADDR_IPX, *PSOCKADDR_IPX,FAR *LPSOCKADDR_IPX;
//
// AF_NETBIOS - NetBIOS name space
//
#define NETBIOS_NAME_LENGTH 16
typedef struct sockaddr_nb {
    short   snb_family;                   // 2 bytes
    u_short snb_type;                     // 2 bytes
    char    snb_name[NETBIOS_NAME_LENGTH]; // 16 bytes
} SOCKADDR_NB, *PSOCKADDR_NB,FAR *LPSOCKADDR_NB;
```

Having selected the address family, specify the type of socket (using the type parameter); use zero (0) to pick the default socket type. The type of socket defines the type of service expected from the socket (for example, connectionless or connection-oriented communication). Table 5-1 lists all socket types, although WinSock only supports the first two.

1. When IPX/SPX protocol is running on Windows or Windows NT.

Table 5-1. *Socket Types*

Type of Socket	*Description*
SOCK_STREAM	Stream or connection-oriented socket; supported by WinSock.
SOCK_DGRAM	Datagram or connectionless socket; supported by WinSock.
SOCK_RAW	Raw or data-transmitted-as-is with little error control socket
SOCK_SEQPACKET	Sequenced packets socket
SOCK_RDM	Reliably delivered message sockets

The third parameter supplied to the socket() call is the protocol. Specify the protocol to be used by the socket library (using the protocol parameter). Use zero (0) to pick the default protocol. When the caller specifies SOCK_STREAM as the type of socket and zero (0) for the protocol, the socket library will use the TCP protocol by default, because TCP provides guaranteed stream data service. On the other hand, specifying SOCK_DGRAM as the socket type and zero (0) for the protocol causes the UDP protocol to be used, because UDP provides datagram-based nonguaranteed data delivery service. Table 5-2 shows the most common combinations of address family, socket type, and protocols supported by Windows Socket libraries.

Table 5-2. *Common Combinations Supported by WinSock Libraries*

Address Family	*Type*	*Protocol*	*Actual Protocol*
AF_INET	SOCK_DGRAM	IPPROTO_UDP	UDP
AF_INET	SOCK_STREAM	IPPROTO_TCP	TCP
AF_IPX	SOCK_DGRAM	NSPROTO_IPX	IPX
AF_IPX	SOCK_STREAM	NSPROTO_SPX	SPX
AF_IPX	SOCK_STREAM	NSPROTO_SPXII	SPX II
AF_NETBIOS	SOCK_DGRAM	NBPROTO_NETBEUI	NetBEUI
AF_NETBIOS	SOCK_STREAM	NBPROTO_NETBEUI	NetBEUI
AF_OSI	SOCK_STREAM	ISOPROTO_TP4	OSI TP4

The following code fragment shows an application creating and then closing the socket. Notice that all socket function calls are encapsulated between the WSAStartup() and the WSACleanup() APIs. This chapter refers often to the **WinSockIntialize()** function.

```
//
// Create a socket
//
#define RETURN_ON_ERROR(err)
    {\
    WSACleanup();\
    return (err);\
}
int WinSockInitialize()
{
    int err;
    WORDxVersionReq;
    WSADATA wsaData;
                    // All applications must call WSAStartup()
                    // before calling any socket functions
    wVersionReq = MAKEWORD (1, 1);
    err = WSAStartup(wVersionReq, &wsaData);
    if (err != 0)
        exit(1);
}
int main (int argc, char **argv)
{
    SOCKET soc;
                    // initialize WinSock library
    WinSockInitialize();
                    // Create a stream socket and (by default) use the TCP
                    // protocol
    soc = socket ( AF_INET, SOCK_STREAM, 0);
    if (soc == INVALID_SOCKET)
        RETURN_ON_ERROR(1);
    < ..further processing ..>
    // Close the socket
    closesocket ( soc );
    // Cleanup
    WSACleanup();
    return (0);
}
```


5.1.2 Associating Address with a Socket or Binding

To form a connected socket between two processes, both processes must create their own endpoints by calling the socket() API, then one of them listens to incoming requests, and the other tries to connect. A socket-based application can be contacted by another socket-based application only after the first application has associated its local address with its endpoint. In a client/server model, a server always waits for the client to connect to it. The server must therefore associate its network addresses with the socket endpoints created by the socket() function. A client can associate its network address with its own endpoint.

The action of associating a network address with the socket is referred to as "binding," and is performed by calling the bind() function. When binding is completed, the socket endpoint becomes a named entity visible in the namespace. This allows the endpoint, and thus the server, to be addressed or to be an object of search. Typically, a server uses the network address of the

host computer as its own address, so the server can be located by any client that knows the network address of the remote computer. For example, a server running on a computer whose assigned TCP/IP address is (11.1.23.67) also has the address (11.1.23.67). The client can specify (11.1.23.67) as the server's address. The prototype for the bind() function is shown here:

```
struct sockaddr {
    u_short sa_family;          // address family
    char    sa_data[14];        // up to 14 bytes of direct address
};
struct in_addr {
    u_long S_addr;
};
struct sockaddr_in {
    short   sin_family;         // 2 bytes
    u_short sin_port;           // 2 bytes port number
    struct  in_addr sin_addr;   // 4 bytes net ID, host ID
    char    sin_zero[8];        // 8 bytes unused
};
int PASCAL FAR
bind ( SOCKET sock, const struct sockaddr FAR *address, int addrlen);
```

The bind() function takes the socket descriptor (using the sock parameter), which must have been previously obtained by calling the socket() function. The address of this socket endpoint can be specified by the address parameter with the addrlen parameter containing the length (in bytes) of the complete address structure. A zero return value indicates successful completion of the call; otherwise SOCKET_ERROR is returned. In the latter case, the WSAGetLastError() API can be used to obtain extended error information.

Note that the address of the socket must be contained within fourteen bytes, while two bytes are used for detecting address family. The format of the addresses differs, depending on the address family chosen. The previous section illustrated the addressing formats for the most common addressing families, such as AF_INET, AF_IPX, AF_NETBIOS, and AF_OSI. While it is beyond the scope of this book to discuss the different addressing formats, we will discuss the address structure (sockaddr_in) for the AF_INET address family as it is used with the TCP/IP protocols (thus most popular).

The Internet address for a socket-based application is composed of two pieces, namely a two-byte port number (sin_port) and a four-byte network address (sin_addr). The network address uniquely identifies one machine among all other machines on the Internet. A port number identifies the process associated with a specific socket on a specific machine. Thus, a machine with the network address of (11.1.121.122) can have two processes, each with their unique port numbers (1025 and 1026).

The port numbers are 16-bit integers associated with a given application (or service) for a specific protocol. Port numbers in the [0,1023] range have been reserved for standard TCP/IP services (FTP, Finger, EchoServer, and so on); you

can find the list of reserved ports in (%SYSTEMROOT %\drivers\etc\services). New applications can use any other port number outside the reserved range. Another way to pick a port number is to ask the system to allocate one to the application at run-time. Do this by specifying zero(0) or INADDR_ANY as the port number when you supply the local address to the bind() function, and the system will assign a port number out of the [1024, 5000] range.

Note that in typical client/server applications, only the server port is important at application level. Recall that in the client/server computing model, the client always attempts to connect to the server by supplying its full address, that is, the port number and network address for TCP/IP-based WinSock applications, and not the other way around. This means that the server port number must be unique for each protocol; the client needs to know the server port number to be able to connect to it; and, while the client port number is used by the underlying system, it is not needed at the application level. In other words, the server needs to have a well-known port number, whereas the client can ask the WinSock drivers to allocate a port number at run time.

The Internet network address can be specified in any one of four formats, depending on the "class" of the address, namely, A, B, C, or D. Figure 5-3 shows the four formats.

For A, B, and C, each address has a netid and a hostid. Netid is used to identify the Internet segment on which the machine resides. Hostid defines the address of the computer. Notice that the netid size grows from seven bits in class A to 21 bits in class C, while the hostid size reduces. This scheme allows both small networks with large numbers of hosts (class A) and large networks with small numbers of hosts (class C). The class D address is used for communicating with a group of applications on the Internet by multicast datagrams. Note that a machine may have multiple network adapters and thus have

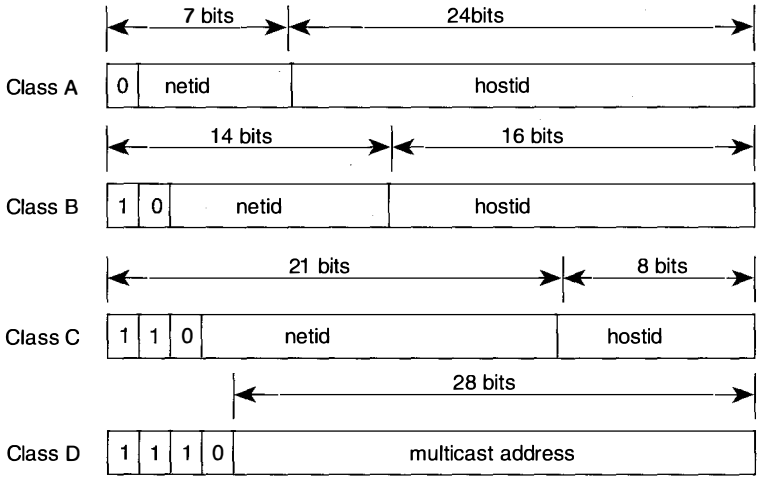


Figure 5-3. *Internet Address Formats*

multiple netids. In such a case, a WinSock application is termed multi-homed, implying that such an application can claim multiple network addresses.

Most applications and users specify the four-byte (32-bit) network address by a string containing four decimals (one per byte) separated by dots (called the dotted address). For example, an address 0x0102FF04 is written as (1.2.255.4). However, the network functions need the address as a 32-bit number, in network byte order. For this purpose, the applications can use the **inet_addr()** API to convert a dotted address (string) to the network address (32-bit number) and the **inet_ntoa()** API for opposite conversion. The prototypes and an example appear here.

```
// prototypes
unsigned long PASCAL FAR   inet_addr (const char FAR * cp);
char FAR * PASCAL FAR      inet_ntoa (struct in_addr in);
//
// Usage of inet_addr() and inet_ntoa() APIs
//
char  szFixedAddress [] = {"1.2.255.4"};
ulong ulNetAddress;
LPSTR pszDottedAddress;
struct in_addr inAddress;
// Convert a dotted network address to a 32-bit number
ulNetAddress = inet_addr ( szFixedAddress );
// Convert a 32-bit network address to dotted address
inAddress.S_addr = ulNetAddress;
pszDottedAddress = inet_ntoa( &inAddress );
ASSERT( 0 = strcmp(pszDottedAddress, szFixedAddress));
```

Before passing the network address, the port number, formats, and conventions to the bind() function, note that the application must ensure that the address is in network byte order (bytes ordered left to right). The inet_addr() API discussed earlier returns the number in the network byte order. The byte-ordering routines handle potential disparities between different computer architectures and multiple network protocols. Thus, for handling heterogeneous machine architectures, all addresses actually sent and received over the network must be specified in network byte order (which is the same as big endian order).

The following list shows the byte-ordering routines that are used to convert long (32-bit) and short (16-bit) numbers from the host computer to Internet format and vice-versa. The routines handling long numbers are used to convert network addresses, and the functions handling short numbers are used with port IDs. The hton prefix of the following functions indicates the conversion of numbers from the host-specific format to the network byte order and ntoh prefix indicates the opposite conversion.

```
u_long   PASCAL FAR       htonl    (u_long hostlong);
u_short  PASCAL FAR       htons    (u_short hostshort);
u_long   PASCAL FAR       htohl    (u_long netlong);
u_short  PASCAL FAR       ntohs    (u_short netshort);
```

The following code fragment shows a client binding to a socket. This shows the simplest way to call the bind() function. The client specifies INADDR_ANY as its network address. This allows the client to assume the network address of the host computer. It also specifies zero as the port number, which causes the WinSock library to allocate a port to the client from the [1024, 5000] range.

```
//
// Client binding a socket
//
// (Some functions and macros omitted when explained earlier)
int main (int argc, char **argv)
{
    SOCKET soc;
    struct in_addr ClientAddr;
    // initialize WinSock library
    WinSockInitialize();
    // Create a stream socket and (by default) use the TCP protocol
    soc = socket ( AF_INET, SOCK_STREAM, 0);
    if (soc == INVALID_SOCKET)
        RETURN_ON_ERROR(1);
    // Bind an address to Client side endpoint
    // Notice the INADDR_ANY which allows
    // Client to assume the network address of the host machine
    memset ( &ClientAddr, 0x00, sizeof(ClientAddr));
    ClientAddr.sin_family = AF_INET;
    ClientAddr.sin_addr.s_addr = htonl(INADDR_ANY);
    ClientAddr.sin_port    = 0;
    if ( bind(soc, &ClientAddr, sizeof(ClientAddr)) == SOCKET_ERROR)
    {
        printf("bind error: %lx\", WSAGetLastError());
        RETURN_ON_ERROR(1);
    }
    < ..further processing ..>
    // Close the socket
    closesocket ( soc );
    // Cleanup
    WSACleanup();
    return (0;
}
```

In WinSock, many database functions are available for obtaining the network address of either a local or remote server. These database functions can also be used to determine the port number associated with a server or to look up the logical names of machines and services. For example, you can query for the network address of a machine named "Phoenix," and ask for the port number for an application named "FaxServer" for the TCP protocol. The prototype of the common database functions follows:

```
//
// Structures used by database functions
```

```
//
struct  hostent { // Host Information
    char        FAR * h_name;               // official name of host
    char        FAR * FAR * h_aliases;      // alias list
    short       h_addrtype;                 // host address type
    short       h_length;                   // length of address
    char        FAR * FAR * h_addr_list;    // list of addresses
#define h_addr   h_addr_list[0]             // address, for backward
                                            // compatibility

};
struct  netent { // Network information
    char        FAR * n_name;               // official name of net
    char        FAR * FAR * n_aliases;      // alias list
    short       n_addrtype;                 // net address type
    u_long      n_net;                      // network #
};
struct  servent { // Service Information
    char        FAR * s_name;               // official service name
    char        FAR * FAR * s_aliases;      // alias list
    short       s_port;                     // port #
    char        FAR * s_proto;              // protocol to use
};
struct  protoent { // Protocol Information
    char        FAR * p_name;               // official protocol name
    char        FAR * FAR * p_aliases;      // alias list
    short       p_proto;                    // protocol #
};
//
// Database Functions
//
struct hostent FAR * PASCAL FAR
    gethostbyaddr(const char FAR * addr,int len, int type);
struct hostent FAR * PASCAL FAR
    gethostbyname(const char FAR * name);
int PASCAL FAR
    gethostname (char FAR * name, int namelen);
struct servent FAR * PASCAL FAR
    getservbyport(int port, const char FAR * proto);
struct servent FAR * PASCAL FAR
    getservbyname(const char FAR * name, const char FAR * proto);
struct protoent FAR * PASCAL FAR
    getprotobynumber(int proto);
struct protoent FAR * PASCAL FAR
    getprotobyname(const char FAR * name);
```

These database functions are most frequently used to determine network address and port numbers in TCP/IP environments. The data for these APIs is kept in the hosts and services files in Windows and Windows NT. The host file contains the addresses of remote computers and gateways, and the service file contains the port number for any service (application) for a given protocol. In Windows NT, these files are in the (%SYSTEMROOT%\drivers\etc) directory. A sample of these files follows. Note that applications can use other (local or remote) name services to obtain addressing information. These methods are covered later.

```
# sample host file for TCP/IP protocol
# Format:
# IP address      Internet name      comments
#
102.54.94.97      rhino.acme.com     # source server
38.25.63.10       x.acme.com         # x client host
127.0.0.2         EchoServer         # a made up number for testing
# sample services file
# Format:
# <service name>  <port number>/<protocol>  [aliases...]   [#<comment>]
echo              7/tcp
echo              7/udp
qotd              17/tcp                    quote
EchoServer        5001/tcp                          #added for testing
```

The following sample program demonstrates these key features:

- By default (DEFAULT_PORT_ADDR option), the program creates a stream socket in the AF_INET address family, and, again by default, uses the TCP/IP protocol. Thus, the program binds to the local machine address by specifying INADDR_ANY. By specifying a zero as the port number, it asks the system to allocate a port number. This is a common bind option for the client end of sockets.
- The next bind option (FIXED_PORT_LOCAL_ADDR) shows the option most commonly used by servers using sockets. The program binds the socket to a local host address and a fixed port number (1045). Notice that the port number is translated from host order to network byte order by calling the **htons()** API. The program determines the local host name by calling the **gethostname()** API, then determines the local host address by calling the **gethostbyname()** API. The gethostbyname() API returns the network address in network byte order, then the user calls the inet_ntoa() API to display the machine address in the dotted address format. Note that binding the socket to the network address has the same effect as binding the socket to INADDR_ANY. Why use the gethostname() and the gethostbyname() APIs? To demonstrate how to use APIs. More importantly, because you will use the gethostbyname() API later to obtain the network address of remote servers.
- Often, TCP/IP-based applications determine the port number at run-time, which adds flexibility in the configuration of such applications. In this case, the port number is specified in the (%SYSTEMROOT%\drivers\etc\services) files and obtained at run-time by calling the **getservbyname()** API. Use the (GET_PORT _AND_LOCAL_ADDR) option for this.
- The (BIND_NETBIOS) option shows how a datagram socket is created in the NetBIOS name space. Use the getservbyname() API

to determine the port number. The service name is registered as a unique NetBIOS name.

- Similarly, the (BIND_IPX) option allows the user to create a datagram socket using the IPX protocol. Note that in an IPX environment, the network address is composed of two elements, the network number (four bytes) and the node number (six bytes).

```
//
// Binding a Socket using database functions
// (corresponds to %BOOK%\winsock\bind.c and sockutil.c)
//
//
////////////////////////////////////////////////////
// FILE: Sockutil.h
// (certain parts continued in the following sections)
//
#ifndef _SOCK_UTIL_H_
#define _SOCK_UTIL_H_
#include <windows.h>
#include <winsock.h>
#include <wsnetbs.h>
#include <wsipx.h>
#include <stdio.h>
// Global Macros
// Bind options
#define DEFAULT_PORT_ADDR          1
#define FIXED_PORT_LOCAL_ADDR      2
#define GET_PORT_AND_LOCAL_ADDR    3
#define BIND_NETBIOS               4
#define BIND_IPX                   5
#define GET_PORT_AND_REMOTE_ADDR   6
// Defaults
#define FIXED_PORT                 1045
#define CLIENT_QUEUE_SIZE          5
#define READ_WRITE_LOOP            1
#define SHUTDOWN                   "ShutDown"
int WinSockInitialize();
int BindSocketToAddr (int               iBindType,
        SOCKET        soc,
        LPSTR         pszServiceName,
        LPSTR         pszProtocolName
                      );
u_short   GetPortForService(LPSTR pszServiceName, LPSTR pszProtocol);
u_long    GetLocalHostAddress();
u_long    GetHostAddress(LPSTR pszHostName);
void      PrintAddressDetails (LPSTR pClientAddr, int iAddrLen );
int       ConnectToServer (int iBindType, SOCKET soc,
                           LPSTR szHostName, LPSTR szServiceName,
                           LPSTR szProtocol);
#endif // _SOCK_UTIL_H_
////////////////////////////////////////////////////
// FILE: Sockutil.c
// (certain parts continued in the following sections)
//
```

```
#include "sockutil.h"
#include <wsnwlink.h>
// Global
SOCKADDR_IPX defIPXAddress  =  { AF_IPX,  // family
                                 // Net Number
                                 {0x00, 0x00, 0x00, 0x00},
                                 // Node Number
                                 {0x00, 0x00, 0x00, 0x00, 0x00, 0x00},
                                 // A Fixed Socket number
                                 0x4040
                                 };
//
// Initialize WinSock library
//
int WinSockInitialize()
{
    int       err;
    WORD      wVersionRequired;
    WSADATA   wsaData;
    wVersionRequired = MAKEWORD(1,1);
    err = WSAStartup(wVersionRequired, &wsaData);
    if (err != 0)
        exit(1);
    return(0);
}
//
// Get local or remote host name
//
u_long GetHostAddress(LPSTR pszHostName)
{
    struct hostent FAR *pHostAddr;
    struct in_addr in;
    //
    // get local host address
    //
    pHostAddr = gethostbyname ( pszHostName );
    if (pHostAddr == NULL)
    {
          printf("gethostbyname error: %ld for %s\n",
                      WSAGetLastError(), pszHostName);
          return(0);
     }
     else // print out details
     {
        printf("Host Name: %s\n",            pHostAddr->h_name);
        printf("Address Type: %x\n",         pHostAddr->h_addrtype);
        printf("Address Length: %d\n",       pHostAddr->h_length);
        printf("Host Address: %lx \n",       pHostAddr->h_addr);
        in.s_addr = *((u_long *)             pHostAddr->h_addr);
        printf("Dotted Address: %s\n", inet_ntoa(in));
    }
    return *((u_long *)pHostAddr->h_addr);
}
//
// Get Local Host Address
//
```

```
// This may cause lookup in %SYSTEMROOT%\drivers\etc\hosts file
u_long GetLocalHostAddress()
{
    char     szHostName [ MAX_PATH ];
    u_long ulHostAddr;
    if (gethostname (szHostName, sizeof(szHostName)) != SOCKET_ERROR)
    {
        printf("Local Host Name: %s\n", szHostName);
        ulHostAddr = GetHostAddress ( szHostName );
    }
    else
    {
        printf("gethostname error: %ld\n", WSAGetLastError());
        return(0);
    }
    return (ulHostAddr);
}
//
// Get port number for a local service
//
// This causes lookup in %SYSTEMROOT%\drivers\etc\services file
u_short GetPortForService(LPSTR pszServiceName, LPSTR pszProtocol)
{
    u_short usPort;
    struct servent FAR* pServAddr;
    pServAddr = getservbyname(pszServiceName, pszProtocol);
    if (pServAddr != NULL)
    {          // Print Service Details
               printf("\nService Name: %s\n", pServAddr->s_name);
               printf("Service Port: %d\n", ntohs(pServAddr->s_port));
               printf("Service Protocol: %s\n", pServAddr->s_proto);
               usPort = pServAddr->s_port;
    }
    else
    {
        printf("getservbyname error: %ld\n", WSAGetLastError());
        return ((u_short)-1);
    }
    return(usPort);
}
//
// Bind a socket to the host address and service port
//
#define PRINT_ERROR_AND_RETURN(s,err) {\
    printf("Error: %d in %s\n", err, s);\
    return(err);\
}
int BindSocketToAddr (int     iBindType,
                      SOCKET  soc,
                      LPSTR   pszServiceName,
                      LPSTR   pszProtocolName,
                      BOOL    fClient
                      )
{
    struct sockaddr_in   LocalAddr;
    SOCKADDR_NB   NetbiosAddr;
```

```
SOCKADDR_IPX  IpxAddr;
int                     err;
u_long                  ulHostAddress;
u_short                 usPortNo;
BOOL                    bReuse = TRUE;
char                    szNBName [ NETBIOS_NAME_LENGTH ];
//
// For TCP/IP connections, client asks the system to allocate
// a port and binds to INADDR_ANY
//
if (fClient)
{
    if ((iBindType != BIND_IPX) && (iBindType != BIND_NETBIOS))
        iBindType = DEFAULT_PORT_ADDR;
}
memset ( &LocalAddr, 0x00, sizeof(LocalAddr));
err = SOCKET_ERROR;
switch (iBindType)
{
case DEFAULT_PORT_ADDR:
    //
    // Bind to local host address and get system to
    // allocate a port number
    //
    LocalAddr.sin_family = AF_INET;
    LocalAddr.sin_addr.s_addr = htonl(INADDR_ANY);
    LocalAddr.sin_port  = 0;
    break;
case FIXED_PORT_LOCAL_ADDR:
    //
    // Bind to the local host address and a fixed port
    //
    if ((ulHostAddress = GetLocalHostAddress()) == 0)
    {
        PRINT_ERROR_AND_RETURN("GetLocalHost", err);
    }
    LocalAddr.sin_family = AF_INET;
    LocalAddr.sin_addr.s_addr = ulHostAddress ;
    LocalAddr.sin_port = htons(FIXED_PORT);
    break;
case GET_PORT_AND_LOCAL_ADDR:
    //
    // Bind to the local host address and a port for local service
    //
    if ((ulHostAddress = GetLocalHostAddress()) == 0)
    {
        PRINT_ERROR_AND_RETURN("GetLocalHost error", err);
    }
    else
    {
        usPortNo = GetPortForService(pszServiceName, pszProtocolName);
    }
    LocalAddr.sin_family = AF_INET;
    LocalAddr.sin_addr.s_addr = ulHostAddress ;
    LocalAddr.sin_port  = usPortNo;
    break;
```

```
    case BIND_NETBIOS:
       //
       // Bind the socket to a NetBIOS address
        usPortNo = GetPortForService(pszServiceName, pszProtocolName);
        if (fClient)
        {
               // We make up a fictitious name as client name does not
               // matter (in this example)
               pszServiceName = "WSockClient";
        }
        // see <wsnetbs.h> for expansion of the following macro
        memset (szNBName, 0x00, sizeof(szNBName));
        memcpy (szNBName, pszServiceName, strlen(pszServiceName));
        SET_NETBIOS_SOCKADDR(&NetbiosAddr,
            NETBIOS_UNIQUE_NAME,szNBName,(char) usPortNo);
        break;
    case BIND_IPX:
        // First copy the basic default address template
        memcpy (&IpxAddr, &defIPXAddress, sizeof(SOCKADDR_IPX));
        // Now, get the node number set in 'szServiceName'
        memcpy (&IpxAddr.sa_nodenum, pszServiceName, 6);
        // Now, get the net number set in 'szProtocolName'
        memcpy (&IpxAddr.sa_netnum, pszProtocolName, 4);
        // If this is a client, bind it to socket number
        // different from that of server (that is 0x4040)
        if (fClient)
        {
               // Ask the system to allocate an IPX socket for the client
               IpxAddr.sa_socket = 0x0000;
        }
        // Reuse a given socket number. Allows for multiple instances
        // of an application to run
        setsockopt(soc, SOL_SOCKET, SO_REUSEADDR,
                                        (const char FAR *)&bReuse,4 );
    };
    // Bind the socket to the local address
    //
    if (iBindType == BIND_NETBIOS)
        err = bind(soc, (struct sockaddr *)&NetbiosAddr,
                    sizeof(NetbiosAddr));
    else if (iBindType == BIND_IPX)
        err = bind(soc, (struct sockaddr *)&IpxAddr, sizeof(IpxAddr));
    else
        err = bind(soc, (struct sockaddr *)&LocalAddr,
                         sizeof(LocalAddr));
    if (err==SOCKET_ERROR)
    {
          PRINT_ERROR_AND_RETURN("bind()", WSAGetLastError());
    }
    else
        printf("Successful Binding\n");
    return (err);
}
////////////////////////////////////////////////////////////
// File: Bind.c
//               (see %BOOK%\winsock\bind.c for full sample)
////////////////////////////////////////////////////////////
```

```
//
// Binding a Socket using database functions
//
#include "sockutil.h"
// Macro
#define RETURN_ON_ERROR(err) {\
    WSACleanup();             \
    return(err);              \
}
// Globals
char szServiceName     [MAX_PATH ] = {"EchoServer"};
char szProtocol        [MAX_PATH ] = {"tcp"};
//
// Usage  - the program
//
int Usage( )
{
    printf("bind [-d | -f |-s:<name>|-n:<name> |-p:<protocol>]\n");
    exit(1);
    return(1); // quiets the compiler
}
//
// Command line parser
//
int ParseCommandLine ( int argc, char **argv)
{
    int iBindType = DEFAULT_PORT_ADDR;
    char* ptr;
    argc--;
    argv++;
    while (argc)
    {
          ptr = argv[0];
          if (*ptr=='-')
          {
               ptr++;
               switch (*ptr)
               {
               case 'd': iBindType = DEFAULT_PORT_ADDR;
                    break;
               case 'f': iBindType = FIXED_PORT_LOCAL_ADDR;
                    break;
               case 's': iBindType = GET_PORT_AND_LOCAL_ADDR;
                    ptr +=2;
                    if (*ptr)
                        strcpy(szServiceName, ptr);
                    break;
               case 'n': iBindType = BIND_NETBIOS;
                    ptr +=2;
                    if (*ptr)
                        strcpy(szServiceName, ptr);
                    break;
               case 'p': ptr +=2;
                    if (*ptr)
                        strcpy(szProtocol, ptr);
                    break;
```

```
                case 'i': iBindType = BIND_IPX;
                     break;
                default:
                   return Usage();
                }
          }
          else
                return Usage();
          argc--;
          argv++;
     }
     return (iBindType);
}
int main (int argc, char **argv)
{
     SOCKET soc = INVALID_SOCKET;
     int    iBindType, err;
     // Parse command line
     iBindType = ParseCommandLine(argc, argv);
     // initialize WinSock library
     WinSockInitialize();
     // Create a socket
     switch (iBindType)
     {
     case BIND_NETBIOS:
          // Create a datagram socket in NetBIOS address family
          soc = socket ( AF_NETBIOS, SOCK_DGRAM, 0);
          break;
     case BIND_IPX:
          // Create a datagram socket in IPX/SPX address family
          soc = socket (AF_IPX, SOCK_DGRAM, NSPROTO_IPX);
          break;
     default:
          // Create a stream socket and (by default) use the TCP protocol
          soc = socket ( AF_INET, SOCK_STREAM, 0);
     }
     if (soc == INVALID_SOCKET)
          RETURN_ON_ERROR(1);
     // Bind the socket
     err = BindSocketToAddr ( iBindType, soc, szServiceName, szProtocol,
                              TRUE);
     // Close the socket
     closesocket ( soc );
     // Cleanup and return
     WSACleanup();
     return (0);
}
```


5.1.3 Server Accepting Calls

Stream sockets are generally used in client/server environments, and datagram sockets are used for broadcasting data or for peer-to-peer communication. This section describes the functions called by stream-socket-based servers that allow clients to establish a connection with the server.

As you know, a server must call the socket() function to create a socket and the bind() function to associate its address with the server-side endpoint. The server now calls the listen() and accept() functions to accept client requests to connect to it. The server calls listen() to establish the acceptable size of the client queue associated with a socket. The server application specifies the socket descriptor (using the soc parameter) and the size of the client queue (with the backlog parameter). Currently, the acceptable values for backlog are in the range of [1,5]. By calling listen(), the server gets ready to receive one or more client calls.

```
int PASCAL FAR listen ( SOCKET soc, int backlog);
SOCKET PASCAL FAR accept ( SOCKET soc,
                           const struct sockaddr FAR *addr,
                           int FAR *addrlen);
```

Next, the server calls the accept() function to accept incoming connection requests from the clients (see Figure 5-2 on page 203). The server specifies the socket descriptor, created during the socket() call, by using the soc parameter. The server supplies a buffer to receive the client's address (via the addr parameter) with addrlen containing the size of the buffer. On successful completion, the socket library fills in the buffer pointed to by the addr parameter and returns the length of the actual address with the addrlen parameter. Of course, servers that do not need to have the address of the connecting client can supply NULL in the (addr and addrlen) parameters.

By default, the accept() API blocks until a client has connected to the server. Clients call the connect() function to establish a connection with the server. Successful completion of the accept() function call results in a new socket descriptor, which comes back as a return value from accept(). A value of INVALID_SOCKET indicates error; the server calls WSAGetLastError() to get extended error information. The server uses the new socket descriptor returned by the accept() API to send and receive messages from the connected client. When the server wants to terminate dialogue with the client, it calls the **closesocket**() function, passing in the socket received from accept(). The server can now loop back to its accept() call and wait for a new client to connect. This is illustrated in the following server code, which blocks on the accept() function call waiting for the clients. Once a client is connected, the server receives the client request [by calling the recv() API] and sends back a reply [using the send() API] up to RWLoop times. Then it disconnects the client by closing the accepted socket handle and waits for the next client.

```
////////////////////////////////////////////////////////
// A simple Windows Socket Server (Server1)
// (Certain code fragments and functions omitted.
// See %BOOK\winsock\Server1.c and sockutil.c for full listing)
```

```
//////////////////////////////////////////////////////////////
//  Functions from sockutil.c used by Server1.c are shown here
//////////////////////////////////////////////////////////////
// Create a socket
SOCKET CreateSocket ( int iBindType, int type)
{
    SOCKET soc;
    // Create a stream socket
    switch (iBindType)
    {
    case BIND_IPX:
        // Create a socket in IPX (NWLink, IPX/SPX) address family
          soc = socket (AF_IPX,
                        type,
                        (type==SOCK_STREAM)?NSPROTO_SPX:NSPROTO_IPX
                        );
          break;
    case BIND_NETBIOS:
        // Create a socket in NetBIOS (NetBEUI) address family
        soc = socket (AF_NETBIOS,
                     (type==SOCK_STREAM)?SOCK_SEQPACKET:SOCK_DGRAM,
                      0
                     );
        break;
    default:
        // Create a stream socket in AF_INET (TCP, UDP) address family
        soc = socket ( AF_INET, type, 0);
    }
    return (soc);
}
//////////////////////////////////////////////////////////////
// File: Server1.c . Use it with Client1.c illustrated below
//////////////////////////////////////////////////////////////
#include "sockutil.h"
// Macro
#define RETURN_ON_ERROR(err) {\
    WSACleanup();      \
    return(err);       \
}
#define PRINT_ERROR_AND_BREAK(s) {\
    printf("Error: %d in %s\n", WSAGetLastError(), s);\
    closesocket(accSoc);\
    break;       \
}
// Globals
char szServiceName         [MAX_PATH ] = {"EchoServer"};
char szProtocol            [MAX_PATH ] = {"tcp"};
char szHostName            [MAX_PATH ];
BOOL fShutDownServer;
int  RWLoop     = READ_WRITE_LOOP ;
#define SERVER_REPLY "Server Reply: %d\n"
//
// Usage  - the program
//
int Usage( )
{
```

```
    printf("Server1   [-f |-s:<name>|-p:<protocol>| [-i:<node>:<net>]\n");
    printf(" |-r:<read/write loop> | -n]\n");
    exit(1);
    return(1); // quiese the compiler
}
int main (int argc, char **argv)
{
    SOCKET soc    = INVALID_SOCKET
    SOCKET accSoc = INVALID_SOCKET;
    int    iBindType,i,
           err;
    char   szBuffer   [ MAX_PATH ];
    char   ClientAddr [ MAX_PATH ];
    int    iAddrLen;
    BOOL   fShutDownServer = FALSE;
    // Parse command line (see parsecmd.c for full listing)
    iBindType = ParseCommandLine(argc, argv, FIXED_PORT_LOCAL_ADDR);
    // initialize WinSock library (see sockutil.c)
    WinSockInitialize();
    // Create a stream socket (see sockutil.c)
    soc =  CreateSocket (iBindType, SOCK_STREAM);
    if (soc == INVALID_SOCKET)
         RETURN_ON_ERROR(1);
    // Bind the socket
    err = BindSocketToAddr (iBindType, soc, szServiceName, szProtocol,
                            FALSE);
    // Create an incoming client connect request queue
    err = listen ( soc, CLIENT_QUEUE_SIZE);
    if (err)
    {
        printf("Listen Error: %d\n", WSAGetLastError());
        closesocket(soc);
        RETURN_ON_ERROR(1);
    }
    // Loop until shutdown message is received
    while ( !fShutDownServer )
    {
        printf("Waiting for Client\n");
        // Wait for clients to call connect
        iAddrLen = sizeof(ClientAddr);
        memset(&ClientAddr, 0x00, iAddrLen);
        accSoc = accept ( soc, (struct sockaddr FAR *)&ClientAddr,
                                                &iAddrLen);
        if (accSoc==INVALID_SOCKET)
               PRINT_ERROR_AND_BREAK("accept");
        for (i = 0; i < RWLoop; i++)
        {
               // Read client message
               err = recv ( accSoc, szBuffer, sizeof(szBuffer), 0);
               if (err == SOCKET_ERROR)
               {
                    // Check if the connection was broken
                    if (WSAGetLastError() ==WSAECONNRESET)
                        fShutDownServer = TRUE;
                    PRINT_ERROR_AND_BREAK("recv");
               }
```

```
            if (err)
                printf("Client Sent: %s\n", szBuffer );
            // Is this a shutdown message? Yes, close socket
            if (0==stricmp(szBuffer, SHUTDOWN))
            {
                closesocket(accSoc);
                fShutDownServer = TRUE;
                break;
            }
            // Write a reply
            sprintf(szBuffer, SERVER_REPLY, i);
            err = send ( accSoc, szBuffer, sizeof(szBuffer), 0);
            if (err == SOCKET_ERROR)
                PRINT_ERROR_AND_BREAK("send");
        }
        // All done. Disconnect the client
        closesocket ( accSoc );
    }
    // Close the socket
    closesocket ( soc );
    // Cleanup and return
    WSACleanup();
    return (0);
}
```

Note that if this is a single-threaded server, it can service only one client at a time using this loop of { accept(), send(), recv(), closesocket() }. If other client requests come in while the first client is still connected to the server, the requests are queued until the number of waiting clients exceeds the server's queue size, as specified by the backlog parameter of the listen() API call. Then client requests are refused, and clients receive an extended error, WSAECONREFUSED.

If a single-threaded server can service only one client at a time, then to design a server that can service multiple clients at the same time the designer might start a new thread for each client. For this model, the server's main thread waits for new clients to arrive by blocking on the accept() API. Once the accept() function completes successfully, the server's main thread can start a thread to service the new client. The main thread then returns to waiting on the accept() API. This creates a disadvantage, because for a large number of clients (say, fifty), the server process spends a large part of its processing time on thread context switches. A better design uses a pool of "worker" threads in which a new client waits in a queue until it is serviced by the next available worker thread. (See Section 5.5 for details.)

An alternative server design capable of servicing multiple clients is based on the fact that a socket's mode can be changed from the (default) blocking mode to the nonblocking mode: the server can change the mode of the socket so that all I/O-bound calls complete asynchronously. Such sockets are referred to as nonblocking sockets. Because these sockets do not block, the server can continue processing (for instance, sending or receiving messages from existing

clients) and only occasionally call the accept() function to check for new client requests. When it is called, the accept() function returns immediately. If there is a client request in the server queue, the accept() function returns a valid socket descriptor. Otherwise, the server receives an extended error (WSAEWOULDBLOCK), and it can return to servicing existing clients. Section 5.1.9, "Asynchronous Operations," illustrates the nonblocking sockets in detail.

5.1.4 Client Connecting to Server

A stream-socket-based client that wants to establish a stream-socket-based server calls the connect() function, supplying the server address. [Before calling connect(), the client must have created a socket by calling the socket() and the bind() functions to establish the client-side endpoint.] The prototype of the connect() Windows Socket API is shown here:

```
int PASCAL FAR connect ( SOCKET soc,
                         const struct sockaddr FAR * name,
                         int namelen );
```

When calling the connect() function, the client supplies the socket descriptor (using the soc parameter) received earlier from the socket() API. [Recall that the socket() function call creates the endpoint, and that the bind() function call associates the local address with the endpoint.] The connect() function joins the client and the server endpoints to provide a logical connection. Upon successful completion of the connect() API, the client can use the socket descriptor for sending and receiving data to and from the server. If there is an error, the connect() API returns (SOCKET_ERROR), and the client can call the WSAGetLastError() API to get extended error information.

The client specifies the server address with the name parameter, with namelen containing the size of the address buffer. The client must know the server's address family and the family's addressing conventions. For example, TCP/IP applications (PF_INET address family) use database functions to determine the server address, and provide the server's port number and 32-bit network address to the connect() API. In this example, the gethostbyname() API returns the network address of the computer hosting the server application, and the getservbyname() API returns the port number associated with a given server. Applications that use other address families (such as AF_NETBIOS or AF_IPX) must use interfaces specific to that address space to locate a server. For example, in the Novell NetWare environments, the clients use the service advertising protocol (SAP) to locate a server (see Chapter 9 for details on SAP). A NetBIOS application could use the NCB.FIND.NAME command to find the full address of a remote server (see Chapter 8 for details on NetBIOS).

These different methods of determining the server address create a lot of confusion. So, starting with Windows NT 3.5, Windows Sockets has a new set of APIs—referred to as the name service provider APIs—to handle server registration (by the server) and finding a server (by clients). These APIs allow the servers to use one set of APIs to register the server details (such as network address, port number, and so on) for each supported address family. Similarly, the clients can use these APIs to determine the server details for each address family. These APIs are discussed in Section 5.4.

The following code fragment shows a client that creates a stream socket and connects with the server described earlier in Section 5.1.3. Notice that by default, the client (and the server shown earlier) uses the AF_INET address family and the FIXED_PORT_LOCAL_ADDR option. In this case, the client assumes the server is local, and fixes the port number (FIXED_PORT). The other options follow:

- Use -h:<host name> command line option to force the client to use the gethostbyname() API to determine the host address and connect to a server running on a remote machine. This causes a lookup in the %SYSTEMROOT%\drivers\etc\hosts file on Windows NT.
- Use -s:<service name> to have the client call the getservbyname() API to determine the port number for the server (service). This causes a lookup in the %SYSTEMROOT%\drivers\etc\services file on Windows NT.
- Use the -i:<node number> :<net number> option to have the client connect to the server through the SPX protocol. When the net number is not specified, assume that the client and server are on the same network (that is, at net number = 0). To test the connection, determine the 6-byte node number by using the adapter number associated with the IPX/SPX (NWLink) protocol (use the net config workstation command on Windows NT). To determine the IPX/SPX address at run-time, the application can call the **IPXGetInternetworkAddress()** Novell NetWare communication API (see Chapter 9).
- Instruct the client to send more than one request to, and receive replies from, the server by using the -r:<Read/Write Loop Count> parameter.
- Use the -k option to send a shutdown message to the (local or remote) server.

```
//
// A simple Windows Socket Client
// (some code fragments omitted.
// See %BOOK%\winsock\ (client1.c & sockutil.c) for full listing)
```

```
////////////////////////////////////////////////////////////
// File: Sockutil.c
// Continued after the listing shown in the previous section
//
// Connect to a local or remote server
////////////////////////////////////////////////////////////
int ConnectToServer (int    iBindType,
                     SOCKET soc,
                     LPSTR  pszHostName,
                     LPSTR  pszServiceName,
                     LPSTR  pszProtocolName
                     )
{
    struct sockaddr_in     RemAddr;
    SOCKADDR_NB            NetbiosAddr;
    SOCKADDR_IPX           IpxAddr;
    int                    err;
    u_long                 ulHostAddress;
    u_short                usPortNo;
    memset ( &RemAddr, 0x00, sizeof(RemAddr));
    err = SOCKET_ERROR;
    switch (iBindType)
    {
    case DEFAULT_PORT_ADDR:
        printf("Invalid option\n"); return(SOCKET_ERROR);
        break;
    case FIXED_PORT_LOCAL_ADDR:
        //
        // Connect to local server with a fixed port
        //
        if ((ulHostAddress = GetLocalHostAddress()) == 0)
        {
               PRINT_ERROR_AND_RETURN("GetHostAddress", err);
        }
        RemAddr.sin_family = AF_INET;
        RemAddr.sin_addr.s_addr = ulHostAddress ;
        RemAddr.sin_port = htons(FIXED_PORT);
        break;
    case GET_PORT_AND_LOCAL_ADDR:
        //
        // Connect to a local server. Get its port at run-time
        //
        if ((ulHostAddress = GetLocalHostAddress()) == 0)
        {
               PRINT_ERROR_AND_RETURN("GetLocalHost error", err);
        }
        else
        {
               usPortNo = GetPortForService(pszServiceName,
                                            szProtocolName);
        }
        RemAddr.sin_family = AF_INET;
        RemAddr.sin_addr.s_addr = ulHostAddress ;
        RemAddr.sin_port = usPortNo;
        break;
    case GET_PORT_AND_REMOTE_ADDR:
```

```
        //
        // Connect to a remote server. Use a Fixed Port.
        //
        if ((ulHostAddress = GetHostAddress(pszHostName)) == 0)
        {
                PRINT_ERROR_AND_RETURN("GetLocalHost error", err);
        }
        RemAddr.sin_family = AF_INET;
        RemAddr.sin_addr.s_addr = ulHostAddress ;
        RemAddr.sin_port = htons(FIXED_PORT);
        break;
    case BIND_IPX:
        // Connect to remote server via IPX address
        memcpy (&IpxAddr, &defIPXAddress, sizeof(SOCKADDR_IPX));
    };
    //
    // Connect the client socket with the remote server
    //
    if (iBindType == BIND_NETBIOS)
        err = connect(soc, (struct sockaddr *)&NetbiosAddr,
                    sizeof(NetbiosAddr));
    else if (iBindType == BIND_IPX)
        err = connect(soc, (struct sockaddr *)&IpxAddr,
                    sizeof(IpxAddr));
    else
        err = connect(soc, (struct sockaddr *)&RemAddr,
                    sizeof(RemAddr));
    if (err==SOCKET_ERROR)
    {
        PRINT_ERROR_AND_RETURN("connect()", WSAGetLastError());
    }
    else
        printf("Successful connection\n");
    return (err);
}
////////////////////////////////////////////////////////////
// File: Client1.c
// A simple Windows Socket client (Client1)
// (see %BOOK%\winsock\client1.mak for full details)
////////////////////////////////////////////////////////////
#include "sockutil.h"

// Macro
#define RETURN_ON_ERROR(err) {\
     WSACleanup();      \
     return(err);       \
}

#define PRINT_ERROR_AND_BREAK(s) {\
     printf("Error: %d in %s\n", WSAGetLastError(), s);\
     break;       \
}
// Globals
char szServiceName      [MAX_PATH ] = {"EchoServer"};
char szProtocol         [MAX_PATH ] = {"tcp"};
char szHostName         [MAX_PATH ] = {"localhost"};
```

```
int  RWLoop = READ_WRITE_LOOP ;
BOOL fShutDownServer = FALSE;
#define CLIENT_REQUEST "Client Request: %d\n"
//
// Usage  - the program
//
int Usage( )
{
    printf("Client1 [-f |-s:<name>|-p:<protocol>\n");
    printf(" |-r:<read/write loop>| -h:<host name>
             |[-i:<node>:<net>]\n");
    printf(" |-k|-n]\n");
    exit(1);
    return(1); // quiese the compiler
}
int main (int argc, char **argv)
{
    SOCKET soc    = INVALID_SOCKET;
    int     iBindType, i,
              err;
    char  szBuffer [ MAX_PATH ];

    // Parse command line
    iBindType = ParseCommandLine(argc, argv, FIXED_PORT_LOCAL_ADDR);

    // initialize WinSock library
    WinSockInitialize();

    // Create a stream socket
    soc =  CreateSocket ( iBindType, SOCK_STREAM);
    if (soc == INVALID_SOCKET)
        RETURN_ON_ERROR(1);

    err = BindSocketToAddr (
                            iBindType,          // BindType
                            soc,                // Socket
                            szServiceName,      // Service Name
                            szProtocol,         // Protocol
                            TRUE                // This is client
                            );

    if (err)
    {
        closesocket(soc);
        RETURN_ON_ERROR(1);
    }
    // Connect to the server
    printf("Connecting to server: %s on Host: %s Protocol: %s\n",
                szServiceName, szHostName, szProtocol);
    err = ConnectToserver ( iBindType, soc, szHostName,
                            szServiceName, szProtocol);
    if (err != 0)
    {
        closesocket(soc);
        RETURN_ON_ERROR(1);
    }
```

```
    else
        printf("Connected\n");
    // Loop 'RWLoop' times
    for (i = 0; i < RWLoop; i++)
    {
        printf("Read/Write Loop :%d\n", i );
        // Send a request to the server
        if (fShutDownServer)
                strcpy(szBuffer, SHUTDOWN);
        else
                sprintf(szBuffer, CLIENT_REQUEST, i);
        err = send ( soc, szBuffer, sizeof(szBuffer), 0);
        if (err == SOCKET_ERROR)
                PRINT_ERROR_AND_BREAK("send");

        // Don't expect a message from server if we shut it down
        if (fShutDownServer)
        {
                // send a finished (FIN) message
                shutdown(soc, 1);
                break;
        }
        // Read Server's  reply
        err = recv ( soc, szBuffer, sizeof(szBuffer), 0);
        if (err == SOCKET_ERROR)
                PRINT_ERROR_AND_BREAK("recv");
        if (err)
                printf("Server Replied: %s\n", szBuffer );

    }

    // Close the socket
    closesocket ( soc );

    // Cleanup and return
    WSACleanup();
    return (0);
}
```

5.1.5 Address Resolution in TCP/IP Environment

Address resolution is triggered by Windows Socket applications, usually clients that need to determine the network address (for example, 11.1.21.34) of a remote server when the application has the name (for example, "BigServer") of the host computer. It can also be triggered by normal Windows NT operations, such as connecting to a remote server over TCP/IP. The section above illustrated the use of the gethostbyname() database API for determining the address of the host and for controlling the name-to-address mapping process by modifying the (%SYSTEMROOT%\drivers\etc\hosts) file. This section expands on address resolution in the Windows NT environment. Note that similar functions are also available on Windows for Workgroups and Windows 95.

There are three address resolution methods: Windows Internetwork Name Service (WINS); domain name system (DNS); or the LMHOSTS (%SYSTEMROOT%\drivers\etc\LMHOSTS) file, which can be configured for direct or indirect use on Windows NT workstations. You can trigger any of the methods directly by executing a command such as ("net use * \\BigServer\FileShare"). An indirect trigger is activated when a client attempts to determine the network address of a remote server (using WinSock interfaces).

Using WINS

In Windows and Windows NT environments, every workstation has a fixed "computer name," such as "BigServer." These names correspond one-to-one with the NetBIOS name of the workstation. Also, every workstation that supports the TCP/IP protocol must have a unique IP address (for example, 11.1.12.34).

A Windows NT server can be configured to provide the mapping between the NetBIOS names and the corresponding IP addresses by activating the WINS server. A WINS server serves the Windows and Windows NT workstations within the same subnet. (It can also serve other subnets, provided a router has been put in place.) At startup time, each workstation registers its NetBIOS name and IP address with the WINS server on its subnet. Workstations remove their names (deregister) prior to shutdown. Thus, when a socket application running on "Savannah" asks for the host address for "BigServer," WinSock can query the nearest WINS server and determine the IP address of "BigServer." This is illustrated in Figure 5-4. The WINS server has been implemented following the guidelines suggested in the RFC[2] 1001/1002. See [Allard 94] for further details on WINS server.

Using DNS

The domain name system is a distributed database that provides a hierarchical naming system for identifying hosts on the Internet. DNS identifies a user by a user-friendly name, such as Jim@columbia.com or Mary@bodega.stanford.edu. In this distributed system, several DNS servers maintain the repository of name-to-address mappings and periodically replicate the repository between themselves. Thus, a DNS client need only connect to one of the DNS servers to determine the address (for instance, 11.2.34.15) associated with a name (for example, BigServer@BigCorp.Com).

A Windows NT workstation can be configured to connect to a DNS server (provided a proper Internet connection has been established between the

2. Request for Information. <nic.ddn.mil> is the official repository for all Internet drafts and RFCs.

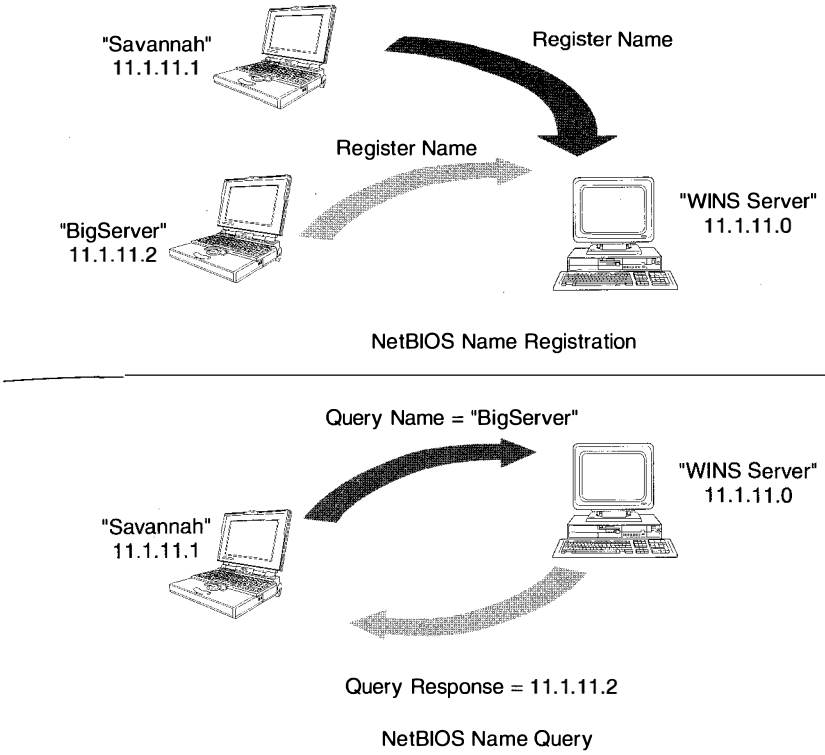


Figure 5-4. *Using WINS Server for Name Resolution Using DNS*

Windows NT workstation and the remote DNS server) and obtain name-to-address mapping. Windows Socket applications (such as FTP) can then accept remote server names in DNS format. This is illustrated by Figure 5-5, in which a DNS name query (for BigServer@ABigCorp.Com) is issued from Savannah.com. The query is satisfied by the Internet, which contains DNS servers, each of which in turn contain name-to-address mappings for machines on the Internet.

Using LMHOSTS File

The third address resolution method in Windows NT is the LMHOSTS file. This can be used for backward compatibility with Microsoft LAN Manager workstations and servers, and is also appropriate in small workgroups. In LMHOSTS, the administrator puts the NetBIOS names of the remote hosts and their corresponding IP addresses in a format similar to that used for the (%SYSTEMROOT\drivers\etc\hosts) files. Thus LMHOSTS can fit into the Windows NT domain model (see #DOM keyword) and performance enhancements by preloading most frequently used names mapping in memory. See [Allard 94] for further details.

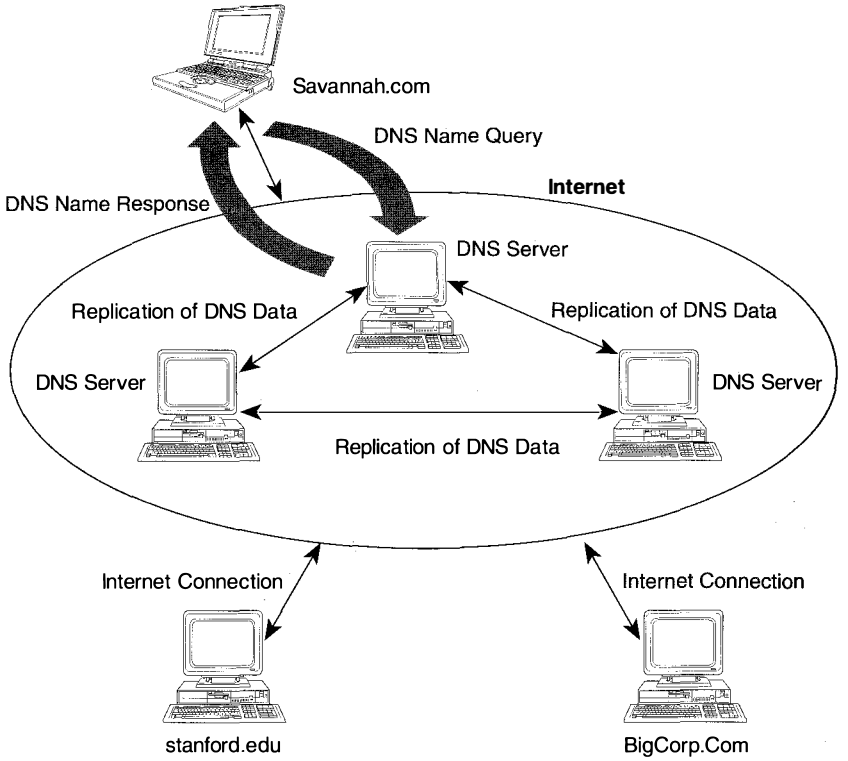


Figure 5-5. *Using an Internet DNS Server for Name Resolution*

5.1.6 Sending and Receiving Data on Stream Socket

Here are the prototypes for the send() and recv() functions:

```
int PASCAL FAR send ( SOCKET              soc,
                      const char FAR *    buffer,
                      int                 len,
                      int                 flags );
int PASCAL FAR recv ( SOCKET              soc,
                      char FAR *          buffer,
                      int                 len,
                      int                 flags );
```

Both functions take a valid socket descriptor, using the soc parameter. For the server, the socket descriptor specified must be the same as the socket descriptor returned by an accept() function. Clients use the socket descriptor returned by a successful connect() function. Both functions return ERROR_SOCKET if an error occurs in the transmission of data; if this happens, applications can call the WSAGetLastError() API to receive extended error information.

Using the buffer parameter with the number of bytes in the data buffer set in the len parameter, the application that sends the data specifies the data to

be transmitted. The send() function returns the number of bytes transmitted upon successful completion, which can be in the range [1, length of data buffer length]. Therefore, applications must be prepared to re-transmit any unsent data if the send() function does not transmit the complete data buffer. In the case of a blocking socket (default), the send() API will block until an application data buffer is either transferred to the recipient or copied to the local system buffer. Note that a number of send() calls can fill up the local system buffer. Any subsequent send() calls will block until the remote recipient empties all or part of the system buffer. Applications should not be designed to depend on local or remote socket system-buffering schemes, protocols, or underlying systems. To avoid blocking on send() or recv() calls, use nonblocking sockets.

Like the send() API, the recv() API returns the number of bytes copied into the caller's address space when the data transmission completes. It can return zero when the socket connection has been closed (or when the remote sender closes the socket). Applications should expect WSAECONNRESET from recv()[3] when a socket connection is abnormally disconnected (due to a network failure, termination of the remote application, or the like).

Be aware that a stream socket is really a stream of bytes with no boundaries between any two messages. The incoming and outgoing socket buffers are distinct so that an incoming message stream and an outgoing message stream do not mix, yet the socket implementation does not maintain boundaries between any two incoming messages. This is much like the byte mode Named Pipe. Therefore, the socket client and server must have an agreed-upon data transmission format so that they can distinguish one message from the next.

The client and server must also have an agreed-upon protocol of their own to eliminate the chance that no data is transmitted. Fortunately, in a typical client/server system, the client and the server alternately call send() and recv(), thereby removing chances of blocking. This closely simulates client requests to the server, with the server responding to client requests.

When both client and server send and receive fixed length data buffers, data-buffer handling becomes much simpler. However, when sending and receiving variable lengths of data between consecutive transmissions, both the client and the server must have an agreed-upon data format so that each end knows the data-buffer length. The data structure below can be used by both client and the server to send and receive variable length buffers.

```
typedef struct _tagData{
      unsigned short     usOpCode;     // This allows client/server to
                                       // distinguish type of data
      unsigned long      ulBufLen;     // Buffer Length
      byte               bData [1];    // Variable length data
};
```

3. Actually, recv() will return ERROR_SOCKET and WSAGetLastError() will return WSAECONNRESET.

Here, the sender (client or server) can put a command in usOpCode and specify the length of the data buffer to follow in usBufLen. The data buffer can be one byte to whatever limit is imposed by the underlying protocol. The receiver can also call recv() with a fixed-size buffer (at least eight bytes in this case). When the sender's data is less than receiver's buffer size, the receiver will block until the recv() API times out. (See the SO_RCVTIMED option of the setsokopt() API.) Recall that recv() returns the bytes copied into receiver's buffer. However, when the incoming data does not fit into the receiver's buffer, recv() will return and fill up the receiver's buffer (that is, the number of bytes copied will equal the size of the receiver's buffer). The receiver must check the ulBufLen parameter to make sure all data has been received. If not, the receiver needs to call recv() again to obtain the rest of the data. The code fragment at the end of this section illustrates how this happens.

Both the send() and the recv() APIs take the flags parameter, which can modify data transmission semantics. If this parameter is not zero, it is formed by OR-ing as one of the following constants:

Parameter	*Description*
MSG_DONTROUTE:	Bypass message routing relevant to send() or sendto()
MGS_OOB:	Send or receive out of band data
MSG_PEEK:	Peek at incoming message relevant to rev() or recvfrom()

MSG_DONTROUTE instructs the Windows Socket library not to use the underlying protocol's routing mechanism, but instead to send the packet directly to the designated recipient at the address specified for the connect() API [or as a parameter to the **sendto**() API]. However, the WinSock library implementors can ignore this instruction, so applications that depend on this function must either verify the existence of this option by calling the **getsockopt**() API, or specify the MSG_OOB flag to send and receive out-of-band data.

When you specify MSG_PEEK flags with the recv() function, recv() returns the incoming data into your buffer without actually removing data from the socket buffers. In other words, subsequent calls to the recv() function will return the same data until the recv() API is called without the MSG_PEEK flag. Thus, the MSG_PEEK parameter allows a nondestructive read of the stream socket. Another attribute of MSG_PEEK is that when the recv() API is called on a blocked socket (default) with the MSG_PEEK flag, the function call does not block, which means you can use the recv() API to determine if a message has arrived without blocking. If a message has arrived, you can ascertain the type of message, length of message, and so on, without having to read the message. If you do this, you can allocate a large (or small) buffer, depend-

ing on the size of the message. You can also prioritize the processing of incoming messages based on their content, especially if a server is receiving data on multiple sockets. Once the receiver is prepared to receive the message and remove it permanently from the system buffer, the receiver can call the recv() API without the MSG_PEEK flag.

The following code fragment illustrates the use of the MSG_PEEK flag when variable-size data packets are being sent and received by stream socket. Variable-size data is sent with the VARDATA structure, using the **SendVariable()** function. OPCODE_BEGIN appears in the usOpCode field of the first packet and OPCODE_LAST in the last; all packets in between are tagged with the OPCODE_MIDDLE opcode. The sender puts the length of the data buffer in the ulBufLen parameter and copies the data bytes into the bData field. The receiver calls the **ReceiveVariable()** function, which first calls the recv() with the MSG_PEEK flag to determine the length of the incoming data. It can thus allocate a buffer large enough to hold the entire incoming data buffer and call recv() with no flags to remove the data from the stream socket buffer.

```
//////////////////////////////////////////////////////
// Functions to send and receive variable length
// data.
// See usage in %BOOK%\winsock\Client2.c and %BOOK%\winsock\Server2.c
//////////////////////////////////////////////////////
#include "sockutil.h"

#define PRINT_ERROR_AND_BREAK(s) {\
    printf("Error: %d in %s\n", WSAGetLastError(), s);\
    break;      \
}

typedef struct _tagData
{
    unsigned short usOpCode;
    unsigned short ulBufLen;
    byte                bData[1];
} VARDATA, *PVARDATA;
#define OPCODE_BEGIN  1
#define OPCODE_LAST   2
#define OPCODE_MIDDLE 3

#define NEXT_DATA_SIZE(sent, Total) {
            ((Total - sent) > 10)?((Total-sent)/2):(Total-sent)\
}
//
// Send variable length data packets
//
int SendVariable ( SOCKET soc)
{
    // Data Buffer
    char     szDataBuffer [ 2 * MAX_PATH ]; // Data Buffer
    // Transmission Buffer
```

```
char     szVarBuffer [ sizeof(VARDATA) + 2 * MAX_PATH ];
VARDATA  *pVarData;

int      i, err, iDataSent, iDataSize;
char*    ptr;
//
// Set a pattern in buffer
//
iDataSize = sizeof(szDataBuffer);
for (i = 0; i < iDataSize ; i++)
{
    // put 0x5A in odd bytes and 0xA5 in even
    szDataBuffer [i] = (i & 1)?0x5A:0xA5;
}
//
// The first packet sent has OPCODE_BEGIN opcode and the
// the last packet sent has OPCODE_LAST opcode. Rest have
// OPCODE_MIDDLE stamp
//
ptr = &szDataBuffer[0];
pVarData = (PVARDATA) &szVarBuffer;
iDataSent = 0;
while (iDataSent < iDataSize )
{
    // Determine the next send buffer size
    i = NEXT_DATA_SIZE(iDataSent, iDataSize);
    printf("Next Data Size: %d\n", i);

    // Set the opcode
    if (iDataSent == 0)
           pVarData->usOpCode = OPCODE_BEGIN;
    else if (iDataSent + i == iDataSize)
           pVarData->usOpCode = OPCODE_LAST;
    else
           pVarData->usOpCode = OPCODE_MIDDLE;

    // Set the buffer length
    pVarData->ulBufLen = i;

    // Copy data from sender's buffer to transmission buffer
    memcpy(pVarData->bData, ptr, i );

    err= send ( soc,
                (LPSTR)&szVarBuffer,       // data
                i + sizeof(VARDATA) -1 ,   // size of buffer
                0);
    if (err==SOCKET_ERROR)
    {
           PRINT_ERROR_AND_BREAK("send");
    }
    else
    {
           iDataSent += i;
           ptr += i;
    }
}
```

```
    if (err != SOCKET_ERROR)
        printf("Total Data Sent: %d\n", iDataSent);

    return(err);
}
//
// Receive variable length data size
//
int ReceiveVariable ( SOCKET soc)
{
    VARDATA  varData, *pVarData;
    int      i, err, iDataRecv, iTotal;
    char*    pszBuffer;

    //
    // We loop until the last data packet has been received
    //
    iTotal = 0;
    do
    {
        //
        // First peek in the input stream to determine the buffer size
        //
        memset (&varData, 0x00, sizeof(varData));
        iDataRecv = sizeof(varData);
        err = recv ( soc, (LPSTR)&varData, iDataRecv, MSG_PEEK);
        if (err==SOCKET_ERROR)
        {
               PRINT_ERROR_AND_BREAK("recv peek");
        }
        if (varData.ulBufLen != 0)
        {
               iDataRecv = varData.ulBufLen + sizeof(VARDATA) - 1;
               printf("Incoming Data Size: %d\n", varData.ulBufLen);

               // Allocate buffer large enough to hold incoming data
               pszBuffer = malloc ( iDataRecv);
               if (pszBuffer==NULL)
               {
                    printf("malloc failed\n"); err = SOCKET_ERROR; break;
               }

               // Receive data
               err = recv ( soc, pszBuffer, iDataRecv, 0);
               if (err==SOCKET_ERROR)
               {
                    PRINT_ERROR_AND_BREAK("recv peek");
               }
               else
               {
                    pVarData = (PVARDATA)pszBuffer;
               }
               // Verify data
               for (i = 0 ; i < pVarData->ulBufLen; i++)
               {
                  // Should have 0x5A in odd bytes and 0xA5 in even bytes
```

```
                if ((((i + iTotal ) & 1)&&(pVarData->bData[i] ==0x5A))||
                                        (pVarData->bData[i] == 0xA5))
                    continue;
                else
                    printf("Invalid data:%x at:%d\n",
                    pVarData->bData[i], i);
            }
            iTotal += pVarData->ulBufLen;
            free (pszBuffer);
            pszBuffer = NULL;
        }
        else
            printf("No data in input stream\n");

      // Loop until we receive last packet
    } while ( varData.usOpCode != OPCODE_LAST);
    if (err != SOCKET_ERROR)
        printf("Total Data Received: %d\n", iTotal);
    return(err);
}
```

5.1.7 Sending and Receiving Data on SOCK_DGRAM Socket

Before discussing the use of datagram sockets to send and receive data, let us review the datagram socket fundamentals. Recall that datagram sockets provide connectionless service, that is, data delivery is not guaranteed. Datagrams that are directed towards a specific recipient by the sender are called directed datagrams; broadcast datagrams are directed towards all active nodes on the network. The protocols such as IPX and UDP provide datagram services. Table 5-3 contrasts datagram sockets with stream sockets.

To send and receive data with a datagram socket, the client and server applications first create a datagram socket (SOCK_DGRAM) by calling the socket() API and binding their address to the socket by calling the bind() API. Then they can use the sendto() and the recvfrom() functions to send and receive data. These functions can be used with stream sockets as well. However, this section focuses mainly on their use with datagram sockets. The prototypes of these functions are shown here:

```
int PASCAL FAR sendto (SOCKET  soc,
                       const  char   FAR  *  buffer,
                       int  len,
                       int  flags,
                       const struct sockaddr FAR * to,
                       int tolen );
int PASCAL FAR recvfrom (SOCKET  soc,
                         char  FAR  *  buffer,
                         int  len,
                         int  flags,
                         struct sockaddr FAR * from,
                         int  FAR  *  fromlen );
```

Table 5-3. Datagram Sockets Compared with Stream Sockets

Issue	*Stream Socket*	*Datagram Socket*
Data delivery	Guaranteed and in order	Not guaranteed. Packets may be lost, duplicated, and/or arrive out of order.
Network packet utilization	Higher than datagram	Lower because receiver and socket sender's network address must be carried in each packet.
Protocol overhead	High because the protocol (for example, TCP) handles errors, transmission and so on	Lower because protocol (for example, IPX) does little error control.
Broadcast overhead	N/A	When broadcast datagrams are sent, these packets are received by every listening node, causing each node to process these packets. This affects the entire network of computers.
Application	Traditional client/server applications that let the protocol take care of error control, data packet ordering, and other guaranteed data delivery issues	Applications that either do not need error control or can achieve higher throughput by writing small applications level error control protocol. Also, applicable on high speed networks, such as Fiber Distribution Data Interface (FDDI) and Asynchronous Transfer Mode (ATM), because the protocol overhead must be kept low to keep up with the high speed of the network.

Both functions use the soc parameter to take a socket descriptor. Specify the recipient's address using the to parameter in sendto(), with tolen containing the length of the address. Specify the data buffer with the buffer parameter. Use the len parameter to specify the length of data in bytes. The maximum size of the datagram data buffer is restricted by the capability of the underlying protocol. For example, in the TCP/IP environments, an application cannot send a data buffer bigger than the largest IP packet allowed in the subnet. If it does, the sendto() API returns with an error (WSAEMSGSIZE). The maximum allowable size of an IP packet is determined at startup time by calling the WSAStartup() API. On datagram sockets, successful completion of the sendto() function does not guarantee that the recipient has received the data. Further, on a blocking (default) socket, the sendto() function blocks

if the system does not have enough space to hold the sender-supplied buffer. On successful completion of the sendto() function, the function returns the number of bytes transmitted to the recipient. On failure, it returns ERROR_SOCKET; the application can call the WSAGetLastError() API for extended error information.

For a nonblocking datagram socket, the sendto() API writes one or more bytes into the local system buffer and returns immediately. An application can use other methods, such as calling the select() API, to determine the number of bytes that can be sent to the recipient and then try sending more data later.

The datagram socket-based server calls the **recvfrom**() API to receive data from a client. If the server supplies a nonzero from parameter when calling the recvfrom() API, then the sender's address is returned in the from parameter with the length of the address in the fromlen parameter. Just like the recv() API, the server indicates with the buffer parameter the address that receives client data. Upon successful completion of the recvfrom() API, the function returns the number of bytes received. When a socket connection has terminated,[4] the recvfrom() API returns zero as the return value. In other error conditions, the recvfrom() API returns ERROR_SOCKET. Finally, if the data sent by a client is larger than the server-supplied data buffer, the system returns to the server only as much as the server's buffer can hold, and discards the rest of the message. In this case, the recvfrom() API will also return an error (WSAEMSGSIZE).

Both the sendto() and the recvfrom() APIs take a flags parameter, which can be used to modify the behavior of data transmission. The flags values and semantics are similar to the send() and recv() functions discussed earlier.

Applications can also use the send() and the recv() functions to transmit data over datagram-based sockets. Table 5-4 shows the difference between the two ways to transmit data with datagram sockets. The {sendto(), recvfrom()} pair is flexible; at any time, the application can send or receive data from any remote end. However, this flexibility requires specifying the address of the recipient on every call. On the other hand, when two applications want to use a datagram socket to communicate exclusively with each other, they can use the {send(), recv()} pair. They need to call the connect() API to establish the address of the recipient address.

It is also possible to use the recvfrom() function to broadcast datagrams over the datagram socket by specifying (INADDR_BROADCAST) as the network address of the server. However, applications should be able to determine the maximum size of the datagram packet they can send and receive using the iMaxUdpDg field of the data structure (WSADATA) returned by the WSAStartup() API.

4. Relevant only when recvfrom() is being used on stream socket, because the datagram socket does not maintain a connection that can be terminated.

Table 5-4. Functions Used with Datagram Sockets

Using recvfrom()/sendto()		*Using send()/recv()*		
Server Calls	**Client Calls**	**Server Calls**	**Client Calls**	***Description***
socket()	socket()	socket()	socket()	Establish SOCK_DGRAM socket
bind()	bind()	bind()	bind()	Bind endpoint to local address
		connect()	connect()	Establish remote endpoint address to be used with send() and recv()
recvfrom()	sendto()	recv()	send()	Client request sent to server
sendto()	recvfrom()	send()	recv()	Server responds back to client
close()	close()	close()	close()	Close socket created by socket()

The following code fragment shows a client and server using datagram sockets to send and receive data. Notice that once the protocol-specific details are handled by the bind() and the connect() APIs, the majority of the WinSock client and server (code) remains identical for all protocols; this is one of the advantages of using the WinSock interfaces. The code demonstrates these programming concerns:

- By default, both the client and the server create a datagram packet over UDP by calling the socket() API. Using the bind() API, the server binds its endpoint with the local host address and a fixed port. The client asks the system to allocate a port for itself and binds to INADDR_ANY. Next, the server uses the recvfrom() and sendto() APIs to transmit messages over the datagram socket. Notice that the server determines the client address by using the pClientAddr parameter to recvfrom(). This allows the server to respond to any client. The client supplies the WinSock implementation with the network address of the server through the connect() API. This allows it to use the send() and recv() API to relay datagrams.
- The program also demonstrates the creation of datagrams sockets over the IPX protocol. (See BIND_IPX option on page 217.) The

server binds to the workstation address (node number, network number) and a fixed IPX socket number (0x4040). The client also binds to the local workstation address, but asks the system (that is, NWLink) to allocate an IPX socket for it by supplying (0x0000) as the IPX socket number (using the sa_socket parameter of SOCKADDR_IPX). This program becomes more sophisticated if the IPXGetInternetworkAddress() NetWare API is used to obtain the workstation IPX address.

- The program also demonstrates creation of datagram socket over the NetBEUI protocol. (See BIND_NETBIOS option on page 217.) It uses the name of the service (EchoServer) and the port number (5001) to create a NetBIOS name for the server. This name is used with the bind() API on the server end. The client name, passed to the bind() API on the client side, is prepared from a fixed component ("WSockClient") and the service port number . The client must supply the server address to the connect() API.

```
////////////////////////////////////////////////////
// A Windows Socket Datagram Server (Server3)
// (see %BOOK%\winsock\server3.mak for full listing)
////////////////////////////////////////////////////
#include "sockutil.h"

// Macro
#define RETURN_ON_ERROR(err) {\
    WSACleanup();      \
    return(err);       \
}
#define PRINT_ERROR_AND_BREAK(s) {\
    printf("Error: %d in %s\n", WSAGetLastError(), s);\
    break;      \
}

// Globals
char szServiceName  [MAX_PATH ] = {"EchoServer"};
char szProtocol     [MAX_PATH ] = {"udp"};
char szHostName     [MAX_PATH ];
BOOL fShutDownServer;
int       RWLoop     = READ_WRITE_LOOP ;
#define SERVER_REPLY "Server Reply: %d\n"
//
// Usage  - the program
//
int Usage( )
{
    printf("Server3 [-f |-s:<name>|-p:<protocol>| [-i:<node>:<net>]\n");
    printf(" |-r:<read/write loop>]\n");
    exit(1);
    return(1); // quiese the compiler
}
int main (int argc, char **argv)
```

```
{
    SOCKET soc = INVALID_SOCKET;
    struct sockaddr ClientAddr, *pClientAddr;
    SOCKADDR_NB ClientNBAddr;

    int     iAddrLen;
    int     iBindType,i, err;
    char    szBuffer [ MAX_PATH ];
    BOOL    fShutDownServer = FALSE;

    // Parse command line
    iBindType = ParseCommandLine(argc, argv, FIXED_PORT_LOCAL_ADDR);

    // initialize WinSock library
    WinSockInitialize();

    // Create a datagram socket
    soc =  CreateSocket ( iBindType, SOCK_DGRAM);
    if (soc == INVALID_SOCKET)
          RETURN_ON_ERROR(1);

    // Bind the socket
    err = BindSocketToAddr ( iBindType, soc, szServiceName, szProtocol,
                             FALSE);
    if (err)
    {
          closesocket(soc);
          RETURN_ON_ERROR(1);
    }
    // Loop until shutdown message is received
    while ( !fShutDownServer )
    {
          printf("Waiting for Client\n");
          for (i = 0; i < RWLoop; i++)
          {
               // Read Client message
               iAddrLen    = sizeof(ClientAddr);
               pClientAddr = &ClientAddr;
               if (iBindType==BIND_NETBIOS)
               {
                    iAddrLen    = sizeof(ClientNBAddr);
                    pClientAddr = (struct sockaddr *)
                                  &ClientNBAddr;
               }
               err = recvfrom ( soc, szBuffer, sizeof(szBuffer), 0,
                                pClientAddr, &iAddrLen);
               if (err == SOCKET_ERROR)
                    PRINT_ERROR_AND_BREAK("recvfrom");
               if (err)
                    printf("Client Sent: %s\n", szBuffer );
               // Is this a shutdown message? Yes, close socket
               if (0==stricmp(szBuffer, SHUTDOWN))
               {
                    fShutDownServer = TRUE;
                    break;
               }
```

```
                // Write back a reply
                sprintf(szBuffer, SERVER_REPLY, i);
                err = sendto( soc, szBuffer, sizeof(szBuffer), 0,
                              pClientAddr, iAddrLen);
                if (err == SOCKET_ERROR)
                     PRINT_ERROR_AND_BREAK("sendto");
           }
     }
     // Close the socket
     closesocket ( soc );
     // Cleanup and return
     WSACleanup();
     return (0);
}
////////////////////////////////////////////////////////
// A simple Windows Socket Datagram Client (Client3)
// (see %BOOK%\winsock\client3.mak for full details)
////////////////////////////////////////////////////
#include "sockutil.h"

//
// Globals
char szServiceName  [MAX_PATH ] = {"EchoServer"};
char szProtocol     [MAX_PATH ] = {"udp"};
char szHostName     [MAX_PATH ] = {"localhost"};

int  RWLoop = READ_WRITE_LOOP ;
BOOL fShutDownServer = FALSE;
#define CLIENT_REQUEST "Client Request: %d\n"
//
// Usage  - the program
//
int Usage( )
{
     printf("Client3 [-f |-s:<name>|-p:<protocol>\n");
     printf(" |-r:<read/write loop>| -h:<host name>|
               [-i:<node>:<net>]\n");
     printf(" |-n|-k]\n");
     exit(1);
     return(1); // quiets the compiler
}
int main (int argc, char **argv)
{
     SOCKET soc = INVALID_SOCKET;
     int iBindType, i, err;
     char szBuffer [ MAX_PATH ];

     // Parse command line
     iBindType = ParseCommandLine(argc, argv, FIXED_PORT_LOCAL_ADDR);

     // initialize WinSock library
     WinSockInitialize();
     // Create a datagram socket
     soc =  CreateSocket ( iBindType, SOCK_DGRAM);
     if (soc == INVALID_SOCKET)
          RETURN_ON_ERROR(1);
```

```
    // Bind the socket
    err = BindSocketToAddr (
                            iBindType,      // Bind Type
                            soc,            // socket
                            szServiceName,  // Service Name
                            szProtocol,     // protocol name
                            TRUE            // This is Client
                            );

    if (err)
    {
         closesocket(soc);
         RETURN_ON_ERROR(1);
    }
    // Connect to the server
    printf("Connecting to Server: %s on Host: %s Protocol: %s\n",
                szServiceName, szHostName, szProtocol);
    err = ConnectToServer ( iBindType, soc, szHostName, szServiceName,
                            szProtocol);
    if (err != 0)
    {
         closesocket(soc);
         RETURN_ON_ERROR(1);
    }
    else
          printf("Connected\n");
    // Loop 'RWLoop' times
    for (i = 0; i < RWLoop; i++)
    {
          printf("Read/Write Loop :%d\n", i );
          // Send a request to the server
          if (fShutDownServer)
               strcpy(szBuffer, SHUTDOWN);
          else
               sprintf(szBuffer, CLIENT_REQUEST, i);
          err = send ( soc, szBuffer, sizeof(szBuffer), 0);
          if (err == SOCKET_ERROR)
               PRINT_ERROR_AND_BREAK("send");

          // Don't expect a message from server if we shut it down
          if (fShutDownServer)
               break;
          // Read server's reply
          err = recv ( soc, szBuffer, sizeof(szBuffer), 0);
          if (err == SOCKET_ERROR)
               PRINT_ERROR_AND_BREAK("recv");
          if (err)
               printf("Server Replied: %s\n", szBuffer );

    }
    // Close the socket
    closesocket ( soc );
    // Cleanup and return
    WSACleanup();
    return (0);
}
```

5.1.8 Terminating a Connection

Once the client and server have completed their communication, they can terminate their connection by calling the shutdown() API to close down the connections and the closesocket() API to close the socket itself. Here are the prototypes of both functions:

```
int PASCAL FAR shutdown     ( SOCKET soc, int how );
int PASCAL FAR closesocket  ( SOCKET soc );
```

The shutdown() function is used to stop data transmissions on the socket specified by the soc parameter. After the shutdown() API has been called, an application cannot send or receive data on the socket. However, the function does not close the socket itself. The application must call the closesocket() function to do this and release any system resources associated with the socket. The shutdown() function is useful for servers, because a busy server that is servicing multiple sockets can shut down a socket and stop receiving client requests. It can then attend to other important business, such as freeing its own data buffers associated with the socket. Note that an application cannot call the connect() API on a shutdown socket and restart a connection with another process. Finally, the shutdown() function does not block even if the socket is a blocking socket.

The shutdown() function takes a parameter, how, which gives more control over how the transmissions are shut down. Setting how to zero causes only the incoming traffic to cease; all recv() function calls by the application calling shutdown() will return an extended error (WSAESHUTDOWN). Note that this may not affect the lower-level protocols, such as TCP or UDP. TCP can continue to receive packets, but may not acknowledge them which leads to network inefficiency, because the remote resends packets from the current TCP window until it gives up and terminates the connection. Similarly, UDP may receive all incoming datagram packets and generate no ICMP (error) packets. On the other hand, setting how to one causes all subsequent sends by the shutdown() API caller to be disallowed. After a socket has been shutdown by setting how to one, all calls to the send() or sendto() APIs will return an extended error (WSAESHUTDOWN). If the socket is using the TCP protocol (that is, the stream socket), the remote end is notified by a FIN command that transmissions are closing. Finally, setting how to two disallows both sends and receives on the socket.

To close a socket, the client or the server calls the closesocket() API.[5] The call releases any system resources associated with the socket, such as binding to protocols, and any queued data. Unlike the shutdown() API with a how value of one, the closesocket() API affects the lower protocols, which shut

5. Note that closesocket() is equivalent to the close() function in UNIX BSD4.3. This is a Windows Socket enhancement.

down by sending an appropriate message to their counterpart on the remote end. Note that the closesocket() API can block on a blocking stream socket until all the unsent data is successfully sent to the recipient.

5.1.9 Asynchronous Operations

While blocking sockets are easy to use and understand, they are not suitable for sophisticated Windows NT client/server applications, nor are they recommended for Windows applications. In a Windows environment, a blocking call causes the application to stop processing user requests, leading to a "hung" program. Due to the single message queue system in the Windows environment, a blocking application adversely affects the entire system. In the Windows NT environment, a single-thread blocking application cannot affect the rest of the system—but the lack of response from the application may lead to an unhappy user. Which means you need to understand how to avoid blocking socket calls.

Table 5-5 shows the calls that may block for an indefinite period of time when called on a blocking socket. Blocking can be avoided by using nonblocking calls on blocking socket [for example, the recv() API called with the MSG_PEEK flag does not block], or by changing the default behavior of the socket from blocking to nonblocking. In this case, the function calls shown in Table 5-5 return immediately with an error (WSAEWOULDBLOCK) whenever the function cannot be completed immediately. In other words, if it is using a nonblocking socket, an application does not block on these functions.

Socket implementations can provide two functions, select() and ioctlsocket(),[6] either of which can be used to avoid blocking on socket calls.

Using a Nonblocking Socket

The simplest way for a function not to block on a socket call is to use a nonblocking socket. A nonblocking socket is created by calling the socket() function to make a socket endpoint, then calling the **ioctlsocket()** function shown here:

```
int PASCAL FAR ioctlsocket ( SOCKET soc, long cmd, u_long FAR * argp );
```

Table 5-5. *Socket Functions That Block*

Socket Function	*Usage Description*
accept()	Used by server
connect()	Used by client
recv()/recvfrom()	Used by both client and server
send()/sendto()	Used by both client and server
close()	Used by both client and server

6. Note that ioctlsocket() is equivalent to the ioctl() function of UNIX 4.3BSD.

The function ioctlsocket() gets a socket descriptor from the soc parameter and can be used to control blocking. The WinSock standard allows only a limited number of commands, which are specified by the cmd parameter. A command may take one or more parameters, which are specified by the argp parameter. Essentially, an application calls the ioctlsocket() function and supplies it with a socket descriptor and a command. This modifies the state of the socket itself. The command that changes the state of the socket from blocking to nonblocking is FIONBIO. When setting the cmd to FIONBIO in the ioctlsocket() API, argp points to an unsigned long number that is set to nonzero to change the socket mode to nonblocking. (Conversely, a nonblocking socket can be changed to a blocking socket by specifying FIONBIO in cmd and setting the unsigned number pointed to by argp to zero.) The following code fragment shows how to use the ioctlsocket() API.

```
SOCKET sock;
// Create a socket
sock = socket ( PF_INET, 0, 0);
if (sock == INVALID_SOCKET)
{
    printf("Error creating socket: %lx\n", WSAGetLastError());
    return (1);
}
// Mark the socket non-blocking
unsigned long ulNonBlocked = 1;
if (ioctlsocket(sock, FIONBIO,
                (u_long FAR *) &ulNonBlocked)==SOCKET_ERROR)
{
    printf("Error making non-blocking socket: %lx\n",
                WSAGetLastError());
    return (1);
}
```

The FIONREAD command can be used to determine the amount of data that can be read from the socket. In this case, argp points at an unsigned long in which the ioctlsocket() function stores the result. For stream sockets, the FIONREAD command returns the total amount of data that can be read in a single recv(); this is usually the same as the total amount of data queued on the socket. For datagram sockets, the FIONREAD command returns the size of the first datagram queued on the socket.

The SIOCATMARK command is used to determine whether all out-of-band data has been read. This applies only to stream sockets that have been configured for in-line reception of any out-of-band data (SO_OOBINLINE). In this case, argp points at a BOOL in which ioctlsocket() stores the result. If no out-of-band data is waiting to be read, the operation returns TRUE. Otherwise, it returns FALSE. The next recv() or recvfrom() performed on the socket retrieves some or all of the data preceding the "mark;" the application should use the SIOCATMARK operation to determine whether any data remains. If there

is any normal data preceding the "urgent" (out-of-band) data, it is received in order. [Note that a recv() or recvfrom() never mixes out-of-band and normal data in the same call.]

Once the socket has been designated as nonblocking, all socket functions return with an error (WSAEWOULDBLOCK) any time a call does not complete immediately. So the good news is that applications do not hang on a given socket call. The not-so-good news is that applications must poll the socket periodically to determine when a function call will complete successfully.

Using a Nonblocking Socket with connect()

Client applications can block on a connect() API call for a long period of time. The time-out value depends on the time-out values and policies of the underlying protocol. Avoid long delays by marking the socket as nonblocking prior to calling the connect() API. If a connection is not available immediately, the connect() function will return the extended error (WSAEWOULDBLOCK) and an application can try to connect at a later time. Once the connection is complete, the application can continue to use the nonblocking socket for all subsequent I/O calls. It is also possible to change the socket to the blocking mode after the connect() function is successful. This is useful in situations when the maximum (or indefinite) delay is encountered in locating and connecting to the server. Subsequent I/O calls complete quickly, allowing use of the blocking socket.

Using select() in Conjunction with accept()

The select() function is the means by which a server can determine if a client request is pending in the request queue. Once the server knows that a client request has arrived, it can call the accept() API and establish a logical connection without blocking on the accept() function.

The select() function prototype appears here. The function is used primarily in the Windows and Windows NT environments to determine the status of one or more sockets and ask for information on read, write, or error status.

```
long PASCAL FAR select ( int nfds,
                         fd_set FAR * readfds,
                         fd_set FAR * writefds,
                         fd_set FAR * exceptfds,
                         const struct timeval FAR * timeout );
typedef struct fd_set {
        u_short fd_count;               /* how many in the SET? */
        SOCKET  fd_array[FD_SETSIZE];   /* an array of SOCKETs */
} fd_set;
struct timeval {
        long     tv_sec;          /* seconds */
        long     tv_usec;         /* and microseconds */
};
```

When using the select() API, specify a set of socket descriptors by using the fd_set structures.[7] Prior to calling the select() API, the socket descriptors are set by using the FD_SET() macro for each socket. The caller can specify up to three such sets in each call to the select() function, one each for determining the read, write, or error status of the sockets. For example, if the caller wants to know if some data has arrived in the incoming message queue of a socket, he will set the socket descriptor of this socket in read socket set. If the caller is interested in determining, say, write status (that is, if the socket can be written into), she can specify null read and error socket set. Finally, the caller can specify a time-out value that includes zero, which allows control over how long select() can block when trying to determine the status before returning. Most applications use a zero time-out value, because they do not want select() to block.

When using the select() function, the read, write, and error socket sets can be specified by the readfds, writefds, and exceptdfs parameters. Currently, most Windows Socket implementations ignore the exception socket set. The time-out value can be specified by the time-out parameter. The maximum number of sockets in the socket set is specified by nfds. Because the maximum number of sockets in a socket set (the fd_set structure) is 64, the nfds parameter is ignored by the WinSock implementations. Finally, the WinSock implementation provides the macros in Table 5-6 to manipulate the socket set (which is an array of socket descriptors).

On successful completion, the select() API returns the number of sockets for which it has information, including zero. It does this by altering the socket set supplied to it prior to the call. It clears from the socket set the socket entries that do not meet the condition and leaves the remaining ones set. For example, say we put four descriptors {a, b, c, d} in the read set and three {a, e, d} in the write set using the FD_SET() macro. At the end of the time-out value of one second, assume that sockets a and d have received data that can be read, and that sockets a and e can have fresh data written into them. In this case, the select() function returns four. Calling the FD_ISSET() function on the {a, d} sockets returns TRUE, indicating that they have met the condition. This is illustrated in Table 5-7.

Table 5-6. *Macros That Manipulate Socket Sets*

Macro	*Description*
FD_SET(sock, *fds)	Sets socket descriptor sock in set pointed by fds
FD_CLR(sock, *fds)	Removes socket descriptor fds in set pointed by fds
FD_ZERO(*fds)	Initializes the set pointed by fds to a NULL set
FD_ISSET(sock, *fds)	Checks if socket descriptor sock is set in fds

7. If you are familiar with UNIX 4.3BSD select(), note that Windows Socket uses sockets arrays, not bitmaps, to define fd_set. Use predefined macros to manipulate the socket set.

Table 5-7. Functions and Return Conditions

Ready to Be Read?	*Ready to Be Written Into?*
FD_ISSET(a, readfds) = TRUE	FD_ISSET(a, writefds) = TRUE
FD_ISSET(b, readfds) = FALSE	N/A
FD_ISSET(c, readfds) = FALSE	N/A
FD_ISSET(d, readfds) = FALSE	FD_ISSET(d, writefds) = TRUE
N/A	FD_ISSET(e, writedfs) = TRUE

There are three ways to use the time-out value to control the amount of time the select() function blocks and the information that is returned by the select() function:

- If the timeout parameter is set to NULL, the select() function blocks indefinitely until one or more of the socket descriptors meet the condition. This option is not too useful, because it does not help an application that is trying to avoid blocking on a socket call.
- If a nonzero time-out value is specified by the time-out parameter, then the select() function blocks until one of the sockets is ready or the specified time has expired. This is useful when an application chooses to block for a (usually small) finite time.
- If a zero time-out value is specified by the time-out parameter, then the select() function returns immediately, collecting status on all sockets specified by socket sets. This is useful for implementing a polling mechanism.

Now that we understand how the select() function works, let us go back to providing a solution for not blocking on the accept() function on a stream socket. To determine if a client is trying to connect, the server puts the socket in a read socket set and calls the select() function. If the client request is pending, then the FD_ISSET() function will return TRUE on the socket, and the server can call the accept() function to establish the connection without blocking. This code fragment shows how this happens.

```
// Assume that the server has already called socket(), listen(), and
// bind() APIs
SOCKET soc;          // created using socket() API
SOCKET socacc;       // Socket descriptor returned by accept() API

// Set socket in the read socket set
fd_set readfds;          // Read socket set
FD_ZERO(&readfds);       // Start with NULL set
FD_SET(soc,&readfds);    // Set the socket descriptor returned from
                         // socket() in read set
//
```

```
// Set zero values in timeout parameter so that select() will return
// immediately. Thus, we are using select() to poll socket status.
struct timeval TimeOut;
TimeOut.tv_sec = 0;
TimeOut.tv_usec = 0;
//
// In this simple example, we loop calling select() until a client
// has connected. In real application, the server can process other
// client requests instead of looping
//
do
{
    // Call select to determine if the socket is ready to be "read"
    if (select(FD_SETSIZE, &readfds, NULL, NULL, &Timeout) !=
          SOCKET_ERROR)
    {
          if (FD_ISSET(soc, &readfds))
          {
               // Client has called connect().
               // Thus, accept() will not block
               sockacc = accept(soc, 0, 0);
               break;
          }
          else
          {
               printf("Waiting for client to connect\n");
               // Process other requests, etc.
          }
    }
    else
    {
          printf("Error in select: %ld\n", WSAGetLastError());
          break;
     }
} while ( TRUE );
```


Using select() with recv()/recvfrom()

Recall that both the recv() and the recvfrom() functions can block for an indefinite time on a blocking socket, and that the select() function can be used to avoid blocking. Recall also that either the recv() or the recvfrom() functions block when the incoming buffer is empty and the receiver has specified a non-null receive buffer. You can avoid blocking on the recv() or recvfrom() functions if you know that one or more bytes are available in the socket. This is where the select() function comes into play. Use the select() function with a small or zero time-out value to determine if a socket is ready to be read (that is, it now contains a stream of bytes that have been sent to it). The following code fragment shows the use of the select() function to determine if a socket is ready to be read by the application receiving data.

```
// Assume that the server has already called socket(), listen(), and
// bind(), and accept() APIs
SOCKET socacc;                //     obtained from accept() API
```

```
// Set socket in the read socket set
fd_set readfds;                 // Read socket set
FD_ZERO(&readfds);              // Start with NULL set
FD_SET(socacc,&readfds);        // Set the socket descriptor returned from
                                // socket() in read set
//
// We wait 100 ms in each call to select()
//
struct timeval TimeOut;
TimeOut.tv_sec = 0;
TimeOut.tv_usec = 100;
//
// In this simple example, we loop calling select() until data
// is available. In a real application, the server can process other
// requests instead of looping.
//
do
{
    // Call select to determine if the socket is ready to be "read"
    if (select(FD_SETSIZE, &readfds, NULL, NULL, &Timeout) !=
          SOCKET_ERROR)
    {
        if (FD_ISSET(socacc, &readfds))
        {
              // One or more bytes available to read.
              // Thus, recv() will not block
              nBytesReceived = recv(socacc,...);
              break;
        }
        else
        {
              printf("Waiting to receive data\n");
              // Process other requests, etc.
        }
    }
    else
    {
        printf("Error in select: %ld\n", WSAGetLastError());
        break;
    }
} while ( TRUE );
```

Using select() with send()/sendto()

You can avoid blocking on send() or sendto() functions if you can be sure no more data can be written into the socket. This is where the select() function comes into play. Use the select() function with small or zero time-out value to determine if a socket to be written to is ready. The following code fragment shows how to use the select() function to check the socket's write status.

```
// Assume the server has already called socket(), listen(), and
// bind(), and accept() APIs
SOCKET socacc;              // obtained from accept() API
// Set socket in the write socket set
fd_set writefds;            // Write socket set
```

```
FD_ZERO(&writefds);          // Start with NULL set
FD_SET(socacc,&writefds);    // Set the socket descriptor returned from
                             // socket() in write set
//
// We wait 100 ms in each call to select()
//
struct timeval TimeOut;
TimeOut.tv_sec = 0;
TimeOut.tv_usec = 100;
//
// In this simple example, we loop calling select() until room becomes
// available for new data to be written into the socket.
//
do
{
    // Call select to determine if the socket is ready to be "written
    // into"
    if (select(FD_SETSIZE, NULL, &writefds, NULL, &Timeout) !=
        SOCKET_ERROR)
    {
        if (FD_ISSET(socacc, &writefds))
        {
               // Data can be sent without blocking
               nBytesSent = send(socacc,...);
               break;
        }
        else
        {
               printf("Waiting to send data\n");
               // Process other requests etc.
        }
    }
    else
    {
        printf("Error in select: %ld\n", WSAGetLastError());
        break;
    }
} while ( TRUE );
```


5.1.10 Out-of-Band Data Processing

Stream sockets are very useful for sending a stream of bytes from one process to another. The data flowing in this manner are called in-line data. However, applications at either end occasionally need to communicate with each other without interrupting the regular flow of in-line data. Such communication can be achieved by **out-of-band (OOB)** data, which are sent at a higher priority to the receiver by the same socket as that used for the in-line data transfer.

The advantage of the OOB data system is that the data goes from sender to receiver directly, without any wait at the end of the incoming data stream. The receiver can recognize the OOB data as it arrives, take the necessary action, and then continue to process the byte stream that is arriving through the normal channel. For example, in a client/server system, both client and server

can use OOB means to communicate exceptional conditions (such as the user pressing <control><c>) and normal channels for data transfer.

However, Microsoft Windows Sockets documentation advises applications *not* to use OOB data unless it is absolutely necessary. This recommendation stems from the fact that the semantics of OOB data first defined in RFC 793 differ from their implementation in the Berkeley Software Distribution; BSD implementation differs from the implementation notes defined in RFC 1122. Additionally, Windows NT supports only one byte of OOB data, not the second channel envisioned in RFC 793.

OOB data can only be sent and received on a stream socket. Specify MSG_OOB in the flags parameter of the send() and recv() functions to do this. The receiver detects the arrival of OOB data by calling the ioctlsocket() API; once the OOB data have arrived, the receiver has two options for processing it. The receiver can check the OOB data without removing the data from the socket buffers, or it can read the out-of-band data using the recv() function that removes the data from the socket buffer. The first option allows the receiver to judge the value of the out-of-band data. If the receiver determines that it must process the data immediately, it can call the recv() function to remove the out-of-band data from the socket stream. It can also choose to ignore the sender-perceived priority of the message and process it at a later time. In any event, a typical OOB receiver calls the recv() function with MSG_OOB set to receive, and processes the OOB data before proceeding to the in-line data. A socket application can specify that OOB data be given to it as in-line or normal data, and leave it to the application to decipher the urgency of the OOB.

5.2 WINDOWS SOCKETS EXTENSIONS

This section focuses on the extensions beyond the UNIX socket interfaces that make it possible for socket-based applications to work well in the Windows and Windows NT environments. See [Stephens 90] for a standard reference for UNIX socket programming. Furthermore, these extensions form the guidelines to which Windows Socket Library providers adhere. These guidelines can lead to communication interoperability among applications that run on different platforms.

WinSock extensions have been designed to "marry" two orthogonal programming paradigms:

- **A nonblocking I/O over a socket requires polling/looping mechanisms.** Earlier sections illustrated the functions [ioctlsocket() and select()] by which nonblocking communication is achieved over Berkeley-style sockets. When the socket mode is changed to nonblocking, the application must implement a polling (and looping)

mechanism to determine the end of an I/O operation. Using the select() function to determine the I/O status of a blocking socket also requires certain looping mechanisms.[8] This indicates a procedural programming paradigm.

- **The message-driven Windows and Windows NT application programming paradigm.** While Berkeley sockets interfaces were designed to work with the UNIX environment's procedural programming paradigm, using the same interfaces in the Windows and Windows NT environments leads to unwieldy applications and complicated code. Note that for Windows NT console applications and WIN32 Services, the Berkeley socket interfaces are the most appropriate.

The WinSock extensions combine the two programming paradigms by defining a new set of interfaces that help to write message-driven socket applications. This allows applications to use nonblocking socket calls without having to implement their own polling/looping mechanisms. Table 5-8 shows all the WinSock extensions.

5.2.1 Startup and Cleanup Functions

Every WinSock application must call the WSAStartup() function before making any socket call, and it must call the WSACleanup() function before exiting. When calling the WSAStartup(), an application supplies the version number (using the wVersionRequired parameter) that the WinSock DLL will support.

```
int PASCAL FAR WSAStartup ( WORD wVersionRequired,
                    LPWSADATA lpWSAData );
int PASCAL FAR WSACleanup ( void );

struct WSAData {
    // WinSock implementation version
    WORD                wVersion;
    // Highest version supported. Usually same as wVersion
    WORD                wHighVersion;
    // An ASCII description of WinSock DLL implementation
    char                szDescription[WSADESCRIPTION_LEN+1];
    // An ASCII status or configuration information string
    char                szSystemStatus[WSASYSSTATUS_LEN+1];
    // Maximum number of sockets that a process can potentially open
    unsigned short      iMaxSockets;
    // The max. size of UDP datagram that WinSock app can send
    // or receive. The minimum is 512.
    unsigned short      iMaxUdpDg;
```

8. Implement the polling mechanism by using the WM_TIMER message. In Windows NT, use threads and events for handling the nonblocking sockets.

Table 5-8. Windows Socket Extension Functions

Windows Socket Extensions	*Brief Description*
Setup and Cleanup Functions (necessary for all WinSock applications)	
WSAStartup()	Registers application with Windows socket implementation
WSACleanup()	Frees any resources allocated by WinSock implementation for this application
Functions for Handling Blocked I/O	
WSAIsBlocking()	Check if a call to socket has blocked
WSACancelBlockingCall()	Cancel a call to Windows socket that might be blocked
WSASetBlockingHook()	Associate an application-supplied function with WinSock that is called when a socket is blocked
WSAUnhookBlockingHook()	Disassociate the application-supplied function called when a socket is blocked
Asynchronous Database Functions	
WSAAsyncGetServByName()	Asynchronous form of getservbyname()
WSAAsyncGetServByPort()	Asynchronous form of getservbyproto()
WSAAsyncGetProtoByName()	Asynchronous form of getprotobyname()
WSAAsyncGetProtoByNumber()	Asynchronous form of getprotobynumber()
WSAAsyncGetHostByName()	Asynchronous form of gethostbyname()
WSAAsyncGetHostByAddr()	Asynchronous form of gethostbyaddr()
Asynchronous I/O Functions	
WSACancelAsyncRequest()	Asynchronously cancel a pending request made
WSAAsyncSelect()	Asynchronous form of select()
WSARecvEx()	Asynchronous version of recv() available in Windows NT only
Error Handling Functions	
WSASetLastError()	Sets error code, maps to GetLastError() Win32 API
WSAGetLastError()	Gets error code, maps to SetLastError() Win32 API

```
    // A pointer to vendor-specific information
     char FAR *        lpVendorInfo
}WSADATA, FAR* LPWSADATA;

// Asking for version 1.1
int WinSockInitialize()
{
     int err;
     WORD wVersionReq;
     WSADATA wsaData;
     // All applications must call WSAStartup()
     // before calling any socket functions
     wVersionReq = MAKEWORD (1, 1);
     err = WSAStartup(wVersionReq, &wsaData);
     if (err != 0)
          exit(1);
     return err;
}
```

The WSAStartup() API causes the application to activate the WinSockDLL. If the WinSock implementation supports the version requested by the application,[9] it returns zero and fills in the WSAData structure that the lpWSAData parameter points to in the WSAStartup() function. The WSA Startup() function allows the WinSock implementation to start keeping track of the application, and causes a reference count to be set to one when called the first time.

The WSAData structure also supplies two pieces of information that are invaluable for an application, the maximum number of sockets (using iMaxSockets) that a given process can open, and the maximum size of datagram packet (using iMaxUdpDg) that the application can safely send or receive. This latter information applies to applications that use datagram sockets, as they can determine packet sizes at run time. Whenever applicable, the maximum datagram size is used as a guide for defining the application data buffer size.

An application can call WSAStartup() many times (presumably to get WSADATA), but it cannot request different version numbers after the first call. Each subsequent WSACleanup() call decrements the reference count. When a WinSock application makes the last WSACleanup() call, the implementation releases any resources it may have allocated for the application. If a socket is blocked, the call to WSACleanup() fails until the blocking call is completed or canceled. If there are data to be sent even after closesocket() has completed on that socket, WSACleanup() will not affect that data's delivery (that is, pending data will still be sent).

9. The implementation may support more versions than the one requested by the application.

5.2.2 Error-Handling Functions

Traditionally, socket implementations and libraries follow Berkeley UNIX conventions, returning minus one on an error and setting *errno* to the reason for error. This scheme has two problems:

- In BSD, all return values are declared as C *int* type. This immediately leads to problems, because the C *int* can be either a 16- or a 32-bit number, depending on the compiler and the operating system. Furthermore, the byte ordering is different between little endian (Intel 80x86) and big endian (Motorola 6800) machines. This causes an acute problem in relaying the results of an operation in a distributed system in which the client may be on a 16-bit x86 platform and the server on a 32-bit DEC Alpha machine.
- While a single process-wide error variable (*errno*) makes sense in a single-thread environment, it leads to increased complexity and inefficiency in a multithread environment such as Windows NT. Every thread in Windows NT has its own error status, which can be determined by calling the GetLastError() API. An error value is set by calling the **SetLastError()** API.

The WinSock extension solves both problems, first by using portable macro ERROR_INVALID_SOCKET and second by specifying that all applications call the WSAGetLastError() API to determine the exact error code. This scheme makes the socket-based application portable across multiple hardware platforms and single or multithread environments. The prototype of WSAGetLastError() and WSASetLastError() functions appear below. The latter function is used by Windows Socket Library implementors to set the error code.

```
int  PASCAL FAR WSAGetLastError ( void );
void PASCAL FAR WSASetLastError ( int iError );
```

Applications should use predefined macros and constants to handle (or set) errors rather than relying on specific values (for example, use WSAEINPROGRESS, not 10036). In the WinSock interface, all error code macros are prefixed with WSA (for example, WSAEALREADY). For compatibility and portability, the standard socket error values have remained the same as here in Table 5-9:

Table 5-9. *Sockets and Their Error Values*

WinSock Extension	*Berkeley Style*	*Error Value*
WSAEWOULDBLOCK	EWOULDBLOCK	10035

For future compatibility, new applications should use the Windows Socket Interface-style error macros rather than Berkeley-style macros.

5.2.3 Functions for Handling Blocked Sockets

There is a large base of UNIX and socket-based public domain, as well as commercial software, so one of the major challenges faced by the socket-based-application writer is porting existing Berkeley-socket-based applications to a Windows environment. Most of these applications use blocked sockets, perhaps devoting a process to handling socket calls. (UNIX programmers typically use the fork() system call to start a child process.) Porting these applications to Windows NT may be comparatively easy, because Windows NT is a multitasking system with a 32-bit system that supports thread. (Because Windows NT does have support for fork(), it may be hard to port some applications.) However, porting these applications to a 16-bit Windows environment is difficult, because there is no multi-threading support. In either case, these ports still do not work with the message-based Windows programming paradigm, because each blocked call hinders the processing of the rest of the messages present in the message queue.[10]

Porting is not the only occasion for using blocked sockets. Sometimes, an application adds a feature—to, say, send and receive files—that uses sockets over TCP/IP. The file transfer capability is not the prime focus of the application, so it does not make sense to add sophisticated code. The designer can use blocked sockets as long as there is some means by which this new feature does not degrade the rest of the system or the application.

To help those applications that use blocked sockets, WinSock provides two features:

- In a nonthreaded Windows environment such as Windows 3.1, while the socket call itself blocks, the WinSock implementation keeps processing the messages for the application and sending it to the application. Thus, an application becomes re-entrant and does not become totally unresponsive. The application can process other messages, such as WM_PAINT. Once the socket call completes, the application can complete the message processing that started the socket call in the first place. Figure 5-6 shows how an application that is blocked on a connect() call can continue to receive messages as if the connect() function were not blocked at all. The application calls the connect() function when it receives the

10. Recall that while Windows NT has a message queue for each Win32 application, it (by default, for compatibility reasons) has only one message queue for all Win16 applications. Starting with Windows NT 3.5, you can ask that NT have a separate message queue for each Win16 application.

user-defined WM_CONNECT command (Figure 5-6a). This call to the connect() function may block for some time. However, the WinSock default message-handling system ensures that the application continues to receive messages. Thus, the application can receive a WM_PAINT message (Figure 5-6b) even though it is still blocked on the connect() function. Once, the connect() call completes, the application continues to execute (Figure 5-6c).

- In multithread Windows environments such as Windows NT, there is no such default nonblocking mechanism in place, and the thread making the socket call blocks while other threads continue. Additionally, in a Windows NT environment a thread can only make one blocked socket call at a time.

Furthermore, the Windows Socket Interface provides the following helper functions that give applications more control over handling a blocking socket.

```
BOOL      PASCAL FAR WSAIsBlocking ( void );
int       PASCAL FAR WSACancelBlockingCall ( void );
FARPROC   PASCAL FAR WSASetBlockingHook ( FARPROC lpBlockFunc );
int       PASCAL FAR WSAUnhookBlockingHook ( void );
```

In a nonmultithread environment (such as Windows 3.x), an application is limited to one blocking socket call at a time. For example, in Figure 5.6, the application cannot close the socket by calling the closesocket() function until after the connect() function has completed (or aborted). In such situations, an application can use the WSAIsBlocking() call to determine if it is currently blocking on any socket call, and the **WSAIsBlocking()** function returns TRUE if there is a pending call. In the Windows 3.x environment, the application can cancel a blocked call by calling the **WSACancelBlockingCall()** API.

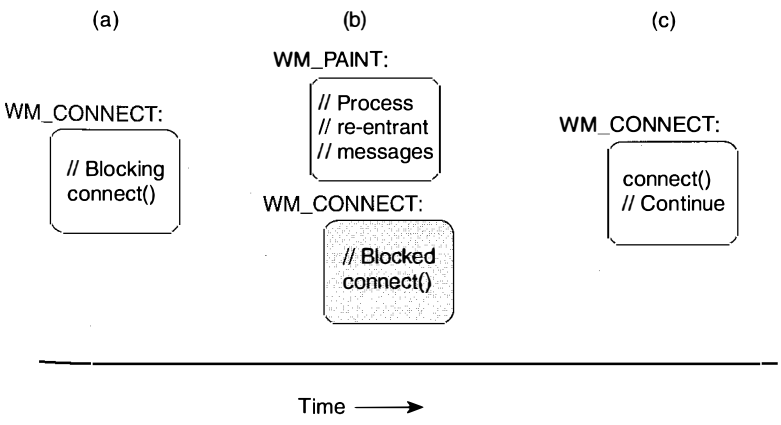


Figure 5-6. Application Using Blocked Windows Socket

A call to the WSACancelBlockingCall() API causes the blocked socket call to return as soon as possible with an error-interupted message, WSAEINTR. When it is cancelled, the state of the socket depends on the particular call that was blocked. If a connect() function is canceled, some resources may not be released until the socket is closed, and this will adversely affect the application when it makes subsequent connect() calls. Canceling the select() or the accept() functions does not affect the socket; the socket can be used later without any restraints. However, any other socket call that is canceled may leave the socket in an indefinite state, and the application may not be allowed any further operations on the socket except to close it. Applications should therefore close a socket after a blocked socket call has been canceled—unless the blocked call was open(), accept(), or select().

Sophisticated applications use their own handlers to deal with a blocked socket and thus override the default blocking socket handling function of Windows Socket Library. They can use the **WSASetBlockingHook()** API to install their own handler, and remove it using the **WSAUnhookBlocking Hook()** API. The custom blocked-socket handling function that is set by WSASetBlockingHook() applies to the entire task in a Windows environment, but it is restricted to the thread calling the WSASetBlockingHook() API in Win32. In Win32, each thread that wants to use its custom blocking function has to call the WSASetBlockingHook() function. (These functions are necessary for only a very few applications and therefore will not be explored any further. See [MS93a] for a comprehensive example of the usage of these functions.)

5.2.4 Asynchronous Database Functions

Windows Socket provides asynchronous database functions that send a message to an application upon completion of a function. This provides the option of combining asynchronous programming with Windows message-driven programming.

The prototype of these functions appears here. Note that WinSock also supports the Berkeley socket (synchronous) database functions for backward compatibility. However, new applications should use the asynchronous database functions.

```
HANDLE PASCAL FAR
     WSAAsyncGetServByName ( HWND hWnd,
                             unsigned int wMsg,
                             const char FAR * name,
                             const char FAR * proto,
                             char FAR * buf,
                             int buflen );
HANDLE PASCAL FAR
     WSAAsyncGetServByPort ( HWND hWnd,
                             unsigned int wMsg,
                             int port,
```

```
                        const char FAR * proto,
                        char FAR * buf,
                        int buflen );
HANDLE PASCAL FAR
     WSAAsyncGetProtoByName ( HWND hWnd,
                        unsigned int wMsg,
                        const char FAR * name,
                        char FAR * buf,
                        int buflen );
HANDLE PASCAL FAR
     WSAAsyncGetProtoByNumber ( HWND hWnd,
                        unsigned int wMsg,
                        int number,
                        char FAR * buf,
                        int buflen );
HANDLE PASCAL FAR
     WSAAsyncGetHostByAddr ( HWND hWnd,
                        unsigned int wMsg,
                        const char FAR * addr,
                        int len,
                        int type,
                        char FAR * buf,
                        int buflen );
HANDLE PASCAL FAR
     WSAAsyncGetHostByName ( HWND hWnd,
                        unsigned int wMsg,
                        const char FAR * name,
                        char FAR * buf,
                        int buflen );
int PASCAL FAR WSACancelAsyncRequest     ( HANDLE hAsyncTaskHandle );
```

These functions execute the database function asynchronously and eventually return the result to a window identified by the first parameter, hwnd. An application supplies the message it wants to receive by using the wMsg parameter (for example, WM_ASYNC_DB). When an asynchronous database lookup function posts successfully, it returns a handle. A zero-value handle indicates an error. You can determine the cause of the error by calling the WSAGetLastError() API. This handle can be used to cancel a pending request by calling the **WSACancelAsyncRequest()** API.

When the asynchronous request completes, the application-specified window receives a completion message. The wParam parameter contains the handle returned by the asynchronous function when the function was first called. An application can use this handle to identify the database function that has completed. The lParam is divided into two logical parts, the high 16 bits and the low 16 bits. The values contained within the lParam are as follows:

- The high 16 bits indicate any error that might have occurred. A zero value indicates success. The following macro can be used to determine an error value.

```
#define WSAGETASYNCERROR(lParam)     HIWORD(lParam)
#define WSAGETASYNCBUFLEN(lParam)    LOWORD(lParam)
```

- The low 16-bit point reports the buffer length that was filled by the database function. The application must supply a sufficiently large buffer[11] when making the asynchronous database function call. It is also imperative that the buffer supplied to the database function is not released until after the completion (or cancellation) of the function. Furthermore, in a Windows 3.1 environment, the buffer should be locked in memory so that it does not move during the asynchronous database function call.

The following code fragment shows the usage of the **WSAAsync GetHostByName()** function. In this example, assume a client is trying to connect to the server (EchoServer) and that it uses the WSAAsyncGetHostByName() function to map the logical name of the server to a network address. The client supplies the network address to the connect() function so it can connect to the server (code not shown). The program calls WSAAsyncGetHostByName() function when it receives a WM_GETHOSTADDRESS (user defined) message. The caller specifies that it wants to receive WM_ASYNC_DB message upon completion of the database function. It supplies a buffer (using the buf variable) in which the database function returns the server address. On completion of the WSAAsyncGetHostByName() function, the application receives WM_ASYNC_DB message.


```
////////////////////////////////////////////////////////////////////////
// PROGRAM: Client4.c
// (see %BOOK%\client4.mak for full details)
////////////////////////////////////////////////////////////////////////

#include "client4.h"
#include "sockutil.h"

// Globals
HINSTANCE hInst;  // current instance

char szAppName[] = "CLIENT4";                // The name of this application
char szTitle[]   = "Test of WSAGetAsyncHostByName";   // The title bar text

// Message Buffer which is used to display the messages
char szMessageBuffer [ MAX_PATH ];
// error messages buffer
char szErrorMsg [ 100 ] ;

LRESULT CALLBACK WndProc( HWND hWnd,UINT message,WPARAM wParam,
LPARAM lParam);
```

11. The recommended buffer size varies depending on the function call being made and is distinctly documented for each call in Windows Socket documentations

```
////////////////////////////////////////////////////////////////////
// FUNCTION: WinMain(HINSTANCE, HINSTANCE, LPSTR, int)
////////////////////////////////////////////////////////////////////
int APIENTRY WinMain( HINSTANCE hInstance, HINSTANCE hPrevInstance,
                      LPSTR lpCmdLine, int nCmdShow)

{
    MSG msg;
    WNDCLASS  wc;
    HWND     hWnd; // Main window handle.
    // If first instance, register app's window class
    if (!hPrevInstance)
    {
        // Fill in window class structure with parameters that
        // describe the main window.

        wc.style         = CS_HREDRAW | CS_VREDRAW;      // Class style(s).
        wc.lpfnWndProc   = (WNDPROC)WndProc;             // Window Procedure
        wc.cbClsExtra    = 0;                    // No per-class extra data.
        wc.cbWndExtra    = 0;                   // No per-window extra data.
        wc.hInstance     = hInstance;                // Owner of this class
        wc.hIcon         = LoadIcon (hInstance, szAppName);     // Icon name
        wc.hCursor       = LoadCursor(NULL, IDC_ARROW);           // Cursor
        wc.hbrBackground = (HBRUSH)(COLOR_WINDOW+1);        // Default color
        wc.lpszMenuName  = szAppName;                 // Menu name from .RC
        wc.lpszClassName = szAppName;              // Name to register as

        // Register the window class and return success/failure code.
        if (!RegisterClass(&wc))
              return (FALSE);              // Exits if unable to register
    }
    // Save the instance handle in static variable,
    hInst = hInstance;

    // Create a main window for this application instance.
    hWnd = CreateWindow(
              szAppName,                      // See RegisterClass() call.
              szTitle,                       // Text for window title bar.
              WS_OVERLAPPEDWINDOW,                        // Window style.
              CW_USEDEFAULT, 0, CW_USEDEFAULT, 0,   //default positioning
              NULL,                  // Overlapped windows have no parent.
              NULL,                           // Use the window class menu.
              hInstance,          .     // This instance owns this window.
              NULL               // We don't use any data in our WM_CREATE
             );

    // If window could not be created, return "failure"
    if (!hWnd)
        return (FALSE);

    // Initialize Windows Socket
    WinSockInitialize();
    // Make the window visible; update its client area;
    ShowWindow(hWnd, nCmdShow);  // Show the window
    UpdateWindow(hWnd);          // Sends WM_PAINT message

    //Acquire and dispatch messages until a WM_QUIT message is received.
    while (GetMessage(&msg,      // message structure
```

```
                        NULL,     // handle of window receiving the message
                        0,        // lowest message to examine
                        0))       // highest message to examine
    {
        TranslateMessage(&msg); // Translates virtual key code
        DispatchMessage(&msg);  // Dispatches message to window
    }
    // WinSock Cleanup
    WSACleanup();
    return (msg.wParam); // Returns the value from PostQuitMessage
}
//////////////////////////////////////////////////////////////////////
//
// FUNCTION: WndProc(HWND, UINT, WPARAM, LPARAM)
//
//////////////////////////////////////////////////////////////////////
LRESULT CALLBACK
WndProc( HWND hWnd,UINT message, WPARAM wParam, LPARAM lParam)
{
    // Window specific variables
    int wmId, wmEvent;
    // Async. DB call specific variables
    static HANDLE     hAsyncDB;
    static char   szBuffer [ MAXGETHOSTSTRUCT ];
    struct hostent FAR *pHostAddr;
    struct in_addr in;
    switch (message)
    {
    case WM_COMMAND:  // message: command from application menu
          wmId        = LOWORD(wParam);
          wmEvent     = HIWORD(wParam);
          switch     (wmId)
          {
          case IDM_EXIT:
               DestroyWindow (hWnd);
               break;
          case IDM_GETHOSTADDRESS:
               hAsyncDB =
                    WSAAsyncGetHostByName (hWnd,              // Window
                                           WM_ASYNC_DB,       // message
                                           "EchoServer",      // host name
                                           (LPSTR)&szBuffer, // buffer
                                           sizeof(szBuffer)// buffer length
                                           );
               if (hAsyncDB == NULL )
               {
                    wsprintf(szErrorMsg, "WSAAsyncGetHost..Error %ld\n",
                         WSAGetLastError());
                    MessageBox ( NULL, szErrorMsg, "Error", MB_OK );
               }
               break;
        default:
               return (DefWindowProc(hWnd, message, wParam, lParam));
        }
        break;
```

```
    case WM_ASYNC_DB:
        // First check the message for the correct DB query
        // by checking wParam with the handle
        if (hAsyncDB != (HANDLE)wParam)
        {
               MessageBox ( NULL, "wParam does not match handle", "Error",
                     MB_OK ); break;
        }
        // Next check if any error occured
        if (WSAGETASYNCERROR(lParam) != 0)
        {
               wsprintf(szErrorMsg, "WSAAsyncGetHost.. Sent Error %ld\n",
                        WSAGETASYNCERROR(lParam));
               MessageBox ( NULL, szErrorMsg, "Error", MB_OK );
               break;
        }
        // Determine the size of the sent buffer
        if (WSAGETASYNCBUFLEN(lParam) >= sizeof(struct hostent))
        {
               pHostAddr = (struct hostent *) &szBuffer[0];
               in.s_addr = *((u_long *)pHostAddr->h_addr);
               wsprintf( szMessageBuffer,
                         "Host Name: %s Address Type: %x "\
                         "Address Length: %d Host Address: %lx "\
                         "Dotted Address: %s",
                         pHostAddr->h_name,
                         pHostAddr->h_addrtype,
                         pHostAddr->h_length,
                         pHostAddr->h_addr,
                         inet_ntoa(in)
                       );
               MessageBox ( NULL, szMessageBuffer, "Result", MB_OK);
        }
        else
        {
               wsprintf(szErrorMsg, "WSAAsyncGetHost.. Returned Buffer
                        Length %ld\n", WSAGETASYNCBUFLEN(lParam));
               MessageBox ( NULL, szErrorMsg, "Error", MB_OK );
        }
        break;
    case WM_DESTROY:  // message: window being destroyed
        PostQuitMessage(0);
        break;
    default:          // Passes it on if unproccessed
        return (DefWindowProc(hWnd, message, wParam, lParam));
    }
    return (0);
}
```

5.2.5 Asynchronous Select

The **WSAAsyncSelect()** API allows applications to be socket-state-driven as well as message-driven. The benefit of being socket-state-driven is that an application reacts to changes in socket state, rather than functioning as a single monolithic code full of if-then-else and while-do constructs. An immediate

benefit, in my opinion, is the readability and maintainability of the code. The advantage of being both socket-state-driven and message-driven is that it fits in nicely with the Windows message-driven paradigm and makes programs easy to understand and debug. Note that the Windows NT console applications and Win32 services are not able to use the message-oriented WSA AsyncSelect() API.

An application calls the WSAAsyncSelect() API to make itself available to receive certain messages when the state of the socket changes. Whenever that state changes, an event occurs and the application receives a specified message. For example, a socket-based server can request, by using the WSAAsyncSelect() API, that it receive a message whenever a new client connection request comes in. Upon receiving such a message, it can then call the accept() function and start communicating with the client. Notice that this contrasts with the server that blocks on the accept() function that is waiting for client requests to come in. Thus, by using the WSAAsyncSelect() function, a server immediately becomes message-driven and nonblocking.

```
int PASCAL FAR WSAAsyncSelect     ( SOCKET   soc,
                                    HWND     hWnd,
                                    unsigned int wMsg,
                                    long     lEvent );
```


As the prototype above shows, when you call the WSAAsyncSelect() function, specify the socket of interest (using the soc parameter). Specify the window (using the hWnd parameter) that should receive the message from WinSock, and specify the message (using the wMsg parameter) that you wish to receive. Finally, specify one or more events (using the lEvent parameter) in which you are interested. Table 5-10 shows several events and their relevance to socket programming. Specify your interest in multiple events by passing (using lEvent) OR-ed event values.

An application can call the WSAAsyncSelect() function many times on the same socket. However, the last call overrides all previous calls: If you use the WSAAsyncSelect() API to receive, say, FD_READ the first time and FD_WRITE the second time, then the application will only receive the FD_WRITE notifications. Similarly, you cannot register varying messages (using the wMsg parameter) for different events. The application distinguishes notifications for different events by using the WSAAsyncSelect() implementation, which sends (using wParam) the socket handle whose state has changed. Furthermore, the low 16-bit part of lParam shows the event itself for which notification was sent, and the high 16-bit part of lParam shows error, if any. The following two macros can be used to determine the event and error value.

```
#define WSAGETSELECTERROR(lParam)      HIWORD(lParam)
#define WSAGETSELECTEVENT(lParam)      LOWORD(lParam)
```

Table 5-10. Events to Register for with WSAAsyncSelect()

Event	*Function Called on Receiving the Event*	*Behavior Desired*
FD_READ	recv() or recvfrom()	Notify application when socket has data to be read
FD_WRITE	send() or sendto()	Notify application when socket buffers are free so that new data can be written into it
FD_CONNECT	send(), recv() etc	Notify application that connect() on a nonblocking socket has completed
FD_ACCEPT	accept()	Notify application that client request to connect has arrived
FD_CLOSE	close()	Notify application that socket is closed
FD_OOB	recv()	Notify application when out-of-band data has arrived

An application can, of course, stop receiving notifications. This is done by passing in zero in the last two parameters:

```
[WSAAsyncSelect (soc,hWnd,0,0)]
```

This forces WinSock to immediately stop sending messages to the application. However, an application may have some unprocessed messages still waiting in its message queue. The application should be prepared to receive some messages even after it has called to cancel all notifications.

WinSock sends only one notification for each state change. For example, when new data arrives in a socket, an application receives only one FD_READ event. However, if an application makes an appropriate socket call (also called a re-enabling function), the socket implementation sends the next message without calling the WSAAsyncSelect() function again. In the example above, the event notification of FD_READ is received by the application after it calls the recv() function to remove data from the socket. In other words, the recv() function re-enables the FD_READ notification. Table 5-11 shows the events and their enabling functions.

The next code shows a server that uses the WSAAsyncSelect() function to process client requests. While the server code can handle as many clients as the implementation allows, the code fragment shows only one variable for the socket (soc) rather than an array of sockets. Similarly, while the server code can accept multiple requests, only one variable (socacc) keeps track of the socket number returned by the accept() function. The server creates a blocking socket. Then, instead of calling the accept() function, it calls the

Table 5-11. Asynchronous Events and Re-enabling Functions

Event	*Re-enabling Function*	*Description*
FD_READ	recv() or recvfrom()	Reading all or part of the data available in the socket re-enables the event notification.
FD_WRITE	send() or sento	Writing one or more bytes into the socket re-enables this event.
FD_ACCEPT	accept()	Once accept() has been called, application should expect this event again when a new client request arrives.
FD_CLOSE	none	This event signals closure of the socket. No further notification is received from change of state of that socket.
FD_CONNECT	none	Once the socket is connected, it cannot be reconnected. Notification is not re-enabled.
FD_OOB	recv()	Reading out-of-band data causes renewal of OOB notification.

WSAAsyncSelect() function to notify it when a client connects. When the client request to connect comes in, the server receives the FD_ACCEPT event, and it can call the accept() API without blocking. Next, it registers its interest in receiving FD_READ, FD_WRITE, and FD_CLOSE notifications on the accepted socket. Notice that you could have registered to receive read/write/close/accept notifications on the listening socket, which, by inheritance, would have allowed the accepted socket to receive this event. However, a listening socket never receives read/write/close notifications, just as an accepted socket never receives an FD_ACCEPT event.

Once a socket has been accepted, the server receives an FD_WRITE event. This is ignored in the code fragment, because there is no data to send to the client yet (fDataToBeSent = FALSE). In this example, the client and the server follow a simple model: client sends a request that is received by the server, who sends a reply back to the client. Once the client-sent data arrives, the server gets an FD_READ event. The server can either read in the entire message, or it can read in part of it [by varying the size of buffer in the recv() function]. When the server reads in only part of the request, the socket implementation sends another FD_READ event at the completion of the recv() function. This mechanism makes it easy for the server to handle messages only as big as its buffer can hold. After processing the client request, the server calls the send() function to reply to the client.

Notice that upon receiving the FD_ACCEPT notification, the server registers to receive FD_CLOSE notifications as well. Thus, should the client call

shutdown() or closesocket(), the server will receive FD_CLOSE notification, and it can throw away pending buffers and close the accepted socket.

```
////////////////////////////////////////////////////////////////////////
// PROGRAM: Server5.c
// Usage: Run Server.exe against Client1.exe
// (See Server5.mak, Server5.rc, Server5.h in %BOOK%\winsock for
// complete listing)
////////////////////////////////////////////////////////////////////////

#include "Server5.h"
#include "sockutil.h"
// Globals
HINSTANCE hInst;     // current instance

char szAppName[]    = "Server5";   // The name of this application
char szTitle[]      = "Server using WSAAsyncSelect";  // The title bar text
char szDialogName[] = "I/O Box";   // Title on dialog box

// Send/Receive Message Buffers
char szMessageBuffer  [ MAX_PATH ];
char szSendBuffer     [ MAX_PATH ];
// error messages buffer
char szErrorMsg [ 100 ] ;
#define SHOW_ERROR(s, err) {\
     wsprintf(szErrorMsg, "Error: %ld in %s at Line: %d\n", \
              err, s, __LINE__);\
     MessageBox (NULL, szErrorMsg, "Error!", MB_OK);\
}
#define SHOW_ERROR_AND_BREAK(s) {\
     { SHOW_ERROR(s, WSAGetLastError()) \
        break;}\
}
#define DISPLAY_ON_SCREEN(s) {\
     {if (s != NULL) strcpy(szMessageBuffer, s);\
      SendMessage(hwndList, LB_ADDSTRING, \
                  0, (LONG)(LPTSTR)szMessageBuffer);} \
}
// internal functions
LRESULT CALLBACK WndProc (HWND hWnd,UINT message,WPARAM wParam,
                          LPARAM lParam);
void     CheckConnection          ( HWND hWnd, SOCKET soc, SOCKET socacc);
void     GetReadyForNewClient     ( HWND hWnd, SOCKET soc, SOCKET socacc);

////////////////////////////////////////////////////////////////////////
// FUNCTION: WinMain(HINSTANCE, HINSTANCE, LPSTR, int)
//
////////////////////////////////////////////////////////////////////////
int APIENTRY WinMain(
                    HINSTANCE hInstance,
                    HINSTANCE hPrevInstance,
                    LPSTR lpCmdLine,
                    int nCmdShow)
{
     MSG msg;
```

```
WNDCLASS  wc;
HWND     hWnd; // Main window handle.
// If first instance, register app's window class
if (!hPrevInstance)
{
     // Fill in window class structure with parameters that
     // describe the main window.
     wc.style         = CS_HREDRAW | CS_VREDRAW;     // Class style(s).
     wc.lpfnWndProc   = (WNDPROC)WndProc; // Window Procedure
     wc.cbClsExtra    = 0;                // No per-class extra data.
     wc.cbWndExtra    = 0;                // No per-window extra data.
     wc.hInstance     = hInstance;        // Owner of this class
     wc.hIcon         = LoadIcon (hInstance, szAppName);  // Icon name
     wc.hCursor       = LoadCursor(NULL, IDC_ARROW);      // Cursor
     wc.hbrBackground = (HBRUSH)(COLOR_WINDOW+1);     // Default color
     wc.lpszMenuName  = szAppName;         // Menu name from .RC
     wc.lpszClassName = szAppName;         // Name to register as

     // Register the window class and return success/failure code.
     if (!RegisterClass(&wc))
          return (FALSE);     // Exits if unable to register
}
// Save the instance handle in static variable,
hInst = hInstance;
// Create a main window for this application instance.
hWnd = CreateWindow(
     szAppName,              // See RegisterClass() call.
     szTitle,                // Text for window title bar.
     WS_OVERLAPPEDWINDOW,    // Window style.
     CW_USEDEFAULT, 0, CW_USEDEFAULT, 0,     // default positioning
     NULL,                   // Overlapped windows have no parent.
     NULL,                   // Use the window class menu.
     hInstance,              // This instance owns this window.
     NULL                    // We don't use any data in our WM_CREATE
   );

// If window could not be created, return "failure"
if (!hWnd)
     return (FALSE);
// Initialize Windows Socket
WinSockInitialize();

// Make the window visible; update its client area
ShowWindow(hWnd, nCmdShow);      // Show the window
UpdateWindow(hWnd);              // Sends WM_PAINT message

// Acquire and dispatch messages until a WM_QUIT message is
// received.
while (GetMessage(&msg,     // message structure
                  NULL,     // handle of window receiving the message
                  0,        // lowest message to examine
                  0))       // highest message to examine
{
 TranslateMessage(&msg);// Translates virtual key code
 DispatchMessage(&msg); // Dispatches message to window
}
```

```
    // WinSock Cleanup
    WSACleanup();

    return (msg.wParam); // Returns the value from PostQuitMessage
}
//
// Procedure to handle list box
//
BOOL CALLBACK
ListBoxProc(HWND hwndDlg, UINT Msg, LONG  wParam, LONG    lParam)
{
    return FALSE;
}
////////////////////////////////////////////////////////////////////
//
// FUNCTION: WndProc(HWND, UINT, WPARAM, LPARAM)
//
////////////////////////////////////////////////////////////////////

LRESULT CALLBACK
    WndProc(HWND hWnd, UINT Msg, LONG  wParam, LONG    lParam)
{
    // Window-specific variables
    int wmId, wmEvent;
    static HWND hListDialog, hwndList;

    // WinSock handling variables
    int iRet;
    static SOCKET soc = INVALID_SOCKET,
    socacc = INVALID_SOCKET;
    int iBindType = FIXED_PORT_LOCAL_ADDR;
    static BOOL fDataToBeSent = FALSE;

    switch (message)
    {
    case WM_CREATE:
        // Create a dialog box which will display I/O information
        hListDialog = CreateDialog ( hInst,
                                     MAKEINTRESOURCE(IDD_LISTDIALOG),hWnd,
                                     (DLGPROC)ListBoxProc
                                     );
        if (hListDialog == NULL)
        {
            MessageBox ( NULL, "Error Creating List Box", "Error",
                MB_OK);
            return (1);
        }
        else
        {
            // Set title and set the list to empty
            SendMessage ( hListDialog, WM_SETTEXT,
                          0, (LPARAM)(LPSTR)szDialogName);
                          hwndList = GetDlgItem (hListDialog, ID_LIST);
                          SendMessage (hwndList, LB_RESETCONTENT, 0,0L);
        }
        break;
```

```
case WM_COMMAND:      // message: command from application menu
     wmId     = LOWORD(wParam);
     wmEvent      = HIWORD(wParam);
     switch (wmId)
     {
     case IDM_EXIT:
          DestroyWindow (hWnd);
          break;

     case IDM_START_SERVER:
          // Create a socket
          soc = CreateSocket (iBindType, SOCK_STREAM);
          if (soc==INVALID_SOCKET)
               SHOW_ERROR_AND_BREAK ("CreateSocket")
          // Bind local address to the socket
          if (BindSocketToAddr (iBindType,
                                soc,
                                "EchoService",
                                "tcp",
                                 FALSE
                                 ) != 0)
          {
               closesocket ( soc );
               SHOW_ERROR_AND_BREAK ("BindSocketToAddr")
          }

          // Start listening to client requests
          if (listen (soc, 5))
          {
               closesocket ( soc );
               SHOW_ERROR_AND_BREAK ("listen")
          }
          // Instead of calling accept() and block,
          // we register interest in receiving FD_ACCEPT
          // notifications
          iRet=
               WSAAsyncSelect( soc,              // socket
                               hWnd,             // Window
                               WM_ASYNC_SELECT,  // message
                               FD_ACCEPT         // event
                               );
          if (iRet != 0)
          {
               SHOW_ERROR_AND_BREAK ("AsyncSelect")
          }
          else
               DISPLAY_ON_SCREEN("Server Started")
          break;
     case IDM_CLOSE_SERVER:
          // Close the accepted socket
          if (socacc != INVALID_SOCKET)
          {
               // Stop Notifications
               WSAAsyncSelect ( socacc, 0, 0, 0); // stop xfer
               closesocket( socacc );
          }
```

```
            else
                WSAAsyncSelect ( soc, 0, 0, 0); // stop accepting
            // Close the socket
            closesocket(soc);
            DISPLAY_ON_SCREEN("Server Stopped");
            break;
        default:
            return (DefWindowProc(hWnd, message, wParam, lParam));

        } // end of WM_COMMAND
        break;

    case WM_ASYNC_SELECT:
        // First check if there was an error
        if (WSAGETSELECTERROR(lParam) != 0)
        {
            SHOW_ERROR("AsyncSelect Returned",
                WSAGETSELECTERROR(lParam));
            // Abort client connection if necessary
            CheckConnection (hWnd, soc, socacc);
            break;
        }
        // Check the notification type
        switch (WSAGETSELECTEVENT(lParam))
        {
        case FD_ACCEPT:
            // A client request to connect has arrived. We can
            // safely call accept() without blocking
            socacc = accept ( soc, NULL, NULL);
            if (socacc == INVALID_SOCKET)
            {
                SHOW_ERROR_AND_BREAK ("accept")
            }
            // Having connected to the client, now the
            // server wants to receive notification on
            // read/write/close
            iRet = WSAAsyncSelect( socacc, // accepted socket
                                   hWnd,
                                   WM_ASYNC_SELECT,
                                   FD_WRITE| FD_READ| FD_CLOSE
                                 );
            if (iRet == SOCKET_ERROR)
            {
                SHOW_ERROR_AND_BREAK ("AsyncSelect")
            }
            DISPLAY_ON_SCREEN("Client Connected");
            break;
        case FD_READ:
            // Data is available in socket so read it
            iRet = recv(socacc, szMessageBuffer,
                        sizeof(szMessageBuffer), 0);
            if (iRet == SOCKET_ERROR)
            {
                 // Abort client connection if necessary
                CheckConnection (hWnd, soc, socacc);
                break;
```

```
                }
                // Display the message received from client
                DISPLAY_ON_SCREEN(NULL);
                // Fall thru to send the data
                fDataToBeSent = TRUE;
            case FD_WRITE:
                // Socket buffer is free. Hence, we will write
                // into it
                if (fDataToBeSent)
                {
                    sprintf(szSendBuffer, "Server Reply to %s\n",
                            szMessageBuffer);
                    iRet = send ( socacc, szSendBuffer,
                                  sizeof(szSendBuffer), 0);
                    if (iRet == SOCKET_ERROR)
                    {
                        // Abort client connection if necessary
                        CheckConnection (hWnd, soc, socacc);
                        break;
                    }
                    else
                        fDataToBeSent = FALSE;
                }
                break;
            case FD_CLOSE:
                // Client closed the socket. So, we will too!
                GetReadyForNewClient ( hWnd, soc, socacc );
                DISPLAY_ON_SCREEN("Socket Closed");
                break;
            }
            break;
        case WM_DESTROY:  // message: window being destroyed
            PostQuitMessage(0);
            break;

    default:     // Passes it on if unproccessed
            return (DefWindowProc(hWnd, message, wParam, lParam));
    }
    return (0);
}
// Close the accepted socket and call WSAAsyncSelect() again
void GetReadyForNewClient ( HWND hWnd, SOCKET soc, SOCKET socacc)
{
    int iRet;
    // Close accepted socket
    if (socacc != INVALID_SOCKET)
        closesocket ( socacc );
    // Register to get new clients
    iRet = WSAAsyncSelect( soc,              // socket
                           hWnd,             // Window
                           WM_ASYNC_SELECT,  // message
                           FD_ACCEPT         // event
                           );
    if (iRet == SOCKET_ERROR)
    {
        SHOW_ERROR("AsyncSelect", WSAGetLastError());
```

```
    }
}
// Check connection with client. If broken, go back to accept new
// clients by registering to receive FD_ACCEPT events.
void CheckConnection (HWND hWnd, SOCKET soc, SOCKET socacc)
{
    int iRet = WSAGetLastError();
    if ((iRet == WSAECONNRESET) || (iRet == WSAECONNABORTED))
    {
        // Client shutdown or aborted. Shutdown this connection
        // and get ready for new connection
        GetReadyForNewClient ( hWnd, soc, socacc);
    }
    else
        SHOW_ERROR("CheckConnection", iRet);
}
```

Clients can use WSAAsyncSelect() to avoid blocking on socket calls as well as to make their programs event-driven. The client can mark the socket as nonblocking after creating it. Thus, when it calls connect(), it does not block. Further, when the client registers interest in receiving a connection notification, it receives the FD_CONNECT event when a connection to the server is established, even though the connect() function immediately returns with (WSAEWOULDBLOCK). Receiving read/write/close notifications is the same as illustrated for the server: upon receiving a FD_CONNECT message, the client calls the connect() function to connect with the server. On successful completion of connect(), the client registers to receive notifications for FD_READ, FD_WRITE, and FD_CLOSE.

5.3 INTEROPERABILITY WITH IPX/SPX-BASED APPLICATIONS

This section briefly investigates interoperability between WinSock/IPX and Novell NetWare applications (called NetWare/IPX applications) that use IPX/SPX client communication APIs.

WinSock/IPX and NetWare/IPX applications do interoperate. In other words, applications written to Novell NetWare IPX/SPX APIs can communicate with Windows and Windows NT applications written to the Windows Socket APIs.

Windows Sockets does not specify the "on the wire" data format. This allows WinSock/IPX applications to use the IPX/SPX protocols (such as Microsoft NWLink suite) to send and receive data in the IPX/SPX data format and thus communicate with NetWare/IPX applications. This is illustrated in Figure 5-7.

Both WinSock and NetWare IPX have similar APIs for similar functions. Table 5-12 illustrates this [Johnson 94].

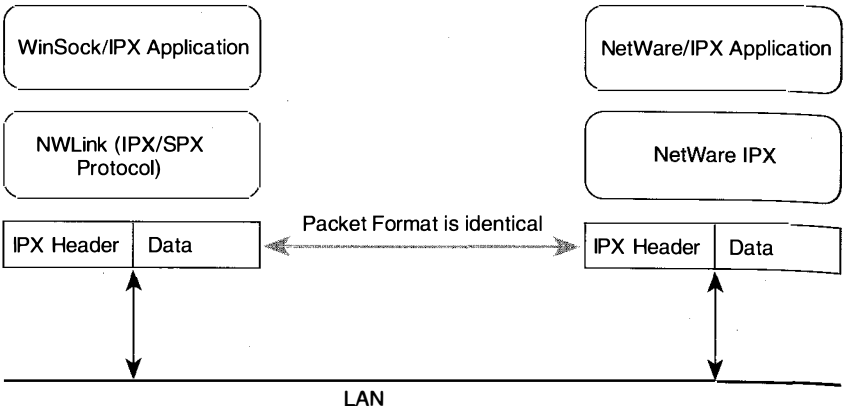


Figure 5-7. *WinSock/IPX and NetWare/IPX Interoperate*

The NWLink protocol allows WinSock/IPX applications to receive the IPX header along with each packet. This is accomplished by calling the setsockopt() API as shown below. This capability allows programmers to use the IPX header information in the same way they use native IPX/SPX programming.

```
// Get IPX header information with each packet received
// Assume that socket has been bound in AF_IPX family
//      and NSPROTO_IPX protocol
// INCLUDE FILE: <wsnwlink.h>
```

Table 5-12. *Novell Communication APIs Compared with Windows Sockets*

Novell Communication API	*Windows Socket API*
Startup and Cleanup	
IPXInitialize()	WSAStartup()
IPXSPXDeinit()	WSACleanup()
Opening and Connecting Sockets	
IPXOpenSocket()	socket(), bind(), connect()
Transmitting Data	
IPXSendPacket()	send(), sendto()
IPXListenPacket()	recv(), recvfrom()
Asynchronous I/O Functions	
Event Service Routines in Windows/MS-DOS	WSAAsyncSelect()
Closing the Socket	
IPXCloseSocket()	closesocket(), shutdown()

```
BOOL  bOn = TRUE;
iRet = setsockopt(soc,                    // socket
                  NSPROTO_IPX,            // level
                  IPX_RECVHDR,            // Option name
                  (const char FAR *)&bOn, // Option value
                  sizeof(bOn);            // Option length
if (iret != SOCKET_ERROR)
     printf("Unable to set option to receive IPX header\n");
```

The NWLink protocol allows WinSock/IPX applications to set the IPX packet type in each outgoing packet. In the IPX/SPX environment, the packet type determines the packet header information as well as the packet data format. For example, NetWare/IPX packets are usually stamped with zero or four, while the SPX packet types are stamped with five. The following code fragment shows how to set the packet type.

```
// Two ways to Set IPX packet type
// Assume that socket has been bound in AF_IPX family
//     and NSPROTO_IPX protocol
// INCLUDE FILE: <wsnwlink.h>

//
// Alternative #1
//
int  pktType = 34;
soc = socket ( AF_IPX, SOCK_DGRAM, NSPROTO_IPX + pktType)
//
// Alternative #2
//
soc = socket ( AF_IPX, SOCK_DRAM, NSPROTO_IPX );
iRet = setsockopt(soc,                          // socket
                  NSPROTO_IPX,                  // level
                  IPX_PTYPE,                    // Option name
                  (const char FAR *)&pktType,   // Option value
                  sizeof(pktType);              // Option length
if (iret != SOCKET_ERROR)
     printf("Unable to set packet type \n");
```

NWLink-supported sequenced packet (SOCK_SEQPACKET) sockets interoperate with NetWare applications using the SPX protocol, which sends and receives messages that can be spread over multiple packets. The sender can send several packets, each of which contains a single message. The receiver waits for all messages to arrive before handing the entire transmission to the receiving application. The receiver knows the transmission has ended when the sender sets the **end-of-message (EOM)** bit on the last message. Because this behavior is not available with TCP/IP stream sockets, NWLink provides the sequenced packet protocol that closely simulates the SPX protocol, which allows WinSock/SPX applications to talk with NetWare/SPX applications.

NWLink allows WinSock/IPX applications to filter special IPX/SPX events, such as file monitoring and directory manipulations, by using IPX_FILTERPTYPE and IPX_STOPFILTERPTYPE socket options.

At present, WinSock/IPX applications cannot use the address resolution techniques used by TCP/IP-based WinSock applications. These include the HOSTS file, WINS server, and the DNS server. NetWare/IPX applications use the **service advertising protocol (SAP)** that are accessed by a set of APIs. These will be integrated with name service provider APIs in the near future, which will allow WinSock/IPX applications to resolve addresses without being forced to use the SAP APIs directly.

5.4 NAME SERVICE PROVIDER APIs

This section presents a brief overview of the **Name Service Provider** APIs (also referred to as **RnR**) that allow WinSock applications to resolve an address regardless of the address family.

Windows Socket APIs were first designed to give applications TCP/IP access in the Windows and Windows NT environments. In the TCP/IP environment, the address of a remote host or a service is programmatically resolved via the database functions such as gethostbyname(). These database functions look at the local file (HOSTS, SERVICES, and the like) or resolve address via a name server (for example, WINS Server or a DNS Server). Thus, in this environment, server lookup is built into the socket programming paradigm.

The simplicity and popularity of WinSock interfaces mean that it is quickly becoming the one common programming interface over multiple protocols and environments. In this chapter, we have demonstrated how WinSock applications can operate over IPX/SPX and NetBios protocols. It is expected that the WinSock interface will also be available over telephony protocols, OSI protocols, and future high-speed **asynchronous transfer mode (ATM)** protocols. While this is a desirable trend, the problem is that each of these environments has a unique address announcement and discovery technique. For example, in the IPX/SPX environment, a server announces its addressing information via the SAP protocol. These announcements are stored in the network routers, allowing the clients to use the SAP discovery protocols to determine the server address given the server's logical name. Thus, Windows Sockets applications using IPX/SPX cannot use the TCP/IP address resolution facilities; they must use the SAP protocol. Worse yet, WinSock applications that use multiple protocols must resort to multiple address resolution techniques. This is where Name Service APIs come into the picture. They promise to provide one common address resolution.

In this scheme, the server (or service) registers its addressing information for each of the address families it supports. For example, a server with

the logical name "EchoServer" could register itself into two name spaces, the TCP/IP name space (address = 11.1.11.128) and the IPX/SPX name space (address = node:12345678 net:1234). The name and address of a server are registered with the name service providers. This is illustrated in Figure 5-8. As you can see, the server, using the name service APIs, registers itself with two providers, IPX/SPX and TCP/IP. Each of these providers announces the presence of the server in their respective name spaces. For example, the IPX provider could send SAP announcements on the server's behalf. These announcements can be cached on the one or more routers.

The client also uses the name service APIs, but it needs only supply the server details to one set of APIs. The name service APIs request each of the name service providers to locate the server. For example, the IPX provider (on client side) sends SAP messages that cause one or more routers to return the server address. The next code shows the prototypes of the two name service APIs. The **GetService()** API is used by the client to locate a server, and the **SetService()** API is used by the server.

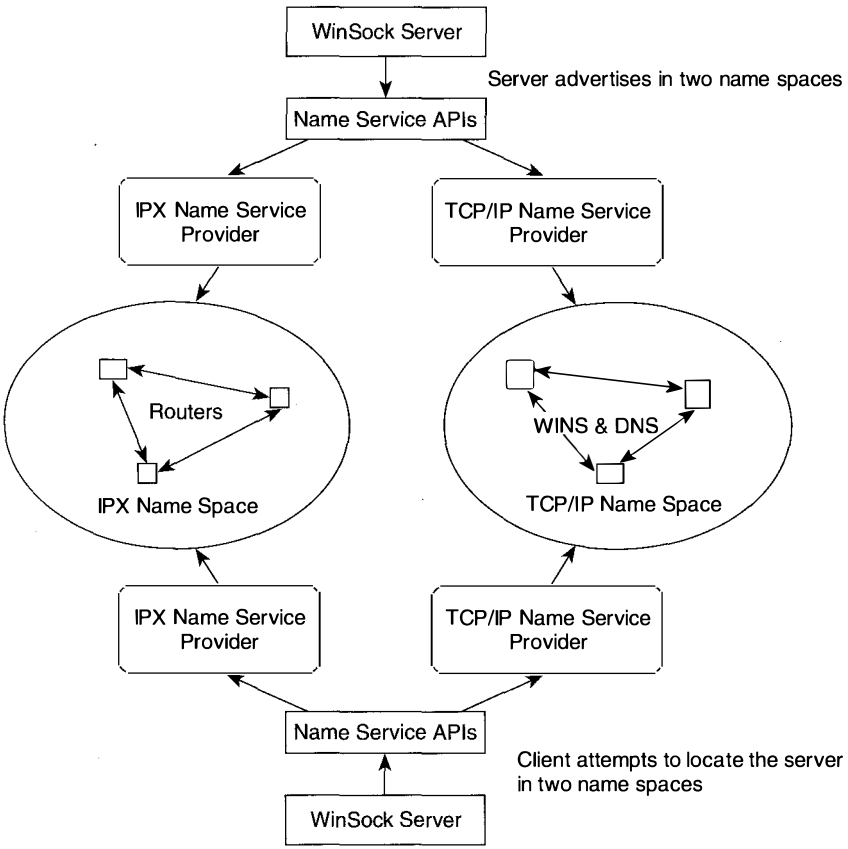


Figure 5-8. *A Schematic View of Name Service Providers*

```
//
// File: \mstools\h\nspapi.h
// source: Windows NT 3.5 SDK
//
typedef VOID (*LPSERVICE_CALLBACK_PROC) (
     IN LPARAM lParam,
     IN HANDLE hAsyncTaskHandle
);
typedef struct _SERVICE_ASYNC_INFO {
     LPSERVICE_CALLBACK_PROC lpServiceCallbackProc;
     LPARAM lParam;
     HANDLE hAsyncTaskHandle;
} SERVICE_ASYNC_INFO, *PSERVICE_ASYNC_INFO, FAR * LPSERVICE_ASYNC_INFO;
INT APIENTRY  GetService (
     IN     DWORD                    dwNameSpace,
     IN     LPGUID                   lpGuid,
     IN     LPWSTR                   lpServiceName,
     IN     DWORD                    dwProperties,
     IN OUT LPVOID                   lpBuffer,
     IN OUT LPDWORD                  lpdwBufferSize,
     IN     LPSERVICE_ASYNC_INFO     lpServiceAsyncInfo
     );
INT APIENTRY
SetServiceW (
     IN     DWORD                    dwNameSpace,
     IN     DWORD                    dwOperation,
     IN     DWORD                    dwFlags,
     IN     LPSERVICE_INFOW          lpServiceInfo,
     IN     LPSERVICE_ASYNC_INFO     lpServiceAsyncInfo,
     IN OUT LPDWORD                  lpdwStatusFlags
     );
//
// Some common name spaces
//
#define NS_DEFAULT     (0)
#define NS_SAP         (1)      // SAP protocol
#define NS_NDS         (2)      // Netware Directory Service
#define NS_TCPIP_LOCAL (10)     // Local TCP/IP
#define NS_TCPIP_HOSTS (11)     // Lookup via HOST file
#define NS_DNS         (12)     // Use DNS Service
#define NS_NETBT       (13)     // Use NetBIOS name server
```


Because the name service APIs are still evolving, they are not covered in great detail. (Inspect the sample program (\samples\win32\rnr) supplied with the Windows NT 3.5 SDK for more information.)

5.5 A WINDOWS-SOCKET-BASED WIN32 SERVICE

This section illustrates how WinSock can be used by a Win32 service.

Recall that the Echo Server Win32 service was introduced in Chapter 2. Chapter 4 showed the Echo Server using RPC to communicate with its clients. This section shows the Echo Server using sockets by expanding the skeletal service code shown in Chapter 2. The code for the Echo Server follows.

Notice that the Echo Server has two parts, the code fragments necessary for the Echo Server to function as a Win32 service, and the code fragments used to create a WinSock server that will accept calls from the socket-based clients. The first part was explained in Chapter 2; the second part of the code fragment appears here.

When the service is started, the InitEchoServer() is executed. Here, the internal function **CreateLocalSocketEndPoint()** is called to create the server-side socket. The function creates the server-side socket by calling the socket() API, and then calls the bind() API to bind the local host address with the socket. Note that database functions are used to determine the local host address and port number associated with the Echo Server service. On successful creation of the socket, the listener thread is started and immediately calls the listen() API to make the socket ready to receive client connect requests. To avoid blocking on the accept() API call, the service uses the select() API in the internal function **WillReadBlock()**. When WillReadBlock() returns FALSE, the server can safely call the accept() API and establish a connection with the client.

When a client calls the connect() API, the listen thread in the service can successfully complete the call to accept(). At this point, the listen thread will put client details (the accepted socket descriptor and the client IP address) in the processing queue and continue to wait for new client requests. The client request is processed by one of the worker threads on the server. After the new client connect request is put in the processing queue, the next available worker thread picks up the request. It reads the client request by calling the recv() function, and sends an echo by calling the send() function. Notice that the ioctlsocket() API is used with the FIONREAD command to determine the size of the client request (in bytes). This allows the worker thread to call recv() with the exact buffer size, and thus not block. The worker thread can also avoid blocking on the send() API by calling the WillWriteBlock() internal function; the WillWriteBlock() function uses the select() API to determine the write status of the socket.

The code is shown for the client (EchoC.c) that calls the WinSock-based Win32 service. It calls the connect() API to connect with the Win32 service. Then it uses the send() and recv() functions to send/receive data to/from the service. It also avoids blocking by calling the WinReadBlock() and WillWriteBlock() functions.

```
//////////////////////////////////////////////////////////////
// NETFUNC.C - WinSock Service utility functions
// (see %BOOK%\echosrv\winsock\echos.mak for full details)
//////////////////////////////////////////////////////////////
#include "netfunc.h"
#Pragma hdrstop

#define SERVER_BACK_LOG 5
```

```
#define ECHO_SERVER     "EchoServer"
#define ECHO_PROTOCOL   "tcp"

//
// Initialize WinSock library
//
int WinSockInitialize()
{
     int       iRc;
     WORD      wVersionRequired;
     WSADATA   wsaData;
     wVersionRequired = MAKEWORD(1,1);
     iRc = WSAStartup(wVersionRequired, &wsaData);
     if (iRc != 0)
          PrintError1("WSAStartup() Error: %d\n", iRc);
     return (iRc);
}
//
// CreateLocalSocketEndPoint() -
// Create a socket and associate the local address
// with the socket
//
int CreateLocalSocketEndPoint ( SOCKET *pSoc, int af, BOOL fServer)
{
     int iRc = 0;
     SOCKET soc;
     struct hostent FAR *pHostAddr;
     struct servent FAR *pServInfo;
     struct sockaddr_in LocalAddr;

     // First create a stream socket over any protocol
     soc = socket ( af, SOCK_STREAM, 0);
     if ( soc != INVALID_SOCKET)
     {
          // Binding information is setup differently, depending
          // on type of caller (Client vs. Server)
          if (fServer)
          {
               // Find ip address corresponding to "EchoServer"
               pHostAddr = gethostbyname ( ECHO_SERVER);
               if (pHostAddr != NULL)
               {
                    // Find the port number
                    pServInfo = getservbyname (ECHO_SERVER, ECHO_PROTOCOL);
                    if (pServInfo != NULL)
                    {
                         // address family
                         LocalAddr.sin_family = af;
                         // Host Address
                         LocalAddr.sin_addr.s_addr =
                              *(u_long *)pHostAddr->h_addr;
                         // Port
                         LocalAddr.sin_port = pServInfo->s_port;
                    }
                    else
                    {
```

```
                    PrintError("getservbyname() error. Check SERVICES
                        file\n");
                    iRc = -1;
                }
            }
            else
            {
                PrintError("getservbyname() error. Check HOSTS file\n");
                    iRc = -1;
            }
        }
        else     // Client
        {
            // Bind to the any port at any local address
            LocalAddr.sin_family = af;
            LocalAddr.sin_addr.s_addr = INADDR_ANY; // Any local address
            LocalAddr.sin_port = 0; // System allocates a port
        }
        // If no error occured, bind the socket to the local address
        if (iRc == 0)
        {
            iRc = bind ( soc, (LPSOCKADDR)&LocalAddr,
                        sizeof(LocalAddr));
            if (iRc == SOCKET_ERROR)
            {
                iRc = WSAGetLastError();
                PrintError1("socket bind error: %d\n", iRc);
            }
            else
            {
                // Return the socket number to caller.
                *pSoc = soc;
            }
        }
    }
    else
    {
        iRc = WSAGetLastError();
        PrintError1("socket create error: %d\n", iRc);
    }
    return(iRc);
}
//
// ConnectToServer() -
//     Connects the client with the EchoServer (local or remote)
//
//
int ConnectToServer ( SOCKET soc, int af)
{
    int iRc = 0;
    struct hostent FAR *pHostAddr;
    struct servent FAR *pServInfo;
    struct sockaddr_in ServAddr;

    // Find ip address corresponding to "EchoServer"
    pHostAddr = gethostbyname ( ECHO_SERVER);
```

```
        if (pHostAddr != NULL)
        {
             // Find the port number
             pServInfo = getservbyname (ECHO_SERVER, ECHO_PROTOCOL);
             if (pServInfo != NULL)
             {
                  // address family
                  ServAddr.sin_family = af;
                  // Host Address
                  ServAddr.sin_addr.s_addr = *(u_long *)pHostAddr->h_addr;
                  // Port
                  ServAddr.sin_port = pServInfo->s_port;
             }
             else
             {
                  PrintError("getservbyname() error. Check SERVICES file\n");
                      iRc = -1;
             }
        }
        else
        {
                  PrintError("getservbyname() error. Check HOSTS file\n");
                      iRc = -1;
        }
        // If no error occurred, bind the socket to the local address
        if (iRc == 0)
        {
             iRc = connect ( soc, (LPSOCKADDR)&ServAddr, sizeof(ServAddr));
             if (iRc == SOCKET_ERROR)
             {
                  iRc = WSAGetLastError();
                  PrintError1("socket connect error: %d\n", iRc);
             }
        }
        return(iRc);
}
//
// ListenForClient()
//     Called by the server to setup a queue and start listening
//
int ListenForClient ( SOCKET soc, BOOL fFirstTime, PSOC_DATA pSocData)
{
     int iRc = 0;
     SOCKADDR_IN ClientAddr;
     int len;
     char *pszClientAddr;

     // If this is the first time, setup a listen queue
     if (fFirstTime)
     {
          iRc = listen ( soc, SERVER_BACK_LOG);
               if (iRc == SOCKET_ERROR)
               {
                    iRc = WSAGetLastError();
                    PrintError1("socket listen error: %d\n", iRc);
               }
```

```
    }
    if (iRc != 0)
        return (iRc);

    // First check if there is any client request pending
    if ( !WillReadBlock( soc ))
    {
        len = sizeof(ClientAddr );
        pSocData->soc = accept ( soc, (LPSOCKADDR)&ClientAddr, &len);
            if (pSocData->soc == INVALID_SOCKET)
            {
                iRc = WSAGetLastError();
                PrintErrorl("socket accept error: %d\n", iRc);
            }
            else
            {
                // Translate client address from network byte order
                // to ASCII
                pszClientAddr = inet_ntoa ( ClientAddr.sin_addr);
                sprintf ( pSocData->chRemoteName, "%s",
                          pszClientAddr);
            }
    }
    else
    {
        // A call to accept() would block
        iRc = WSAEWOULDBLOCK;
    }
    return(iRc);
}
//
// WillReadBlock() -
//     Checks if a call to read data from socket will block
//
//
BOOL WillReadBlock ( SOCKET soc )
{
    int iRc = 0;
    fd_set     ReadSet;
    struct     timeval Timeout;
    BOOL       fRet = TRUE;

    // immediately return from select()
    Timeout.tv_sec = 0;
    Timeout.tv_usec = 0;
    // set out socket descriptor 'bit' in the read mask
    FD_ZERO ( (fd_set FAR *)&ReadSet );
    FD_SET  ( soc, (fd_set FAR *)&ReadSet);

    // Check if "read" will block
    iRc = select ( FD_SETSIZE, &ReadSet, NULL, NULL,
                   &Timeout);
    if (iRc == SOCKET_ERROR)
    {
        iRc = WSAGetLastError();
        PrintErrorl("socket select error: %d\n", iRc);
    }
```

```
    else
    {
        if (FD_ISSET ( soc, (fd_set FAR *)&ReadSet))
        {
          fRet = FALSE; // Read operation will succeed without blocking
        }
    }
    return(fRet);
}
//
// WillWriteBlock() -
//     Checks if a call to write data into socket will block
//
//
BOOL WillWriteBlock ( SOCKET soc )
{
    int iRc = 0;
    fd_set  WriteSet;
    struct  timeval Timeout;
    BOOL    fRet = TRUE;

    // immediately return from select()
    Timeout.tv_sec = 0;
    Timeout.tv_usec = 0;
    // set out socket descriptor 'bit' in the write mask
    FD_ZERO ( (fd_set FAR *)&WriteSet );
    FD_SET  ( soc, (fd_set FAR *)&WriteSet);

    // Check if "write" will block
    iRc = select ( FD_SETSIZE, NULL, &WriteSet, NULL,
                   &Timeout);
    if (iRc == SOCKET_ERROR)
    {
        iRc = WSAGetLastError();
            PrintError1("socket select error: %d\n", iRc);
    }
    else
    {
        if (FD_ISSET ( soc, (fd_set FAR *)&WriteSet))
        {
          fRet = FALSE;// Write operation will succeed without blocking
        }
    }
    return(fRet);
}

//
// SendData ()
// - Sends data stream over stream socket
//
int SendData ( SOCKET soc, char far * pchData, unsigned short usDataLen)
{
    int iRc = 0;
    // First check if a write would block
    if (!WillWriteBlock(soc))
    {
```

```
        // No blocking. Let us send it.
        iRc = send ( soc, pchData, usDataLen, 0);
        if ( iRc == SOCKET_ERROR)
        {
             iRc = WSAGetLastError();
             PrintError1("socket send error: %d\n", iRc);
        }
        else
             iRc = 0; // contains the number of bytes sent!
    }
    else     // Write will block
    {
        iRc = WSAEWOULDBLOCK;
    }
    return(iRc);
}
//
// ReceiveData ()
//     - Receives data stream from stream socket
//
int ReceiveData ( SOCKET soc, char far * pchData,
                  unsigned long *plDataLen)
{
    int iRc = 0;
    u_long lBytesInSocket = 0;

    // First check if a read would block
    if (!WillReadBlock(soc))
    {
        // No blocking. Let us find out how many bytes we can read
        iRc = ioctlsocket ( soc, FIONREAD, &lBytesInSocket );

        if ((iRc != SOCKET_ERROR) && (lBytesInSocket))
        {
             iRc = recv ( soc, pchData, lBytesInSocket, 0);
             if (iRc != SOCKET_ERROR)
                  {
                       *plDataLen = lBytesInSocket;
                  iRc = 0; // iRc contains the number of bytes received!
             }
        }
        if ( iRc == SOCKET_ERROR)
        {
             iRc = WSAGetLastError();
             PrintError1("socket recv/ioctl error: %d\n", iRc);
        }
        else if ( lBytesInSocket == 0)
        {
             // The other side closed or reset the socket
             iRc = WSAEDISCON;
        }
    }
    else     // Read will block
    {
             iRc = WSAEWOULDBLOCK;
    }
```

```
    return(iRc);
}
//
// CloseSocket()
//     - Closes a socket which has been earlier opened
//
int CloseSocket ( SOCKET soc )
{
    int iRc = 0;
    iRc = closesocket ( soc );
    if (iRc == SOCKET_ERROR)
    {
            iRc = WSAGetLastError();
            PrintError1("socket close error: %d\n", iRc);
    }
    return(iRc);
}
////////////////////////////////////////////////////////////////
// ECHOC.C - WinSock Service Server Program
// (see %BOOK%\echosrv\winsock\echos.mak for full details)
////////////////////////////////////////////////////////////////
#include "netfunc.h"
#pragma hdrstop

// Global Data
#define ECHO_SERVER_A "EchoServer"
SOCKET     soc;
BOOL fStopService = FALSE;
#define MAX_INSTANCES (5)
// Create a pool of SOC_DATA shared between Listen thread
// and worker threads
INITIALIZE_POOL(MAX_INSTANCES, sizeof(SOC_DATA))

// Internal functions
void  WorkerThread(VOID *pF);
void  ListenThread(VOID * pF);
void  ShutDown(void);
DWORD InitializeSocket();

//
// InitEchoServer - Called when the service is started
//
BOOL InitEchoServer()
{
    DWORD dwRc;
    BOOL  bRet = TRUE;

    // Initialize SOC_DATA Resource Pool
    InitializePool();

    // First initialize the Client Request Pool
    dwRc = InitializeQueue();
    if (dwRc == 0)
    {
        // Get the WinSock Service Started
        dwRc =  InitializeSocket( );
```

```
          // Finally create the worker thread pool
          dwRc = InitializeThreads ( WorkerThread);
          if (dwRc)
               bRet = FALSE;
     }
     return(bRet);
}
DWORD InitializeSocket()
{
     DWORD dwRc;
     // First Reset
     dwRc = WinSockInitialize();
     if (dwRc == 0)
     {
          // Create a socket, bind to local address
          dwRc  = CreateLocalSocketEndPoint ( &soc, AF_INET, TRUE);
          if (dwRc ==0)
          {
               // Start out "Listen Thread"
               _beginthread ( ListenThread,
               4098,
               0
               );
          }
     }
     return(dwRc);
}
//
// In this thread, the server listens to incoming client
// request. Once the request arrives, this thread will
// put the incoming request in the request pool for
// worker threads which, in turn, will service the client
//

// Define Listen state transitions
typedef enum
{
      enListen = 0,
      enWaitListen,
      enAccepted
} enListenState;

void ListenThread(VOID * pF)
{
     DWORD     dwRc     = 0,
               dwEventIndex;
     enListenState ListenState;
     SOC_DATA  SocData,     *pSocData;
     BOOL      fFirstTime = TRUE;

     // Create an event to synchronize with Service Thread
     dwEventIndex = GetThreadSyncEvent ();
     if (dwEventIndex == -1)
     {
          PrintError("Could not create Thread Sync. Event\n");
          return;
```

```
}
//
// Loop forever listening to incoming clients
//
ListenState = enListen;
do
{
     // First check if Service needs to be stopped
     if (fStopService)
     {
          SetThreadSyncEvent (dwEventIndex); // We can be closed
          dwRc = 0; // break out
          break;
     }
     //
     // State driven Asynchronous Listen
     //
     switch(ListenState)
     {
     case enListen: // Post a Listen
          dwRc = ListenForClient ( soc, fFirstTime, &SocData);
          if (fFirstTime)
               fFirstTime = FALSE;
          if (dwRc == 0)
          {
               ListenState = enAccepted; // Client connected!
                         dwRc = WAIT_TIMEOUT;
          }
          else if (dwRc==WSAEWOULDBLOCK)
          {
               ListenState = enWaitListen; // wait for a while
                         dwRc = WAIT_TIMEOUT;
          }
          else
          {
               PrintErrorl("Listen Failed: %lx\n", dwRc);
          }
          break;
     case enWaitListen: // Waiting for a client
          SleepEx ( 300, TRUE ); // Alertable wait
          ListenState = enListen;
          dwRc = WAIT_TIMEOUT;
          break;

     case enAccepted:
          // A client has come in. Put his request
          // in the Request Pool for the next free worker
          if (GetPoolResource (&pSocData))
          {
                    pSocData->soc = SocData.soc;
                    memcpy(pSocData->chRemoteName,
                         SocData.chRemoteName,
                         strlen(SocData.chRemoteName)+1
                        );
                    // put this in the worker pool
                    if (dwRc = QueueRequest ( pSocData,
```

```
                    sizeof(SOC_DATA))!=0)
                {
                    // Free the resource and break the session
                    PrintErrorl("Canceling Session", dwRc);
                    FreePoolResource ( pSocData );
                    CloseSocket ( SocData.soc );
                }
            }
            // Now post a new Listen
            ListenState = enListen;
            dwRc = WAIT_TIMEOUT;
        }
    } while (dwRc == WAIT_TIMEOUT);

    // Close Thread sync. event
    CloseThreadSyncEvent ( dwEventIndex );
    _endthread();
}
//
// The worker threads wait to get requests from the
// listen thread.
//
// Define Worker Thread state transitions
typedef enum
{
    enWaitOnConnect = 0,      // Wait for client to connect
    enReceiveRequest,         // Receive client request (async)
    enSendReply,              // Send a response (async)
    enHangupSession           // End session with client
} enWorkerState;

void WorkerThread(VOID *pF)
{
    DWORD dwRc;
    ThreadData *pTData;

    enWorkerState WorkerState = enWaitOnConnect;
    PSOC_DATA     pSoc = NULL;
    DWORD         dwDataSize;
    char          chReceiveBuffer    [ MAX_TALK_BUFFER_LENGTH ],
                  chSendBuffer       [ MAX_TALK_BUFFER_LENGTH ];

    // Set the thread status
    pTData = (ThreadData *)pF;
    if (pTData == NULL)
    {
        PrintError("Null Thread Data\n");
        return;
    }
    do
    {
        // First check if Service needs to be stopped
        if (fStopService)
        {   dwRc = 0;
            WorkerState = enHangupSession;
        }
```

```
        switch (WorkerState)
        {
        case enWaitOnConnect:
             // Wait for Listen Thread to put data on queue
             dwRc = GetNextRequest ( &pSoc, &dwDataSize);
             if (dwRc ==0)
             {     // Verify the data integrity
                if ((pSoc == NULL) || (dwDataSize < sizeof(SOC_DATA)))
                {
                   dwRc = 0;
                   PrintError("Data Corruption\n");
                }
                else
                { // We have successfully gotten a new client request
                     WorkerState = enReceiveRequest;
                     dwRc = WAIT_TIMEOUT;
                }
             }
             else if (dwRc != WAIT_TIMEOUT)
                 PrintErrorl("GetNextRequest Error: %lx\n", dwRc);
        break;
        case enReceiveRequest:
             dwDataSize = MAX_TALK_BUFFER_LENGTH;
             dwRc = ReceiveData (
                                  pSoc->soc, // socket
                                  chReceiveBuffer,
                                  &dwDataSize
                                 );
             if (dwRc == 0)
             {
                 char szMsg [ 2 * MAX_TALK_BUFFER_LENGTH ];
                 sprintf(szMsg, "Client: [%s] Sent: [%s]\n",
                           pSoc->chRemoteName, chReceiveBuffer);
                 PrintInfoString("Client Msg: %s", szMsg);
                 dwRc = WAIT_TIMEOUT;
                 WorkerState = enSendReply;
             }
             else if (dwRc == WSAEWOULDBLOCK)
             {
                 dwRc = WAIT_TIMEOUT;     // just wait a while
                 SleepEx (300, TRUE);     // alertable wait
             }
             else
             {
                 PrintErrorl("Receive Error: %lx\n", dwRc);
                 dwRc = 0;
             }
             break;
    case enSendReply:
        sprintf(chSendBuffer, "Server Thread: %ld Reply: [%s]\n",
                 pTData->hThread, chReceiveBuffer
                );
        dwRc = SendData (
                          pSoc->soc,     // socket
                          chSendBuffer,
                          strlen(chSendBuffer) + 1
                         );
```

```
        if (dwRc == 0)
            {
                        // Sent the request,
                        // now disconnect from client
                        dwRc = WAIT_TIMEOUT;
                        WorkerState = enHangupSession;
                }
                else if (dwRc == WSAEWOULDBLOCK)
                {
                        dwRc = WAIT_TIMEOUT;
                        SleepEx ( 300, TRUE );
                }
                else
                {
                        PrintError1("Error Sending Reply: %lx\n", dwRc);
                        dwRc = 0;
            }
        break;
        case enHangupSession: // Disconnect with client
            if (pSoc)
        {
                // Disconnect from the client
                dwRc = CloseSocket ( pSoc->soc);
                if (dwRc != 0)
                     PrintError1("Error Closing socket: %lx\n",
                                pSoc->soc);
                // Release the buffer
                FreePoolResource ( pSoc );
                pSoc = NULL;

                // Get ready for next client
                WorkerState = enWaitOnConnect;
                dwRc = WAIT_TIMEOUT;
            }
            if (fStopService)
            {
                 // Notify the service thread that we can be closed
                 SetThreadSyncEvent (pTData->dwEventIndex);
                 dwRc = 0;
            }
            break;
        } // end of switch
    } while (dwRc == WAIT_TIMEOUT);

    // Close Thread sync. event
    CloseThreadSyncEvent ( pTData->dwEventIndex );
    _endthread();
}
// called when service is paused via Service Control Applet
BOOL PauseEchoServer()
{

    printf("Pausing the WinSock Service is not supported\n");
    return(TRUE);
}
// called when service is resumed via Service Control Applet
BOOL ResumeEchoServer()
```

```
{
     printf("Resuming the WinSock Service is not supported\n");
     return(TRUE);
}
// called when service is stopped that is net stop echoserv
BOOL StopEchoServer()
{
     DWORD dwRc;
     // Signal all threads to quit
     fStopService = TRUE;
     WaitForAllThreadsToTerminate();

     // Close the socket
     dwRc = CloseSocket ( soc );
     if (dwRc != 0)
     {
          PrintError1("Error Closing Socket: %lx\n", dwRc);
     }
     // Delink from WinSock DLL
     WSACleanup();
     return(TRUE);
}
////////////////////////////////////////////////////////////
// ECHOC.C - WinSock Service Client Program
// (see %BOOK%\echosrv\winsock\echoc.mak for full details)
////////////////////////////////////////////////////////////
#include "netfunc.h"
#pragma hdrstop
#define ECHO_CLIENT_A "EchoClient"

// Global Data
static SOCKET soc;
static char chClientName [ 20 ];
int ContactServer ();

int main (int argc, char ** argv)
{
     DWORD dwRc;
     static BYTE bClientNo;

     // determine client number
     argc--;
     argv++;
     if (argc)
          bClientNo = (BYTE) atoi(argv[0]);
     else
          bClientNo  = 0;
     // Set client name
     sprintf(chClientName, "%s%d", ECHO_CLIENT_A, bClientNo);
     printf("Socket Echo Client: [%s] starting\n", chClientName);

     // First get WinSock DLL ready
     dwRc = WinSockInitialize();
     if (dwRc == 0)
     {
          // Create client side of socket
```

```
        dwRc = CreateLocalSocketEndPoint ( &soc, AF_INET, FALSE);
        if (dwRc != 0)
        {
             printf("Error creating socket : %ld\n", dwRc);
             return (dwRc);
        }

        // Contact a server
        dwRc = ContactServer ( );

        // Disenage from WinSock DLL
        WSACleanup();
    }
    return(dwRc);
}
// Function: ContactServer
//     It calls the Server, sends a message and receives
//     a message.
//
typedef enum // Declare Client states
{
        enDataToSend = 0,
        enDataToReceive
}ClientState;

int ContactServer ()
{
    DWORD dwRc;
    DWORD dwBufSize;
    char  chReceiveBuffer     [ MAX_TALK_BUFFER_LENGTH ],
          chSendBuffer        [ MAX_TALK_BUFFER_LENGTH ];
    ClientState enClientState = enDataToSend;

    printf("Calling Echo Server\n");
    // Connect is synchronous operation in this example
    dwRc = ConnectToServer ( soc, AF_INET );
    if (dwRc)
    {
        printf("ConnectToServer Failed: %lx\n", dwRc);
        return(dwRc);
    }

    // Successfully connected!
    printf("Connected! to the server. socket no: %d\n", soc);

    // Send a message to the server
    sprintf(chSendBuffer, "Message From: %s\n", chClientName);
    do
    {
        switch (enClientState)
        {
        case enDataToSend:
             dwRc = SendData ( soc,                  // socket
                               chSendBuffer,         // data to be sent
                               strlen(chSendBuffer)+1 // sizeof data
                             );
```

```
                if (dwRc != 0)
                {
                    printf("Error sending data to the Server: %ld\n", dwRc);
                     break;
                }
                else
                     printf("Data sent to Echo Server\n");
                // Now proceed to receive data
                enClientState = enDataToReceive;
                // Simply fall thru!
           case enDataToReceive:
                // Receive Reply
                dwRc = ReceiveData (
                                    soc,   // Socket
                                    chReceiveBuffer,
                                    &dwBufSize
                                   );
                if (dwRc == 0)
                {
                     if (dwBufSize != 0)
                     printf("Data Received From Server\n[%s]\n",
                            chReceiveBuffer);
                     else
                     printf("Server disconnected\n");
                     // We are done. Get out
                     break;
                }
                else
                     printf("Error receiving data from the Server: %ld\n",
                            dwRc);
                break;
           };

           // Wait a bit
           if (dwRc == WSAEWOULDBLOCK)
           {
                Sleep ( 300 );
           }
      } while (dwRc == WSAEWOULDBLOCK );

      // Close the socket
      dwRc = CloseSocket ( soc );
      if (dwRc != 0)
      {
           printf("Error terminating socket: %ld\n", dwRc);
      }
      return (dwRc);
}
```

5.6 GUIDELINES FOR 16-BIT WINDOWS SOCKETS APPLICATIONS

This chapter has mentioned programming issues that affect all 16-bit Windows Socket applications. This section summarizes these guidelines.

Unless specifically noted, all Windows Sockets interfaces discussed in this chapter are available and functional in the Windows 3.x environment. Microsoft (as well as other vendors) has provided 16-bit implementations of the Windows Sockets libraries, DLLs, and drivers. In particular, Microsoft provides multiple WinSock-compatible protocols (such as TCP/IP, IPX/SPX, NetBEUI) and name resolution using WINS on the Windows for Workgroups platform.

- Unless necessary, Windows applications should avoid blocking socket calls such as connect(), send() or recv(). Windows Socket asynchronous interfaces, such as WSAAsyncSelect(), should be used as much as possible. Applications can also make a socket non-blocking by using the ioctlsocket() API, or they can determine the status of the socket by using the select() API.
- When blocking calls are made on a socket, the Windows Socket implementation provides the default Windows message-handling function. This allows the system to function without hanging. Application designers should consider providing their own message-handling function with the WSASetBlockingHook() function. This will allow them to present a responsive GUI to the user. This can enable them to provide a dialog box to cancel a (potentially) long network call.
- Applications should lock memory buffers and structures associated with an asynchronous socket I/O call, such as the WSAAsyncGetHostByName() API.
- While Windows Socket-based servers can be implemented to function on a Windows 3.x workstation, mission-critical client/server systems should have their servers hosted on a robust operating system, such as Windows NT, OS/2, or UNIX.

5.7 SUMMARY

This chapter has presented the concepts and the use of the Windows Socket interfaces to build the communication infrastructure of Windows and Windows NT client/server systems. It illustrated, using sample programs and diagrams, the essentials of socket programming, including creating and opening local or remote sockets, sending and receiving data by a socket, and Win32 services that use sockets. The Windows Socket interface supports both datagram-based communication and stream-based communication. It allows applications to transmit data over a number of popular protocols, such as TCP/IP, IPX/SPX, AppleTalk, and NetBIOS. In the near future, applications will be able to use the Windows Socket interfaces to communicate over emerging mediums such as ISDN and ATM.

Chapter Six

PIPES IN WINDOWS NT

A pipe is an application-level programming construct or interface that can be used to build an IPC channel between a client and a server. This chapter presents the concept of pipes and illustrates how pipes are used in Windows and Windows NT. It begins with an in-depth presentation on the Named Pipe architecture in Windows NT, then demonstrates how pipes can be used across heterogeneous environments (such as Windows NT and Novell NetWare). It presents aspects, from simple to the most sophisticated, of programming pipes in Windows and Windows NT. The end of the chapter demonstrates a Named Pipe-based Win32 service (EchoServer) that was first introduced in Chapter 2. It closes with a comparison of Named Pipes with UNIX pipes.

Pipes were developed in the UNIX environment to allow a child process to communicate with its parent process. Usually, a parent process redirected its output to a pipe instead of the **standard output device (stdout)**, and a child process redirected its standard device input **(stdin)** to come from the pipe, thereby reading the instruction sent by the parent. In most operating systems, the pipe character (" | ") is a manifestation of the pipe programming interfaces. In the following illustration, the output of the "type" command is being piped into the "more" command.

```
C:> type large.txt | more
```

In Windows NT, an application writer has two choices for using pipes for IPC. Named Pipes, as the name implies, are named, and can be used for inter-machine communication. Another type of pipe is an anonymous pipe, which can only be used on the same workstation. This chapter discusses both types of pipes, their use, and their advantages and disadvantages, as compared with other IPC mechanisms.

6.1 NAMED PIPES IN WINDOWS NT

Named Pipe is an application programming interface that can be used for bi-directional IPC. The Named Pipe interface lies above, and is independent from the transport protocols; maintaining sessions between communicating

processes is left to session layer interfaces, such as NetBIOS or TCP/IP. Named Pipes exist at the presentation layer in OSI. However, the presentation layer in OSI is responsible for data massaging, such as data compression, encryption, and conversion of data from one format to another (for example, big endian to little endian). Named Pipe implementations do not perform the duties of the presentation layer; RPC performs the presentation layer work. In fact, RPC considers Named Pipe as another IPC mechanism, like IPX/SPX or NetBIOS.

Named Pipes are used to transmit data between two distinct types of entities, servers and clients. The main difference is that only a server can "create" a pipe. Once a pipe has been created by the server, one or more clients can open it. After the pipe has been opened, both the server and the client can call the normal read/write functions to read data from, and to write data into, the pipe. Figure 6-1 shows a model of a Named Pipe client and server transferring data. In addition to some special calls to create and manipulate the Named Pipe, the applications can read from and write to the Named Pipe as if it were a file.

The Named Pipe concept is similar to the WinSock concepts introduced in Chapter 5. The major differences between Named Pipe and WinSock are that Named Pipe implementations allow the Named Pipe-based server to exist only on designated platforms (such as Windows NT or OS/2), but a socket-based server can exist on every platform that supports sockets.

A Named Pipe is always named according to the UNC. For pipes, the form is \\<computer name>**PIPE**\<pipe name>. The <computer name> field can be the name of the remote computer such as "\\DATA_SERVER". When referring to a local pipe, the client supplies "." in place of the local computer name. PIPE is a keyword distinguishing Named Pipe as a distinct Windows NT object. The <pipe name> field can be any name, such as "MyPipe" or "James\Bond.007."

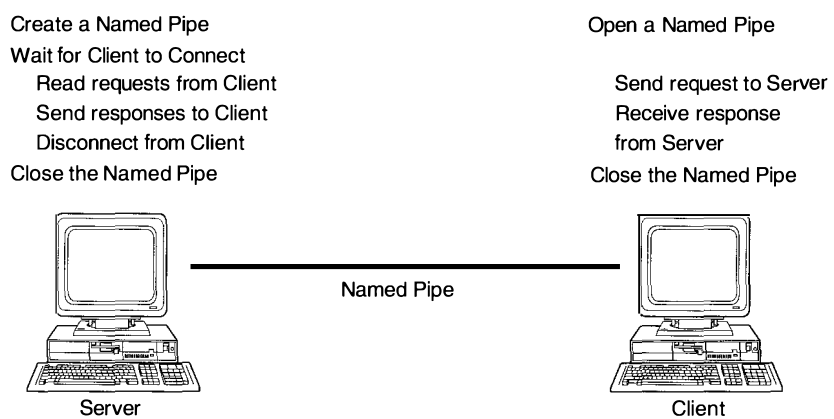


Figure 6-1. *Named Pipe Client and Server*

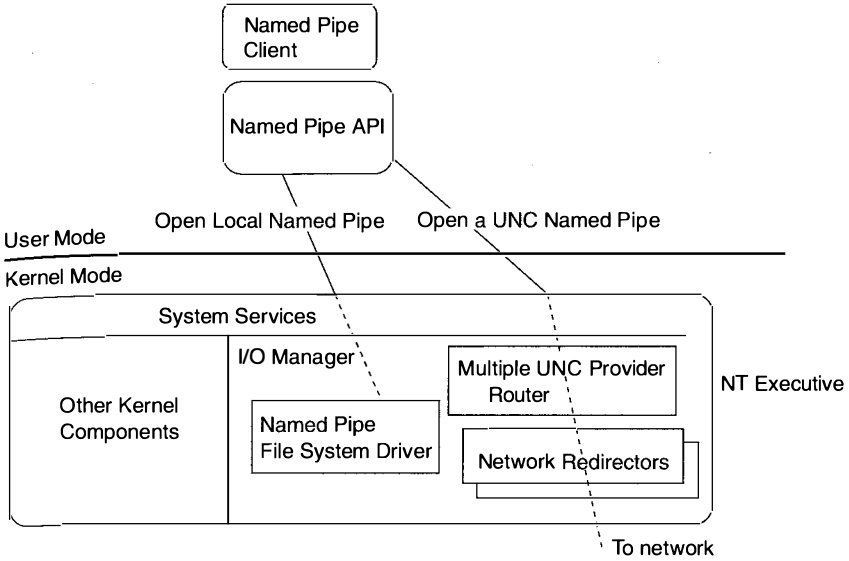


Figure 6-2. *Named Pipe File System Driver*

When an application opens a pipe (for example, "\\.\PIPE\MyPipe"), it is serviced by the local Named Pipe file system device shown in Figure 6-2. When a remote pipe is opened, the MUP driver redirects this request, using a local redirector, to the Named Pipe file system driver running on a remote machine.[1] The Named Pipe file system driver is a pseudo file system driver that maintains the data buffers for the Named Pipes and keeps track of the current state and the owner(s) of the Named Pipes. When a server application writes to a local pipe, the data is stored in the internal buffers of the Named Pipe driver. When a client application reads the pipe, it is actually reading the data off the internal buffers of the Named Pipe driver.

6.1.1 Platforms Supported

In addition to Windows NT, Named Pipe is available for various other platforms. Named Pipe is a powerful IPC mechanism that frees the application writer from low-level details, such as transport protocol (TCP/IP, NetBEUI, IPX/SPX, and so on), LAN software (Microsoft LAN manager, Novell NetWare, and so on) and operating systems (MS-DOS, Windows, Windows NT, and others). Thus, client/server applications written to the Named Pipe interface are highly portable. Table 6-1 shows the availability of Named Pipe server or client support in myriad environments. Note that on operating systems supporting the server side of the Named Pipe (UNIX, Windows NT, and

1. Provided the remote system is NT as well. For a non-NT Named Pipe, servers can be any other driver. For example, a Microsoft LAN manager server running on OS/2 fields both Named Pipe and file I/O requests.

Table 6-1. Platforms Supporting Named Pipe

Operating System	*Network Software*	*Client/Server*
Windows NT	Basic workstation	Client/Server
OS/2	Microsoft LAN Manager	IBM LAN Server Client/Server
OS/2	Novell Redirector	Banyan Redirector Client/Server
MS-DOS	Microsoft LAN Manager	Client
Windows	Microsoft LAN Manager	Novell NetWare Banyan Vines Client
Windows for Workgroups	Basic workstation	Client
UNIX (SCO, AT & T)	LAN Manager for UNIX (LM/U)	Client/Server

OS/2), you can create both a local server and one or more local clients, regardless of the network software type.

As long as a remote server can understand the SMB protocol and respond in a dialect specified by the protocol, a local redirector can talk to the remote server. (See Figure 6-3.) But the Named Pipe interface operates over many operating systems. In Figure 6-3, the Named Pipe created on the Windows NT server can be opened by an OS/2 client as well as by a Windows NT client. Assume that the OS/2 client uses the TCP/IP transport protocol to connect to the Windows NT server and the Windows NT client uses NetBIOS protocol drivers. Note that the NT (file) server uses its own protocol (SMB) to communicate with its peers (NT Redirector and OS/2 Redirector), but uses

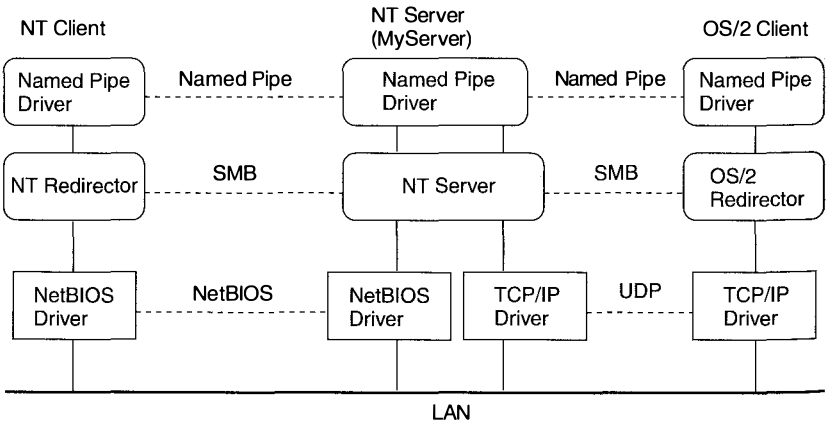


Figure 6-3. *Named Pipe Drivers Communicating by SMB Protocol*

different transport level drivers (NetBIOS and UDP drivers). The Named Pipe driver communicates with its peers using the Named Pipe protocol.

The SMB protocol is used by all the Microsoft redirectors and file servers, including the Microsoft LAN manager redirectors and file servers, Windows NT Workstation and Advanced Servers, and Windows for Workgroups. In other words, the Named Pipe client side of a client/server application can be running on a Windows NT machine, while the server end of the application can be on an OS/2 machine—provided proper drivers are present on both sides.

It is not necessary for every redirector to use the SMB protocol to support a Named Pipe communication. For example, the Novell Redirector uses **network control protocol (NCP)** to communicate to the Novell NetWare servers. Thus, a Named Pipe client running on Windows NT (or Windows 3.1) can communicate with a Named Pipe server resident on a NetWare server, provided both sides make the appropriate Named Pipe calls.[2] The same client can also communicate with another Named Pipe server that is running on a remote Windows NT or an OS/2 machine. This open architecture leads to portable applications, because they are not dependent on either the network software or the underlying transport.

6.1.2 Simple Named Pipe Client and Server

This section begins with the design of the most basic Named Pipe client/server application, one that calls the minimum (but essential) Named Pipe functions.

Each Named Pipe has two ends, the server and the client. The server end must be created first. Figure 6-4 shows the system calls in the order in which they are made. The server first creates a pipe with the **CreateNamedPipe()** API and

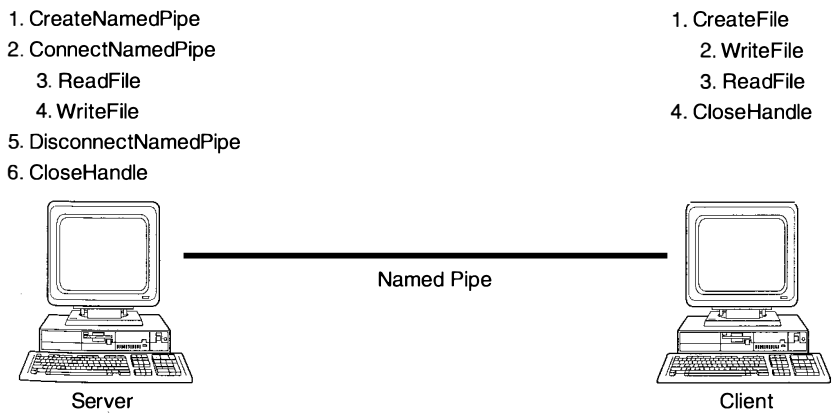


Figure 6-4. *Sequence of Calls by Named Pipe Client and Server*

2. On a NetWare server, the Named Pipe server can use a Named Pipe emulation API so that Named Pipe clients can communicate with the server.

then puts itself into a listening mode by calling the ConnectNamedPipe() API. Any client can now connect to and open the pipe by calling, for example, CreateFile() Win32 API, **_lopen()** Windows 3.x API, the open() C function, and others. If no network errors occur, the server and client ends of the Named Pipe are connected, and both client and server can read and write into the pipe as if it were a file. The server calls the **DisconnectNamedPipe()** API, stopping itself from receiving any more requests. Then, the server releases its end of Named Pipe by calling the CloseHandle() API. Finally, the client disconnects its end of the pipe by calling the **CloseHandle()** API.

6.1.3 Creating a Named Pipe

A Named Pipe is created by using the CreateNamedPipe() API whose prototype is shown here:

```
HANDLE CreateNamedPipe ( LPSTR lpszPipeName,
                         DWORD fdwOpenMode,
                         DWORD fdwPipeMode,
                         DWORD nMaxInstances,
                         DWORD cbOutBuf,
                         DWORD cbInBuf,
                         DWORD dwTimeOut,
                         LPSECURITY_ATTRIBUTES lpsa);
```


The name of the pipe (supplied by the lpszPipeName parameter) in the CreateNamedPipe() API must be in the form of \\.\PIPE\<pipe name>. The "\\." signifies a local pipe. The pipe names are case-insensitive. The Named Pipe is created by the server, and can be accessed when clients specify the name in the \\<computer name>\PIPE\<pipe name> format. For example, "\\.\PIPE\MyPipe" can be used to access a local Named Pipe. The name "\\MyServer\PIPE\MyPipe can be used to access a Named Pipe created on a remote server ("\\MyServer").

Table 6-2 shows various flags used with the CreateNamedPipe() API.

The second parameter (fdwOpenMode) of the CreateNamedPipe() API defines the open mode of a pipe. The open mode of a pipe specifies the direction of flow of data in the pipe (for example, unidirectional, from the client to the server, bidirectional, and so on). The pipe can also be put in a write-through mode. This is pertinent only when client and server are on two distinct machines. When the pipe open mode is set to write-through, any writes into the pipe do not return until the data has actually been written to the remote end's system buffers. If this flag is not set, the system enhances the efficiency of network operations by buffering data until a minimum number of bytes have accumulated, or until a maximum time period has expired. Write-through mode is useful for transaction-oriented applications, such as database servers. Finally, you can set Named Pipe to the overlapped mode (FILE_FLAG_OVERLAPPED), which allows I/O requests on a Named Pipe to be nonblocking when an I/O operation cannot be completed immediately.

Table 6-2. CreateNamedPipe() Flags

Flags	*Description*
Open Mode	
PIPE_ACCESS_DUPLEX	The pipe is bidirectional; both server and client processes can read from and write to the pipe.
PIPE_ACCESS_INBOUND	The flow of data in the pipe goes from client to server only.
PIPE_ACCESS_OUTBOUND	The flow of data in the pipe goes from server to client only.
FILE_FLAG_WRITE_THROUGH	Write-through mode is enabled.
FILE_FLAG_OVERLAPPED	Overlapped mode is enabled.
Pipe Mode	
PIPE_TYPE_BYTE	Data is written to the pipe as a stream of bytes.
PIPE_TYPE_MESSAGE	Data is written to the pipe as a stream of messages.
PIPE_READMODE_BYTE	Data is read from the pipe as a stream of bytes.
PIPE_READMODE_MESSAGE	Data is read from the pipe as a stream of messages.
PIPE_WAIT	Blocking mode is enabled.
PIPE_NOWAIT	Nonblocking mode is enabled.

The pipe mode of the pipe (specified by the fdwPipeMode parameter) defines how to read and write into the pipe and whether it supports asynchronous operations. The PIPE_TYPE_xxx constants define the write mode of the pipe, or, more precisely, the way data is written to the pipe. Similarly, the constants PIPE_READMODE_xxx define the read mode of the pipe, or, more precisely, the way it is read. By default, all data is written to the pipe buffer as a stream of bytes (PIPE_TYPE_BYTE) and the data is accessed as a stream of bytes (PIPE_READMODE_BYTE). However, it makes sense for some applications to send and receive messages with clear boundaries. Most applications that receive and send transactions, such as database applications and transaction processing applications, prefer to use message-based Named Pipes. For these pipes, set the pipe write mode as PIPE_TYPE_MESSAGE and read mode as PIPE_READMODE_MESSAGE; each data block written into the pipe is distinct from the previous or the following data block. Further, the entire message transmits at one time—as long as there is some agreement on the maximum size of a message. In message mode, the reader can only read a complete message, so if the reader's read buffer size is smaller than an incoming message, the message cannot

be read in, and ReadFile() API will return an error of ERROR_MORE_DATA. While it is good practice for most applications to have the same read and write modes on both ends of the pipe, there is one exception: a message write mode (PIPE_TYPE_MESSAGE) pipe can be read in byte read mode (PIPE_READMODE_BYTE). This mode is needed when the reader does not have enough memory for reading in an entire message and changes the read mode from message read mode to byte read mode. Then, the reader can read in the entire message as a stream of bytes and later change the pipe state back to message read mode.

A pipe can be specified as a synchronous or asynchronous type by OR-ing PIPE_WAIT or PIPE_NOWAIT with the pipe mode parameter (specified by the fdwPipeMode parameter) of the CreateNamedPipe() API. By default, all pipes are synchronous, which means that any I/O operation requested on them will block until the I/O is complete. On asynchronous pipes, all I/O requests on the pipe return right away, without blocking, and the application must use either a polling technique or synchronization objects (such as semaphores) to determine when the I/O operation has completed.

While creating a Named Pipe, the server is able to specify (using the nMaxInstances parameter) the pipe's number of instances, from one to PIPE_UNLIMITED_INSTANCES. Figure 6-5 shows the difference between multiple pipes and a pipe with multiple instances. In Figure 6-5(a), the server supports multiple Named Pipes. The client that wants to connect to the server needs to know the name of the pipe (such as\\.\PIPE\pipe1, \\.\\PIPE\pipe2, and so on). In Figure 6-5(b), the client always connects to same name (\\.\PIPE\pipe1, if it is a local pipe) although the server has created multiple instances of the same pipe to support multiple clients.

Figure 6-5(b) shows that all instances of a multi-instance pipe have the same name (in the figure, \\.\PIPE\pipe1). Thus, a client of a pipe connects

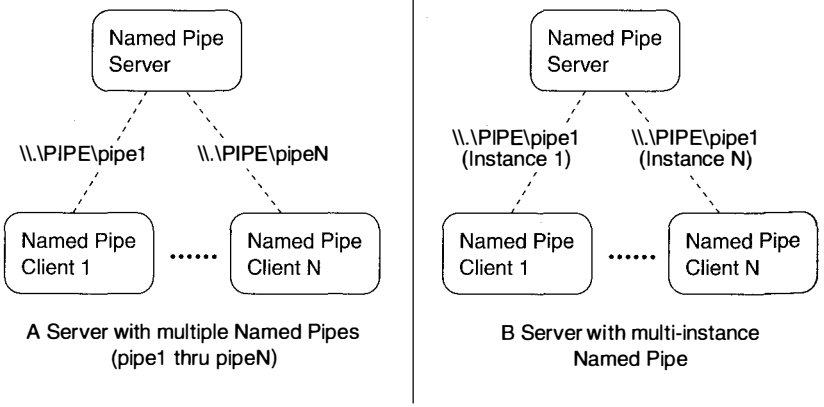


Figure 6-5. *Multiple Pipes and Multiple Instances of the Same Pipe*

by specifying the well-known name of the pipe. The server must manage a connection on each instance of a multi-instance pipe. The great benefit of having multiple instances of a Named Pipe is that the server can support one or more clients at the same time. Most importantly, the client application can depend on a single pipe name.

When you create a Named Pipe, specify the default size of the input and output buffer sizes (using the cbOutBuf and cbInBuf parameters) that the system should set aside for this Named Pipe. For message mode pipes, if you expect to be sending and receiving data blocks whose sizes can vary from a couple of bytes to a maximum of 8192 bytes, specify the input and output buffer size at (2 x 8192), thus allowing the system to store up to two of the largest data packets at a time without having to allocate more internal memory at run time. The system takes these numbers as "suggestions" and optimizes the buffers and their sizes dynamically.

You can also specify the default open time-out value (using the dwTimeout parameter) in milliseconds when creating a Named Pipe. The open time-out parameter gives the server a say on how long one or more clients should wait on a request to connect with the server. The time-out value affects the clients who try to connect to a Named Pipe by using the **WaitNamedPipe()** API.

Finally, while creating the Named Pipe you can attach Windows NT security access controls (using the lpsa parameter) with the CreateNamedPipe() function. This allows the server application to integrate with the standard account database of a Windows NTAS to control and limit access. If NULL is specified in this field, then the Named Pipe has the same security limits and access controls as the process that created it. Notice that the currently logged-on user must have the privileges to create a pipe (FILE_CREATE_ PIPE_INSTANCE).

When the call to the CreateNamedPipe() API completes, a Named Pipe is created and a handle to the server end of the pipe is returned. You must call the CreateNamedPipe() API for each instance of a Named Pipe.

6.1.4 Accepting Incoming Open Requests from Clients

As soon as the server end of the pipe has been created, the Named Pipe object becomes visible in the name space, and clients can try to open the Named Pipe. However, on the server end, the server controls when it will start to process incoming requests. The server indicates that it is open and ready to accept a request from a client by calling the ConnectNamedPipe() API, the prototype for which appears below. Notice the use of an overlapped structure parameter. Overlapped structures allow I/O operations to proceed asynchronously. Additionally, the caller can arrange to be called back at a certain function when the asynchronous overlapped I/O operation completes.

```
BOOL ConnectNamedPipe ( HANDLE hNamedPipe, LPOVERLAPPED lpo);
```

When it calls the ConnectNamedPipe() function, the server specifies the handle to the newly created Named Pipe (using the hNamedPipe parameter) on which the server wants to accept calls. A pointer can also be supplied to a structure (using the lpo parameter) (OVERLAPPED) if the pipe has been created with an overlapped mode (FILE_FLAG_OVERLAPPED). If the server supplies a NULL parameter in this field and the Named Pipe has been created as a synchronous type (PIPE_WAIT) pipe, then a call to the ConnectNamedPipe() function blocks until the client makes a call to open the pipe. Upon successful connection, the function returns TRUE. (A successful connection means that both the server and client ends of the Named Pipe are now joined, and both can read from, or write into, the pipe.) It is possible for the client end to call to connect to the pipe before the server calls the ConnectNamedPipe() API—that is, the client call comes between the CreateNamedPipe() API and the ConnectNamedPipe() API calls made on the server. If this happens, the ConnectNamedPipe() function returns FALSE, but a subsequent call to the GetLastError() function will return ERROR_PIPE_CONNECTED, and client and server can start using the pipe. This behavior is similar for a Named Pipe created in the overlapped mode. When the pipe is in the overlapped mode, and no client has made calls to connect to the pipe, the ConnectNamedPipe() API returns FALSE and a subsequent call to the GetLastError() API returns ERROR_IO_PENDING.

When the pipe is nonblocking (PIPE_NOWAIT), the ConnectNamedPipe() API displays one of two behaviors, depending on when it is called. In all cases, it returns immediately. When the ConnectNamedPipe() API is first called after calling the DisconnectNamedPipe() API to disconnect the previous client from a pipe, the ConnectNamedPipe() API returns TRUE. The rest of the time, it returns a value of FALSE. Subsequent calls to the GetLastError() API yield ERROR_PIPE_LISTENING if no client has made a call to connect. If a client has made a call to connect, the GetLastError() API returns ERROR_PIPE_CONNECTED.

The server disconnects from the client with the DisconnectNamedPipe() API, whose prototype follows:

```
BOOL DisconnectNamedPipe( HANDLE hNamedPipe);
```

This call, made on the server end, breaks the connection between the server and client ends of the pipe. However, the client may not yet know it and still may have its end open. In this case, it will get an error such as ERROR_PIPE_NOT_CONNECTED the next time it tries to perform any I/O operation at its end of the pipe. The client terminates its end of the pipe by calling the CloseHandle() API. Once the server has disconnected with the client, it cannot reconnect with the same client by calling the ConnectNamedPipe() API. The "old" client, which still has not called the CloseHandle() API, must close the pipe handle and reopen the pipe as if it were a new client.

The server can remove its end of the Named Pipe by calling the CloseHandle() API on the handle of the Named Pipe. However, if the server wants to continue servicing other clients, it can use the same handle it just supplied to the DisconnectNamedPipe() API for the next call to the ConnectNamedPipe() API. Thus, the server can reuse the pipe without having to call the CloseHandle() API, the CreateNamedPipe() API, and the ConnectNamedPipe() API. The client, however, does not have this luxury. Once the pipe has been closed, by either client or server, the client cannot reuse the handle originally obtained by opening the pipe. It must close the handle by calling the CloseHandle() API and then make a fresh call to some open function, such as the CreateFile() API. The code that follows illustrates client and server ends connecting on the pipe, and summarizes the major points discussed in this section.

Notice that the code fragment shows code paths for all possible pipe modes. This is for illustration only. By default, the client and the server work with blocked message-mode Named Pipes. In this example, the server creates a Named Pipe, then waits for the client to connect to it by calling ConnectNamedPipe(). Once the client connects to it, the server reads the client's requests and processes them. Finally, it disconnects the pipe from the client. The client opens the pipe, writes one request into it, and then closes it. This example uses a design in which the client sends a request to the server to shut down. In the real world, the server would ask for input from a local console (or through a local GUI).

```
////////////////////////////////////////////////////////////////
// FILE: Server1.c
//
// Creating a server which simply creates a Named Pipe and
// connects to the client(s).
//
// Usage: (use it with client1 or client2)
//     Server1 -n:<pipe name> -p:<pipe mode> -o:<open mode>
//             -m:<max instance>
//
#include <windows.h>
#include <stdio.h>
// Constants used when creating Named Pipe
#define WAIT_NAMED_PIPE_TIMEOUT (2 * 60 * 1000)     // 2 Minutes
#define OUT_BUFFER_SIZE   (80L)
#define IN_BUFFER_SIZE    (80L)
#define MESSAGE_SIZE      (80L)
#define SHUT_DOWN_SERVER "Shutdown"
// Macros used
#define REPORT_ERROR(string) {\
    printf("Error [%ld] in %s\n", GetLastError(), string); \
    CloseHandle (hNamedPipe);                              \
    return (1);                                            \
}\

#define DEFAULT_PIPE_NAME "\\\\.\\PIPE\\Server1"
//
```

```
// Function to retrieve data from command line
BOOL ParseCommandLine ( int     argc,
                        char    **argv,
                        char    **pszPipeName,
                        DWORD   *pdwPipeMode,
                        DWORD   *pdwOpenMode,
                        DWORD   *pdwMaxInstance
                      )
{
    char *ptr;
    argc--;
    argv++;
    // Setup Defaults
    *pszPipeName = DEFAULT_PIPE_NAME;
    *pdwPipeMode = PIPE_TYPE_MESSAGE | PIPE_READMODE_MESSAGE |
                   PIPE_WAIT;
    *pdwOpenMode = PIPE_ACCESS_DUPLEX;
    *pdwMaxInstance = PIPE_UNLIMITED_INSTANCES;
    while (argc)
    {
          ptr = argv[0];
          if (*ptr=='-')
          {
               ptr++;
               switch (*ptr)
               {
               case 'n': ptr+=2;*pszPipeName = ptr; break;
               case 'p': ptr+=2;*pdwPipeMode = atol(ptr);break;
               case 'o': ptr+=2;*pdwOpenMode = atol(ptr);break;
               case 'm': ptr+=2;*pdwMaxInstance = atol(ptr);break;
               default:
                    return FALSE;
               }
          }
          else
               return FALSE;
          argc--;
          argv++;
    }
    return TRUE;
}
int main (int argc, char **argv)
{
    char     *pszPipeName;        // Name of Pipe
    DWORD    dwPipeMode=0;        // Pipe mode such as PIPE_READMODE_BYTE
    DWORD    dwOpenMode=0;        // Open mode such as PIPE_ACCESS_DUPLEX
    DWORD    dwMaxInstance=0;     // Number of instances of the pipe
    HANDLE   hNamedPipe= NULL;    // Handle to an instance of pipe
    BOOL     fEndServer = FALSE; // Flag determines when to end server app.
    BOOL     fRc;                 // Return Code
    BOOL     fClientConnected=FALSE; // Indicates if client has connected

    char     szReadBuffer [MESSAGE_SIZE];
    DWORD    dwBytesRead;

    // Call a helper routine which will parse command line and
    // give us values
```

```
if (!ParseCommandLine ( argc,
                        argv,
                        &pszPipeName,
                        &dwPipeMode,
                        &dwOpenMode,
                        &dwMaxInstance
                      ))
{
      printf("Error parsing command line\n"); return (1);
}
// Create an instance Named Pipe.
// Note: We are not truly dealing with multiple instance case even
// if we could be creating multi-instance pipe.
hNamedPipe = CreateNamedPipe( pszPipeName,
                              dwOpenMode,
                              dwPipeMode,
                              dwMaxInstance,
                              OUT_BUFFER_SIZE,
                              IN_BUFFER_SIZE,
                              WAIT_NAMED_PIPE_TIMEOUT,
                              // Security Descriptor
                              NULL
                            );
if (hNamedPipe==INVALID_HANDLE_VALUE)
{
      printf("Error creating Named Pipe: [%ld]\n", GetLastError());
      return (1);
}

// We loop until some client sends us a request to end the program.
// Thru each loop, we connect to a client and read its request,
// print out the request, and disconnect from him. Of course,
// we check if client sent a request to shut down the server itself.

while (fEndServer==FALSE)
{
        // Make a call to start accepting call from clients.
        // We are not dealing with overlapped mode in this sample.

        printf("Calling ConnectNamedPipe\n");
        fRc = ConnectNamedPipe (hNamedPipe, NULL);

        //  Deal with return code depending upon pipe mode
        if (!(dwPipeMode & PIPE_NOWAIT))
        {
                // Synchronous Mode
                if (fRc==TRUE)
                    fClientConnected=TRUE; // client connected
                else
                    REPORT_ERROR("Wait Mode ConnectNamedPipe")
        }
        else if (dwPipeMode & PIPE_NOWAIT)
        {
                if (fRc==FALSE)
                {
                     if (GetLastError()==ERROR_PIPE_LISTENING)
                     {
```

```
                                        Sleep ( 10 );// No clients connected yet
                                        continue;
                                }
                        else if (GetLastError()==ERROR_PIPE_CONNECTED)
                                fClientConnected=TRUE;      // client connected
                        else if (GetLastError()==ERROR_NO_DATA)
                        {
                                // Client has closed handle on this instance
                                // and server did not disconnect
                                DisconnectNamedPipe (hNamedPipe);
                                continue;
                        }
                        else
                                REPORT_ERROR("No Wait Mode ConnectNamedPipe")
                }
                else
                        ;       // State same as ERROR_PIPE_LISTENING
        }
        // Return codes for overlapped mode. In this simple
        // example, following code will never be executed
        // since we had supplied NULL OVERLAPPED stucture when
        // calling ConnectNamedPipe()
    else if (dwOpenMode & FILE_FLAG_OVERLAPPED)
    {
                if (fRc==TRUE)
                        // Not documented if this return code
                        // is possible. If it is, then it should
                        // mean that client connected
                        fClientConnected=TRUE;
                else if (GetLastError()==ERROR_IO_PENDING)
                {
                        // No clients connected right away. Will have to use
                        // overlapped structures and events.
                        ;
                }
                else if (GetLastError()==ERROR_PIPE_CONNECTED)
                        fClientConnected=TRUE;
                else
                        REPORT_ERROR("Overlapped ConnectNamedPipe")
    }
    // At this point, we will take care of -
    //    Asynchronous Named Pipe
    //    Overlapped Named Pipe
    // until a client connects. We will elaborate on this
    // in a later example.

    // If client already connected, we can proceed to receive
    // request from it
    if (fClientConnected)
    {
         // In the following calls, we handle error
         // simply by closing pipe. In later examples, we
         // demonstrate handing read/write errors properly

         // Receive a request from client
         fRc = ReadFile(hNamedPipe,
                        szReadBuffer,      // Data read in here
```

```
                    MESSAGE_SIZE,      // Bytes to read
                    &dwBytesRead,      // Bytes read by ReadFile()
                    NULL               // Overlapped structure
                   );
        if (!fRc)
                     REPORT_ERROR("ReadFile")

            // Process the request
            if (dwBytesRead)
            {
                printf("Client Request is : [%s]\n", szReadBuffer);
                // Check if we need to close down
                if (stricmp(szReadBuffer, SHUT_DOWN_SERVER)==0)
                fEndServer = TRUE;
            }
            else
                printf("Zero bytes read from Client\n");
    }// end of if(connected)
    // At this point, we have services this client. Hence, we
    // will disconnect him
    fRc = DisconnectNamedPipe( hNamedPipe );
    if (fRc==FALSE)
        REPORT_ERROR("DisconnectNamedPipe")
    else
        fClientConnected=FALSE;
    }
    // Need to close the server end of pipe
    CloseHandle( hNamedPipe );

    printf("Ending Server application\n");
    return (0);
}
////////////////////////////////////////////////////////////////////////
// FILE: Client1.c
// Creating a client which simply opens a Named Pipe and
// connects to the server.
// Usage:
//    Client1 -n:<pipe name>  -a:<access mode> -s:<share mode>
//            -f[<End Server>]
//
#include <windows.h>
#include <stdio.h>
// Constants used when creating Named Pipe
#define MESSAGE_SIZE     (80L)
#define SHUT_DOWN_SERVER "Shutdown"

// Macros used
#define REPORT_ERROR(string) {\
                printf("Error [%ld] in %s\n", GetLastError(), string); \
                CloseHandle (hNamedPipe);    \
                return (1);    \
}

#define DEFAULT_PIPE_NAME "\\\\.\\PIPE\\Server1"
//
// Function to retrieve data from command line
BOOL ParseCommandLineForClient ( int argc, char **argv,
```

```
                                        char    **pszPipeName,
                                        DWORD   *pdwAccessMode
                                        DWORD   *pdwShareMode,BOOL  *pfEndServer
                                      )
{
    char *ptr;
    argc--;
    argv++;
    // Setup Defaults
    *pszPipeName   = DEFAULT_PIPE_NAME;
    *pdwAccessMode = GENERIC_READ|GENERIC_WRITE; // read/write
    *pdwShareMode  = 0;                          // no sharing
    *pfEndServer   = FALSE;                   // Don't terminate the server
    while (argc)
    {
          ptr = argv[0];
          if (*ptr=='-')
          {     ptr++;
                switch (*ptr)
                {
                case 'n': ptr+=2; *pszPipeName = ptr; break;
                case 'a': ptr+=2; *pdwAccessMode = atol(ptr); break;
                case 's': ptr+=2; *pdwShareMode = atol(ptr); break;
                case 'f':         *pfEndServer = TRUE; break;
                default:
                     return FALSE;
                }
          }
          else
                return FALSE;
          argc--;
          argv++;
    }
    return TRUE;
}
int main (int argc, char **argv)
{
    char     *pszPipeName; // Name of Pipe
    DWORD    dwAccessMode; // Access Mode
    DWORD    dwShareMode;  // Share Mode
    HANDLE   hNamedPipe;   // Handle to an instance of pipe
    BOOL     fRc;          // Return Code
    BOOL     fEndServer;   // Determines if client will shut down the server
    char     szWriteBuffer [MESSAGE_SIZE] = {"Hello From Client"};
    DWORD    dwBytesWritten;
    // Call a helper routine which will parse command line and
    // give us values
    if (!ParseCommandLineForClient ( argc, argv,&pszPipeName,
                                     &dwAccessMode, &dwShareMode,
                                     &fEndServer
                                   ))
    {
          printf("Error parsing command line\n"); return (1);
    }
    // Open an instance Named Pipe.
    hNamedPipe = CreateFile ( pszPipeName,
                              dwAccessMode,
```

```
                                dwShareMode,
                                NULL,     // Security Descriptor
                                          // Pipe must have created already
                                OPEN_EXISTING,
                                FILE_ATTRIBUTE_NORMAL,
                                NULL      // No Template
                               );
    if (hNamedPipe==INVALID_HANDLE_VALUE)
    {
          printf("Error opening Named Pipe: [%ld]\n", GetLastError());
          return (1);
    }
    // We will send the server one request. Prepare the request
    if (fEndServer)
    {
          strcpy(szWriteBuffer, SHUT_DOWN_SERVER);
    }
    // For this example, we do not follow rigorous error checking
    fRc = WriteFile ( hNamedPipe,
                      szWriteBuffer,
                      strlen(szWriteBuffer) + 1, // Bytes to write
                      &dwBytesWritten,
                      NULL
                    );
    if (fRc==FALSE)
        REPORT_ERROR("WriteFile");

    // We are done. Let us close the pipe
    CloseHandle ( hNamedPipe );
    printf("Ending Client application\n");
    return(0);
}
```

6.1.5 Opening a Pipe

A Win32 client opens a pipe by using the function CreateFile(). A non-Windows NT client opens a pipe using its native open functions, such as the _lopen() API in Windows 3.1 and the **DosOpen()** API in OS/2. Note that opening a pipe from the client end causes two actions: it creates a Named Pipe client end, and it connects the client and server ends, both of which can be either local or remote. Creating the client side of the Named Pipe causes some system resources to be allocated. Additionally, the connection to a remote server lends itself to myriad possible network failures and can involve long delays before connection is actually established. Hence, application writers must be aware that while opening a pipe looks much like opening a file, it has implications of system overhead and potential failure.

The code fragment above shows a typical Windows NT client connecting to the Named Pipe server using CreateFile(), whose prototype is:

```
HANDLE CreateFile( LPCTSTR                lpszName,
                   DWORD                  fdwAccess,
                   DWORD                  fdwShareMode,
                   LPSECURITY_ATTRIBUTES  lpsa,
```

```
DWORD                   fdwCreate,
DWORD                   fdwCreate,
DWORD                   fdwAttrsAndFlags,
HANDLE                  hTemplateFile);
```

As mentioned earlier, the client supplies the name of the pipe (specified with the lpszName parameter) in the format \\.\PIPE\<pipe name> for local servers and \\<remote computer>\PIPE\<pipe name> for remote servers. Table 6-3 shows the valid flags used with the CreateFile() API for opening a Named Pipe.

Table 6-3. *CreateFile() Flags for Opening Named Pipes*

Flags	*Description*
Access Flags	
0	Queries access without actually accessing the pipe.
GENERIC_READ	Specifies read access to the pipe.
GENERIC_WRITE	Specifies write access to the pipe.
Sharing Flags	
0	Prevents the pipe from being shared.
FILE_SHARE_READ	Allows other open operations to be performed on the pipe for read access.
FILE_SHARE_WRITE	Allows other open operations to be performed on the pipe for write access.
Create Flags	
CREATE_NEW	Creates a new pipe. Invalid for pipes.
CREATE_ALWAYS	Creates a new pipe. Invalid for pipes.
OPEN_EXISTING	Opens the pipe. Fails if the pipe does not exist.
OPEN_ALWAYS	Opens the pipe, if it exists. If not, same as CREATE_NEW
TRUNCATE_EXISTING	Opens the pipe and truncates it so that its size is zero bytes.
Relevant Attributes and Flags	
FILE_ATTRIBUTE_NORMAL	Normal pipe.
FILE_ATTRIBUTE_READONLY	The pipe is read only.
FILE_FLAG_WRITE_THROUGH	Instructs the operating system to write through any intermediate cache and go directly to the server. The operating system can still cache writes but cannot lazily flush the writes.

(continued)

Table 6-3 continued

Flags	*Description*
FILE_FLAG_OVERLAPPED	Instructs the operating system to initialize the file, so the operations ReadFile, WriteFile, ConnectNamedPipe, and TransactNamedPipe, which take a significant amount of time to process, return ERROR_IO_PENDING. When the operation is finished, an event is set to the signaled state.
SECURITY_SQOS_PRESENT	Used in conjunction with one or more of the following flags.
SECURITY_ANONYMOUS	Specifies to impersonate the client at the Anonymous impersonation level.
SECURITY_IDENTIFICATION	Specifies to impersonate the client at the Identification impersonation level.
SECURITY_IMPERSONATION	Specifies to impersonate the client at the Impersonation impersonation level.
SECURITY_DELEGATION	Specifies to impersonate the client at the Delegation impersonation level.
SECURITY_CONTEXT_TRACKING	Specifies that the security tracking mode is dynamic. If this flag is not specified, Security Tracking Mode is static.
SECURITY_EFFECTIVE_ONLY	Specifies that only the enabled aspects of the client's security context are available to the server. If this flag is not specified, all aspects of the client's security context are available. This flag allows the client to limit the groups and privileges a server can use while impersonating the client.

The client must also declare its mode of access up front when opening the pipe (specified by the fdwAccess parameter). Table 6-4 shows the interconnection between the Named Pipe open mode (specified by the server when it creates the Named Pipe) and the client access mode. Boxes marked with (↔) are legal. Boxes marked with (✥) are not a valid combination of the pipe open mode and the client access mode.

The only share mode of the pipe (specified by the fdwShareMode parameter) that makes sense is zero (which means the Named Pipe cannot be shared

Table 6-4. Legal Combinations of the Named Pipe Open Mode and the Client Access Mode

	Open Mode on Server		
Client Access Mode	*PIPE_ACCESS _INBOUND*	*PIPE_ACCESS _OUTBOUND*	*PIPE_ACCESS _DUPLEX*
GENERIC_READ	✥	↔	↔
GENERIC_WRITE	↔	✥	↔
GENERIC_READ | GENERIC_WRITE	↔	↔	↔

by any other process). By specifying NULL as a security descriptor (with the lpsa parameter) in the call to the CreateFile() API, the Named Pipe is given the same privileges as the client processes.

The client can specify the file create mode (using the fdwCreate parameter) when opening a Named Pipe. The create mode (OPEN_EXISTING) should be specified so the client connects to the server only if such a Named Pipe exists. The client applications can also specify file attributes (with the fdwAttrsAndFlags parameter). Most applications will use the (FILE_ATTRIBUTE_NORMAL) flag as the attribute of the pipe. However, transaction-oriented applications may wish to OR the attribute field with the (FILE_FLAG_WRITE_THROUGH) flag to ensure all the data written by the client [with, for instance, the WriteFile() API] is flushed to the server-side system buffers immediately upon a write. As explained earlier, without this flag, the client-side Named Pipe drivers may cache the written data for optimizing network efficiency. Further, the client applications can OR the attribute field with the (FILE_FLAG_OVERLAPPED) flag so all subsequent client-side I/O operations on a Named Pipe return immediately with ERROR_IO_PENDING. When the I/O operation completes, the system informs the client, and the client can perform other operations while an I/O operation is pending. Finally, security-sensitive applications can OR the attribute field with other parameters discussed later).

When calling the CreateFile() API, you can specify a template file (with the hTemplateFile parameter), but it is unlikely many Named Pipe clients will use this feature. Template files allow an application to clone a new file based on a template that has the desired attributes.

On successful completion of the CreateFile() function, the function returns a valid handle to the Named Pipe. This handle can be used to send and receive messages with the server. This handle is closed, and the associated system resources deallocated, by calling the CloseHandle() API. If the server does not exist or has not yet created a Named Pipe, the CreateFile() API fails, returning INVALID_HANDLE_VALUE as a handle. An immediate call to the

GetLastError() API will return ERROR_PATH_NOT_FOUND. If the server is present, but has no free instances of Named Pipe available to accept a client's call, then the CreateFile() API fails with the GetLastError() API returning ERROR_PIPE_BUSY. At this point, the client application can take various measures, such as asking the user to try again later.

However, the Named Pipe communication interface does provide a programmatic way to deal with server unavailability. Client applications can call the WaitNamedPipe() API, which allows one or more clients to wait for a free Named Pipe. Its prototype follows:

```
BOOL WaitNamedPipe ( LPSTR lpszPipeName, DWORD dwTimeOut);
```

A client specifies the full name of a local or remote Named Pipe (using the lpszPipeName parameter) and a time-out value (in milliseconds, using the dwTimeOut parameter). Thus, one or more clients can wait for a free Named Pipe. When a Named Pipe becomes available at the server end, the WaitNamedPipe() API completes with TRUE, and the client calls the CreateFile() function to connect to the Named Pipe. However, note that when a pipe becomes free, all clients waiting on a Named Pipe are notified at the same time. The client whose connection request comes to the server first gets the pipe connection, while the rest of the clients get ERROR_PIPE_BUSY. The following code fragment shows a modified client that uses the WaitNamedPipe() API to wait when the client gets ERROR_PIPE_BUSY error upon CreateFile().

```
// (Full sample can be found in CLIENT2.C in the sample sources)
// Modification around CreateFile() API shown in the client
// shown earlier
int main (int argc, char **argv)
{
    // get pipe details
    <...>
    // Now loop trying to find a free Named Pipe instance
    int cbNumberOfRetries = NUMBER_OF_OPEN_RETRIES;
    while (cbNumberOfRetries--)
    {
        hNamedPipe = CreateFile ( pszPipeName,
                                  fdwAccessMode,
                                  fdwSharedMode,
                                  NULL,              // Security Descriptor
                                  CREATE_EXISTING,   // Pipe must have
                                                     // created already
                                  FILE_ATTRIBUTE_NORMAL,
                                  NULL               // No Template
                );
        if (hNamedPipe==INVALID_HANDLE_VALUE)
        {
            switch (GetLastError())
            {
            case ERROR_PIPE_BUSY:
                // Server is busy. Let us wait
                if (WaitNamedPipe(pszPipeName,
                        NMPWAIT_USE_DEFAULT_WAIT)==TRUE)
```

```
                continue;  // Server is free. Try CreateFile() again
            else
                break;     // Wait timed out or could be any number
                           // of failures such as network failure etc.
                           // In this example, we will try CreateFile()
                           // again and exit program on failures. Else
                           // we will wait again.
        default:
          printf("Error opening Named Pipe: [%ld]\n",
                 GetLastError());return (1);
          }
      }
      else
        break;     // We have connected so break out of loop
  }
  // Connected with the pipe
  <..Rest of the Client code ..>
}
```

A client can specify the flag (NMPWAIT_USE_DEFAULT_WAIT) as the time-out value when it calls the WaitNamedPipe() function. If it does this the client remains blocked for the duration specified as the time-out value by the server. This leads to simpler synchronization between server and client. However, clients can specify any other time-out value, including the flag (NMPWAIT_WAIT_FOREVER), which causes the client to block until the server becomes free. While this option leads to simpler design, it makes it possible for the client to hang forever, without any hope that user intervention will abort the operation.

6.1.6 Reading/Writing Named Pipe

Once the pipe has been connected, Named Pipe client/server programs can use several functions to transmit data to, or receive data from, each other using the Named Pipe. These include Win32 functions, such as ReadFile(), ReadFileEx(), WriteFile(), WriteFileEx(), and C-Runtime functions such as **read()** and **write()**.

There are three issues that affect I/O operations using Named Pipes. They are synchronous or asynchronous pipes, byte or message mode pipes, and overlapped or nonoverlapped pipes. By default, all pipes are opened in byte read mode and block on all I/O operations until either the operation completes or times out. The default on the client side of the pipe can be changed by calling the **SetNamedPipeHandleState()** function. On the server side, the Named Pipe characteristics are specified when the server creates a Named Pipe, and can be changed by calling the SetNamedPipeHandleState() API.

Most Win32 client/server programs find the ReadFile() and the WriteFile() APIs adequate for their needs. Sophisticated Win32 client/server programs can use overlapped I/O, which is discussed later.

```
BOOL ReadFile ( HANDLE hFile,
                LPVOID lpBuffer,
                DWORD nNumberOfBytesToRead,
                LPDWORD lpBytesRead,
                LPOVERLAPPED lpOverLapped);
```

```
BOOL WriteFile ( HANDLE hFile,
                 LPVOID lpBuffer,
                 DWORD nNumberOfBytesToWrite,
                 LPDWORD lpBytesWritten,
                 LPOVERLAPPED lpOverLapped);
```

Both functions take a handle to the Named Pipe as a parameter (using the hFile parameter) and return TRUE on success. Both take a buffer (specified by the lpBuffer parameter) that is the target of the I/O operations; the ReadFile() function reads incoming data into the buffer and the WriteFile() API writes out the data in the buffer. A caller must specify the number of bytes to read when calling the ReadFile function (with the nNumberOfBytesToRead parameter). Similarly, the write function must specify the number of bytes to write out from the buffer into the pipe when calling the WriteFile() API (with the nNumberOfBytesToWrite parameter). Both functions take a pointer to a DWORD in which the actual number of bytes transferred can be returned [lpBytesRead in case of ReadFile() and lpBytesWritten in the case of WriteFile()]. These handle resource constraints and network failures in the case of nonblocking pipes.

Recall that based on the blocking behavior of a pipe, pipes can be classified into two categories, blocking and nonblocking. In the case of blocking pipes (default), a read operation will always block until the write operation has written all the bytes requested by the reader (using nBytesToRead). The only exception to this rule is when the pipe is in message (as well as in blocking) mode, and the message size is greater than the supplied read buffer. In this case, the ReadFile() API returns FALSE and the GetLastError() API returns ERROR_MORE_DATA. The following code fragment shows a modified version of the previous client, using (ERROR_MORE_DATA) error returned by the ReadFile()API on a message-mode Named Pipe. To illustrate the point, in this example a small buffer is supplied to force this error. You can extrapolate the way this feature of Named Pipes can be used to receive variable length data. Note that because the Named Pipe is opened in byte mode by the CreateFile() API on the client end, you should set the Named Pipe in message mode by calling the SetNamedPipeHandleState() API.

```
// FILE: client3.c
// (See %BOOK%\nmpcode\client3.c for full code. Some lines omitted
// as they have been explained earlier and are repetitive)
//
// Creating a client which uses message mode variable length data
// and uses ERROR_MORE_DATA to understand that large response
// is coming from the server
//
// Usage: (use it with Server3 with -r:1)
//
// Client3 -n:<pipe name>  -a:<access mode> -s:<share mode>
//         -f[<End Server>]
//
#define NUMBER_OF_OPEN_RETRIES 5
// Macros used
```

```
#define DEFAULT_PIPE_NAME "\\\\.\\PIPE\\Server3"
#define CLIENT_REQUEST  "Client request %d"

//
// Function to retrieve data from command line
BOOL ParseCommandLineForClient ( int    argc,char **argv,
                                 char **pszPipeName,DWORD *pdwAccessMode,
                                 DWORD *pdwShareMode,BOOL  *pfEndServer
                               )
{
    char    *ptr;
    argc--;
    argv++;
    // Setup Defaults
    *pszPipeName  = DEFAULT_PIPE_NAME;
    *pdwAccessMode= GENERIC_READ|GENERIC_WRITE;// read/write
    *pdwShareMode = 0;                         // no sharing
    *pfEndServer  = FALSE;                     // Don't terminate the server
    while (argc)
    {
        < process command line parameters >
    }
    return TRUE;
}
int main (int argc, char **argv)
{
    char     *pszPipeName; // Name of Pipe
    DWORD    dwAccessMode; // Access Mode
    DWORD    dwShareMode;  // Share Mode
    HANDLE   hNamedPipe;   // Handle to an instance of pipe
    BOOL     fRc;          // Return Code
    BOOL     fEndServer;   // Determines if client will shut down the server

    char     szWriteBuffer [MESSAGE_SIZE] = {"Hello from Client"},
             szReadBuffer  [MESSAGE_SIZE];
    DWORD  dwBytesRead, dwBytesWritten, dwPipeMode, dwRc, i;
    // Call a helper routine which will parse command line and
    // give us values
    if (!ParseCommandLineForClient( argc,
                                    argv,
                                    &pszPipeName,
                                    &dwAccessMode,
                                    &dwShareMode,
                                    &fEndServer
                                  ))
    {
        printf("Error parsing command line\n"); return (1);
    }
    // Open an instance Named Pipe.
    hNamedPipe = CreateFile ( pszPipeName,
                              dwAccessMode,
                              dwShareMode,
                              NULL,                  // Security Descriptor
                              OPEN_EXISTING,         // Pipe should exist
                              FILE_ATTRIBUTE_NORMAL,
                              NULL     // No Template
                            );
```

```
if (hNamedPipe==INVALID_HANDLE_VALUE)
      REPORT_ERROR("opening Named Pipe");
//
// By default, pipe is opened in byte mode.
// Hence, we need to set it to message
// mode so we get the error ERROR_MORE_DATA
// when read buffer is smaller than incoming data
//
dwPipeMode = PIPE_READMODE_MESSAGE | PIPE_WAIT;
if (SetNamedPipeHandleState (hNamedPipe, &dwPipeMode, NULL,
                             NULL)==FALSE)
{
      REPORT_ERROR("SetNamedPipeHandleState");
}

// We will send the server one request.
// Prepare the request
if (fEndServer)
{
      strcpy(szWriteBuffer, SHUT_DOWN_SERVER);
}

 // Send a request and wait for response
 fRc = WriteFile ( hNamedPipe,
                   szWriteBuffer,
                   strlen(szWriteBuffer) + 1,     // Bytes to write
                   &dwBytesWritten,
                   NULL
                 );
if (fRc==FALSE)
      REPORT_ERROR("WriteFile");

// Now we are going to read response from the server.
// In order to demonstrate the usage of ERROR_MORE_DATA
// we will purposely supply a very small read buffer
//
do
{
      dwRc = dwBytesRead = 0;
      fRc = ReadFile ( hNamedPipe,
                       szReadBuffer,
                       4,     // purposely supplying small buffer
                       &dwBytesRead,
                       NULL
                     );
      if (fRc==FALSE)
      {
           dwRc = GetLastError();
           if (dwRc != ERROR_MORE_DATA)
           {
               REPORT_ERROR("ReadFile");
           }
           else
               printf("More data available. <");
      }
      else
           printf("Last part. <");
```

```
            if (dwBytesRead)
            {
                 for (i = 0; i < dwBytesRead; i++)
                      printf("%c", szReadBuffer[i]);
                 printf(">\n");
            }
       } while (dwRc == ERROR_MORE_DATA);

       // We are done. Let us close the pipe
       CloseHandle ( hNamedPipe );
       printf("Ending Client application\n");
       return(0);
}
```

A write operation on a blocking mode pipe blocks until all the bytes written into the pipe have been read. In fact, a write to a blocked mode Named Pipe does not block unless the system buffer is so full it can't accommodate the data buffer passed in by the WriteFile() API. Hence, Windows NT "cheats"—it does not really wait for all the data to be read out. For most applications this is not a critical issue. However, applications that target sensitive or mission-critical operations (such as bank transactions) can use the flag (FILE_FLAG_WRITE_THROUGH) when creating and opening a Named Pipe. This ensures the WriteFile() API forces data to be truly sent, and does not return until the remote end has received the data.

It is important to understand the system buffers associated with Named Pipe. Figure 6-6 shows a schematic of the system buffers associated with each instance of a duplex or bidirectional Named Pipe. As the figure shows, each end of the pipe has two logical buffers—an incoming data buffer and an outgoing data buffer. When a program writes into the pipe, it first goes to the local outgoing buffer unless write-through has been enabled, in which case all writes go directly to the receiver's incoming data buffer. After some time has elapsed, or when the outgoing data buffer becomes full,[3] the system transfers

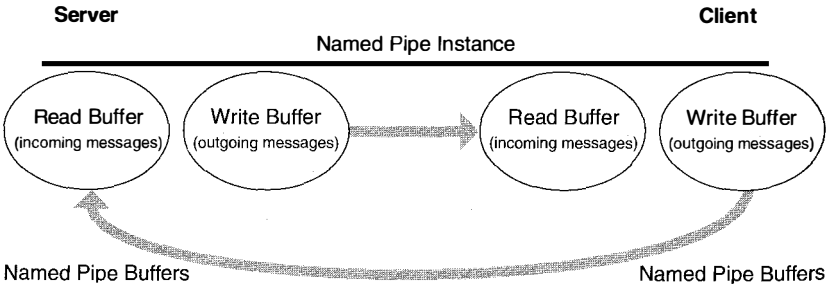


Figure 6-6. *System Buffers Associated with Duplex Named Pipe*

3. The underlying operating system may use a number of optimization algorithms to determine when to transfer data to the remote buffer. Application writers must not rely on any specific implementation but should understand the concepts.

the contents of this outgoing buffer into the receiver's incoming buffer. All read operations on a Named Pipe return the data present in the system incoming data buffer. When the local incoming data buffer does not have sufficient data to satisfy a read, Windows NT can request the remote end to send any data written by the remote writer but cached in the remote end's outgoing buffer. Windows NT uses various caching and polling mechanisms to optimize Named Pipe data transfer.

In a nonblocking Named Pipe, the reader of the pipe might not have read all the data present in its incoming data buffer, and so it may not be empty enough to accommodate all the data sent by the writer's end. When this happens, the underlying system writes out as much as possible into the reader's incoming buffer if the Named Pipe is in byte mode. In message mode, if the reader end does not have enough room for the entire message, no bytes will be transferred from the writer to the reader. Thus, during a write operation on a nonblocking byte mode pipe, it is possible to write out only part of the buffer (that is, nBytesToWrite > *lpBytesWritten).

Also note that in a nonblocking Named Pipe, when a reader supplies a buffer larger than the data present in the incoming data buffer, the ReadFile() API returns with fewer bytes than expected (that is, nBytesToRead > *lpBytesRead). On the other hand, the reader may have supplied a smaller buffer than the data present in the local incoming data buffer. If the pipe is in byte mode (default), the ReadFile() API returns TRUE with the supplied buffer filled with data from the incoming data buffer (that is, *lpBytesRead = nBytesToRead). This case is no different than if the incoming data buffer size was truly the same size as that of the supplied read buffer (lpBuffer).

There are many possible solutions for dealing with mismatched buffer sizes in nonblocked byte mode pipes. The first is for the client and server to agree on the exact size of each data buffer transmitted or received. Alternatively, the client and the server can maintain an application-defined protocol to send and receive variable-length data blocks to each other. For example, the server may always send a fixed-size control data packet that contains the length of the variable length data packet that is to follow. If variable-length data buffers are the norm for the client/server application, then they can use message-mode Named Pipes, because each message has clear boundaries irrespective of the message size. However, note that the overhead for message-mode Named Pipe is about two bytes per message.

Finally, both functions—ReadFile() and WriteFile()—take an OVERLAPPED structure (specified by the lpOverLapped parameter). This structure can be supplied if a Named Pipe has been marked for overlapped file I/O (FILE_FLAG_OVERLAPPED). In such cases, if a read or write cannot be completed immediately, both functions return FALSE right away and the GetLastError() API returns ERROR_IO_PENDING. For non-overlapped Named Pipe, supply NULL in place of the OVERLAPPED structure.

One of the common mistakes made in using the ReadFile() API or the WriteFile() API on Named Pipes is that the application writer forgets to check if the read/write operations failed. This is natural, because read/write operations seldom fail when used on local files. Note that Named Pipes are used as the communication mechanism between two processes across a network where the communication can fail in many ways. Fortunately for the application writer, the underlying operating system takes care of these errors and returns ERROR_BROKEN_PIPE whenever communications fail between the client and the server. When it receives FALSE from the ReadFile()/WriteFile() APIs, and the GetLastError() API returning ERROR_BROKEN_PIPE, the application should close the pipe handle so the user can restart the operation.

6.1.7 "Peeking" into a Named Pipe

The **PeekNamedPipe()** API provides a general mechanism for either the client or the server to determine the size of incoming messages and lets them nondestructively "peek" at the contents of the message. A peek does not remove the message from the pipe, but returns a copy so a reader can take appropriate action, based on the content. The reader must call the ReadFile() API (or another appropriate function) to remove the message from the Named Pipe. Furthermore, the PeekNamedPipe() API never blocks; it returns right away. Its prototype follows:

```
BOOL     PeekNamedPipe (
                        HANDLE hPipe,
                        LPVOID lpvBuffer,
                        DWORD cbBuffer,
                        LPDWORD lpcbRead,
                        LPDWORD lpcbAvail,
                        LPDWORD lpcbMessage);
```

The function takes a handle to the Named Pipe (using the hPipe parameter). The caller can specify a buffer (using the lpvBuffer parameter) and its size (with the cbBuffer parameter) and the PeekNamedPipe() function places a copy of the available data in a caller-supplied buffer. This data can be examined by the caller and its actions prioritized. For example, if a server application has fifty instances of a Named Pipe, it can peek into each incoming message from each client, determine whose request is most urgent, and service the most urgent request first. A caller can also supply NULL as the buffer (using lpvBuffer), in which case a copy of the data is not returned. This option is used to determine the number of bytes available in the pipe without actually getting a copy. The PeekNamedPipe() API returns the number of bytes copied into the caller-supplied buffer (with the lpcbRead parameter).

The PeekNamedPipe() API returns the number of bytes available for reading from the pipe buffer if a non-NULL parameter is specified (by the lpcbAvail parameter). For simple blocked Named Pipe operations, this field is the most useful parameter. If there are no bytes available in the pipe, then the caller need not

call the ReadFile() API (or other read functions). This prevents the read function from blocking on the ReadFile() API indefinitely, a great boon in Windows 3.1 where synchronous (and long lasting) operations block the entire system.

Finally, in message-based pipes, the PeekNamedPipe() API returns the number of bytes left in the message (using the lpcbMessage parameter). This can be used to determine precisely the size of a message and allocate read buffers accordingly. Zero is returned in this field in the case of a byte-mode pipe, because byte mode is a stream of bytes without any message boundaries. This parameter can be very valuable for handing variable-length messages between the client and the server. The following server code fragment shows how to use the PeekNamedPipe() API so it will never block on read. The code is a modification of the full server code shown earlier in this chapter. This code also shows one way to handle variable-length messages between the client and the server by using the PeekNamedPipe() API.

```
////////////////////////////////////////////////////////////////////
// FILE: Server2.c
// (See %BOOK%\nmpcode\server2.c for full code. Some lines omitted
// as they have been explained earlier and are repetitive)
//
// Creating a server which simply creates a blocked Named Pipe and
// uses PeekNmPipe() before calling ReadFile() so as to not block on
// read
//
// Usage: (use it with Client1 or Client2)
// Server2    -n:<pipe name> -m:<max instance>
//
#define DEFAULT_PIPE_NAME "\\\\.\\PIPE\\Server1"

//
// Function to retrieve data from command line
BOOL ParseCommandLine2 ( int     argc,
                         char    **argv,
                         char    **pszPipeName,DWORD *pdwMaxInstance
                       )
{
    char   *ptr;
    argc--;
    argv++;
    // Setup Defaults
    *pszPipeName = DEFAULT_PIPE_NAME;
    *pdwMaxInstance = PIPE_UNLIMITED_INSTANCES;

    while (argc)
    {
          < process command line parameters >
    }
    return TRUE;
}
// internal function
BOOL DataAvailable( HANDLE     hNamedPipe,
                    DWORD      dwPipeMode,
                    LPDWORD    lpdwBytesAvail);
```

```
int main (int argc, char **argv)
{
  char     *pszPipeName;           // Name of Pipe
  DWORD    dwPipeMode=0;           // Pipe mode such as PIPE_READMODE_BYTE
  DWORD    dwOpenMode=0;           // Open mode such as PIPE_ACCESS_DUPLEX
  DWORD    dwMaxInstance=0;        // Number of instances of the pipe
  HANDLE   hNamedPipe = NULL;      // Handle to an instance of pipe
  BOOL     fEndServer = FALSE;     // Flag determines when to end server app.
  BOOL     fRc;                    // Return Code

  char     szReadBuffer [MESSAGE_SIZE];
  DWORD    dwBytesRead, dwBytesPresent;

  // Open duplex blocked byte mode Named Pipe
  dwPipeMode = PIPE_TYPE_BYTE | PIPE_READMODE_BYTE;
  dwOpenMode = PIPE_ACCESS_DUPLEX;

  // Call a helper routine which will parse command line and
  // give us from values
  if (!ParseCommandLine2( argc,
                          argv,
                          &pszPipeName,
                          &dwMaxInstance
                        ))
  {
        printf("Error parsing command line\n"); return (1);
  }

  // Create an instance Named Pipe.
  hNamedPipe = CreateNamedPipe ( pszPipeName,
                                 dwOpenMode,
                                 dwPipeMode,
                                 dwMaxInstance,
                                 OUT_BUFFER_SIZE,
                                 IN_BUFFER_SIZE,
                                 WAIT_NAMED_PIPE_TIMEOUT,
                                 NULL
                               );
  if (hNamedPipe==INVALID_HANDLE_VALUE)
  {
      printf("Error creating Named Pipe: [%ld]\n", GetLastError());
      return (1);
  }

  // We loop until some client sends us a request to end the program
  // Through each loop, we connect to a client and read its request
  // print out the request and disconnect from him. Of course,
  // we check if client sent a request to shut down the server itself.
  printf("Calling ConnectNamedPipe\n");
  while (fEndServer==FALSE)
  {
      // Make a call to start accepting call from clients
      fRc = ConnectNamedPipe ( hNamedPipe, NULL);

      if  (fRc == FALSE)
      {
```

```
                REPORT_ERROR("ConnectNamedPipe")
        }
        // At this point, we have connected with a client
        // Check if there is data to be read from client
        while (TRUE)
        {
                if (DataAvailable(hNamedPipe, dwPipeMode,
                                &dwBytesPresent))
                {
                     // Data is available. ReadFile() will not block
                     fRc = ReadFile(hNamedPipe,
                                  szReadBuffer,  // Data read in here
                                  dwBytesPresent,// Bytes to read
                                  &dwBytesRead,  // Bytes read
                                  NULL         // Overlapped structure
                                 );
                     if (!fRc)
                     {
                          REPORT_ERROR("ReadFile")
                     }
                     // Process the request
                     if (dwBytesRead)
                     {
                         printf("Client Request is : [%s]\n",
                                szReadBuffer);
                         // Check if we need to close down
                         if (stricmp(szReadBuffer,
                             SHUT_DOWN_SERVER)==0)
                             fEndServer = TRUE;
                         break; // break out of while loop
                     }
                     else
                         printf("Zero bytes read from client\n");
                }
                else
                {
                     // In this example, simply sleep. In real
                     // application do some useful work on the server.
                     printf("No Data yet\n");
                     Sleep ( 10);
                }
            }
            // At this point, we have serviced this client.
            // Hence, we will disconnect it.
            fRc = DisconnectNamedPipe( hNamedPipe );
            if (fRc==FALSE)
                    REPORT_ERROR("DisconnectNamedPipe")
    }
    // Need to close the server end of pipe
    CloseHandle( hNamedPipe );
    printf("Ending Server application\n");
    return (0);
}
//  Function - DataAvailable
//  Synopsis - The function determines if there are any bytes avail-
//             able in the pipe. If the pipe is in message mode, it
```

```
//            determines if there are any more bytes in message
// Returns  - TRUE if there is data available in the pipe, else
//            FALSE. If there is data available, it returns that
//            in 'lpdwBytesAvail' parameter
BOOL DataAvailable( HANDLE    hNamedPipe,
                    DWORD     dwPipeMode,
                    LPDWORD   lpdwBytesAvail
                  )
{
    DWORD dwBytesAvail = 0;
    DWORD dwBytesInMessage = 0;
    BOOL  fRc;

    // If this is a non-blocking pipe. Can't call PeekNamedPipe() on it
    if (dwPipeMode & PIPE_NOWAIT)
    {
          *lpdwBytesAvail = IN_BUFFER_SIZE;
          return(TRUE);
    }
    // Set bytes available to zero
    *lpdwBytesAvail = 0;

    // In this simple example, we do not want to look at the
    // contents of the data, if available. Hence, we
    // supply no data buffer to copy data into

    fRc = PeekNamedPipe ( hNamedPipe,
                          NULL,              // No data to be read
                          0,                 // Size of data buffer
                          NULL,              // No data to be read
                          &dwBytesAvail,     // Bytes available in pipe
                          &dwBytesInMessage  // Bytes left in message
                        );
    // Check if error occurred. While, we do not do any
    // specific error checking, PeekNamedPipe() function
    // can return ERROR_BROKEN_PIPE error among other errors.
    // Hence, this function can be modified to verify
    // sanity of Named Pipe connection between client and server
    // without reading into or writing from the pipe.
    if (fRc==FALSE)
    {
          printf("dwBytesAvail dwBytesinMes %ld %ld\n", dwBytesAvail,
                 dwBytesInMessage);
          printf("Error: [%ld] in PeekNamedPipe\n", GetLastError());
          exit(1);
    }
    // Return bytes available in the pipe
    if (dwPipeMode & PIPE_TYPE_MESSAGE)
    {
          *lpdwBytesAvail = dwBytesInMessage;
    }
    else
          *lpdwBytesAvail = dwBytesAvail;
    return TRUE;
}
```

6.1.8 Transaction-Oriented Client/Server Applications

This section looks at the transaction-oriented applications that follow the pattern of the client sending one request (containing the transaction data) to the server, which sends a response back to the client.

In the client/server applications shown above, the complexity of the code increases with the degree of flexibility desired. Experienced developers know that for network applications, it is easy to write five to ten lines of error-handling code for every single network-aware function call. At the same time, client/server applications cause significant network traffic during interprocess communications. Unless it follows some optimization technique, a client/server application can cause excessive network traffic. For example, every read or write operation on the Named Pipe may cause network traffic. It is easy for a client to trigger extra network traffic when it sends multiple requests to the server to complete one transaction (as opposed to a single write followed by a single read). To reduce code complexity, the Named Pipe facility has two function calls for transaction-oriented applications. These functions can also enhance network efficiency because Windows NT optimizes the network traffic for the application. These two functions are the **CallNamedPipe()** API and the **TransactNamedPipe()** API; their prototypes follow. While the CallNamedPipe() API is useful only to Named Pipe clients, the TransactNamedPipe() API can be used by both client and server.

```
BOOL TransactNamedPipe (
     HANDLE      hNamedPipe,
     LPVOID      lpvWriteBuf,
     DWORD       cbWriteBuf,
     LPVOID      lpvReadBuf,
     LPDWORD     lpcbRead,
     LPOVERLAPPED lpo);
BOOL CallNamedPipe (
     LPTSTR      lpszPipeName,
     LPVOID      lpvWriteBuf,
     DWORD       cbWriteBuf,
     LPVOID      lpvReadBuf,
     LPDWORD     lpcbRead,
     DWORD       dwTimeOut);
```

The TransactNamedPipe() function is a single function that accomplishes the task of the WriteFile() API followed by the ReadFile() API on a Named Pipe. The TransactNamedPipe() API takes as parameters one buffer (using the lpvWriteBuf parameter) whose contents are to be written into the pipe, and another buffer (using the lpvReadBuf parameter) in which to place the data that are read from the pipe. This assumes that a proper Named Pipe connection has already been established. Hence, it is useful to the clients who send

requests to the server and then wait to get results from the server. It has two advantages over the WriteFile()-ReadFile() API pair. First, less code needs to be written. Second, Windows NT can optimize the network traffic. Recall from earlier discussions that the default Windows NT redirector communicates with a remote Windows NT server by sending and receiving data packets encapsulated in SMBs. Of course, each SMB packet itself is encapsulated in protocol-dependent packets before transmission over the network. As Figure 6-7 shows, while the WriteFile()-ReadFile() API pair can cause up to four SMBs to be transmitted between the client and the server, the TransactNamedPipe() API can reduce the network transmission by half, cutting down the number of SMBs to be transmitted between client and server. There are some constraints. To use the TransactNamedPipe() API, the pipe must be in message mode. Furthermore, the TransactNamedPipe() API always blocks until the supplied read buffer is filled, even if the pipe is in nonblocking mode.

The TransactNamedPipe() function is suitable for clients that need to invoke multiple transactions on the server during the lifetime of a connection. The CallNamedPipe() API is a good choice when the client needs to connect to a server for a single transaction. The CallNamedPipe() function reduces application code complexity and optimizes network traffic by doing all the work of the functions {CreateFile()/WaitNamedPipe(), TransactNamedPipe(), and CloseHandle()}.

For example, the application running on a bank's automatic teller machine may use the TransactNamedPipe() API, because the ATM services a steady stream of people. But an automated remote weather station may use the CallNamedPipe() API to send data to a server, because it updates the central weather station infrequently.

Client	Network Traffic	Server
1. WriteFile()	SMB Sent =>	ReadFile()
	<=SMB Received	
2. ReadFile()	SMB Sent =>	WriteFile()
	<=SMB Received	
1. TransactNamedPipe()	SMB Sent =>	ReadFile() WriteFile()
	<=SMB Received	

Figure 6-7. *TransactNamedPipe() Compared with WriteFile()-ReadFile() Pair*

Applications that use the TransactNamedPipe() function must supply a handle to the Named Pipe (using the hNamedPipe parameter). Specify the write data buffer (using lpvWriteBuf parameter) and its size (with the cbWriteBuf parameter). This buffer is written into the pipe first. The TransactNamedPipe() API waits to read in the specified number of bytes (specified by the cbReadBuf parameter) into the read data buffer (specified by the lpvReadRead parameter). Note that the TransactNamedPipe() API only works if the pipe is in message mode, otherwise it fails, and the subsequent call to the GetLastError() API returns (ERROR_BAD_PIPE). By design, the TransactNamedPipe() function always blocks, even if the pipe is in nonblocking mode. The function returns TRUE on success and FALSE on error. When the incoming message is larger than the buffer supplied, then the TransactNamedPipe() API returns FALSE and a subsequent call to the GetLastError() API returns (ERROR_MORE_DATA). If the pipe has been marked (at create time and when it is opened) as an overlapped pipe (that is, FILE_FLAGGED_OVERLAPPED), then a pointer can be supplied to an OVERLAPPED structure (using the lpo parameter). In this case, the TransactNamedPipe() API returns immediately with FALSE and the GetLastError() API returns (ERROR_IO_PENDING). Applications can then use events to determine the completion of I/O.

The following code fragment illustrates a typical server and client using the TransactNamedPipe() API. In this example, the server creates one instance of Named Pipe and connects to one client at a time. Once connected, it iterates through the ReadFile()/WriteFile() APIs a given number of times (set to 5 by default). The client calls the CreateFile() API to connect to the server and then uses the TransactNamedPipe() API, instead of the WriteFile()/ReadFile() pair, to send a request to the server and receive a reply. Note that on the client side, the pipe mode is changed from byte to message mode by using the SetNamedPipeHandleState() API.

```
////////////////////////////////////////////////////////////////
// FILE: Server3.c
//
// Creating a server which is used by clients calling
// TransactNamedPipe() or CallNamedPipe()
//
// Usage:
// Server3 -n:<pipe name> -p:<pipe mode> -o:<open mode>
//         -m:<max instance> -r:<r/w loop>
//
// <r/w loop > = A number which determines how many times the
//               server loops doing a Read followed by write.
//
#define SERVER_RESPONSE  "Server Response no: [%ld] to Client: [%ld]"
#define DEFAULT_PIPE_NAME "\\\\.\\PIPE\\Server3"
//
```

```
// Function to retrieve data from command line
BOOL ParseCommandLine3 ( int     argc,
                         char    **argv,
                         char    **pszPipeName,
                         DWORD   *pdwPipeMode,
                         DWORD   *pdwOpenMode,
                         DWORD   *pdwMaxInstance,
                         int     *pnRWLoop
                       )
{
    char *ptr;
    argc--;
    argv++;
    // Setup Defaults
    *pszPipeName     = DEFAULT_PIPE_NAME;
    *pdwPipeMode     = PIPE_TYPE_MESSAGE | PIPE_READMODE_MESSAGE |
                       PIPE_WAIT;
    *pdwOpenMode     = PIPE_ACCESS_DUPLEX;
    *pdwMaxInstance  = PIPE_UNLIMITED_INSTANCES;
    *pnRWLoop        = 5;
    while (argc)
    {
          < process command line parameters >
    }
    return TRUE;
}

int main (int argc, char **argv)
{
    char   *pszPipeName;             // Name of Pipe
    DWORD  dwPipeMode = 0;           // Pipe mode such as PIPE_READMODE_BYTE
    DWORD  dwOpenMode = 0;           // Open mode such as PIPE_ACCESS_DUPLEX
    DWORD  dwMaxInstance = 0;        // Number of instances of the pipe
    HANDLE hNamedPipe = NULL;        // Handle to an instance of pipe
    BOOL   fEndServer = FALSE;       // Flag determines when to end server app.
    BOOL   fRc;                      // Return Code
    BOOL   fClientConnected = FALSE;// Indicates if client has connected

    char     szBuffer [MESSAGE_SIZE];
    DWORD  dwBytesRead, dwBytesWritten;
    int     nRWLoop, i, j;
    // Call a helper routine which will parse command line and
    // give us from values
    if (!ParseCommandLine3 ( argc,
                             argv,
                             &pszPipeName,
                             &dwPipeMode,
                             &dwOpenMode,
                             &dwMaxInstance,
                             &nRWLoop
                           ))
    {
          printf("Error parsing command line\n"); return (1);
    }

    // Create an instance of Named Pipe.
```

```
hNamedPipe = CreateNamedPipe ( pszPipeName,
                               dwOpenMode,
                               dwMaxInstance,
                               OUT_BUFFER_SIZE,
                               IN_BUFFER_SIZE,
                               WAIT_NAMED_PIPE_TIMEOUT,
                               NULL
                             );
if (hNamedPipe==INVALID_HANDLE_VALUE)
{
      printf("Error creating Named Pipe: [%ld]\n", GetLastError());
      return (1);
}
//
// We loop forever servicing clients until shutdown
// message is received
//
i = 0;
while ( fEndServer==FALSE)
{
      printf("Waiting for client to connect\n");
      fRc = ConnectNamedPipe ( hNamedPipe, NULL);

      //  Deal with return code depending upon pipe mode
      //  Remember we are in blocked mode
      if (fRc==FALSE)
           REPORT_ERROR("Wait Mode ConnectNamedPipe");

      // We are connected to the client. We will now
      // loop for 'nRWLoop' times calling ReadFile()
      // WriteFile() pair.
      for (j  =0 ; j< nRWLoop ; j++)
      {
           // Read the request
           if (ReadFile(hNamedPipe,
                        &szBuffer,
                        MESSAGE_SIZE,
                        &dwBytesRead,
                        NULL
                       )==FALSE)
           {
               // Unless the pipe broke, we continue
               if (GetLastError()==ERROR_BROKEN_PIPE)
                   break;  // Proceed to disconnect
               else
                   REPORT_ERROR("ReadFile");
           }

      // Process the request
      if (dwBytesRead)
      {
           printf("\nClient Request is : [%s]\n", szBuffer);
           // Check if we need to close down
           if (stricmp(szBuffer, SHUT_DOWN_SERVER)==0)
               fEndServer = TRUE;
      }
```

```
            // Write back a response
            sprintf(szBuffer, SERVER_RESPONSE, j, i);
            if (WriteFile(hNamedPipe,
                szBuffer,
                strlen(szBuffer)+1,
                &dwBytesWritten,
                NULL
               )==FALSE)
            {
                 // Unless the pipe broke, we continue
                 if (GetLastError()==ERROR_BROKEN_PIPE)
                      break;  // Proceed to disconnect
                 else
                      REPORT_ERROR("WriteFile");
            }
          } // for nRWLoop times

          // At this point, we have serviced this client. Hence, we
          // will disconnect him
          if (fRc = DisconnectNamedPipe( hNamedPipe ) == FALSE)
          {
                 // It is possible that a client has already died or
                 // closed its handle. We want to handle those
                 // cases gracefully
                 if (GetLastError()==ERROR_BROKEN_PIPE)
                     ; // do nothing
                 else
                     REPORT_ERROR("DisconnectNamedPipe");
          }
          i++; // Next Client
     }
     // Need to close the server end of pipe
     CloseHandle( hNamedPipe );

     printf("Ending server application\n");
     return (0);
}
//////////////////////////////////////////////////////////////////
// FILE: Client4.c
// (See %BOOK%\nmpcode\client4.c for full code. Some lines omitted
// as they have been explained earlier and are repetitive)
//
// Creating a client which  calls TransactNamedPipe()
//
// Usage: (use it with Server3)
//
// Client4 -n:<pipe name>  -a:<access mode> -s:<share mode>
//         -f[<End Server>] -r:<r/w loop>
//
// <r/w loop > = A number which determines how many times the
//               client loops doing TransactNamedPipe()

#define DEFAULT_PIPE_NAME "\\\\.\\PIPE\\Server3"
#define CLIENT_REQUEST  "Client request %d"
```

```
// Function to retrieve data from command line
BOOL ParseCommandLineForClient2 ( int    argc,
                                  char   **argv,
                                  char   **pszPipeName,
                                  DWORD  *pdwAccessMode,
                                  DWORD  *pdwShareMode,
                                  int    *pnRWLoop,
                                  BOOL   *pfEndServer
                                )
{
    char   *ptr;
    argc--;
    argv++;
    // Setup Defaults
    *pszPipeName   = DEFAULT_PIPE_NAME;
    *pdwAccessMode = GENERIC_READ|GENERIC_WRITE; // read/write
    *pdwShareMode  = 0;                  // no sharing
    *pfEndServer   = FALSE;              // Don't terminate the server
    *pnRWLoop      = 5;                  // Call TransactNamedPipe() 5 times
    while (argc)
    {
          < process command line parameters >
    }
    return TRUE;
}
int main (int argc, char **argv)
{
    char     *pszPipeName; // Name of Pipe
    DWORD    dwAccessMode; // Access Mode
    DWORD    dwShareMode;  // Share Mode
    HANDLE   hNamedPipe;   // Handle to an instance of pipe
    BOOL     fRc;          // Return Code
    BOOL     fEndServer;   // Determines if client will shut down server
    char     szWriteBuffer [MESSAGE_SIZE],
             szReadBuffer  [MESSAGE_SIZE];
    DWORD    dwBytesRead, dwPipeMode;
    int      nRetries, nRWLoop, i;
    // Call a helper routine which will parse command line and
    // give us from values
    if (!ParseCommandLineForClient2( argc,
                                     argv,
                                     &pszPipeName,
                                     &dwAccessMode,
                                     &dwShareMode,
                                     &nRWLoop,
                                     &fEndServer
                                   ))
    {
              printf("Error parsing command line\n"); return (1);
    }
    // Open an instance Named Pipe. It we get an error ERROR_PIPE_BUSY
    // we call WaitNamedPipe() and then try to open the pipe again
    nRetries = NUMBER_OF_OPEN_RETRIES;
    while (nRetries--)
```

```
{
      hNamedPipe = CreateFile ( pszPipeName,
                                dwAccessMode,
                                dwShareMode,
                                NULL,            // Security Descriptor
                                OPEN_EXISTING,   // Pipe must exist
                                FILE_ATTRIBUTE_NORMAL,
                                NULL             // No Template
                               );
      if (hNamedPipe==INVALID_HANDLE_VALUE)
      {
           switch (GetLastError())
           {

           case ERROR_PIPE_BUSY:    // Server is busy. Let us wait
                printf("Pipe busy. Waiting on pipe\n");
                if (WaitNamedPipe(pszPipeName,
                                  NMPWAIT_USE_DEFAULT_WAIT)==TRUE)
                   continue; // Server is free. Try CreateFile again
                else
                   break;    // Wait timed out or could be any number
                             // of failures such network failure etc.
                             // In this example, we will try CreateFile()
                             // again and exit program on failures. Else
                             // we will wait again.
           default:
                REPORT_ERROR("opening Named Pipe");
           }
      }
      else
           break;  // We have connected so break out of loop
} // while

// Check if we connected
if (hNamedPipe == INVALID_HANDLE_VALUE)
{
      REPORT_ERROR("opening Named Pipe");
}

// By default, pipe is opened in byte mode. Hence, we need
// to set it to message mode
dwPipeMode = PIPE_READMODE_MESSAGE | PIPE_WAIT;
if (SetNamedPipeHandleState (hNamedPipe, &dwPipeMode, NULL,
                             NULL) == FALSE)
{
           REPORT_ERROR("SetNamedPipeHandleState");
}
// We loop nRWLoop times calling the server with TransactNamedPipe()
for (i = 0; i < nRWLoop; i++)
{
      printf("Calling Server\n");
      // We will send the server one request. Prepare the request
      if (fEndServer)
      {
           strcpy(szWriteBuffer, SHUT_DOWN_SERVER);
           nRWLoop = 1;
      }
```

```
        else
             sprintf(szWriteBuffer, CLIENT_REQUEST, i);

        // Send a request and wait for response
        if (TransactNamedPipe(hNamedPipe,
                              szWriteBuffer,          // Request
                              strlen(szWriteBuffer)+1,// Request size
                              szReadBuffer,          // Buffer for Response
                              MESSAGE_SIZE,          // Max. bytes expected
                              &dwBytesRead,           // Actual bytes read
                              NULL                    // No overlap I/O
                             )==FALSE)
        {
             // Unless the pipe broke, we continue
             if (GetLastError()==ERROR_BROKEN_PIPE)
                  break;  // Proceed to disconnect
             else
                  REPORT_ERROR("TransactNamedPipe");
        }
        else if (dwBytesRead)
        {
             printf("Server's Response was :[%s]\n", szReadBuffer);
        }
   } // for nRWLoop times

   // We are done. Let us close the pipe
   CloseHandle ( hNamedPipe );
   printf("Ending Client application\n");
   return(0);
}
```

Client applications that need to communicate only occasionally with the server can use the CallNamedPipe() function, which takes a pipe name as a parameter (using lpszPipeName). Most of the other parameters are the same as those used with the TransactNamedPipe() API. Unlike the TransactNamedPipe() API, the CallNamedPipe() API takes a time-out value (specified in milliseconds with dwTimeOut), after which it returns with an error if an instance of the Named Pipe is not available. This time-out value dictates the time a client waits for the server end of the pipe to become available. It has the same semantics as that of the WaitNamedPipe() API discussed earlier in this chapter. The CallNamedPipe() function opens the pipe with the (GENERIC_READ | GENERIC_WRITE) open flags. Hence, the pipe must have been created at the server end with the pipe open mode of (PIPE_ACCESS_DUPLEX).

The following code fragment shows a client that calls the CallNamedPipe() API to communicate with a remote server (Server3.c, shown earlier). By calling the CallNamedPipe() API, the client (Client5.c) is functionally equivalent to the client shown above (Client4.c). The CallNamedPipe() API connects to the server, performs a transaction, and then closes the pipe. Thus, the new client can operate with the server (Server3.c) shown earlier (with server side read/write loop count set to 1).

```
//////////////////////////////////////////////////////////////////////
// FILE: Client5.c
// (See %BOOK%\nmpcode\client5.c for full code. Some lines omitted
// as they have been explained earlier and would be repetitive)
//
// Creating a client which calls CallNamedPipe()
//
// Usage: (use it with Server3)
//
// Client5 -n:<pipe name>  -a:<access mode> -s:<share mode>
//         -f[<End Server>] -r:<r/w loop>
//
// <r/w loop > = A number which determines how many times the
//               client loops doing CallNamedPipe()
//
#define DEFAULT_PIPE_NAME "\\\\.\\PIPE\\Server3"
#define CLIENT_REQUEST  "Client request %d"
// Function to retrieve data from command line
BOOL ParseCommandLineForClient2 ( int      argc,
                                  char   **argv,
                                  char   **pszPipeName,
                                  DWORD   *pdwAccessMode,
                                  DWORD   *pdwShareMode,
                                  int     *pnRWLoop,
                                  BOOL    *pfEndServer
                                 )
{
    char   *ptr;
    argc--;
    argv++;
    // Setup Defaults
    *pszPipeName    = DEFAULT_PIPE_NAME;
    *pdwAccessMode = GENERIC_READ|GENERIC_WRITE; // read/write
    *pdwShareMode  = 0;                  // no sharing
    *pfEndServer   = FALSE;              // Don't terminate the server
    *pnRWLoop      = 5;                  // Call CallNamedPipe() 5 times
    while (argc)
    {
          < process command line parameters >
    }
    return TRUE;
}
int main (int argc, char **argv)
{
    char    *pszPipeName;  // Name of Pipe
    DWORD    dwAccessMode; // Access Mode
    DWORD    dwShareMode;  // Share Mode
    HANDLE   hNamedPipe;   // Handle to an instance of pipe
    BOOL     fRc;          // Return Code
    BOOL     fEndServer;   //Determines if client will shut down the server

    char     szWriteBuffer [MESSAGE_SIZE],
             szReadBuffer  [MESSAGE_SIZE];
    DWORD    dwBytesRead,  dwPipeMode;
    int      nRWLoop, i;
```

```
    // Call a helper routine which will parse command line and
    // give us from values
    if (!ParseCommandLineForClient2( argc,
                                     argv,
                                     &pszPipeName,
                                     &dwAccessMode,     // we don't use this
                                     &dwShareMode,      // we don't use this
                                     &nRWLoop,
                                     &fEndServer
                                   ))
    {
          printf("Error parsing command line\n"); return (1);
    }
    //
    // Call CallNamedPipe(). We loop nRWLoop times
    //
    for ( i = 0; i < nRWLoop; i++)
    {
          printf("Calling Server\n");
          // We will send the server one request. Prepare the request
          if (fEndServer)
          {
               strcpy(szWriteBuffer, SHUT_DOWN_SERVER);
               nRWLoop = 1;
          }
          else
               sprintf(szWriteBuffer, CLIENT_REQUEST, i);

          // Send a request to server and receive her response
          if (CallNamedPipe( pszPipeName,
                             szWriteBuffer,          // Request
                             strlen(szWriteBuffer)+1,// Request size
                             szReadBuffer,           // Buffer for Response
                             MESSAGE_SIZE,           // Max. bytes expected
                             &dwBytesRead,           // Actual bytes read
                             NMPWAIT_USE_DEFAULT_WAIT
                                                     // Time to wait for Server
                           )==FALSE)
          {
               // if the server is busy, we will simply call it again
             if (GetLastError()==ERROR_PIPE_BUSY)
                 continue;
             else
                 REPORT_ERROR("CallNamedPipe");
          }
          else if (dwBytesRead)
          {
               printf("Server's Response was :[%s]\n", szReadBuffer);
          }
    } // for nRWLoop times

    printf("Ending client application\n");
    return (0);
}
```

6.1.9 Dynamically Getting and Setting Pipe Mode

This section studies how to change the modes of a Named Pipe at run time using certain Named Pipe APIs. These allow the applications to be more flexible in their use of Named Pipe for client/server communication.

Recall from earlier discussions that a Named Pipe has two distinct characteristics or modes, wait mode and read mode. Wait mode dictates whether an I/O call on a pipe blocks until the call completes or executes asynchronously, and read mode defines if a pipe data can be read a byte or message at a time. Two functions—the **GetNamedPipeHandleState()** API and the SetNamedPipeHandleState() API—have been provided to inspect, and change if necessary, the pipe mode (specified by the dwPipeMode parameter in the CreateNamedPipe() function). These functions can be called on both client and server. The most common usage of the SetNamedPipeHandleState() function is to change a pipe from blocking to nonblocking and vice versa; and to change from byte mode to message mode and vice versa. The prototypes of these functions are:

```
BOOL GetNamedPipeHandleState (
     HANDLE hNamedPipe,
     LPDWORD lpdwMode,
     LPDWORD lpdwCurInstance,
     LPDWORD lpcbMaxCollect,
     LPDWORD lpdwCollectDataTimeout,
     LPTSTR lpszUser,
     DWORD cchMaxUser);
BOOL SetNamedPipeHandleState (
     HANDLE hNamedPipe,
     LPDWORD lpdwMode,
     LPDWORD lpcbMaxCollect,
     LPDWORD lpdwCollectDataTimeout);
```

Both functions take a handle to a Named Pipe as the first parameter (hNamedPipe). The handle must be created or opened with (GENERIC_READ) access to use with the GetNamedPipeHandleState() API, and must be created or opened with (GENERIC_WRITE) access to use with the SetNamedPipeHandleState() API. A Named Pipe opened or created with both read and write access can be fully manipulated by both the GetNamedPipeHandleState() API and the SetNamedPipeHandleState() API functions.

The most important information returned by the GetNamedPipeHandleState() API concerns the Named Pipe mode (using lpdwMode)—blocking or nonblocking, and byte or message mode. Hence, when the pipe is in nonblocked mode, (PIPE_NOWAIT) is present in this state variable. The absence of this flag (PIPE_NOWAIT) means that the pipe is in blocking mode. Similarly, the (PIPE_READMODE_MESSAGE) flag is present, which means the pipe is in message mode. A NULL can be passed as a value for the mode parameter (using lpdwMode) if no information is needed.

The caller determines the number of Named Pipes (using the lpdwCurInstances parameter). A server can determine the user name (using the lpszUserName parameter) associated with clients that are connected to an instance of the Named Pipe. When calling the GetNamedPipeHandleState() API on the client, this parameter must be set to NULL. A server that does not want to receive the user name can set the value of the parameter to NULL prior to calling the GetNamedPipeHandleState() API function. The last parameter (cchMaxUser) specifies the size of the buffer (pointed to by lpszUserName) passed by the caller of the GetNamedPipeHandleState() API.

Windows NT employs a data caching mechanism on the client end of the Named Pipe to improve network efficiency. In this mechanism, all writes to the pipe go into the local outgoing buffer associated with the Named Pipe until the buffer is full. When the local outgoing buffer can no longer hold the data to be written into the Named Pipe, the locally cached data buffer is transmitted to the remote end, using the underlying protocols.

This brings up the concept of data buffer threshold value. It is possible that some application only needs to write a small amount of data into the Named Pipe and then waits for a response. Because the local write buffer is not full, the data will never be sent to the remote end if the Named Pipe driver monitors only the data buffer threshold. To assist such Named Pipe applications, the Named Pipe driver maintains a time-out threshold. When the application writes no data into the pipe for longer than the time-out threshold, the local buffer is sent to the remote end whether the buffer is full or not.

The Named Pipe default caching behavior can be either completely turned off or controlled to some extent. A Named Pipe client application that does not want to use the Named Pipe data caching mechanism can turn it off by setting the flag (FILE_FLAG_WRITETHROUGH) when it opens a Named Pipe. Examples of such applications are real-time applications that need to eliminate any time delays in data transfer, and database applications that need to guarantee actual transmission of data to the server. Client applications that want to use data caching, but need to control it, can use the GetNamedPipeHandleState() API and the SetNamedPipeHandleState() API functions to guide the caching mechanism. In this context, the GetNamedPipeHandleState() API function returns the data threshold value (using the lpcbMaxCollect parameter) for locally cached bytes on a client. Because the local pipes do not provide this caching behavior, this parameter must be NULL when calling the GetNamedPipeHandleState() API on a local pipe. This parameter must also be NULL on servers, because no caching is employed on the server end of the Named Pipe. An application can always pass NULL in this parameter when it is not interested in the threshold value. The GetNamedPipeHandleState() API also returns the time threshold value (using the lpdwCollectTimeout parameter) for locally cached bytes on the client.

Restrictions regarding passing NULL are the same as for lpcbMaxCollect. The SetNamedPipeHandleState() API function allows Named Pipe clients to set both buffer and time threshold values (using the lpcbMaxCollect and lpdwCollectDataTimeout parameters, respectively). The restrictions on passing NULL in these two parameters are the same for the GetNamedPipeHandleState() API.

The SetNamedPipeHandleState() API function can be used to change the mode of the pipe from blocking mode to nonblocking mode (PIPE_WAIT or PIPE_NOWAIT) and from the byte read mode to the message read mode (PIPE_READMODE_BYTE or PIPE_READMODE_MESSAGE) (using the lpdwMode parameter). Table 6-2 on page 304 shows a valid combination of pipe modes on Named Pipe clients and servers that applies here as well.

6.1.10 Using Information Available on Named Pipe

The Named Pipe function set includes the GetNamedPipeInfo() function, which can be used to determine additional information on the Named Pipe that is not available through the GetNamedPipeHandleState() API. The GetNamedPipeInfo() prototype follows:

```
BOOL GetNamedPipeInfo(
      HANDLE  hNamedPipe,
      LPDWORD lpdwType,
      LPDWORD lpcbOutBuf,
      LPDWORD lpcbInBuf,
      LPDWORD lpcMaxInstances);
```

The caller must supply a valid handle to the GetNamedPipeInfo() function (using the hNamedPipe parameter) and can supply NULL into the rest of the parameters. The information returned from this function is:

- The pipe type (using the lpdwType parameter). This field is OR-ed with the (PIPE_SERVER_END) flag when the GetNamedPipeInfo() API is called on a server. The bit (PIPE_TYPE_MESSAGE) is also set when the pipe is in a message mode.
- Outgoing data buffer size (using the lpcbOutBuf parameter). The caller can determine the outgoing data buffer size associated with a given pipe.
- Incoming data buffer size (using the lpcbInBuf parameter). The caller can determine the incoming data buffer size associated with a given pipe.
- Maximum instances of the pipe (using lpcMaxInstances). This determines the maximum number of instances of the Named Pipe that can be created by the application.

6.1.11 Using Events and Overlapped I/O

This section looks at how to use Win32 events and overlapped I/O with Named Pipe to develop client/server applications that do not block on Named Pipe I/O functions such as ReadFile(), WriteFile(), and so on. Additionally, overlapped I/O can be used to enhance the I/O throughput of a client/server application using Named Pipes. The section also illustrates the use of asynchronous procedures with Named Pipes in overlapped mode. (Events, overlapped I/O, and asynchronous procedures were discussed in Chapter 2.)

As you know, a Named Pipe is either blocked or nonblocked. While blocking mode Named Pipes are useful for simple applications, asynchronous or nonblocking Named Pipes are essential for designing efficient and sophisticated server applications. One way to create a nonblocking Named Pipe is to create an asynchronous pipe. Another way (recommended for Windows NT-based servers) is to use overlapped I/O with Named Pipes. This section illustrates Named Pipes using overlapped I/O.

Overlapped I/O allows a Win32 event to be associated with each I/O operation (for example, reading data from a pipe). This allows the I/O call to return immediately without blocking. The caller can check the status of the event associated with the I/O call at a later time. When the I/O operation completes, Windows NT puts the event in a signaled state; an application can either wait for the event to complete, or it can check periodically on the status of the event. For example, while a server using simple blocked pipes will block waiting for clients' requests, the server can post an overlapped read request and carry on other tasks, periodically checking on the status of the event associated with the read operation. Once a client sends a request, the event associated with the read operation is set to the signaled state by the Named Pipe driver, and the server takes the appropriate action.

To use a Named Pipe with overlapped I/O, the server sets the (FILE_FLAG_OVERLAPPED) bit in the pipe open mode (dwOpenMode) parameter of the CreateNamedPipe() API. After that, the server application must use the ReadFile(), WriteFile(), ConnectNamedPipe(), and TransactNamedPipe() APIs in overlapped mode. Client applications can use the Named Pipe in overlapped mode by setting the (FILE_FLAG_OVERLAPPED) bit in the file attributes (fdwAttrsAndFlags) parameter of the CreateFile() API. Then they must use the ReadFile(), TransactNamedPipe(), and WriteFile() APIs in overlapped mode. A special structure (OVERLAPPED) is used with each I/O operation on a Named Pipe using overlapped I/O. The prototype is shown here:

```
typedef struct _OVERLAPPED { /* o */
        DWORD Internal;
        DWORD InternalHigh;
        DWORD Offset;
        DWORD OffsetHigh;
        HANDLE hEvent;
} OVERLAPPED;
```

A caller usually associates an event with the overlapped I/O operation by setting the handle to a Win32 event in the hEvent field of the OVERLAPPED structure. The application sets the event to a nonsignaled state before making the overlapped I/O call, then uses wait functions (such as the WaitForSingleObject() API) to detect transition of the event from the nonsignaled to the signaled state. The state transition of the event indicates completion of Named Pipe I/O. Note that when multiple overlapped I/O calls are made on the same Named Pipe during the same time period, each I/O should have its distinct OVERLAPPED structure and (more importantly) its distinct event. When multiple I/O calls on a Named Pipe share an event, the Named Pipe driver will set the event multiple times when each I/O completes. (Otherwise, due to timing problems, the application might not be able to detect each transition.)

It is possible to use the handle of the Named Pipe itself as an event. In this case, the caller puts the handle of the Named Pipe in the hEvent field of the OVERLAPPED structure. When the I/O operation completes, the Named Pipe is put in the signaled state, and an application can wait on the pipe handle to detect completion of I/O. This is a good technique to use for a client awaiting server response. However, it is not recommended in either a client or server application when two or more overlapped I/O operations can be performed on the same Named Pipe simultaneously, because there could be a loss of information when two or more overlapped I/O operations are performed on the same instance of the pipe (within the same application). Typically, only the server side of the Named Pipe deals with multiple instances of a Named Pipe. Thus, using a pipe handle as an event may not be appropriate in the Named Pipe server.

When used with Named Pipe, all fields of the OVERLAPPED structure (except the hEvent field) should be set to zero. Once the OVERLAPPED structure has been prepared, call the ReadFile(), WriteFile(), ConnectNamedPipe(), or TransactNamedPipe() APIs and pass in the pointer to this structure. [The CallNamedPipe() API does not have any provision for handling overlapped I/O]. When making an I/O-bound call on an overlapped mode pipe, all I/O-bound functions return FALSE if NULL is passed in as compared with a valid OVERLAPPED structure. In this case, the GetLastError() API returns (ERROR_INVALID_PARAMETER). After a call to these functions is submitted, they may return TRUE only if the I/O operation completes right away. For example, when calling the ReadFile() API in overlapped mode, the function could return TRUE with the read buffer filled with data if the read could complete without blocking. In all other cases, these functions return FALSE and subsequent GetLastError() API calls return (ERROR_IO_PENDING), indicating normal queuing of the overlapped I/O call. At this point, the caller can continue with other activities. The completion of overlapped I/O can be determined by using one of the following two options:

- Use the WaitForSingleObject() or WaitForMultipleObject() APIs, which block on the event handle passed in the OVERLAPPED structure until it transitions to the signaled state at the completion of I/O.

- Call the GetOverlappedResult() API periodically to determine the status of overlapped I/O. The prototype is shown here:

```
BOOL GetOverlappedResult (
      HANDLE          hFile,
      LPOVERLAPPED    lpo,
      LPDWORD         lpcbTransfer,
      BOOL            fWait);
```

The GetOverlappedResult() API takes the handle to the instance of a pipe (client or server end) and a pointer to the OVERLAPPED structure that has been previously passed in an I/O-bound function, such as the ReadFile() API. It returns TRUE if the I/O function has completed, and FALSE if the I/O operation has not completed. Non-completion of overlapped I/O can be verified by calling the GetLastError() API, which will return (ERROR_IO_INCOMPLETE) to indicate I/O is incomplete. The GetOverlappedResult() API returns the number of bytes (using the lpcbTransfer parameter) transferred at the time of the call. This informs users about the progress of the network I/O. To avoid polling, the fWait parameter can be set to TRUE, which will cause the GetOverlappedResult() API to block until the I/O operation completes (or aborts for some reason).

The following code shows the implementation of a server (Server5.c) that uses overlapped mode Named Pipe and Win32 events to create nonblocking Named Pipes.

```
////////////////////////////////////////////////////////////////////////
// FILE: Server4.c
// (See %BOOK%\nmpcode\server4.c for full code. Some lines omitted
// as they have been explained earlier and are repetitive)
//
// Creating a server which shows usage of an overlapped
// I/O with Named Pipe. It shows usage of GetOverlappedResult()
// API in blocking and non-blocking modes. We also show
// how to simply wait on overlapped events without
// using GetOverlappedResult() API.
//
// Usage:     (This can be run against Client4 or Client5 with RWLoop count
//             set to 1)
// Server4  -n:<pipe name> -p:<pipe mode> -o:<open mode>
//          -m:<max instance>  -r:<r/w loop>
//
#define SERVER_RESPONSE  "Server Response no: [%ld] to Client: [%ld]"

#define DEFAULT_PIPE_NAME "\\\\.\\PIPE\\Server3"
```

```
//
// Function to retrieve data from command line
BOOL ParseCommandLine3 ( int    argc,
                         char   **argv,
                         char   **pszPipeName,
                         DWORD  *pdwPipeMode,
                         DWORD  *pdwOpenMode,
                         DWORD  *pdwMaxInstance,
                         int    *pnRWLoop
                       )
{
    char   *ptr;
    argc--;
    argv++;
    // Setup Defaults
    *pszPipeName = DEFAULT_PIPE_NAME;
    *pdwPipeMode = PIPE_TYPE_MESSAGE | PIPE_READMODE_MESSAGE |
                   PIPE_WAIT;
    *pdwOpenMode = PIPE_ACCESS_DUPLEX;
    *pdwMaxInstance = PIPE_UNLIMITED_INSTANCES;
    *pnRWLoop = 5;

    while (argc)
    {
          < process other command line parameters >
          case 'r': ptr+=2;
                    *pnRWLoop  = atoi(ptr);
                    break;
    }
    return TRUE;
}

// The following function is used to wait on
// overlapped I/O completion. In real applications
// one would perform useful tasks and occasionally
// check the status of I/O operations.
typedef enum
{    enWaitLooping,
     enWaitBlocking,
     enWaitOnEvent
}enWaitType;
int WaitForIOCompletion (
          HANDLE          hNamedPipe,
          LPOVERLAPPED    pOverlap,
          LPDWORD         pdwBytesTransferred,
          enWaitType      enWait)
{
    DWORD dwRc;
    switch ( enWait)
    {
    case enWaitLooping: // Use the GetOverlappedResult() API
          while (GetOverlappedResult(hNamedPipe,
                                     pOverlap,
                                     pdwBytesTransferred,
                                     FALSE    // Don't want to block
                                    ) == FALSE )
```

```
        {
             printf(".");
             Sleep ( 10 );   // sleep for a while
        }
        break;
    case enWaitOnEvent: // Wait on an event
        dwRc = WaitForSingleObject(pOverlap->hEvent,
                                   INFINITE
                                  );
        if (dwRc !=0)
             REPORT_ERROR("WaitForSingleObject");
        break;
    case enWaitBlocking: // Use GetOverlappedResult() API to block
        GetOverlappedResult(hNamedPipe,
                            pOverlap,
                            pdwBytesTransferred,
                            TRUE     // Block
                           );

    }
    // Must set the event to non-signaled state for next I/O
    ResetEvent ( pOverlap->hEvent );
    return 0;
}
int main (int argc, char **argv)
{
    char    *pszPipeName;          // Name of Pipe
    DWORD   dwPipeMode=0;          // Pipe mode such as PIPE_READMODE_BYTE
    DWORD   dwOpenMode=0;          // Open mode such as PIPE_ACCESS_DUPLEX
    DWORD   dwMaxInstance=0;       // Number of instances of the pipe
    HANDLE  hNamedPipe= NULL;      // Handle to an instance of pipe
    BOOL    fEndServer = FALSE;    // Flag determines when to end server app.
    BOOL    fRc;                   // Return Code
    BOOL    fClientConnected=FALSE; // Indicates if client has connected

    char          szBuffer   [MESSAGE_SIZE];
    DWORD         dwBytesRead, dwBytesWritten;
    int           nRWLoop, i, j;
    HANDLE        hEvent;
    OVERLAPPED    oOverlap;

    // Call a helper routine which will parse command line and
    // give us values
    if (!ParseCommandLine3 ( argc,
                             argv,
                             &pszPipeName,
                             &dwPipeMode,
                             &dwOpenMode,
                             &dwMaxInstance,
                             &nRWLoop
                            ))
    {
            printf("Error parsing command line\n");
            return (1);
    }
```

```
// Create an instance of Named Pipe in overlapped Mode
hNamedPipe = CreateNamedPipe ( pszPipeName,
                               dwOpenMode | FILE_FLAG_OVERLAPPED,
                               dwPipeMode,
                               dwMaxInstance,
                               OUT_BUFFER_SIZE,
                               IN_BUFFER_SIZE,
                               WAIT_NAMED_PIPE_TIMEOUT,
                               NULL
                             );
if (hNamedPipe==INVALID_HANDLE_VALUE)
{
          printf("Error creating Named Pipe: [%ld]\n", GetLastError());
          return (1);
}
// Now, we create an event object to associate with this
// instance of Named Pipe. The event will remain in non-signaled
// state until a client does some operation such as connecting
// to the pipe
hEvent  = CreateEvent( NULL, // Security descriptor
                       TRUE, // Manual Reset
                       FALSE, // Start with non-signaled state
                       NULL   // unnamed event
                     );
if (hEvent==NULL)
{
     REPORT_ERROR("CreateEvent");
}
else    // set up the overlapped structure
{
     memset (&oOverlap, 0x00, sizeof(oOverlap));
     oOverlap.hEvent = hEvent;
}
//
// We loop forever, servicing clients until shutdown
// message is received
//
i = 0;
while (fEndServer==FALSE)
{

     if (ConnectNamedPipe (hNamedPipe, &oOverlap) == FALSE)
     {
          // If I/O is pending, we loop occasionally checking I/O
          // status
          if (GetLastError() == ERROR_IO_PENDING)
          {
               printf("Waiting for client to connect\n");
               WaitForIOCompletion (hNamedPipe, &oOverlap,
                                    &dwBytesRead, enWaitLooping);
          }
          else
               REPORT_ERROR("ConnectNamedPipe");
     }

     // We are connected to the client. We will now
     // loop for 'nRWLoop' times calling ReadFile()
```

```
// WriteFile() pair.
for (j  =0 ; j< nRWLoop ; j++)
{
          // Read the request
          if (ReadFile(hNamedPipe,
                       &szBuffer,
                       MESSAGE_SIZE,
                       &dwBytesRead,
                       &oOverlap
                      )==FALSE)
     {
          // We wait on the event waiting for read to complete
          if (GetLastError() == ERROR_IO_PENDING)
          {
               printf("waiting for read to complete\n");
               WaitForIOCompletion (hNamedPipe, &oOverlap,
                                    &dwBytesRead, enWaitOnEvent );
          }
          else     // Unexpected error
               REPORT_ERROR("ReadFile");
     }

     // Process the request
     if (dwBytesRead)
     {
          printf("\nClient Request is : [%s]\n", szBuffer);
          // Check if we need to close down
          if (stricmp(szBuffer, SHUT_DOWN_SERVER)==0)
                     fEndServer = TRUE;
     }
     // Write back a response
     sprintf(szBuffer, SERVER_RESPONSE, j, i);
     if (WriteFile(hNamedPipe,
                   szBuffer,
                   strlen(szBuffer)+1,
                   &dwBytesWritten,
                   NULL
                  )==FALSE)
     {
          // We block using GetOverlappedStatus() waiting for
          // write to complete
          if (GetLastError()==ERROR_IO_PENDING)
          {
               printf("waiting for write to complete\n");
               WaitForIOCompletion (hNamedPipe, &oOverlap,
                                    &dwBytesWritten, enWaitBlocking );
          }
          else
               REPORT_ERROR("WriteFile");
     }
} // for nRWLoop times

// At this point, we have serviced this client. Hence, we
// will disconnect him
if (fRc = DisconnectNamedPipe( hNamedPipe ) == FALSE)
{
     // It is possible that a client has already died or
```

```
                // closed its handle. We want to handle those
                // cases gracefully
                if (GetLastError()==ERROR_BROKEN_PIPE)
                     ; // do nothing
                else
                     REPORT_ERROR("DisconnectNamedPipe");
           }
           i++; // Next Client
      }
     // Need to close the server end of pipe
     CloseHandle( hNamedPipe );
     CloseHandle(hEvent);

     printf("Ending server application\n");
     return (0);
}
```

As you know from Chapter 2, asynchronous procedures can be used in overlapped I/O mode to enhance data I/O throughput. The first prerequisite of using asynchronous procedures is that the Named Pipe must be created in overlapped mode (by the server) or opened in the overlapped mode (by the client), then the ReadFileEx() and the WriteFileEx() APIs can be used for asynchronous read or write operations (also referred to as overlapped completion routines). The prototypes of the functions follow:

```
BOOL WriteFileEx(
     HANDLE                              hFile,
     LPVOID                              lpBuffer,
     DWORD                               nNumberofBytesToWrite,
     LPOVERLAPPED                        lpOverlapped,
     LPOVERLAPPED_COMPLETION_ROUTINE     lpCompletionRoutine);
BOOL ReadFileEx(
     HANDLE                              hFile,
     LPVOID                              lpBuffer,
     DWORD                               nNumberofBytesToRead,
     LPOVERLAPPED                        lpOverlapped,
     LPOVERLAPPED_COMPLETION_ROUTINE     lpCompletionRoutine);
VOID IoCompletionRoutine (
     DWORD                               dwErrorCode,
     DWORD                               dwNumberOfBytesTransferred,
     LPOVERLAPPED                        lpOverlapped);
```

Note that all but the last parameters of the ReadFileEx() and the WriteFileEx() APIs are similar to those of the ReadFile() and the WriteFile() API, respectively. The last parameter (lpCompletionRoutine) allows you to specify the address of an asynchronous procedure that is called when the write or read operation completes. Prior to calling the asynchronous procedure, the Win32 system ensures that the proper data is transferred to the process address space. Hence, when reading data from a pipe, all the data must be transferred from system buffer to caller's buffer before the asynchronous function is called. Similarly, when writing data into a pipe, all of a caller's data must be transferred from the caller's buffer to the Named Pipe's outgoing data buffer, if not to the ultimate recipient's buffer. (Recall that NT may perform optimization and may mark a write "completed" before the data is really written out to the remote recipient buffer.)

The following code fragment demonstrates the use of an asynchronous routine—ReadCompletionRoutine()—in conjunction with the ReadFileEx() API. The functions are used to read data from a Named Pipe that is assumed to have been created (or opened) in overlapped mode. Notice that ReadFileEx() passes in the address of the completion routine. This function is called when the read operation is complete. In this example, after calling ReadFileEx(), wait in an alertable state by calling the WaitForSingleObjectEx() API. When the completion routine has been executed, the WaitForSingleObjectEx() completes.

```
// The following code fragment shows asynchronous read from a Named Pipe
// which has already been opened in overlapped mode. Further assume
// that the function is being called by a thread other than the
// main thread of the application
//
// Asynchronous Procedure - Called by Win32 when read completes
//                          and before WaitForSingleObject in
//                          AsyncRead() completes.
//
VOID ReadCompletionRoutine(DWORD dwErrorCode,
                           DWORD dwNumberOfBytesTransfered,
                           LPOVERLAPPED lpOverlapped)
{
    // If an I/O error occurred, display the error and then exit

    if (dwErrorCode) {
          printf("FATAL I/O Error %ld I/O Context %lx.%lx\n",
                 dwErrorCode, lpOverlapped, lpOverlapped->hEvent);
          ExitProcess(dwErrorCode);
    }
    // Release the overlapped structure allocated in AsyncRead()
    LocalFree(lpOverlapped);
}


//
// AsyncRead() - The function reads a Named Pipe which has been
//               opened in overlapped mode already
//

DWORD AsyncRead(HANDLE hNamedPipe,
                LPVOID lpvBuffer,
                DWORD dwBufferSize,
                DWORD dwBytesRead)
{
    HANDLE hEvent;
    DWORD CompletionStatus;
    LPOVERLAPPED Overlapped;
    BOOL IoOperationStatus;

    // Create an event
    hEvent = CreateEvent (NULL, TRUE, FALSE, NULL)
    if (hEvent == NULL) {
          printf("Error creating event\n");
          return GetLastError();
    }
    // Allocate an overlapped structure
```

```
    Overlapped = LocalAlloc(LMEM_ZEROINIT,
                            sizeof(OVERLAPPED));
  if (!Overlapped) {
    printf("FATAL allocation error\n");
    return GetLastError();
    }
    Overlapped->Offset = 0 ;
    Overlapped->OffsetHigh = 0;
    Overlapped->hEvent = hEvent;

     //  Post the asynchronous read
     IoOperationStatus = ReadFileEx(hNamedPipe,
                                    lpBuffer,
                                    dwBufferSize,
                                    Overlapped,
                                    ReadCompletionRoutine
                                   );
    // Test to see if I/O was queued successfully
    if (!IoOperationStatus) {
         printf("FATAL I/O Error %ld I/O Context %lx.%lx\n",
         GetLastError(), Overlapped, Overlapped->hEvent);
         return GetLastError();
     }
     // Do an alertable wait on the event.
     CompletionStatus = WaitForSingleObjectsEx(
                                        hEvent,
                                        INFINITE,// Time out period
                                        TRUE     // return if I/O completes
                                             );
    // If the wait failed, error out
    if (CompletionStatus == 0xFFFFFFFF) {
         printf("FATAL WAIT ERROR %ld\n", GetLastError());
         return GetLastError();
    }
    // I/O completed successfully
    return (0);
}
```


6.1.12 Using Asynchronous Pipes

This section covers asynchronous Named Pipes and the behavior of the Named Pipe functions in such a pipe. (Note: I recommend that designers writing new Win32 applications use overlapped I/O rather than asynchronous nonblocking pipes.)

The Win32 subsystem provides asynchronous pipes for backward compatibility with the Microsoft LAN manager. However, the Win32 API documentation discourages new applications from using this feature. Asynchronous (nonblocking) pipes are created with the CreateNamedPipe() API with pipe mode set to (PIPE_NOWAIT), or by changing a blocking pipe to nonblocking mode by calling the SetNamedPipeHandleState() API after a pipe has been created. Once a pipe is in nonblocking mode, any I/O-bound call made on the pipe returns immediately without blocking. If the request can be satisfied immediately, then the function will return TRUE; otherwise, it will return FALSE.

The main drawback of asynchronous Named Pipes is that you need to build another layer of infrastructure on top of the basic I/O functions. Such an infrastructure will probably resemble Named Pipe with overlapped I/O, hence the recommendation to use overlapped I/O.

The ConnectNamedPipe() API on an asynchronous pipe returns immediately and puts the pipe in the listening mode, ready to receive calls from clients. If a client request has already come through, then ConnectNamedPipe() returns FALSE with the GetLastError() API returning (ERROR_PIPE_CONNECTED). If the pipe has already been closed by the client, the GetLastError() API returns ERROR_NO_DATA. If the pipe is already in the listening state, the ConnectNamedPipe() API returns FALSE with the GetLastError() API returning (ERROR_PIPE_LISTENING). Finally, if the DisconnectNamedPipe() API has already been called on the pipe, the ConnectNamedPipe() API returns TRUE. It is possible to periodically call the ConnectNamedPipe() API on an asynchronous Named Pipe until it returns FALSE with the GetLastError() API returning (ERROR_PIPE_CONNECTED), indicating that a client has connected.

Once the pipe has been connected, the server can use the ReadFile(), WriteFile(), or TransactNamedPipe() functions to transfer data from or to the client. The WriteFile() API call on an asynchronous pipe will return TRUE if all the data can be written into the pipe's outgoing buffer immediately. If the pipe is in byte mode and the outgoing buffer is full, the WriteFile() API returns TRUE, with lpNumberOfBytesWritten parameter containing the number of bytes actually written into the pipe. In all other cases, the WriteFile() API returns FALSE with the GetLastError() API returning an appropriate error. The ReadFile() API reads off as many bytes that are immediately available without blocking, unless the pipe is in message mode. If it is, read will fail if a complete message cannot be read into the caller's buffer. Hence, the lpNumberOfBytesRead parameter of the ReadFile() API should be checked to determine the number of bytes really read. Note that the TransactNamedPipe() API blocks on write even on a nonblocked pipe.

Given the complexity of read and write operations on asynchronous pipes, it can be difficult to use asynchronous pipes. However, because of the nonblocking nature of the pipe, asynchronous Named Pipes can certainly lead to more responsive programs than can blocking Named Pipes.

6.1.13 Multiple Instance Management

This section discusses the implementation of efficient servers that use Named Pipes with multiple instances.

In multiple instances of the Named Pipe, the server creates a Named Pipe with multiple instances all having the *same name*. Hence, the clients can connect by the same name while they may be connecting to the twelfth or twentieth instance of the pipe. In other words, the clients do not know what

instance of the Named Pipe they are connecting with. However, the server may need to know precisely which client connected to an instance of a pipe so that it can better serve each client. This information is also needed by the other servers that need to perform client authentication before servicing them.

The number of instances of the pipe is specified when calling the CreateNamedPipe() API. Note that while the sample programs have been creating multiple instance Named Pipes, they do not have handles to all of the instances on the server end. In fact, the sample programs only deal with one instance of the pipe on the server end!

The solution is to call the CreateNamedPipe() API one time per instance and keep an array of instance handles. The caller specifies the same parameters every time, also keeping the nMaxInstance parameter identical. Once all instances of the pipe have been created, they can be put in a listening mode by calling the ConnectNamedPipe() API, ready to accept connect calls from clients. Here is the first catch: suppose each pipe instance was created in blocking mode. In this case, the ConnectNamedPipe() call on the first instance of the pipe will block the entire server until a client has connected to the first instance of the pipe. A second client could not connect to a second (or any) instance of the pipe (assuming single-thread server). This defeats the purpose of creating a multi-instance Named Pipe. The problem can be solved in one of the following ways:

Create one thread per Named Pipe instance

In this case, each thread deals with only one instance of the pipe servicing only one client. Once the client has been serviced, the thread can be terminated or it can call the DisconnectNamedPipe() API followed by the ConnectNamedPipe() API, thereby blocking until a new client makes a request to connect. The advantage of this scheme is that there are blocking pipes with each server thread handling a distinct client. However, the disadvantage of this scheme becomes evident if the server needs to serve a large number of clients (say, fifty or one hundred). In this case, a large number of threads could be running simultaneously, which might bog down the system itself due to the (system and application) overhead associated with each thread.

Create asynchronous nonblocking pipes

In this case, the server calls the ConnectNamedPipe() API on each instance, one by one, without waiting for clients to get connected. This model takes less process overhead as compared to the multithread counterpart described above. However, it is less efficient in servicing clients because server processing is serialized.

Create Named Pipes in overlapped I/O mode

Supply a distinct OVERLAPPED structure to every ConnectNamedPipe() API called (once per instance of a Named Pipe).

The following sample server application shows multiple instances handled by the server by using overlapped Named Pipes. The server first calls the CreateNamedPipe() API to create the necessary number of overlapped Named Pipes instances, and puts the handles in an array. Next, it calls the ConnectNamedPipe() API and associates an overlapped structure (each containing a handle to a Win32 manual reset event) with each Named Pipe instance. This puts all the instances in the listening state, ready to accept new client connect requests. As the Named Pipe is in overlapped mode, the ConnectNamedPipe() API does not block. After that, the server loops forever, checking the status of the events associated with Named Pipe by using the WaitForMultipleObject() API. When a client connects to the server, one of the pipe instances is connected to the client, causing the event associated with that instance to be set to the signaled state. This triggers the WaitForMultipleObject() to complete. At this point, the server serves the client by using the ReadFile() and the WriteFile() APIs. Note that because the pipe is in overlapped mode, the ReadFile()/WriteFile() APIs must also be supplied a valid overlapped structure. This example uses the same overlapped structure (and the event) for all I/O operations on a given pipe instance. Finally, the server calls the DisconnectNamedPipe() API to disconnect the Named Pipe, and makes the instance available for the next client by calling the ConnectNamedPipe() again. Set the events to a nonsignaled state before calling the ConnectNamedPipe() API.

```
//////////////////////////////////////////////////////////////////////
// FILE: Server5.c
// (See %BOOK%\nmpcode\server5.c for full code. Some lines omitted
// as they have been explained earlier and are repetitive)
//
// Creating a server which shows usage of a multiple instance
// Named Pipe in conjunction with using overlapped I/O.
//
// This can be run against Client4 or Client5 applications with
// <R/W Loop> count of 1
//
// Usage:
//  Server5 -n:<pipe name> -p:<pipe mode> -o:<open mode>
//          -m:<max instance>
//
//          <Max Instance > = A number which determines how many instances
//                            of this Named Pipe should be created
//
#define CLEANUP_AND_RETURN(string) {\
    printf("Error [%ld] in %s\n", GetLastError(), string);\
    for (j=0; j < dwMaxInstance; j++) \
    {                                          \
          if ( hPipeArray [j] != NULL)     \
               CloseHandle ( hPipeArray [j]);\
          if ( hEventArray [j]!= NULL)\
               CloseHandle ( hEventArray [j]);\
    }\
```

```
    return(1);\
}\

#define DEFAULT_PIPE_NAME "\\\\.\\PIPE\\Server3"
//
// Function to retrieve data from command line
BOOL ParseCommandLine4 ( int     argc,
                         char    **argv,
                         char    **pszPipeName,
                         DWORD   *pdwPipeMode,
                         DWORD   *pdwOpenMode,
                         DWORD   *pdwMaxInstance
                       )
{
   char   *ptr;
    argc--; argv++;
    // Setup Defaults
    *pszPipeName = DEFAULT_PIPE_NAME;
    *pdwPipeMode = PIPE_TYPE_MESSAGE | PIPE_READMODE_MESSAGE |
                   PIPE_WAIT;
    *pdwOpenMode = PIPE_ACCESS_DUPLEX;
    *pdwMaxInstance = PIPE_UNLIMITED_INSTANCES;
    while (argc)
    {
         < parse command line as shown earlier >
    }
    return TRUE;
}
int main (int argc, char **argv)
{
    char    *pszPipeName;      // Name of Pipe
    DWORD    dwMaxInstance;    // Nos. of pipe instances
    DWORD    i,j;              // indices

    BOOL     fEndServer= FALSE;// Should the Server terminate?
    CHAR     szBuffer  [MESSAGE_SIZE];
    DWORD    dwBytesRead, dwBytesWritten,
             dwPipeMode, dwOpenMode,
             dwActivePipe;    // Holds currently active handle
    // Array of Named Pipe instance handles
    HANDLE hPipeArray [MAXIMUM_WAIT_OBJECTS],
    // Array of events
    hEventArray[MAXIMUM_WAIT_OBJECTS];
    // Array of Overlap structures
    OVERLAPPED oOverLap[MAXIMUM_WAIT_OBJECTS];
    DWORD  dwRc;     // return code

    // Call a helper routine which will parse command line and
    // give us values
    if (!ParseCommandLine4 (argc,
                            argv,
                            &pszPipeName,
                            &dwPipeMode,
                            &dwOpenMode,
                            &dwMaxInstance
                           ))
    {
```

```
            printf("Error parsing command line\n"); return (1);
}

if (dwMaxInstance > MAXIMUM_WAIT_OBJECTS)
{
      printf("More that [%ld] instances can not be created in this
                  example\n", MAXIMUM_WAIT_OBJECTS);
      return (1);
}
// Setup the array of handles, events, and overlapped structures
memset ( &hPipeArray,  0x00, dwMaxInstance * sizeof(HANDLE));
memset ( &hEventArray, 0x00, dwMaxInstance * sizeof(HANDLE));
memset ( &oOverLap,    0x00, dwMaxInstance * sizeof(OVERLAPPED));

// We create multiple instances of the pipe an instance of named
// pipe. Each instance of pipe is a blocking pipe
// which is bidirectional and in message mode.
for ( i = 0; i< dwMaxInstance; i++)
{
      hPipeArray [i]= CreateNamedPipe ( pszPipeName,
                                        dwOpenMode |
                                        FILE_FLAG_OVERLAPPED,
                                        dwPipeMode,
                                        dwMaxInstance,
                                        OUT_BUFFER_SIZE,
                                        IN_BUFFER_SIZE,
                                        WAIT_NAMED_PIPE_TIMEOUT,
                                        NULL
                                      );
      if (hPipeArray[i]==INVALID_HANDLE_VALUE)
      {
           printf("Error creating [%d]th Named Pipe\n", i);
           CLEANUP_AND_RETURN("CreateNamedPipe");
      }
      // Now, we create an event object to associate with each
      // Named Pipe. This remains in non-signaled state until
      // a client does some operation such as connecting
      // to the pipe
      hEventArray[i] = CreateEvent( NULL,     // Security descriptor
                                    TRUE,     // Manual Reset
                              FALSE,     // Start with nonsignaled state
                                    NULL     // unnamed event
                                  );
      if (hEventArray[i]==NULL)
      {
           printf("Error creating [%d]th event\n", i);
           CLEANUP_AND_RETURN ("CreateEvent");
      }

      // Put this event in an Overlapped structure before calling
      // ConnectNamedPipe

      oOverLap[i].hEvent = hEventArray [i];

      // Put each instance of pipe in listening mode to receive
      // requests from clients. As Named Pipe is in overlapped mode,
      // ConnectNamedPipe() will return right away setting the
```

```
        // event in non-signaled state
        //
        // However, we are purposefully
        // not checking for FALSE return as some client
        // may have already connected. We will deal with
        // them as during wait event below

        ConnectNamedPipe ( hPipeArray[i], &oOverLap[i]);
    }

    // We loop until some client sends us a request to end the program
    // Through each loop, we connect to a client, read its request
    // and write back a response. We loop until
    // fEndServer is set to TRUE.
    while ( fEndServer==FALSE)
    {
        // At this time, we wait for any pipe to get connected.
        // Then, we operate on that pipe
        printf("Waiting for Clients to connect\n");
        dwRc = WaitForMultipleObjects (dwMaxInstance,// Count of Objects
                        (const HANDLE*)&hEventArray, // Object Handles
                                    FALSE,        // "wait any"
                                    INFINITE
                                   );
        if (dwRc==WAIT_TIMEOUT) // We simply timed out. No Client
            continue;
        if (dwRc==0xFFFFFFFF)   // some error
            CLEANUP_AND_RETURN("WaitForMultipleObjects");


        // We now know which Named Pipe instance was
        // connected to the client. Set the event to non-signaled
        // state again
        dwActivePipe = dwRc;
        ResetEvent (hEventArray [ dwActivePipe ]);
        printf("Pipe instance: [%ld] has been connected\n",
                  dwActivePipe);

        // Read the request. Because we created the pipe
        // in overlapped mode, read must supply
        // an overlapped structure
        if (ReadFile(hPipeArray[ dwActivePipe ],
                     &szBuffer,
                     MESSAGE_SIZE,
                     &dwBytesRead,
                     &oOverLap[dwActivePipe]    // Overlap structure
                    )==FALSE)
        {
            // If the pipe broke, we disconnect
            if (GetLastError()==ERROR_BROKEN_PIPE)
            {
                printf("Pipe disconnected\n");
                go to Disconnect;  // Proceed to disconnect
            }
            else if (GetLastError()==ERROR_IO_PENDING)
            {
                printf("waiting for read to complete\n");
```

```
                if (GetOverlappedResult( hPipeArray[dwActivePipe],
                                         &oOverLap[dwActivePipe],
                                         &dwBytesRead,
                                         TRUE // block
                                        )==FALSE)
                {
                    CLEANUP_AND_RETURN("GetOverlappedResult");
                }

            }
            else     // Unexpected error
                CLEANUP_AND_RETURN("ReadFile");
        }
        // Process the request
        if (dwBytesRead)
        {
            printf("\nClient Request is : [%s]\n", szBuffer);
            // Check if we need to close down
            if (stricmp(szBuffer, SHUT_DOWN_SERVER)==0)
                fEndServer = TRUE;
        }
        // Write back a response. Again, because we opened the
        // pipe in overlapped mode, we have to supply
        // an overlapped structure
        ResetEvent ( hEventArray [ dwActivePipe ] );
        sprintf(szBuffer, SERVER_RESPONSE, j, i);
        if (WriteFile(hPipeArray [ dwActivePipe],
                      szBuffer,
                      strlen(szBuffer)+1,
                      &dwBytesWritten,
                      &oOverLap[dwActivePipe]
                     )==FALSE)
        {
            // If the pipe broke, we will disconnect
            if (GetLastError()==ERROR_BROKEN_PIPE)
            {    // Proceed to disconnect
                printf("Pipe disconnected\n");
                goto Disconnect;  // Proceed to disconnect
            }
            else if (GetLastError()==ERROR_IO_PENDING)
            {
                printf("waiting for write to complete\n");
                if (GetOverlappedResult(hPipeArray[dwActivePipe],
                                        &oOverLap[dwActivePipe],
                                        &dwBytesWritten,
                                        TRUE // block
                                       )==FALSE)
                {
                    CLEANUP_AND_RETURN("GetOverlappedResult");
                }
            }
            else    // Unexpected error
                CLEANUP_AND_RETURN("WriteFile");
        }
Disconnect:
        // At this point, we have serviced this client or have
        // somehow gotten broken pipe. Hence, we
```

```
            // will disconnect him
            DisconnectNamedPipe(hPipeArray[ dwActivePipe ]);
            ResetEvent ( hEventArray [ dwActivePipe ] );

            // Finally, we will put this pipe in listen mode
            ConnectNamedPipe (hPipeArray[ dwActivePipe ],
                              &oOverLap [ dwActivePipe]);
        } // while

        // Need to close all the server end pipe instances
        for (j = 0; j < dwMaxInstance; j++)
        {
            if (hPipeArray [j] != NULL)
                CloseHandle ( hPipeArray [j]);
            if (hEventArray [j]!= NULL)
                CloseHandle (hEventArray [j]);
        }
        printf("Ending Server application\n");

}
```

6.1.14 Security Considerations with Named Pipes

This section discusses ways in which a Named Pipe-based client/server application can be made secure on Windows NT. Windows NT provides means by which a designer can secure access to a Named Pipe instance and control the resources accessed by a Named Pipe server for its client.

The access control to a Named Pipe instance is made possible by tightly integrating Named Pipe operations with the Windows NT security system in which each user, or group of users, can be granted (or denied) specific rights and privileges. (See Chapter 2 for a brief overview of Windows NT security features.) A server application ties the Named Pipe access control with Windows NT security by supplying (with the lpsa parameter) a non-NULL SECURITY_ATTRIBUTES structure with the CreateNamedPipe() API. The SECURITY_ATTRIBUTES structure (shown below) contains a SECURITY_DESCRIPTOR field by which an application can specify the users and groups that are allowed (or denied) access to the instance of Named Pipe created by the CreateNamedPipe() API. The bInheritedHandle field of SECURITY_ATTRIBUTES is used to specify if a child process will inherit (and thus use) the Named Pipe handle.

```
typedef struct _SECURITY_ATTRIBUTES { /* sa */
        DWORD    nLength;
        LPVOID   lpSecurityDescriptor;
        BOOL     bInheritedHandle;
} SECURITY_ATTRIBUTES;
HANDLE CreateNamedPipe ( LPSTR lpszPipeName,
                         DWORD fdwOpenMode,
                         DWORD fdwPipeMode,
                         DWORD nMaxInstances,
                         DWORD cbOutBuf,
                         DWORD cbInBuf,
                         DWORD dwTimeOut,
                         LPSECURITY_ATTRIBUTES lpsa);
```

When a remote client tries to connect to the Named Pipe by calling the CreateFile() API, the Windows NT security subsystem verifies that the user who invoked the client has proper access rights to the Named Pipe. For example, an **accounting database server (ADS)** using Named Pipes might only give read/write access rights to the "accountants" user group so all accountants can access the server. In this example, any user who is not part of the "accountants" user group will not be able to access the Named Pipe on the server and will be denied access to the ADS. Notice that the CreateFile() API takes a SECURITY_ATTRIBUTES parameter as well. When this parameter is not NULL, the security descriptor part of the structure is ignored. In other words, the client end cannot specify any security attributes to override or change Named Pipe security attributes specified by the Named Pipe server.

```
HANDLE CreateFile( LPCTSTR lpszName,
                   DWORD fdwAccess,
                   DWORD fdwShareMode,
                   LPSECURITY_ATTRIBUTES lpsa,
                   DWORD fdwCreate,
                   DWORD fdwCreate,
                   DWORD fdwAttrsAndFlags,
                   HANDLE hTemplateFile);
```

Windows NT also provides means by which a thread in the server can impersonate the security context of the client. This allows the Named Pipe server to limit (or expand) its rights and privileges to those of the user accessing the server. This is useful when the server, after accepting a request from the client, operates on certain resources for the client. For example, the ADS may have full administrative access to all the resources necessary to operate the database on a Windows NTAS. This might lead to a security breach, especially if the ADS accesses highly sensitive data within the database. In this case, a security-conscious designer could grant (or deny) all users very specific rights, and then ask the Named Pipe server to impersonate the user. This way, even if ADS has full access to sensitive information, the thread servicing the client will have the same rights as the user using the Named Pipe server. Thus, an intruder will be denied access to sensitive information even if he can connect with the Named Pipe server.

A Named Pipe server can impersonate the remote user by calling the **ImpersonateNamedPipe()** API, whose prototype is shown below. The thread calling this API will impersonate the client until it calls the **RevertToSelf()** API. Thus, all client data processing should be encapsulated between the ImpersonateNamedPipe() API and the RevertToSelf() API. The pseudocode of a Named Pipe server thread routine shows the use of both the APIs.

```
BOOL ImpersonateNamedPipeClient(HANDLE hNamedPipe);
BOOL RevertToSelf(VOID);
//
// Pseudocode for a Named Pipe Server
// thread impersonating the client
//
```

```
NamedPipe_ThreadRoutine()
{
    HANDLE hNamedPipe;
    BOOL   fRc;
    // Create a Named Pipe
    hNamedPipe = CreateNamedPipe(..);
    // Wait for client to connect
    do
    {
          fRc = ConnectNamedPipe()
          // Now impersonate the client
          if (fRc)
          {
            if (!ImpersonateNamedPipeClient ( hNamedPipe))
            {
                    printf("Impersonation Failure. %d\n" GetLastError());
                    return;
            }
            else
            {
                    // process client request
                    <....>
                    // Now revert back
                    if (!RevertToSelf())
                    {
                         printf("Revert Failure. %d\n", GetLastError());
                         return;
                    }
            }
            fRc = DisconnectNamedPipe ( hNamedPipe );
          }
    } while ( fRc );
}
```

In the context of impersonation, note that the client may also wish to control what the Named Pipe server can do for the client. For example, the president of a company may have complete administrative rights on a server machine, but for security concerns, he may allow the ADS limited rights sufficient to operate the database. This will prevent rogue servers ("Trojan Horses") from misusing the access rights of a given user. For this purpose, the client can specify (by OR-ing) a number of flags in the file attributes parameter (fdwAttrsAndFlags) of the CreateFile() API in conjunction with the security quality of service flag (SECURITY_SQOS_PRESENT). Table 6-3 (on page 318) shows the security flags that a client can specify to control server impersonation.

6.2 ANONYMOUS PIPES

This section presents anonymous pipes, which are a useful IPC mechanism between local processes. Anonymous pipes cannot be used between remote processes. They are presented in this chapter only to complete the discussion of Named Pipes.

Anonymous pipes (unNamed Pipes) are functionally similar to their named counterparts, except that they are designed for communication among processes that exist on the same workstation. Because the anonymous pipes do not have a name, they cannot be uniquely identified or located. Hence, anonymous pipes cannot be used between processes resident on two separate workstations. Once a process has created an anonymous pipe, it is difficult for another process on the same machine to get a handle to this pipe. Anonymous pipes are usually used between a parent process and its child process(es), because a parent process can pass on a handle to the anonymous pipe to its child process(es) at process creation time [see the **CreateProcess()** Win32 API].

The mechanisms used for the anonymous pipe are similar to those in a Named Pipe, albeit much simpler. A pipe is first created by the server end with the **CreatePipe()** API. The server end can immediately begin using the pipe through the ReadFile() and the WriteFile() APIs. The pipe can be closed by calling the CloseHandle() API. The client end must somehow get the handle to the pipe since it cannot use the CreateFile() API to open the client end of the pipe. After it receives the handle, it can use the ReadFile() and WriteFile() APIs to send data to, and receive data from, the server end. Both server and client end can use the PeekNamedPipe() API on an anonymous pipe. The rest of the Named Pipe functions cannot be used with anonymous pipes.

6.2.1 Creating an Anonymous Pipe

The CreatePipe() API creates an anonymous pipe. Its prototype follows:

```
BOOL CreatePipe(
     PHANDLE phRead,
     PHANDLE phWrite,
     LPSECURITY_ATTRIBUTES lpsa,
     DWORD    cbPipe);
```

On successful completion of the function, indicated by a return value of TRUE, the CreatePipe() API returns two handles to the pipe—one for reading from the pipe (using the phRead parameter) and another for writing into the pipe (using the phWrite parameter). This is unlike Named Pipe, which returns only one handle. [UNIX users should find this familiar. Please see the following section on comparison with UNIX **pipe()** for details.]

Specify NULL for SECURITY_ATTRIBUTES (using the lpsa parameter) if you plan to use an anonymous pipe within the same process, for example, between the main thread and some worker thread. However, be sure to set the fInheritedHandle in the SECURITY_ATTRIBUTES parameter to TRUE if the child process must inherit the pipe handle.

Finally, the pipe creator can suggest a buffer size that the system should set aside for the anonymous pipe. Alternatively, it can leave it at zero, in which case a default buffer size is used by the system.

6.2.2 Reading/Writing Anonymous Pipe

Use the ReadFile() API to read from the pipe and the WriteFile() API to write into the pipe. By default, the pipe behaves like a blocking and byte-mode Named Pipe. A read will block until it is satisfied and similarly, writes into the pipe will block if insufficient space is available in the anonymous pipe to accommodate all the data to be written. Hence, ReadFile()/WriteFile() API calls on the pipe must be synchronized between the server and the client.

The following example demonstrates the use of anonymous pipes between a parent process and its child processes. The anonymous pipe and the child processes are created so child processes can inherit the pipe read/write handles. Only the write handle is passed from the parent process (server) to the child processes (client) as a command line parameter. (The *value* of an inherited handle to an object remains the same between a parent process and its children.)

```
//////////////////////////////////////////////////////////////////////
// FILE: parent.c

#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <memory.h>
#define MESSAGE_SIZE (80L)
int cdecl main (int argc, char **argv)
{
    DWORD                dwRc, dwBytesRead;
    int                  iChildren,     i;
    SECURITY_ATTRIBUTES  sa;
    STARTUPINFO          siInfo;
    PROCESS_INFORMATION  piChild;
    HANDLE               hPipeRead;
    HANDLE               hPipeWrite;
    // Go past program name
    argc--;
    argv++;
    if (argc < 1)
          return (1);
    // Determine the number of child processes to spawn
    iChildren = atoi( *argv );
    printf("Nos. children of this process: [%d]\n", iChildren);
    // Set up security attribute so that pipe is inherited
    memset (&sa, 0x00, sizeof(SECURITY_ATTRIBUTES) );
    sa.nLength = sizeof(SECURITY_ATTRIBUTES);
    sa.bInheritHandle = TRUE;
    // Create an anonymous pipe
    if (CreatePipe ( &hPipeRead,      // Handle for reading from pipe
                     &hPipeWrite,     // Handle for writing into pipe
                     &sa,             // Security attribute
                     MESSAGE_SIZE     // Pipe buffer size
                   ) == FALSE)
    {
          printf("Error [%d] creating pipe\n", GetLastError());
          return (1);
```

```
}
// We send each client its sequence number and
// a pipe write handle who can write
// back to us.
for ( i = 0; i < iChildren; i++)
{
      // Create a command line
      TCHAR wszCommandLine [ MAX_PATH ];
      swprintf( wszCommandLine,
                L"Child %d %d",
                i,
                hPipeWrite
              );
      // Create a child process
      memset( &siInfo, 0x00,sizeof ( STARTUPINFO));
      siInfo.cb = sizeof ( STARTUPINFO);
      siInfo.lpTitle = L"Child Process";
      if ( CreateProcess (NULL,              // Image file name
                          wszCommandLine,    // Command line
                          &sa,               // process sec. descrp.
                          &sa,               // thread  sec. descrp.
                          TRUE,              // Inherit all handles
                          CREATE_NEW_CONSOLE,// Additional create attr.
                          NULL,              // environment variable
                          NULL,              // current directory
                          &siInfo,           // startup info
                          &piChild           // Process Information
                         )== FALSE)
      {
           printf("Error: [%d] creating a child process\n",
                GetLastError());
           break;
      }
      else
           printf("Child Process Handle [%lx]\n", piChild.hProcess);
      // Read the message sent from client using the Read Pipe Handle
      if (ReadFile ( hPipeRead,
                     &wszCommandLine,
                     MESSAGE_SIZE,
                     &dwBytesRead,
                     NULL
                   ) == FALSE)
      {
           printf("Error: [%d] Read from Pipe\n", GetLastError());
           break;
      }
      else if (dwBytesRead)
      {
           wprintf(L"Client: [%d] sent: [%s]\n", i, wszCommandLine);
      }
      // We now wait for child process to complete
      do
      {
           if (GetExitCodeProcess (piChild.hProcess,  // Child Process
                                   &dwRc
                                  )== FALSE)
```

```
                {
                    printf("Error: [%d] GetExitCodeProcess\n",
                                GetLastError());
                    break;
                }
                // Give up the time slice
                if (dwRc == STILL_ACTIVE)
                    Sleep (1);
            } while ( dwRc == STILL_ACTIVE );
            // Close handles
            CloseHandle ( piChild.hProcess ) ;
        }    // For all children
        // Cleanup
        CloseHandle ( hPipeRead  );
        CloseHandle ( hPipeWrite );

        printf("Parent process ending\n");
        return (0);
}
////////////////////////////////////////////////////////////////////
// FILE: child.c
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <memory.h>

#define MESSAGE_SIZE (80L)
int cdecl main (int argc, char **argv)
{
    DWORD dwRc, dwBytesWritten;
    HANDLE hPipeWrite;
    TCHAR wszCommandLine [ MAX_PATH ];

    // Go past program name
    argc--;
    argv++;
    // Print out the number of this child
    swprintf( wszCommandLine,
              L"I am Child Process No: [%d]",
              atoi(*argv)
            );
    wprintf(L"%s\n", wszCommandLine);
    // Determine the pipe handle with which we can write to Parent
    argc--;
    argv++;
    hPipeWrite = (HANDLE) atol(*argv);

    // Write the message to server
    if (WriteFile ( hPipeWrite,
                    &wszCommandLine,
                    MESSAGE_SIZE,
                    &dwBytesWritten,
                    NULL
                  )== FALSE)
    {
          printf("Error: [%d] Write into Pipe\n", GetLastError());
```

```
    else if (dwBytesWritten)
    {
          printf("Client wrote [%d]\n", dwBytesWritten);
    }

    printf("Child process ending\n");
    return (0);

}
```

In the next example, we demonstrate that both the server and the client can use the same read/write handles returned via the CreatePipe() API. Hence, for creating a duplex communication between a server and a client, one does not need to create two instances of an anonymous pipe. Of course, when the client process is not a child process, or is not a child process that inherits parent (server) handles, one must devise some IPC mechanism to pass the handle from the server to the client. Additionally, the server should use the **DuplicateHandle()** API function to make a copy of the anonymous pipe read/write handle that it can pass onto the client process.

```
//////////////////////////////////////////////////////////////////
// FILE: parent2.c (see anonpipe\parent2.c for full code)
//    Here we only present the differences between the parent.c and
//    parent2.c so as to focus on the point that both parent and
//    children use the same read/write handle.
//
int cdecl main (int argc, char **argv)
{

    HANDLE hPipeRead;
    HANDLE hPipeWrite;

    < Create a pipe to get the read and write handles>

    // We send each client its sequence number and
    // a pipe write handle that can write
    // back to us.
    for ( i = 0; i < iChildren; i++)
    {

          // Create a command line
          TCHAR wszCommandLine [ MAX_PATH ];
          swprintf( wszCommandLine,
                    L"Child %d %d %d",
                    i,
                    hPipeWrite,
                    hPipeRead
                   );

          <Create a process with this command line parameter >

          // Send a message to the client
          if (WriteFile(hPipeWrite,
                        &wszCommandLine,
```

```
                        MESSAGE_SIZE,
                        &dwBytesRead,
                        NULL
                       )== FALSE)
            {
                printf("Error: [%d] Write into Pipe\n", GetLastError());
                break;
            }

            // Read the message sent from client
            if (ReadFile ( hPipeRead,
                        &wszCommandLine,
                        MESSAGE_SIZE,
                        &dwBytesRead,
                        NULL
                       )== FALSE)
            {
                printf("Error: [%d] Read from Pipe\n", GetLastError());
                break;
            }
            else if (dwBytesRead)
            {
                wprintf(L"Client: [%d] sent: [%s]\n", i, wszCommandLine);
            }
            < Wait for all processes to terminate >
            return (0);
}
////////////////////////////////////////////////////////////////
// FILE: child2.c (see anonpipe\child2.c for full code)
//    Here we only present the differences between the child.c and
//    child2.c so as to focus on the point that both parent and
//    children use the same read/write handle.
//
int cdecl main (int argc, char **argv)
{
    int    iChildNo;
    TCHAR wszCommandLine [MESSAGE_SIZE];
    HANDLE hPipeWrite,hPipeRead;
    // Go past program name
    argc--;
    argv++;

    // Determine the sequence number of this child
    iChildNo = atoi(*argv);

    // Determine the pipe handle with which we can write to Parent
    argc--;
    argv++;
    hPipeWrite = (HANDLE) atol(*argv);

    // Determine the pipe handle with which we can read from Parent
    argc--;
    argv++;
    hPipeRead = (HANDLE) atol(*argv);

     // We first read a message sent by the Parent
```

```
    if (ReadFile ( hPipeRead, &wszCommandLine,MESSAGE_SIZE,
                   &dwBytesRead,
                   NULL
                 )== FALSE)
    {
          printf("Error: [%d] Read from Pipe\n", GetLastError());
          return (1);
    }
    else if (dwBytesRead)
          wprintf(L"Server sent: [%s] \n", wszCommandLine);

    // Next, we send a message to the Parent
    swprintf( wszCommandLine, L"I am Child Process No: [%d]",iChildNo );
    if (WriteFile ( hPipeWrite,&wszCommandLine,MESSAGE_SIZE,
                   &dwBytesWritten,
                   NULL
                 )== FALSE)
          printf("Error: [%d] Write into Pipe\n", GetLastError());
    else if (dwBytesWritten)
          printf("Client wrote [%d]\n", dwBytesWritten);
    return (0);
}
```

6.3 WIN32 SERVICE USING NAMED PIPES

This section illustrates how Named Pipes can be used by a Win32 service.

Recall that the Echo Server Win32 service was introduced in Chapter 2. Chapter 4 featured the Echo Server using RPC to communicate with its clients. This section shows the Echo Server using Named Pipes by expanding the skeletal service code shown in Chapter 2. The code for the Echo Server follows. Notice that the Echo Server has two parts, the code fragments necessary for the Echo Server to function as a Win32 service, and the code fragments used to create a Named Pipe server that will accept calls from the Named Pipe's clients. Because the first part was explained in Chapter 2, only the second part of the code fragment is studied here.

First of all, the listener thread creates multiple instances of Named Pipe in overlapped mode by calling the CreateNamedPipe() API; then, it calls the ConnectNamedPipe() API to make the Named Pipe instances ready to receive client connect requests. Notice that because the pipe is in overlapped mode, the ConnectNamedPipe() API does not block; one overlapped I/O event is associated with each instance of the Named Pipe. This thread will loop, waiting on the overlapped I/O events. When a client calls the CreateFile() API, a given instance of Named Pipe is connected, and the corresponding event will transition to the signaled state. At this point, the listener thread will put client details (an index to the global Named Pipe array indicating the connected pipe instance) in the processing queue and continue to process new client requests. When the ConnectNamedPipe() API succeeds on the Echo Server, the client successfully completes the CreateFile() API and sends a string to the server by calling the WriteFile() API. The client Echo request is processed by one of the

worker threads on the server. When a new client connect request is put in the processing queue (by the listener thread), the next available worker thread will pick up the request and issue the ReadFile() API on the Named Pipe instance (which was connected with the client in the listener thread). Next, it will create an echo reply and write back the string to the client by calling the WriteFile() API. Then it disconnects the client by calling the DisconnectNamedPipe() API. At this point, the worker thread releases the Named Pipe handle and waits for the next client request. Note that because the EchoServer is a Win32 service, a security descriptor must be associated with each Named Pipe created by the server so that all clients can communicate with the server.

The code for the client (EchoC.c) calls the Named Pipe-based Echo Server. It calls the CallNamedPipe() API to connect and communicate with the server.

```
////////////////////////////////////////////////////////////////////
// FILE: EchoS.c
//    See %BOOK%\echosrv\namepipe directory for compiling details
//
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <process.h>
#include "service.h"   // See chapter 2 for detail
#include "printcmd.h"  // See chapter 2 for detail


#define DEFAULT_PIPE_NAME    L"\\\\.\\PIPE\\EchoServer"
#define MAX_INSTANCES  5
#define WAIT_NAMED_PIPE_TIMEOUT (2 * 60 * 1000)    // 2 Minutes
#define OUT_BUFFER_SIZE (80L)
#define IN_BUFFER_SIZE  (80L)
#define MESSAGE_SIZE    (80L)
#define SERVER_RESPONSE "Echo: %s by NMP Thread: %ld Pipe %ld"

// The data shared between threads
typedef struct _PipeData
{
    DWORD dwPipeIndex; // index into global pipe handle array
} PIPE_DATA, *PPIPE_DATA;

// Create a pool of PIPE_DATA shared between Listen thread
// and worker threads
INITIALISE_POOL(MAX_INSTANCES, sizeof(PIPE_DATA))

// Global Data
HANDLE     hPipeArray [MAX_INSTANCES]; // Array of Named Pipe instance handles
HANDLE     hEventArray[MAX_INSTANCES]; // Array of events
OVERLAPPED oLapArray  [MAX_INSTANCES]; // Array of overlapped structures
BOOL       fStopService = FALSE;

// Internal functions
```

```
void ListenThread(VOID * pF);
void WorkerThread(VOID *pF);
void ShutDown(void);
DWORD InitializeNamedPipe();

#define CLEANUP_AND_RETURN(string) {\
                    PrintError1(string, GetLastError());\
                    for (j=0; j < MAX_INSTANCES; j++) \
                    {     \
                         if ( hPipeArray [j] != NULL)     \
                              CloseHandle ( hPipeArray [j]);\
                         if ( hEventArray[j] != NULL)\
                              CloseHandle ( hEventArray[j]);\
                    }\
        return(1);\
        }\

BOOL InitEchoServer()
{
     DWORD dwRc;
     BOOL  bRet = TRUE;
     // First initialize the Client Request Pool
     dwRc = InitializeQueue();
     if (dwRc == 0)
     {
          // Get the Named Pipe Service Started
          dwRc = InitializeNamedPipe ( );

          // Finally Create the worker thread pool
          dwRc = InitializeThreads (WorkerThread);
          if (dwRc)
               bRet = FALSE;
    }
    return(bRet);
}
//
// Function: CreateSecurityDesc()
// Purpose:
// Because the EchoServer runs as a service, it is logged on by the system
// as a special user (LocalSystem). Thus, a Named Pipe created by
// the server is not accessible to the normally logged on user
// unless the service allows them by specifying a proper
// security descriptor. In the function below, we create
// a NULL Discretionary Access Control List (DACL) which,
// when associated with the Named Pipe, allows all users
// to access the pipe.
//
SID_IDENTIFIER_AUTHORITY    NtAuthority = SECURITY_NT_AUTHORITY;
SID_IDENTIFIER_AUTHORITY    LocalAuthority = SECURITY_LOCAL_SID_AUTHORITY;
PSID    SystemSid = NULL;
DWORD CreateSecurityDesc(PSECURITY_DESCRIPTOR * ppsdPort)
{
    PSECURITY_DESCRIPTOR    psdPort;
    BOOL Status;

    *ppsdPort = NULL;
    // Allocate and initialize a security descriptor
```

```
    psdPort = malloc(sizeof(SECURITY_DESCRIPTOR));
    if (!psdPort)
        return(ERROR_OUTOFMEMORY);

    if (!InitializeSecurityDescriptor(psdPort,
                                      SECURITY_DESCRIPTOR_REVISION))
    {
        return(GetLastError());
    }
    //
    // Okay, now we have the basic structure ready.  First, we add an
    // owner of Named Pipe to the descriptor
    if (!SystemSid)
    {
        Status = AllocateAndInitializeSid(
            &NtAuthority,
            1, SECURITY_LOCAL_SYSTEM_RID, // "LocalSystem" is owner
            0, 0, 0, 0, 0, 0, 0,
            &SystemSid);
        if (!Status)
        {
            free(psdPort);
            return(GetLastError());
        }
    }
    if (!SetSecurityDescriptorOwner(psdPort, SystemSid, FALSE))
    {
        free(psdPort);
        return(GetLastError());
    }

    //
    // Set the primary group information (LocalSystem)
    //
    if (!SetSecurityDescriptorGroup(psdPort, SystemSid, FALSE))
    {
        free(psdPort);
        return(GetLastError());
    }
    //
    // Set a NULL Dacl, which is supposed to indicate all access granted
    // as opposed to an empty ACL which is no access
    //
    if (!SetSecurityDescriptorDacl(psdPort, TRUE, NULL, TRUE))
    {
        free(psdPort);
        return(GetLastError());
    }
    *ppsdPort = psdPort;
    return(0);
}
DWORD InitializeNamedPipe()
{
    DWORD dwRc;
    int   i,j;
    PSECURITY_DESCRIPTOR pSD;
```

```
SECURITY_ATTRIBUTES  secAttr;
// Initialize PIPE_DATA Resource Pool
InitializePool();
// Initialize the array of handles and overlapped structures
memset ( &hPipeArray,  0x00, sizeof(hPipeArray));
memset ( &hEventArray, 0x00, sizeof(hEventArray));
memset ( &oLapArray,   0x00, sizeof(oLapArray));

// Create a security descriptor to associate with Named Pipe
if (dwRc = CreateSecurityDesc(&pSD))
{
      PrintErrorl("CreateSecurityDesc", dwRc);
      return (dwRc);
}
else
{
      secAttr.nLength = sizeof(secAttr);
      secAttr.lpSecurityDescriptor = pSD;
}
// Create MAX_INSTANCE pipes and put them in listen mode
for (i = 0; i< MAX_INSTANCES; i++)
{
      // create overlapped mode Named Pipe instances
      hPipeArray [i]= CreateNamedPipe ( DEFAULT_PIPE_NAME,
                                        PIPE_ACCESS_DUPLEX|
                                        FILE_FLAG_OVERLAPPED,
                                        PIPE_TYPE_MESSAGE |
                                        PIPE_READMODE_MESSAGE |
                                        PIPE_WAIT,
                                        MAX_INSTANCES,
                                        OUT_BUFFER_SIZE,
                                        IN_BUFFER_SIZE,
                                        WAIT_NAMED_PIPE_TIMEOUT,
                                        // Security Attribute
                                        &secAttr
                                      );
      if (hPipeArray[i]==INVALID_HANDLE_VALUE)
      {
           PrintErrorl("Error creating [%d]th Named Pipe\n", i);
           CLEANUP_AND_RETURN("CreateNamedPipe");
      }
      // Now, we create an event object to associate with each
      // Named Pipe. This remains in non-signaled state until
      // a client does some operation such as connecting
      // to the pipe
      hEventArray [i]  = CreateEvent( NULL,  // Security descriptor
                                      TRUE,  // Manual Reset
                                      FALSE, // Start with
                                             // nonsignaled state
                                      NULL   // unnamed event
                                    );
      if (hEventArray[i]==NULL)
      {
           PrintErrorl("Error creating [%d]th event\n", i);
           CLEANUP_AND_RETURN ("CreateEvent");
      }
```

```
            // Put each instance of pipe in listening mode to receive
            // requests from clients. As Named Pipe is in overlapped mode,
            // ConnectNamedPipe() will return right away setting the
            // event in non-signaled state
            //
            // However, we are purposefully
            // not checking for FALSE return as some client
            // may have already connected. We will deal with
            // them as during wait for event below


            oLapArray [i].hEvent = hEventArray [ i ];
            ConnectNamedPipe ( hPipeArray[i], &oLapArray[i]);
        }

        // Now begin the Listen thread
        _beginthread ( ListenThread,
                       4098,
                       0
                     );
        free (pSD); // release the security descriptor
        return(0);
}
//
// In this thread, the server listens to incoming client
// request. Once the request arrives, this thread will
// put the incoming request in the request pool for
// worker threads which, in turn, will service the client
//

// Define Listen state transitions
typedef enum
{
    enCancelListen,
    enWaitOnListen,
} enListenState;

void ListenThread(VOID * pF)
{
    DWORD dwRc = 0, dwEventIndex, dwActivePipe;
    enListenState   ListenState;
    PPIPE_DATA      pPipeData = NULL;
    int i;
    // Create an event by which synchronize with Service Thread
    dwEventIndex = GetThreadSyncEvent ();
    if (dwEventIndex == -1)
    {
          PrintError("Could not create Thread Sync. Event\n");
          return;
    }
    //
    // Loop forever listening to incoming clients
    //
    ListenState = enWaitOnListen;
    do
    {
```

```
// First check if service needs to be stopped
if (fStopService)
{
     ListenState = enCancelListen;
}
//
// State driven Asynchronous Listen
//
switch(ListenState)
{
case enCancelListen:
     // Close the pipes and events
     for (i=0; i < MAX_INSTANCES; i++)
     {
          if (hPipeArray [i] != NULL)
              CloseHandle (hPipeArray [i]);
          if (hEventArray[i] != NULL)
              CloseHandle (hEventArray[i]);
     }
     // Signal end of this thread
     SetThreadSyncEvent (dwEventIndex);
     dwRc = 0;
     break;

case enWaitOnListen: //
     dwRc = WaitForMultipleObjects(
                    MAX_INSTANCES,               // nos. events
                    (const HANDLE*)&hEventArray, // Object Handles
                    FALSE,                       // "wait any"
                    INFINITE
                    );
     if (dwRc==WAIT_TIMEOUT) // We simply timed out. No client
          continue;
     if (dwRc==0xFFFFFFFF)   // some error
     {
          ListenState = enCancelListen;
          PrintErrorl("WaitForMultipleObject", dwRc);
          continue;
     }

     // We now know which Named Pipe instance was
     // connected to the client. Set the event to non-signaled
     // state again
     dwActivePipe = dwRc;

     ResetEvent (hEventArray [ dwActivePipe ]);

     PrintInfol("Pipe instance: [%ld] has been connected\n",
                       dwActivePipe);

     if (GetPoolResource (&pPipeData))
     {
          pPipeData->dwPipeIndex = dwActivePipe;
          // put this in the Client Queue for Worker threads
          if ((dwRc = QueueRequest(pPipeData,
                                   sizeof(PIPE_DATA)))!=0)
```

```
                    {
                        // Free the resource and break the session
                        PrintError1("Canceling Session", dwRc);
                        FreePoolResource ( pPipeData );
                        DisconnectNamedPipe ( hPipeArray [ dwActivePipe] );
                    }
                }
                // Wait for other clients
                ListenState = enWaitOnListen;
                dwRc = WAIT_TIMEOUT;

            } // end of switch

        } while (dwRc == WAIT_TIMEOUT);

        // Close Thread sync. event
        CloseThreadSyncEvent ( dwEventIndex );
        _endthread();
    }
    //
    // The worker threads wait to get requests from the
    //
    //
    // Define Worker Thread state transitions
    typedef enum
    {
        enWaitOnConnect = 0,     // Wait for client to connect
        enReceiveRequest,        // Receive client request (async)
        enSendReply,             // Send a response (synchronously)
        enHangupSession          // End session with client
    } enWorkerState;

    void WorkerThread(VOID *pF)
    {
        DWORD dwRc;
        ThreadData *pTData;

        enWorkerState WorkerState = enWaitOnConnect;
        PPIPE_DATA    pPipeData = NULL;
        DWORD         dwDataSize, dwActivePipe,
                      dwBytesRead, dwBytesWritten;
        HANDLE        hReceiveEvent = NULL;
        char          szReadBuffer [ MESSAGE_SIZE],
                      szWriteBuffer[ MESSAGE_SIZE];
        OVERLAPPED    oOverlap;

        // Set the thread status
        pTData = (ThreadData *)pF;
        if (pTData == NULL)
        {
              PrintError("Null Thread Data\n");
              return;
        }
        //
        // Create a special event which we will wait on after posting
        // an asynchronous receive
        //
```

```
hReceiveEvent= CreateEvent(NULL, TRUE, FALSE, NULL);
if (hReceiveEvent == NULL)
{
      PrintWin32Error("CreateEvent");
      return;
}
do
{
      // First check if service needs to be stopped
      if (fStopService)
      {
           WorkerState = enHangupSession;
      }
      switch (WorkerState)
      {
      case enWaitOnConnect:// Wait for Listen Thread to put data on
                           // queue
           dwRc = GetNextRequest (&pPipeData, &dwDataSize);
           if (dwRc ==0)
           {    // Verify the data integrity
                if ((pPipeData == NULL) ||
                   (dwDataSize < sizeof(PIPE_DATA)))
               {
                     dwRc = 0;
                     PrintError("Data Corruption\n");
               }
               else
               {    // We have successfully gotten a new client request
                     dwActivePipe = pPipeData->dwPipeIndex;
                     WorkerState = enReceiveRequest;
                     dwRc = WAIT_TIMEOUT;
               }
           }
           else if (dwRc != WAIT_TIMEOUT)
                PrintError1("GetNextRequest", dwRc);
           break;
      case enReceiveRequest:
           // Read the request. Because we created the pipe
           // in overlapped mode, read must supply
           // an overlapped structure.
           ResetEvent ( hReceiveEvent );
           memset (&oOverlap, 0x00, sizeof(oOverlap));
           oOverlap.hEvent = hReceiveEvent;
           if (ReadFile(hPipeArray[ dwActivePipe ],
                        &szReadBuffer,
                        MESSAGE_SIZE,
                        &dwBytesRead,
                        &oOverlap     // Overlap structure
                       )==FALSE)
           {
                // If the pipe broke, we disconnect
                if (GetLastError()==ERROR_BROKEN_PIPE)
                {
                     PrintInfo("Pipe disconnected\n");
                     WorkerState = enHangupSession;
                     dwRc = WAIT_TIMEOUT;
                }
```

```
                    else if (GetLastError()==ERROR_IO_PENDING)
                    {
                       PrintInfo("waiting for read to complete\n");
                       if (GetOverlappedResult( hPipeArray[dwActivePipe],
                                                &oOverlap,
                                                &dwBytesRead,
                                                TRUE // block
                                               )==FALSE)
                         {
                              PrintError1("GetOverlappedResult",
                                          GetLastError());
                              WorkerState=enHangupSession;
                              continue;
                         }
                    }
                    else    // Unexpected error
                    {
                         PrintError1("ReadFile", GetLastError());
                         WorkerState=enHangupSession;
                         continue;
                    }
               }
               // Process the request
               if (dwBytesRead)
               {
                    PrintInfoString("\nClient Request is : [%s]\n",
                                    szReadBuffer);
                    WorkerState = enSendReply;
               }
               break;
          case enSendReply:
               ResetEvent ( hReceiveEvent );
               memset (&oOverlap, 0x00, sizeof(oOverlap));
               oOverlap.hEvent = hReceiveEvent;
               sprintf(szWriteBuffer, SERVER_RESPONSE, szReadBuffer,
                         pTData->dwEventIndex, dwActivePipe);
               if (WriteFile(hPipeArray [ dwActivePipe],
                             szWriteBuffer,
                             strlen(szWriteBuffer)+1,
                             &dwBytesWritten,
                             &oOverlap
                            )==FALSE)
               {
                    // If the pipe broke, we will disconnect
                    if (GetLastError()==ERROR_BROKEN_PIPE)
                    {    // Proceed to disconnect
                         PrintInfo("Pipe disconnected\n");
                         WorkerState = enHangupSession;
                         dwRc = WAIT_TIMEOUT;
                    }
                    else if (GetLastError()==ERROR_IO_PENDING)
                    {
                       PrintInfo("waiting for write to complete\n");
                       if (GetOverlappedResult(   hPipeArray[dwActivePipe],
                                                  &oOverlap,
                                                  &dwBytesWritten,
```

```
                                        TRUE // block
                                    )==FALSE)
                    {
                        PrintError1("GetOverlappedResult",
                                        GetLastError());
                        WorkerState=enHangupSession;
                        continue;
                    }
                    else
                    {
                        WorkerState = enHangupSession;
                        dwRc = WAIT_TIMEOUT;
                    }
                }
                else       // Unexpected error
                {
                    PrintError1("WriteFile", GetLastError());
                    WorkerState = enHangupSession;
                    continue;
                }
            }
            break;
        case enHangupSession: // Disconnect with Client

            if (pPipeData)
            {
                // Allow next client to connect to the pipe
                DisconnectNamedPipe (hPipeArray [dwActivePipe ]);
                ConnectNamedPipe    (hPipeArray [dwActivePipe ],
                                     &oLapArray [dwActivePipe ]
                                    );
                ResetEvent (hEventArray[dwActivePipe]);

                // Release the buffer
                FreePoolResource ( pPipeData );
                pPipeData = NULL;
            }
            // Get ready for next client
            WorkerState = enWaitOnConnect;
            dwRc = WAIT_TIMEOUT;

            if (fStopService)
            {
                // Notify the service thread that we can be closed
                SetThreadSyncEvent (pTData->dwEventIndex);
                dwRc = 0;
            }
            break;
        }
    } while (dwRc == WAIT_TIMEOUT);

    // Close Thread sync. event
    CloseThreadSyncEvent ( pTData->dwEventIndex );
    _endthread();
}
BOOL PauseEchoServer()
```

```
{
    PrintInfo("Pausing the Named Pipe Service is not supported\n");
    return(TRUE);
}
BOOL ResumeEchoServer()
{
    PrintInfo("Resuming the Namped Pipe Service is not supported\n");
    return(TRUE);
}

BOOL StopEchoServer()
{
    // Signal all threads to quit
    fStopService = TRUE;
    WaitForAllThreadsToTerminate();


    return(TRUE);
}
////////////////////////////////////////////////////////////
// FILE: EchoC.c
//
//
#include <header.h>
#pragma hdrstop

#define DEFAULT_PIPE_NAME    L"\\\\.\\PIPE\\EchoServer"
#define MESSAGE_SIZE    (80L)
#define MAX_RETRIES    (5)
void Usage(char *ptr)
{
    printf("%s -n <pipe name> -s <echo string>\n", ptr);
    exit(1);
}
int _CRTAPI1 main(int argc, char **argv)
{
    LPSTR    pszString    = "Ping from Client";
    TCHAR    wszPipeName [MAX_PATH];
    LPTSTR   pszPipeName = DEFAULT_PIPE_NAME;
    char     szReadBuffer [ MESSAGE_SIZE ];
    DWORD    dwBytesRead;
    int      i, nRetries;

    // allow the user to supply EchoServer Named Pipe name
    // or provide a new string override settings with command line
    // switches
    for (i = 1; i < argc; i++)
    {
          if ((*argv[i] == '-') || (*argv[i] == '/'))
          {
               switch (tolower(*(argv[i]+1)))
               {
               case 'n':    // pipe name
                    ++i;
                    mbstowcs ( wszPipeName, argv[i], strlen(argv[i])+1);
```

```
                pszPipeName = &wszPipeName[0];
                break;
            case 's':
                pszString = argv[++i];
                break;
            case 'h':
            case '?':
            default:
                Usage(argv[0]);
            }
        }
        else
            Usage(argv[0]);
    }
    // Send a request to EchoServer and receive response
    nRetries = MAX_RETRIES;
    while (nRetries--)
    {
        if (CallNamedPipe( pszPipeName,
                           pszString,    // Request
                           strlen(pszString)+1,    // Request size
                           szReadBuffer,    // Buffer for Response
                           MESSAGE_SIZE,    // Max. bytes expected
                           &dwBytesRead,    // Actual bytes read
                NMPWAIT_USE_DEFAULT_WAIT  // Time to wait for Server
                      )==FALSE)
        {
            // if the server is busy, we will simply call it again
            if (GetLastError()==ERROR_PIPE_BUSY)
                continue;
            else
            {
                printf("CallNamedPipe Error: %ld", GetLastError());
                break;
            }
        }
        else if (dwBytesRead)
        {
            printf("Server's Response was :[%s]\n", szReadBuffer);
            break;
        }
    }
    return (0);
}  // end main()
```

6.4 NAMED PIPES IN WINDOWS

This section discusses using Named Pipes from Windows 3.x applications. The concepts and general use of Named Pipes in Windows are the same as in Windows NT. One remarkable difference is that the Named Pipe servers are not supported on Windows 3.x workstations, so Windows applications can only be Named Pipe clients. Table 6-5 lists the Windows APIs corresponding to the Win32 APIs discussed earlier in this chapter.

Table 6-5. Windows Named Pipe APIs

WIN32 API	*Windows API*
The following APIs work in all versions of Windows.	
CreateFile()	OpenFile()
ReadFile()	_lread()
WriteFile(	_lwrite()
CloseHandle()	_lclose()
The following APIs are available in Microsoft Windows for Workgroups only.	
PeekNamedPipe()	DosPeekNmPipe()
WaitNamedPipe()	DosWaitNmPipe()
CallNamedPipe()	DosCallNmPipe()
TransactNamedPipe()	DosTransactNmPipe()
GetNamedPipeHandleState()	DosQNmPHandState()
SetNamedPipeHandleState()	DosSetNmPipeHandState()
GetNamedPipeInfo()	DosQNmPipeInfo()
Special Asychronous APIs available in Windows for Workgroups.	
ReadFile() with overlapped I/O completion routine	DosReadAsyncNmPipe()
WriteFile() with overlapped I/O completion routine	DosWriteAsyncNmPipe()

The following code fragment shows the **NamedPipeTransaction()** function that a Windows client calls to complete a transaction. The function opens a blocking Named Pipe and completes one transaction (one write followed by a read) by using the **OpenFile()**, **_lwrite()**, and **_lread()** APIs. The Named Pipe is closed by calling the **_lclose()** API.

```
#define REPORT_ERROR_AND_RETURN(szMsg) {\
    MessageBox(NULL, szMsg, "Named Pipe Error", MB_OK);\
    if (hPipe != -1) _lclose (hPipe);\
    return(-1);\
}
int NamedPipeTransaction( LPSTR  pszPipeName,
                          LPVOID lpSendBUffer,
                          USHORT usSendBufferLen,
                          LPVOID lpRecvBuffer,
                          USHORT usRecvBufferLen
                        )
{
    // open the Named Pipe
    OFSTRUCT of;
    int   hPipe = -1,
          nBytesWritten,
          nBytesRead;
```

```
    hPipe = OpenFile (pszPipeName, &of, OF_PIPE);
    if (hPipe == -1)
    {
                REPORT_ERROR_AND_RETURN("Opening the Pipe")
    }
    // Send request to the server
    nBytesWritten = -lwrite (hPipe, lpSendBuffer, usSendBuffferLen);
    if (nBytesWritten == -1 )
    {
               REPORT_ERROR_AND_RETURN( "Writing into pipe")
    }
    // Wait for a reply from the server
    nBytesRead = _lread (hPipe, lpRecvBuffer, usRecvBufferLen);
    if (nBytesRead == -1 )
    {
               REPORT_ERROR_AND_RETURN("Reading from pipe")
    }
    // Close the Named Pipe
    _lclose ( hPipe );
    return(0);
}
```

The code fragment can have problems due to the blocking nature of the Named Pipe. Recall that Windows has a nonpreemptive scheduling environment. Thus, any blocking function causes the application to become nonresponsive. This leads to a "hung" Windows system, because the user cannot switch tasks until the blocked application yields or completes the blocked function call. To avoid this problem, Windows clients can use the **DosPeekNmPipe()** API before attempting read or write operations on the Named Pipe. The prototype of the DosPeekNmPipe() API follows. The DosPeekNmPipe() API returns the state of the pipe (using the pfsState parameter). A client should verify that the Named Pipe is in a connected state (NP_CONNECTED) before performing I/O on the pipe. Additionally, the DosPeekNmPipe() API returns a AVAILDATA structure for the Named Pipe (using the pAvail parameter). On successful completion of the DosPeekNmPipe() API, the AVAILDATA structure contains the number of bytes present in the Named Pipe, which allows clients to avoid calling the _lread() API on an empty pipe (and thus blocking). Finally, the caller can also supply a buffer (using the lpBuffer parameter) into which data (if any) from the pipe is copied without actually removing the data from the Named Pipe.


```
// Named Pipe states returned by DosPeekNmPipe() in pfsState
#define NP_DISCONNECTED    1 // after pipe creation or Disconnect
#define NP_LISTENING       2 // after DosNmPipeConnect
#define NP_CONNECTED       3 // after client open
#define NP_CLOSING         4 // after client or server close

// DosPeekNMPipe() Available record returned in pAvail
typedef struct _AVAILDATA  {
```

```
    unsigned short  cbpipe;     // bytes left in the pipe
    unsigned short  cbmessage;  // bytes left in current message
} AVAILDATA, far* PAVAILDATA;
unsigned short
DosPeekNmPipe(HPIPE                  hp,         // Named pipe handle
              unsigned char far *    lpBuffer,   // Peeked data buffer
              unsigned short         cbBuf,      // Peeked buffer size
              unsigned short far *   pcbRead,    // Bytes read into buffer
              PAVAILDATA             pAvail,     // Availability record
              unsigned short far *   pfsState);  // Named pipe state
//
// pseudocode for using DosPeekNmPipe() with _lread()
//
USHORT       usState, usRet;
HANDLE       hPipe;
char         szBuffer [ MAX_SIZE ];
AVAILDATA    Avail;
// Check if there is any data in the pipe before calling _lread()
usRet = DosPeekNmPipe ( hPipe, NULL,  0, NULL,
                        &Avail,&usState);
if ((usRet ==0) && (usState == NP_CONNECTED) && (Avail.cbPipe))
{
    nBytesRead = _lread(  hPipe, szBuffer, min(Avail.cbPipe, MAX_SIZE));
}
```


As Table 6-5 shows, a designer has an array of Named Pipe APIs corresponding to the Win32 Named Pipe API. Preceding sections have explained the concepts and the general use of Named Pipe APIs, such as TransactNamedPipe() or SetNamedPipeHandleState(). The prototypes of the corresponding Windows Named Pipe APIs can be found in the WFW software development kit. Note that, given the nonpreemptive nature of Windows, a designer should consider the Named Pipe asynchronous read and write APIs—**DosReadAsyncNmPipe()** and **DosWriteAsyncNmPipe()**—when building applications directed for WFW workstations. The prototypes of these two APIs are shown below. They follow the same concept as that of Win32 overlapped I/O completion routines. The caller supplies a callback function with these APIs which should be properly exported by using the **MakeProcInstance()** API.

```
// Callback Function prototype
void FAR PASCAL  AsyncCallback(unsigned char far *pbBuffer);

unsigned short DosReadAsyncNmPipe (
HPIPE hHandle,                      // identifies a Named Pipe
API_CALLBACK pfnCallBack,           // called when a read operation ends
unsigned short far *pusErrCode,     // receives an error value for read
unsigned char far *pbBuffer,        // buffer to receive data
unsigned short cbBuffer,            // size of buffer, in bytes
unsigned short far *pcbBytesRead    // receives size of data read, in bytes
);
unsigned short DosWriteAsyncNmPipe (
HPIPE hHandle,                        // identifies a Named Pipe
API_CALLBACK pfnCallBack,             // called when a write operation ends
```

```
    unsigned short far *pusErrCode,     // receives an error value for write
    unsigned char far *pbBuffer,        // buffer to send
    unsigned short cbBuffer,            // size of buffer, in bytes
    unsigned short far *pcbBytesWrite   // receives bytes written
);
```

When the asynchronous read or write I/O completes, the Named Pipe driver calls the asynchronous procedure with a pointer to the buffer originally supplied by the caller to the DosReadAsyncNmPipe() or the DosWriteAsyncNmPipe() API (using the pbBuffer parameter). When the callback procedure is called, the Named Pipe I/O has completed (or aborted). Within the callback function, an application can set global variables and check the error code status (pusErrCode), which gets set before the asynchronous procedure is called. However, applications should not return from the callback function with any registers modified. Additionally, the callback procedure should not call any MS-DOS system functions, BIOS functions, and any C function that can call an MS-DOS function. Use the **PostMessage()** API within the asynchronous procedure to post a completion message to the main window. This allows the application to complete processing the results of I/O at a later time, and so quickly ends the asynchronous procedure.

6.5 COMPARISON WITH UNIX PIPES

This section compares Windows NT (or Win32) pipes with UNIX pipes. The focus is on Berkeley UNIX 4.3 BSD and AT&T System V for these comparisons. The information about UNIX pipes is derived from [Stevens 90]. This section is presented for those who are familiar with UNIX systems and are curious about similarities between UNIX and Windows NT pipes.

Table 6-6 presents a comparison of UNIX pipes functions with Windows NT pipes. The most noticeable distinction (with the exception of STREAM pipes) is that Windows NT pipes are bidirectional. For the UNIX pipe client and server to both send and receive data between each other, they must have two pipes created between them, but a single Windows NT pipe can be used by both client and server to send and receive data. Like Windows NT pipes, UNIX STREAM-based pipes are bidirectional.

The next most obvious difference between Windows NT pipes and UNIX pipes is that Windows NT pipes operate over any transport protocol that may have been loaded on the workstation; UNIX pipes are generally dependent on TCP/IP. Furthermore, Windows NT allows applications using pipes to specify the data buffer size to set aside for this instance of pipe. As discussed earlier, this allows the operating system to be more responsive and efficient. The caller can also ask Windows NT to choose a default size if there is no need for optimization or if data packets are close to the buffer size set aside by Windows NT.

Table 6-6. *Comparison of UNIX Pipes with Windows NT (Win32) Pipes*

Attributes	*UNIX pipe*	*UNIX named pipe (FIFO)**
Call used to create server end	pipe()	mknod()
Buffer specified by caller††	no	no
Unidirectional	yes	yes
Bidirectional	no	no
Local client	yes	yes
Typically used between	parent and child processes	any two processes
Remote client	no	no
Transport protocol dependent	n/a	n/a
Number of descriptors or handles per pipe	2	2
Type of descriptor or handle for read/write	separate descriptor	separate descriptor
Stream I/O support	yes	yes
Message or record I/O support	no	no
Access control support‡	no	yes

* Available in AT&T System V and not in 4.3 BSD.

** Available only on AT&T System V as part of STREAMS I/O system.

† Available only on AT&T System V as part of STREAMS I/O system.

6.6 SUMMARY

Named Pipes are application-level communication constructs that can be used to build IPC infrastructure in Windows and Windows NT. The Named Pipe consists of a server end and a client end. The server ends must be created before a client can connect to it. Once a connection has been established between the server and the client (that is, once Named Pipe is connected), both server and client can use normal read and write I/O functions on the Named Pipe to transmit data back and forth.

*UNIX STREAM pipe***	*UNIX named STREAM pipe*†	*Windows NT anonymous pipe*	*Windows NT named pipe*
socketpair()	socketpair() & bind()	CreatePipe()	CreateNamedPipe()
no	no	yes	yes
no	no	no	no
yes	yes	yes	yes
yes	yes	yes	yes
any two processes	any two processes	parent and child processes	any two processes
no	yes	no	yes
TCP/IP	TCP/IP	not applicable	independent of transport protocol
2	2	2	1
either descriptor	either descriptor	separate handles	same handle
yes	yes	yes	yes
yes	yes	no	yes
yes	yes	yes	yes

†† In order for pipes to work, there must be some data buffers set aside in the system kernel. Some operating systems allow the calling program to specify the buffer size for a pipe while others always set aside a set size of buffer per instance of pipe.

‡ Access control can be specified directly when creating such an entity. Not including operations that can be performed by suitable ioctl() or fcntl() calls.

This chapter presented the use of Named Pipes ranging from simple client/server applications to sophisticated Win32 services (Echo Server). It discussed the Win32 Named Pipe APIs and the Windows for Workgroups Named Pipe APIs. To complete the discussion of pipes in Windows NT, it provided a brief analysis of anonymous pipes. Finally, it presented a comparison between Windows NT Named Pipes and UNIX pipes.

Chapter Seven

USING MAILSLOT IN WINDOWS NT

The Mailslot API allows users to send unidirectional data to a specific computer or a set of computers. The Named Pipe and Windows Socket interfaces provide a guaranteed two-way communication channel; Mailslot, in contrast, provides a best-effort one-way communication channel. That is, it uses broadcast datagrams to provide unreliable message transfer service.

In the rest of the chapter, we will illustrate use of the Mailslot programming interfaces by Windows NT applications. We will then discuss use of Mailslot interfaces from Windows for Workgroups workstations. We will end the chapter with a Win32 Service (Echo Server) which uses the Mailslot interface to communicate with clients.

We have seen that each network layer provides a service to its upper layer, and it makes this service public. The service can be reliable or unreliable. The layer above an unreliable service provider may implement its own error-handling algorithms to provide reliable service to its users. Datagram- or connectionless-based services generally provide unreliable delivery of data packets. (There are also reliable datagram protocols.) Thus, there is no guarantee the data packet will arrive at the destination, and even if it does, it may not arrive in the order in which it was sent. For example, packet 1 can arrive later than packet 3. Additionally, the receiver may receive duplicate packets. In connection-oriented or virtual-circuit-based communication, the client end may use broadcast and routing techniques to contact the server end. However, once the connection has been established, delivery is guaranteed, and the packets arrive in order.

Datagram packets sent to a specific recipient are referred to as directed datagrams. Although these datagrams are received by only one (target) node on the network, the packet traverses the entire network. Broadcast datagrams, on the other hand, are directed toward every listening node on the network and are processed by every node. They cause inefficiency because all nodes on the network must process each broadcast packet. So when do we use directed datagrams versus broadcast datagram? Directed datagrams are useful when the recipient's address is known and the client/server system uses a datagram-based protocol to communicate. Broadcast datagrams are useful

when the node wishes to send the same information (for example, stock quotes) to every node on the network or when trying to locate a remote entity. In fact, most connection-oriented protocol implementations (such as those of TCP, NetBIOS, and SPX) first use broadcast datagrams to locate the remote node before starting virtual-circuit-based communication. Furthermore, when data transmission is limited to a LAN and the data transmitted is not crucial, using a broadcast-based communication methodology may lead to low programming overhead.

In the corporate world today, offices are connected via WANs more often than not. Each LAN is connected to one or more LANs via routers or bridges, which can filter data packets going from one LAN to a neighbor. Traditionally, these routers are set up so they forward only directed packets, but not broadcast packets, from one LAN to another. In this way, all computers in a given LAN can use broadcast to, say, locate each other without flooding the complete WAN and thereby usurping scarce company data bandwidth. This efficient solution to avoid flooding the complete WAN with broadcast packets, however, can be sabotaged by poorly designed network-aware applications any time the client/server application uses broadcast-based communication and the client and the server are running in separate LANs. This is illustrated in Figure 7-1, which shows two LANs---one with all engineers and another with all accountants—connected via a router, and a database server on the accountants' LAN needs to be accessed from the engineers' LAN. If the database server and its clients use broadcast-based communication, the router must be configured to allow all broadcast packets to cross LAN boundaries. Thus, the company's WAN will be flooded with broadcast packets from all corners of the company. This is not a problem when occasional broadcast traffic is generated by clients attempting to locate a remote server. However, the WAN will be seriously affected when the server periodically (say, every five minutes) broadcasts information (for example, stock quotes).

Designers of CSSs should be aware of the topography of the target installation sites. Typically applications that are used only within a department will take place within the LAN. However, applications that will be used by several departments will be spread over WANs, so the choice of communication methodology is important.

An alternative to using broadcasts is multicasting, whereby a community of nodes joins a logical group. In this case, a datagram sent by a member of the group is received by all the other members but not by every active node on the network. Thus, the sender can transmit the information to all the interested nodes in a single "send" but only the members of the multicast group receive the multicast packet. Typically, this operation is implemented in the network layer that passes to the application only multi-cast messages directed toward a locally registered multicast name (for example, "StockQuotes"). As a result, CPU overhead on noninterested nodes will be reduced because applications need not process all incoming broadcast traffic. This form of datagram

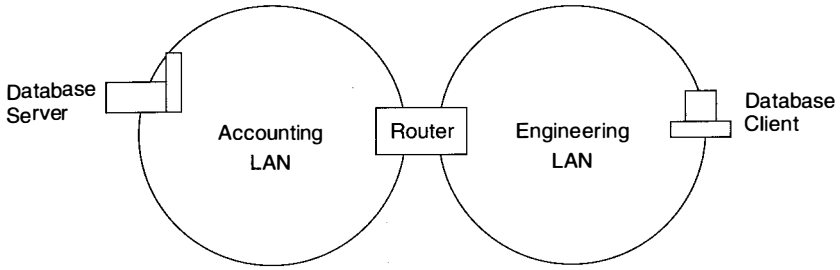


Figure 7-1. *Wide Area Network in a Corporation*

communication is quite popular on the Internet, where one node transmits multimedia information (for example, Rolling Stone® Live) to all other interested nodes. (See Appendix B on multicasting via WinSock.)

Mailslot-based communication can certainly be used for client/server communication and between any two peer processes (local or remote). However, in this chapter, we will continue to use the client/server term with the understanding that a server process receives a client process request and may respond to the client. This nomenclature is consistent with client/server designs presented elsewhere in the book.

7.1 ARCHITECTURE OF MAILSLOT

Mailslot, an application-level broadcast and multicast facility provided by Windows NT, originated in Microsoft LAN Manager software.

The Mailslot applications communicate with each other using the Mailslot interface, which is supported by the Windows NT server[1] and the Windows NT redirector. The Windows NT server and Windows NT communicate via SMB protocol using the services of the underlying transport layer. As Figure 7-2 shows, a Windows NT workstation can simultaneously support multiple transport services. As long as both server and client have at least one common transport service (for example, NetBIOS), they can send and receive Mailslot messages. When a computer has multiple transport services, the Windows NT server or the redirector will attempt to deliver the Mailslot packet to the other end by trying each transport service one after the other in the order the transport services were configured. Thus, when the client and the server have more than one common transport protocol, the recipient may receive duplicate packets (one packet per common protocol); in this case, CCS designers have to filter out duplicate Mailslot messages when they arrive.

1. The basic Windows NT package contains a peer server, which can be used for limited server capablities, including creating Mailslots. Windows NT Advanced Server is the complete server package. Thus, every Windows NT station has a server, albeit with varying capabilities.

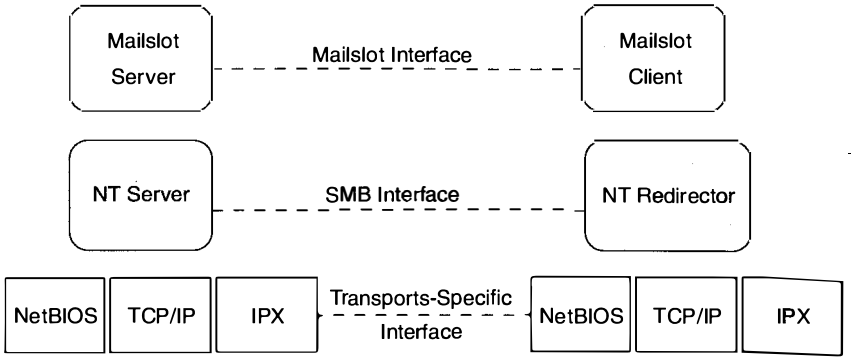


Figure 7-2. *Mailslot Architecture in Windows NT*

In setting up a Mailslot channel, the server must create the Mailslot and then can read messages sent by local or remote clients, which must "open" the Mailslot before sending messages. Thus, a server can create a Mailslot and read from it, or the client can open a Mailslot and send messages to the server. But the server cannot send a message to the client using the Mailslot it has created, nor can the client read from the Mailslot it has opened to write to the server. In short, we have a one-way nonguaranteed communication channel with Mailslots. For there to be two-way nonguaranteed communication, the client and the server each must create a Mailslot. This is much like real-life mailboxes in front of houses. To receive care packages from Mom, you must own a mailbox. And your mother needs a mailbox as well if she wants to receive your infrequent replies!

The Mailslot provides message-based communication much like Named Pipe does; each message can have zero or more bytes. In other words, the reader of a Mailslot must read the whole message at once and not one or more bytes at a time. (Contrast this with the Windows Socket interface, which exposes a byte stream.) Well, then, what constitutes a message?

Every successful "write" into the Mailslot is considered a complete message. When a message arrives at the server, the Mailslot system queues it. Thus, if a client has sent three messages to the server and two of those messages arrived at the server end, the server will have to issue two "read" I/O calls to retrieve them.

The Mailslot name indicates the range and type of communication. The server always creates a Mailslot using the following format: `\\.\mailslot\<name>`—for example, `\\.\mailslot\mymailslot` or `\\.\mailslot\apps\spreadsheet`. The name must be unique on a given machine. The client chooses one of the formats in Table 7-1 depending on the range and the type of datagram desired.

Table 7-1. Mailslot Name Formats Used by Mailslot Clients

Syntax	*Description*
`\\.\mailslot\<name>`	Used to communicate to a server resident on the same machine. The communication is local to the machine.
`\\<ComputerName>\mailslot\<name>`	Used to communicate to a server present on a specified computer. The communication is done via broadcast datagrams.
`\\<Domain Name>\mailslot\<name>`	Used to communicate to any Mailslot server in the domain that has the same <name> component. The communication is achieved via broadcasting (ideally this should be done by multicasting). Applications running on many machines, all sharing the same name in the specified domain, will receive the message.
`\\*\mailslot\<name>`	For broadcasting to a specific application on all systems on network.

Based on the name format of the Mailslot, the client requests may be directed to a specific application on a local machine or to all computers in the client's domain.

7.2 PROGRAMMING MAILSLOT

Mailslot programming interface can be used to design client/server applications as well as applications that need peer-to-peer communication. It allows datagram-based nonguaranteed communication in which datagrams can be directed to a specific process or a group of processes running in a domain. This section begins with the design of a Mailslot server, then presents a Mailslot client, and ends with an examination of certain Mailslot APIs that can be used to improve the efficiency of Mailslot I/O operations.

7.2.1 Creating a Mailslot Server

A Mailslot server can create a Mailslot and read messages sent by clients. The server and the client can reside on the same machine (local) or be running on distinct computers (remote). The server uses the **CreateMailslot()**

API to create a Mailslot and the ReadFile() API to read from any requests sent by the client. Finally, it closes the Mailslot using the CloseHandle() API:

```
HANDLE CreateMailslot( LPCTSTR lpszName, DWORD cbMaxMsg,
                       DWORD dwReadTimeout, LPSECURITY_ATTRIBUTES lpsa );
BOOL ReadFile(HANDLE hMailSlot, LPVOID lpBuffer,
              DWORD nNumberOfBytesToRead,
              LPDWORD lpNumberOfBytesRead, LPOVERLAPPED lpOverlapped );
BOOL CloseHandle ( HANDLE hMailSlot );
```

While creating a Mailslot, the server specifies the name of the Mailslot (via `lpszName`), which must follow the syntax `\\.\mailslot\<name>`, and the maximum size of messages in bytes (via cbMaxMsg) that it expects to receive from the clients. The server Mailslot can receive one or more messages, each of which must have equal or fewer bytes than the maximum specified. There is no limit on number of messages that a Mailslot can receive. The server can specify NULL maximum size, which will allow it to receive messages of any size allowed by the system. For the NetBEUI transport protocol, the maximum size of a directed Mailslot message cannot be more than 64 Kbytes. When Mailslot messages are sent via the underlying NetBEUI protocol, a broadcast message to a domain is limited to 400 bytes.

The server also specifies the default time-out period in milliseconds (via dwReadTimeout) associated with reading a message from the Mailslot when Mailslot is empty. A value of 0 signifies that ReadFile() will return immediately without an error when the Mailslot is empty. By specifying MAILSLOT_WAIT_FOREVER the server ensures ReadFile() will not block until a message is available. When any messages are already in the message queue, the ReadFile() call will complete immediately without being affected by the default read-time-out value of the Mailslot. Finally, just like any Win32 API, CreateMailslot() API allows SECURITY_ATTRIBUTES to be associated with the Mailslot (via lpsa). The server can restrict access to the Mailslot. By specifying NULL as a security parameter, the server delegates the responsibility of access control to the base system; a client with access to the server machine will also have access (permission to write) to the server's Mailslot. Mailslot servers that are also Win32 services will need to specify a security descriptor. (See Chapter 2 for a general discussion on Windows NT security.)

On successful completion of the CreateMailslot() API, the server receives a handle to its newly created Mailslot that it can use to read messages from the Mailslot using the ReadFile() API, besides doing other file system operations. The server supplies the handle (via hMailslot) to the ReadFile() function and a buffer (pointed to via the lpBuffer parameter) into which it wishes to receive the Mailslot message. The server also specifies the number of bytes of messages that it wishes to receive in its buffer (via the nNumberOfBytesToRead parameter); it is then prudent for the server to supply a buffer that is equal to or larger in size than that of the message it wishes

to receive. The ReadFile() API returns the number of bytes it actually placed in the server's buffer (via lpNumberOfBytesRead). If other messages are already in the message queue, the ReadFile() API fills the supplied buffer and returns immediately. If the message queue is empty, it will block until the requested number of bytes have arrived in the Mailslot. The wait is determined by the read-time-out value specified when creating the Mailslot. The ReadFile() API can return FALSE with the GetLastError() API returning ERROR_SEM_TIMEOUT when messages do not arrive within default read-time-out time. Finally, the ReadFile() API returns FALSE with GetLastError() returning ERROR_INSUFFICIENT_BUFFER when the server-supplied buffer is smaller than the client-sent message. On the other hand, the server can request to read more bytes (for example, maximum Mailslot message size) than incoming message(s), and ReadFile() will return the message indicating (via lpNumberOfBytesRead) the size of the message.

Recall that the server cannot read the Mailslot as a byte stream; rather, it reads one message at a time. Then the Mailslot is destroyed as soon as all the handles referring to it are closed using CloseHandle() calls.[2] Now the clients can no longer send messages to the server; furthermore, all messages that had arrived in the server's mail prior to closing the Mailslot are discarded. This can be a problem if the client message was important and the server has not read it before the Mailslot was closed. To ensure this does not happen, the clients and the server can adopt a simple protocol by which each client communicates to the server end of its dialog by sending a special, and last, Mailslot message. And the server can adopt a simple policy at the time it needs to destroy the Mailslot:

- It does not acknowledge any new clients. Thus, it can simply discard any messages from new clients.
- It can process data from the current clients until they send an end-of-dialog message. (Of course, there is no guarantee that the client's "end dialog" message will arrive at the server end given nonguaranteed data service from Mailslot.)

When the client/server communication is strategic and loss of data packets is unacceptable, the designer should build a guaranteed delivery protocol on top of the Mailslots or use guaranteed delivery services offered by other communication methods, such as Named Pipe or stream sockets.

The following code fragment shows a simple Mailslot server that receives variable-sized messages (up to MAX_MESSAGE_SIZE) from the clients and prints them on the screen. It shuts down when the client sends it a shutdown message.

2. A child process may have inherited the Mailslot handle from the parent process. Should the child process not explicitly close the handle, the handle will be closed automatically when the child process terminates.

```
//
// See %BOOK%\mailslot\server.mak for full details
//
#include <windows.h>
#include <stdio.h>
#define MAX_MESSAGE_SIZE (4800)          // 4800 bytes
#define READ_TIMEOUT     (5 * 60 * 1000) // 5 minutes
#define MAILSLOT_NAME     "\\\\.\\mailslot\\MyCompany\\ThisServer"
#define SHUTDOWN_MESSAGE "shutdown"

int main ( char argc, char ** argv)
{
    HANDLE   hMailSlot;
    char     szMessageBuffer [ MAX_MESSAGE_SIZE + 1];
    DWORD    dwBytesRead,
             dwRc;
    BOOL     fShutDown = FALSE;
          // Create a Maislot
    if ( (hMailSlot = CreateMailslot( MAILSLOT_NAME,     // name of mailslot
                                      MAX_MESSAGE_SIZE, // Limits msg size
                                      READ_TIMEOUT,     // Max wait on Read
                                      NULL              // Default security
                                                ))== INVALID_HANDLE_VALUE)
    {
            printf("Error: %ld creating mailslot\n", GetLastError());
            return (1);
    }
    // Loop forever until a client sends shutdown message
    do
    {
            printf("Waiting for Client Message\n");
            // Read Message from client(s)
            if (! ReadFile ( hMailSlot,     // MailSlot handle
                        szMessageBuffer,// Buffer to receive message in
                        MAX_MESSAGE_SIZE,// Bytes to read in
                        &dwBytesRead,     // Bytes really read
                        NULL              // no Overlapped I/O
               ))
            {
                   // Try reading again if we simply timeout waiting
                   if ((dwRc = GetLastError()) == WAIT_TIMEOUT)
                        continue;
                   // Other error. Abort.
                   printf("Error: %ld reading from Mailslot\n", dwRc);
                   break;
              }

              // Print out client message
              if (dwBytesRead)
              {
                   printf("Client Message: %s\n", szMessageBuffer);
                   szMessageBuffer  [dwBytesRead] = '\0';
              }
              // Check if a client sent Shutdown message
              if (strstr(szMessageBuffer, SHUTDOWN_MESSAGE))
                        fShutDown = TRUE;
```

```
            // Get ready for next Message
            dwBytesRead = 0;
} while ( fShutDown == FALSE);
// Close the Mailslot
CloseHandle ( hMailSlot);
return (0);
}
```

7.2.2 Creating a Mailslot Client

A Mailslot client can write messages to the server's Mailslot. The message size is limited to 64K bytes when writing to a specific Mailslot in which directed datagrams carry the message from the client to the server. The size is limited to 400 bytes when the client wants to broadcast the message to any server in a domain with the specified Mailslot name.[3] In this case, a broadcast datagram packet may be transmitted to all recipients on the network, at least those on the local network. Because Mailslot does not guarantee message delivery, there is no assurance any message sent to the server's Mailslot will be received by the server.

A client first opens the Mailslot using the CreateFile() API. The form for the name of the Mailslot varies:

`\\.\mailslot\<mailslot name>`	For local Mailslots
`\\<Computer Name>\mailslot\<mailslot name>`	For a remote Mailslot previously created on a specific computer
`\\<Domain Name>\mailslot\<mailslot name>`	For all similar-named Mailslots created in all systems
`\\*\mailslot\<mailslot name>`	For sending messages to any Mailslot in the client's primary domain

Once the client has received a valid handle to the recipient's Mailslot, it can start sending messages to the server using the WriteFile() API. It closes the handle using the CloseHandle() API. The function prototypes of these APIs follow:

```
HANDLE CreateFile(LPCTSTR lpszName, DWORD fdwAccess, DWORD fdwShareMode,
                  LPSECURITY_ATTRIBUTES lpsa, DWORD fdwCreate,
                  DWORD fdwAttrsAndFlags, HANDLE hTemplateFile);
BOOL WriteFile(HANDLE hMailSlot, LPVPOID lpBuffer, DWORD nNumberOfBytesToWrite,
             LPDWORD lpNumberOfBytesWritten, LPOVERLAPPED lpOverlapped);
BOOL CloseHandle (HANDLE hMailSlot);
```

3. This is a limitation posed by the NetBEUI protocol. When the Windows NT workstation is using other protocols, such as TCP/IP or IPX/SPX, the maximum size may be different.

When calling the CreateFile() API, the client specifies the name of the Mailslot (via the lpszName parameter). The access mode (specified via the fdwAccess parameter) can be GENERIC_WRITE because the client cannot perform other operations, such as read, on the Mailslot. The Mailslot client must specify FILE_SHARE_READ as the sharing mode (by means of the fdwShareMode parameter) for opening a Mailslot so the server can open the Mailslot for read operations. Unlike Named Pipes, a client can open the Mailslot even before it has been created and thus will always get a handle to the Mailslot. To ensure failure to open Mailslot when it has not been created, the client can set the create mode (via fdwCreate) to OPEN_EXISTING. However, even with create mode set to OPEN_EXISTING, the client will always get a handle when opening a nonlocal Mailslot. The client can specify the security attributes (via the lpsa parameter), although most clients specify NULL as security attributes to get default access to the server Mailslot. Finally, one specifies FILE_ATTRIBUTE_NORMAL as a Mailslot attribute (via fdwAttrsAndFlags) and NULL as the template file (via hTemplateFile).

Once a handle to the Mailslot has been received, the client uses it (via the hMailslot parameter) to write to the Mailslot (via the WriteFile() API). The client can write zero to multiple bytes into the Mailslot by specifying the message buffer (via lpBuffer) and the number of bytes to write (via nNumberOfBytesToWrite). On successful completion of the WriteFile() API, it informs the client about the number of bytes sent to the server's Mailslot (via lpNumberOfBytesWritten). WriteFile() will succeed (that is, return TRUE) should the client write zero bytes into the Mailslot.

A client can have a handle to a Mailslot even if no server is associated with it. Also, one can easily envision a situation when the client still has a handle to the Mailslot, but the server has terminated. When this happens and the server and the client are coresident on the same machine, writing into a nonexistent (or terminated) Mailslot will cause the WriteFile() API to return an extended error (ERROR_HANDLE_EOF). However, when the client writes into a nonexistent (or terminated) nonlocal Mailslot, the WriteFile() API always succeeds! Why? Recall that the Windows NT redirector can be presented the name of the remote Mailslot in three formats: (1) a computer name format (for example, \\BigServer\Mailslot\MyMailslot), (2) a domain name format (for example, \\PayrollDomain\Mailslot\MyMailslot), or (3) a broadcast name (*\Mailslot\MyMailslot). In the last case, the user obviously wishes to send broadcast messages in the primary domain. However, the Windows NT redirector is not designed to distinguish between the first two cases. (Ideally, one would expect a directed datagram to the specified computer in the first case and a multicast datagram in the second case.) Hence, the Windows NT redirector sends broadcast messages in all three cases. It does not use any

mechanism to verify the existence of the remote recipient or the successful receipt of the broadcast messages. The result is that the WriteFile() API always returns the success code. Client/server system designers should understand the implications of this nonguaranteed data delivery system. The only way to work around it is to build an application-level protocol (on top of the Mailslot system) in which the recipient acknowledges the receipt of the messages (much like registered mail).

The following code fragment shows a client that sends variable-sized messages to the server shown in subsection above. The client sends messages typed in by the user.

```
//
// See %BOOK%\mailslot\client.mak for full details
//
#include <windows.h>
#include <stdio.h>

#define MAX_MESSAGE_SIZE (4800)                                  // 4800 bytes
#define READ_TIMEOUT     (5 * 60 * 1000)                         // 5 minutes
#define MAILSLOT_NAME     "\\\\.\\mailslot\\MyCompany\\ThisServer"
#define SHUTDOWN_MESSAGE  "shutdown"
#define QUIT              "quit"

int main (char argc, char ** argv)
{
    HANDLE     hMailSlot, hConsole;
    char       szMessageBuffer [ MAX_MESSAGE_SIZE +1 ];
    DWORD      dwBytesToWrite,
               dwBytesWritten;
    BOOL       fShutDown = FALSE;
    LPSTR      lpszMailSlotName = MAILSLOT_NAME;   // Default mailslot name

    if (argc ==2)
    {     // User Supplied Mailslot name
          lpszMailSlotName = argv[1];
    }
    printf("Connecting to %s\n", lpszMailSlotName);
          // Open the Mailslot
    if ((hMailSlot = CreateFile( lpszMailSlotName, // name of mailslot
                                 GENERIC_WRITE,    // Access mode
                                 FILE_SHARE_READ,  // Share mode
                                 NULL,             // Default security
                                 OPEN_EXISTING,    // Open only if exists
                                 FILE_ATTRIBUTE_NORMAL,// Normal File I/O
                                 NULL              // No Template
                               ))== INVALID_HANDLE_VALUE)
    {
              printf("Error: %ld opening mailslot\n", GetLastError());
              return (1);
    }
    // We open the console so we can read-in user-typed messages
    if (( hConsole = CreateFile( "CON",         // open the console to read
```

```
                                    GENERIC_READ,     // Access mode
                                    FILE_SHARE_WRITE,// Share mode
                                    NULL,             // Default security
                                    OPEN_EXISTING,    // Open only if exists
                                    FILE_ATTRIBUTE_NORMAL, // Normal File I/O
                                    NULL              // No Template
                                    ))== INVALID_HANDLE_VALUE)
    {
            printf("Error: %ld opening console\n", GetLastError());
            return (1);
    }

    // Loop forever until a User types "quit" or sends "shutdown" message
    do
        {
        // Request input from user
        printf("\nMessage For Server: ");
        if (!ReadFile ( hConsole,             // Read from console
                        szMessageBuffer,      // User message buffer
                        MAX_MESSAGE_SIZE,     // Maximum buffer size
                        &dwBytesToWrite,      // Bytes typed in User
                        NULL                  // No Overlapped I/O
                      ))
        {    // Error reading user message
        printf("Error reading from Console: %ld\n", GetLastError());
        break;
        }
        // Send user's message to the server mailslot
        if (!WriteFile ( hMailSlot,         // MailSlot handle
                         szMessageBuffer,   // Buffer with message
                         dwBytesToWrite,    // Bytes to write
                         &dwBytesWritten,   // Bytes really written
                         NULL               // no overlapped I/O
                       ))
        {
             // Error Writing into the mailslot
             printf("Error: %ld Writing into Mailslot\n", GetLastError());
        }
             // Show the size of message that was sent to the server
             printf("Total Bytes: %ld Bytes Actually Sent: %ld\n",
                     dwBytesToWrite, dwBytesWritten);

             // Check if a client sent Shutdown message or wants to quit
             if (strstr( szMessageBuffer, SHUTDOWN_MESSAGE) ||
                 strstr( szMessageBuffer, QUIT))
                       fShutDown = TRUE;

             // Get ready for next Message
             dwBytesToWrite = 0;
             } while ( fShutDown == FALSE);
             // Close the Mailslot and Console
             CloseHandle (hMailSlot);
             CloseHandle (hConsole);
             return (0);
}
```

7.2.3 Efficient Use of Mailslot

In the illustrations of typical Mailslot client and server interaction, both client and server agreed on the maximum size (4800 bytes) of the Mailslot message. However, when the client may send variable-length messages, ranging from a few bytes to the maximum allowable bytes (64K bytes), the server must be able to determine the message size. Two APIs, **GetMailslotInfo()** and **SetMailslotInfo()**, can be used to design Mailslot clients and servers using variable-length messages. They also can be used to poll the Mailslot to detect the arrival of new messages. Recall that when no messages are available in the Mailslot, the ReadFile() API will block. The polling mechanism allows the server to call the ReadFile() API after a new message has arrived. The prototypes of these APIs follow.

```
BOOL GetMailslotInfo(HANDLE hMailslot, LPDWORD lpcbMaxMsg,
                     LPDWORD lpcbNextMsg,
                     LPDWORD lpcMsg, LPDWORD lpdwReadTimeout);
BOOL SetMailslotInfo(HANDLE hMailslot, DWORD dwReadTimeout);
```

The server application can use the SetMailslotInfo() API to change the read-time-out value (via dwReadTimeOut) of a Mailslot at run time and thus control the amount of time the ReadFile() API call waits for incoming Mailslot messages. The read-time-out values are specified in milliseconds and can be in the interval of [0, MAILSLOT_WAIT_FOREVER]. When the time-out value is set to 0, the ReadFile() API returns immediately when there are no messages. For a time-out interval of MAILSLOT_WAIT_FOREVER, the ReadFile() API blocks until a message arrives. An alternative to changing the time-out value is to use the GetMailslotInfo() API to detect (via the lpcMsg parameter) pending messages. This allows the application to call the ReadFile() API only when pending messages are detected.

The GetMailslotInfo() can be used to determine the size of the next Mailslot message pending in the queue (via the lpcbNextMsg parameter). This allows the client to send a variable-sized buffer to the server. The server can dynamically allocate the correct-sized buffer based on the size of the next message information returned by the GetMailslotInfo() API. Finally, the GetMailslotInfo() can be used to determine the read-time-out value for the Mailslot (via lpdwReadTimeout) as well as the maximum Mailslot message size for the given Mailslot (via lpcbMaxMsg). Recall that the maximum message size is set when Mailslot is created. The GetMailslotInfo() API returns information whenever the corresponding parameter is not NULL. For example, calling GetMailslotInfo (hMailslot, NULL, &dwNextMessageSize, &dwNosMessages, NULL) will return the number of pending messages in dwNextMessage and the size of the next message in dwNosMessages.

The following code fragment shows a server that uses the GetMailslotInfo() to avoid calling ReadFile() on empty Mailslot and dynamically allocates a

buffer to contain an incoming message. We show one optimization technique in that the server defaults to using an 80-byte stack-based buffer until the incoming message is larger, so the server does not have to allocate buffers dynamically all the time. The server is a slight modification from the example shown earlier.

```
//
// See %BOOK%\mailslot\server2.mak for full details
//
#include <windows.h>
#include <stdio.h>

#define MAX_MESSAGE_SIZE     (4800) // Max Message Size
#define OPTIMUM_MESSAGE_SIZE (80)   // Most often client sends fewer than
                                    // 80 bytes
#define READ_TIMEOUT         (5 * 60 * 1000) // 5 minutes
#define MAILSLOT_NAME        "\\\\.\\mailslot\\MyCompany\\ThisServer"
#define SHUTDOWN_MESSAGE     "shutdown"

int main ( char argc, char ** argv)
{
    HANDLE     hMailSlot;
    char       szMessageBuffer [ OPTIMUM_MESSAGE_SIZE ];
    char*      pszMsgPtr = szMessageBuffer;
    DWORD      dwBytesRead = 0,
               dwNosMessages = 0,
               dwNextMessageSize     = 0,
               dwRc = 0,
               dwBufferSize = OPTIMUM_MESSAGE_SIZE;
    BOOL       fShutDown = FALSE;
    // Create a Mailslot
if (( hMailSlot = CreateMailslot(MAILSLOT_NAME,     // name of mailslot
                                 MAX_MESSAGE_SIZE, // Limits msg size
                                 READ_TIMEOUT,     // Max wait on Read
                                 NULL              // Default security
                                ))== INVALID_HANDLE_VALUE)
{
    printf("Error: %ld creating mailslot\n", GetLastError());
                    return (1);
}
    // Loop forever until a client sends shutdown message
printf("Waiting for Client \n");
do
{
        // Determine if one or more messages are pending
        if (!GetMailslotInfo( hMailSlot,
                              NULL,
                              &dwNextMessageSize,
                              &dwNosMessages,
                              NULL))
        {     // We will simply bail out on this error
    printf("Error: %ld in GetMailslotInfo\n", GetLastError());
           break;
```

```
        }
        else if (!dwNosMessages)
        {
            // No message. Do some chores
            continue;
        }
        else if (!dwNextMessageSize)
        {
            printf("Next Message has null length\n");
            continue;
        }
        // We have at least 1 message of non-null size
        if (dwNextMessageSize > OPTIMUM_MESSAGE_SIZE)
        {
            pszMsgPtr = (char *)
            malloc ( dwBufferSize = dwNextMessageSize +1 );
        }
        // Read Message from client
        if (! ReadFile ( hMailSlot,     // MailSlot handle
                         pszMsgPtr,     // Buffer to receive message in
                         dwBufferSize, // Bytes to read in
                         &dwBytesRead, // Bytes really read
                         NULL           // no Overlapped I/O
        ))
        {
            // Try reading again if we simply timeout waiting
            if ((dwRc = GetLastError()) == WAIT_TIMEOUT)
                        continue;
            // Other error. Abort.
            printf("Error: %ld reading from Mailslot\n", dwRc);
            break;
        }

        // Print out client message
        if (dwBytesRead)
        {
            printf("Client Message: %s\n", pszMsgPtr);
            pszMsgPtr [ dwBytesRead ] = '\0';
        }
        // Check if a client sent Shutdown message
        if (strstr(pszMsgPtr, SHUTDOWN_MESSAGE))
            fShutDown = TRUE;

        // Free the buffer if we allocated it
        if (dwNextMessageSize > OPTIMUM_MESSAGE_SIZE)
        {
            free(pszMsgPtr);
            pszMsgPtr = szMessageBuffer;
            dwBufferSize = OPTIMUM_MESSAGE_SIZE;
        }
        dwBytesRead = 0;
    } while (fShutDown == FALSE);
    // Close the Mailslot
    CloseHandle (hMailSlot);
    return (0);
}
```

7.3 A WIN32 SERVICE USING MAILSLOT

Now let us see how Mailslot can be used by a Win32 service, a multithreaded program in which a given thread services a given client. I have deliberately chosen to illustrate a complex multithreaded Mailslot server here.

In this section, we will see the Echo Server using the Mailslot by expanding the skeletal service code provided in Chapter 2. In the code that follows, notice that the Echo Server has two parts: (1) the code fragments necessary for it to function as a Win32 Service, and (2) the code fragments used to create a Mailslot server that will accept calls from the Mailslot clients. Because the first part was explained in Chapter 2, we will concentrate here on the second part of the code fragment.

The Echo Server Win32 Service creates a "listener thread," which then creates a well-known server Mailslot (\\.\Mailslot\EchoServer) to which new clients will send Mailslot messages. The listener thread forever loops, waiting for new client requests. To avoid blocking on the ReadFile() API, the listener thread calls the GetMailslotInfo() API to detect any new client requests. When a message arrives in its Mailslot, the listener thread uses the ReadFile() API to extract the client message and puts this new message in the client queue. Who services these client queues? Well, at start-up time, the Echo Server starts a number of "worker threads" (see **WorkerThread()** function), which handle echoing client requests.

The first action by each worker thread is to create a Mailslot for itself (\\.\Mailslot\EchoWorker_X where X = thread number). Recall that Mailslot is unidirectional, so both the client and the server need to "open" each other's Mailslots before two-way communication can begin. Thus, the client needs to know the server's Mailslot name, and vice versa. The technique for achieving this without having to hard-code the client Mailslot name in the server is illustrated in Figure 7-3.

As Figure 7-3 shows, in order to handle multiple clients running at the same time on the same workstation, a new client creates its own Mailslot (\\.\Mailslot\EchoClient_X where X = Client no) and then connects with the Mailslot (\\.\Mailslot\EchoServer) created by the listener thread by calling the CreateFile() API. The client then uses the WriteFile() API to send the name of its own Mailslot as the first message to the Echo Server. This message is received by the listener thread on the server, which puts the client Mailslot name in the client queue. Momentarily, the next available worker thread picks the new arrival and connects with the client Mailslot by calling the CreateFile() API. On successfully connecting to the client Mailslot, the worker threads sends (to the client) its own Mailslot name (\\.\Mailslot\EchoWorker_X). Why? Recall that initially the client is connected to the Mailslot of the listener thread, so any more messages the client sends will be serviced by the listener thread, not by the worker thread. Hence, the client must close the handle of the listener thread Mailslot. The client, in fact, simply calls

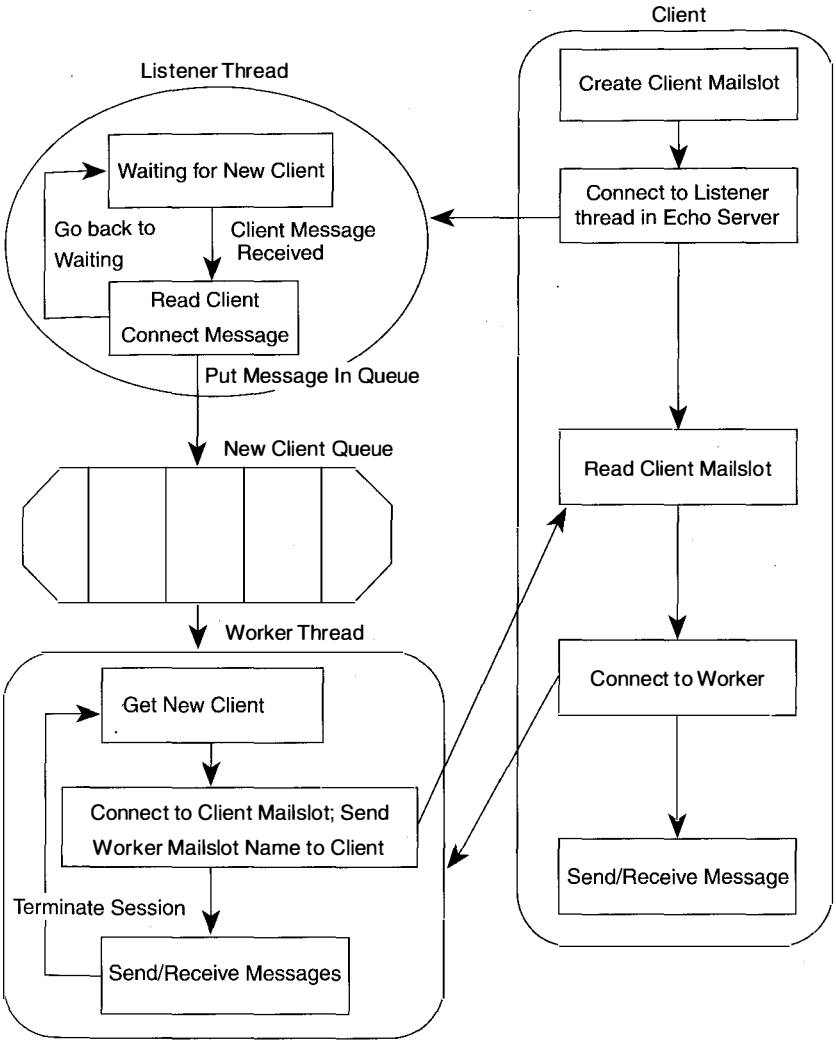


Figure 7-3. *Mailslot-based Echo Server Win32 Service*

the ReadFile() API on its own Mailslot, waiting to receive a message from the worker thread. Upon receiving the message, the client opens the Mailslot of the worker thread. Now both the client and the worker thread are connected to each others' Mailslot. At this point, the client sends a message to the Mailslot of the worker thread, which simply sends an echo back to the client.

In the example, we illustrate the following additional features:

- Use of security descriptor. Because the Echo Server runs as a Win32 service, it is logged on by the system as a special user (LocalSystem). Thus, a Mailslot created by the server is not accessible to the normally logged-on users unless the service allows

them by specifying a proper security descriptor. In the function below, we create a NULL **Discretionary Access Control List (DACL)** that, when associated with the Mailslot, allows all users to access it.

- Multiple clients simultaneously accessing the server Mailslot. Because multiple clients may be writing to the listener thread's Mailslot simultaneously, each client must use the FILE_SHARE_WRITE sharing mode with the CreateFile() API when connecting to the listener thread Mailslot. This is in addition to the FILE_SHARE_READ sharing mode that all Mailslot clients must use (to allow the server to read the contents of the Mailslot).

```
////////////////////////////////////////////////////////////
// FILE: %BOOK%\echosrv\mailslot\EchoS.c
//
//
#include <header.h>
#pragma hdrstop
#define MAX_INSTANCES     (5)
#define LISTEN_MAILSLOT_NAME     L"\\\\.\\MAILSLOT\\EchoServer"
#define WORKER_MAILSLOT_NAME     "\\\\.\\MAILSLOT\\EchoWorker_%d"
#define MAX_MESSAGE_SIZE (4800)
#define CLIENT_MAILSLOT_SIZE    (MAX_PATH + 4)
#define READ_TIMEOUT     (1 * 60 * 1000) // 1 minutes
#define WRITE_TIMEOUT    (1 * 60 * 1000) // 1 minutes
#define SERVER_RESPONSE "Echo: %s by Thread: %ld"
// The data shared between threads
typedef struct _MailData
{
    TCHAR szClientMailslotName [ MAX_PATH ];
} MAIL_DATA, *PMAIL_DATA;

// Create a pool of MAIL_DATA shared between Listener thread
// and worker threads
INITIALISE_POOL(MAX_INSTANCES, sizeof(MAIL_DATA))

// Global Data
HANDLE    hListenMailslot;
BOOL      fStopService = FALSE;

// Internal functions
void ListenThread(VOID * pF);
void WorkerThread(VOID *pF);
void ShutDown(void);
DWORD InitializeMailslot();

BOOL InitEchoServer()
{
    DWORD   dwRc;
    BOOL    bRet = TRUE;
    // First initialize the Client Request Pool
    dwRc = InitializeQueue();
    if (dwRc == 0)
```

```
    {
          // Get the Mailslot Service Started
          dwRc = InitializeMailslot ( );
          if (dwRc == 0)
               // Finally Create the worker thread pool
               dwRc = InitializeThreads (WorkerThread);
          if (dwRc)
               bRet = FALSE;
    }
    return(bRet);
}
// Function: CreateSecurityDesc()
// Purpose:
// Because the EchoServer runs as a service, it is logged on by the system
// as a special user (LocalSystem). Thus, a Mailslot created by the server
// is not accessible to the normally logged on user unless the
// service allows them by specifying a proper security descriptor.
// In the function below, we create a NULL Discretionary
// Access Control List (DACL) which, when associated with
// the Mailslot, allows all users to access the Mailslot.
//
SID_IDENTIFIER_AUTHORITY    NtAuthority = SECURITY_NT_AUTHORITY;
SID_IDENTIFIER_AUTHORITY    LocalAuthority = SECURITY_LOCAL_SID_AUTHORITY;
PSID SystemSid = NULL;
DWORD CreateSecurityDesc(PSECURITY_DESCRIPTOR * ppsdPort)
{
    PSECURITY_DESCRIPTOR psdPort;
    BOOL Status;

    *ppsdPort = NULL;
    // Allocate and initialize a security descriptor
    psdPort = malloc(sizeof(SECURITY_DESCRIPTOR));
    if (!psdPort)
          return(ERROR_OUTOFMEMORY);

    if (!InitializeSecurityDescriptor(psdPort, SECURITY_DESCRIPTOR_REVISION))
    {
          return(GetLastError());
    }
    //
    // Okay, now we have the basic structure ready.  First, we add an owner
    // of named pipe to the descriptor
    if (!SystemSid)
    {
          Status = AllocateAndInitializeSid(
                      &NtAuthority,
                      1, SECURITY_LOCAL_SYSTEM_RID, // "LocalSystem" is
                                                    // owner
                      0, 0, 0, 0, 0, 0, 0,
                      &SystemSid);
         if (!Status)
          {
               free(psdPort);
               return(GetLastError());
          }
    }
```

```
    if (!SetSecurityDescriptorOwner(psdPort, SystemSid, FALSE))
    {
          free(psdPort);
          return(GetLastError());
    }

    //
    // Set the primary group information (LocalSystem)
    //
    if (!SetSecurityDescriptorGroup(psdPort, SystemSid, FALSE))
    {
          free(psdPort);
          return(GetLastError());
    }
    //
    // Set a NULL Dacl, which is supposed to indicate all access granted
    // as opposed to an empty ACL which is no access:
    //
    if (!SetSecurityDescriptorDacl(psdPort, TRUE, NULL, TRUE))
    {
          free(psdPort);
          return(GetLastError());
    }
    *ppsdPort = psdPort;
    return(0);
}
DWORD InitializeMailslot()
{
    DWORD dwRc;
    PSECURITY_DESCRIPTOR pSD;
    SECURITY_ATTRIBUTES  secAttr;

    // Initialize MAIL_DATA Resource Pool
    InitializePool();

    // Create a security descriptor to associate with Mailslot
    if (dwRc = CreateSecurityDesc(&pSD))
    {
          PrintError1("CreateSecurityDesc", dwRc); return (dwRc);
    }
    else
    {
          secAttr.nLength = sizeof(secAttr);
          secAttr.lpSecurityDescriptor = pSD;
    }

    //
    // Create a Mailslot which will receive the connect
    // messages from new clients
    //
hListenMailslot = CreateMailslot( LISTEN_MAILSLOT_NAME,  // name of mailslot
                                  MAX_MESSAGE_SIZE,      // Limits msg size
                                  READ_TIMEOUT,          // Max wait on Read
                                  &secAttr               // Security
                                );
    if (hListenMailslot == INVALID_HANDLE_VALUE)
```

```
    {
          dwRc = GetLastError();
          PrintError1("Listen CreateMailslot", dwRc);
          return (dwRc);
    }
    // Now begin the Listen thread
    _beginthread ( ListenThread,
                   4098,
                   0
                 );
    free (pSD);
    return(0);
}
//
// In this thread, the server listens to the incoming client
// request. Once the request arrives, this thread will
// put the incoming request in the request pool for
// worker threads which, in turn, will service the client.
//

// Define Listen state transitions
typedef enum
{
    enCancelListen,
    enWaitOnListen,
} enListenState;

void ListenThread(VOID * pF)
{
    DWORD dwRc   = 0,
    dwEventIndex,
    dwNosMessages,
    dwBytesRead;
    enListenState ListenState;
    PMAIL_DATA pMailData = NULL;
    char szMessageBuffer [ CLIENT_MAILSLOT_SIZE ];

    // Create an event by which to synchronize with Service Thread
    dwEventIndex = GetThreadSyncEvent ();
    if (dwEventIndex == -1)
    {
          PrintError("Could not create Thread Sync. Event\n");
          return;
    }
    //
    // Loop forever listening to incoming clients
    //
    ListenState = enWaitOnListen;
    do
    {
         // First check if Service needs to be stopped
          if (fStopService)
          {
               ListenState = enCancelListen;
          }
          //
```

```
// State driven Asynchronous Listen
//
switch(ListenState)
{
case enCancelListen:
     // Close the listening Mailslot
     CloseHandle ( hListenMailslot );

     // Signal end of this thread
     SetThreadSyncEvent (dwEventIndex);
     dwRc = 0;
     break;

case enWaitOnListen:
     // First check if there are any messages
     if (!GetMailslotInfo( hListenMailslot,
                           NULL,
                           NULL,
                           &dwNosMessages,
                           NULL))
     {   // We will simply bail out on this error
         PrintError1("GetMailslotInfo Error: %d\n",
                     GetLastError());
         break;
     }
     else if (!dwNosMessages)
     {
         // No message. Alter Sleep for a bit
         SleepEx ( 10, TRUE );
         dwRc = WAIT_TIMEOUT;
         continue;
     }
     if (! ReadFile ( hListenMailslot, // Mailslot handle
                      szMessageBuffer, // Buffer to receive
                                       // message in
                      CLIENT_MAILSLOT_SIZE, // Bytes to read in
                      &dwBytesRead,    // Bytes really read
                      NULL             // no Overlapped I/O
                    ))
     {
          // Try reading again if we simply timeout waiting
          if ((dwRc = GetLastError()) == WAIT_TIMEOUT)
               continue;
          // Other error. Abort.
          PrintError1("Error: %ld reading from Mailslot\n", dwRc);
          ListenState = enCancelListen;
          continue;
     }
     //
     // We have received a message from a new client. New clients
     // send their mailslot name in the connection request
     //
     if (GetPoolResource (&pMailData))
     {
       // Client mailslot name is in ASCII. Convert to Wide Char
          mbstowcs (pMailData->szClientMailslotName,
```

```
                    szMessageBuffer, strlen(szMessageBuffer));
                PrintInfoString("Client Mailslot is : [%s]\n",
                    szMessageBuffer);
                    // put this in the Client Queue for Worker threads
                 if ((dwRc = QueueRequest ( pMailData,
                                        sizeof(MAIL_DATA)))!=0)
                   {
                        // Free the resource and break the session
                        PrintError1("Canceling Client Request", dwRc);
                        FreePoolResource ( pMailData );
                   }
            }
            // Wait for other clients
            ListenState = enWaitOnListen;
            dwRc = WAIT_TIMEOUT;

        } // end of switch

    } while (dwRc == WAIT_TIMEOUT);

    // Close Thread sync. event
    CloseThreadSyncEvent ( dwEventIndex );
    _endthread();
}
//
// The worker threads wait to get requests from the
//
//
// Define Worker Thread state transitions
typedef enum
{
    enWaitOnConnect = 0,  // Wait for client to send Connect Msg
    enAckConnectMsg,      // Acknowledge connect and send new Mailslot name
    enReceiveRequest,     // Receive Client Echo message on new Mailslot
    enSendReply,          // Send a response. This also works as an
                          // acknowledgment that client message was
                          // received
    enHangupSession       // End session with client. Close Mailslot
} enWorkerState;

void WorkerThread(VOID *pF)
{
    DWORD dwRc;
    ThreadData *pTData;

    HANDLE     hWorkerMailslot;
    HANDLE     hClientMailslot;
    enWorkerState WorkerState = enWaitOnConnect;

    PMAIL_DATA pMailData = NULL;
    DWORD      dwDataSize,
               dwBytesRead,    dwBytesWritten;
    char       szReadBuffer    [ MAX_MESSAGE_SIZE],
               szWriteBuffer   [ MAX_MESSAGE_SIZE];
    char       szMailslotName  [ MAX_PATH ];
    TCHAR      wszMailslotName [ MAX_PATH ];
```

```
PSECURITY_DESCRIPTOR pSD;
SECURITY_ATTRIBUTES  secAttr;

// Create a security descriptor to associate with Mailslot
if (dwRc = CreateSecurityDesc(&pSD))
{
      PrintErrorl("CreateSecurityDesc", dwRc);
      return;
}
else
{
      secAttr.nLength = sizeof(secAttr);
      secAttr.lpSecurityDescriptor = pSD;
}

// Set the thread status
pTData = (ThreadData *)pF;
if (pTData == NULL)
{
      PrintError("Null Thread Data\n");
      return;
}

//
// Each worker has its own Mailslot independent of the
// Listen Mailslot. When a worker thread picks up
// a new client connection request, it will send
// its own Mailslot name to the client. After
// that, the client communicates with the
// worker Mailslot and hence with the worker thread.
//
sprintf(szMailslotName, WORKER_MAILSLOT_NAME, pTData->dwEventIndex);
mbstowcs(wszMailslotName, szMailslotName, strlen(szMailslotName) + 1);
PrintInfoString("Worker Mailslot Name %s", szMailslotName);
hWorkerMailslot = CreateMailslot(
                                  wszMailslotName,  // Name of mailslot
                                  MAX_MESSAGE_SIZE, // Limits msg size
                                  READ_TIMEOUT,     // Max wait on Read
                                  &secAttr          // Security
                                 );
if (hWorkerMailslot == INVALID_HANDLE_VALUE)
{
      dwRc = GetLastError();
      PrintErrorl("CreateMailslot", dwRc);
      return ;
}
do
{
      // First check if Service needs to be stopped
      if (fStopService)
      {
           WorkerState = enHangupSession;
      }
      switch (WorkerState)
      {
      // Wait for Listen Thread to put data on queue
```

```
case enWaitOnConnect:
     dwRc = GetNextRequest ( &pMailData, &dwDataSize);
     if (dwRc ==0)
     {   // Verify the data integrity
         if ((pMailData == NULL) ||
             (dwDataSize < sizeof(MAIL_DATA)))
          {
               dwRc = 0;
               PrintError("Data Corruption\n");
          }
          else
          {
              // We have successfully gotten a new client request
               WorkerState = enAckConnectMsg;
               dwRc = WAIT_TIMEOUT;
          }
     }
     else if (dwRc != WAIT_TIMEOUT)
          PrintError1("GetNextRequest", dwRc);
     break;
case enAckConnectMsg:
     // Connect to client and Send it Worker's Mailslot name
     if (( hClientMailslot = CreateFile    (
         pMailData->szClientMailslotName,// name of mailslot
         GENERIC_WRITE,                  // Access mode
         FILE_SHARE_READ,                // Share mode
                                         // Default security
         OPEN_EXISTING,                  // Open only if exists
         FILE_ATTRIBUTE_NORMAL,          // Normal File I/O
         NULL                            // No Template
         ))== INVALID_HANDLE_VALUE)
     {
          dwRc = GetLastError();
          PrintError1("Opening mailslot\n", dwRc);
          // Error. Free Pool Resource
          dwRc = WAIT_TIMEOUT;
          WorkerState = enHangupSession;
          continue;
     }
     dwDataSize = strlen(szMailslotName) +1;
     dwBytesWritten = 0;
     if ( !WriteFile ( hClientMailslot,   // Mailslot handle
                       szMailslotName,    // Buffer containing
                                          // message
                       dwDataSize,        // Bytes to Write
                       &dwBytesWritten,   // Bytes really written
                       NULL               // no Overlapped I/O
                     ) ||
        (dwBytesWritten < dwDataSize))
     {
          // Error Writing into the mailslot
          dwRc = GetLastError();
          PrintError1("Writing into Mailslot\n", dwRc);
          // Close connection
          dwRc = WAIT_TIMEOUT;
          WorkerState = enHangupSession;
```

```
        }
        else
        {
            dwRc = WAIT_TIMEOUT;
            WorkerState = enReceiveRequest;
        }
        break;
    case enReceiveRequest:
        //
        // Check a) if client has sent request, b) Receive time out
        //                period expired (Client dead, etc.)
        //
        dwBytesRead = 0;
        if (!ReadFile ( hWorkerMailslot,  // Read from console
                        szReadBuffer,     // User message buffer
                        MAX_MESSAGE_SIZE, // Maximum buffer size
                        &dwBytesRead,     // Client message size
                        NULL              // No overlapped I/O
                      ) ||
           (dwBytesRead == 0)
         )
        {   // Error reading client message
            dwRc = GetLastError();
            PrintErrorl("Error reading from Mailslot\n", dwRc);
            dwRc = WAIT_TIMEOUT;
            WorkerState = enHangupSession;
            break;
        }
        // Process the request
        PrintInfoString("\nClient Request is : [%s]\n",
                        szReadBuffer);
        WorkerState = enSendReply;
        break;
    case enSendReply:
        //
        // We received a message from client. Send it a reply
        // and close connection (since we are 'Echo' Server only
        //
        sprintf(szWriteBuffer, SERVER_RESPONSE,
                szReadBuffer,  pTData->dwEventIndex
               );
        dwDataSize = strlen(szWriteBuffer) + 1;
        dwBytesWritten = 0;
        if (!WriteFile (hClientMailslot,// Mailslot handle
                        szWriteBuffer,  // Buffer containing message
                        dwDataSize,     // Bytes to Write
                        &dwBytesWritten,// Bytes really written
                        NULL            // no Overlapped I/O
                       ) ||
            (dwBytesWritten < dwDataSize))
        {
            // Error Writing into the mailslot
            dwRc = GetLastError();
            PrintError1("Writing into Mailslot\n", dwRc);
        }
```

```
                // Close connection
                dwRc = WAIT_TIMEOUT;
                WorkerState = enHangupSession;
                break;
          case enHangupSession: // Disconnect with client

                // Disconnect from client's Mailslot
                CloseHandle ( hClientMailslot );

                if (pMailData)
                {
                     // Release the buffer
                     FreePoolResource ( pMailData );
                     pMailData = NULL;
                }
                // Get ready for next client
                WorkerState = enWaitOnConnect;
                dwRc = WAIT_TIMEOUT;

                if (fStopService)
                {
                     // Notify the service thread that we can be closed
                     SetThreadSyncEvent (pTData->dwEventIndex);
                     dwRc = 0;
                }
                break;
          }; // end switch
     } while (dwRc == WAIT_TIMEOUT);

    // Free security descriptor
    free (pSD);
    // Close Thread sync. event
    CloseThreadSyncEvent ( pTData->dwEventIndex );
    _endthread();
}
BOOL PauseEchoServer()
{

    PrintInfo("Pausing the Mailslot Service is not supported\n");
    return(TRUE);
}
BOOL ResumeEchoServer()
{
    PrintInfo("Resuming the Mailslot Service is not supported\n");
    return(TRUE);
}

BOOL StopEchoServer()
{
    // Signal all threads to quit
    fStopService = TRUE;
    WaitForAllThreadsToTerminate();


    return(TRUE);
}
```

```
//////////////////////////////////////////////////////////
// FILE: %BOOK%\echosrv\mailslot\EchoC.c
// Mailslot client program
//
#include <windows.h>
#include <stdio.h>
#define CLIENT_MAILSLOT_SIZE      (MAX_PATH + 4)
#define LISTEN_MAILSLOT_NAME      "\\\\.\\MAILSLOT\\EchoServer"
#define MAX_MESSAGE_SIZE (4800)           // 4800 bytes
#define READ_TIMEOUT      (1 * 60 * 1000) // 5 minutes
#define MAILSLOT_NAME      "\\\\.\\MAILSLOT\\EchoClient_%d"
#define CLIENT_REQUEST     "Hello from Client"

void Usage(char *ptr)
{
    printf("%s -n <Server Mailslot name> -s <echo string>\n", ptr);
    exit(1);
}
int main (char argc, char ** argv)
{
    HANDLE      hServerMailslot,
                hClientMailslot;
    char        szMessageBuffer [ MAX_MESSAGE_SIZE +1 ];
    DWORD       dwBytesToWrite, dwRc,
                dwBytesRead,
                dwBytesWritten;
    char        szMailslotName [ CLIENT_MAILSLOT_SIZE];
    char*       pszServerMailslotName = LISTEN_MAILSLOT_NAME;
    char*       pszString = CLIENT_REQUEST;
    int         i, iClient = 0;
    for (i = 1; i < argc; i++)
    {
          if ((*argv[i] == '-') || (*argv[i] == '/'))
          {
               switch (tolower(*(argv[i]+1)))
               {
               case 'n':  // Mailslot name
                    ++i;  // user-supplied Mailslot
                          // name break;
                    pszServerMailslotName = argv[i];
               case 's':
                    pszString = argv[++i];
                    break;
               case 'h':
               case '?':
               default:
                    Usage(argv[0]);
               }
          }
          else
               Usage(argv[0]);
    }

    //
    // Open the client's Mailslot. As there can be multiple
    // clients on the same machine, we will keep trying to
```

```
// find a unique name for the client
//
do
{
      sprintf(szMailslotName, MAILSLOT_NAME, iClient);
      printf("Creating Client Mailslot: %s\n", szMailslotName );
      hClientMailslot = CreateMailslot(
                                 szMailslotName,  // name of mailslot
                                 MAX_MESSAGE_SIZE,// Limits msg size
                                 READ_TIMEOUT,    // Max wait on Read
                                 NULL             // Default security
                                    );
      if (hClientMailslot == INVALID_HANDLE_VALUE)
      {
           dwRc = GetLastError();
           if (dwRc != ERROR_ALREADY_EXISTS)
           {
                printf("CreateMailslot error: %ld\n", dwRc);
                return (1);
           }
           else
                iClient++; // try the next name
      }
      else
           dwRc = 0;
}
while ( dwRc == ERROR_ALREADY_EXISTS );

//
// Open the Echo Server Maislot
//
printf("Connecting to Echo Server Mailslot: %s\n",
       pszServerMailslotName);
if (( hServerMailslot = CreateFile (
                        pszServerMailslotName,   // name of mailslot
                        GENERIC_WRITE,           // Access mode
                        FILE_SHARE_WRITE|FILE_SHARE_READ, // Share mode
                        NULL,                    // Default security
                        OPEN_EXISTING,           // Open only if exists
                        FILE_ATTRIBUTE_NORMAL,   // Normal File I/O
                        NULL                     // No Template
                                  ))== INVALID_HANDLE_VALUE)
{
           printf("Error: %ld opening mailslot\n", GetLastError());
           return (1);
}
//
// Send the Server our Mailslot name and receive the worker threads'
// mailslot name
//
dwBytesToWrite = strlen(szMailslotName) + 1;
if (!WriteFile ( hServerMailslot,  // Mailslot handle
                 szMailslotName,   // Contains Client Mailslot name
                 dwBytesToWrite,   // Bytes to Write
                 &dwBytesWritten,  // Bytes really written
                 NULL              // no Overlapped I/O
```

```
                ) | |
            (dwBytesWritten < dwBytesToWrite)
        )
 {
        // Error Writing into the mailslot
        printf("Error: %ld Writing into Mailslot\n", GetLastError());
        return (1);
}
//
// Now receive worker threads Mailslot Name
//
dwBytesRead = 0;
if (!ReadFile ( hClientMailslot,       // Client Mailslot
                szMessageBuffer,       // Message
                MAX_MESSAGE_SIZE,      // Maximum buffer size
                &dwBytesRead,          // Bytes really read
                NULL                   // No Overlapped I/O
              ) | |
    (dwBytesRead == 0)
   )
{  // Error reading server response
   printf("Error reading from client Mailslot: %ld\n", GetLastError());
      return(1);
}
printf("Server Thread Mailslot Name: %s\n", szMessageBuffer);
//
// Close the handle to Server Listen Thread Mailslot and
// open handle to Worker Thread Handle
//
CloseHandle ( hServerMailslot );
if (( hServerMailslot = CreateFile (
                                    szMessageBuffer,// name of mailslot
                                    GENERIC_WRITE,  // Access mode
                                    FILE_SHARE_READ,// Share mode
                                    NULL,           // Default security
                                   OPEN_EXISTING,// Open only if exists
                               FILE_ATTRIBUTE_NORMAL,// Normal File I/O
                                    NULL            // No Template
                                   ))== INVALID_HANDLE_VALUE)
{
        printf("Error: %ld opening Worker mailslot\n", GetLastError());
        return (1);
}
//
// We have established two-way communication handles.
// Now send the Worker a message
//
dwBytesToWrite = strlen(pszString) + 1;
printf("Sending Message: %s\n", pszString);
if (!WriteFile ( hServerMailslot,   // Mailslot handle
                 pszString,         // Contains Client Echo Request
                 dwBytesToWrite,    // Bytes to Write
                 &dwBytesWritten,   // Bytes really written
                 NULL               // no Overlapped I/O
               ) | |
      (dwBytesWritten < dwBytesToWrite)
   )
```

```
    {
          // Error Writing into the mailslot
          printf("Error: %ld Writing into Worker Mailslot\n",
                 GetLastError());
          return (1);
    }
    //
    // Now receive receive an 'Echo' reply from EchoServer
    //
    dwBytesRead = 0;
    if (!ReadFile ( hClientMailslot,     // Client Mailslot
                    szMessageBuffer,     // Message
                    MAX_MESSAGE_SIZE,    // Maximum buffer size
                    &dwBytesRead,        // Bytes really read
                    NULL                 // No Overlapped I/O
                  ) ||
          (dwBytesRead == 0)
       )
    {   // Error reading Server response
          printf("Error reading from Client Mailslot: %ld\n",
                 GetLastError());
          return(1);
    }
    printf("Server Replied: %s", szMessageBuffer);
    // Close the Mailslots
    CloseHandle (hClientMailslot);
    CloseHandle (hServerMailslot);
          return (0);
}
```

7.4 MAILSLOTS IN WINDOWS FOR WORKGROUPS

The Mailslot function in WFW is similar to those offered in Microsoft's LAN Manager; it is different from Mailslot in Windows NT in the following ways:

- Data delivery via Mailslot can be guaranteed or not guaranteed. The "first-class" Mailslots guarantee data delivery; the "second-class" does not. In this respect, Windows NT Mailslot provides second-class functionality.
- Each Mailslot message can have a priority from zero to nine, assigned by the client when it is sending the message. This priority determines the position of each new message in the server's Mailslot, with high-priority messages generally placed before low-priority ones. Windows NT provides no such priority scheme.

A Windows NT–based Mailslot client or server can communicate with its Windows for Workgroups counterpart. In this case, only second-class Mailslots are supported, and the priority of messages is ignored. Table 7-2 shows the WFW APIs used to manipulate the Mailslot. (It is beyond the scope of this book to present the programming details of these APIs. Interested readers will find these Windows for Workgroups APIs to be similar to the Windows NT counterparts and should refer to [MS93d] for details.)

Table 7-2. Windows for Workgroups APIs

Windows for Workgroups API	*Corresponding Windows NT API*	*Comments*
DosMakeMailslot()	CreateMailslot()	Create a Mailslot of a given class
DosDeleteMailslot()	CloseHandle()	Close a Created Mailslot
N/A	CreateFile()	DosWriteFile() is comparable to Windows NT 3 APIs: CreateFile(), WriteFile(), and CloseHandle()
DosReadMailslot()	ReadFile()	Read a message from Mailslot
DosWriteMailslot()	WriteFile()	Write a message to the Mailslot; can assign priority and choose the Mailslot class
DosMailslotInfo()	GetMailslotInfo()	Get Mailslot information
None	SetMailslotInfo()	Change Mailslot characteristics
DosPeekMailslot()	GetMailslotInfo()	Detect pending messages

7.5 SUMMARY

Mailslot provides an application-level programming construct to broadcast messages on the network. The Mailslot in Windows NT and Windows provides this broadcasting functionality irrespective of the underlying transport protocols (for example, NetBIOS, IPX/SPX, TCP/IP). This capability is traditionally used by applications to locate or "discover" other applications.

Mailslot in Windows NT does not guarantee message delivery. Thus, client/server applications will find Mailslot useful only for occasional broadcast purposes, using other guaranteed IPC mechanisms, such as Named Pipes and Windows Sockets, for normal client/server communications.

Chapter Eight

NETBIOS PROGRAMMING IN WINDOWS NT

Windows NT supports the NetBIOS programming interface, which allows IPC. This interface has been provided primarily for backward compatibility and to ease porting MS-DOS and 16-bit Windows applications to the Win32 environment. Historically, Microsoft and IBM operating systems and networking products have been built around NetBEUI protocol (Microsoft's implementation of the NetBIOS protocol), while Novell has based its products around SPX/IPX protocol. As a result of this strategy, network-aware applications can always expect to have NetBIOS support in Microsoft/IBM environments. In fact, Novell NetWare environments also support a NetBIOS programming interface provided proper drivers have been loaded. For this reason, a large number of MS-DOS and Windows applications have been designed to use NetBIOS as the IPC mechanism. Thus, Windows NT provides a means for porting existing NetBIOS-based applications to Windows NT while recommending new applications to be designed with Windows Sockets. Why Windows Sockets? Windows Sockets does not force any over-the-wire protocol, unlike protocols such as NetBIOS or IPX/SPX. (See Chapter 5 for use of Windows Sockets over NetBIOS protocol.)

This chapter presents an overview of NetBIOS programming and then illustrates NetBIOS programming in the Windows NT environment. The chapter goes on to demonstrate the use of NetBIOS as an IPC mechanism in a client/server system and presents a multi-threaded Win32 Service (Echo Server) which communicates with clients using the NetBIOS IPC interface. The chapter ends with an overview of the use of NetBIOS in Windows 3.x environments.

8.1 OVERVIEW OF NETBIOS PROGRAMMING

In 1984, IBM defined NetBIOS (Network Basic Input/Output System) as a session layer network protocol specification to operate over IEEE/ANSI 802 standards [Sinha92a]. The specifications detailed the services offered by the NetBIOS layer and included a programming interface that allowed

applications to use NetBIOS services for communicating over LAN using modules implementing the NetBIOS protocol. This chapter examines the NetBIOS programming interface, not the protocol specifications or their implementation. (See [IBM 84] for details on specifications.) Furthermore, we will treat the entire collection of modules (drivers, DLLs, programs) needed for supporting NetBIOS programming interface as one entity, referred to as the NetBIOS driver.

In early PC-based computing, NetBIOS was quite prevalent in network operating systems. In fact, until a couple of years ago, it was the transport protocol of choice in Microsoft/IBM-supplied network operating systems (Microsoft's implementation of NetBIOS was the NetBEUI protocol). With the pervasive presence of NetBIOS protocol in PC networks, various non-NetBIOS-based transport protocols, such as IPX or TCP/IP, also provided NetBIOS programming interfaces, which allow NetBIOS-based applications to operate in a variety of transport environments. The "pure" NetBIOS transports, such as NetBEUI, both provide the NetBIOS programming interface, per [IBM 84], and communicate with their remote counterpart in a protocol specified by [IBM 84]. However, non-NetBIOS transports may communicate using their own dialect, as shown in Figure 8-1. In the first case, the application is using a NetBEUI protocol while in the second case, the underlying transport is IPX. The NetBIOS programming interface allows the application to execute unmodified in either configuration.

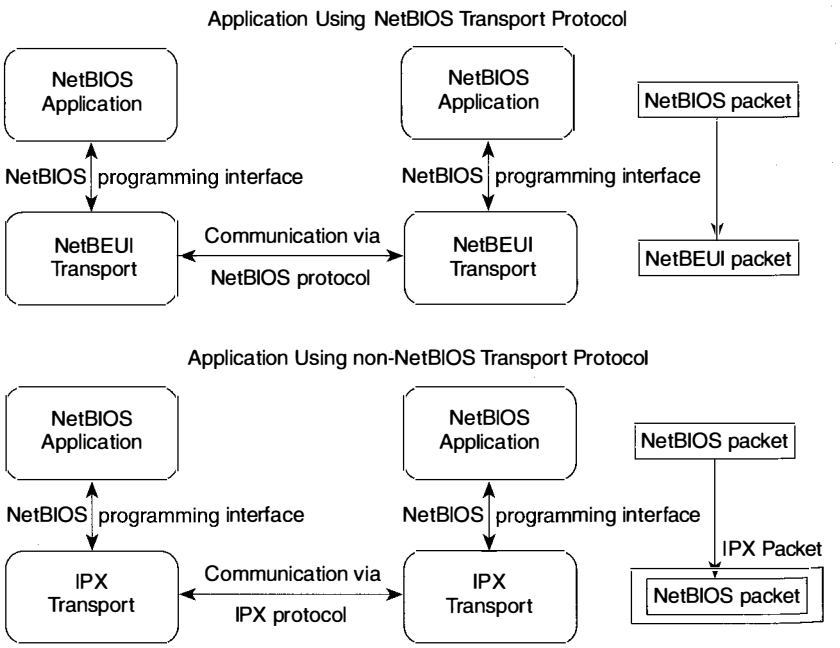


Figure 8-1. *NetBIOS Programming Interface vs. Transport Protocol*

NetBIOS enforces a certain data format (referred to as the "on-the-wire" data format) in packets irrespective of the underlying transport protocol. The data packets are encapsulated within the frames provided by the transport protocols. Thus, in the case of the NetBEUI protocol, each NetBIOS packet fits perfectly in a NetBEUI packet. When the IPX transport is used, each NetBIOS packet may be encapsulated within one or more IPX packets. Contrast this with Windows-Sockets specifications, which do not impose any on-the-wire data format. This difference is important because an absence of on-the-wire format allows Windows-Socket–based applications to match the on-the-wire format of the actual transport protocol (such as TCP/IP or IPX/SPX) that is being used.

Having seen that the NetBIOS programming interface can be exposed by a non-NetBIOS transport protocol (such as IPX/SPX or TCP/IP), you'll now explore the services offered by NetBIOS. NetBIOS services are exploited by applications via the NetBIOS programming interfaces. Thus, any transport protocol which exposes the NetBIOS programming interface provides the NetBIOS services which are described as you go on.

NetBIOS primarily offers a connectionless or connection-oriented communication mechanism and a multicast communication mechanism whereby a process can send a message to a collection of processes simultaneously. Additionally, processes can use a NetBIOS broadcast facility to send messages to all systems on the LAN. NetBIOS offers four services: connection-oriented communication service, connectionless communication service, name management service, and general-purpose services.

Connection-oriented services allow two processes to establish a session or virtual circuit between each other. Once a session has been established, both parties can send and receive messages, with delivery guaranteed. Thus, NetBIOS implementation handles all the issues associated with providing guaranteed message transmission, such as error recovery and message retransmission. Each message can be up to 64K–1 bytes large for normal transmissions and 128K-2 bytes for chained sequences. Figure 8-2 shows the life cycle of a session-based communication between a client and a server. Both the client and the server register their respective names with the underlying network. Then the server waits as it "listens" for new client requests. When a client "calls" a specific server (who must be listening), a virtual circuit is established and the client and server can send and receive data using NetBIOS commands. At the end, both the client and the server terminate their end of the session.

Connectionless service provides means for transmitting datagrams from one process to another. It does not guarantee delivery of each datagram to the recipient, or maintain the order of datagrams. Thus, the recipient may receive the third datagram after receiving the seventh datagram. And when both sender and receivers are in different LANs interconnected by routers and gateways, the network overhead may be higher than that of virtual circuit

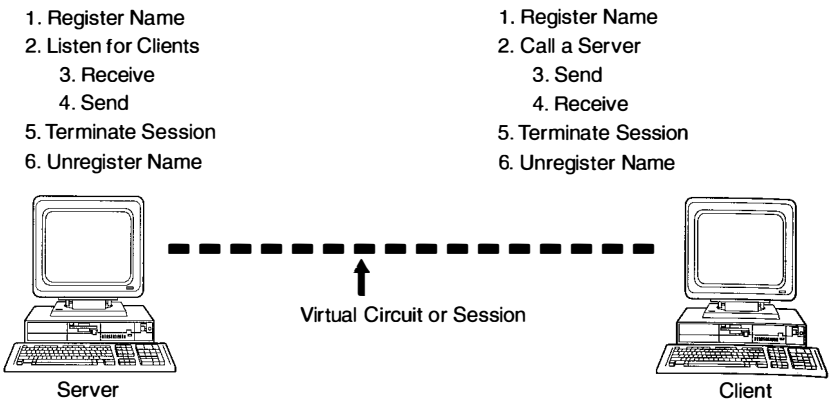


Figure 8-2. *NetBIOS Client and Server Life Cycle*

because each packet has to be routed from the source to the destination independent of its predecessor or follower datagram.

A datagram-based communication service can send directed datagrams (one-to-one) from a sender to the receiver, and to a group of recipients (multicast), and to all entities on the network (broadcast). Broadcast messages up to 512 bytes in length are supported in all environments.[1] Multicasting and broadcasting are often used by communicating processes to locate one another on the LAN. In this respect, datagrams are essential to NetBIOS-based communications. We will revisit this topic shortly.

Name management services provide the means for each communicating entity to be distinctly named and the means for an entity to address other entities on the network. A NetBIOS name is a 16-byte logical name that must be unique on the entire network. An entity can be a process, a thread within a process, or a workstation. In NetBIOS-based communication, only named entities can send and receive messages. These logical names are stored in a table, referred to as the name table, and maintained for each network adapter card present in a workstation. Most transports hold the name in a list. Each name table can contain 254 names. In MS-DOS and Windows environments, the name table can contain up to 254 names across the entire system (for a given network adapter). In the Windows NT environment, a name table is maintained per process for each network adapter, so the entire system can

1. Many transport protocols, such as XNS or TCP/IP, also provide a NetBIOS interface. These transports may support larger datagram packet sizes. The datagram packet size can also be affected by the underlying media (for example, Ethernet, Token Ring). If the length supplied is too much for the transport/media, the transport layer can truncate the message or return an error to the sender.

have many more than 254 names.[2] Every logical name is indexed in the table by a number, referred to as the name number.

In the NetBIOS environment, each process has a logical name so that other processes on the network can address it easily. The network adapter, along with the MAC driver for the card, can use the name table to dispatch the incoming packets to the correct process. This concept is illustrated in Figure 8-3, where five processes are communicating via NetBIOS. The first computer (PC 1) has two processes, which have associated their logical names, Alok and Bonnie, to the card. These two processes are communicating on the first LAN (Net 1) with the two processes, Cynthia and Craig on the second workstation. Notice that the second workstation has two network adapters, so Craig has associated his name with both cards so he can communicate with processes on both the first and second LAN.

NetBIOS offers some services that can be used to determine an adapter's status and to reset an adapter to a predefined state. The adapter status information comprises the physical address of the card, statistics on the packets received by and sent via the adapter, and logical names in the adapter name table.

NetBIOS specifications lay out the NetBIOS programming interface in great detail [see IBM 84]. Unlike other communication interfaces studied in this book, which have multiple APIs, NetBIOS programming uses one operating-system-specific function and one data structure. Examples of NetBIOS functions are **INT 5C** for MS-DOS, **NetBiosCall()** in Windows, **NetBiosSubmit()** in OS/2, and **Netbios()** in Win32. Although each operating environment may provide a unique implementation and interface for NetBIOS, each must adhere strictly to NetBIOS specifications, which go into exacting details on the outcome of executing each command. This

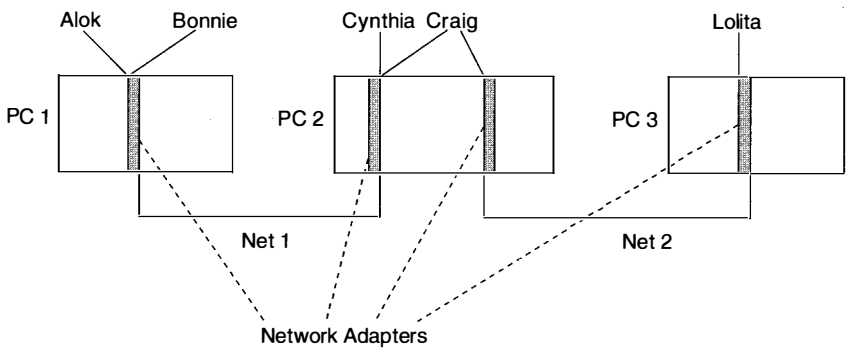


Figure 8-3. *NetBIOS Logical Names*

2. For NetBIOS transport, each network adapter is assigned a logical number starting with zero. For other transports (such as TCP/IP or IPX) supporting a NetBIOS interface, the logical network adapter card number remains the same irrespective of the number of active cards in the system.

allows NetBIOS-based applications to be portable and allows them to communicate with other NetBIOS-aware applications resident on a heterogeneous hardware and software environment. Figure 8-4 shows a NetBIOS-based Mail program that can communicate with its peers hosted on different environments on the same LAN. Note in the figure that each system exposes a NetBIOS interface so the applications can interoperate. This is necessary because NetBIOS forces an on-the-wire data format. Contrast this situation with that in Figure 8-5, in which applications interoperate because Windows Sockets or IPX or **TLI (transport layer interface)** do not force an on-the-wire protocol.

Each NetBIOS function accepts a single parameter—a pointer to a special data structure, the **NetBIOS Control Block (NCB).** The prototype for NCB is shown here.

```
typedef struct _NCB{
          BYTE          ncb_command;
          BYTE          ncb_retcode;
          BYTE          ncb_lsn;
          BYTE          ncb_num;
          DWORD         ncb_buffer;
          WORD          ncb_length;
          BYTE          ncb_callname[16];
          BYTE          ncb_name[16];
          BYTE          ncb_rto;
          BYTE          ncb_sto;
          DWORD         ncb_post;
          BYTE          ncb_lana_num;
          BYTE          ncb_cmd_cplt;
          BYTE          ncb_reserve[14];
} NCB;
```

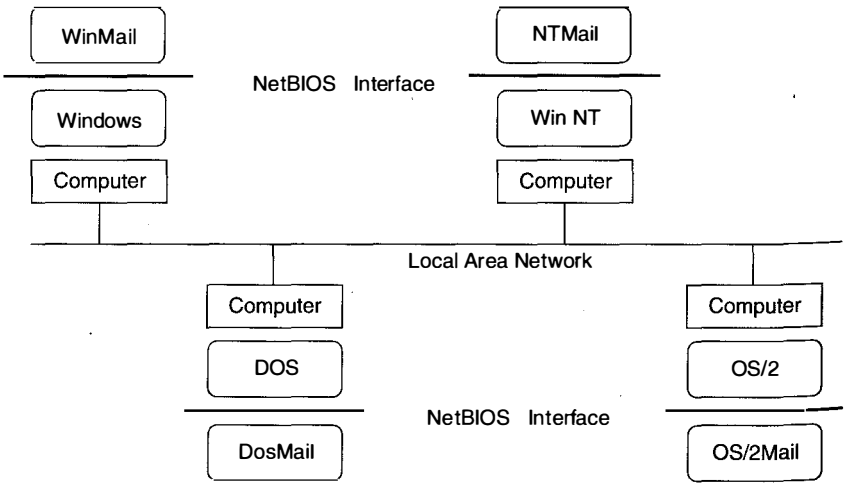



Figure 8-4. *Common NetBIOS Interfaces in Diverse Environments*

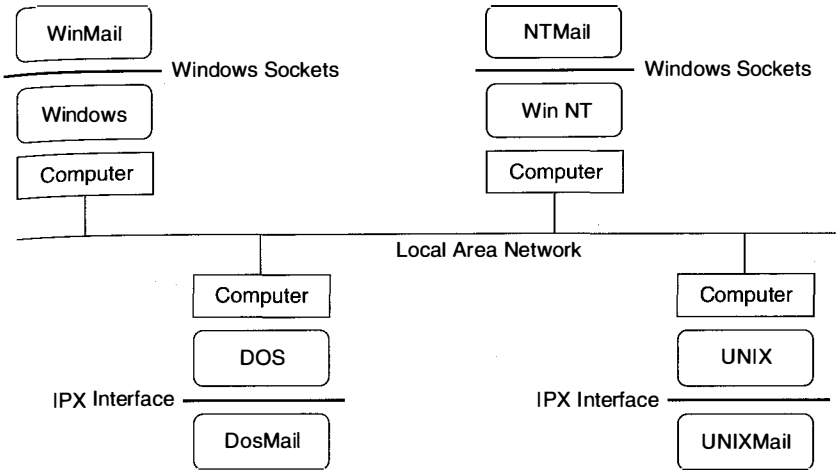


Figure 8-5. *Interoperability Among Multiple Protocols*

8.1.1 Elements of an NCB

An NCB structure is passed to a NetBIOS programming interface to execute a single NetBIOS command such as sending a data packet from one process to another. The NetBIOS command is specified in the **ncb_command** field of the NCB. Depending on the command, one or more fields of the NCB structure may be filled with appropriate values. For example to send data, the caller will put the pointer to the data buffer in the **ncb_buffer** field of NCB and length of data (in bytes) in the **ncb_length** field. Once an NCB has been properly filled, an operating-system-specific NetBIOS function is called, a process often referred to as "submitting an NCB." A NetBIOS function can return success or failure return codes via the **ncb_retcode** field. On successful completion of a command, the NCB may contain data returned by the NetBIOS driver. For example, the NetBIOS driver will return a pointer to received data via the ncb_buffer field of an NCB when the caller issues a receive data command.

All NetBIOS services can be used by executing a NetBIOS command. Table 8-1 shows the commands categorized by the service. (The table shows only the NetBIOS commands in NetBIOS 3.0 specifications.) Almost all NetBIOS commands can be executed synchronously or asynchronously. When a caller submits a synchronous command, the calling thread blocks indefinitely until the command completes or aborts with an error. To avoid blocking on execution of a NetBIOS command, a caller can use an asynchronous command, which causes the NetBIOS function to return immediately. The caller can then periodically check the status of the pending command by examining the **ncb_cmd_cplt** field of the NCB. Note that each asynchronous command is created by OR-ing 128 (80h) with its corresponding synchronous command.

Table 8-1. NetBIOS Commands Categorized by Service

NetBIOS Command	*Synchronous Code*	*Asynchronous Code*
General-purpose services		
NCB.RESET	32H	N/A
NCB.ADAPTOR.STATUS	33H	B3H
NCB.CANCEL	35H	N/A
NCB.UNLINK	70H	N/A
Name management services		
NCB.ADD.NAME	30H	B0H
NCB.DELETE.NAME	31H	B1H
NCB.ADD.GROUP.NAME	36H	B6H
NCB.FIND.NAME	78H	F8H
Session services		
NCB.CALL	10H	90H
NCB.LISTEN	11H	91H
NCB.HANGUP	12H	92H
NCB.SEND	14H	94H
NCB.RECEIVE	15H	95H
NCB.RECEIVE.ANY	16H	96H
NCB.CHAIN.SEND	17H	97H
NCB.SESSION.STATUS	34H	B4H
Datagram services		
NCB.SEND.DATAGRAM	20H	A0H
NCB.RECEIVE.DATAGRAM	21H	A1H
NCB.SEND.BROADCAST. DATAGRAM	22H	A2H
NCB.RECEIVE.BROADCAST. DATAGRAM	23H	A3H

The NetBIOS driver notes the success or failure of the NetBIOS command via the ncb_retcode field. This field can contain zero on success and a nonzero number in the range [1h, FFh] in case of failure. A special return code of 255 (FFh) is returned by the driver in the ncb_retcode field when an asynchronous command has been submitted that cannot be completed immediately.

A program seeking to communicate to a peer needs to register its name by specifying a proper command via the ncb_command field and its logical NetBIOS name via the **ncb_name** field. When it receives a request, the NetBIOS driver searches to see if the name has already been used. If it has not, it is added to the adapter name table, and the name number is returned via the **ncb_num** field.

In a client/server environment, a client initiates the actions to set up a session with the server; the server merely replies to the client. This means that the server needs a well-known name so clients can locate it. The client name is not important in this communication because the server rarely needs to locate a given client by name. Thus, a client can choose an arbitrary name (Client0, Client1, and so on). The issue of naming will become important when multiple instances of a client can run in the network. If the first instance of the client application registers a single well-known name, the second (or nth) instance of any NetBIOS application cannot use the same name (this is referred to as name clash). Because client names can be any arbitrary 16-byte-long string, each client application can attempt to register a well-known name; on failure, it can modify the name programmatically until a name is successfully registered. (See the following pseudocode for an example.)

```
#define CLIENT_NAME "Client"
char szClientName [ 16 ] = {CLIENT_NAME);
for (i = 0; i < M ; i++)
{
    err= RegisterName ( szClientName );
    if (err)
        sprintf(szClientName,"%s%d", CLIENT_NAME, i);
    else
        break;      // successful registration of name
}
```

Another process employed by some applications (for example, Microsoft MSMail™) is always to use the special name number (1) without registering its name in the name table. The name number slot (1) is always occupied by the workstation name and is guaranteed to be present whenever the network is functional. We will revisit this issue when discussing the NCB.RESET command.

To use guaranteed delivery of data packets, two processes—one the listener and the other a caller—set up a virtual circuit between each other. The listener specifies the caller's NetBIOS name via the ncb_name field, and the caller specifies the callee via the **ncb_callname** field of an NCB. When a session has been set up between the two processes, both receive a unique number local to their workstation via the **ncb_lsn** field of an NCB. This number, granted by the NetBIOS driver, is called the **local session number (LSN)**. In subsequent communications between the two processes, both sides use the LSN rather than full names to identify each other. The LSN value can range from 1 through 254 (0 and 255 are never assigned). In the MS-DOS and Windows environment, an LSN is primarily an index into the session table maintained by each workstation per network adapter card. In Windows NT, the LSN is assigned on a per-process basis; thus, the total number of LSNs per system can be more than 254. In Windows NT, the LSN is a handle that identifies to the transport protocol the appropriate destination. The values are recycled when an existing session is disconnected. Figure 8-6 illustrates the concepts of name numbers and local session numbers. Sue (name number 2)

communicates with Jim (name number 6 on Workstation 2). The LSN for the session on Workstation 1 is 1; the LSN on Workstation 2 is 39.

The **ncb_lana_num** field is used to specify the **local area network adapter (LANA)** number in a workstation. The numbering normally starts with 0, although this can be overridden by using the Network Control Panel Applet. Thus, usually nth adapter is specified by the LANA of N-1. Also, when transport protocols such as TCP/IP or IPX support NetBIOS programming interfaces, there is usually one LANA for all adapters.

The **ncb_rto** and **ncb_sto** fields are used to specify the send time-out and receive time-out values respectively. The time-out value for the session is established at session setup time by each end and remains in effect for the duration of the session. Of course, each session partner determines the time-out value for its end of the session. The time-out values are specified in 500-millisecond (1/2 second) increments for each session.[3] A time-out value of 00h causes the NetBIOS command to wait until it completes. Applications should not block NetBIOS commands, which may complete after an indefinite period of time. When the NetBIOS interface of a non-NetBIOS transport protocol (for example, XNS or IPX) is being used, these transports may impose additional time-outs. In such cases, the underlying transport may time out before the time-outs specified via the ncb_rto and ncb_sto fields.

Applications can use asynchronous commands to avoid blocking on network I/O commands. When an asynchronous command is submitted, the NetBIOS function immediately returns the NCB with both its ncb_retcode

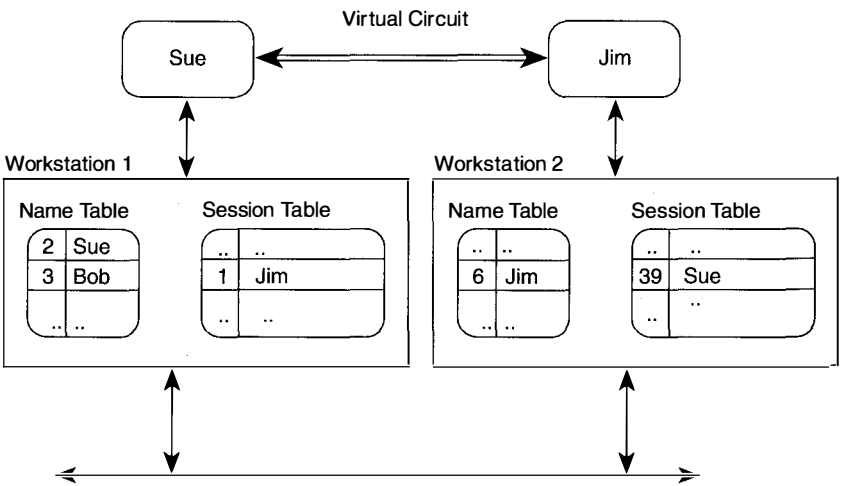


Figure 8-6. *Name Numbers and Local Session Numbers*

3. Because ncb_rto and ncb_sto are byte fields, their maximum value is limited to 255, which allows an application to specify a maximum time-out value of about 127 seconds.

and ncb_cmd_cplt fields filled with 255 (FFh). The NetBIOS driver saves a pointer to the NCB so it can update the ncb_cmd_cplt field of the NCB at the completion of the asynchronous command. Thus, an application can periodically check the status of the ncb_cmd_cplt field of the submitted NCB to detect completion of an asynchronous command. On completion or error, the ncb_cmd_cplt field is set to the final return code other than FFh. An application should not affect an NCB data structure from the time its address has been passed to the NetBIOS function for executing an asynchronous command until the asynchronous command completes or aborts. In the Windows/MS-DOS environment, the NCB must be locked in memory until the asynchronous command completes.

An asynchronous command remains pending until it completes or is canceled. The time-out values are determined by send/receive time-outs established at session setup time and are no different for asynchronous commands than for synchronous commands. Of course, the advantage of using asynchronous commands is that an application does not block on a command. Furthermore, an asynchronous command can be canceled using the NetBIOS cancel (NCB.CANCEL) command, which gives great flexibility to the application. Finally, an application can submit multiple asynchronous commands. Each asynchronous command must have its own NCB and associated buffers, if any. This powerful feature allows users to implement applications that can service multiple network operations asynchronously. Most applications use NetBIOS to communicate with peer processes in which the protocols are simple (for example, each send-of-data followed by wait-for-reply). On the other hand, one can envision a NetBIOS-based server that can service more than one NetBIOS-based client. In such a case, the server will use multiple asynchronous commands to set up sessions with, and service requests of, multiple clients. Later in this chapter, you will see examples of designing such NetBIOS servers using multiple asynchronous commands.

8.2 NETBIOS SUPPORT IN WINDOWS NT

Windows NT provides NetBIOS programming support to MS-DOS, Windows (16-bit), OS/2, and Win32 applications. More specifically, Windows NT supports the INT 5Ch interface for MS-DOS programs, the NetBiosCall() WIN16 API for Windows applications, NetBiosSubmit() API for OS/2 applications, and Netbios() API for Win32 applications. Notice in Figure 8-7 that MS-DOS and Win16 applications get NetBIOS support via the REDIR.EXE TSR. The REDIR.EXE is loaded via the autoexec.nt whenever a VDM starts. It then hooks the INT 5Ch interrupt and processes it by eventually calling the Netbios() Win32 API. Similarly, the OS/2 subsystem processes the NetBiosSubmit() API (as well as the **NetBiosOpen()** and **NetBiosClose()** APIs) and calls the Netbios() Win32 API.

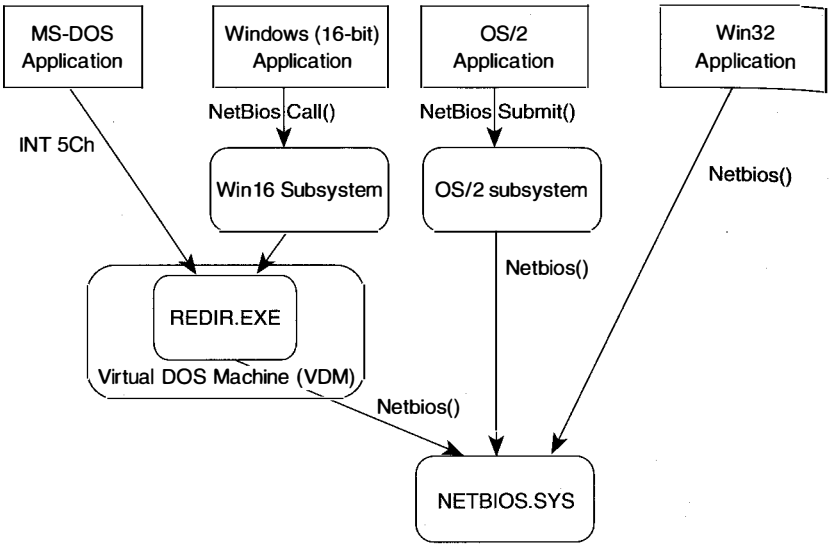


Figure 8-7. *NetBIOS Support in Windows NT*

All Win32 applications use the Netbios() Win32 API to submit NCB to the underlying system. The Netbios() API interacts with the Windows NT NetBIOS driver (NETBIOS.SYS) instead of directly communicating with the transport protocols. By default, the NetBIOS driver uses the NetBEUI protocol. However, a user can set up a Windows NT workstation to use other transport protocols, such as TCP/IP or IPX. Additionally, the Windows NT networking architecture allows a workstation to support multiple adapters and protocols.

In Windows NT, each transport protocol is assigned a LANA at setup time. The LANA normally begins with 0 and is incremented by the Windows NT setup program. Indeed, an administrator can change the LANA sequences using the Network Control Applet. Some protocols (for example, NetBEUI) choose to use one LANA per adapter; others (for example, IPX) may use the same LANA for all adapters in a system. In all applications, however, the same LANA can never be used to describe two transports. Thus, an application can choose one transport and adapter combination by specifying the correct LANA in the ncb_lana_num field. In Windows NT, a program can determine the valid LANAs (and their characteristics) at run time using a nonstandard NetBIOS command (NCB.ENUM) (discussed in detail in Appendix A).

Figure 8-8 shows the concept of LANA in the Windows NT environment. Workstation A has only one network card and sports the NetBEUI transport stack. It has one valid LANA number. Workstation B has two network adapters and three transport protocols, and it has four valid LANA numbers (0, 1, 2, and 3).

Because Windows NT is backward compatible, NetBIOS-based MS-DOS, Win16, and OS/2 applications should function on it with no modification. I

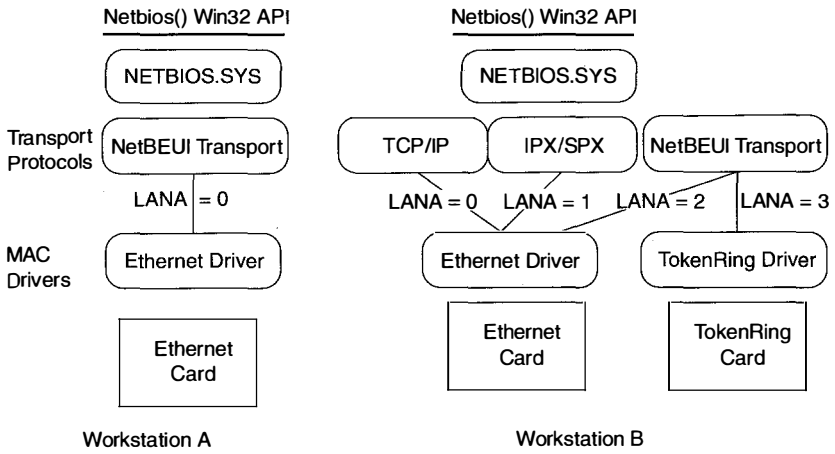


Figure 8-8. *Relationship of Protocols with the NetBIOS() API*

will not discuss non-Win32 issues any further; I assume readers trying to execute their non-Win32 applications have access to references pertinent to that environment [for example, IBM 84, Sinha92b, Schwaderer 88]. I recommend that new NetBIOS applications should use the Netbios() Win32 API because it is supported by Windows NT and Windows95 and because the Netbios() API provides new features, such as the use of Win32 events.

8.3 NETBIOS PROGRAMMING IN WINDOWS NT

This section focuses on NetBIOS programming and its services, which can be used for IPC. If you are familiar with NetBIOS programming in other environments (such as OS/2 and MS-DOS), you may use this section to refresh your knowledge in the context of Windows NT. In particular, I delve into the details of NetBIOS name management service, datagram service, and session service.

8.3.1 Setting the NetBIOS Application Environment

On Windows NT, an application must set its NetBIOS environment prior to using the services offered by the NetBIOS system (this is also true for other multitasking operating systems such as OS/2) by issuing the NCB.RESET command at the outset of the application. An application can completely reset its environment by calling NCB.RESET at any time during execution. Although an application can set up its programming environment in a number of ways, we will follow the recommendations of the Win32 API manuals.

A program can use the NCB.RESET command to set the maximum size of the name table, session table, maximum NetBIOS commands, and use of name number 1. In Windows NT, these limits are on a per-process basis; for

example, a process (Mail) can set its name table size to 128, and another process (WordProcessor) can choose the default size for its name table. Recall that a name table contains the logical names for communicating processes, while name numbers are the indexes into the table. In Windows NT, the maximum size of a name table associated with a process for a given LANA is 255. Because the 0 and 255 are reserved name numbers, each process can register up to 253 names. Similarly, a process can have up to a maximum of 253 sessions per LANA, with 0 and 255 as reserved session numbers; for example, in a Windows NT workstation set up with the NetBEUI protocol over two network adapters, each process will be able to register 506 unique names and set up 506 sessions (253 names per LANA and 253 sessions per LANA).

The NetBIOS specifications prescribe that a program can change the number of maximum pending asynchronous NetBIOS commands via the NCB.RESET command. However, in Windows NT, this number is ignored.

Via the NCB.RESET command, a process can request a name number (1) to be issued to it (and thus allow it to register up to 254 names). This feature is used by some client applications (such as Microsoft MSMail) to avoid registering their own logical names and thus avoid encountering name clashes. This process works because the name number (1) is always granted to the logical name of the workstation (also referred to as the computer name). Thus, the name table for every process already has the computer name in name number (1). Additionally, in a client/server environment, the client's name is not important because the client always locates and calls the server, which simply replies to the client. Another benefit of using the reserved name number (1) is that the application reduces start-up time because it does not need to register its own name by calling the NCB.ADD.NAME command.

A program first sets the appropriate fields of an NCB and then calls the Netbios() function to submit an NCB.RESET command. It needs to set the ncb_command to NCB.RESET and ncb_lana_num to the network adapter (or protocol) that it will use. The value in ncb_lsn dictates how the ncb_callname field is used to set upper limits (see Table 8-2).

Table 8-2. *Setting NetBIOS-Related Resources*

	ncb_lsn: 0	***ncb_lsn: Nonzero***
Resources	Free existing resources; allocate new resources	Free existing resources
ncb_callname[0]	Max sessions	Ignored
ncb_callname[1]	Max NetBIOS command	Ignored
ncb_callname[2]	Max Name Table Size	Ignored
ncb_callname[3]	Usage of name number 1	Ignored

The following code fragment shows setting the program NetBIOS environment by issuing the NCB.RESET command. Note that an application may not get all the resources it requests. The NetBIOS driver returns the actual resources allocated to the program via the ncb.name field, as shown in the code. The code fragments additionally demonstrate requesting name number (1) by using the fUseNameNumberOne parameter.

```
//
// FILE: %BOOK%\netbios\reset.c
//
#include <windows.h>
#include <stdio.h>

#define MAX_SESSIONS  12
#define MAX_NAMES       12
static BYTE bLanaNum = 0;
static BOOL fUseNameNumberOne = FALSE;
BYTE Reset ()
{
    BYTE bRc;
    NCB  ncb;
    // Clean out the NCB
    memset (&ncb, 0x00, sizeof(NCB));
    // Call NCB.RESET
    ncb.ncb_command = NCBRESET;
    // Reset LAN Adaptor
    ncb.ncb_lana_num   = bLanaNum;
    // Free all resources
    ncb.ncb_lsn = 0;
    // Set Max Sessions and Names
    ncb.ncb_callname[0] = MAX_SESSIONS;
    ncb.ncb_callname[2] = MAX_NAMES;
    ncb.ncb_callname[3] = fUseNameNumberOne;

    if ((bRc = Netbios (&ncb)) != NRC_GOODRET)
        printf("Reset Failed: %x\n", bRc);
    else
    {
        printf("Reset Completed\n");
        printf("Resources::"\
               "Actual Sessions: 0x%X Actual Commands: 0x%X Actual Names:
               0x%X\n", ncb.ncb_name[0], ncb.ncb_name[1], ncb.ncb_name[2]);
        printf("Application does %s Name Number 1\n",
              (ncb.ncb_name[3]) ? "have" : "not have");
    }
    return(bRc);
}
//
//  Main - Demonstrates usage of NCB.RESET command
//  Usage - reset <LAN adapter number> <fUseNameNumberOne>
//
int _cdecl main (int argc, char ** argv)
{
    argc--;
    argv++;
```

```
    if (argc >= 1)
        {
                bLanaNum = (BYTE) atoi(argv [0]);
                argc--;
                argv++;
                if (argc)
                     fUseNameNumberOne = TRUE;
        }
        // Reset the chosen adapter
       Reset();
       return 0;
}
```

To summarize, all Windows NT processes must call the NCB.RESET command prior to issuing any other NetBIOS commands. This command is used to reserve the maximum number of names and sessions that a process might need during its lifetime.

8.3.2 Name Management Services

Processes (or threads) using NetBIOS must register their well-known logical name with the underlying system prior to using data transfer services. To establish end-to-end, datagram-based, or session-based communication, both parties must register their respective unique names with the NetBIOS system. As the name suggests, no two entities resident on the network can have the same name. On the other hand, a process can register itself to be part of a group by specifying the group name. Additionally, a collection of entities can share a group name, which allows for one-to-many datagram-based group communication. Group names are not normally used for session-based data transfer channels.

Every character in the case-sensitive 16-byte NetBIOS logical name is significant.[4] By convention, the NetBIOS names are padded with an ASCII space (20h) character. A name must not contain "IBM" in the first 3 bytes and 00h through 1Fh in the sixteenth byte. Most NetBIOS software vendors, such as Microsoft and IBM, use the sixteenth byte to indicate the type of NetBIOS entity, so only 15 bytes are useful. Table 8-3 shows some of the encoding convention.

A NetBIOS name is a logical name used by applications to locate each other. A transport-specific address (for example, 11.1.121.124 in TCP/IP or 0001:000012 for IPX/SPX) allows the transports to locate their counterpart on the network. NetBIOS protocols such as NetBEUI have one-to-one correspondence between the name and the transport-specific address; that is, there is no distinction between the NetBIOS name and the NetBIOS transport address. Non-NetBIOS transports, however, maintain the distinction between the NetBIOS name and the transport-specific address, so they must

4. This is somewhat misleading. Although the bytes are not case sensitive, the characters are. The system uses the OEM character set page to convert characters to the bytes used in the name.

Table 8-3. Significance of the Sixteenth Byte in a NetBIOS Name

Value in the Sixteenth Byte of NetBIOS Name	*Significance*
00h	SMB-based redirector name AKA computer name
03h	User name
05h	Forwarded name
20h	Server name

handle the mapping between the NetBIOS name and the transport-specific addresses. For example, a **WINS (Windows Internetwork Name Service)** server is used to map the NetBIOS names to their corresponding TCP/IP addresses (Figure 8-9).

NetBIOS offers a flat address space that spans the entire network—be it a single LAN or a WAN composed of interconnected LANs. To put the restriction of flat address space into proper perspective, imagine an organization with offices spanning the globe and all computers connected in a tightly integrated network. Should this organization adopt NetBIOS as the primary network communication protocol, no two processes in the entire organization can have the same logical (NetBIOS) name, such as MEDIA_SERVER. By the same token, when workstations are set up with the NetBEUI protocol, no two workstations can have the same computer name because the Windows NT redirector registers a variant of the computer name as its NetBIOS logical name.

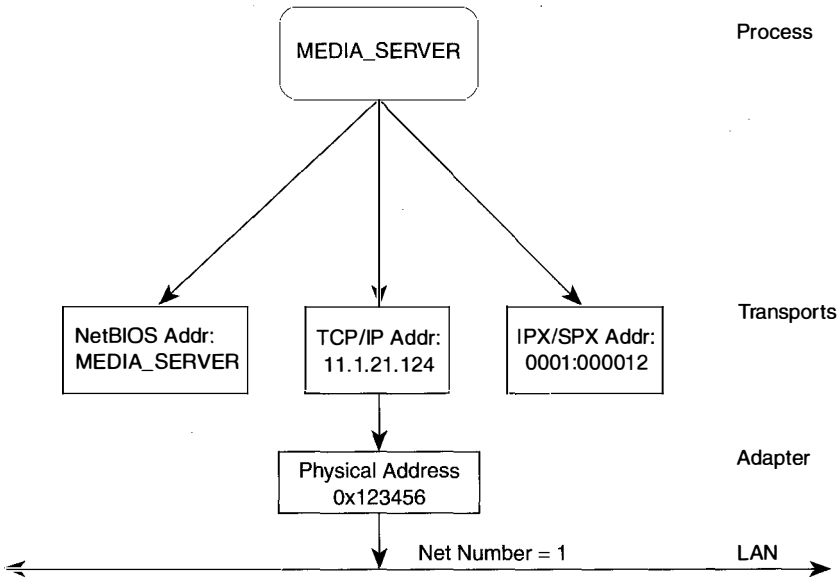


Figure 8-9. NetBIOS Names and Transport Addresses

The NetBIOS command **NCB.ADD.NAME** is used to register a unique name, and **NCB.ADD.GROUP.NAME** is used to register a group name. **NCB.DELETE.NAME** is used to unregister NetBIOS names. When a program tries to register a unique name, the underlying NetBIOS driver sends a broadcast message on the network querying for any preexisting owners of the name. If a workstation on the network believes it owns the name, it will send a response to the originator of the query, and the NCB.ADD.NAME command will fail with "duplicate name" error. If no responses are received by the inquiring NetBIOS driver within a system-defined time limit, the NetBIOS name is added to the name table. In Windows NT, each process has its own name table, thereby allowing each process to add up to 254 names to its name table. Each process can add names as long as the names put into its own name table do not duplicate an existing name on the network even if it is added by another process on the same machine. This is in marked contrast with MS-DOS/Windows environments, where the name table is a system-wide resource allowing a workstation to have a maximum of 254 names registered in it. Once a name has been added to the name table, the NetBIOS driver returns the name number in the ncb_num field. This number can be used for subsequent NetBIOS commands.

The following code fragment shows how to use the NCB.ADD.NAME and NCB.DELETE.NAME commands. The program takes the first command parameter as a NetBIOS name and adds the name to the name table using the synchronous NCB.ADD.NAME command. On successful completion of NCB.ADD.NAME, the NetBIOS driver returns a name number in the ncb_num field, which is used to remove the name from the name table using NCB.DELETE.NAME. Note that the Reset() function is assumed to be the same as the one shown in the previous code fragment.


```
//
// FILE: %BOOK%\netbios\addname.c
//
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>
BYTE bLanaNum = 0;
//
// AddName - Adds a NetBIOS name into name table.
//
BYTE AddName ( PCHAR pchName, BYTE *pbNameNum )
{
     BYTE bRc = NRC_BUFLEN;
     NCB  ncb;
     if (( pchName ) && (strlen(pchName) <= NCBNAMSZ))
     {
          // Set all fields to zero
          memset (&ncb, 0x00, sizeof(NCB));
          // Command = NCB.ADD.NAME
          ncb.ncb_command = NCBADDNAME;
          // Blank pad the NetBIOS name
```

```
        memset (&ncb.ncb_name, 0x20, NCBNAMSZ);
        memcpy (ncb.ncb_name, pchName, strlen(pchName));
        // Use first Lan Adaptor
        ncb.ncb_lana_num = bLanaNum;
        // Submit the command
        bRc = Netbios (&ncb);
        // Parse the success and most common error codes.
        switch (bRc)
        {
        case NRC_GOODRET:
                printf("Name added successfully. \n");
                printf("Name Number: %x\n", ncb.ncb_num);
                *pbNameNum = ncb.ncb_num;
                break;
        case NRC_DUPNAME:
                printf("Duplicate Name. Name present on network\n");
                break;
        case NRC_NAMTFUL:
                printf("Name table is full\n");
                break;
        case NRC_NOWILD:
                printf("Cannot use * in Name\n");
                break;
        default: printf("Add Name Error: %x\n", bRc);
        }
    }
    return bRc;
}

//
// DeleteName - Remove a NetBIOS name from the name table.
//
BYTE DeleteName ( PCHAR pchName, BYTE bNameNum )
{
    BYTE bRc;
    NCB  ncb;
    // Set all fields to zero
    memset (&ncb, 0x00, sizeof(NCB));
    // Command = NCB.DELETE.NAME
    ncb.ncb_command = NCBDELNAME;
    // Set the Name Number
    ncb.ncb_num = bNameNum;
    // Copy the name in place
    memset (&ncb.ncb_name, 0x20, NCBNAMSZ);
    memcpy (ncb.ncb_name, pchName, strlen(pchName));

    // Set the LANA
    ncb.ncb_lana_num = bLanaNum;
    // Submit the command
    bRc = Netbios ( &ncb);
    // Parse the success and most common error codes.
    switch (bRc)
    {
    case NRC_GOODRET:
        printf("Name Removed successfully\n");
        break;
    case NRC_ILLNN:
```

```
            printf("Illegal Name Number: %x\n", bNameNum);
            break;
        case NRC_ACTSES:
            printf("Name deregistered but has active sessions\n");
            break;
        case NRC_NOWILD:
            printf("Name not found.\n");
            break;
        default:
            printf("Delete Name Error: %x\n", bRc);
        }
        return bRc;
}
//
// Main - Adds a NetBIOS name into name table and then removes it.
// Usage - addname <name> <lana>
//
int _cdecl main (int argc, char **argv)
{
        BYTE  bRc, bNameNum;
        argc--;
        argv++;
        if (argc)
        {
            // Check if Lan Adapter number has been supplied
            if (argc >=2)
            {
                    bLanaNum = (BYTE) atoi(argv[1]);
            }
            // Reset
            if ((bRc = Reset()) != NRC_GOODRET)
            {
                    return(1);
            }
            printf("Adding [%s] to Name Table\n", *argv);
            if (( bRc = AddName (*argv, &bNameNum)) == NRC_GOODRET)
            {
                    printf("Deleteing [%s] from Name Table\n", *argv);
                    bRc = DeleteName ( *argv, bNameNum );
            }
        }
        else
            printf("addname <name to add to name table> <lana number>\n");
        return 0;
}
```

Applications can use the **NCB.FIND.NAME** NetBIOS command to enumerate the workstations that have registered a particular name in their name table. The command causes the NetBIOS driver to send a query on the network in much the same way as when a name is being registered. At the end of the system-dependent time limit, the NetBIOS driver returns a list of workstations that responded to the query. If no workstations respond, the command returns "time out" (5h) error. The number of workstations responding to a query also depends on the type of the name. If the name is unique,

then at most one workstation will respond. On the other hand, one or more workstations will respond when a group name is queried for.

The command is useful in locating the origin of a unique name, and it can be used to create and then maintain a list of members of a group. The caller must specify (via the ncb_buffer field) a large enough buffer to contain the returned data. On successful completion of the NCB.FIND.NAME command, the caller-supplied buffer is filled with responses in a special format, which follows.

```
// Buffer returned by NCB.FIND.NAME contains Find name header block
// followed by
// one or more Find name buffers
//
typedef struct _FIND_NAME_HEADER { /* fnh */
    WORD  node_count;
    UCHAR reserved;
    UCHAR unique_group;
} FIND_NAME_HEADER;
typedef struct _FIND_NAME_BUFFER { /* fnb */
    UCHAR length;
    UCHAR access_control;
    UCHAR frame_control;
    UCHAR destination_addr[6];
    UCHAR source_addr[6];
    UCHAR routing_info[18];
} FIND_NAME_BUFFER;
```

The buffer contains FIND_NAME_HEADER followed by one or more FIND_NAME_BUFFER structures. The number of FIND_NAME_BUFFER structures following the header is indicated by the value in the node_count field, whereas the value in unique_group (0 for unique, 1 for group) indicates the type of name. Each FIND_NAME_BUFFER contains the valid buffer length in the length field because the whole buffer may not have valid information, even if each buffer is 33 bytes long. Most of the addressing and routing information present in the buffer is useful at the NetBIOS protocol level and has little use for applications. The source_addr field of FIND_NAME_BUFFER contains the physical address of the network adapter when the name has been registered. The destination_addr field contains the address of the adapter where the name query was issued. I will shortly show the result of calling the NCB.FIND.NAME command. For further information, see [IBM 84].

The following code fragment shows how to use the NCB.FIND.NAME command. It takes a name and tries to locate it. On a NetBIOS network, LAN software vendors and applications simply blank (20h) pad a name, with the exception that a special character is put in the sixteenth byte to segregate distinct entities. For example, Windows NT server names have 20h in the sixteenth byte, whereas group names, such as Domain Name, have 00 in the sixteenth byte. The code fragment thus allows a user to put a special

character in the sixteenth byte. Furthermore, a user can specify a LAN adapter on the command line. Note that the Reset() function has not been shown since it is the same as in previous code fragments. I have also shown the output from two runs of a findname program locating a unique name (ALOKSDEV) and a group name (ALOKS-DOMAIN) that is also the Windows NT domain name.

```
// FILE: See %BOOK%\netbios\findname.c
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
//
// PrintNameBuffer - Prints the contents of the buffer returned on
//                   successful completion of NCB.FIND.NAME
//
static BYTE bLanaNum = 0;
void PrintNameBuffer (UCHAR * pBuffer)
{
    FIND_NAME_HEADER *pfh;
    FIND_NAME_BUFFER *pfb;
    WORD i, j, Count;
    // Print Find Name Header information
    pfh = (FIND_NAME_HEADER *) pBuffer;
    Count = pfh->node_count;
    printf("Node Count: %d Name Type: %s\n",
                Count, (pfh->unique_group)?"Group ":"Unique");


    // Print Find Name Buffer Information
    pfb = (FIND_NAME_BUFFER *) ((PCHAR)pBuffer + sizeof(FIND_NAME_HEADER));
    for (i = 0; i< Count; i++)
    {
        printf("Length: %x Access Control: %x Frame Control: %x\n",
                pfb->length, pfb->access_control, pfb->frame_control);
        printf("Source Address: ");
        for ( j = 0; j< 6; j++)
                printf("%2.2x", pfb->source_addr[j]);
        printf("\nDestination Address: ");
        for ( j = 0; j< 6; j++)
                printf("%2.2x", pfb->destination_addr[j]);
        printf("\nRouting Information: ");
        for ( j = 0; j< 18; j++)
                printf("%2.2x", pfb->routing_info[j]);
        printf("\n\n");
        // Next Buffer
        pfb++;
    }
}

//
// FindName - Issues NCB.FIND.NAME on the supplied name
//
#define NOS_MAX_NAMES 20
static UCHAR aucNcbBuffer [sizeof(FIND_NAME_HEADER)
                        + NOS_MAX_NAMES * sizeof(FIND_NAME_BUFFER)];
```

```
BYTE FindName(UCHAR *puchName)
{
    BYTE bRc;
    NCB  ncb;
    // Clean the NCB
    memset (&ncb, 0x00, sizeof(NCB));
    // call NCB.FIND.NAME
    ncb.ncb_command = NCBFINDNAME;
    // Adapter to use
    ncb.ncb_lana_num = bLanaNum;
    // Name to locate
    memcpy (ncb.ncb_callname, puchName, NCBNAMSZ);

    // Specify the buffer to collect information
    ncb.ncb_buffer = aucNcbBuffer;
    ncb.ncb_length = sizeof(aucNcbBuffer);

    // Submit a synchronous command
    bRc = Netbios ( &ncb );
    switch (bRc)
    {
    case NRC_GOODRET:
            printf("Find Name successful\n");
            PrintNameBuffer ( aucNcbBuffer );
            break;
    case NRC_CMDTMO:
            printf("Find Name timed out.Maybe no such name exists\n");
            break;
    case NRC_BUFLEN:
            printf("Illegal buffer length.\n");
            break;
    default:
            printf("Find Name Error: %x\n", bRc);
     }

    return(bRc);
}
//
// Main - Locates presence of a NetBIOS name and prints
          addessing information.
// Usage - findname <name> <lan adapter number> <16th Byte>
//
int _cdecl main (int argc, char **argv)
{
    BYTE  bRc, bSixteen;
    WORD  dwLength;
    UCHAR achName [ NCBNAMSZ ];
    argc--;
    argv++;
    if ((argc) && (strlen(*argv) <= NCBNAMSZ))
    {
        // argv[0] = Name
        printf("Locating %s\n", argv[0]);
        dwLength = strlen(argv[0]
        memset(achName, 0x20,  NCBNAMSZ);
        memcpy(achName, argv[0], dwLength);
```

```
        // argv[1] = Lana
        if (argc >= 2)
        {
                bLanaNum = (BYTE) atoi(argv[1]);
        }
        printf("Lana Adaptor: %d\n", bLanaNum);
        // argv[2] = 16th Byte
        if (argc >= 3)
        {
                // Pick up the byte for 16th byte
                bSixteen = (BYTE) atoi(argv[2]);
                achName [15] = bSixteen;
                dwLength = NCBNAMSZ;
                printf("Sixteen-th byte: 0x%x\n", bSixteen);
        }
        if (Reset() == NRC_GOODRET)
        {
                bRc = FindName(achName);
        }
    }
    else
        printf("findname <name> <lana> <16th byte>\n");
     return 0;
}
//
// Output from findname on argument: ALOKSDEV
// Note: 1) ALOKSDEV is Windows NT computer name
//       2) Adapter in ALOKSDEV is 00dd010f394a
//       3) Program run on ALOKSTEST machine whose MAC address is
           02608c9fed74
//       4) Each machine is configured with NetBEUI protocol on a
//          single Ethernet adapter (that is, NetBEUI/Ethernet LANA = 0)
//
Locating ALOKSDOS
Lana Adaptor: 0
Find Name successful
Node Count: 1 Name Type: Unique
Length: e Access Control: 0 Frame Control: 0
Source Address:           00dd010f394a
Destination Address:      02608c9fed74
Routing Information:      ff240000000062d0fc000000000000000000
//
// Output from findname on argument: ALOKS-DOMAIN
// Note:     1) ALOKS-DOMAIN is a Windows NT Domain name.
//           2) Three machines in this domain - ALOKSDEV, ALOKSTEST,ALOKSNT
//           3) Program was run on ALOKSTEST
Locating ALOKS-DOMAIN
Lana Adapter:   0
Sixteenth byte: 0x0
Find Name successful

Node Count: 3 Name Type: Group
Length: e Access Control: 0 Frame Control: 0
Source Address:           02608c9fed74
Destination Address:      02608c9fed74  <-- This is from ALOKSTEST to self.
Routing Information:      ff240000000062d0fc000000000000000000
```

```
Length: e Access Control: 0 Frame Control: 0
Source Address:           00dd010f394a
Destination Address:      02608c9fed74     <-- This is from ALOKSDEV
Routing Information:      0000000000000000000000009460bdffdc60

Length: e Access Control: 0 Frame Control: 0
Source Address:           02608c6cf5de     <-- Third machine is ALOKSNT
Destination Address:      02608c9fed74
Routing Information:      00000000000000000000000000000000000000
```

8.3.3 Exploiting Session Service

This section examines sending and receiving data packets between two processes by using the NetBIOS session services. Session service allows two NetBIOS entities to establish a virtual circuit between themselves for using the ordered and guaranteed data delivery. This implies that the NetBIOS layer manages error recovery and other transmission control operations. (Readers familiar with Windows Sockets will realize that this is similar to WinSock stream service.)

To use session services, one process has to be a "listener" and the other a "caller." Once a session has been established between the two processes, both ends can send and receive data packets. Traditionally, the server in a CSS plays the role of the listener, and one or more clients call the server. (Hence, in the following discussion, listener or server will be used to refer to one entity and caller or client for the other.)

A server issues the **NCB.LISTEN** command to indicate that it is ready to receive calls from clients. The server specifies its own name (via the ncb_name field) and the caller's name (via the ncb_callname field). An asterisk (*) in the ncb_callname field indicates that the server will accept calls from any caller. When the NCB.LISTEN NetBIOS command is submitted, the server blocks waiting for the specified client (or any caller depending on the value of ncb_callname). Alternatively, servers can use an asynchronous version of the NCB.LISTEN command so the listening thread in the server does not block.

The server can specify the time-out values associated with data transmission at this time: data packet send time-out values via the ncb_sto field and data packet receive time-out values via the ncb_rto field. These values take effect if and when a session is established with a caller. A 0 value in these fields indicates that a send, or receive, command should not time out. Recall from the discussion about the fields of NCB that the time-out values are specified in 500-millisecond increments. Also, some transport protocols (such as TCP/IP) may have additional time-outs to avoid indefinite retransmissions when the remote machine has no receive pending.

A client typically calls a server by using the **NCB.CALL** command. It specifies its own name (via the ncb_name field) and the server name (via the ncb_callname field). Similar to the NCB.LISTEN command, an NCB.CALL command blocks the client calling thread until the NetBIOS times out looking for a server with the specified name. To eliminate blocking the calling thread, most NetBIOS clients use an asynchronous version of the NCB.CALL command. A client can specify the time-out values (via ncb_rto and ncb_sto) associated with data transmission when and if a session is established with a server. Note that an NCB.CALL command times out after a system-defined time-out period should there be no matching NCB.LISTEN (from a server). Furthermore, it is possible to establish more than one session between two NetBIOS entities so long as each NCB.LISTEN is matched with a corresponding NCB.CALL. Figure 8-10 shows the relationship between the client and the server.

When a server with a specified name is located, by means similar to that discussed in the section on NCB.FIND.NAME, the call command succeeds; a virtual circuit or session is now established between the caller and the listener. Both commands return to their respective programs with local session numbers in the ncb_lsn field. Each LSN is independent of the other and local to the executing process. Once a session has been established between the client and the server, both can use the same session to send and receive data packets using some application-level protocol. In a traditional CSS, a client initiates a dialog and sends requests. On receiving a request, a server sends back responses. The length of messages sent, or received, can range from 0 to 65,535 bytes. A 0-byte message is sometimes used by applications to acknowledge receipt of a message.

To send a data packet between two NetBIOS entities that have established a session, the sender uses the **NCB.SEND** command. The sender specifies the local session number (via the ncb_lsn field), a pointer to the data

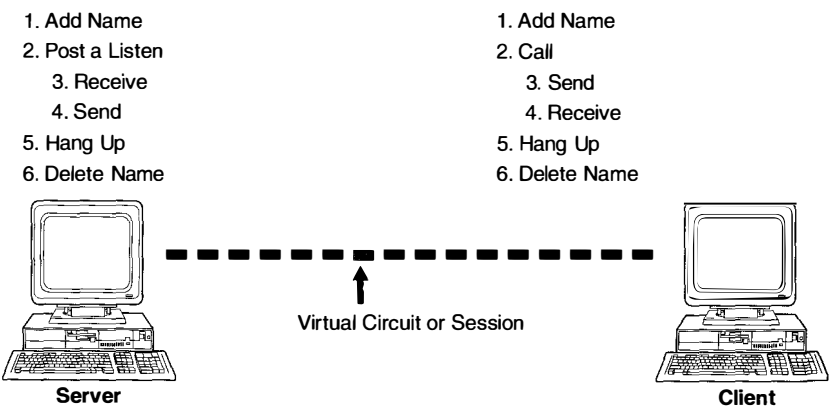


Figure 8-10. *Client and Server Setting Up a Session*

buffer (via the ncb_buffer field), and the length of the buffer (via the ncb_length field) in bytes. In its generic synchronous form, the NCB.SEND command blocks (the calling thread) until the data has been received by the receiver or the send times out. Successful completion of the NCB.SEND command does not guarantee that the receiver program received the data. It merely signifies that the data has been sent from the driver on the sender side to the driver on the receiver side. The completion also indicates that the buffer is not going to be used by NetBIOS anymore and can be reused or freed by the application.

Similar to the sender, a receiver blocks waiting to receive a packet after it has submitted an **NCB.RECEIVE** command. The receiver has to supply its local session number (via the ncb_lsn field), a pointer to the receiver buffer (via the ncb_buffer field) where the data are to be received, and the length of the buffer (via the ncb_length field) in bytes. On successful completion of the NCB.RECEIVE command, the receiver can determine the actual number of bytes received via the ncb_length field.

When the sender submits multiple NCB.SEND commands on the session, the data packets are sent on a first-come, first-serve basis. Similarly, multiple pending NCB.RECEIVE commands are serviced on a first-come, first-serve basis. When the receiving end does not have an NCB.RECEIVE pending corresponding to each NCB.SEND, the data packets are stored by the remote NetBIOS driver. The effect on the performance of the remote (receiver) system can be adverse because storing received packets usurps system resources. When a send cannot be completed in its entirety despite the error control and retransmission mechanisms built into the NetBIOS protocol, the session is terminated to ensure data integrity. And when the incoming data size is larger than the buffer specified by the receiving process, the NetBIOS driver returns data in the caller-supplied buffer as well as an error code (NRC_INCOMP). Upon receiving this error, an application can submit another NCB.RECEIVE command to retrieve the rest of the data sent by the sending process.

A variation from the generic NCB.SEND is **NCB.SEND.NO.ACK**, which in Windows NT gives a hint to the transport protocol that probably there is no response to this send. This can increase throughput if one application is sending lots of data to the other machine when the other machine is not sending data for each packet received. In OS/2 systems, the latter command is used to achieve lower network traffic by reducing NetBIOS-level acknowledgments. The command **NCB.CHAIN.SEND** is used to transmit large data streams. In the Windows NT environment, **NCB.CHAIN.SEND.NO.ACK** is equivalent to the NCB.CHAIN.SEND command. Finally, the **NCB.RECEIVE.ANY** command is used to design efficient servers, which can serve more than one client.

Once a session has been established, it should be terminated using the **NCB.HANG.UP** command. This command must be executed by both session partners. The caller specifies the session number (via the ncb_lsn field) that it

wishes to terminate. Upon successful completion of the NCB.HANG.UP command, the session number cannot be used for accessing NetBIOS session services. After one partner terminates its end of the session, all pending NetBIOS commands at the other end terminate with a distinguished error (NRC_SCLOSED). When an NCB.HANG.UP command is issued by a process, all of the pending NCB.RECEIVE commands terminate with the error NRC_SCLOSED. A hang-up command blocks until all NCB.SEND commands complete, time out, or abort. An abnormal termination of a session causes all the pending NetBIOS commands to terminate with NRC_SABORT. In a classical CSS, the client indicates the end of the dialog using an application-level protocol, at which time both the server and client proceed to hang up the session at each end. This approach ensures that neither side has to worry about abnormal termination save due to networking problems beyond their control.

The following code fragment shows a client ("Client1") and a server ("Server1") who use the NetBIOS session services to communicate with each other. Initially, the server waits, listening (via NCB.LISTEN) to client connect requests. The client connects to the server by posting the NCB.CALL command. The client then sends a message (via NCB.SEND) to the server and waits to receive a reply (via NCB.RECEIVE). The server always waits to hear from the client, then responds with a reply. Both the client and the server loop through the send/receive cycle RWLoop times. Both the client and the server use synchronous commands (hence all functions are named BlockedFunction).

Notice that the client and the server register their names before proceeding to connect to each other because a name number must be used with the NCB.LISTEN or NCB.CALL commands. Alternatively, they can use the reserved name number (1). Finally, the server program can listen to a specific client by using an asterisk in ncb_callname for the NCB.LISTEN command or specifying a client (for example, Client1). The former is preferable in most client/server communications where a server simply services any client that contacts the server. The latter method is useful in peer-to-peer communications when both parties know each other's names and wish to communicate only with each other. Note that when the server is listening for a specific client (for example, Client1), all connection attempts by other clients (for example, Client2) will fail with an error (NRC_REMTFUL—"Remote Session Table is Full").


```
// Readers should see %BOOK%\netbios\Client1.mak and Server1.mak for
// building fully working programs.
//
// FILE: %BOOK%\netbios\netfunc.c.
//       Only those functions that are used by Client1.c
//       and Server1.c are shown here. I will show the rest of the functions
//       as needed throughout this chapter. The functions AddName(),
//       DeleteName(), and Reset() are similar to the functions shown above
//       and hence not shown here.
//
```

```
#include "netfunc.h"

#define MAX_SESSIONS      12
#define MAX_NAMES         12
#define SEND_TIMEOUT      120 // (120 * 500 ms )/ 1000 = 60 sec = 1 minute
#define RECV_TIMEOUT      120 // 1 minute
#define ONE_SECOND        1000*1 // 1000 ms = 1 second
// Global variable
UCHAR ucLanaNum = 0;

//
// PrintError() - Prints messages for most common NetBIOS errors
//                        (Only a partial listing is shown here)
void PrintError(UCHAR ucRc, LPSTR pszError)
{
     printf("Error in %s:", pszError);
     switch (ucRc)
     {
     case NRC_BUFLEN:
          printf("illegal buffer length.\n");
          break;
     default:
          printf("Unknown error: %x\n", ucRc);
     }
}
//
// BlockingSend() - Send a packet using Session Services (NCB.SEND)
//

UCHAR BlockingSend(
                   UCHAR ucLsn,
                   LPSTR pchData,
                   USHORT usDataLen
                  )
{
     NCB Ncb;
     UCHAR ucRc;
     // First clear out the buffer
     memset (&Ncb, 0x00, sizeof(NCB));

     //     Set the Command. Synchronous NCB.SEND
     Ncb.ncb_command = NCBSEND;

     // set the LAN Adapter number
     Ncb.ncb_lana_num = ucLanaNum;

     // set the Local Session Number
     Ncb.ncb_lsn = ucLsn;

     // Set  Data buffer pointer and its length
     Ncb.ncb_buffer = pchData;
     Ncb.ncb_length = usDataLen;

     // Make the NetBIOS call
     if ((ucRc=Netbios( &Ncb))!= NRC_GOODRET)
     {
          PrintError(ucRc, "Blocking Send");
          return ucRc ;
```

```
    }
    return Ncb.ncb_retcode;
}
//
// BlockingReceive() -
//   This function will call synchronous NCB.RECEIVE
//
UCHAR     BlockingReceive(
                          UCHAR ucLsn,
                          LPSTR pchData,
                          USHORT *pusDataLen
                         )
{
    NCB  Ncb;
    UCHAR ucRc;
    // First clear out the buffer
    memset (&Ncb, 0x00, sizeof(NCB));

    //   Set the Command ( NCB.RECEIVE )
    Ncb.ncb_command = NCBRECV;

    // set the LAN Adapter number
    Ncb.ncb_lana_num = ucLanaNum;

    // set the Local Session Number
    Ncb.ncb_lsn = ucLsn;

    // Set  Data buffer pointer and its length
    Ncb.ncb_buffer = pchData;
    Ncb.ncb_length = *pusDataLen;

    // Make the NetBIOS call
    if ((ucRc=Netbios (&Ncb)) != NO_ERROR)
    {
        PrintError(ucRc, "BlockingReceive");
        return ucRc;
    }
    // return actual size of data buffer if call was successful
    if (Ncb.ncb_retcode == NRC_GOODRET)
        *pusDataLen = Ncb.ncb_length;

    return Ncb.ncb_retcode;
}
// CancelCommand() -
//   This function will execute synchronous NCB.CANCEL command.
//
UCHAR  CancelCommand(PNCB pNcb)
{
    NCB Ncb;
    UCHAR ucRc;
    // First clear out the buffer
    memset (&Ncb, 0x00, sizeof(NCB));

    //   Set the Command synchronous NCB.HANGUP
    Ncb.ncb_command = NCBCANCEL;

    // set the LAN Adapter number
```

```
    Ncb.ncb_lana_num = ucLanaNum;

    // set the ptr to NCB to cancel in the buffer
    *(DWORD *)&(Ncb.ncb_buffer) = (DWORD)pNcb;

    // Make the NetBIOS call
    if ((ucRc=Netbios (&Ncb))!= NRC_GOODRET)
    {
        PrintError(ucRc, "CancelCommand");
        return ucRc ;
    }
    return Ncb.ncb_retcode;
}
//
//  HangSess() -
//      This function will execute synchronous NCB.HANGUP command.
//
UCHAR    HangSess( UCHAR ucLsn )
{
    NCB  Ncb;
    UCHAR ucRc;

    // First clear out the buffer
    memset (&Ncb, 0x00, sizeof(NCB));

    //   Set the Command synchronous NCB.HANGUP
    Ncb.ncb_command = NCBHANGUP;

    // set the LAN Adapter number
    Ncb.ncb_lana_num = ucLanaNum;

    // set the Local Session Number
    Ncb.ncb_lsn = ucLsn;

    // Make the NetBIOS call
    if ((ucRc=Netbios (&Ncb))!= NRC_GOODRET)
    {
        PrintError(ucRc, "HangSession");
        return ucRc;
    }
    return Ncb.ncb_retcode;
}
//
// BlockingCall()
//   This function will execute synchronous NCB.CALL command.
//
UCHAR    BlockingCall(
                    LPSTR pchRemoteName,
                    LPSTR pchLocalName,
                    UCHAR *ucLsn
                   )
{
    NCB Ncb;
    UCHAR ucRc;
    // First clear out the buffer
    memset (&Ncb, 0x00, sizeof(NCB));
```

```
        //Set the Command synchronous NCB.CALL
        Ncb.ncb_command = NCBCALL;

        // Set the send/receive timeout values
        Ncb.ncb_sto = SEND_TIMEOUT;
        Ncb.ncb_rto = RECV_TIMEOUT;

        // set the LAN Adapter number
        Ncb.ncb_lana_num = ucLanaNum;

        // Set Remote name in ncb_callname
        memcpy(&Ncb.ncb_callname[0], pchRemoteName, NCBNAMSZ);

        // Set Local name in ncb_name
        memcpy(&Ncb.ncb_name[0], pchLocalName, NCBNAMSZ);

        // Make the NetBIOS call
        If ((ucRc=Netbios (&Ncb))!= NRC_GOODRET)
        {
             PrintError(ucRc, "BlockingCall");
             return ucRc ;
        }
        // Return the Local Session Number
        if (Ncb.ncb_retcode == NO_ERROR)
        {
             *ucLsn = Ncb.ncb_lsn;
             printf("LSN on Call side: %d\n", *ucLsn);
        }
        return Ncb.ncb_retcode;
}
//
// BlockingListen() -
//   This function will execute synchronous NCB.LISTEN command
//
UCHAR     BlockingListen(
                         LPSTR pchRemoteName,
                         LPSTR pchLocalName,
                         UCHAR *ucLsn
                        )
{
     NCB  Ncb;
     UCHAR ucRc;
     BOOL fReturnCallerName;
     // First clear out the buffer
     memset (&Ncb, 0x00, sizeof(NCB));

     //   Set the Command ( NCB.LISTEN )
     Ncb.ncb_command = NCBLISTEN;

     // set the LAN Adapter number
     Ncb.ncb_lana_num = ucLanaNum;

     // Set the send/receive timeout values
     Ncb.ncb_sto = SEND_TIMEOUT;
     Ncb.ncb_rto = RECV_TIMEOUT;

     // Set Remote name in ncb_callname
```

```
    if (pchRemoteName[0]=='*')
    {
        memset ( Ncb.ncb_callname, 0x20, NCBNAMSZ);
        Ncb.ncb_callname[0]='*';
        fReturnCallerName = TRUE;
    }
    else
    {
        memcpy(&(Ncb.ncb_callname[0]), pchRemoteName, NCBNAMSZ);
        fReturnCallerName = FALSE;
    }
    // Set Local name in ncb_name
    memcpy(Ncb.ncb_name, pchLocalName, NCBNAMSZ);

    // Make the NetBIOS call
    if ((ucRc=Netbios (&Ncb))!= NRC_GOODRET)
    {
        PrintError(ucRc,"BlockingListen");
        return ucRc;
    }
    // Return the Local Session Number
    // and return caller's name if pchRemoteName was set '*'
    if (Ncb.ncb_retcode == NRC_GOODRET)
    {
        *ucLsn = Ncb.ncb_lsn;
        printf("LSN on Listen side: %d\n", *ucLsn);
        if (fReturnCallerName)
        {
           memcpy(pchRemoteName, &Ncb.ncb_callname[0], NCBNAMSZ);
           pchRemoteName[NCBNAMSZ] = '\0'; // purely for ease of printing
        }
    }
    return Ncb.ncb_retcode;
}
//
// FILE: %BOOK%\netbios\parsecmd.c
//
#include "netfunc.h"
// Globals declared in the caller
extern char            szClientName [ ];
extern char            szServerName [ ];
extern int             RWLoop;
extern UCHAR           ucLanaNum;
extern BOOL            fShutDownServer;
extern int Usage();
//
// Create a blank padded NetBIOS name from a NULL-terminated string
//
void CopyName (PCHAR pszTo, PCHAR pszFrom)
{
    int len = strlen(pszFrom);
    // First set space in the target name
    memset (pszTo, 0x20, NCBNAMSZ);
    // now only copy the characters
    memcpy (pszTo, pszFrom, len);
}
//
```

```
// ParseCommandLine() - Command line parser. Sets the defaults.
//
int ParseCommandLine ( int argc, char **argv, BOOL fServer)
{
     char* ptr;
     argc--;
     argv++;
     // Set Defaults
     if (fServer)
     {    // Allow Server to communicate with all clients
          CopyName(szClientName, "*");
     }
     else
          CopyName(szClientName, "Client1");
     CopyName(szServerName, "Server1");
     RWLoop = 1;
     fShutDownServer = FALSE;
     ucLanaNum = 0;
     while (argc)
     {
          ptr = argv[0];
          if (*ptr=='-')
          {
               ptr++;
               switch (*ptr)
               {
               case 's': ptr += 2; // Server Name
                         if (*ptr)
                              CopyName(szServerName, ptr);
                         break;
               case 'c': ptr +=2;  // Client Name
                         if (*ptr)
                              CopyName(szClientName, ptr);
                         break;
               case 'l': ptr +=2;  // LANA number
                         if (*ptr)
                              ucLanaNum = atoi(ptr);
                         break;
               case 'r': ptr += 2;  // Read/Write Loop
                         RWLoop = atoi (ptr);
                         break;
               case 'k': fShutDownServer = TRUE; // Shutdown the Server
                         break;
               default:
                    return Usage();
               }
          }
          else
               return Usage();
          argc--;
          argv++;
     }
     // Put a NULL at the end of names to ease printing
     szServerName[NCBNAMSZ] = '\0';
     szClientName[NCBNAMSZ]  = '\0';
     if (!fBroadcast)
          printf("Client: [%s] Server: [%s]\nLana: %d "\
```

```
                "RWLoop: %d ShutdownServer: %s\n",
                 szClientName,
                 szServerName,
                 ucLanaNum,
                 RWLoop,
                (fShutDownServer) ? "yes" : "no"
              );
     return (0);
}
//
// FILE: %BOOK%\netbios\CLIENT1.C
//
#include "netfunc.h"

char        szClientName [NCBNAMSZ + 1] ;
char        szServerName [NCBNAMSZ + 1] ;
int         RWLoop;
BOOL        fShutDownServer;

#define MESSAGE_SIZE 80
#define SHUTDOWN_MSG "shutdown"

int Usage()
{
     printf("Clientl -s:<Server Name> -c:<Client Name> -l:<LANA num>\n");
     printf("            -k\n");
     return (1);
}

int _cdecl main (int argc, char ** argv)
{
    UCHAR       ucLsn, ucNameNum;
    UCHAR       ucRc = NRC_GOODRET;
    USHORT      usBytesRead;
    char        chBuffer [ MESSAGE_SIZE ];
    int         i;

    // Parse the command line
    if (ParseCommandLine (argc, argv, FALSE))
          return(1);

    // call NCB.RESET
    if (Reset())
          return (1);
     // Register the Client name by NCB.ADD.NAME
     if (AddName (szClientName, &ucNameNum) == NRC_GOODRET)
     {
          // post an NCB.CALL
          printf("Calling Server\n");
          ucRc = BlockingCall (szServerName, szClientName, &ucLsn);
          if (ucRc==NRC_GOODRET)
          {
               printf("Connected!\n");
               for ( i  = 0; i  < RWLoop; i++ )
               {
                    // Send a message to server via NCB.SEND
                    if (fShutDownServer)
```

```
                    sprintf(chBuffer, SHUTDOWN_MSG);
               else
                    sprintf(chBuffer, "Client Msg: %d", i);
               ucRc = BlockingSend ( ucLsn,
                                     &chBuffer[0],
                                     strlen(chBuffer) + 1
                                   );
               if ( (ucRc != NRC_GOODRET) || (fShutDownServer) )
                    break; // hangup

               // Receive Reply from Server via NCB.RECEIVE
               usBytesRead = sizeof(chBuffer);
               ucRc = BlockingReceive ( ucLsn,
                                        &chBuffer[0],
                                        &usBytesRead
                                      );
               if ((ucRc == NRC_GOODRET) && (usBytesRead))
               {
                    printf("Server Replied: %s\n", chBuffer);
               }
               else
                    break; // hangup the session
          }
          // Terminate the session via NCB.HANGUP
          HangSess ( ucLsn );
     }
     // Deregister the name via NCB.DELETE.NAME
     DeleteName (szClientName, ucNameNum);
  }
  return 0;
}
//
// FILE: %BOOK%\netbios\SERVER1.C
//
#include "netfunc.h"

char        szClientName [NCBNAMSZ + 1];
char        szServerName [NCBNAMSZ + 1];
int         RWLoop;
BOOL        fShutDownServer;

#define MESSAGE_SIZE 80
#define SHUTDOWN_MSG "shutdown"

int Usage()
{
     printf("Server1 -s:<Server Name> -c:<Client Name> -l:<LANA num>\n");
     return (1);
}
int _cdecl main (int argc, char ** argv)
{
     UCHAR       ucLsn, ucNameNum;
     UCHAR       ucRc = NRC_GOODRET;
     USHORT      usBytesRead;
     char        chBuffer [ MESSAGE_SIZE ];
     int         i;
```

```
// Parse the command line
if (ParseCommandLine ( argc, argv, TRUE))
      return(1);

// call NCB.RESET
if (Reset())
      return (1);

// Register the Server name by NCB.ADD.NAME
if (AddName (szServerName, &ucNameNum)==NRC_GOODRET)
{
      while ((fShutDownServer == FALSE) && (ucRc == NRC_GOODRET))
      {
            // post an NCB.LISTEN
            printf("Waiting in NCB.LISTEN\n");
            ucRc = BlockingListen ( szClientName, szServerName, &ucLsn);
            if (ucRc==NRC_GOODRET)
            {
                  printf("%s Connected\n", szClientName);
                  for ( i  = 0; i  < RWLoop; i++ )
                  {
                        // Receive messages from client via NCB.RECEIVE
                        usBytesRead = sizeof(chBuffer);
                        ucRc = BlockingReceive ( ucLsn,
                                                 &chBuffer[0],
                                                 &usBytesRead
                                                 );
                        if (ucRc == NRC_GOODRET)
                        {
                              if (usBytesRead)
                              {
                                    printf("Client Sent: %s\n", chBuffer);
                                    if (stricmp(chBuffer, SHUTDOWN_MSG)==0)
                                    {
                                          fShutDownServer = TRUE;
                                          break; // end Server
                                    }
                              }
                        }
                        else
                              break; // hangup the session

                        // Now send a reply via NCB.SEND
                        sprintf(chBuffer, "Server Reply: %d", i);
                        ucRc = BlockingSend ( ucLsn,
                                              &chBuffer[0],
                                              strlen(chBuffer) + 1
                                             );
                        if (ucRc != NRC_GOODRET)
                              break; // hangup
                  }
                  // Terminate the session via NCB.HANGUP
                  HangSess ( ucLsn );
            }     // end while
}
// Deregister the name via NCB.DELETE.NAME
```

```
        DeleteName (szServerName, ucNameNum);
        }
        return 0;
}
```

In summary, the NCB.LISTEN, NCB.CALL, and NCB.HANG.UP commands are used to set up or tear down a session. The NCB.SEND and NCB.RECEIVE commands are used to send and receive data.

8.3.4 Designing Programs Using Asynchronous Commands

You probably noticed in the previous section that the program is not friendly because each synchronous NetBIOS command will block until it completes. This is not necessarily a problem. Using the synchronous name management NetBIOS command (for example, NCB.ADD.NAME) is usually good practice because the program will not be able to do much without registering its logical name and these commands complete within a few seconds (depending on the NetBIOS driver settings and network topology). On the other hand, an NCB.LISTEN command issued by a server can block indefinitely until a client issues an NCB.CALL command. In general, a network-aware program should not block on network I/O because it cannot predict the behavior of other participants and the network. Hence, applications should use asynchronous I/O or "peeking" facilities whenever available and deemed useful.

Often, asynchronous I/O involves additional operational, monitoring, and management code. Thus, for simple client/server applications, the performance may be negatively affected because of the extra management. However, this is often insignificant due to the slow relative speed of the network compared to CPU speeds. In general, asynchronous I/O is most appropriate for sophisticated and complex servers that are simultaneously serving multiple clients.

Most NetBIOS command can be executed asynchronously. To do so, the caller fills in the ncb_command field with the proper NetBIOS command OR-ed with 80h and the rest of the NCB structure with the required set of values corresponding to the command as if submitting a blocking command. The NCB is then submitted to the Netbios() API, which returns immediately, allowing the caller to continue. In this case, Netbios() API returns (00h) indicating it has accepted the request. A nonzero value returned by the Netbios() API indicates that an error occurred while processing the NCB or the command completed as soon as the nonblocking NetBIOS command was submitted; in the later cases, the immediate return value is the final one. The NetBIOS driver sets the ncb_retcode as well as ncb_cmd_cplt fields with the immediate return code—255 (FFh) for pending commands. When an asynchronous NetBIOS command completes, the NetBIOS driver sets the ncb_cmd_cplt field with the final return code. Hence, a simple way for an application to determine the completion of a pending NetBIOS command is

to poll the ncb_cmd_cplt field until the value changes from pending (FFh) to a success code (0h) or an error code.

The following code fragment shows a function, **PollingSend()** which sends messages to the server using an asynchronous send command. Specifically, the PollingSend() function submits an asynchronous NCB.SEND command and then uses ncb_cmd_cplt–based polling to determine completion of the send command. The PollingSend() function is used in the Client2.c program, a slight modification of the Client1.c program.

```
// FILE: %BOOK%\netbios\netfunc.c
//
// PollingSend() - Send a packet using asynchronous NCB.SEND
//                          and then pool for command to complete
UCHAR PollingSend(
                  UCHAR ucLsn,
                  LPSTR pchData,
                  USHORT usDataLen
                 )
{
    NCB Ncb;
    UCHAR ucRc;
    // First clear out the buffer
    memset (&Ncb, 0x00, sizeof(NCB));

    //   Set the Command. Asynchronous NCB.SEND
    Ncb.ncb_command = NCBSEND | ASYNCH;

    // set the LAN Adapter number
    Ncb.ncb_lana_num = ucLanaNum;

    // set the Local Session Number
    Ncb.ncb_lsn = ucLsn;

    // Set  Data buffer pointer and its length
    Ncb.ncb_buffer = pchData;
    Ncb.ncb_length = usDataLen;

    // Make the NetBIOS call
    ucRc=Netbios(&Ncb);
    if (Ncb.ncb_retcode == NRC_PENDING)
    {
         // Wait for command to complete
         while (Ncb.ncb_cmd_cplt == NRC_PENDING)
         {
              // Do other processing. Here, we simply sleep
              Sleep (1);
              printf(".");
         }
         printf("\n");
         ucRc =  Ncb.ncb_cmd_cplt;
    }
    if (ucRc != NRC_GOODRET)
    {
         PrintError(ucRc, "Polling Send");
```

```
            return ucRc ;
        }
        return Ncb.ncb_retcode;
}
//
// FILE: %BOOK%\netbios\c1ient2.c
//
#include "netfunc.h"
char        szClientName [NCBNAMSZ + 1] ;
char        szServerName [NCBNAMSZ + 1] ;
int         RWLoop;
BOOL        fShutDownServer;
#define MESSAGE_SIZE 80
#define SHUTDOWN_MSG "shutdown"
int Usage()
{
     printf("Client2 -s:<Server Name> -c:<Client Name> -l:<LANA num>\n");
     printf("        -k\n");
     return (1);
}
int _cdecl main (int argc, char ** argv)
{
     UCHAR       ucLsn, ucNameNum;
     UCHAR       ucRc = NRC_GOODRET;
     USHORT      usBytesRead;
     char        chBuffer [ MESSAGE_SIZE ];
     int         i;
     // Parse the command line
     if (ParseCommandLine (argc, argv, FALSE))
          return(1);

     // call NCB.RESET
     if (Reset())
          return (1);
     // Register the Client name by NCB.ADD.NAME
     if (AddName (szClientName, &ucNameNum) == NRC_GOODRET)
     {
          // post an NCB.CALL
          printf("Calling Server\n");
          ucRc = BlockingCall (szServerName, szClientName, &ucLsn);
          if (ucRc==NRC_GOODRET)
          {
               printf("Connected!\n");
               for ( i  = 0; i  < RWLoop; i++ )
               {
                    // Send a message to server via NCB.SEND
                    if (fShutDownServer)
                         sprintf(chBuffer, SHUTDOWN_MSG);
                    else
                         sprintf(chBuffer, "Client Msg: %d", i);
                    // Asynchronous NCB.SEND and poll to detect completion
                    ucRc = PollingSend ( ucLsn,
                                         &chBuffer[0],
                                         strlen(chBuffer) + 1
                                       );
                    if ((ucRc != NRC_GOODRET) || (fShutDownServer))
```

```
                    break; // hangup

                // Receive Reply from Server via NCB.RECEIVE
                usBytesRead = sizeof(chBuffer);
                ucRc = BlockingReceive ( ucLsn,
                                         &chBuffer[0],
                                         &usBytesRead
                                       );
                if ((ucRc == NRC_GOODRET) && (usBytesRead))
                {
                    printf("Server Replied: %s\n", chBuffer);
                }
                else
                    break; // hangup the session
            }
            // Terminate the session via NCB.HANGUP
            HangSess ( ucLsn );
        }
        // Deregister the name via NCB.DELETE.NAME
        DeleteName (szClientName, ucNameNum);
    }
    return 0;
}
```

Although the use of the ncb_cmd_cplt field allowed us to use an asynchronous command execution facility, it added a cumbersome method, polling, to determine the completion of the asynchronous commands. Fortunately, NetBIOS provides another means to determine the completion of asynchronous commands. An application can associate a function with an asynchronous NCB so that upon completion of the NCB, NetBIOS drivers call the application back to the supplied function. Such functions are commonly known as post routines (or callbacks).

A post routine is associated with an asynchronous NetBIOS command via the **ncb_post** field. The post routine is always called at the completion of the NetBIOS command, whether the command completes immediately (that is, the return code from Netbios() is 0) or later. In other words, the post routine is not called if the Netbios() function returns an error code immediately. The prototype of a post routine and NCB for Windows NT follows.

```
// NCB structure for Windows NT
typedef struct _NCB { /* ncb */
    UCHAR     ncb_command;
    UCHAR     ncb_retcode;
    UCHAR     ncb_lsn;
    UCHAR     ncb_num;
    PUCHAR    ncb_buffer;
    WORD      ncb_length;
    UCHAR     ncb_callname[NCBNAMSZ];
    UCHAR     ncb_name[NCBNAMSZ];
    UCHAR     ncb_rto;
    UCHAR     ncb_sto;
    void      (CALLBACK *ncb_post)( struct _NCB * );
    UCHAR     ncb_lana_num;
```

```
    UCHAR      ncb_cmd_cplt;
    UCHAR      ncb_reserve[10];
    HANDLE     ncb_event;
} NCB, *PNCB;
//
#define NCB_POST void CALLBACK
// usage: NCB_POST MyPostRoutine( PNCB pncb );
//
```

Note that NCB in Windows NT takes a Win32 event handle via the ncb_event field. The total size of the NCB data structure for Windows NT is the same as presented earlier to maintain backward compatibility. The only difference is that instead of 14 bytes of reserved space, the Windows NT NCB uses 10 bytes as reserved space and 4 bytes for holding the event handle.

The post routine receives, as the only parameter, a pointer to the NCB that completed. The function is called in the application's context so the caller can access normal resources from this call-back routine. It is generally recommended to return from the post routine quickly because the NetBIOS driver cannot discard the resources associated with an NCB until the post routine completes. Note that another NetBIOS command should not be posted from within a post routine. Additionally, the post routine should not make a blocking call. In the Windows NT environment, a post routine may use a critical section to protect internal data structures. Often it is sufficient to set an event to the signaled state.

In the following code fragments, I demonstrate usage of a post routine by tying in asynchronous NetBIOS programming with a typical Windows program. I first show a modified version of the PollingSend() function, which submits an asynchronous NCB.SEND and associates a post routine with the NCB. The main application message processing routine (WndProc) creates a special structure (NOTIFYNCB) containing the window handle, a notification message, and the NCB. For posting an asynchronous NCB.SEND, the pointer to the NCB contained in NOTIFYNCB is sent to the **PostSend()** routine. The PostSend() function associates a post routine with the NCB and submits it. The post routine (**PostRoutine()**), called at the completion of NCB.SEND, extracts the window handle from the pointer passed to it and sends the main message processing routine a user-defined message (WM_TRANSITION_STATE). This mechanism allows the application to return from the post routine quickly and yet retain full ability to process completion messages in its regular message processing routine. In other words, using the PostMessage() mechanism from the post routine, nonblocking NetBIOS completion becomes yet another window message and meshes well with the message-driven programming paradigm of Windows.

Does this technique look familiar? It should. The Windows Socket WSAAsyncSelect() API uses a similar mechanism to inform the application on completion of I/O. Indeed, Windows Socket libraries hide the details from us. Here we are directly dealing with the Netbios() API and hence need to design an asynchronous I/O handling mechanism.

For single-threaded Win16 applications, this is the recommended method for handling asynchronous NetBIOS commands. By using the post routine, the application does not block on NetBIOS command and by using the PostMessage() API, it can handle completion of I/O as a normal Windows message.

```
//
// FILE: %BOOK%\netbios\netfunc.c
//
//
// AllocNcbAndBuffer() -
// Allocate an NCB and put the data in the NCB. I also embed
// the Windows handle and a notification message with each
// NCB so that PostRoutine() can correctly route the completion
// message
//
//
typedef struct
{
    HWND   hPostWindow;
    UINT   uiPostMessage;
    NCB    Ncb;
} NOTIFYNCB, *PNOTIFYNCB;
#define NCB_PTR(p)     (PNCB)        ((LPSTR )p + sizeof(HWND) + sizeof(UINT))
#define NOTIFY_PTR(p)  (PNOTIFYNCB) ((LPSTR )p - sizeof(HWND) - sizeof(UINT))

PNCB AllocNcbAndBuffer ( LPSTR pszString,
                         HWND  hPostWindow,
                         UINT  uiPostMessage
                       )
{
    PNCB          pNcb = NULL;
    PNOTIFYNCB    pNotify;
    int           iLen = strlen ( pszString ) + 1;
    LPSTR         ptr;
    // Allocate global memory and lock it
    pNotify = (PNOTIFYNCB) GlobalAllocPtr ( GHND, sizeof(NOTIFYNCB)
                                                  + iLen );
    if (pNotify)
    {
          // Put the Post Message and Window alongside the NCB
          pNotify->hPostWindow   = hPostWindow;
          pNotify->uiPostMessage = uiPostMessage;
          // Next get a pointer to the NCB embedded in the NOTIFYNCB
          pNcb = NCB_PTR(pNotify);

          // Copy string into buffer associated with the NCB
          ptr = (LPSTR) pNcb + sizeof(NCB);
          memcpy(ptr, pszString, iLen);
          pNcb->ncb_buffer = ptr;
          pNcb->ncb_length = iLen;
     }
     return (pNcb);
}
//
// FreeNcbAndBuffer() -
```

```
//   Free the NOTIFYNCB and its associated buffer
//   (see AllocateNcbAndBuffer() for structures)
//
void FreeNcbAndBuffer (PNCB pNcb)
{
    if (pNcb)
    {
          PNOTIFYNCB pNotify = NOTIFY_PTR ( pNcb );
          // Unlock and free the memory
          GlobalFreePtr (pNotify);
    }
}
//
// PostRoutine() - This function is called by NetBIOS driver when
//                 a NetBIOS command asynchronously completes or aborts
NCB_POST PostRoutine ( PNCB pNcb )
{
    if (pNcb)
    {
          // Get to the NOTIFYNCB structure
          PNOTIFYNCB pNotify = NOTIFY_PTR ( pNcb );
          // Send notification message
          if (pNotify)
               PostMessage ( pNotify->hPostWindow,
                             pNotify->uiPostMessage,
                             0,
                             (LPARAM) pNcb
                           );
    }
}
//
// PostSend() -   Send a packet using asynchronous NCB.SEND
//                using a Post Routine called asynchronously
//                when send completes or aborts
//
UCHAR PostSend( PNCB   pNcb, UCHAR  ucLsn )
{
    UCHAR ucRc;
    // The NCB already has data buffer and length set

    //  Set the Command. Asynchronous NCB.SEND
    pNcb->ncb_command = NCBSEND | ASYNCH;

    // set the LAN Adapter number
    pNcb->ncb_lana_num = ucLanaNum;

     // set the Local Session Number
     pNcb->ncb_lsn = ucLsn;

     // Set the Post Routine
     pNcb->ncb_post = PostRoutine;

     // Make the NetBIOS call
     ucRc=Netbios( pNcb);
    return pNcb->ncb_retcode;
}
```

```
//
// FILE: %BOOK%\netbios\client3.c. client3.mak is used to build the program
//
// Usage - Use it with Server1
//         Server1 -c:Client3 -s:Server1 -r:2 -l:0
//         Client3
//
#include "client3.h"
#include "netfunc.h"

char        szClientName [NCBNAMSZ + 1] ;
char        szServerName [NCBNAMSZ + 1] ;
int         RWLoop;
BOOL        fShutDownServer;
extern UCHAR ucLanaNum;
#define MESSAGE_SIZE 80

// Globals
HINSTANCE hInst;                              // current instance
char szAppName[] = "Client3";                 // The name of this application
char szTitle[] = "Client using PostRoutine"   // The title bar text
char szDialogName[] = "I/O Box";              // Title on dialog box

// Display/Error Message Buffer
char szMessageBuffer [ MAX_PATH ];
char szErrorMsg [ 100 ] ;
// Macro to show an error with message box
#define SHOW_ERROR(s, err) {\
    wsprintf(szErrorMsg, "Error: %ld in %s at Line: %d\n", \
                    err,s);}
// Macro to send message to I/O List Box
static HWND hListDialog, hwndList;
#define DISPLAY_ON_SCREEN(s) {\
    {if (s != NULL) strcpy(szMessageBuffer, s);\
    SendMessage(hwndList, LB_ADDSTRING, \
                0, (LONG)(LPTSTR)szMessageBuffer);} \
}

LRESULT CALLBACK WndProc    ( HWND hWnd,
                              UINT message,
                              WPARAM wParam,
                              LPARAM lParam);

int Usage ()
{
    return(1);
}


////////////////////////////////////////////////////////////////////////
//
// FUNCTION: WinMain(HINSTANCE, HINSTANCE, LPSTR, int)
//
////////////////////////////////////////////////////////////////////////
int APIENTRY WinMain( HINSTANCE hInstance, HINSTANCE hPrevInstance,
                      LPSTR lpCmdLine,int nCmdShow)
```

```
{
    MSG msg;
    WNDCLASS  wc;
    HWND      hWnd;          // Main window handle.
    UCHAR     ucNameNum;     // Name Number
    UCHAR     ucRc;

    // If first instance, register app's window class
    if (!hPrevInstance)
    {
          // Fill in window class structure with parameters that
          // describe the main window.
          wc.style         = CS_HREDRAW | CS_VREDRAW; // Class style(s).
          wc.lpfnWndProc   = (WNDPROC)WndProc;        // Window Procedure
          wc.cbClsExtra    = 0;                  // No per-class extra data.
          wc.cbWndExtra    = 0;                  // No per-window extra data.
          wc.hInstance     = hInstance;               // Owner of this class
          wc.hIcon     = LoadIcon (hInstance, szAppName)// Icon name from .RC
          wc.hCursor   = LoadCursor(NULL, IDC_ARROW);  // Cursor
          wc.hbrBackground = (HBRUSH)(COLOR_WINDOW+1);// Default color
          wc.lpszMenuName  = szAppName;               // Menu name from .RC
          wc.lpszClassName = szAppName;               // Name to register as

          // Register the window class and return success/failure code.
          if (!RegisterClass(&wc))
               return (FALSE);     // Exits if unable to register
     }
     // Save the instance handle in static variable, which will be used in
     // many subsequent calls from this application to Windows.

     hInst = hInstance; // Store instance handle in our global variable

     // Create a main window for this application instance.
     hWnd = CreateWindow(
               szAppName,            // See RegisterClass() call.
               szTitle,              // Text for window title bar.
               WS_OVERLAPPEDWINDOW,  // Window style.
               CW_USEDEFAULT,   0,   //   positioning
               CW_USEDEFAULT,   0,   //   positioning
               NULL,                 // Overlapped windows have no parent.
               NULL,                 // Use the window class menu.
               hInstance,            // This instance owns this window.
               NULL                  // We don't use any data in WM_CREATE
                         );

     // If window could not be created, return "failure"
     if (!hWnd)
          return (FALSE);
     // Set defaults (ideally we should get them by displaying a dialog)
     CopyName (szClientName, "Client1");
     CopyName (szServerName, "Server1");
     ucLanaNum = 0;
     RWLoop = 2;
     fShutDownServer = FALSE;

     // call NCB.RESET
     if (ucRc = Reset( ))
```

```
    {
          SHOW_ERROR("Reset", ucRc);
          return (1);
    }

    // Register the client name by NCB.ADD.NAME
    if ((ucRc = AddName (szClientName, &ucNameNum)) != NRC_GOODRET)
    {
          SHOW_ERROR("AddName", ucRc);
          return (1);
    }
    // Make the window visible; update its client area; and return "success"
     ShowWindow(hWnd, nCmdShow);   // Show the window
     UpdateWindow(hWnd);           // Sends WM_PAINT message

     // Acquire and dispatch messages until a WM_QUIT message is received.
     while (GetMessage(&msg,      // message structure
               NULL,              // handle of window receiving the message
               0,                 // lowest message to examine
               0))                // highest message to examine
    {
          TranslateMessage(&msg);// Translates virtual key code
          DispatchMessage(&msg); // Dispatches message to window
    }

    // Deregister the name via NCB.DELETE.NAME
    DeleteName (szClientName, ucNameNum);
    return (msg.wParam); // Returns the value from PostQuitMessage
}

//
// Procedure to handle list box
//
BOOL CALLBACK
ListBoxProc(HWND hwndDlg, UINT Msg, LONG  wParam, LONG lParam)
{
    return FALSE;
}
//
// Globals related to state transition, net traffic
//
static UCHAR ucLsn    = 0;
static PNCB  pNcbSend = NULL;
// The state transitions which describe how to handle
// a given state
typedef enum
{
     eStartSession = 0,
     ePostSend,
     eWaitSendComplete,
     eReceive,
     eCancelSession,
     eDone
}enNetState;
typedef struct _StateTable
{
    // The current state
```

```
    enNetState eCurState;
    // When an action taken in current state causes error, go to
    //eErrorState
    enNetState eErrorState;
    // When an action taken in current state succeeds, go to eNextState
    // enNetState eNextState;
} StateTable;
StateTable STATE_TABLE [] =
{
    {eStartSession,          eDone,              ePostSend},
    {ePostSend,              eCancelSession,     eWaitSendComplete},
    {eWaitSendComplete,      eCancelSession,     eReceive},
    {eReceive,               eCancelSession,     ePostSend},
    {eCancelSession,         eDone,              eDone}
};
enNetState eCurState = eDone; // Current state
//
// HandleTransition() - Handles all NetBIOS-related transitions
//
UCHAR HandleTransition( HWND hWnd, LPARAM lParam )
{
    UCHAR ucRc;
    char  chBuffer [ MESSAGE_SIZE ];
    USHORT usBytesRead;
    static int iSendRecv;

    switch (eCurState)
    {
    case eStartSession:
          ucRc = BlockingCall (szServerName, szClientName, &ucLsn);
          if (ucRc==NRC_GOODRET)
          {
               iSendRecv = RWLoop;
               DISPLAY_ON_SCREEN("Connected!");
               eCurState = ePostSend;
               PostMessage (hWnd, WM_STATE_TRANSITION, 0, 0);
          }
          else
               SHOW_ERROR("Blocking Call", ucRc);
          break;
    case ePostSend:
          // Send a message to server via NCB.SEND
          sprintf(chBuffer, "Client Msg: %d", iSendRecv);

          // Allocate an NCB, buffer for data, and set
          // buffer pointer in NCB
          pNcbSend = AllocNcbAndBuffer (
                                        chBuffer,
                                        // This window gets notification
                                        hWnd,
                                        // Message sent by PostRoutine()
                                        WM_STATE_TRANSITION
                                       );
          if (pNcbSend == NULL)
          {
               eCurState = eCancelSession;
               PostMessage (hWnd, WM_STATE_TRANSITION, 0, 0);
```

```
            break;
        }
        // Asynchronous NCB.SEND with post routine
        ucRc = PostSend  ( pNcbSend, ucLsn  );
        if (ucRc != NRC_PENDING)
        {
            eCurState = eCancelSession;
            PostMessage (hWnd, WM_STATE_TRANSITION, 0, 0);
        }
        else
        {
            eCurState = eWaitSendComplete;
            DISPLAY_ON_SCREEN("Send Posted.");
        }
        break;
    case eWaitSendComplete:
        // This message is received once send completes

        // LPARAM = Pointer to NCB
        pNcbSend = (PNCB) (lParam);
        // Free the send NCB and its buffer
        if (pNcbSend)
        {
            ucRc = pNcbSend->ncb_cmd_cplt;
            FreeNcbAndBuffer ( pNcbSend );
            pNcbSend = NULL;
            if (ucRc != NRC_GOODRET)
        {
            SHOW_ERROR("Posted Send", ucRc);
        }
            else
                eCurState = eReceive;
        }
        else
        {
            DISPLAY_ON_SCREEN("Error! NULL pNcb");
            ucRc = NRC_GOODRET + 1;
        }
        if (ucRc != NRC_GOODRET)
        {
            eCurState = eCancelSession;
        }
        PostMessage (hWnd, WM_STATE_TRANSITION, 0, 0);
        break;
    case eReceive:
        usBytesRead = sizeof(chBuffer);
        ucRc = BlockingReceive ( ucLsn,
                                 &chBuffer[0],
                                 &usBytesRead
                               );
        if ((ucRc == NRC_GOODRET) && (usBytesRead))
        {
            sprintf(szMessageBuffer, "Server Replied: %s\n", chBuffer);
            DISPLAY_ON_SCREEN(NULL);
            // Now check if we need to do one more round of send/receive
            iSendRecv--;
            if (iSendRecv)
```

```
                    eCurState = ePostSend; // another cycle of Send/Receive
               else
                    eCurState = eCancelSession;  // done
          }
          else
          {
               SHOW_ERROR("Blocking Receive", ucRc);
               eCurState = eCancelSession;
          }
          PostMessage (hWnd, WM_STATE_TRANSITION, 0, 0);
          break;
     case eCancelSession:
          if (pNcbSend)
          {
               FreeNcbAndBuffer ( pNcbSend );
               pNcbSend = NULL;
          }
          HangSess ( ucLsn );
          eCurState = eDone;
          break;
     case eDone:
          break;
     }
     return (ucRc);
}
////////////////////////////////////////////////////////////////////////
//
// FUNCTION: WndProc(HWND, UINT, WPARAM, LPARAM)
//
////////////////////////////////////////////////////////////////////////
LRESULT CALLBACK WndProc( HWND hWnd,            // window handle
                          UINT message,         // type of message
                          WPARAM wParam,        // additional information
                          LPARAM lParam)        // additional information
{
     // Window specific variables
     int wmId, wmEvent;
     switch (message)
     {
     case WM_CREATE:   // Message sent when window is first created

          // create a dialog box which will display I/O information
          hListDialog = CreateDialog (
                        hInst,
                        MAKEINTRESOURCE(IDD_LISTDIALOG),
                        hWnd,
                       (DLGPROC)ListBoxProc
                       );
          if (hListDialog == NULL)
          {
               MessageBox (NULL, "Error Creating List Box", "Error", MB_OK);
               return (1);
          }
          else
          {
               // Set title and set the list to empty
               SendMessage (hListDialog, WM_SETTEXT,
```

```
                              0, (LPARAM)(LPSTR)szDialogName);
                hwndList = GetDlgItem (hListDialog, ID_LIST);
                SendMessage (hwndList, LB_RESETCONTENT, 0, 0L);
          }
          break;
     case WM_COMMAND:  // message: command from application menu
          wmId          = LOWORD(wParam);
          wmEvent       = HIWORD(wParam);
          switch (wmId)
          {
          case IDM_EXIT:
               DestroyWindow (hWnd);
               break;
          case IDM_START:
               eCurState = eStartSession;
               HandleTransition (hWnd, (LPARAM)0);
          default:
               return (DefWindowProc(hWnd, message, wParam, lParam));
          } // end of WM_COMMAND
          break;
     case WM_STATE_TRANSITION:
          // For debugging only
          HandleTransition (hWnd, lParam);
          break;
     case WM_DESTROY:  // message: window being destroyed
          if (eCurState != eDone)
          {    SHOW_ERROR ("Net Traffic Active", eCurState);}
          else
               PostQuitMessage(0);
          break;
     default:          // Passes it on if unprocessed
          return (DefWindowProc(hWnd, message, wParam, lParam));
     }
     return (0);
}
```

The program has shown the following key concepts:

- The Windows program (Client3.c) uses synchronous commands for almost all NetBIOS actions such as connecting to the server or receiving a reply; however, it uses the PostSend() function to send a message to the server. The PostSend() function associates a post routine (PostRoutine()) with the asynchronous NCB.SEND command; the PostRoutine() function is called by the NetBIOS driver on completion of the NCB.SEND command.
- When the PostRoutine() function is called, the user sends a completion notification to the application via the PostMessage() Windows API. How do you know which window to send the message to? By associating with each NCB a Window handle (PostWindow) and a message (PostMessage) via the NOTIFYNCB structure. Thus, within the PostRoutine() function, users can quickly extract the window handle and the message to be sent to the window.

- In the **AllocNcbAndBuffer()** function, the user allocates enough memory to contain a NOTIFYNCB structure and the string to be sent to the server. At this time, the window handle and the notification message are put in the NOTIFYNCB structure so that PostRoutine() can use them. Additionally, the string to be sent to the server is attached to the bottom of the NCB structure and is pointed to by the ncb_buffer field of the NCB.
- In the client program (Client3.c), a simple state-driven mechanism meshes asynchronous NetBIOS programming with the Windows messaging paradigm. Each state change is caused by a message (WM_ STATE_TRANSITION) received by the **WndProc()** function, which is generated by the PostRoutine() function as well as from within the **HandleTransition()** function to indicate state change. The STATE_TABLE structure shows the complete state transition. The program uses synchronous NetBIOS commands for all purposes except for sending data to the server (via NCB.SEND). However, the program has been specifically designed so that you can easily extend it to use asynchronous commands for any other time-consuming net activity (such as NCB.CALL, NCB.RECEIVE).

The two mechanisms of executing asynchronous NetBIOS commands—using the ncb_cmd_cplt field and a post-processing routine—are specified by the NetBIOS specifications [IBM 84]. Hence, programmers porting their existing applications to Windows NT will find it convenient to use these mechanisms in Windows NT as well.

In the Windows NT environment, programs using a post routine do take a resource and performance hit because a thread is created by the underlying NetBIOS driver for each NCB containing a post routine. To expand, when an NCB containing a post routine is submitted, the NetBIOS driver forwards the I/O request to the proper transport protocol driver. It also creates a thread, which waits for the transport driver to complete the request. When the I/O completes (or aborts), the thread started by the NetBIOS driver calls the original application at the supplied post routine. At the completion of the post routine, the thread within the NetBIOS driver is terminated. Thus, the application using the post routine (indirectly) causes a new thread to start per the asynchronous NetBIOS call. This can negatively affect the performance of the application.

How, then, can you efficiently use asynchronous NetBIOS commands in Windows NT? In Windows NT, an application can associate a Win32 event handle with an NCB via the ncb_event field. This allows the application to determine the completion of a nonblocking NetBIOS command. By using a Win32 event technique, the application avoids creating a new thread per asynchronous command.

In using this technique, the caller creates a manual-reset event and associates the asynchronous NCB via the ncb_event field. The NetBIOS driver sets the event to a nonsignaled state if the command is successfully queued. Hence, an application can wait until the event state changes to the signaled state when the NetBIOS command completes. When an application uses Win32 events to wait on NCB completion, the overall system resources are better used.

An interesting side effect of using events with nonblocking NetBIOS is that when a thread terminates, all waiting NetBIOS commands are canceled automatically if they have an event associated with them. This leads to automatic system cleanup should an application or thread terminate prematurely. This feature is not available using the other two modes of executing asynchronous NetBIOS commands wherein all pending NetBIOS commands must be explicitly canceled. Win32 event–based programming allows an application to conduct network I/O from one or more worker threads while the main thread responds to, say, user input. These extra features of event-based nonblocking NetBIOS programming are highly useful for designing efficient Win32 applications. These applications will not only perform efficient nonblocking network I/O but also boast of noncumbersome programs.

The following code fragment shows my favorite routine, **WaitOnSend()**, using a Win32 event. The client code, which calls WaitOnSend(), creates a manual-reset event and passes its handle to WaitOnSend(). The WaitOnSend() function associates the Win32 events with the NCB and submits an asynchronous NCB.SEND command. It then waits on the Win32 event to determine completion of I/O. The rest of the functions are the same as shown in the previous client/server application.

```
// FILE: %BOOK%\netbios\netfunc.c
// WaitOnSend() - Send a packet using asynchronous NCB.SEND
//                and then wait for an event to signal indicating
//                command completion
//
UCHAR WaitOnSend(
                 UCHAR  ucLsn,
                 LPSTR  pchData,
                 USHORT usDataLen,
                 HANDLE hEvent
                )
{
     NCB Ncb;
     UCHAR ucRc;
     // First clear out the buffer
     memset (&Ncb, 0x00, sizeof(NCB));

     //   Set the Command. Asynchronous NCB.SEND
     Ncb.ncb_command = NCBSEND | ASYNCH;

     // set the LAN Adapter number
```

```
        Ncb.ncb_lana_num = ucLanaNum;

        // set the Local Session Number
        Ncb.ncb_lsn = ucLsn;

        // Set  Data buffer pointer and its length
        Ncb.ncb_buffer = pchData;
        Ncb.ncb_length = usDataLen;

        // Set the Win32 event
        Ncb.ncb_event = hEvent;

        // Make the NetBIOS call
        ucRc=Netbios(&Ncb);
        if (Ncb.ncb_retcode == NRC_PENDING)
        {
             // Wait for command to complete
             while ( WaitForSingleObject(hEvent, ONE_SECOND) == WAIT_TIMEOUT )
             {
                  // Do other processing. Here, we simply sleep
                  Sleep (1);
                  printf(".");
             }
             printf("\n");
             ucRc =  Ncb.ncb_cmd_cplt;
        }
        if (ucRc != NRC_GOODRET)
        {
             PrintError(ucRc, "Polling Send");
             return ucRc;
        }
        return Ncb.ncb_retcode;
}
//
// FILE: %BOOK%\netbios\client4.c
#include "netfunc.h"
char      szClientName [NCBNAMSZ + 1] ;
char      szServerName [NCBNAMSZ + 1] ;
int       RWLoop;
BOOL      fShutDownServer;
#define MESSAGE_SIZE 80
#define SHUTDOWN_MSG "shutdown"
int Usage()
{
     printf("Client4 -s:<Server Name> -c:<Client Name> -l:<LANA num>\n");
     printf("            -k\n");
     return (1);
}
int _cdecl main (int argc, char ** argv)
{
     UCHAR     ucLsn, ucNameNum;
     UCHAR     ucRc = NRC_GOODRET;
     USHORT    usBytesRead;
     char      chBuffer [ MESSAGE_SIZE ];
     int       i;
     HANDLE    hEvent;
```

```
// Parse the command line
if (ParseCommandLine (argc, argv, FALSE))
     return(1);

// call NCB.RESET
if (Reset())
     return (1);
// Create a manual-reset event
hEvent = CreateEvent ( NULL,
                       TRUE,      // manual-reset
                       FALSE,     // initial state = non-signaled
                       NULL       // unnamed event
                     );
if (hEvent == NULL)
{
     printf("Error: %d Creating Event\n", GetLastError());
     return(1);
}
// Register the client name by NCB.ADD.NAME
if (AddName (szClientName, &ucNameNum) == NRC_GOODRET)
{
     // post an NCB.CALL
     printf("Calling Server\n");
     ucRc = BlockingCall (szServerName, szClientName, &ucLsn);
     if (ucRc==NRC_GOODRET)
     {
          printf("Connected!\n");
          for ( i  = 0; i  < RWLoop; i++ )
          {
               // Send a message to server via NCB.SEND
               if (fShutDownServer)
                    sprintf(chBuffer, SHUTDOWN_MSG);
               else
                  sprintf(chBuffer, "Client Msg: %d", i);
               // Asynchronous NCB.SEND and wait on event to signal
               // completion of command. In this example, we simply
               // sleep. Real app will continue processing
               ResetEvent ( hEvent );
               ucRc = WaitOnSend ( ucLsn,
                                   &chBuffer[0],
                                   strlen(chBuffer) + 1,
                                   hEvent
                                 );
               if ((ucRc != NRC_GOODRET) || (fShutDownServer))
                    break; // hangup

               // Receive Reply from Server via NCB.RECEIVE
               usBytesRead = sizeof(chBuffer);
               ucRc = BlockingReceive ( ucLsn,
                                        &chBuffer[0],
                                        &usBytesRead
                                      );
               if ((ucRc == NRC_GOODRET) && (usBytesRead))
               {
                    printf("Server Replied: %s\n", chBuffer);
               }
```

```
                    else
                        break; // hangup the session
                }
                // Terminate the session via NCB.HANGUP
                HangSess ( ucLsn );
            }
            // Deregister the name via NCB.DELETE.NAME
            DeleteName (szClientName, ucNameNum);
            }
        CloseHandle (hEvent);
        return 0;
}
```

Now that you have seen three ways to submit asynchronous NetBIOS commands, let us focus on how to cancel a pending asynchronous command. Imagine a server that needs to shut down but has one or more pending NCB.LISTEN commands, or a client application that needs to be terminated while it has posted one or more NCB.RECEIVE commands and is awaiting a server response. NetBIOS provides a special command, **NCB.CANCEL**, which can be used to terminate pending commands. The main difference between NCB.HANG.UP and NCB.CANCEL is that while the former can be used to trigger all pending NetBIOS commands on a given local session number, the latter can be used selectively to cancel pending NetBIOS commands, whether a session exists or not.

In order to cancel a NetBIOS command, the caller must provide the address of the NCB. The address can be supplied via the ncb_buffer field. Of course, the ncb_command field is set to NCB.CANCEL, and the adapter number is set in the ncb_lana_num field. However, it is not valid to issue NCB.CANCEL on certain commands, listed in Table 8.4. When you examine the table, you will realize that NetBIOS programs are encouraged to use NCB.HANG.UP to terminate existing sessions, a good application programming practice of negotiating to set up or break up a session. A careful examination of the table will indicate that an application can cancel a pending NCB.RECEIVE (or a variant of receive) command. Thus, you should basically use the NCB.CANCEL command to end a pending receive command (probably prior to terminating the program).

8.3.5 Handling Multiple Clients and Large Data Transfers

Now that you have seen the basics of NetBIOS session services, we will concentrate on designing powerful client and server applications that use these services. In particular, I discuss the designing servers, which can service multiple clients simultaneously, and CSSs, which can handle large data transfers.

The sample code fragments shown so far demonstrated simple client and server applications in which the server serviced one client at a time. In actual

Table 8-4. NetBIOS Commands That Should Not Be Canceled

Service Type	*Commands*
Name management	NCB.ADD.NAME
	NCB.ADD.GROUP.NAME
	NCB.DELETE.NAME
	NCB.FIND.NAME
Session services	NCB.SEND
	NCB.SEND.NO.ACK
	NCB.CHAIN.SEND
	NCB.CHAIN.SEND.NO.ACK
	NCB.SESSION.STATUS
Datagram services	NCB.SEND.BROADCAST.DATAGRAM
	NCB.SEND.DATAGRAM
Miscellaneous	NCB.CANCEL
	NCB.RESET
	NCB.CANCEL

CSSs, more often than not, a server must service multiple clients simultaneously. If you examine the server code fragment (Server1.c) shown earlier, you will notice that the server posts an NCB.LISTEN command and waits for a client to connect to it. Once a client connects to the server, the server services the same client until the session is terminated. Thus, to serve multiple clients, the server needs to start multiple sessions—one with each client.

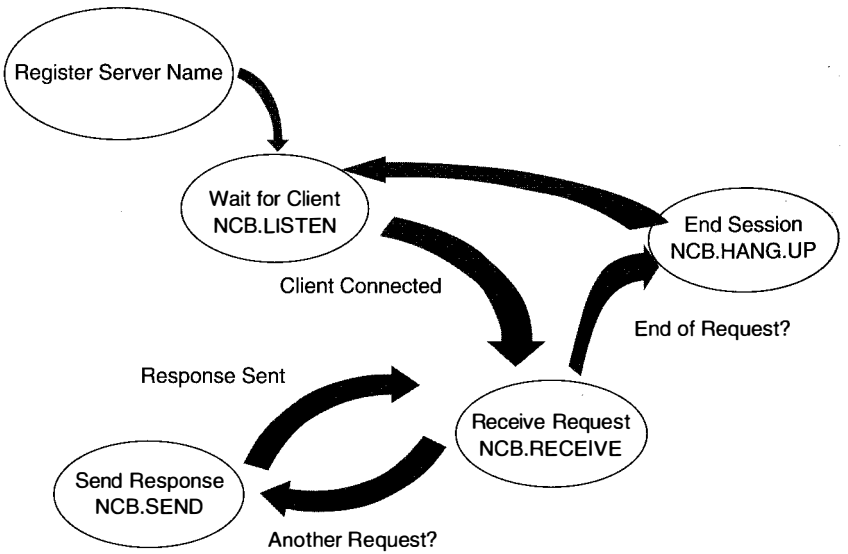
A designer can design a server in different ways so it services multiple clients. One method is for the server to start multiple threads, with each thread posting a blocking NCB.LISTEN command. In this case, a client call to connect will be serviced by one of the threads in the order the NCB.LISTEN calls were submitted. On successful connect, the connecting thread will receive a local session number, which it can use for further communication with the client. Although this scheme will lead to a design whereby a server can service multiple clients, the threads add to the overall application overhead, and eventually performance deteriorates as the number of threads increase. Additionally, it forces *N* threads to service *N* clients up front.

In a second approach, a modification of the first, the server maintains a pool of worker threads. The main thread posts a nonblocking NCB.LISTEN command. When a client connects with the server, the main thread hands the connection over to the next available worker thread, which (within the server) communicates with the client. This model leads to a controlled growth of threads as the number of clients increases; hence, it is recommended. It was

used in designing the NetBIOS-based EchoServer presented in Section 8.4, "A NetBIOS-Based Win32 Service."

A third approach is more common and conservative in system resource consumption. In this design, a server submits multiple asynchronous NCB.LISTEN commands and waits for new clients to connect using one of the techniques already described. When a client connects, one of the pending NCB.LISTEN commands completes returning a session number. The server can then service this new session. This design leads to complexity because a server has to keep track of both pending NCB.LISTEN commands and currently active sessions. Nevertheless, the complexity can be controlled easily by writing a state-driven server. This design works especially well with a post routine that posts a Windows message on completion. Indeed, Win32 applications can use Win32 events instead of post routines. In either case, the server is moved from one state to another by using the NetBIOS completion notification as the trigger (or edge). See [Sinha 92a] for an example of a state-driven server. Figure 8.11 shows a coarse-grained state machine for a server that uses asynchronous NetBIOS commands to service multiple clients.

In implementing the state diagram, the server would have to maintain an array of active sessions and for each active session submit a nonblocking NCB.RECEIVE command to receive commands from the respective client. Recall that to call NCB.RECEIVE, the server has to specify the local session number established when the client connected to it via the NCB.CALL command. An alternative design is simply to use the NCB.RECEIVE.ANY command, which, as the name suggests, can receive data from any active session.



Note: All session-related NetBIOS commands are assumed to be asynchronous.

Figure 8-11. *State Diagram for a Server Servicing Multiple Clients*

The use of NCB.RECEIVE.ANY is much like NCB.RECEIVE except that the caller does not specify the local session number. On the other hand, unlike NCB.RECEIVE, this command needs to have the name number associated with caller's name. At the minimum, the caller specifies the LAN adapter number, a buffer and its length in bytes, and the name number. Additionally, the caller can specify a post routine or an event for asynchronous execution of the command. On successful completion of the command, the NetBIOS driver returns the local session number (via the ncb_lsn field) identifying the session partner who sent the packet. This command thus allows the application to maintain a queue of pending NCB.RECEIVE.ANY and defer the session identification until a packet has been received from the session partner.

NCB.RECEIVE.ANY is extremely useful in designing applications that must service multiple clients that send and receive a large volume of data. Recall that when a sender sends a data packet via NCB.SEND (or a similar command), the NetBIOS driver on the receiver end collects the data packet in its own buffer pool until the receiver program issues NCB.RECEIVE (or a similar command). Hence, a large number of clients sending requests to a server will quickly usurp the resources of the NetBIOS driver unless the server has a pending NCB.RECEIVE command, which allows the NetBIOS driver to put incoming data directly into the server buffer. Hence, a typical server keeps two or more NCB.RECEIVE commands pending per session so that it can pick up client transmission (say, via NCB.SEND) quickly. This scheme improves the network I/O performance but places high resource demand on the server application since the server needs to maintain at least 2*N* pending NCBs at any given time, assuming *N* active session. This design will incur high overhead because the server needs to maintain the memory associated with 2*N* NCBs and its associated data buffers (maximum data buffer size of 2*N* x 64K). If events are used with NCB, the server needs to maintain 2*N* events. When post routines are used, the system will create 2*N* threads for the server.

This is where NCB.RECEIVE.ANY is handy because it allows the number of pending NCBs to M (2<= M <= N), thereby reducing the resource overhead from both the server and the underlying system. (The improvement in operation efficiency and reduced resources needed to service multiple clients is a well-known concept in queuing theory.)

One of the limitations of NCB.SEND-based data transmission is that the data buffer size is limited to 65,535 bytes (64K –1), primarily because MS-DOS programs were unable to handle a large buffer when NetBIOS was designed. Later versions of NetBIOS have a new command, **NCB.CHAIN.SEND**, which enables the sender to send up to 131,070 bytes (128K –2). Thus, a sender can transmit large data buffers that the receiver can pick up by using two NCB.RECEIVE or NCB.RECEIVE.ANY commands. Designers of an application that use the NCB.CHAIN.SEND command must be aware that without corresponding NCB.RECEIVE (or NCB.RECEIVE.ANY) commands on the receiver side, the NetBIOS driver

resources on the receiving end will be unnecessarily tied up, thus adversely affecting network I/O performance.

Because a non-NetBIOS transport protocol (such as IPX or TCP/IP) can provide a NetBIOS interface, the actual transport mechanism (thus the efficiency of transmission via NCB.CHAIN.SEND) may be quite different from one transport to another. Additionally, the NCB.CHAIN.SEND command is more expensive (in terms of CPU and memory usage) than a simple NCB.SEND command. Thus, designers are advised to determine the effect of using the NCB.CHAIN.SEND command on performance as well as resource usage for a particular application.

The use of the NCB.CHAIN.SEND command is very similar to that of the NCB.SEND command except that it allows two data buffers to be associated with each NCB. The two data buffers need not be consecutive, although both are sent as a single message. The caller specifies the first data buffer via ncb_buffer field, with ncb_length containing the length of the buffer. The second data buffer address is specified via the ncb_callname field; the first two bytes contain the length of the second data buffer, with the next four bytes containing the address of the second data buffer. Contrast this use with NCB.SEND, which does not use the ncb_callname field at all. The semantics and functioning of NCB.CHAIN.SEND are no different from NCB.SEND except for its ability to send twice the amount of data.

The following code fragments show the use of the NCB.CHAIN.SEND command. The client sends two buffers (although they are quite small) via the ChainSend() function, which associates the first buffer with the ncb_buffer field of the NCB, with the buffer length being specified by the ncb_length field. The second buffer (and its length) is pointed to by the ncb_callname field of the NCB. The client (Client5.c) works with the server (Server1.c) demonstrated earlier. In other words, the server that uses the NCB.RECEIVE command to receive messages does not need to use any special commands to receive double buffers. However, the client can send up to 128K –2 bytes via a single NCB.CHAIN.SEND command, so the receiving end must be prepared. When the receiver's buffer is smaller than the size of the message sent, NCB.RECEIVE will fill the receiver's buffer and return a special error (NRC_INCOMP). The sample program does not handle this error.

```
//
// ChainSend() -  Send two buffers using asynchronous NCB.SEND
//                and then wait for an event to signal indicating
//                command completion
//
UCHAR ChainSend(  UCHAR  ucLsn,
                  LPSTR  pchData1,
                  USHORT usDataLen1,
                  LPSTR  pchData2,
                  USHORT usDataLen2,
                  HANDLE hEvent
                )
```

```
{
     NCB Ncb;
     UCHAR ucRc;
     // First clear out the buffer
     memset (&Ncb, 0x00, sizeof(NCB));

     //  Set the Command. Asynchronous NCB.CHAIN.SEND
     Ncb.ncb_command = NCBCHAINSEND | ASYNCH;

     // set the LAN Adapter number
     Ncb.ncb_lana_num = ucLanaNum;

     // set the Local Session Number
     Ncb.ncb_lsn = ucLsn;

     // Set  first Data buffer pointer and its length
     Ncb.ncb_buffer = pchData1;
     Ncb.ncb_length = usDataLen1;

     // Set the 2nd data buffer pointer and its length
     *(USHORT*)(&(Ncb.ncb_callname [ 0 ]))  = usDataLen2;
     *(DWORD *)(&(Ncb.ncb_callname [ 2 ])) = (DWORD)pchData2;

     // Set the Win32 event
     Ncb.ncb_event = hEvent;

     // Make the NetBIOS call
     ucRc=Netbios(&Ncb);
     if (Ncb.ncb_retcode == NRC_PENDING)
     {
          // Wait for command to complete
          while ( WaitForSingleObject(hEvent, ONE_SECOND) == WAIT_TIMEOUT )
          {
               // Do other processing. Here, we simply sleep
               Sleep(1);
               printf(".");
          }
          printf("\n");
          ucRc = Ncb.ncb_cmd_cplt;
     }
     if (ucRc != NRC_GOODRET)
     {
          PrintError(ucRc, "Polling Send");
          return ucRc ;
     }
     return Ncb.ncb_retcode;
}
//
// FILE: %BOOK%\netbios\client5.c
//
//
// Client5 - Shows usage of asynchronous NCB.CHAIN.SEND
//                used with a Win32 event
//
#include "netfunc.h"
char      szClientName [NCBNAMSZ + 1] ;
char      szServerName [NCBNAMSZ + 1] ;
```

```
int       RWLoop;
BOOL      fShutDownServer;
#define MESSAGE_SIZE 80
#define SHUTDOWN_MSG "shutdown"
int Usage()
{
     printf("Client5 -s:<Server Name> -c:<Client Name> -l:<LANA num>\n");
     printf("        -k\n");
     return (1);
}
int _cdecl main (int argc, char ** argv)
{
     UCHAR     ucLsn, ucNameNum;
     UCHAR     ucRc = NRC_GOODRET;
     USHORT    usBytesRead;
     char      chBuffer  [ MESSAGE_SIZE ];
     char      chBuffer2 [ MESSAGE_SIZE ];
     int       i;
     HANDLE hEvent;

     // Parse the command line
     if (ParseCommandLine (argc, argv, FALSE))
         return(1);
     // call NCB.RESET
     if (Reset())
         return (1);
     // Create a manual-reset event
     hEvent = CreateEvent ( NULL,
                            TRUE,     // manual-reset
                            FALSE,    // initial state = non-signaled
                            NULL      // unnamed event
                          );
     if (hEvent == NULL)
     {
          printf("Error: %d Creating Event\n", GetLastError());
          return (1);
     }
     // Register the client name by NCB.ADD.NAME
     if (AddName (szClientName, &ucNameNum) == NRC_GOODRET)
     {
          // post an NCB.CALL
          printf("Calling Server\n");
          ucRc = BlockingCall (szServerName, szClientName, &ucLsn);
          if (ucRc==NRC_GOODRET)
          {
               printf("Connected!\n");
               for ( i  = 0; i  < RWLoop; i++ )
               {
                    // Send TWO messages to server via NCB.CHAIN.SEND
                    if (fShutDownServer)
                    {
                         sprintf(chBuffer, SHUTDOWN_MSG);
                         chBuffer2[0] = '\0';
                    }
                    else
                    {
                   sprintf(chBuffer,
                           "First Buffer >Client Msg: %d.<\n", i);
```

```
                // 2nd Buffer
                sprintf(chBuffer2,
                        "Second Buffer...
                >This is 2nd Message from Client.<");
            }

            // Asynchronous NCB.CHAIN.SEND and wait on event to signal
            // completion of command. In this example, we simply
            // sleep. Real app will continue processing
            ResetEvent ( hEvent );
            ucRc = ChainSend ( ucLsn,
                               &chBuffer[0], // first buffer
                               strlen(chBuffer),
                               &chBuffer2[0], // second buffer
                               strlen(chBuffer2) + 1,
                               hEvent
                             );
                 if ((ucRc != NRC_GOODRET) || (fShutDownServer))
                      break; // hangup

                 // Receive Reply from Server via NCB.RECEIVE
                 usBytesRead = sizeof(chBuffer);
                 ucRc = BlockingReceive (ucLsn,
                                         &chBuffer[0],
                                         &usBytesRead
                                        );
                 if ((ucRc == NRC_GOODRET) && (usBytesRead))
                 {
                      printf("Server Replied: %s\n", chBuffer);
                 }
                 else
                      break; // hangup the session
            }
            // Terminate the session via NCB.HANGUP
            HangSess ( ucLsn );
       }
       // Deregister the name via NCB.DELETE.NAME
       DeleteName (szClientName, ucNameNum);
  }
  CloseHandle (hEvent);
  return 0;

}
```

8.3.6 Communication Using Datagram Services

Datagrams or connectionless services allow any two processes to communicate with each other without setting up a session between themselves. An advantage of datagram services is the low system overhead (in terms of algorithmic complexity and software size) associated with transmitting data because the service does not guarantee data delivery. When an application needs to build a guaranteed data delivery channel on top of the datagram services, the total overhead will turn out to be more (user mode code as opposed to kernel mode) than that incurred during session-based communication. Thus, NetBIOS datagram service is not recommended for general client/server communication.

Datagrams are useful for occasional multicasting and broadcasting operations. For example, chat programs are often based on the NetBIOS multicasting (that is, group name usage) facility. It is also useful for locating a network entity so that a session can be established with it. For example, when a client posts an NCB.CALL.NAME command, the NetBEUI transport uses NCB.FIND.NAME to discover the location of the server; the NCB.FIND.NAME command causes broadcast datagrams to be sent over the network, and qualified workstations respond via datagrams as well. However, broadcast datagrams increase the CPU processing overhead in all the active nodes on the network since these nodes need to determine if the broadcast is for them. Thus, frequent broadcast datagrams adversely affect the performance of the network. Additionally, in large networks consisting of multiple LANs connected via bridges and routers, the broadcast datagrams need to be transported from one LAN to another, in the process affecting the throughput of the routers (or bridges).

Datagrams can be sent from one entity to another (one to one); they are referred to as directed datagrams. When they are sent from one entity to a collection of entities (one to many), they are referred to as multicast or group datagrams. Finally, a process may wish simply to notify all interested parties on the network (one to all). In such a case, it will use broadcast datagrams. NetBIOS supports all three datagram modes, and in each mode, the maximum size of data buffer associated with a datagram is at least 512 bytes.[5]

Directed datagrams can be sent using the **NCB.SEND.DATAGRAM** command and received by posting the **NCB.RECEIVE.DATAGRAM** command. The sender registers its name and must specify its name number (via the ncb_num field) when issuing the NCB.SEND.DATAGRAM command. It specifies the data buffer address (via the ncb_buffer field) and the length of the data buffer (via the ncb_length field). The sender also needs to specify the receiver's name (via the ncb_callname field). The sender can use the blocking or nonblocking version of NCB.SEND.DATAGRAM (via the ncb_command field). Unless the specified receiver has NCB.RECEIVE.DATAGRAM pending, the datagram sent by the sender is "lost," although the NCB.SEND.DATAGRAM command will complete without error. The receiver of a directed datagram must have submitted the NCB.RECEIVE.DATAGRAM command in order to catch the datagram sent by the sender. Normal practice is to have two or more pending receive commands to minimize packet loss.

5. Some systems such as Windows NT may have a higher limit. Applications can use the NCB.ADAPTER.STATUS command to determine the maximum datagram data size. This is an inherent weakness of NetBIOS because it uses broadcast datagrams to locate session partners. Hence, most commercial routers and bridges allow smart filtering of broadcast datagrams in that they do not forward a datagram from one LAN to another unless it contains NCB.FIND.NAME-related messages.

A receiver can specify a name number (via the ncb_num field), implying that it wishes to receive datagrams sent to a specific name. Alternatively, it can specify 255 (FFh) in the name number, indicating that it wishes to receive datagrams sent to any name registered on the receiver's process.

The following code fragment shows datagram-based **DatagramSend()** and **DatagramRecv()** functions, which can be used to send (or receive) broadcast datagrams as well as directed datagrams. The DatagramSend() function executes synchronous datagram commands (NCB.SEND.DATAGRAM or NCB.SEND.BROADCAST.DATAGRAM) as datagram transmission (traditionally) completes quickly. However, an application can seldom predict when it should post a command to receive a datagram message. Thus, the DatagramRecv() function associates a Win32 event with the NCB and submits an asynchronous command. The datagram command completion can then be detected by using the **CheckAsyncNcbStatus()** function, which simply waits on the Win32 event for a certain amount of time. When the datagram is not received within an acceptable time limit in this example, we simply cancel the pending NCB.RECEIVE.DATAGRAM command (or NCB.RECEIVE.BROADCAST.DATAGRAM).

These functions have been used in the datagram-based client (Client6.C) and the server (Server2.C). Notice that the client posts an NCB.RECEIVE.DATAGRAM (or any receive) command *before* sending a message to the server via the DatagramSend() function. This technique is often employed in NetBIOS-based communication to avoid missing a reply from the other end and is not limited to datagram-based communication.

```
//
// FILE: %BOOK%\netbios\netfunc.c
//
//
// DatagramRecv() - Receive a broadcast message via
//                  asynchronous NCB.RECEIVE.BROADCAST.DATAGRAM or
//                  asynchronous NCB.RECEIVE.DATAGRAM
//
UCHAR DatagramReceive ( PNCB          pNcb,
                        UCHAR         ucNameNum,
                        LPSTR         pchData,
                        USHORT       *pusDataLen,
                        HANDLE        hEvent,
                        BOOL          fBroadcast
                      )
{
     UCHAR ucRc;
     // First clear out the buffer
     memset (pNcb, 0x00, sizeof(NCB));
     //  Set the Command
     if (fBroadcast)
          pNcb->ncb_command = NCBDGRECVBC | ASYNCH;
     else
          pNcb->ncb_command = NCBDGRECV | ASYNCH;
```

```
    // set the LAN Adapter number
    pNcb->ncb_lana_num = ucLanaNum;

    // set the Name Number
    pNcb->ncb_num = ucNameNum;

    // Set  Data buffer pointer and its length
    pNcb->ncb_buffer = pchData;
    pNcb->ncb_length = *pusDataLen;

    // Set the Win32 event
    pNcb->ncb_event = hEvent;

    // Make the NetBIOS call
    ucRc=Netbios ( pNcb );
    if ( (pNcb->ncb_retcode !=  NRC_PENDING) &&
         (pNcb->ncb_retcode !=  NRC_GOODRET))
    {
         PrintError(ucRc, "DatagramReceive");
         return ucRc ;
    }
    return pNcb->ncb_retcode;
}

//
// CheckAsyncNCBStatus()
//   - Checks the status of pending async. NetBIOS command
//      After ~10 seconds, it cancels the pending command
//   - Assumes that a Win32 event was associated with the command
//
UCHAR CheckAsyncNCBStatus ( PNCB pNcb, HANDLE hEvent )
{
    int iRetries = 10;
    // Wait to receive a datagram
    while ( (iRetries) &&
            (WaitForSingleObject(hEvent, ONE_SECOND) == WAIT_TIMEOUT)
          )
    {
         // Do other processing. Here, we simply sleep
         Sleep (1);
         iRetries--;
    }
    if (pNcb->ncb_retcode == NRC_PENDING)
    {
         // Cancel the command
         printf("Canceling pending NCB.\n");
         CancelCommand ( pNcb );
    }
    return  pNcb->ncb_cmd_cplt;
}
//
// DatagramSend - Send a broadcast message via synchronous
//                NCB.SEND.BROADCAST.DATAGRAM or
//                NCB.SEND.DATAGRAM
//
UCHAR DatagramSend(
                  UCHAR       ucNameNum,
```

```
                    LPSTR       pchData,
                    USHORT      usDataLen,
                    BOOL        fBroadcast,
                    LPSTR       pchRemoteName // only for nonbroadcast
                   )
{
     NCB Ncb;
     UCHAR ucRc;

     // First clear out the buffer
     memset (&Ncb, 0x00, sizeof(NCB));

     //   Set the Command.
     if (fBroadcast)
          Ncb.ncb_command = NCBDGSENDBC;
     else
          Ncb.ncb_command = NCBDGSEND;

     // Set the recipient's name (for nonbroadcast only)
     if (!fBroadcast)
          memcpy(Ncb.ncb_callname, pchRemoteName, NCBNAMSZ);

     // set the LAN Adapter number
     Ncb.ncb_lana_num = ucLanaNum;

     // set the Name Number
     Ncb.ncb_num = ucNameNum;

     // Set  Data buffer pointer and its length
     Ncb.ncb_buffer = pchData;
     Ncb.ncb_length = usDataLen;

     // Make the NetBIOS call
     ucRc=Netbios(&Ncb);
     if (ucRc != NRC_GOODRET)
     {
          PrintError(ucRc, "DatagramSend");
          return ucRc ;
     }
     return Ncb.ncb_retcode;
}
//
// FILE: %NETBIOS%\book\client6.c
//
// Client6 - Shows usage of NetBIOS commands to send/receive
//                data via broadcast datagram.
// Usage   - Use with Server2
//
#include "netfunc.h"
char       szClientName [NCBNAMSZ + 1] ;
char       szServerName [NCBNAMSZ + 1] ;
int        RWLoop;
BOOL       fShutDownServer;
extern BOOL fBroadcast;
#define MESSAGE_SIZE 80
#define SHUTDOWN_MSG "shutdown"
int Usage()
```

```
{
     printf("Client6 -b -c:<Client Name> -s:<Server Name> -l:<LANA num> -"\
           "k\n");
     printf("            -b initiates broadcast datagrams\n");
     return (1);
}
int _cdecl main (int argc, char ** argv)
{
     UCHAR       ucNameNum;
     UCHAR       ucRc = NRC_GOODRET;
     USHORT      usBytesRead;
     char        chBuffer [ MESSAGE_SIZE ];
     int         i;
     HANDLE      hEvent;
     NCB         NcbReceive;

     // Parse the command line (to pick up LANA)
     if (ParseCommandLine (argc, argv, FALSE))
          return(1);

     // Check if the user wishes to send/receive broadcast datagrams
     if (fBroadcast)
         printf("Using Broadcast datagrams\n");

     // call NCB.RESET
     if (Reset())
         return (1);

     // Create a manual-reset event
     hEvent = CreateEvent ( NULL,
                            TRUE,     // manual-reset
                            FALSE,    // initial state = non-signaled
                            NULL      // unnamed event
                          );
     if (hEvent == NULL)
     {
          printf("Error: %d Creating Event\n", GetLastError());
          return (1);
     }
     // Register the client name by NCB.ADD.NAME
     if (AddName (szClientName, &ucNameNum) == NRC_GOODRET)
     {
          for ( i  = 0; i  < RWLoop; i++ )
          {
               // First Post an async. NCB.RECEIVE.DATAGRAM
               ResetEvent ( hEvent );
               usBytesRead = sizeof(chBuffer);
               ucRc = DatagramReceive ( &NcbReceive,
                                        ucNameNum,
                                        &chBuffer[0],
                                        &usBytesRead,
                                        hEvent,
                                        fBroadcast
                                       );
               if ((ucRc != NRC_PENDING) &&(ucRc != NRC_GOODRET))
                    break; // unknown error
```

```
            // Send a message to server via NCB.SEND.DATAGRAM
            if (fShutDownServer)
                sprintf(chBuffer, SHUTDOWN_MSG);
            else
                sprintf(chBuffer, "Client Msg: %d", i);

            printf("Sending Message\n");
            ucRc = DatagramSend  ( ucNameNum,
                                   &chBuffer[0],
                                   strlen(chBuffer) + 1,
                                   fBroadcast,
                                   (fBroadcast) ? NULL : szServerName
                                 );
            if ((ucRc != NRC_GOODRET) || (fShutDownServer))
                break;

            // Wait to receive datagram reply
            printf("Waiting to receive message\n");

            ucRc = CheckAsyncNCBStatus ( &NcbReceive, hEvent );
            if (ucRc == NRC_GOODRET)
            {
                if (NcbReceive.ncb_length)
                    printf("Server Replied: %s\n", chBuffer);
            }
            else if (ucRc == NRC_CMDCAN)
                continue; // we timed out waiting to receive msg
            else
                break; // get out.
        }
        // If Receive is pending, cancel the command
        if (NcbReceive.ncb_cmd_cplt == NRC_PENDING)
            CancelCommand (&NcbReceive);

        // Deregister the name via NCB.DELETE.NAME
        DeleteName (szClientName, ucNameNum);
    }
    CloseHandle (hEvent);
    return 0;
}

//
// FILE: %BOOK%\netbios\server2.c
//
// Server2 - Shows usage of NetBIOS command to send/receive
//           broadcast data.
// Usage   - Use with Client6
#include "netfunc.h"
char       szClientName [NCBNAMSZ + 1];
char       szServerName [NCBNAMSZ + 1];
int        RWLoop;
BOOL       fShutDownServer;
extern BOOL fBroadcast;
#define MESSAGE_SIZE 80
#define SHUTDOWN_MSG "shutdown"
int Usage()
```

```
{
     printf("Server2 -b -c:<Client Name> -s:<Server Name> -l:<LANA"\
          "num>\n");
     printf("        -b initiates broadcast datagrams\n");
     return (1);
}
int _cdecl main (int argc, char ** argv)
{
     UCHAR     ucNameNum;
     UCHAR     ucRc = NRC_GOODRET;
     USHORT    usBytesRead;
     char      chBuffer [ MESSAGE_SIZE ];
     int       i;
     HANDLE    hEvent;
     NCB       NcbReceive;

     // Parse the command line (to pick up LANA)
     if (ParseCommandLine (argc, argv, TRUE))
         return(1);

     // Check if the user wishes to send/receive broadcast datagrams
     if (fBroadcast)
          printf("Using Broadcast datagrams\n");

     // call NCB.RESET
     if (Reset())
          return (1);
     // Create a manual-reset event
     hEvent = CreateEvent ( NULL,
                            TRUE,     // manual-reset
                            FALSE,    // initial state = non-signaled
                            NULL      // unnamed event
                          );
     if (hEvent == NULL)
     {
          printf("Error: %d Creating Event\n", GetLastError());
          return (1);
     }

     // Register the server name by NCB.ADD.NAME
     if (AddName (szServerName, &ucNameNum)==NRC_GOODRET)
     {
          while ((fShutDownServer == FALSE) && (ucRc == NRC_GOODRET))
          {
               for (i = 0; i< RWLoop; i++)
               {
                    // post an NCB.RECEIVE.DATAGRAM
                    printf("Waiting to receive datagram\n");
                    usBytesRead = sizeof(chBuffer);
                    ResetEvent ( hEvent );
                    ucRc = DatagramReceive ( &NcbReceive,
                                             ucNameNum,
                                             &chBuffer[0],
                                             &usBytesRead,
                                             hEvent,
                                             fBroadcast
                                           );
```

```
                if (ucRc == NRC_PENDING)
                    ucRc = CheckAsyncNCBStatus ( &NcbReceive, hEvent );
                if (ucRc == NRC_GOODRET)
                {
                     if (NcbReceive.ncb_length)
                     {
                         printf("Client Sent: %s\n", chBuffer);
                          if (stricmp(chBuffer, SHUTDOWN_MSG)==0)
                          {
                               fShutDownServer = TRUE;
                               break; // end Server
                          }
                     }
                     else
                          printf("Blank broadcast received !\n");
                }
                else if (ucRc == NRC_CMDCAN)
                     continue; // we timed out waiting to receive msg
                else
                     break;

                // Now send a reply via NCB.SEND.DATAGRAM
                sprintf(chBuffer, "Server Reply: %d", i);

                printf("Sending Reply\n");
                ucRc = DatagramSend ( ucNameNum,
                                      &chBuffer[0],
                                      strlen(chBuffer) + 1,
                                      fBroadcast,
                                      (fBroadcast)?NULL:szClientName
                                    );
            } // end of for
        }
        // Deregister the name via NCB.DELETE.NAME
    DeleteName (szServerName, ucNameNum);
    }
    CloseHandle ( hEvent );
    return 0;
}
```

The NCB.SEND.DATAGRAM and NCB.RECEIVE.DATAGRAM commands can be used, respectively, to send datagrams to a group and receive datagrams sent to a group. Sending datagrams to a group is useful in implementing group communication software (that is, multicasting). For example, imagine two or more database servers that belong to a NetBIOS group DATABASE_SRVS. A typical database client can then locate a server willing to service it by sending a datagram to the group DATABASE_SRVS and picking the first responder as the nearest database server. Thus, to send multicast datagrams, the sender must specify a group name, whereas the receiver(s) must specify the name number they received upon successful registration of the group name on their respective systems. Thus, the previous code fragments showing to send and receive directed datagrams can be used unmodified to send and receive multicast datagrams.

In the previous program, **NCB.SEND.BROADCAST.DATAGRAM** and **NCB.RECEIVE.BROADCAST.DATAGRAM** are used to send and receive broadcast datagrams. The usage of broadcast datagrams are similar to directed datagrams. Generally, broadcast datagrams are used to locate a partner so that a session can be established with it. Note that most bridges and routers prevent broadcast datagrams from migrating from one physical LAN to another; thus, two workstations stationed on distinct LAN segments may not be able to communicate with one another if they use broadcast datagrams.[6]

8.4 A NETBIOS-BASED WIN32 SERVICE

In this section, we explore a Win32 server (EchoServer) that uses NetBIOS as the IPC mechanism to communicate with clients. The core structure of the service was presented in Chapter 2, and throughout the succeeding chapters, we have examined a version of Echo Server (and Echo Client) that uses various IPC mechanisms (RPC, Windows Sockets, Named Pipe, and Mailslot). In this chapter, we extend the EchoServer to use NetBIOS as the IPC mechanism. (See Chapter 2 for a description of the management of threads, resource pools, and client queues.)

In the sample code that follows, the Echo Server Win32 service creates one "listener" thread and multiple "worker" threads. The listener thread calls the WaitingListen() function, which posts an asynchronous NCB.LISTEN command. When an Echo Client connects with the server by calling the NCB.CALL command, the listener thread puts the new client information in the client queue. The client information contains the name of the client and the LSN number received by the listener thread. The client name is returned by the NCB.LISTEN command, because the command was posted with an asterisk in the ncb_callname field.

Once the listener thread puts the new client information in the client queue, the next available worker thread picks up the client and starts servicing it. In particular, it posts an asynchronous NCB.RECEIVE command via the **WaitingReceive()** function. The client, of course, is not aware of the worker thread (or the listener thread for that matter). Upon successful connection, it sends a message to the Echo Server via the NCB.SEND command. This message is received by the worker thread, which sends an echo of the original client message back to the client. At this time, the worker thread terminates the server side of the session.

```
//
// FILE: %BOOK\echosrv\netbios\echos.c - NetBIOS Based EchoServer
```

6. This is an inherent weakness of NetBIOS because it uses broadcast datagrams to locate session partners. Hence, most commercial routers and bridges allow smart filtering of broadcast datagrams in that they do not forward a datagram from one LAN to another unless it contains NCB.FIND.NAME-related messages.

```
//
typedef struct _sessData
{
     UCHAR ucLsn;     // Session established with client
     char chRemoteName[NCBNAMSZ + 1]; // One extra char for '\0'
}SESSION_DATA, *PSESS_DATA;


// The thread which blocks waiting for RPC calls
VOID CDECL WorkerThread(PVOID pF);
VOID CDECL ListenThread(PVOID pF);
void ShutDown(void);
DWORD InitializeNetBios();

// Global Data
static      UCHAR ucNameNum = 0;
#define     ECHO_SERVER_A "EchoServer"
#define     MAX_TALK_BUFFER_LENGTH 80
static      BOOL fStopService = FALSE;
static      char chServerName [NCBNAMSZ];
//
// Declare the Data Pool for NetBIOS
// Server
INITIALIZE_POOL(5, sizeof(SESSION_DATA))

BOOL InitEchoServer()
{
     DWORD  dwRc;
     BOOL   bRet = TRUE;
     // First initialize the Client Request Pool
     dwRc = InitializeQueue();
     if (dwRc == 0)
     {
          // Initialize the Data Pool
          InitializePool();
          // Get the NetBIOS Service Started
          dwRc = InitializeNetBios();

          // Finally Create the worker thread pool
          dwRc = InitializeThreads (WorkerThread);
          if (dwRc)
               bRet = FALSE;
     }
     return(bRet);
}
DWORD InitializeNetBios()
{
     DWORD dwRc = 0;

     // First Reset
     dwRc = Reset();
     if (dwRc == 0)
     {
          // Register a Name
          CopyName ( chServerName, ECHO_SERVER_A );
          dwRc = AddName ( chServerName, &ucNameNum );
          if (dwRc ==0)
```

```
            {
                // Start out "Listen Thread"
                _beginthread ( ListenThread,
                               4098,
                               NULL
                             );
            }
            else
                PrintError1("Error Adding Name: %lx\n", dwRc);
        }
        else
            PrintError1("Error Reset: %lx\n", dwRc);
        return(dwRc);
}
//
// In this thread, the server listens to incoming client
// request. Once the request arrives, this thread will
// put the incoming request in the request pool for
// worker threads which, in turn, will service the client

// Define Listen state transitions
typedef enum
{
    enPostListen = 0,
    enCancelListen,
    enWaitOnListen
} enListenState;

VOID CDECL ListenThread(PVOID  pF)
{
    DWORD             dwRc     = 0,
                      dwEventIndex;
    enListenState     ListenState;
    HANDLE            hListenEvent = NULL;
    PSESS_DATA        pSessionData = NULL;
    NCB               Ncb;
    UCHAR             ucLsn;
    char              chRemoteName [NCBNAMSZ];
    char              szMessage [ 256 ];

    // Create an event by which synchronize with Service Thread
    dwEventIndex = GetThreadSyncEvent ();
    if (dwEventIndex == -1)
    {
        PrintError("Could not create Thread Sync. Event\n");
        return;
    }
    //
    // Create a special event which we will wait on after posting
    // an asynchronous Listen
    hListenEvent  = CreateEvent(NULL, TRUE, FALSE, NULL);
    if (hListenEvent == NULL)
    {
        PrintWin32Error("CreateEvent");
        return;
    }
    //
```

```
// Loop forever listening to incoming clients
//
ListenState = enPostListen;
do
{
     // First check if Service needs to be stopped
     if (fStopService)
     {
          ListenState = enCancelListen;
     }
//
// State driven Asynchronous Listen
//
switch(ListenState)
{
case enPostListen: // Post a Listen

     //CopyName ( szRemoteName, ECHO_SERVER_A);
     CopyName ( chRemoteName, "*");
     dwRc = WaitingListen  ( &Ncb,
                             &chRemoteName[0],   // Client Name
                             &chServerName[0],   // Local name
                             &ucLsn,             // session number
                             hListenEvent
                           );
     if ((dwRc==NRC_GOODRET) || (dwRc==NRC_PENDING))
     {
          ListenState = enWaitOnListen;
          dwRc = WAIT_TIMEOUT;
     }
     else
     {
          PrintErrorl("Listen Failed: %lx\n", dwRc);
     }
     break;

case enCancelListen: // Cancel Pending Listen
     // in Win32 environment. The pending Listen
     // will automatically cancel when the thread
     // terminates. We will still try to clean up ourselves

     CancelCommand ( &Ncb );
     // Notify the service thread that we can be closed
     SetThreadSyncEvent (dwEventIndex);
     dwRc = 0;
     break;

case enWaitOnListen: // Wait on a posted Listen
     dwRc = WaitForSingleObject ( hListenEvent, 3000);
     if (dwRc == 0)
     {
     if (Ncb.ncb_retcode != NRC_GOODRET)
     {
               dwRc =  Ncb.ncb_retcode;
               PrintErrorl("Listen Error: %lx\n", dwRc);
               break;
     }
```

```
                    // A client has come in. Put his request
                    // in the Request Pool for the next free worker
                    if (GetPoolResource (&pSessionData))
                    {
                        // save the session number
                        pSessionData->ucLsn = Ncb.ncb_lsn;
                        // save the name of the client
                           memcpy(pSessionData->chRemoteName,
                                  Ncb.ncb_callname,
                                  NCBNAMSZ
                                 );
                        // pad the name with NULL for ease of printing
                        pSessionData->chRemoteName[NCBNAMSZ] = '\0';
                        sprintf(szMessage,
                             "Client connected: %s\n",
                             pSessionData->chRemoteName);
                        PrintInfoString("%s", szMessage);

                        // put this in the worker pool
                        if ((dwRc = QueueRequest ( pSessionData,
                             sizeof(SESSION_DATA)))!=0)
                        {
                             // Free the resource and break the session
                             PrintError1("Canceling Session", dwRc);
                             FreePoolResource ( pSessionData );
                             HangSess ( Ncb.ncb_lsn );
                         }
                    }
                    //
                    // Now post a new Listen
                    ListenState = enPostListen;
                    dwRc = WAIT_TIMEOUT;
               }
               else if (dwRc != WAIT_TIMEOUT)
               {
                    PrintError1("WaitFor..Listen : %lx\n", dwRc);
               }
          }
     } while (dwRc == WAIT_TIMEOUT);

     // Close the Listen Event
     CloseHandle ( hListenEvent );

     // Close Thread sync. event
     CloseThreadSyncEvent ( dwEventIndex );

     _endthread();
     return;

}
//
// The worker threads wait to get
// requests from the clients
//
// Define Worker Thread state transitions
typedef enum
```

```
{
     enWaitOnConnect = 0,       // Wait for client to connect
     enReceiveRequest,          // Receive client request (async)
     enWaitOnRecvComp1t,        // Wait for Receive to complete
     enSendReply,               // Send a response (synchronously)
     enHangupSession            // End session with Client
} enWorkerState;
VOID CDECL WorkerThread(PVOID pF)
{
     DWORD dwRc,    dwDataSize;
     ThreadData     *pTData;
     enWorkerState WorkerState = enWaitOnConnect;
     PSESS_DATA     pSession = NULL;
     unsigned short usDataSize;
     HANDLE         hReceiveEvent = NULL;
     char           chReceiveBuffer [ MAX_TALK_BUFFER_LENGTH ],
                    chSendBuffer [ MAX_TALK_BUFFER_LENGTH ];
     char           szMessage [ 256 ];
     NCB            ncbReceive;
     // Set the thread status
     pTData = (ThreadData *)pF;
     if (pTData == NULL)
     {
          PrintError("Null Thread Data\n");
          return;
     }
     else
          PrintInfo1("Thread Number: %d\n", pTData->dwEventIndex);
     // Create a special event which we will wait on after posting
     // an asynchronous receive
     hReceiveEvent= CreateEvent(NULL, TRUE, FALSE, NULL);
     if (hReceiveEvent == NULL)
     {
          PrintWin32Error("CreateEvent");
          return;
     }
     do
     {
          // First check if Service needs to be stopped
          if (fStopService)
          {
               WorkerState = enHangupSession;
          }
          switch (WorkerState)
          {
      case enWaitOnConnect: // Wait for Listen Thread
                            // to put data on queue
               dwRc = GetNextRequest ( &pSession, &dwDataSize);
               if (dwRc ==0)
               {     // Verify the data integrity
         if ((pSession== NULL) ||
             (dwDataSize < sizeof(SESSION_DATA)))
               {
               dwRc = 0;
                       PrintError("Data Corruption\n");
               }
```

```
            else
               {    // We have successfully gotten a new client request
                    WorkerState = enReceiveRequest;
                    dwRc = WAIT_TIMEOUT;
               }
            // end dwRc = 0
            else if (dwRc != WAIT_TIMEOUT)
                 PrintError1("GetNextRequest Error: %lx\n", dwRc);
            break;
       case enReceiveRequest:
            usDataSize = sizeof(chReceiveBuffer);
            dwRc = WaitingReceive (
                                 &ncbReceive,
                                 pSession->ucLsn,
                                 &chReceiveBuffer[0],
                                 &usDataSize,
                                 hReceiveEvent
                                 );
            if ((dwRc == NRC_GOODRET) || (dwRc == NRC_PENDING))
            {
                 dwRc = WAIT_TIMEOUT;
                 WorkerState = enWaitOnRecvComplt;
            }
            else
            {
                 PrintErrorl("Receive Error: %lx\n", dwRc);
                 dwRc = 0;
            }
            break;
       case enWaitOnRecvComplt:
            dwRc = WaitForSingleObject (hReceiveEvent, 3000);
            if (dwRc == 0)
            {
                 dwRc = ncbReceive.ncb_retcode;
                 if (dwRc == NRC_GOODRET)
                 {
                      // Receive is completed successfully
                      sprintf (szMessage, "Client: [%s] Sent: [%s]\n",
                      pSession->chRemoteName, chReceiveBuffer);
                      PrintInfoString("%s", szMessage);
                      dwRc = WAIT_TIMEOUT;
                      WorkerState = enSendReply;
                 }
                 else
                 {
                      PrintError1("Receive Error: %lx\n", dwRc);
                      dwRc = 0;
                 }
            }
            else if (dwRc != WAIT_TIMEOUT)
            {
                 PrintErrorl("WaitOn..Receive: %lx\n", dwRc);
                 dwRc = 0;
            }
            break;1
```

```
        case enSendReply:
             sprintf(chSendBuffer, "[%s] Echoed: %s",
                     ECHO_SERVER_A, chReceiveBuffer
                    );
             dwRc = BlockingSend(
                                 pSession->ucLsn,
                                 &chSendBuffer[0],
                                 (unsigned short)(strlen(chSendBuffer) + 1)
                                 );
             if (dwRc == 0)
             {
                  // Sent the request, now disconnect from the client
                  dwRc = WAIT_TIMEOUT;
                  WorkerState = enHangupSession;
             }
             else
             {
                  PrintError1("Error Sending Reply: %lx\n", dwRc);
                  dwRc = 0;
             }
             break;
        case enHangupSession: // Disconnect with client
             if (pSession)
             {
                  // Hangup the session
                  dwRc = HangSess (pSession->ucLsn);
                  if (dwRc != 0)
                  {
                      sprintf(szMessage,
                            "Error: %x on Hangup of Session: %x\n",
                             dwRc, pSession->ucLsn);
                      PrintErrorStr("%s", szMessage);
                  }
                  // Release the buffer
                  FreePoolResource ( pSession );
                  pSession = NULL;
                  // Get ready for next client
                  WorkerState = enWaitOnConnect;
                  dwRc = WAIT_TIMEOUT;
             }
             if (fStopService)
             {
                  // Notify the service thread that we can be closed
                  SetThreadSyncEvent (pTData->dwEventIndex);
                  dwRc = 0;
             }
             break;
        }    // end switch

    } while (dwRc == WAIT_TIMEOUT);
    // Close Thread sync. event
    CloseThreadSyncEvent ( pTData->dwEventIndex );
    _endthread();
    return;
}
```

```
BOOL PauseEchoServer()
{

     PrintError("Pausing the NetBIOS Service is not supported\n");
     return(TRUE);
}
BOOL ResumeEchoServer()
{
     PrintError("Resuming the NetBIOS Service is not supported\n");
     return(TRUE);
}
BOOL StopEchoServer()
{
     DWORD dwRc;
     // Signal all threads to quit
     fStopService = TRUE;
     WaitForAllThreadsToTerminate();
     // Remove the NetBIOS name from the Name Table
     dwRc = DeleteName (chServerName, ucNameNum);
     if (dwRc != 0)
     {
          PrintError1("Error Deleting Name: %lx\n", dwRc);
     }
     return(TRUE);
}
//
// FILE: %BOOK\echosrv\netbios\echoc.c - NetBIOS Based Echo Client
//
// Global Dat=
static char (                    ``MSZ + 1 ]; // One extra space for NULL
#define ECHO                       r"
#define ECHO                       :"
#define MAX_TALK_Burr...           30)
UCHAR   ContactServer ();
int main (int argc, char ** argv)
{
     UCHAR   ucRc;
     BYTE    bClientNo;
     char    chTemp [ NCBNAMSZ ];
     UCHAR   ucNameNum;
     // determine client number
     argc--;
     argv++;
     if (argc)
          bClientNo = (BYTE) atoi(argv[0]);
     else
          bClientNo  = 0;
     // Set client name (must be blank padded)
     sprintf(chTemp, "%s%d", ECHO_CLIENT_A, bClientNo);
     CopyName ( chClientName, chTemp );
     // pad the name with NULL for ease of printing
     chClientName [ NCBNAMSZ ] = '\0';
     printf("NetBIOS Echo Client: [%s] starting\n", chClientName);

     // First Reset NetBIOS
     ucRc = Reset();
     if (ucRc == 0)
```

```
    {
        // Register a Name
        ucRc = AddName ( chClientName, &ucNameNum );
        if (ucRc != 0)
        {
            printf("Error Adding Name: %x\n", ucRc);
            return (ucRc);
        }

        // Contact a server
        ucRc = ContactServer ();
        // Remove the NetBIOS name from the Name Table
        ucRc = DeleteName (chClientName, ucNameNum);
        if (ucRc != 0)
        {
            printf("Error Deleting Name: %x\n", ucRc);
        }
    }
    return(ucRc);
}
// Function: ContactServer
// It calls the server, sends a message and receives
// a message.
// For simplicity, we are using synchronous
// calls in the client code
//
UCHAR ContactServer ()
{
    UCHAR       ucRc;
    DWORD       dwRc;
    UCHAR       ucLsn;
    unsigned    short
                usBufSize;
    char        chReceiveBuffer [ MAX_TALK_BUFFER_LENGTH ],
                chSendBuffer    [ MAX_TALK_BUFFER_LENGTH ];
    char        chServerName    [ NCBNAMSZ ];
    HANDLE      hEvent;
    NCB         ncbReceive;
    // Create a manual reset event to be used with NCB.RECEIVE
    hEvent  = CreateEvent(NULL, TRUE, FALSE, NULL);
    if (hEvent == NULL)
    {
        printf("Error: %lx CreateEvent", GetLastError());
        return 1;
    }
    printf("Calling Echo Server\n");
    CopyName ( chServerName, ECHO_SERVER_A );
    ucRc = BlockingCall (
                        chServerName,
                        chClientName,
                        &ucLsn      // session number
                        );
    if (ucRc)
    {
        printf("Call Failed: %x\n", ucRc);
        return(ucRc);
    }
```

```
        // Successfully connected!
        printf("Connected! to the Server. Session no: %x\n", ucLsn);

        // Send a message to the server
        sprintf(chSendBuffer, "Hello From: %s!", chClientName );
        ucRc = BlockingSend(
             ucLsn,                                        // session number
             &chSendBuffer[0],                             // data to be sent
             (unsigned short)(strlen(chSendBuffer)+1)      // sizeof data
             );
        if (ucRc == 0)
        {
             // Receive Reply using asynchronous NCB.RECEIVE
             usBufSize = sizeof(chReceiveBuffer);
             ucRc = WaitingReceive(
                                  &ncbReceive,
                                  ucLsn,    // Session
                                  &chReceiveBuffer[0],
                                  &usBufSize,
                                  hEvent
                                 );
             // Wait for receive to complete
             do
             {
                  // wait for 3 seconds
                  dwRc = WaitForSingleObject ( hEvent, 3000 );
                  if (dwRc == 0)
                  {
                       // Check for errors
                       ucRc = ncbReceive.ncb_retcode;
                       if (ucRc == NRC_GOODRET)
                            // Print the data received from the server
                            printf("Data Received From server\n[%s]\n",
                                  chReceiveBuffer);
                      else
                     printf("Error receiving data from server: %x\n",
                             ucRc);
                  }
                  else if (dwRc != WAIT_TIMEOUT)
                       printf("WaitForSingleObject Error: %lx\n", dwRc);

             } while ( dwRc == WAIT_TIMEOUT ) ;
        }
        else
             printf("Error sending data to the Server: %lx\n", ucRc);

        // Hangup the session
        ucRc = HangSess ( ucLsn );
        if (ucRc != 0)
        {
             printf("Error hanging up session: %x\n", ucRc);
        }
        // Close the event
        CloseHandle ( hEvent );
        return (ucRc);
}
```

8.5 NETBIOS PROGRAMMING IN WINDOWS

This section presents guidelines for designers using NetBIOS programming in Windows (16-bit). Readers interested only in Win32 APIs can skip this section.

Although the architecture of Windows NT is quite different from Windows 3.x, the NetBIOS services offered on Windows are quite similar to those already illustrated in this chapter. All the NetBIOS commands presented (except for NCB.ENUM) are available on the Windows 3.x platform, so all of the programming concepts examined thus far are pertinent and immediately useful for Win16 programming, with the following exceptions:

- NetBIOS interfaces in the Win16 environment do not support associating a Win32 event via the ncb_event field of the NCB. In fact, the ncb_event field is pertinent only in Win32 environments. This means that applications can use only the ncb_cmd_cplt or the ncb_post field of the NCB to determine completion on nonblocking NetBIOS commands. I recommend using the post routines with asynchronous commands. The Client3.C program can be easily compiled and made to work in Win16 environments.
- In Windows 3.x, applications use the NetBiosCall() API to submit an NCB to the NetBIOS drivers. (The NetBiosCall() API expects the pointer in the ES:BX registers.) The following assembly program shows how to map the Netbios() API to the NetBiosCall() Win16 API.

```
;=============================================================
; FILE: %BOOK%\netbios\netbios.asm
;
; PURPOSE: Provides a NetBIOS command submission procedure for Win16
;          applications.
;
; FUNCTIONS:
;                     Netbios() - Corresponding to Netbios() WIN32 API
;=============================================================
;=============================================================
;         SEGMENT DEFINITIONS
;=============================================================
_TEXT          SEGMENT   WORD PUBLIC 'CODE'
_TEXT          ENDS
_DATA          SEGMENT   WORD PUBLIC 'DATA'
_DATA          ENDS
CONST          SEGMENT   WORD PUBLIC 'CONST'
CONST          ENDS
_BSS           SEGMENT   WORD PUBLIC 'BSS'
_BSS           ENDS
DGROUP         GROUP     CONST, _BSS, _DATA
               ASSUME    DS: DGROUP
               ASSUME    SS: NOTHING
.286                                 ; 80286 and above ONLY.
EXTRN NETBIOSCALL: FAR
_TEXT SEGMENT WORD PUBLIC 'CODE'
```

```
 ASSUME CS:_TEXT, DS:DGROUP

PUBLIC  Netbios

;=============================================================
; UCHAR _far _Pascal Netbios(PNCB pNcb);
;=============================================================
Netbios PROC FAR
     enter     0, 0
     push      di
     push      si

     les       bx, DWORD PTR [bp+6]
     call      FAR PTR NETBIOSCALL
     sub       ah, ah

     ;;;       AX = the NetBIOS return code.
     xor       ah, ah
     mov       al, BYTE PTR es:[bx+1] ; return the NCB return code
     pop si
     pop di
     leave
     ret
Netbios ENDP
_TEXT ENDS
     END
```


- Because of single-threaded, nonpreemptive Windows 3.x environments, applications using NetBIOS should avoid issuing blocking NetBIOS commands. This situation is aggravated by a single system-wide Windows message queue. Thus, an application that calls a synchronous NetBIOS command (for example, NCB.RECEIVE) can potentially hang the entire Windows workstation. Usually it is safe to call synchronous versions of the name management (NCB.ADD.NAME, NCB.DELETE.NAME) and cancellation (NCB.CANCEL, NCB.HANGUP) commands. All other commands are good candidates for post routine-based asynchronous programming.
- Whenever an NCB and its associated data buffers are used with asynchronous NetBIOS programming, the NCB as well as the data buffers should be locked in memory. An alternative is to use fixed data segments (DATA SEGMENT FIXED in .def file). The reason is that windows can move, swap, or discard memory during the lifetime of a blocking NetBIOS command. When the command completes, the NetBIOS driver will try to put the results in the original NCB. When the memory is not locked, a system crash is inevitable. See [Sinha 92b] for architectural details. The following code fragments show how to allocate and lock memory.

```
// Extracted from windowsx.h
#define     GlobalAllocPtr(flags, cb)  \
                  (GlobalLock(GlobalAlloc((flags), (cb))))
#define     GlobalReAllocPtr(lp, cbNew, flags)  \
                  (GlobalUnlockPtr(lp),
GlobalLock(GlobalReAlloc(GlobalPtrHandle(lp),
           (cbNew), (flags))))
#define  GlobalFreePtr(lp)                \
                  (GlobalUnlockPtr(lp),
(BOOL)GlobalFree(GlobalPtrHandle(lp)))

// Sample usage
// Allocate an NCB and its assocaited data as a single contiguous memory
PNCB AllocateNCBAndBuffer (DWORD dwBufSize)
{
         PNCB pNcb = GlobalAllocPtr (GHND, sizeof(NCB) + dwBufSize);
         pNcb->ncb_buffer = (LPSTR)(pNcb) + sizeof(NCB);
}
// Release the NCB and its associated data
void FreeNcbAndBuffer (PNCB pNcb)
{
         GlobalFreePtr ( pNcb );
}
```

- The post routine used with an NCB should also be locked in memory. Alternatively, it can be put in a DLL whose code segment is declared FIXED. This is necessary because the post routine is called by NetBIOS drivers at interrupt time and the code must not move, be swapped, or be discarded during the lifetime of an asynchronous NetBIOS command. Additionally, the post routine should not affect any register and must return with RETF. The application should do minimum processing work in the post routine. The following code fragments illustrate a Win16 post routine using the PostMessage() API to send messages to the main Window. This works in the same way as illustrated in the Client3.c sample given earlier in this chapter.

```
//
// PostRoutine() - This function is called by NetBIOS driver when
//                 a NetBIOS command asynchronously completes or aborts
//
// Entry:    ES:BX pointed to completed NCB.
// Exit:     must return with RETF.
// Comments: One must use one's own stack if large stack usage is expected.
//           The stack must be restored before returning.
//
//            See %BOOK%\netbios\netfunc.c for NOTIFYNCB structure
//
void  _loadds cdecl interrupt far
PostRoutine(unsigned es,unsigned ds, unsigned di, unsigned si, unsigned bp,
            unsigned sp,unsigned bx, unsigned dx, unsigned cx, unsigned ax,
            unsigned ip, unsigned cs, unsigned  flags
           )
```

```
{
     // Upon entry, ES:BX points to completed ncb
     PNCB pncb;
     pncb = (PNCB) MAKELONG (bx, es);
     if (pNcb)
     {
          // Get to the NOTIFYNCB structure
          PNOTIFYNCB pNotify = NOTIFY_PTR ( pNcb );
          // Send notification message
          if (pNotify)
               PostMessage ( pNotify->hPostWindow,
                             pNotify->uiPostMessage,
                             0,
                             (LPARAM) pNcb
                           );
     }
}
```

The NetBIOS programming interface is fully functional in Windows 3.x workstations. Designers using NetBIOS in Windows must take certain steps to ensure stable application. In particular, the applications should use asynchronous commands as much as possible. When nonblocking commands are being used with post routines, the post routine code segment must be locked in memory by creating a fixed code segment DLL. Similarly, the NCB and its associated buffers should be locked in memory during the lifetime of asynchronous commands.

8.6 SUMMARY

The chief advantage of NetBIOS is that it is supported by all major commercial operating systems. Thus, a client/server designer can design a communication layer that can be easily ported to multiple environments. Additionally, clients and servers in heterogeneous platforms can communicate with each other via NetBIOS. The major drawback of using NetBIOS as an IPC mechanism is that it is a low-level mechanism tied to a transport and is hard to use and program with. A designer is forced to write communication software layers on top of NetBIOS. On the other hand, high-level IPC mechanisms like RPC or Windows Sockets provide simple and easy-to-use interfaces. More important, RPC and Windows Sockets are not tied to a specific transport protocol.

Chapter Nine

SPX/IPX PROGRAMMING IN WINDOWS NT

The SPX (Sequenced Packet Exchange)/IPX (Internetwork Packet Exchange) programming interface, popularized by Novell NetWare in the MS-DOS, Windows, OS/2, UNIX, and now Windows NT environments, can be compared to the Windows Sockets IPC mechanism in capabilities and strengths. They are the most prevalent IPC mechanism in Novell NetWare LAN environments.

In summer 1995, the Novell NetWare Client for Windows NT was released. A software development kit (Novell SDK Vol. 3) containing the SPX/IPX headers for Windows NT indicates that the SPX/IPX programming APIs have not changed between Windows 3.x and Windows NT. However, IPX/SPX libraries and DLLs have not yet been released. For this chapter, I have taken the following steps:

- I have assumed that the SPX/IPX programming interfaces on NetWare Client for Windows NT will be the same as those on a NetWare Client for Windows 3.x. The early software development kit confirms that this is a safe assumption. We can also safely assume that the NetWare Client for Windows NT will support existing Win16 SPX/IPX-based applications. In other words, Win16 applications written to the SPX/IPX interfaces illustrated in this chapter will function when executed on a Windows NT workstation.
- The sample program presented in this chapter is a Win16 application and has been tested in the Windows 3.x environment only. Within 45 days of the release of the final IPX/SPX libraries and DLLs for Novell NetWare Client for Windows NT, I will put the Win32 version on **ftp://ftp.aw.com/cseng/sinha/windowsnt.**
- Within 45 days of release of the final IPX/SPX libraries and DLLs for Novell NetWare Client for Windows NT, I will put on **ftp://ftp.aw.com/cseng/sinha/windowsnt** any changes in this chapter that pertain to SPX/IPX-based Win32 applications and services. At the same time, I will present a Win32 service (Echo Server) that uses the SPX/IPX interfaces.

9.1 AN OVERVIEW OF NOVELL NETWARE

In the area of PC network software, Novell NetWare is the dominant LAN software used to interconnect computers over LANs and WANs. While Novell NetWare version 3.x provides traditional file/print server services to MS-DOS, Windows, and OS/2 Clients, NetWare 4.x has the additional capability to provide directory services. In the NetWare environment, one or more computers are dedicated to being a NetWare server. These servers provide secure access to their services to clients on the network. Although clients can communicate with each other using IPC schemes, the architecture is geared toward CSSs. It is natural for companies that wish to provide additional services, such as database servers and tape backup servers, to run their programs on the NetWare servers, thus allowing clients to communicate to their servers in the same way clients access servers to access files. In NetWare 3.x, programs running on a NetWare server are called **network loadable modules (NLMs)**. A typical NetWare server has multiple NLMs, each providing a specific service. The workstations, usually referred to as clients in Novell literature, run software that allows them to access the servers.

A client can be an MS-DOS, Windows, Window NT, UNIX, or OS/2 workstation. In each such client environment, Novell provides a redirector, which redirects network requests to the appropriate NetWare server. For example, an MS-DOS Novell Netware 3.x redirector (netx.com) is a protected mode TSR, while in the Windows environment, a virtual device driver (netware.drv) handles network requests from Windows applications. In the Windows NT environment, the Novell redirector is a device driver much like the Windows NT default redirector. (See Chapter 2, "Understanding Windows NT Architecture," for details on the Windows NT redirector.) Once the appropriate redirector is in place and the user has been authenticated by a NetWare server, users and applications can access one or more NetWare file servers in the same way as they access a local drive.

As shown in Figure 9-1, NetWare clients and servers communicate with each other via the **NetWare core protocol (NCP)**, with the protocol packets transmitted using the IPX protocol. In NetWare clients using TCP/IP protocol, the IPX packets themselves are wrapped in an IP packet. In other words, a client does not necessarily need the IPX/SPX transport protocol to communicate with a NetWare server as long as they share at least one common transport protocol (for example, TCP/IP). This abstraction layer allows diverse workstations to communicate with the NetWare server.

Microsoft-supplied Client Services for NetWare (CSNW) and NetWare SPX/IPX-compatible protocol (NWLink) fit into this picture. The CSNW package contains a Microsoft-supplied redirector (and certain DLLs) that can

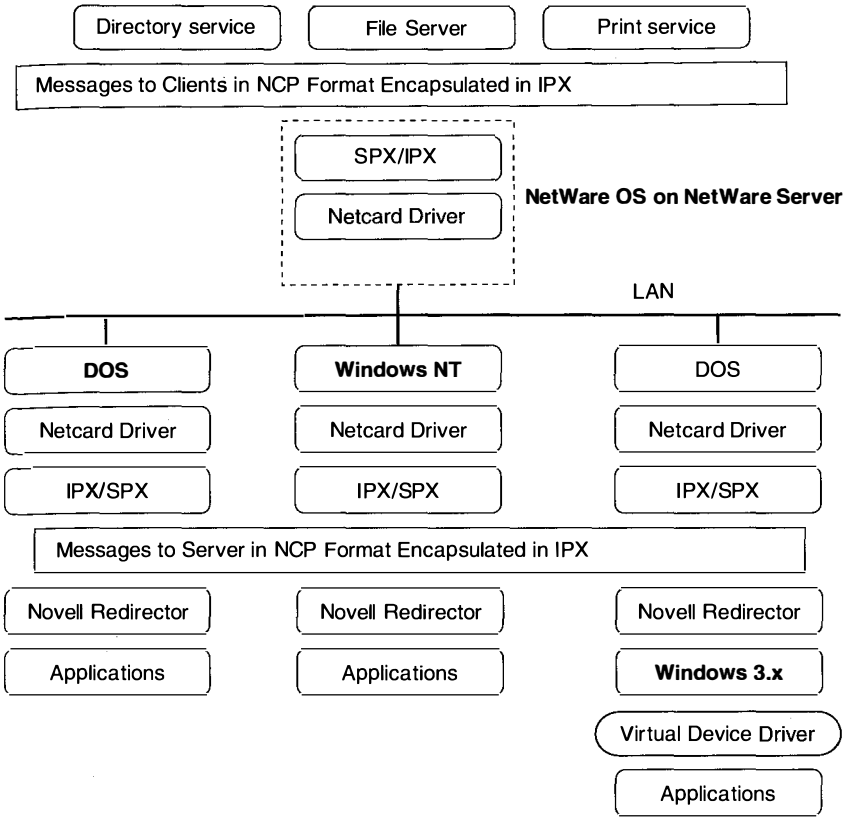


Figure 9-1. *Schematic View of NetWare Server and Workstations*

be used in lieu of the Novell-supplied NetWare client for Windows NT to communicate with NetWare 3.x Servers. In Windows NT 3.5, the CSNW does not support client access to high-end services (for example, directory services) of NetWare 4.x servers. Similarly, the NWLink protocol provides an SPX/IPX transport facility to enable Windows NT workstations to communicate with NetWare servers. It also allows simple NetWare utilities to function on a Windows NT workstation to access the file/print services of a NetWare server.

Essentially, the CSNW and NWLink software has been provided to help users move to the Windows NT platform immediately while awaiting release of NetWare Client software from Novell. The NWLink/CSNW package, however, does not attempt to support all the Novell NetWare client-side APIs. Thus, applications that directly use the Novell NetWare APIs will need a Novell-supplied NetWare client.

Years ago, every NetWare-enabled workstation communicated with the NetWare server only via the IPX transport protocol. The IPX transport provides a nonguaranteed datagram service and is a derivative of the **Xerox network systems (XNS)** protocol. This transport protocol was used by the NetWare redirector via a programming interface aptly termed the IPX programming interface. Later, the SPX interface was added, leading to the SPX/IPX interface; the SPX provides a guaranteed data delivery service. These interfaces were also made available to applications that used these interfaces to create peer-to-peer and workstation-to-server communication links. However, with the need of other transports on a workstation (for example, TCP/IP, NetBIOS), the Novell redirector was made capable of communicating over any transport as long as it supported the IPX interface. Thus, the SPX/IPX programming interface may be available on a workstation even when no SPX/IPX transport protocol is being used. (This may not be supported in NetWare Client for Windows NT. However, it is supported in Windows 3.x.) Additionally, when a Windows NT (or Windows95 or Windows 3.x) workstation uses NWLink, the NWLink provides access to SPX/IPX only via the Windows Sockets interfaces. These concepts are illustrated in Figure 9-2.

In Chapter 5, we explored the access of the SPX/IPX programming interface via the Windows Socket interface. In this chapter, we concentrate on using the SPX/IPX directly via the NetWare client communication APIs. The term IPX driver or SPX driver will be used to denote any or all drivers and DLLs needed to provide an application SPX or IPX programming support. Given the focus of this book on IPC, most of the chapter is devoted to SPX/IPX programming except the following section, which gives a brief description of various services offered by NetWare servers.

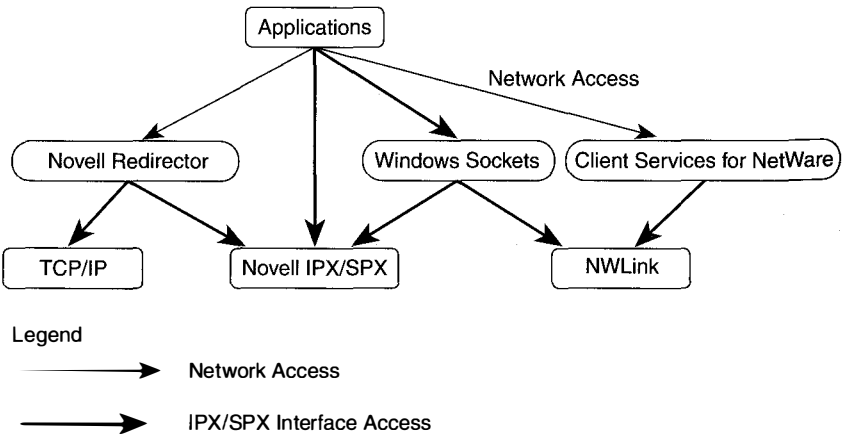


Figure 9-2. *SPX/IPX Programming Interface Schematic*

9.2 NETWARE SERVICES

Novell networking solutions are concentrated around NetWare servers. This approach is in contrast with the Windows NT approach wherein each workstation has file print/server capabilities. It should therefore come as no surprise that most of the NetWare services are offered by a NetWare server. (Personal NetWare and UnixWare are exceptions. They provide file share/print capability to the workstation much as Windows NT or WFW does.) The advantage of this approach is that each workstation has minimal NetWare-specific software—a winning strategy for low-end workstations. The NetWare services can be used by a series of tools provided by Novell.

These services can be used by application programs with a set of application programming interfaces. Novell has provided C and assembly interfaces to NetWare services for MS-DOS, Windows, and OS/2 applications [Novell 95]. Novell has already released the API for Windows NT. However, in all likelihood, it will simply port its existing Windows APIs to the Windows NT environment with some possible enhancements. Thus, programmers familiar with OS/2, MS-DOS, or Windows NetWare APIs should have relative ease in porting their applications to the Windows NT environment. The following list describes the most interesting services:

- *Directory service*—Allows information about organization-wide resources to be put in one commonly accessible information base. Novell directory service is X.500-like, while not being completely compatible.
- *Accounting service*—A bindery-based accounting system used for charging users of NetWare servers for resources such as connection time and disk storage.
- *Apple file service*—Allows users of Macintosh computers to access files on the NetWare server using **AppleTalk Filing Protocol (AFP)**; one key feature is support for long file names (1 to 31 characters).
- *Auditing service*—Provides a means to audit use of network resources such as files and to track special events such as security events.
- *Bindery service*—Before directory service, this was used as the database of network objects containing information on global resources. Bindery is also used by servers to advertise the availability of their services to potential clients.
- *NetWare server environment service*—Allows workstations to attach to NetWare servers, for users to log in to servers, and for controlling a server.

- *File system service*—Defines platform- and version-independent functions to access file system services on a NetWare server (for example, setting special access controls on files and directories).
- *Name space service*—Allows workstations to maintain files on a NetWare server in a file format compatible with their native system (for example, installing a Macintosh name space allows Macintosh workstations to use Macintosh file-naming conventions).
- *Print service*—Exposes platform-independent and version-independent functions and services to redirect print jobs to printers attached to NetWare servers.
- *Queue service*—A generic queue-management system that allows one or more workstations to send jobs to a queue attached to a server. Print service is based on the queue-management system.
- *Synchronization service*—Synchronization of file access by multiple users is provided via file/record locking. Also offers network semaphores, which can be applied to nonfile resources as well.
- *Transaction tracking service*—Gives applications and administrator a means to maintain file integrity by backing out an interrupted or an incomplete transaction due to application, server, network, or workstation failure.
- *Workstation service*—Used to configure and maintain the workstation environment by managing server connections, local drive mapping to server resources, and messages received by a workstation.
- *Communication services*—A range of client/server and peer-to-peer communication services, including SPX/IPX, TLI, and Named Pipe.

Consult the following references for help on making a selection:

- Davis, Ralph. *Windows Network Programming: How to Survive in a World of Windows, MS-DOS, and Networks*. Reading, MA: Addison-Wesley, 1993. Covers SPX/IPX programming in Windows. Readers must be familiar with NetWare.
- Davis, Ralph. *NetWare 386 Programmer's Guide*. Reading, MA: Addison-Wesley, 1991.
- McCann, John. *NetWare Programmer's Guide*. Redwood, CA: M&T Publishing, 1990. Explains NetWare services in terms of real-world problems. It is not an in-depth report on each service by APIs. Included are sample code fragments suitable for the MS-DOS environment.
- Day, Michael, et al. *Novell's Guide to NetWare 4.0 NLM Programming*. Salt Lake City, UT: Sybox/Novell Press, 1993. The authoritative Novell NetWare 4.0 reference manual.

9.3 IPX PROGRAMMING

In this section, we present the basic overview of IPX programming and the use of IPX programming interfaces available on NetWare clients. In the next section, we discuss SPX interfaces. Note that the SPX functions are built on the IPX functions, so an SPX-based program will use IPX functions as well as SPX-based functions. Thus, if you are unfamiliar with SPX/IPX programming, peruse this section before proceeding to the SPX programming section.

IPX, a connectionless or datagram-based communication protocol, is the most commonly used protocol between the NetWare workstations and servers. It limits the data packet size to 576 bytes, which includes protocol-specific header information and thus limits the maximum user data size to 512 bytes. The packet size limitations are determined by the underlying MAC driver; thus, the MAC driver may allow larger packets. Additionally, larger packet sizes are also allowed with the SPX II protocol and LIP (large Internet packet) protocols. For the rest of the chapter, we will assume that the IPX packet is limited to 576 bytes, with data sizes being limited to 512 bytes, a safe and common assumption in Ethernet/Token Ring–based networking. (Readers are encouraged to determine the exact limits in their environment.)

Because IPX is a datagram-based protocol, the application using it must implement its own mechanisms to handle problems such as lost, duplicate, and out-of-sequence packets. Over the years, Novell has gained high user satisfaction by using IPX to achieve high file transfer rates due to the low overhead of IPX and general low network error on a LAN. (As compared to the Internet and WANs). Hence, it is natural that Novell exposed the IPX protocol at the application level so that applications could use IPX for fast IPC. Compared to the Named Pipe, RPC, and Windows Sockets IPC interfaces, the IPX programming interface is a low-level interface (comparable to the NetBIOS programming interface). However, it is one of the most frequently used IPC mechanisms because of its low overhead, platform- and version-independent programming interface, and the predominance of NetWare in PC networking markets.

An application can use IPX to communicate with a peer application running on another workstation or with a server present on a NetWare Server. IPX programming interfaces are built around the IPX socket construct, which is similar to the Windows Socket. An application can read from, or write into, the IPX socket to receive and send data to the other end of the socket. Following the client/server programming paradigm, a server must have created one end of the IPX socket before the client can connect to it. Thus, schematically, the life cycle of an IPX connection or socket can be described by Figure 9-3.

To begin communication, an application must know the address of its counterpart. In the NetWare environment, applications use the **service advertising protocol (SAP)** to find the address of a remote server. Alternatively,

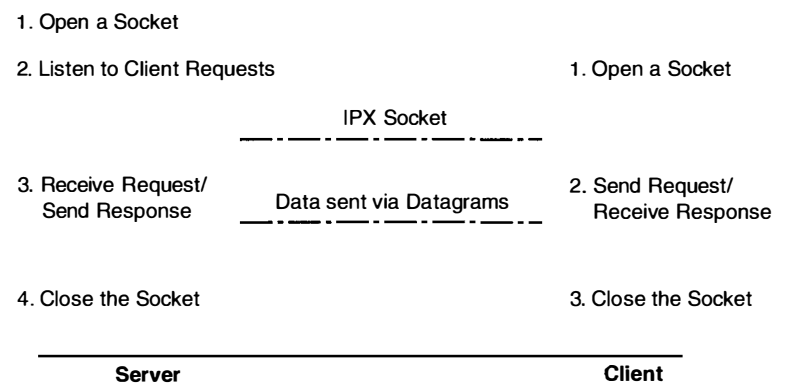


Figure 9-3. *Life Cycle of an IPX Socket*

it can query the user for addresses or revert to a fixed address. The former approach is flexible, allowing any number of servers to provide a specific service (for example, print service); the latter is suitable during the design and development of client/server systems.

A network entity address for IPX-based communication comprises three elements:

1. *Network address:* Identifies a particular LAN (analogous to the subnet in TCP/IP addressing schemes).
2. *Node address:* Pinpoints a particular workstation on the LAN.
3. *Socket number:* Uniquely defines the process on a workstation that needs to receive data.

Of course, an application can determine its own address by calling a simple local function. With both source and destination address in hand, an application can begin to send and receive datagrams.

9.3.1 IPX Programming APIs

The IPX programming interfaces can be used to build a datagram-based communication substrate in a client/server system. They are similar to the Windows Socket functions and provide a highly efficient, asynchronous data transfer mechanism. In this model, the servers create one end of an IPX socket and wait for the client packets to arrive. Hence, the servers "listen" to the incoming packets, and the clients "send" packets. Indeed, when duplex communication is desirable, the distinction between the actions of a client and a server fades because both tend to send and receive packets. Nevertheless, we will continue to focus on the client/server communication model wherein the server does not initiate a conversation; instead, it always awaits client's requests, sending back one or more responses upon receiving a query from the client.

Table 9-1 shows the most common Novell NetWare IPX functions for Windows and Windows NT. (In Novell literature, these functions are also referred to as the NetWare local management functions.) Every application needs to call the **IPXInitialize()** function first. Every IPXInitialize() call must be matched by a corresponding **IPXSPXDeinit()** call. The IPXSPXDeinit() function is usually called after an application does not need to use IPX (or SPX) functions anymore.

Table 9-1. *IPX Functions Categorized by Usage*

Function	*Category*	*Description*
Initialization and cleanup functions		
IPXInitialize()	All	Initialize IPX driver and assign resources to the application
IPXSPXDeinit()	All	Release all resources associated with the application
IPX Socket manipulation functions		
IPXOpenSocket()	All	The function that opens a socket (a socket must be opened before packets can be received)
IPXListenForPacket()	Receiver	Listens for datagram packets sent by any entity; on receiving packets, the IPX driver invokes the ESR of the receiver
IPXCloseSocket()	All	Closes the socket; the server will not receive any further packets
IPXSendPacket()	Sender	Sends data packets; the sender can send up to 546 bytes of data to a node whose address is specified in the ECB
IPXDisconnectFromTarget()	Sender	A courtesy call by the sender notifying the IPX driver that it will stop transmitting packets
Management functions		
IPXGetLocalTarget()	Sender	Returns the value to be placed in the immediateAddress field of ECB; the function takes the full address of the intended receiver (server)
IPXGetLocalTargetAsync()	Sender	Calls the listen ESR; a useful function when ESR is involved in sending packets

(continued)

Table 9-1 continued

Function	*Category*	*Description*
IPXScheduleIPXEvent()	All	An event can be associated with an IPX socket so that the ESR gets called after elapse of the specified time; useful for accomplishing periodic tasks
IPXCancelEvent()	All	Cancels a pending event associated with an IPX socket (includes canceling a pending listen/send ECB)
IPXGetIntervalMarker()	All	Determines the time elapsed between two system events; the granularity is limited to a PC clock tick (1/18 second)
IPXRelinquishControl()	All	Called by an application to yield CPU to the IPX driver; essential in nonpreemptive environments such as MS-DOS and Windows 3.x
IPXGetMaxPacketSize()	All	Can be called to determine the maximum packet size allowed by the underlying topology and network configuration
IPXGetInternetworkAddress()	All	To set the address of the calling application so the address can be sent to remote applications

An application wishing to receive datagram messages via IPX uses the **IPXOpenSocket()** and **IPXListenForPacket()** APIs. It can stop receiving data by calling the **IPXCloseSocket()** API. The sender uses the **IPXSendPacket()** function to send datagrams to one or all (broadcast) recipients.

A sender supplies the full network address of the receiver to the **IPXGetLocalTarget()** function, which returns the 6-byte network node address of the target or the address of the router via which IPX packets are sent to the target. Finally, applications can call the **IPXScheduleIPXEvent()** to receive IPX events at certain intervals so they can complete periodic functions or delay execution of a task until the system is idle.

9.3.2 Data Structures Relevant to IPX Programming

Unbeknownst to most applications, when they use IPX to send and receive data packets, the data are prepended with the data structure that follows.

```
typedef struct IPXAddress {
          BYTE network [4];
          BYTE node [6];
          BYTE socket [2];
};
typedef struct IPXHeader {
          WORD checkSum;
          WORD length;
          BYTE transportControl;
          BYTE packetType;
          IPXAddress destination;
          IPXAddress  source;
};
```

Each data packet sent and received by applications contains two parts: a fixed-size IPX header, followed by 0 or more bytes of data. The IPX header contains the address of the sender (via the source field) and the receiver (via the destination field). In addition, each header contains the length of the entire packet in bytes (see the length field). Recall that the maximum size of an IPX packet is 576 bytes, which leaves only 546 bytes for data given 30 bytes of IPX header. Most applications restrict themselves to 512 bytes of data per packet. IPX itself ignores the checksum field. The packetType field is used internally to distinguish between protocols. Table 9-2 shows the packet types and the corresponding protocols. For IPX-based communication, the packet type can be 0 or 4.

The transportControl field contains all network routing information.[1] Fortunately, Novell provides APIs to initialize the values in the header. Additionally, the IPX programming paradigm encourages the packet structures to be initialized once and then used multiple times by changing only the data

Table 9-2. *IPX Packet Types*

Packet Type	*Description*
0	Unknown packet type
1	Routing information packet
2	Echo packet
3	Error packet
4	Packet exchange packet (IPX)
5	Sequenced packet protocol packet
16–31	Experimental protocols
17	NetWare core protocol (NCP)

1. It contains the number of times a packet went from one network to another as it traveled from the source to the destination. This measurement is also referred to as the hop count.

or the destination address fields. This reuse is not like NetBIOS programming, whereby the NCB is cleared and all values are explicitly set prior to each call.

Programmatically, applications submit an **event control block (ECB)** to the IPX APIs, (similar to submitting NCBs to the Netbios() API). An application specifies the IPX header as well as the data buffer via an ECB to the IPX driver. A prototype of the ECB structure follows:

```
typedef struct tagECBFragment
{
        LPVOID                  address;
        WORD                    size;
} ECBFragment, FAR *PECBFragment;
typedef struct tagECB{
        LPVOID                  linkAddress;
        FARPROC                 ESRAddress;
        BYTE                    inUseFlag;
        BYTE                    completionCode;
        WORD                    socketNumber;
        BYTE                    IPXWorkspace [4];
        BYTE                    driverWorkspace [12];
        BYTE                    immediateAddress [6];
        WORD                    fragmentCount;
        ECBFragment             fragmentDescriptor [2];
} ECB, FAR *PECB;
```


Each ECB contains all the information needed to send or receive data packets. Most of the fields shown above are initialized when the application calls an initialization API. The most interesting part of the ECB is the fragmentDescriptor field, which allows an application to associate an unlimited number of buffers with each ECB. Each buffer size is specified by an ECBFragment data structure, which contains the size and a pointer to the buffer. The number of fragments are specified by the fragmentCount field. Hence, instead of supplying one continuous block of bytes (say, 576 bytes long), an application can set up its data in five separate buffers. The only constraints are that the first 30 bytes of the first buffer must be left aside for the IPX header and the total packet length must not exceed 576 bytes (or the limit imposed by the protocol being used, such as SPX II). Traditionally, a user sets up the first fragment to contain the IPX header and the second fragment to contain the data buffer. When transmitting the packet, the IPX protocol driver creates one continuous packet from all the fragments. The application receiving the IPX packet can specify its own set of fragments, totally disregarding the fragment strategy used by the sending application. For example, the sender could have two buffers: one for the IPX header (30 bytes) and another for data (546 bytes). The receiver, on the other hand, could ask to receive data into three buffers (30, 300, and 246 bytes). In this case, the IPX driver will put the IPX header in the first buffer, 300 bytes in the second, and 246 bytes in the third.

The IPXWorkspace and driverWorkspace fields are reserved for use by the IPX protocol driver. The immediateAddress field contains the node part of the source address when the source and destination are on the same network.

Otherwise, it contains the node part of the address of the IPX router that allowed the packet to come to the destination network. The socketNumber field contains the socket associated with the ECB. Thus, an application will need at least one distinct ECB per socket to send or receive data over IPX sockets. The linkAddress field is internally used by IPX to maintain a linked list of ECBs while they are being processed. An application can also use the same field to maintain a list of free ECBs.

All IPX commands are executed asynchronously. Hence, an ECB contains three fields— ESRAddress, inUseFlag, and completionCode—that accord an application control over nonblocked execution of IPX commands. All IPX functions set the inUseFlag to a nonzero value before returning to the caller. This field is set to zero when the IPX driver has completed (or aborted) the command. The final status code of an ECB execution is placed in the completionCode field. Hence, an application can submit an ECB and then use the polling technique to determine completion of an IPX command by monitoring the status of the inUseFlag field of the ECB. Table 9-3 shows some possible values of inUseFlag.

An alternative to the polling technique is to specify the address of an **event service routine (ESR)** via the ESRAddress field. The ESR is comparable to the post routines in NetBIOS programming. Thus, the ESR functionality is not quite like the WSAAsyncSelect() function of Windows Sockets because the IPX driver does not send a Windows message on completion of a command. Indeed, an application writer can design an ESR that sends a message to the main application via the PostMessage() Windows API.

When the ESRAddress field is nonzero, IPX calls this routine resident in the application when it has completed processing an ECB. The ESR is given

Table 9-3. *Values of the ECBs inUseFlag Field*

inUseFlag Value	*Description*
00h	IPX is not using the ECB
F8h	IPX is busy and has queued the send packet and the ECB
FAh	IPX is processing the ECB
FBh	IPX has dedicated the ECB to an event that has occurred, and ECB has been queued for processing
FCh	IPX has dedicated the ECB to an event scheduled by the AES that has not occurred
FDh	IPX has dedicated the ECB to a scheduled event that has not occurred
FEh	IPX is listening on a socket specified in the ECB, waiting for incoming packets
FFh	IPX is using the ECB to send the packet

Table 9-4. Significance of completionCode Values

completionCode Values	Action		
	Sending Packet	*Receiving Packet*	*Scheduling an Event*
00h	Success	Success	Success
F9h	Undefined	Failure	Undefined
FCh	Canceled	Canceled	Canceled
FDh	Malformed packet	Overflow	Undefined
FEh	Undelivered packet	Undefined	Undefined
FFh	Failure	Closed socket	Undefined

the address of the ECB in the ES:SI address pair. In the Windows 3.x environment, the ESR is called at the interrupt time and hence must complete its processing quickly and return to the IPX driver. Note that IPX sets the inUseFlag field to zero and sets the completionCode field before calling the ESR if one has been associated with an ECB. Table 9-4 shows some of the possible values that can be returned in an ECB's completionCode field.

9.3.3 Asynchronous Processing of ECBs

All IPX commands are scheduled to be executed asynchronously. Relevant IPX APIs take a pointer to an ECB and return instantly with an immediate return code. If the API successfully submitted an ECB to the IPX driver, it will return success (0h). At this point, the inUseFlag field of the ECB is set to a nonzero value. The inUseFlag field is set to zero once the ECB has been processed by the IPX driver and the final return code is placed in the completionCode field of the ECB.

A Windows NT application can start a thread that can periodically check the value of inUseFlag. This will allow the main thread to continue to process the application message queue and appear responsive to user inputs.

A Windows 3.x application has two choices on how to determine the completion of a submitted ECB. One is based on periodically polling the inUseFlag field of the submitted ECB. This method can, however, prevent other applications from processing user input messages because Windows 3.x has a single message queue for the entire system. Furthermore, frequent polling of the inUseFlag field can prevent the IPX driver from getting enough CPU cycles. While the former can be avoided by relinquishing control of the message queue by the calling the **PeekMessage()/TranslateMessage() /DispatchMessage()** APIs (code fragment shown below), the latter can be avoided by simply calling the **IPXRelinquishControl()** API. When the program does not need to process user inputs (via mouse or keyboard), the program may simply the **yield()** Windows API instead use the PeekMessage() method.

```
//
// Outline of a routine which polls the unUseFlag to determine
// if an ECB has been processed by IPX driver
//
BYTE ProcessECB (ECB *pecb)
{
     MSG msg;
     // We wait for ECB to complete
     do
     {
#ifndef _WIN32_ // That is, Windows 3.x
#ifdef INTERACTIVE_WAIT
          // Relinquish control to IPX driver
          // This is primarily needed for IPX Bridge programs.
          // Most Windows applications need not call it.
          IPXRelinquishControl();

          // Process message queue so that application
          // does not hang the rest of the system
          while (PeekMessage (&msg, NULL, 0, 0, PM_REMOVE)
          {
               TranslateMessage ( &msg );
               DispatchMessage  ( &msg );
          }
#else // Noninteractive
          // We don't want to process any messages, so yield to other
          // applications
          Yield();
#endif
#else // Windows NT
          Sleep ( SLEEP_TIME );
#endif
     } while (pecb->inUseFlag);

     // Return the final completion code EC
     return (pecb->completionCode);
}
```

The other method to determine completion of ECBs is to associate an ESR with an ECB so that the IPX driver calls the ESR when the ECB has been processed. This mechanism sets the application free from polling and makes the program event driven. An ESR is called at interrupt time; hence, the amount of processing should be kept to a minimum. Frequently programmers simply post a Window message to their application from an ESR sending the ECB address via lParam. This allows the ESR to complete right away, and the application can process completion of the ECB in the same manner that it reacts to other Windows messages. The advantage of using an ESR is that the application does not need to wait, polling for completion of a command. Instead, it can conduct network I/O in an asynchronous manner, which allows it to perform other tasks, such as responding to user requests. Once an application has received a message indicating completion of an ECB, it can process the results as appropriate.

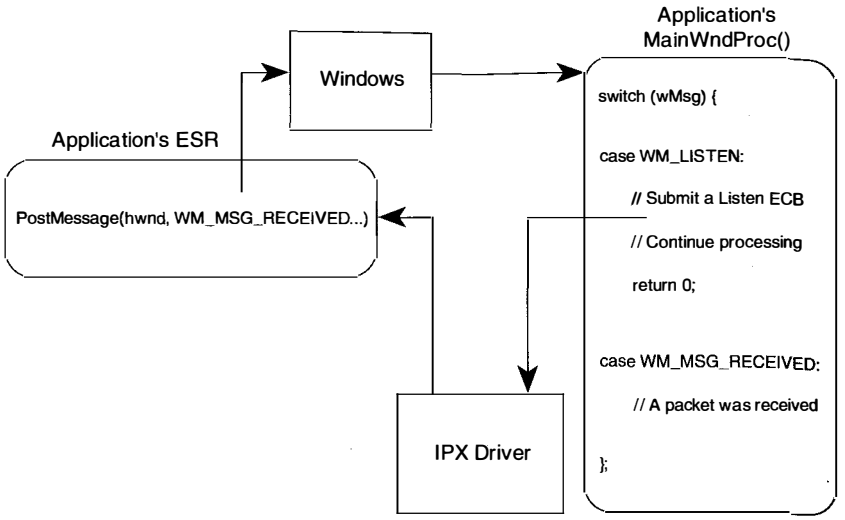


Figure 9-4. *Flow of Control When Using an ESR*

Figure 9-4 shows the flow of control for a server application that uses an ESR to listen to clients and processes requests as they arrive. It should be clear there is a potentially unbounded time lag between submitting an ECB to listen to clients and finally receiving a packet via the ESR. Therefore it is essential that the ESR code segment be constantly present. This is not an issue for Windows NT applications. However, in Windows 3.x environments, code and data segments can move, be discarded, or be swapped. In enhanced mode Windows, the code segment (and the data segment if ESR uses data segment) for ESR must be pagelocked in memory and be resident. An alternative is to put the ESR code, and any data, in a DLL whose code/data segment is marked FIXED to eliminate moving of the ESR.

In the code fragment that follows, we choose a strategy similar to the one used to handle NetBIOS post routines. In this scheme, we create a special structure (MYECB) that contains an ECB and a handle to a window that is to receive the completion message. In the Windows 3.x environment, the ESR gets the address of the ECB via ES:SI. In the Windows NT environment, it simply receives a pointer to the ECB. In both cases, the ESR determines the handle of the window and posts a message to it with lParam containing the address of the MYECB structure that contains the ECB. These concepts are shown here.

```
typedef struct tagmyECB{
    ECB       ECB;            // ECB
    HANDLE    hWnd;           // Handle of window to receive message
    UINT      wMsg;           // Message sent to window on completion
} MYECB, *PMYECB;
```

```
#ifndef _WIN32_ // that is Windows 3.x
//
// IPX_ESRoutine : - The function is called when an ECB completes in
//                   Windows 3.x environment
//
// Entry:     ES:SI points to the ECB submitted via an SPX/IPX function
//
void  _loads cdecl interrupt far IPX_ESRoutine
(
    unsigned es, unsigned ds,
    unsigned di, unsigned si,
    unsigned bp, unsigned sp,
    unsigned bx, unsigned dx,
    unsigned cx, unsigned ax,
    unsigned ip, unsigned cs,
    unsigned flags
)
{
    PMYECB pmyecb;

    // ES:SI contains pointer to just-completed ECB
    pmyecb = (PMYECB) MAKELONG (es, si);
    // Post a message to the window with wParam = 0 and lParam = ptr to MYECB
    // containing the completed ECB.
    PostMessage (pmyecb.hWnd,
                 pmyecb.wMsg,
                 0,
                 (DWORD) pmyecb
                );
}
#else // Windows NT
//
// IPX_ESRoutine : - The function is called when an ECB completes in Win32
//                   environment
//
// Entry:            PECB points to the completed ECB
//
VOID IPX_ESRoutine ( PECB pecb )
{
    // Map from ECB address to MYECB address
    PMYECB pmyecb = (PMYECB) pecb;
    // Post a message to the window with wParam = 0 and lParam = ptr to MYECB
    // containing the completed ECB.
    PostMessage (pmyecb.hWnd,
                 pmyecb.wMsg,
                 0,
                 (DWORD) pmyecb
                );
}
#endif // _WIN32_

//
// SubmitECB - A sketch of a routine which uses the ESR above.
//
BYTE SubmitECB (PMYECB *pmyecb, HANDLE hInstance)
{
```

```
    FARPROC lpProc;
    BYTE    bRc;
    // Set various fields in ECB (to be discussed in later sections)
    ...
    // Get the address of ESR routine
    lpProc = MakeProcInstance ((FARPROC) IPX_ESRoutine), hInstance);
#ifndef _WIN32_
    // Lock the ESR code segment in memory so that it is always
    // available to IPX driver to call back on. IPX Driver calls
    // ESR at interrupt time and hence nonavailability of ESR
    // will cause the entire system to crash.
    GlobalPageLock ((HGLOBAL) SELECTOROF(IPX_ESRoutine);
#endif
    // Associate the ECB with the ESR
    pmyecb->ESRAddress = (FARPROC) IPX_ESRoutine;
    // Submit the ECB
    bRc = ...
    return ( bRc );
}
```

9.3.4 Initializing and Deinitializing an IPX Application

The IPXInitialize() and IPXSPXDeinit() functions must be called by all IPX-based applications. Later, in the section on SPX programming, you will learn that the IPXSPXDeinit() API is also used to deinitialize SPX-based programs.

All IPX applications must call the IPXInitialize() function before calling any other IPX function. The application indicates the maximum number of ECBs (via the maxECB parameter) that it anticipates submitting to IPX during the lifetime of the application. Suppose an application wishes to have five ECBs submitted to receive incoming packets sent from many clients. In this case, it can be recommended that the application register (and manage) ten ECBs with the IPX, because processing the ECBs is much slower than receiving them. The application can set a maximum size of its packets (via the maxPacketSize parameter), which is constrained by the 576-byte limit for IPX-based transmission. In the Windows environment, IPXInitialize() returns a task ID (via IPXTaskID) that the application needs to save since it must supply this task ID as a parameter to subsequent IPX calls. The task ID essentially allows IPX to control and manage resources associated with given applications. These resources are freed when the application calls the IPXSPXDeinit() API supplying it the task ID assigned by IPXInitialize(). An application needs to specify the way IPX allocates the resources by setting specific values (in IPXTaskID) before calling the IPXInitialize() function. A value of (FFFFFFFFh) signifies that the resources are allocated from a common pool shared by multiple applications. The application can specify a value (FFFFFFFEh) to signal that all resources are directly associated with it; however, multiple initialization (of the same application) causes the application to share this resource pool. Finally, a value (0h) indicates that all resources are directly associated with the application. Most applications will find it

appropriate to share the resource pool with other applications. Finally, an application can call the IPXInitialize() function multiple times. However, it must call the IPXSPXDeinit() function corresponding to each IPXInitialize() call. This feature is useful when a DLL manages IPX network requests for multiple applications. In such a case, the DLL can use multiple initializations to keep track of IPX transactions for multiple applications.

```
int FAR PASCAL IPXInitialize( DWORD FAR *IPXTaskID,
                              WORD maxECBs,
                              WORD maxPacketsize
                            );
int FAR PASCAL IPXSPXDeinit(DWORD IPXTaskID);
int FAR PASCAL IPXCancelEvent (DWORD IPXTaskID,
                               ECB FAR *eventControlBlock);
```

Readers familiar with Windows Sockets will find that the IPXInitialize() and IPXSPXDeinit() APIs are similar to the WSAStartup() and WSACleanup() Windows Sockets APIs.

An application should cancel any pending ECBs prior to calling the IPXSPXDeinit() API. All pending send/listen ECBs can be canceled using the **IPXCancelEvent()** API, which takes the task ID (via the IPXTaskID parameter) and the ECB to cancel (via the eventControlBlock parameter). An incoming IPX packet will cause the IPX driver to call the application's ESR unless all pending ECBs have been completed or canceled. When an application quits without properly canceling ECBs, an incoming IPX packet will cause the IPX driver to call a (now) nonexistent ESR, leading to certain collapse of the entire Windows 3.x system. In the Windows NT environment, the Win16 subsystem may shut down, thereby possibly affecting every Win16 program running in it. The IPXCancelEvent() API safeguards against these events and simply removes a pending ECB from the IPX processing queue. It sets the inUseFlag field of the canceled ECB to zero. The IPXCancelEvent() API does not call an ESR at the address set in the ECB to be canceled.

9.3.5 Opening and Closing IPX Sockets

The IPXOpenSocket() and IPXCloseSocket() functions are used to open and close IPX sockets, respectively. (The same APIs are used to open and close SPX sockets as well.)

A server wishing to receive IPX packets first needs to open an IPX socket using the IPXOpenSocket() API. And a client wishing to send an IPX packet can open a socket so that it can pass the (client-side) socket number for subsequent IPX operations. The socket number at the client end need not be the same as that on the server end. For example, a server may listen on socket 3845h, whereas the client may open local socket 2345h. However, a client must clearly identify the server's socket number when sending the server a packet. (Readers familiar with Windows Sockets will find the IPXOpen-Socket() function to be comparable to the socket() Windows Socket API.)

The caller of the IPXOpenSocket() function passes in the task ID assigned to it via the IPXInitialize() API as the first parameter. It also needs to specify a socket number (via the socketNumber parameter). Well-known servers, such as a database server, name server, or file server, use well-defined or static socket numbers. On the hand, applications can use a dynamic socket, in which case the IPX driver assigns a socket number, if one is available. Traditionally, dynamic sockets are appropriate for the clients in a CSS, whereas static sockets are useful for servers. The reason is that in a typical CSS, the clients always initiate a dialog and thus need to know the server address (node number, network number, and socket number). Hence, a static server address is useful because it makes it easy for clients to locate a server. On the other hand, the server always responds to a client request, which contains the client IPX socket number. Thus, the client end socket number can be dynamic. Dynamic sockets are also useful for applications that can have multiple instances on the same workstation.[2]

To receive a dynamic socket number, the caller sets zero in the socketNumber field before calling the IPXOpenSocket() API. In this case, the IPX driver returns a socket number via the socketNumber field; the dynamic socket numbers lie in the range 4000h–5000h. A caller marks the socket life span (via the socketType parameter) as short lived (socketType = 0) or long lived (socketType = FFh).

```
int FAR PASCAL IPXOpenSocket ( DWORD       IPXTaskID,
                               WORD FAR    *socketNumber,
                               BYTE        socketType);
int FAR PASCAL IPXCloseSocket( DWORD IPXTaskID,
                               WORD        socketNumber);
```

A short-lived socket is terminated when an application terminates and is suitable for most cases. A long-lived socket is typically useful for MS-DOS–based TSR applications so that the main application can terminate and become active when a packet arrives. In the latter case, the designer must ensure that ESR remains fixed in memory even after the application terminates. On successful completion of the IPXOpenSocket() function, a socket number is returned in the socketNumber parameter. This socket number can now be assigned to the socketNumber field of an ECB and used for IPX-based communication.

You close an opened socket by calling the IPXCloseSocket() API, causing all pending ECBs associated with a socket to be canceled. The completionCode field of ECBs is set to (FCh), and the inUseFlag is set to zero. However, the ESR associated with the socket is not called. Hence, after a socket has been closed, the ESR associated with it can be safely discarded or destroyed.

2. The IPX driver manages the resource pool by sockets. Hence, it cannot use a single static socket number to manage resources for multiple initializations of an application.

9.3.6 Receiving IPX Packets

The IPXListenForPacket() API is used to receive IPX packets (and SPX packets as well).

Before an application can receive an IPX packet, it must have opened a valid socket by calling the IPXOpenSocket() function. To receive the incoming datagram IPX packets, the receiver (either the client or the server) calls the IPXListenForPacket() function. When calling this API, the caller supplies the task ID (via the IPXTaskID field) and an ECB (via the eventControlBlock parameter).

```
void FAR PASCAL IPXListenForPacket(DWORD IPXTaskID,
                                   ECB FAR *eventControlBlock
                                  );
```

The ECB must have been properly set up before passing it to the IPXListenForPacket() function. The caller needs to set the following fields of the ECB:

- The socketNumber field with the socket number received from the IPXOpenSocket() API.
- The fragmentCount field with the number of fragments associated with the ECB.
- The fragmentDescriptor field with the address and size of fragments.
- The ESRAddress field with the address of the ESR routine.

Most programs find it convenient to associate two fragments with each receive ECB: one for the IPXHeader (30 bytes) and another for data (up to 546 bytes). An application can choose not to use an ESR with the ECB, in which case it must set the ESRAddress field to NULL. When the application does wish to associate an ESR with the ECB, the ESR address must be obtained by calling the MakeProcInstance() Windows API so that the routine address is Windows callable.

In Windows 3.x environments, the ECB and all buffers (fragments) associated with the ECB must be locked in place so that they do not move, get swapped out, or get discarded at any time during which an IPX packet might arrive. Furthermore, the ESR segment must be fixed or locked in memory as well.

Once an ECB has been passed to the IPX driver via the IPXListenForPacket() function, it is put in a queue of ECBs for the given socket, and the inUseFlag of the ECB is set to (FEh). The ECB remains in the queue until either the IPX driver associates an incoming IPX packet with it or the application calls the IPXCloseSocket() API on the socket. When a new IPX packet arrives at the node for the socket, the IPX driver takes an ECB off the queue and associates the packet with it. The driver does not guarantee to process ECBs in the order they were submitted. The driver sets the fields of the ECB as follows:

- The IPX packet is copied to the fragments associated with the ECB.

- The immediateAddress field of the ECB is set with the address of the sender of the IPX.
- The completionCode field of the ECB is set to an appropriate value, as shown in Table 9-4.
- The inUseFlag field of the ECB is set to zero prior to calling the ESR.

At this time, the IPX driver calls the ESR associated with the ECB, if one has been provided by the application. The ESR is called, with the ES:SI register pair pointing to the ECB.

An application submitting a listen ECB cannot specify the type of packet that it wishes to receive. Thus, a listen ECB will complete whether the incoming packet is specifically directed to the receiver or it is a broadcast message sent to all nodes. This behavior is in marked contrast with NetBIOS, whereby the receiver can specify to receive unicast or broadcast messages. Thus, applications using IPX to send and receive messages will have to ensure that the data they received are from their counterpart and ignore broadcast messages propagated on the network by other applications and utilities. This can be accomplished by examining the IPXHeader structure present in the first 30 bytes of the first fragment and discarding any packet with a destination address marked for broadcast (the node is set to FFh). Similarly, the packetType field of the IPXHeader structure allows applications to discriminate between protocols (for example, IPX versus SPX).

9.3.7 Locating a Specific Server Receiving IPX Packets

In this section, we will see how an IPX-based client can discover the server address (so that it can send packets to the server). The same methods are applicable for SPX-based clients.

Recall that a client needs to specify the complete network address (via the IPXAddress structure) of the server to send packets to it. While building a prototype of a CSS, one can simply ask for the server address or "hardcode" it. However, in an actual system, a client must be able to determine the server address at run time. Before discussing one of the many possible ways to determine the address, let us revisit the network address.

A network address is composed of three parts:

1. *Network number* (4 bytes). *Simple or demo* applications can assume that both the sender and receiver exist on the same subnet; *real-world* applications cannot make this assumption. In *simple* applications, a sender can call the IPXGetInternetworkAddress() API. The prototype is shown here:

```
void     IPXGetInternetworkAddress(BYTE far *networkAddress);
```

This API determines its own network number and assumes that the receiver is also on the same network. In *demo* cases, the application can ask the user at the sender end to supply the network number when it is beginning transmission. However, most users will find such an application daunting because they do not understand networking, much less the network number.

2. *Node number* (6 bytes). Determining the network node number of a remote machine is truly difficult. If the user is asked to supply the address, determining the network node numbers will lead to nonintuitive user interfaces.
3. *Socket number* (2 bytes). A socket number can be a well-known number (static) or created at run time (dynamic). Typically, well-known services such as name services have well-known socket numbers, falling in the range of 0h through 3FFFh. A dynamic socket number falls in the range of 4000h through 7FFFh. Thus, one can specify a fixed socket number when designing a client for a well-known service. When designing a CSS that need not interact with other applications, both ends can pick a specific socket number, say, 4545h. The problem with this approach is that another application may already be using this socket number on a given workstation. Hence, an application may resort to using a dynamic socket number generated by the IPX driver at run time. Now the problem is how an IPX packet server (that is, the receiver) tells the IPX packet clients (senders) the dynamic socket number it has been granted. One way is to use a fixed socket number by registering it with Novell/Xerox.

It must be clear by now that determining the addressing information is difficult. One solution is for server applications to broadcast their addressing information on the network and clients to broadcast on the network asking for a specific server or a specific server type. Novell has provided a basic infrastructure for dynamically advertising and locating servers on the network. Called the **service advertising protocol (SAP)**, it is supported in the Windows and the Windows NT environments by appropriate drivers. This concept is comparable to the Name Service of Microsoft RPC (see Chapter 4) and Name Service providers of Windows Sockets (see Chapter 5). In fact, the Windows- Sockets–based name service provider for the Novell NetWare environment uses the underlying SAP services.

SAP allows servers to announce their addressing information on the network and for clients to query for a list of servers matching a specific type. The three functions shown below allow servers and clients to access SAP infrastructure.

```
// APIs used by servers
int AdvertiseService ( WORD serverType,
                       char *serverName,
                       BYTE *serverSocket
                     );
int ShutdownSAP ( char             *serverName);
// APIs used by clients
int QueryServices ( WORD queryType,
                    WORD serverType,
                    WORD returnSize,
                    SAP *serviceBuffer);
// Some of the pre-defined service types
#define OT_WILD                  0xFFFF
#define OT_FILE_SERVER           0x0400
#define OT_PRINT_SERVER          0X0700
// Query type values
// 1 = general service query
// 3 = nearest server query (most commonly used query type)
// SAP data structure
typedef struct {
                IPXHeader    Header;
                WORD         ResponseType;
                BYTE         ServerName [48];
                BYTE         Network [4];
                BYTE         Node [6];
                BYTE         Socket [2];
                WORD         InterveningNetworks;
} SAP;
```


A server typically calls the **AdvertiseService()** API when it starts and the **ShutdownSAP()** prior to terminating. The server identifies itself by the {serverType, serverName} pair. A server name can be up to 48 bytes long and can be composed of any characters in the range 21h–7Fh, with some exceptions. (See [Novell 95].) A client can query for the nearest server of a certain type via the QueryServices() API. As a result of a query, it can receive zero or more SAP structures, each containing the full network address of the server. Figure 9-5 shows a typical network with multiple subnets. A client (ClientA) in subnet A will find PrintServerB because there are no print servers on subnet A. On the other hand, the nearest FAX server for this client will be FaxServerA, which is present on subnet A. Each SAP router exchanges knowledge of the servers present in a particular subnet with other routers so each server advertisement is visible to all clients in all subnets. Thus, even the most remote server can be located by a client via the **QueryServices()** API. For implementation details of SAP infrastructure support in the Novell NetWare environment, consult [Novell 95].

On successful query of service, the client can use the network address returned in the SAP structure to determine the immediateAddress of the server using the IPXGetLocalTarget() API. The prototype of the function follows. On supplying the caller's IPX task ID (via the IPXTaskID parameter)

and the network address (via the networkAddress parameter), the function returns the immediate address of the target machine (via the immediateAddress parameter). This address contains the node number of the target machine, if the machine is on the same subnet as the client. If the machine is on a different subnet, then the immediate address of the target contains the node number of the router via which the target node can be reached. For example, when the client shown in Figure 9-5 calls the IPXGetLocalTarget() based on the SAP returned by the QueryServices() API, it will get the address of Router AB via the immediateAddress parameter.

```
int FAR PASCAL IPXGetLocalTarget ( DWORD        IPXTaskID,
                                   BYTE FAR     *networkAddress,
                                   BYTE FAR     *immediateAddress,
                                   int FAR      *transportTime
                                 );
```

The IPXGetLocalTarget() API also returns an estimate of communication time (via the transportTime parameter). The unit of measurement of transport time is system clock ticks. It estimates the time a 576-byte packet takes to arrive at the target node. Clients can then use this value to select a server with the least transportation cost, should a number of SAP buffers be presented by the QueryServices() API. This scheme will allow the client to connect to the nearest server.

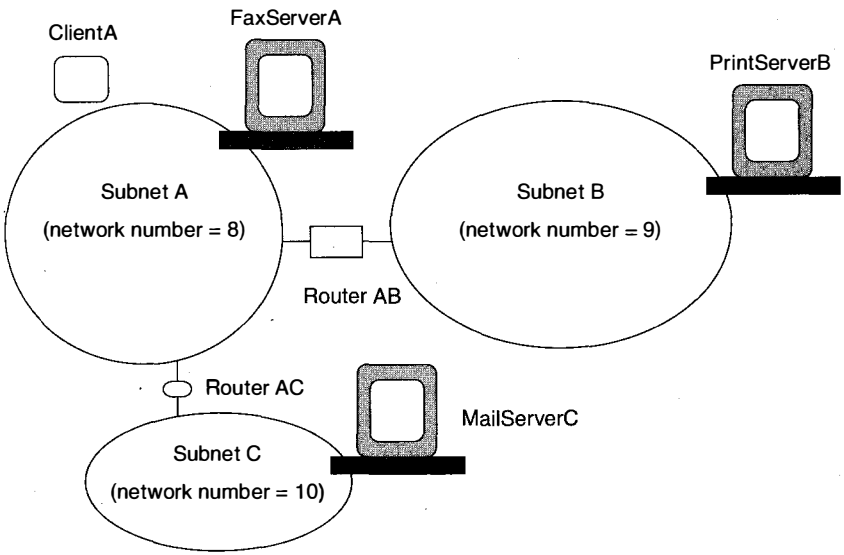


Figure 9-5. *A Typical Network with Multiple Subnets*

9.3.8 Sending Datagram Packets

The IPXSendPacket() API opens the unreliable datagram transmission infrastructure NetWare uses for all applications.[3] An application can send an IPX packet directly to a server or broadcast the data to all nodes on the network. Here is the prototype of the IPXSendPacket() API.

```
int FAR PASCAL IPXSendPacket ( DWORD    IPXTaskID,
                               ECB FAR  *eventControlBlock);
int FAR PASCAL IPXDisconnectFromTarget ( DWORD     IPXTaskID,
                                         BYTE FAR  *networkAddress
                                       );
```

The IPXSendPacket() function takes the task ID (via the IPXTaskID parameter) and the prepared ECB (via the eventControlBlock parameter). The caller must set proper fields in the ECB before calling this function. We have already seen the means by which a client can dynamically determine the network address of a server or a service. Alternatively, it can use a static or user-supplied server address. In all cases, the sender must put the receiver's address in the ECB. The fields in the ECB that must be filled before the IPXSendPacket() API are:

- The immediateAddress field with the target address.
- The socketNumber field with the local (client side) socket to be associated with the ECB.
- The fragmentCount and fragmentDescriptor fields.
- The ESRAddress field containing the address of the ESR to be associated with the ECB, if any.

For sending a directed (unicast) datagram, the sender must know the full network address of the target. It can then call the IPXGetLocalTarget() function to determine the address of the target, which is in the immediateAddress field of the ECB. In addition, the caller must specify the destination address (IPXAddress) via the first fragment descriptor containing the IPXHeader. To send a broadcast packet that will be received by every listening node in the entire network, the caller sets the node field of destination address to FFh. When the sender wants to broadcast the packet to every node in its network, it sets the immediateAddress field of the ECB to FFh as well.

Just like an application receiving datagrams, the sender must call the IPXOpenSocket() API to get a local socket number that can be associated with each send ECB. The IPX driver uses the socket numbers to index through the queued send or listen ECB and hence needs the sender to open a socket. The socket opened by the sender is not used for communication

3. Novell literature asserts that the IPX transmission success rate is usually 95% [Novell NetWare Programmer's Guide for C, 1993].

purposes (unless the sender or receiver wishes to have duplex communication). The socket number of the receiving end is specified in the socket number field of the destination address and is used to locate the receiver on the target node.

The sender sets the number of fragments (that is, buffers) associated with the ECBs via the fragmentCount field and the fragments via the fragmentDescriptor array. The sender must supply at least one fragment large enough to hold the IPXHeader (30 bytes), which among other things contains the destination address. When one or more fragment descriptors are supplied, the IPX driver concatenates the buffers pointed to by all fragments to form a single IPX packet with the IPXHeader followed by data buffers. Thus, the total data buffer cannot exceed 546 bytes. Most applications find it convenient to have one fragment pointing to the IPXHeader and the other pointing to the data buffer.

The sender can associate an ESR with the send ECB, which gets called when the send completes. The address must be obtained by calling the MakeProcInstance() Windows API. When no such ESR exists, the caller must set the ESRAddress field of ECB to NULL.

After an application has submitted a send ECB, the IPX driver puts the ECB on its queue of ECBs to be processed and returns immediately with the inUseFlag field of ECB set to (FFh). The application can then proceed to other operations. In the meantime, the IPX driver attempts to send the IPX packet. It keeps trying until the timer associated with the send ECB times out. If the send is successful, the driver sets the completionCode and inUseFlag fields (of ECB) to zero before calling the ESR associated with the ECB, if one is provided. Due to the nature of datagram service, a completionCode code of zero does not guarantee that the receiver has actually received the datagram. It merely indicates that the local IPX driver was successful in sending the IPX packet toward the remote target. In failure cases, the driver sets a final error code in the completionCode field of the ECB and calls the ESR after setting the inUseFlag field to zero. A typical error code of FEh indicates that the packet was undeliverable, either because of the unavailability of a bridge connecting to the remote network or because the target does not exist.

When the calling application does not provide an ESR with the send ECB, it must periodically check the inUseFlag field (of ECB) to determine the completion of the send. When the inUseFlag field is set to zero, the caller should check the completionCode field to determine the outcome of the send.

When a sending application completes sending packets, it should call the **IPXDisconnectFromTarget()** API. This is a courtesy call to the network drivers, which are made aware of the sender's desire to unwind the operations. This allows the drivers to dismantle any connections made to the network adapters and make intelligent decisions regarding reallocating resources.

9.3.9 IPX Sample Program

Now we will examine a simple demonstration program to understand the use of IPX functions in their simplest form. The application itself can be used to send or receive datagrams via IPX services. Using the simple menu commands, the user needs to select each IPX function in the correct sequence (for example, a socket must be opened prior to receiving IPX packets). When the user selects to receive datagram packets, the local network address is presented on the screen. The user must then feed this address to the sender of the application via a dialog box so that the sender can correctly target the receiver. For simplicity, we use a static (0x4040) socket number, which both sender and receiver are aware of. When the sender is started with no parameters, it is assumed to be sending broadcast data.

The sender sends a fabricated line of text to the receiver, which displays the line and the immediate address of the sender. Note that the WM_CRATE and WM_PAINT commands have been intercepted so the receiver can display the incoming messages via simple message buffering and the **TextOut()** Windows API.

The **SetupAppIPXResources()** function allocates a number of ECBs and two fragments (data buffers) per ECB. The first fragment will contain the IPXHeader, and the second fragment will contain data. Each ECB is associated with an ESR (see **IPX_ESRoutine()** in common.c); the ESR itself is locked in memory by the **GlobalPageLock()** Windows API. We also put the handle of the main window in each MYECB structure so the ESR can send the message to the window. The **ReleaseAppReleaseResources()** function releases the buffers and unlocks the ESR routine.

The StartIPXListen() function submits the listen ECBs with the address of an ESR associated with each ECB. At this time, each MYECB structure contains the handle of the main window. Then the StartIPXListen() function associates a message (WM_MSG_RECEIVED) with each listen ECB. Thus, when the IPX packet arrives, the ESR sends the message to the main window (that is, the message is processed by the WndProc() function). The WM_MSG_RECEIVED message causes the **ProcessIPXPacket()** function to be invoked, which simply copies the contents into a global buffer and sends a WM_PAINT message to the main window, which displays the newly arrived message.

The SendIPXPacket() function is used to send a fabricated string to the receiver. Notice that we cycle through the send ECBs until we find an ECB that has not been submitted. This determination is made by checking the inUseFlag field of the ECBs. We also assume that send ECBs are indexed [MAX_RECV_ECBS , MAX_ECBS], whereas listen ECBs are indexed through [0, MAX_RECV_ECBS]. The user-supplied destination addresses are put in the destination field of IPXHeader, which is permanently associated

with the fragmentDescriptor[0] field of all ECBs. Similar to listen ECBs, each send ECB is associated with an ESR, a main window handle, and a message (MW_MSG_SENT). The ESR sends the message (WM_MSG_SENT) when the send completes or aborts.

Finally, note that we save the receiver's address in the SendIPXPacket() function. This address is later used to send a courtesy disconnect message (via the IPXDisconnectFromTarget() API) to the IPX driver prior to closing the local socket.

```
// For complete project, see %BOOK\ipxspx\ipxdemo.mak
//
// FILE: common.h
//
#if !defined(NWWIN) // NetWare Client SDK
#define NWWIN
#endif

#include <nwipxspx.h>

//
// Constants and defines
//
#define IDM_EXIT                106
#define IDM_IPXINITIALIZE       200
#define IDM_IPXDEINIT           201
#define IDM_OPEN                202
#define IDM_CLOSE               203
#define IDM_LISTEN              204
#define IDM_SEND                205

#define WM_MSG_RECEIVED         250
#define WM_MSG_SENT             251

#define MAX_ECBS                 10
#define MAX_RCV_ECBS              5
#define MAX_SND_ECBS              5
#define MAX_PACKET_SIZE         512
#define MAX_DATA_SIZE           472
#define IPX_DEMO_SOCKET       0x4040

#define IPX_DEMO_STRING     "This message was sent via IPXSendPacket"

//
// Data structures
//
// IPX Address
#define NODE_SIZE        (6)
#define NETWORK_SIZE     (4)
#define SOCKET_SIZE      (2)

typedef IPXAddress FAR * PIPXAddress;

// IPX Header
```

```
typedef IPXHeader FAR * PIPXHeader;

// SPX Header
typedef SPXHeader FAR *PSPXHeader;

// ECB Fragment
typedef ECBFragment FAR *PECBFragment>
// ECB
typedef ECB FAR *PECB;

// My ECB structure. The structure assumes ECB size is fixed.
// For example, in the define above, each ECB has two fragments.
typedef struct tagmyECB{
    ECB       ECB;      // ECB
    HANDLE    hWnd;     // Handle of window to receive message
    UINT      wMsg;     // Message sent to window on completion
} MYECB, FAR *PMYECB;

// In order to GlobalUnlock() and GlobalFree() a piece of memory,
// we need the handle of the object. We therefore store the handle
// returned from GlobalAlloc() in each piece of memory allocated
typedef struct {
    HGLOBAL    hMem;
    BYTE       bMem [0];
} MY_MEMORY, FAR *PMY_MEMORY;
//
// Common functions
//
LRESULT CALLBACK    WndProc(HWND, UINT, WPARAM, LPARAM);
VOID CALLBACK       IPX_ESRoutine (PECB pecb);
LPVOID    AllocAndLock (DWORD dwBytes);
VOID      UnLockAndFree (LPVOID lpMem);
BOOL      AcquireReceiverAddress (HWND hWnd, PIPXAddress pReceiver);
//
// Macros
//
#define DISPLAY_ERROR_MSG(str, rc) {\
    wsprintf(szErrorMsg, "Error: %d in %s\n", rc, str);\
    MessageBox ( NULL, szErrorMsg, szAppName, MB_OK);\
}
#define RETURN_IF_MEM_FAILED(ptr) {if (ptr == NULL) return FALSE;}

#if !defined (ESUCCESS)
#define ESUCCESS (0)
#endif

#if !defined (EFAILURE)
#define EFAILURE (-1)
#endif
// FILE: common.c
// Contains functions used by this sample as well as by the SPX sample
// shown in the SPX section
#include "common.h"
#include "resource.h"
#include <stdio.h>
```

```
////////////////////////////////////////////////////////////////////////
// PROCEDURE : InitDlgProc (HWND hDlg, WORD wMsg, LONG wParam, LONG lParam)
//
// PURPOSE:  This dialog box prompts the user for the node number
//           and network address of the recipient. These values are
//           placed into global strings szNode and szNetwork.
////////////////////////////////////////////////////////////////////////
static char szNode        [ NODE_SIZE   ];
static char szNetwork     [ NETWORK_SIZE];
LONG CALLBACK
GetAddressDlgProc (HWND hDlg, UINT wMsg, WPARAM wParam, LPARAM lParam)
{
    UNREFERENCED_PARAMETER(lParam);
    switch (wMsg)
    {
    case WM_INITDIALOG:
            memset(szNode, 0x00, sizeof(szNode));
            memset(szNetwork, 0x00, sizeof(szNetwork));

            PostMessage (GetDlgItem (hDlg, IDC_EDIT_NODE),
                         EM_LIMITTEXT, NODE_SIZE, 0);

            PostMessage (GetDlgItem (hDlg, IDC_EDIT_NET),
                         EM_LIMITTEXT, NETWORK_SIZE, 0);
    case WM_COMMAND:
            switch (LOWORD(wParam))
               {              // When the user clicks okay, get the
               case IDOK:     // node address and network address
                              // edit fields.
                   GetWindowText (GetDlgItem (hDlg, IDC_EDIT_NODE), szNode,
                              NODE_SIZE);
                   GetWindowText (GetDlgItem (hDlg, IDC_EDIT_NET),
                              szNetwork, NETWORK_SIZE);
                   EndDialog(hDlg, IDOK);
                   return (0);
               case IDCANCEL:
                   EndDialog(hDlg, IDABORT);
               default:
                   return (0);
               }
    default:
            return (0);
    }
   return (0);
}
////////////////////////////////////////////////////////////////////////
//
// FUNCTION: AcquireReceiverAddress(LPSTR)
//
// PURPOSE:  Put up a dialog box and get (node, network) of receiver
//
// COMMENTS: The data is supposed to be in hex
//
//
////////////////////////////////////////////////////////////////////////
```

```
#define CHARTOHEX(ptr, ch) {\
    if   (isdigit(ch)) { *ptr = ch - '0';}\
    else { *ptr = ch - 'A';}\
}
BOOL AcquireReceiverAddress (HWND hWnd, PIPXAddress pReceiver)
{
    BOOL     fRet  = FALSE;
    int      i;
    LPSTR    pszPtr;
    char     szMsg [ 100 ];
    int      iRc = DialogBox ( (HANDLE)GetModuleHandle(NULL),
                               LPSTR)"IDD_GET_ADDRESS",
                               hWnd,
                               (DLGPROC)GetAddressDlgProc
                             );
    if (iRc == IDOK)
    {
        // Get the node address
        pszPtr = szNode;
        for (i = 0; ( (i<NODE_SIZE) ) && (pszPtr) && (*pszPtr != 0x20);
             i++, pszPtr++)
        {
             CHARTOHEX((LPSTR)&(pReceiver->node[i]), *pszPtr)
        }

        // get the network
        pszPtr = szNetwork;
        for (i = 0; ( (i< NETWORK_SIZE) ) && (pszPtr) && (*pszPtr != 0x20);
             i++, pszPtr++)
        {
             CHARTOHEX((LPSTR)&(pReceiver->network[i]), *pszPtr)
        }
        fRet = TRUE;
        // Show the address
        sprintf ( szMsg,
                  "Receiver's Network Number: "\
                  "%02X%02X%02X%02X"\
                  " Node Number: "\
                  "%02X%02X%02X%02X%02X%02X\n",
                  pReceiver->network[0], pReceiver->network[1],
                  pReceiver->network[2],pReceiver->network[3],
                  pReceiver->node[0], pReceiver->node[1],
                  pReceiver->node[2],
                  pReceiver->node[3], pReceiver->node[4],
                  pReceiver->node[5]
                );
        // If not correct, we will put up the dialog again
        if (MessageBox (NULL, szMsg, "Receiver Address", MB_YESNO)==IDYES)
             fRet = TRUE;
        else
             fRet = FALSE;
    }
    return (fRet);
}
```

```
////////////////////////////////////////////////////////////////////////
/
// FUNCTION: AllocAndLock(DWORD)
//
// PURPOSE:  Allocate fixed and zero-initialized memory.
//
// COMMENTS: We allocate 4 extra bytes per memory allocation and save
// the handle at the beginning of allocated memory
////////////////////////////////////////////////////////////////////////
LPVOID AllocAndLock (DWORD dwBytes)
{
    PMY_MEMORY pmyMem = NULL;
    LPVOID     lpMem  = NULL;
    HGLOBAL    hMem   = 0;

    if (hMem = GlobalAlloc (GPTR, dwBytes + sizeof(MY_MEMORY)))
    {
        if ((pmyMem = (PMY_MEMORY)GlobalLock ( hMem )) == NULL)
            GlobalFree ( hMem );
        else
        {     // Save the handle in upper 4 bytes
              pmyMem->hMem = hMem;
              lpMem = (LPBYTE)pmyMem->bMem;
          }
     }
     return lpMem;
}
////////////////////////////////////////////////////////////////////////
//
// FUNCTION: UnLockAndFree(LPVOID )
//
// PURPOSE:  Free previously locked memory
//
// COMMENTS: Real applications must put extra sanity check before
// attempting to free memory such as signatures.
//
////////////////////////////////////////////////////////////////////////
/
VOID UnLockAndFree (LPVOID lpMem)
{
    // Recall that we had the handle of memory in the upper 4 bytes
    // and hence we extract that handle
    HGLOBAL hMem;
    PMY_MEMORY pmyMem = NULL;
    pmyMem = (PMY_MEMORY)((LPBYTE)lpMem - sizeof(HGLOBAL));
    if (pmyMem != NULL)
    {
        // Unlock and Free the memory
           hMem = pmyMem->hMem;
           if (GlobalUnlock (hMem))
               GlobalFree (hMem);
     }
     return;
}
```

```
////////////////////////////////////////////////////////////////////////
// FUNCTION: IPX_ESRoutine
//
// PURPOSE:  Processes asynchronous messages when called from IPX driver
// on completion of an ECB
////////////////////////////////////////////////////////////////////////
#if defined(WIN32)
VOID CALLBACK IPX_ESRoutine (PECB pecb)
#else
VOID _export CALLBACK IPX_ESRoutine (PECB pecb)
#endif
{
    // We have just received an ECB pointer which is really contained in
    // the MYECB structure. Thus, by pointer arithmetic
    // we determine the handle and WPARAM from MYECB
    PMYECB pmyEcb = (PMYECB) pecb;
    if (pmyEcb)
    {
        // Simply post a message to the main app which will complete
        // processing the ECB.
        PostMessage (pmyEcb->hWnd, WM_MSG_RECEIVED, pmyEcb->wMsg, 0);
    }
}
//
// FILE: ipxdemo.c
//
////////////////////////////////////////////////////////////////////////
// PROGRAM: IPXDEMO.c
// PURPOSE: The application prepares the application to send/receive
//          IPX messages.
// USAGE:
//      ; for starting the IPX receiver
//        ipxdemo
//             Menu Command in proper Sequence
//                  Starting: Initialize, OpenSocket, receive
//                  Stopping: CloseSocket, deinit, exit
//      ; starting Sender which sends IPX packet to a specific destination
//        ipxdemo
//             Menu Command in proper Sequence
//                  Starting: Initialize, OpenSocket, send
//                  Stopping: CloseSocket, deinit, exit
////////////////////////////////////////////////////////////////////////
#include "ipxdemo.h"     // specific to this program
////////////////////////////////////////////////////////////////////////
//
// Global Data
//
////////////////////////////////////////////////////////////////////////
HINSTANCE hInst;          // current instance

char szAppName[] = "ipxdemo";               // The name of this application
char szTitle[]   = "IPX Message Receiver";  // The title bar text

// Array of MYECB Structures. MYECB structures are themselves dynamically
// allocated and locked in memory
```

```
PMYECB     apmyECB [MAX_ECBS];
// With each ECB, we associate two fragments - one for IPX Header and
// another for data
PIPXHeader     apipxHeader [MAX_ECBS];
LPBYTE         aDataBuffer  [MAX_ECBS];

// ESR Proc Instance
void (far * ESRAddress)() = NULL;

// Receiver Address
static IPXAddress Receiver;
static BOOL       fSendBroadcast;
static BOOL       fSendMode = FALSE;

// Message Buffer which is used to display the messages
char szMessageBuffer [ 2 * MAX_PACKET_SIZE ];

// error messages buffer
char szErrorMsg [ 100 ] ;

////////////////////////////////////////////////////////////////////////
//
// FUNCTION: WinMain(HINSTANCE, HINSTANCE, LPSTR, int)
//
// PURPOSE: calls initialization function, processes message loop
//
////////////////////////////////////////////////////////////////////////

int APIENTRY WinMain(
          HINSTANCE hInstance,
          HINSTANCE hPrevInstance,
          LPSTR lpCmdLine,
          int nCmdShow)
{

     MSG          msg;
     WNDCLASS     wc;
     HWND         hWnd; // Main window handle.
     // If first instance, register app's window class
     if (!hPrevInstance)
     {
          // Fill in window class structure with parameters that
          // describe the main window.

            wc.style         = CS_HREDRAW | CS_VREDRAW;  // Class style(s).
            wc.lpfnWndProc   = (WNDPROC)WndProc;        // Window Procedure
            wc.cbClsExtra    = 0;                // No per-class extra data.
            wc.cbWndExtra    = 0;               // No per-window extra data.
            wc.hInstance     = hInstance;            // Owner of this class
            wc.hIcon         = LoadIcon (hInstance, szAppName);
                                                  // Icon name from .RC
            wc.hCursor       = LoadCursor(NULL, IDC_ARROW);       // Cursor
            wc.hbrBackground = (HBRUSH)(COLOR_WINDOW+1);   // Default color
            wc.lpszMenuName  = szAppName;             // Menu name from .RC
            wc.lpszClassName = szAppName;            // Name to register as
```

```
        // Register the window class and return success/failure code.
           if (!RegisterClass(&wc))
                return (FALSE);     // Exits if unable to register
    }
    // Save the instance handle in static variable, which will be used in
    // many subsequent calls from this application to Windows.

    hInst = hInstance; // Store instance handle in our global variable

    // Create a main window for this application instance.
    hWnd = CreateWindow(
                        szAppName,             // See RegisterClass() call.
                        szTitle,               // Text for window title bar.
                        WS_OVERLAPPEDWINDOW,               // Window style.
                        CW_USEDEFAULT,
                        0, CW_USEDEFAULT,   0,   // Use default positioning
                        NULL,          // Overlapped windows have no parent.
                        NULL,                  // Use the window class menu.
                        hInstance,       // This instance owns this window.
                        NULL      // We don't use any data in our WM_CREATE
                       );

    // If window could not be created, return "failure"
    if (!hWnd)
         return (FALSE);

    // Make the window visible; update its client area; return "success"
    ShowWindow(hWnd, nCmdShow);     // Show the window
    UpdateWindow(hWnd);             // Sends WM_PAINT message

     // Acquire and dispatch messages until a WM_QUIT message is received.
     while (GetMessage(&msg,  // message structure
                       NULL,  // handle of window receiving the message
                       0,     // lowest message to examine
                       0))    // highest message to examine
     {
        TranslateMessage(&msg);// Translates virtual key code
        DispatchMessage(&msg); // Dispatches message to window
     }


    return (msg.wParam); // Returns the value from PostQuitMessage
}
////////////////////////////////////////////////////////////////////////
// FUNCTION: WndProc(HWND, UINT, WPARAM, LPARAM)
//
// PURPOSE:  Processes messages
//
// COMMENTS: The code has been designed to handle simple user actions.
// The menu items should be selected in exact order
//    a) To start listening to packets:
//       Initialize, open socket, listen
//    b) To close IPX interaction:
//       close socket, deinit
////////////////////////////////////////////////////////////////////////
```

```
LRESULT CALLBACK WndProc(
                        HWND hWnd,          // window handle
                        UINT message,       // type of message
                        WPARAM uParam,      // additional information
                        LPARAM lParam)      // additional information
{
    int wmId, wmEvent;
    int iRc;
    HDC hDC;
    PAINTSTRUCT ps;
    TEXTMETRIC tm;
    static DWORD IPXTaskID;
    static WORD socket;
    static char xChar, yChar;
    static WORD NumLines;
    PMYECB pmyEcb;

    switch (message)
    {
    case WM_CREATE:   // Message sent when window is first created

        hDC = GetDC (hWnd);
        GetTextMetrics (hDC, &tm);
        xChar = (char)tm.tmAveCharWidth;
        yChar = (char)(tm.tmHeight + tm.tmExternalLeading);
        NumLines = 1;
        ReleaseDC (hWnd, hDC);
        szMessageBuffer [0] = '\0';
        break;

    case WM_COMMAND:  // message: command from application menu

// Message packing of uParam and lParam have changed for Win32; let us
// handle the differences in a conditional compilation:
#if defined (_WIN32) || defined(WIN32)
          wmId       = LOWORD(uParam);
          wmEvent    = HIWORD(uParam);
#else
          wmId       = uParam;
          wmEvent    = HIWORD(lParam);
#endif

          switch (wmId)
          {
          case IDM_EXIT:
               DestroyWindow (hWnd);
               break;

          // Call IPXInitialize and save task ID
          case IDM_IPXINITIALIZE:
               // Application will share resource pool
               IPXTaskID = 0xFFFFFFFFL;
               iRc = IPXInitialize (&IPXTaskID,
                                    MAX_ECBS,
                                    MAX_PACKET_SIZE);
               if (iRc)
```

```
        {
             DISPLAY_ERROR_MSG ("IPxInitialixe", iRc);
        }

        else

        {   // Set up ECBs and associated buffers
            if (!(SetupAppIPXResources(hWnd)))
            {
                 DISPLAY_ERROR_MSG ("SetupAppIPXResources",
                 0);
            }
        }
        break;
        case IDM_IPXDEINIT:
        // Release any resources we've allocated
        ReleaseAppIPXResources ();

        // Release resources allocated to use by IPX
        iRc = IPXSPXDeinit ( IPXTaskID );
        if (iRc)
        {
             DISPLAY_ERROR_MSG ("IPXSPXDeinit", iRc);
        }
        break;
        case IDM_OPEN:
        // Open a short-lived socket
        // We specify a fixed socket number so that
        // "sender" can always specify "receiver's"
        // socket number. One can use well-defined
        // socket number for well-known services
        // or dynamic sockets.
        socket  = IPX_DEMO_SOCKET;
        iRc = IPXOpenSocket (IPXTaskID,
                             &socket,
                             0);     // short-lived
        if (iRc)
        {
             DISPLAY_ERROR_MSG ("IPXOpenSocket", iRc);
        }
        else
        {
             if (!AcquireReceiverAddress (hWnd, &Receiver))
                  fSendBroadcast = FALSE;
             else
                  fSendBroadcast = TRUE;
        }
        break;
    case IDM_CLOSE:
        // If we were sending data, we notify the
        // network drivers that we are not going to send
        // any more data.
        if (fSendMode)
             IPXDisconnectFromTarget (IPXTaskID,
                                      (LPBYTE)&Receiver);
```

```
            // Close the socket
            IPXCloseSocket (IPXTaskID, socket);
            break;
        case IDM_LISTEN:
            // Start Listening to incoming IPX Messages
            iRc = StartIPXListen (IPXTaskID, socket);
            if (iRc)
            {
                DISPLAY_ERROR_MSG ("StartIPXListen", iRc);
            }
            break;
        case IDM_SEND:
            // Begin sending data
            iRc = SendIPXPacket (IPXTaskID, socket);
            if (iRc)
            {
                DISPLAY_ERROR_MSG ("SendPacket", iRc);
            }
            else
                fSendMode = TRUE;
            break;
        default:
            return (DefWindowProc(hWnd, message, uParam, lParam));
        }
        break;
    case WM_MSG_RECEIVED:
        // Received an IPX packet with LParam pointing to MYECB
        pmyEcb = (PMYECB) lParam;
        // Process the packet
        ProcessIPXPackets (hWnd, IPXTaskID, pmyEcb);
        break;
    case WM_MSG_SENT:
        // Check error status
        pmyEcb = (PMYECB) lParam;
        iRc = pmyEcb->ECB.completionCode;
        if (iRc != 0)
        {
            DISPLAY_ERROR_MSG ("Send Completed with Error!", iRc);
        }
        break;
    case WM_PAINT:
        // Repaint the window pane
        if (szMessageBuffer [ 0 ] != '\0')
        {
            BeginPaint (hWnd, &ps);
            // Compute lines to print assuming message is properly
            // formatted with newlines and each line has no more than
            // 80 chars.
            NumLines = NumLines + strlen (szMessageBuffer) / 80;
            TextOut (ps.hdc,
                     xChar,                    // Staring at x-coord.
                     yChar * NumLines,         // staring at y-coord.
                     szMessageBuffer,          // Message
                     strlen(szMessageBuffer)   // Message length
                    );
```

```
                // Clear out the message buffer
                szMessageBuffer [ 0 ] = '\0';
                EndPaint(hWnd, &ps);
           }
           break;
      case WM_DESTROY: // message: window being destroyed
           PostQuitMessage(0);
           break;

      default: // Passes it on if unprocessed
           return (DefWindowProc(hWnd, message, uParam, lParam));
      }
      return (0);
}
////////////////////////////////////////////////////////////////////////
// FUNCTION: SetupAppIPXResources(HANDLE )
//
// PURPOSE:  Allocates listen ECBs and associates data buffers with them.
// An ESR is also associated with each ECB.
//
// COMMENTS: We put each ECB in our private structure MYECB, which includes
// the handle of the window to receive notification and a message
////////////////////////////////////////////////////////////////////////

BOOL SetupAppIPXResources(HANDLE hWnd)
{
      int i;
      // Get the address of ESR and save in a global variable
      ESRAddress = (void (far *)())
                   MakeProcInstance ((FARPROC) IPX_ESRoutine, hInst);
      if (ESRAddress == NULL)
           return FALSE;

     // In Windows environment, lock the ESR code segment
     // so that it does not move during the
     // the lifetime of application (or until ReleaseResources() is called)

#if !defined(WIN32) && !defined(_WIN32)
      GlobalPageLock ((HANDLE)SELECTOROF (ESRAddress));
#endif

      //Allocate memory for the MYECB structures each with one ECB
      for ( i = 0; i < MAX_ECBS; i++)
      {
           apmyECB [ i ] = (PMYECB) AllocAndLock (sizeof(MYECB));
           RETURN_IF_MEM_FAILED ( apmyECB [ i ] );
           // Each ECB will have two fragments
           apmyECB [ i ]->ECB.fragmentCount = 2;

           // Each ECB is associated with same ESR
           apmyECB [ i ]->ECB.ESRAddress = ESRAddress;
           // In this case, each ECB sends message back to same window
           apmyECB [i]->hWnd = hWnd;
      }
```

```
    // Allocate memory for the IPXHeader structures per ECB
    for ( i = 0; i < MAX_ECBS; i++)
    {
        apipxHeader [ i ] = (PIPXHeader) AllocAndLock (sizeof(IPXHeader));
        RETURN_IF_MEM_FAILED ( apipxHeader [ i ] );

        // This is the first fragment associated with ECB
        apmyECB [ i ]->ECB.fragmentDescriptor [0].address = apipxHeader [i];
        apmyECB [ i ]->ECB.fragmentDescriptor [0].size = sizeof(IPXHeader);
    }

    // Allocate memory for the data for each ECB
    for ( i = 0; i < MAX_ECBS; i++)
    {
         aDataBuffer [ i ] = (LPBYTE) AllocAndLock (MAX_DATA_SIZE);
         RETURN_IF_MEM_FAILED ( aDataBuffer [ i ] );

         // This is the second fragment associated with ECB
         apmyECB [ i ]->ECB.fragmentDescriptor [1].address = aDataBuffer [i];
         apmyECB [ i ]->ECB.fragmentDescriptor [1].size = MAX_DATA_SIZE;
    }

}
////////////////////////////////////////////////////////////////////////////
//
// FUNCTION: ReleaseAppIPXResources( )
//
// PURPOSE:  Free all listen ECBs and associate data buffers with them.
//
// COMMENTS: Verify that each ECB is still not in use
//
////////////////////////////////////////////////////////////////////////////
VOID ReleaseAppIPXResources( )
{
    int i;

    // Release all memory allocated by SetupAddResources
    for ( i = 0; i < MAX_ECBS; i++)
    {
        // Verify that ECB is not in use
        if (apmyECB [ i ]->ECB.inUseFlag)
        {
            DISPLAY_ERROR_MSG("ECB in Use!",
                              apmyECB [ i ]->ECB.inUseFlag);
        }
        else
            UnLockAndFree ( (LPVOID) apmyECB [ i ] );
            UnLockAndFree ( (LPVOID) apipxHeader [ i ] );
            UnLockAndFree ( (LPVOID) aDataBuffer [ i ] );
    }

    // Release the ESR. We had stowed away the pointer in a global variable
    FreeProcInstance ( (FARPROC) ESRAddress );
}
```

```
////////////////////////////////////////////////////////////////////////
// FUNCTION: StartIPXListen(DWORD, WORD )
//
// PURPOSE:  Post multiple listen ECBs for the specified socket
//
// COMMENTS: Let user know the local address so that user can feed it to
//           sender
////////////////////////////////////////////////////////////////////////
int StartIPXListen (DWORD IPXTaskID, WORD socket)
{
     int i, iRc =0;
     IPXAddress MyAddress;
     // Associate the socket with each ECB and post a listen
     for (i = 0; i <= MAX_RCV_ECBS; i++)
     {
          apmyECB [ i ]->ECB.socketNumber = socket;
          // Message sent from ESR on completion of listen
          apmyECB [i]->wMsg = WM_MSG_RECEIVED;
          IPXListenForPacket (IPXTaskID, (PECB)&(apmyECB [ i ]->ECB));
     }
     // Show network address of the receiver
     IPXGetInternetworkAddress (IPXTaskID, (BYTE FAR *) &MyAddress);
     return ESUCCESS;
}
////////////////////////////////////////////////////////////////////////
// FUNCTION: ProcessIPXPackets(HANDLE, DWORD, PMYECB )
//
// PURPOSE:  In this demo application, we simply display the message that
// has arrived and then resubmit the completed ECB.
//
// COMMENTS: If ECB has encountered an error, we do not resubmit the ECB
////////////////////////////////////////////////////////////////////////
int ProcessIPXPackets ( HANDLE hWnd, DWORD IPXTaskID, PMYECB pmyEcb)
{
    PECB           pecb;
    int            iRc =0;
    IPXHeader      *pipxHeader;
    IPXAddress     *Source;
    LPSTR          lpszData;
    // Check the completion code
    pecb = (PECB)&(pmyEcb->ECB);
    if (pecb->completionCode != 0)
    {
        DISPLAY_ERROR_MSG ("Completion Code", pecb->completionCode);
    }

    // Create a display message showing the address of sender
    // and the message.
    pipxHeader = (IPXHeader *)(pecb->fragmentDescriptor[0].address);
    Source     = &(pipxHeader->source);
    wsprintf(szMessageBuffer,
        "Network Number: %02X%02X%02X%02X"      \
        "Node Number: %02X%02X%02X%02X%02X%02X\n"\
        "sent a message:\n%s",
        Source->network[0], Source->network[1],
```

```
        Source->network[2], Source->network[3],
        Source->node[0], Source->node[1], Source->node[2],
        Source->node[3], Source->node[4], Source->node[5],
        pecb->fragmentDescriptor[1].address
        );
    // Display the message (Only for small messages)
    InvalidateRect ( hWnd, NULL, TRUE);
    // Clear out data that has arrived
    lpszData = (LPSTR)pecb->fragmentDescriptor[1].address;
    *lpszData = '\0';

    // Resubmit the ECB to listen to incoming packets
    IPXListenForPacket (IPXTaskID, pecb);
    return (iRc);
}
////////////////////////////////////////////////////////////////////////
// FUNCTION: SendIPXPacket(DWORD, WORD )
//
// PURPOSE:  Send an IPX packet
//
// COMMENTS: When starting the application, the user can supply the
//           receiver's address. When no address is supplied, it is assumed
//           that the user wishes to broadcast messages.
//
////////////////////////////////////////////////////////////////////////
int SendIPXPacket (DWORD IPXTaskID, WORD socket)
{
    int             i, iRc, time;
    PECB            pecb = NULL;
    PIPXHeader      pipxHeader;

    PIPXAddress Dest, MyAddress;
    // Find a free send ECB
    for (i = MAX_RCV_ECBS; i <= MAX_ECBS; i++)
    {
        if (apmyECB [ i ]->ECB.inUseFlag == 0)
        {
               pecb = (PECB) &apmyECB [i]->ECB;
               // Message sent by ESR when send completes
               apmyECB [i]->wMsg = WM_MSG_SENT;

               // Set a fixed string as data for IPX
               pecb->fragmentDescriptor[1].size = strlen(IPX_DEMO_STRING);
               strcpy ( aDataBuffer [i], IPX_DEMO_STRING);
               break;
        }
    }
    if (pecb == NULL)
    {
        DISPLAY_ERROR_MSG ("No Free Send ECBs", 0);
        return 1;
    }
    // Setup destination address
    pipxHeader = (PIPXHeader) pecb->fragmentDescriptor[0].address;
    Dest       = (PIPXAddress) &(pipxHeader->destination);
```

```
    // Set the socket number
    *(WORD *)&(Dest->socket) = socket;
    if (fSendBroadcast)
    {
        // Set immediateAddress of ECB and node of destination
        memset (&pecb->immediateAddress, 0xFF, 6);
        memset (Dest->node, 0xFF, NODE_SIZE);

        // Determine sender's network number
        IPXGetInternetworkAddress (IPXTaskID, (BYTE FAR *) MyAddress);
        // Send broadcast on sender's network
        memcpy(&Dest->network, &MyAddress->network, NETWORK_SIZE);
    }
    else
    {
        // Set the destination address
        memcpy (&Dest->node,    &Receiver.node, NODE_SIZE) ;
        memcpy (&Dest->network, &Receiver.network, NETWORK_SIZE);

        // Determine immediate address
        iRc = IPXGetLocalTarget (IPXTaskID,
                                 (LPBYTE)Dest,
                                 (LPBYTE)&pecb->immediateAddress,
                                 &time
                                );
        if (iRc)
        {
               DISPLAY_ERROR_MSG ("GetLocalTarget", iRc);
               return iRc;
        }
    }
    // Save the receiver address in a global
    // variable so that IPXDisconnectFromTarget() can
    // be called
    Receiver = *Dest;

    // Submit a send ECB
    IPXSendPacket (IPXTaskID, pecb);
    return ESUCCESS;
}
```


9.4 SPX PROGRAMMING

So far, we have examined IPX programming, which allows applications to send and receive datagrams. Although the IPX data transmission service is unreliable, quite a few applications choose to use it due to its low overhead and efficient transmission rates. However, a significant number of applications wish to write minimal networking code and rely on the underlying system to provide a reliable data transmission service. Such applications can use the SPX interfaces, which provide guaranteed data delivery service.

Figure 9-6 shows the lifecycle of a typical client/server system that uses SPX as the IPC mechanism. In such a system, the server creates its end of the SPX socket and waits for clients to make a connection. The client makes a

local socket for sending and receiving data. It then tries to establish a connection with the server. When the specified server is located, an SPX connection is set up between the client and the server.

After a connection has been made, both client and server can send and receive messages via the SPX APIs. Either can break the connection at any time, which causes the other end to be notified. Once a connection with a client is terminated, the server can close the socket or start listening to new incoming connection requests. The server can service multiple clients (up to 100) at the same time by posting multiple "listen-for-connection" ECBs.

The SPX-based communication is comparable to the stream Windows Sockets. Indeed, a user can open Windows Sockets with SPX as the protocol to get a reliable IPC channel. There are also similarities between the connection-oriented SPX and session services of NetBIOS. Finally, the duplex reliable data service provided by SPX is comparable to that provided by Named Pipe.

SPX itself uses the datagram services of IPX. Thus, SPX applications enjoy a connection-oriented data transfer service, while, unbeknownst to them, directed datagrams are sent back and forth between the client and the server. Each SPX connection between a client and a server is identified by connection IDs unique to each side, yet together they define the connection between the two.[4] When a connection has been established with the remote end, the application receives a connection ID from its local SPX driver. In Figure 9-6, the server connection ID is 8, whereas the client uses a connection ID of 67

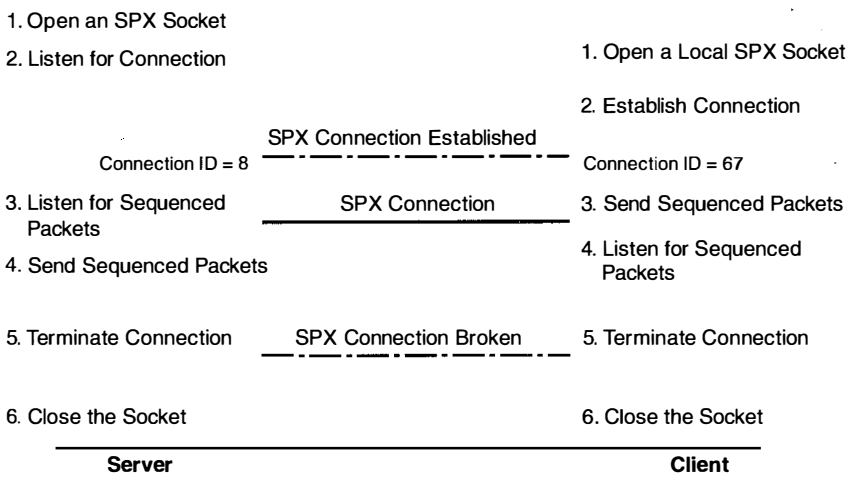


Figure 9-6. *A Typical Client/Server System That Uses SPX*

4. Readers familiar with NetBIOS will find the concept of connection IDs similar to local session number (LSN). Readers familiar with Named Pipes can find connection IDs similar to the handle received when a named pipe is connected.

to identify the same connection. An application specifies this local connection ID in subsequent send/receive functions to identify the other end, as opposed to specifying the complete address.

Table 9-5 shows the SPX functions that can be used by Windows applications. Certain IPX functions (for example, IPXOpenSocket()) are listed in the table because they are used by SPX applications as well.

Table 9-5. SPX Functions

API	*Caller Type*	*Description*
SPXInitialize()	All	All applications using SPX must call this function to initialize it. It is generally called at the beginning of a network-aware application.
IPXSPXDeinit()	All	All applications using SPX must call this function once for every SPXInitialize() made. It is generally called prior to terminating the application.
IPXOpenSocket()	All	Create a socket. Used to set up an SPX connection with the remote socket.
IPXCloseSocket()	All	Close the socket. Applications should close a socket after the SPX connection has been terminated or aborted.
SPXListenForConnection()	Server	An SPX-based server uses this function to make itself available to the clients so that an SPX connection can be established between the two ends.
SPXEstablishConnection()	Client	An SPX client uses this function to make a reliable and guaranteed data delivery connection with the server.
SPXGetConnectionStatus()	All	Used to determine the status of a connection.

(continued)

Table 9-5 continued

API	*Caller Type*	*Description*
SPXListenForSequencedPacket()	All	Allows the caller to receive data from its SPX partner.
SPXSendSequencedPacket()	All	Allows the caller to send data on an SPX connection.
SPXTerminateConnection()	All	Allows the caller to break the SPX connection gracefully. The remote application is notified by sending the termination packet.
SPXAbortConnection()	All	Can be used by applications to break a connection in case of abnormal conditions. The remote partner is not notified and will discover the broken connection after it times out sending or receiving data.

9.4.1 Data Structures and Asynchronous Processing

The data structures needed for SPX programming are similar to those needed for IPX programming. SPX-based applications pass the ECB data structure to the IPX or SPX interfaces. Thus, the details about ECBs, fragmentDescriptor, and asynchronous processing ECBs via ESR, discussed in the context of IPX programming, are applicable to SPX applications, with some exceptions. Programmatically, applications submit an ECB to an SPX/IPX interface to execute a certain command. The interfaces themselves return immediately after queuing the ECB and setting the inUseFlag field of the ECB to a nonzero value. After the ECB has been processed, timed out, or canceled, the driver sets the inUseFlag field to zero and sets the completionCode field of ECB. If an ESR has been associated with the ECB, the SPX driver calls the application-supplied ESR.

From a data structure point of view, the only noticeable difference between IPX and SPX programming is in the header prepended to all SPX packets. Recall that each IPX packet is prepended with a 30-byte IPXHeader data structure that limits the maximum user data size to 546 bytes. SPX packets are prepended with a 42-byte SPXHeader data structure, shown below. Notice that the SPXHeader is a superset of the IPXHeader. The additional fields are needed for maintaining an SPX connection. Similar to IPX packets, SPX

packets are also constrained by a 576-byte maximum packet size. Thus, SPX-based applications can send and receive only up to 532-bytes of data at a time.

```
typedef struct SPXHeader {
// First 30 bytes are similar IPXHeader
    WORD          checkSum;
    WORD          length;
    BYTE          transportControl;
    BYTE          packetType;
    IPXAddress    destination;
    IPXAddress    source;
// Following fields unique to SPXHeader
    BYTE          connectionControl;
    BYTE          dataStreamType;
    WORD          sourceConnectionID;
    WORD          destConnectionID;
    WORD          sequenceNumber;
    WORD          acknowledgeNumber;
    WORD          allocationNumber;
};
#define is_end_of_message(SPXHeader) { SPXHeader.connectionControl & 0x10 }
#define is_end_of_connection(SPXHeader) { SPXHeader.dataStreamType & 0xFE )
```

In the SPXHeader, the connectionControl flags control the bidirectional flow of data across a connection and are mostly used by the underlying system. However, the end-of-message bit (10h) within a connectionControl flag can be used by applications and is passed without change from the sender to the receiver. A typical application sets this bit to one to indicate to its SPX counterpart that it is sending the last message (see **is_end_of_message()** macro above).

"Why do I need the end of message bit?" The SPX protocol is geared toward sending and receiving messages, not streams of bytes (as the message-mode Named Pipe is). Each message can be contained in one SPX packet or may span multiple packets. Thus, whenever the sender starts to send a new message, the SPX driver on the receiver side begins putting the data in each incoming SPX packet in a buffer. Each subsequent SPX packet is presumed to contain the next fragment of data for the same message and is concatenated with the data sent earlier. When the sender notifies the receiver about the end of the message, the SPX driver knows that the message is over and can hand it over to the application (on the receiver side) and get ready for the next message.

The dataStreamType field of the SPXHeader can be used by the applications to classify the data being sent within an SPX packet. The values falling in the range [00h–FDh] can be used by applications. Two values (FEh, FFh) are reserved for the SPX protocol itself. The SPX driver sets the dataStreamType field to (FEh) to indicate the end of the connection (see is_end_of_connection() macro above), usually when the **SPXTerminate-Connection()** API is called by an application. The value (FFh) is used by the SPX driver on the remote end to acknowledge receiving the connection termination message.

The destConnectionID and sourceConnectionID fields contain values by which an SPX driver on either side can uniquely identify an SPX connection and are not set or accessed by applications. Similarly, the sequenceNumber, acknowledgeNumber, and allocationNumber fields are used by the SPX driver for providing guaranteed data delivery and are not to be used by applications.

9.4.2 Initializing SPX Applications

Applications using SPX must call the **SPXInitialize()** function before any other function. Similarly, it should call the IPXSPXDeinit() function once for every corresponding SPXInitialize() call. Typically, applications call the SPXInitialize() function at start-up time and the IPXSPXDeinit() API prior to termination.

```
int FAR PASCAL SPXInitialize( DWORD FAR *IPXTaskID,
                              WORD          maxECBs,
                              WORD          maxPacketSize,
                              BYTE FAR      *majorRevisionNumber,
                              BYTE FAR      *minorRevisionNumber,
                              WORD FAR      *maxConnections,
                              WORD FAR      *availableConnections);
int FAR PASCAL IPXSPXDeinit (DWORD IPXTaskID);
```

A successful return code from the SPXInitialize() function indicates that the SPX driver and environment is present. The SPXInitialize() function returns a task ID (via IPXTaskID parameter), which must be supplied to all subsequent SPX or IPX functions. The caller informs the SPX driver of the maximum number of ECBs (via the maxECBs parameter) that it anticipates using during this initialization context. The application also informs about the maximum packet size that it plans to generate. The default maximum packet size is 576 bytes. The SPX driver returns to the caller its own version number (via the majorRevisionNumber and minorRevisionNumber parameters). It then informs the application about the maximum connections available on the node (via the maxConnections parameter) and the number of SPX connections that this particular initialization context, or the application if it calls SPXInitialize() only once, can set up (via the availableConnections parameter).

The caller can set values in the IPXTaskID parameter prior to calling the SPXInitialize() function to indicate how it wants the SPX driver to allocate resources. A value of zero indicates that the application does not wish to share resources; by setting a value of (FFh) in IPXTaskID, the caller indicates that it wishes to share the resource pool with other applications.

An application should not mix and match task ID returned by the SPXInitialize() and the IPXInitialize() functions. Nor should it try to use an initialization context created by SPXInitialize() to use IPX functions that send and receive datagrams. Conversely, it should not use the task ID returned by the IPXInitialize() function to send and receive SPX packets.

Applications that need to send datagrams simultaneously as well as seek guaranteed data delivery should maintain distinct initialization contexts. They can, however, share the same set of ECBs and the same ESRs so long as they can distinguish the completion of an IPX event from an SPX event. This can be achieved by checking the packetType field of the ECB; the values (0, 4) indicate IPX packets, and the value 5 indicates an SPX packet. Such applications should explicitly set the packetType field of all their ECBs prior to submitting the ECBs to SPX or IPX functions.

9.4.3 Opening and Closing SPX Sockets

Like the IPX applications, SPX applications need to open a socket using the IPXOpenSocket() functions and close the opened sockets with the IPXCloseSocket() API. Unlike IPX applications, however, the IPXTaskID parameter must have been obtained by calling the SPXInitialize() function.

Typically, well-known services open a well-defined static socket number. This allows the clients to specify this socket number when locating the server. Servers can also use SAP services to broadcast their network address, including the dynamic socket number on the network. In this case, clients can obtain the server's network address by using the QueryServices() SAP API. Clients must also open a local socket because all SPX/IPX functions dealing with network I/O need a socket number.

9.4.4 Listen For Connections

SPX, we have seen, provides guaranteed data delivery once a connection has been set up between the client and the server. To set up a connection, the server end opens a socket and then actively listens for incoming connection requests by calling the **SPXListenForConnection()** API. Thus, the server allows clients to set up an SPX connection with itself. This section elaborates on server-side connections. The next section discusses operations a client must perform before it can connect to a server. Indeed, once the connection is set up, both ends can send and receive messages.

The SPXListenForConnection() function is comparable to the accept() Windows Sockets API and the ConnectNamedPipe() Named Pipe API, or the NCB.LISTEN NetBIOS command. The following code gives the prototype of the SPXListenFor-Connection() API:

```
int FAR PASCAL SPXListenForConnection ( DWORD       IPXTaskID,
                                        BYTE        retryCount,
                                        BYTE        watchDog,
                                        ECB FAR     *eventControlBlock );
```

The caller needs to specify the ESR address associated with the ECB but need not associate any fragments with the ECBs. Hence, when submitting "Connection" ECBs, the caller needs only to set the ESR address and socket number in the ECB. Thus, a server needs to call the SPXInitialize() function

to get a task ID and the IPXOpenSocket() API to get a socket number before it can call the SPXListenForConnection() function. The caller can also specify the number of times the SPX driver tries to establish a connection with the remote end (via the retryCount parameter).

When the SPX driver receives an SPXListenForConnection() call, it tries to establish the connection in two steps—a synchronous step followed by an asynchronous step, as follows:

1. It checks the parameters passed in and returns immediately on error. Then it checks the local connection table to see if there is room for one more connection. At this time, it enqueues the Connection ECB and returns the call immediately after setting the inUseFlag field of the ECB. It also places a connection identification (id) identifying the server half of the connection in the IPXWorkspace field of the ECB before returning from the SPXListenForConnection() API. The SPXListenForConnection() can return a number of immediate error codes, in which case the SPX driver does not process the ECB any further. A most common error encountered by busy servers is "Connection Table is Full" (EFh), which indicates that the node already has 100 SPX connections. This could also mean that some errant application is not terminating connections, leading to nonavailability of connections to other applications.
2. On successful completion of the first step, the SPX driver waits for a client. If a client is waiting to connect, the connection is established. Otherwise, it simply waits to receive a client request. When the client request does arrive, the SPX driver will call (asynchronously) the ESR associated with the ECB. Any pending ECB posted via the SPXListenForConnection() can be canceled by calling the IPXCancelEvent() API.

On successful completion of the **SPXEstablishConnection()** API, a server can periodically determine the status of the connection by calling the **SPXGetConnectionStatus()** API by passing in the connection ID returned by the SPXListenForConnection() API. The prototype of the SPXGetConnectionStatus() API and significance of the connectionState field in the CONNECTION_INFO structure follow. When the server has successfully called the SPXListenForConnection() API but no client has connected yet, the connect state will be WAITING.

```
int FAR PASCAL SPXGetConnectionStatus( DWORD     IPXTaskID,
                                       WORD      SPXConnectionID,
                                       CONNECTION_INFO FAR
                                                 *connectionInfo );
typedef struct {
          BYTE connectionState;
          BYTE connectionFlags;
```

```
        WORD sourceConnectionID;
        WORD destinationConnectionID;
        WORD sequenceNumber;
        WORD acknowledgeNumber;
        WORD remoteAcknowledgeNumber;
        WORD remoteAllocationNumber;
        WORD connectionSocket;
        BYTE immediateAddress [6];
        IPXAddress destination;
        WORD retransmissionCount;
        WORD estimatedRoundTripDelay;
        WORD retransmittedPackets;
        WORD suppressedPackets;
} CONNECTION_INFO;
// Connection State value and description
#define WAITING       0x01  // SPX is listening for connection, waiting
                            // for SPXEstablishConnection()
#define STARTING      0x02  // SPX is attempting to establish connection
#define ESTABLISHED   0x03  // SPX has established connection
#define TERMINATING   0x04  // Remote SPX has terminated connection, local
                            // SPX has not yet called
                            // SPXTerminateConnection()
```

When a client request to connect arrives, the SPX driver sends a confirmation of connection to the client. At this time, the 12-byte address of the client is placed in the driverWorkspace field of the ECB and is used by the SPX driver itself to send and receive IPX packets to the remote client. It puts the final return code in the completionCode field of the ECB, resets the inUseFlag, and calls the ESR associated with the connection ECB. At this time, the connection ID present in the IPXWorkspace field can be used by the server to send to, and receive packets from, the client.

The server application can control certain aspects of a connection behavior once it has been established with a client. First, it can specify the maximum number (0–255) of SPX driver resends of unacknowledged packets before the SPX driver will declare the remote partner dead. Usually this value is set to zero to allow the SPX driver to use the default/internal retry count.

One frequent problem associated with PC-based clients is that active connections suddenly become useless when the client crashes or is rebooted without properly shutting down the application using SPX. In such cases, the server application has no way of detecting the loss of partner until it has tried sending packets to the client (retryCount a number of times). The problem gets worse in CSSs in which servers themselves remain passive, responding only to client requests. In these cases, the server will not detect a loss of client and will continue to maintain its end of the connection, leading to the server-side connection table's being cluttered with nonactive connections. New clients may be refused connection with the server due to lack of available connections. In order to avoid these kinds of problems, a server can use a "watchdog" with the connection by setting the watchDog parameter to a nonzero value when calling the SPXListenForConnection() API. The watch-

dog process monitors all active connections and attempts to detect problems with active connections. In case of a dead or unavailable partner, the watchdog will put EDh (Failed Connection) in any pending connection ECBs and call the server's ESR.

Finally, applications are recommended to post at least two connection ECBs: one to confirm the connection request to the remote node and the other to receive acknowledgment of the confirmation. The number of pending ECBs should increase in proportion to the number of clients the server wishes to serve concurrently (for example, for serving five clients concurrently, a server should post ten connection ECBs).

9.4.5 Establishing a Connection

We have seen that by calling the SPXListenForConnection(), the server establishes its half of the connection. The client creates its half by calling the **SPXEstablishConnection()** API. Once the connection is made, client and server can send and receive data on the SPX connection.

This API can be compared to the connect() Windows Socket API, OpenFile() Win32 API (to connect to a Named Pipe), or NCB.CALL NetBIOS command. The prototype of the function follows:

```
int FAR PASCAL SPXEstablishConnection( DWORD IPXTaskID,
                                       BYTE     retryCount,
                                       BYTE     watchDog,
                                       WORD     FAR   *SPXConnectionID,
                                       ECB      FAR   *eventControlBlock );
```

The API takes a task ID (via the IPXTaskID parameter) obtained earlier by calling the SPXInitialize() API. The watchDog and retryCounts parameters are the same as the parameters passed to the SPXListenForConnection() API. This API also takes an ECB, which must be prepared before calling the SPXEstablishConnection() API, and a pointer to a variable where the connection ID can be returned.

The ECB must have a valid local socket number obtained via the IPXOpenSocket() API. The caller also needs to set a full 12-byte destination address (see IPXAddress structure) for the server in the destination field of the SPXHeader. The ECB must have the ESRAddress field initialized; more often than not, applications supply an ESR with the ECB. Finally, the ECB must be associated with exactly one fragment (fragmentCount = 1) whose size should be 42 bytes so that it can hold the SPXHeader.

Like the SPXListenForConnection() API, the SPXEstablish-Connection() API executes in two steps: a synchronous step followed by an asynchronous step:

1. The API verifies the passed-in parameters and returns immediately on error. It then checks the local connection table and can return an error if the table is full. Otherwise, it creates a local half of the

connection and returns the value via the SPXConnectionID parameter. Finally it places a nonzero value in the inUseFlag field of the ECB and returns to the client, which can check the status of the connection using the SPXGetConnectionStatus() API, which will return STARTING (2h) in the connectionState field of the CONNECT_INFO structure.

2. Subsequently, the SPX driver will attempt to connect to the server. If it can contact the server, the server will send it a confirmation of connection. At this point, the SPX driver on the client side will acknowledge receiving the confirmation and proceed to set fields in the connect ECB. It will reset the inUseFlag field, put the final completion code in the completionCode field, and call the ESR associated with the pending ECB, if any. After a successful completion of the ESR, the client can use the connection ID for sending and receiving data over the SPX connection. To cancel a pending connection ECB, a client can call the **SPXAbortConnection()** API, passing in the local connection ID returned via by the SPXEstablishConnection() API.

When the watchdog feature is used with the SPXEstablishConnection() API, the connection is monitored during its lifetime. On detecting an error in the connection, the watchdog can cause any ECB that is posted via the SPXListenForSequencedPacket() to be completed, with the field for completionCode set to EDh (SPX_CONNECTION_FAILED).

9.4.6 Breaking a Connection

Applications can use the **SPXTerminateConnection()** API to terminate a connection. The caller needs to specify the connection ID, which must have been obtained from successful completion of the SPXListenForConnection() or SPXEstablishConnection() API. Recall that each of these APIs immediately returns the local half of the connection even before an SPX connection has been established with the remote end. Once a connection has been established, the connection ID refers to the SPX connection between the two ends as opposed to the local half of the connection. The application can terminate the local half (or unconnected) connection or the full SPX connection by calling this API.

A client usually terminates its connection after it has completed communicating with the server, as does a server once it has completely serviced the client. In a well-designed CSS, one end communicates to the other the desire to end the session (by means of some message) before calling the SPXTerminateConnection() API. This allows each end to release resources gracefully. Indeed, both ends should be prepared to recover from a failed connection (for example, due to network error). In either case, applications

should call the SPXTerminateConnection() API once a connection has been established. Here is the prototype of the function.

```
int FAR PASCAL SPXTerminateConnection( DWORD     IPXTaskID,
                                       WORD      SPXConnectionID,
                                       ECB FAR   *eventControlBlock);
```

Besides the IPX task ID and the connection ID, the caller needs to provide a well-prepared ECB to the SPXTerminateConnection() API. The ECB must have one fragment (fragmentCount = 1), which should have a size equal to the SPXHeader (42 bytes). At this call, the SPX driver sets the dataStreamType field of the SPXHeader to FEh (TERMINATE_CONNECTION) and delivers the packet to the SPX partner. It then resets the inUseFlag field and puts a final status code in the completionCode field of the ECB. Applications calling the SPXTerminateConnection() API must be prepared in case the SPX driver fails to inform the remote end, due to either network failure or a failure of the remote application, indicated by error EDh in the completionCode field.

On completion of this API, the SPX driver can reuse the connection ID given to the application. Thus, the application should not use the connection ID specified when calling the SPXTerminateConnection() function.

9.4.7 Send and Receive Data

Client and server applications can send and receive data once an SPX connection has been established between them. The **SPXSendSequencedPacket()** API is used to send data buffers and the SPXListenForSequencedPacket() API to receive data. Because SPX protocol provides guaranteed data delivery service (in marked contrast to the IPX APIs), successful completion of these APIs ensures data transfer. Additionally, like the other SPX/IPX APIs, these APIs are executed asynchronously, allowing an application to continue to respond to new requests while performing network I/O in the background. The prototype of the functions follows:

```
int FAR PASCAL SPXListenForSequencedPacket ( DWORD     IPXTaskID,
                                             ECB FAR  *eventControlBlock);
int FAR PASCAL SPXSendSequencedPacket( DWORD     IPXTaskID,
                                       WORD      SPXConnectionID,
                                       ECB FAR   *eventControlBlock);
```

Due to the asynchronous nature of these APIs, we will refer to the SPXSendSequencedPacket() API as "posting a send" and to the SPXListenForSequencedPacket() API as "posting a listen." The ECBs (via the eventControlBlock parameter) posted via these APIs will be referred to as "Send ECB" and "Listen ECB," respectively.

The basic model of calling these APIs is fairly simple. In a one-way communication, one end posts a listen, while the other posts a send. In a duplex communication, both ends will post a send and a listen simultaneously. However, it is not necessary for the receiver to post a listen before the sender

calls the SPXSendSequencedPacket(). Indeed, the send will not complete until the receiver has posted a listen ECB and received the sent data. Another important point is that although the sender always specifies an SPX connection ID when posting a send, the receiver does not need to specify the connection ID *a priori*, so a receiver can post multiple listen ECBs and respond to each completed ECB as it occurs. The SPX driver puts the connection ID in the driverWorkspace field of a listen ECB so that the receiver can identify the sender if it needs to. This works well for a server (for example, database server) that must respond to multiple connected clients. Instead of posting a listen ECB per each connection, the server can always maintain a pool of listen ECBs and keep them posted. As each listen ECB completes, the server detects the identity of the client and responds in a proper context.

Each API takes a task ID (via the IPXTaskID parameter) that must have been received by calling the SPXInitialize() function. Before posting a listen ECB, the caller needs to specify the number of fragments, as well as their size and address, via the fragmentCount and the fragmentDescriptor fields. On successful completion of a listen ECB, the buffers pointed to by the fragmentDescriptor will contain the SPXHeader and the data following the header. The application posting a listen also specifies the socket number and the ESR address, if any. When a non-NULL ESR address is specified, the SPX driver calls the ESR on receiving an SPX packet for the given socket. The driver sets the inUseFlag to zero and a final return code in the completionCode field of the ECB. When no ESR has been specified, the SPX driver resets the inUseFlag and sets the completionCode fields. The SPX driver does not maintain an order in which the listen ECBs are posted by the receiving application. Hence, on arrival of an SPX packet, the SPX driver can pick any of the posted listen ECBs to provide incoming data to the receiving application. The application, however, is itself guaranteed to receive data from its SPX partner in the exact order they were sent. For example, the receiver may receive packet 1 on the seventeenth listen ECB and packet 2 on the third. However, it is guaranteed to receive packet 2 after packet 1 and before packet 3.

Another important point about SPX connection-based communication is that, as the remote end terminates the connection (when it calls the SPXTerminateConnection() API), the local receiver will receive a completed listen ECB in which the dataStreamType field will be set to FEh (end-of-connection). On receiving the end-of-connection message, the local SPX driver sends an acknowledgment to the node, terminating the connection. This dictates that applications must post at least two listen ECBs per connection so that the SPX driver can use one to receive the end-of-connection message and another to send acknowledgment.

The applications sending data over an established connection need to specify the data via the fragments associated with the send ECB. The first fragment descriptor must be at least 42 bytes large so it can hold the

SPXHeader. Most applications find it convenient to have the first fragment point to a 42-byte buffer and the second fragment pointing to the user data. The sender also needs to initialize the dataStreamType field of SPXHeader. An application can send an SPX packet that contains only an SPXHeader, that is, 0 bytes of data. This option is usually chosen to send end-of-message notification (by setting the connectionControl field of the SPXHeader).

On receiving a send ECB, the SPX driver sets the inUseFlag field to a nonzero value and returns control to the calling application. The SPX driver remembers the sequence in which the send ECBs are posted so applications can send and receive data packets in order. At system-defined intervals, the SPX driver attempts to send a packet to the remote application as identified by the SPX connection ID. It is thus necessary for the sender to post a send ECB only after a valid connection has been established with the remote end. If the SPX driver detects that the remote end cannot be reached despite a number of attempts, it will abort the connection and complete the send ECB with error FDh set in the completionCode field. On receiving this error, the application should stop using the aborted connection.

9.4.8 SPX Sample Program

The sample Windows 3.x (SPXDemo.c) program is very similar to the sample shown in the IPX programming section. The given instance of the program can function as a "sender" (the client) or as a "receiver" (the server). The receiver creates an SPX socket and awaits sender connect requests. The sender connects with the receiver and sends it an SPX packet via the SPX connection. At the end, both sides terminate their respective connections. In this simple example, the user is expected to execute the correct action [for example, call SPXInitialize() before calling IPXOpenSocket()].

The sender puts up a dialog box to obtain the network and node numbers of the receiver. To make it easy, the receiver prints its own local numbers on screen, which can then be easily fed into the sender program. The program uses a static socket number (0x4040). Just like the IPX sample program, all functions execute asynchronously and call the ESR (IPX_ESRoutine() in common.c), which in turn sends a message to the main window.

The program (SPXDemo.c) follows. Note that it uses the same common functions (see common.c) shown earlier in the IPX sample program.

The **SetupAppSPXResources()** function allocates the ECBs and the two data buffers associated with each ECB. It also associates each ECB with the main window handle (see MYECB structure) so that the ESR can send the main window a message. **ReleaseAppSPXResources()** releases the buffers.

The StartSPXListen() function is used to post listen ECBs via the SPXListenForSequencedPacket() API. Note that we post multiple ECBs to improve the efficiency of the program. When an SPX packet arrives, the ESR

sends a message (WM_MSG_RECEIVED) to the main window, which calls the **ProcessSPXPacket()** function.

ProcessSPXPacket() displays the content of the incoming SPX packet and detects the end-of-message and end-of-connection signals. When neither is true, the function resubmits the ECB to receive the next incoming SPX packet.

The SendPacket() function is used to send an SPX packet. A successful completion of the SPXSendSequencedPacket() API causes the sender to receive a message (WM_MSG_SENT). Finally, the **SetupSPXConnection()** and **TerminateSPXConnection()** functions are used to set up and tear down an SPX connection. Note that we associate the special messages WM_CONN_SETUP and WM_CONN_TERMINATED with the connection and termination APIs. By associating a window message with all the asynchronous SPX/IPX APIs, we make the SPX/IPX programming similar to asynchronous Windows Socket programming.

```
// See %BOOK%\ipxspx\spxdemo.mak for full project details
// FILE: SPXDemo.c
////////////////////////////////////////////////////////////////////////
// PROGRAM: SPXDEMO.c
//
// PURPOSE: The application prepares the application to send/receive
// SPX messages.
//
// FUNCTIONS:
//
// WinMain() - calls initialization function, processes message loop
// WndProc() - processes messages
//
//
// Usage:
// ; for starting the SPX receiver
// spxdemo
// Menu Command in proper Sequence
// Starting: Initialize, Opensocket,
// ListenForConnection, receive
// Stopping: TerminateSPXConnection, CloseSocket,
// deinit, exit
// ; starting Sender which sends SPX packet to a specific destination
// spxdemo
// Menu Command in proper Sequence
// Starting: Initialize, OpenSocket,
// EstablishConnection, end
// Stopping: TerminateSPXConnection, CloseSocket,
// deinit, exit
////////////////////////////////////////////////////////////////////////

#include "spxdemo.h"     // specific to this program
#include <ctype.h>
////////////////////////////////////////////////////////////////////////
// Global Data
////////////////////////////////////////////////////////////////////////
```

```
HINSTANCE hInst;     // current instance

char szAppName[] = "spxdemo";             // The name of this application
char szTitle[] = "SPX Message Receiver";  // The title bar text

// Array of MYECB Structures. MYECB structures are themselves dynamically
// allocated and locked in memory
PMYECB     apmyECB [ MAX_ECBS];
// With each ECB, we initialize two fragments-one for SPX Header
// and another
// for data. However, if an ECB needs to be used with
// EstablishConnection(),
// we set the fragment count to 1.
PSPXHeader apspxHeader [MAX_ECBS];
LPBYTE     aDataBuffer [MAX_ECBS];

// ESR Proc Instance
FARPROC ESRAddress = NULL;

// Receiver Address
static IPXAddress Receiver;
// Send or Receive Data
static BOOL      fSendMode = FALSE;
// SPX Connection ID
static DWORD     SPXConnID = 0;
// Listening or Establishing Connection
static BOOL fListenForConnection = FALSE;
static BOOL fSendBroadcast = FALSE;

// Message Buffer which is used to display the messages
char szMessageBuffer [ 2 * MAX_PACKET_SIZE ];

//  error messages buffer
char szErrorMsg [ 100 ] ;
////////////////////////////////////////////////////////////////////////
//
// FUNCTION: WinMain(HINSTANCE, HINSTANCE, LPSTR, int)
//
// PURPOSE: calls initialization function, processes message loop
//
////////////////////////////////////////////////////////////////////////

int APIENTRY WinMain(
      HINSTANCE hInstance,
      HINSTANCE hPrevInstance,
      LPSTR lpCmdLine,
      int nCmdShow)
{

    MSG         msg;
    WNDCLASS    wc;
    HWND        hWnd; // Main window handle.
    // If first instance, register app's window class
    if (!hPrevInstance)
```

```
{
    // Fill in window class structure with parameters that
    // describe the main window.

      wc.style       = CS_HREDRAW | CS_VREDRAW;// Class style(s).
      wc.lpfnWndProc = (WNDPROC)WndProc;      // Window Procedure
      wc.cbClsExtra  = 0;                  // No per-class extra data.
      wc.cbWndExtra  = 0;                  // No per-window extra data.
      wc.hInstance   = hInstance;          // Owner of this class
      wc.hIcon = LoadIcon (hInstance, szAppName); // Icon name from .RC
      wc.hCursor     = LoadCursor(NULL, IDC_ARROW);// Cursor
      wc.hbrBackground = (HBRUSH)(COLOR_WINDOW+1); // Default color
      wc.lpszMenuName = szAppName;                // Menu name from .RC
      wc.lpszClassName = szAppName;              // Name to register as

         // Register the window class and return success/failure code.
         if (!RegisterClass(&wc))
             return (FALSE);     // Exits if unable to register

}
// Save the instance handle in static variable, which will be used in
// many subsequent calls from this application to Windows.

hInst = hInstance; // Store instance handle in our global variable

// Create a main window for this application instance.
hWnd = CreateWindow(
                    szAppName,            // See RegisterClass() call.
                    szTitle,              // Text for window title bar.
                    WS_OVERLAPPEDWINDOW,  // Window style.
                    CW_USEDEFAULT, 0, CW_USEDEFAULT, 0,
                                          // Use default positioning.
                    NULL,          // Overlapped windows have no parent.
                    NULL,                 // Use the window class menu.
                    hInstance,       // This instance owns this window.
                    NULL     // We don't use any data in our WM_CREATE
                   );

// If window could not be created, return "failure"
if (!hWnd)
    return (FALSE);
// Make the window visible; update its client area; return "success"
ShowWindow(hWnd, nCmdShow);     // Show the window
UpdateWindow(hWnd);             // Sends WM_PAINT message

// Acquire and dispatch messages until a WM_QUIT message is received.

while (GetMessage(&msg,       // message structure
                  NULL,       // handle of window receiving the message
                  0,          // lowest message to examine
                  0))         // highest message to examine
{
      TranslateMessage(&msg);// Translates virtual key code
      DispatchMessage(&msg); // Dispatches message to window
}
return (msg.wParam); // Returns the value from PostQuitMessage
}
```

```
//////////////////////////////////////////////////////////////////////
//
// FUNCTION: WndProc(HWND, UINT, WPARAM, LPARAM)
//
// PURPOSE:  Processes messages
// COMMENTS: The code has been designed to handle simple user actions.
// The menu items should be selected in exact order
//        a) To start listening to packets
//           Initialize, open socket, listen
//        b) To close IPX interaction
//           close socket, deinit
//////////////////////////////////////////////////////////////////////

LRESULT CALLBACK WndProc(
                        HWND hWnd,          // window handle
                        UINT message,       // type of message
                        WPARAM uParam,      // additional information
                        LPARAM lParam)      // additional information
{
     int              wmId, wmEvent;
     int              iRc;
     HDC              hDC;
     PAINTSTRUCT      ps;
     TEXTMETRIC       tm;
     static   DWORD   SPXTaskID;
     static   WORD    socket;
     static   char    xChar,   yChar;
     static   WORD    NumLines;
     BYTE  majorrev,  minorrev;
     UINT  maxConn,   availCon;
     PIPXAddress      Client;
     PMYECB           pmyEcb;

     switch (message)
     {
     case WM_CREATE: // Message sent when window is first created

          hDC = GetDC (hWnd);
          GetTextMetrics (hDC, &tm);
          xChar = (char)tm.tmAveCharWidth;
          yChar = (char)(tm.tmHeight + tm.tmExternalLeading);
          NumLines = 1;
          ReleaseDC (hWnd, hDC);
          szMessageBuffer [0] = '\0';
          break;

     case WM_COMMAND:  // message: command from application menu

// Message packing of uParam and lParam have changed for Win32, let us
// handle the differences in a conditional compilation:
#if defined (_WIN32) || defined(WIN32)
          wmId          = LOWORD(uParam);
          wmEvent       = HIWORD(uParam);
#else
          wmId          = uParam;
          wmEvent       = HIWORD(lParam);
#endif
```

```
switch (wmId)
{
case IDM_EXIT:
     DestroyWindow (hWnd);
     break;

// Call SPXInitialize and save task ID
case IDM_IPXINITIALIZE:
     // Application will share resource pool
     SPXTaskID = 0xFFFFFFFFL;
     iRc = SPXInitialize (&SPXTaskID,
                          MAX_ECBS,
                          MAX_PACKET_SIZE
                          &majorrev, &minorrev,
                          &maxConn, &availCon
                         );

     if (iRc)
     {
          DISPLAY_ERROR_MSG ( "SPxInitialixe", iRc);
     }
     else
     {
          // Check that we have available connections
          if (availCon <=0)
          {
               DISPLAY_ERROR_MSG("No Avail SPX Connections",
                                 availCon);
               break;
          }
          // Set up ECBs and associated buffers
          if (!(SetupAppSPXResources(hWnd)))
          {
               DISPLAY_ERROR_MSG ("SetupAppSPXResources", 0);
          }
     }
     break;
case IDM_IPXDEINIT:
     // Release any resources allocated
     ReleaseAppSPXResources ();

     // Release resources we have allocated to use by SPX
     iRc = IPXSPXDeinit ( SPXTaskID );
     if (iRc)
     {
          DISPLAY_ERROR_MSG ("IPXSPXDeinit", iRc);
     }
     break;
case IDM_OPEN:
     // Open a short-lived socket
     // We specify a fixed socket number so that
     // "sender" can always specify "receiver's"
     // socket number. One can use well-defined
     // socket number for well-known services
     // or dynamic sockets.
     socket  = IPX_DEMO_SOCKET;
```

```
        iRc = IPXOpenSocket (SPXTaskID,
                             &socket,
                             0);      // short-lived
        if (iRc)
        {
             DISPLAY_ERROR_MSG ("IPXOpenSocket", iRc);
        }
        else
        {
             if (!AcquireReceiverAddress (hWnd, &Receiver))
                  fSendBroadcast = TRUE;
             else
                  fSendBroadcast = FALSE;
        }
        break;
   case IDM_LISTEN_CONN:
        // In this case, we are going to be the 'Server'
        // and call SPXListenForConnection() waiting
        // for 'Client' to set up a connection
        iRc = SetupSPXConnection ( SPXTaskID, socket, TRUE );
        if (iRc)
        {
             DISPLAY_ERROR_MSG("SPXListenForConnection", iRc);
        }
        else
             fListenForConnection = TRUE;
        break;
   case IDM_ESTB_CONN:
        // In this case, we are going to be the 'Client'
        // and call SPXEstablishConnection() attempting
        // to set up a connection with a 'Server'
        iRc = SetupSPXConnection ( SPXTaskID, socket, FALSE );
        if (iRc)
        {
             DISPLAY_ERROR_MSG("SPXEstablishConnection", iRc);
        }
        break;
   case IDM_TERM_CONN:
        // If we have opened a connection,
        // we need to terminate the connection
        // prior to closing the socket
        iRc = TerminateSPXConnection ( SPXTaskID );
        if (iRc)
        {
             DISPLAY_ERROR_MSG("TerminateSPXConnection", iRc);
        }
        // We cannot use this SPX Connection ID anymore
        SPXConnID = 0;
        break;
   case IDM_CLOSE:
        // Close the socket
        IPXCloseSocket (SPXTaskID, socket);
        break;
   case IDM_LISTEN:
        // Start Listening to incoming SPX Messages
        iRc = StartSPXListen (SPXTaskID, socket);
```

```
                if (iRc)
                {
                    DISPLAY_ERROR_MSG ("StartSPXListen", iRc);
                }
                break;

            case IDM_SEND:
                // Begin sending data
                iRc = SendSPXPacket (SPXTaskID, socket);
                if (iRc)
                {
                    DISPLAY_ERROR_MSG ("SendSPXPacket", iRc);
                }
                else
                    fSendMode = TRUE;
                break;
            default:
                return (DefWindowProc(hWnd, message, uParam, lParam));
            }
            break;

        case WM_MSG_RECEIVED:
            // Received an SPX packet with LParam pointing to
            // MYECB
            pmyEcb = (PMYECB) lParam;
            // Process the packet
            ProcessSPXPackets ( hWnd, SPXTaskID, pmyEcb);
            break;
        case WM_MSG_SENT:
            // Check error status
            pmyEcb = (PMYECB) lParam;
            iRc = pmyEcb->ECB.completionCode;
            if (iRc != 0)
            {
                DISPLAY_ERROR_MSG ("Send Completed with Error!", iRc);
            }
            break;
        case WM_CONN_SETUP:
            // Check error status
            pmyEcb = (PMYECB) lParam;
            iRc = pmyEcb->ECB.completionCode;
            if (iRc != 0)
            {
                DISPLAY_ERROR_MSG ("SPX Connection Setup Error!", iRc);
            }
            else if (fListenForConnection)
            {
                // This is the server who was connected by a client
                // Extract connection ID from IPXWorkspace field
                memcpy(&SPXConnID, pmyEcb->ECB.IPXWorkspace, sizeof(DWORD));
                // Display the client address which is in
                // driverWorkspace field of ECB
                Client = (PIPXAddress)&(pmyEcb->ECB.driverWorkspace[0]);
                wsprintf( szMessageBuffer,
                          "A client connected to us " \
                          "Network Number: %02X%02X%02X%02X" \
```

```
                        "Node Number: %02X%02X%02X%02X%02X%02X\n",
                        Client->network[0], Client->network[1],
                        Client->network[2], Client->network[3],
                        Client->node[0], Client->node[1], Client->node[2],
                        Client->node[3], Client->node[4], Client->node[5]
                        );
                // Display the message
                InvalidateRect ( hWnd, NULL, TRUE);
        }

        // Set the fragment count back to 2
        // (It was set to 1 in SetupSPXConnection() function)
        pmyEcb->ECB.fragmentCount = 2;
        break;
    case WM_CONN_TERMINATED:
        // Check error status
        pmyEcb = (PMYECB) lParam;
        iRc = pmyEcb->ECB.completionCode;
        if (iRc != 0)
        {
        DISPLAY_ERROR_MSG ("SPXTerminateConn. Completed with Error!", iRc);
        }
        // Set the fragment count back to 2
        // (It was set to 1 in TerminateSPXConnection() function)
        pmyEcb->ECB.fragmentCount = 2;
        break;
    case WM_PAINT:
        // Repaint the window pane
        if (szMessageBuffer [ 0 ] != '\0')
        {
               BeginPaint (hWnd, &ps);
               // Compute lines to print assuming message is properly
               // formatted with new lines and each line has no more than
               // 80 chars.
               NumLines = NumLines + strlen (szMessageBuffer) / 80;
               TextOut ( ps.hdc,
                         xChar,                    // Staring at x-coord.
                         yChar * NumLines,         // staring at y-coord.
                         szMessageBuffer,          // Message
                         strlen(szMessageBuffer)   //Message length
                        );

               // Clear out the message buffer
               szMessageBuffer [ 0 ] = '\0';
               EndPaint(hWnd, &ps);
         }
         break;
    case WM_DESTROY:  // message: window being destroyed
          PostQuitMessage(0);
          break;

     efault:     // Passes it on if unproccessed
          return (DefWindowProc(hWnd, message, uParam, lParam));
    }
    return (0);
}
```

```
//////////////////////////////////////////////////////////////////////////
// FUNCTION: SetupAppSPXResources(HANDLE )
//
// PURPOSE:  Allocate listen ECBs and associate data buffers with them.
// An ESR is also associated with each ECB.
//
// COMMENTS: We put each ECB in our private structure MYECB which includes
// handle of the window to receive notification and a message
//////////////////////////////////////////////////////////////////////////
BOOL SetupAppSPXResources(HANDLE hWnd)
{
    int i;
    // Get the address of ESR and save in a global variable
    ESRAddress = MakeProcInstance ( (FARPROC) IPX_ESRoutine, hInst);
    if (ESRAddress == NULL)
        return FALSE;

    // In Windows environment, lock the ESR code segment
    // so that it does not move during the
    // the lifetime of application (or until ReleaseResources() is called)

#if !defined(WIN32) && !defined(_WIN32)
    GlobalPageLock ((HANDLE)SELECTOROF (ESRAddress));
#endif

   //Allocate memory for the MYECB structures each of which contain one ECB
     for ( i = 0; i < MAX_ECBS; i++)
    {
          apmyECB [ i ] = (PMYECB) AllocAndLock (sizeof(MYECB));
          RETURN_IF_MEM_FAILED ( apmyECB [ i ] );
          // Each ECB will have two fragments
          apmyECB [ i ]->ECB.fragmentCount = 2;

          // Each ECB is associated with same ESR
          apmyECB [ i ]->ECB.ESRAddress = ESRAddress;
          // In this case, each ECB sends message back to same window
          apmyECB [i]->hWnd = hWnd;

    }
    // Allocate memory for the SPXHeader structures per ECB
    for ( i = 0; i < MAX_ECBS; i++)
    {
          apspxHeader [ i ] = (PSPXHeader)
                              AllocAndLock (sizeof(SPXHeader));
          RETURN_IF_MEM_FAILED ( apspxHeader [ i ] );

          // This is the first fragment associated with ECB
          apmyECB [ i ]->ECB.fragmentDescriptor [0].address = apspxHeader [i];
          apmyECB [ i ]->ECB.fragmentDescriptor [0].size = sizeof(SPXHeader);
    }

    // Allocate memory for the data for each ECB
    for ( i = 0; i < MAX_ECBS; i++)
    {
          aDataBuffer [ i ] = (LPBYTE) AllocAndLock (MAX_DATA_SIZE);
          RETURN_IF_MEM_FAILED ( aDataBuffer [ i ] );
```

```
        // This is the second fragment associated with ECB
      apmyECB [ i ]->ECB.fragmentDescriptor [1].address = aDataBuffer [i];
      apmyECB [ i ]->ECB.fragmentDescriptor [1].size    = MAX_DATA_SIZE;
    }

}
////////////////////////////////////////////////////////////////////////////
//
// FUNCTION: ReleaseAppSPXResources( )
//
// PURPOSE:  Free all listen ECBs and associate data buffers with them.
//
// COMMENTS: Verify that each ECB is still not in use
//
////////////////////////////////////////////////////////////////////////////
VOID ReleaseAppSPXResources( )
{
     int i;

     // Release all memory allocated by SetupAddResources
     for (i = 0; i < MAX_ECBS; i++)
     {
          // Verify that ECB is not in use
          if (apmyECB [ i ]->ECB.inUseFlag)
          {
              DISPLAY_ERROR_MSG("ECB in Use!",
                                apmyECB [ i ]->ECB.inUseFlag);
          }
          else
               UnLockAndFree ( (LPVOID) apmyECB [ i ] );
               UnLockAndFree ( (LPVOID) apspxHeader [ i ] );
               UnLockAndFree ( (LPVOID) aDataBuffer [ i ] );
     }
   // Release the ESR. We had stowed away the pointer in a global variable
     FreeProcInstance ( ESRAddress );
}
////////////////////////////////////////////////////////////////////////////
//
// FUNCTION: StartSPXListen(DWORD, WORD )
//
// PURPOSE:  Post multiple Listen ECBs for the specified socket
//
// COMMENTS: Let user know the local address to feed it to sender
//
////////////////////////////////////////////////////////////////////////////
int StartSPXListen (DWORD SPXTaskID, WORD socket)
{
    int i, iRc =0;
    IPXAddress MyAddress;
    // Associate the socket with each ECB and post a listen
    for (i = 0; i <= MAX_RCV_ECBS; i++)
    {
          apmyECB [ i ]->ECB.socketNumber = socket;
          // Message sent from ESR on completion of listen
          apmyECB [ i ]->wMsg = WM_MSG_RECEIVED;
          SPXListenForSequencedPacket (SPXTaskID,
```

```
                    (PECB)&(apmyECB [ i ]->ECB));
    }
    // Show network address of the receiver
    iRc = IPXGetInternetworkAddress (SPXTaskID, &MyAddress);
    if (iRc ==0)
    {
          sprintf (szErrorMsg,
                   "My Network Number:" \
                   "%02X%02X%02X%02X Node"\
                   "Number: %02X%02X%02X%02X%02X%02X\n",
                   MyAddress.network[0], MyAddress.network[1],
                   MyAddress.network[2], MyAddress.network[3],
                   MyAddress.node[0], MyAddress.node[1], MyAddress.node[2],
                   MyAddress.node[3], MyAddress.node[4], MyAddress.node[5]
                  );
          MessageBox (GetActiveWindow (),
                      szErrorMsg,
                      "My Address",
                      MB_OK
                     );
    }
    return iRc;
}
////////////////////////////////////////////////////////////////////////////
//
// FUNCTION: ProcessSPXPackets(HANDLE, DWORD, PMYECB )
//
// PURPOSE:  In this demo application, we simply display the message that
// has arrived and then resubmit the completed ECB.
//
// COMMENTS: If ECB has encountered an error, we do not resubmit the ECB
////////////////////////////////////////////////////////////////////////////
int ProcessSPXPackets ( HANDLE hWnd, DWORD SPXTaskID, PMYECB pmyEcb)
{
    PECB          pecb;
    int           iRc =0;
    PSPXHeader    pspxHeader;
    PIPXAddress   Source;
    LPSTR         lpszData;
    // Check the completion code
    pecb = (PECB)&(pmyEcb->ECB);
    if (pecb->completionCode != 0)
    {
        DISPLAY_ERROR_MSG ("Completion Code", pecb->completionCode);
        return (-1);
    }

    // Create a display message showing the message
    // sent by the sender
    pspxHeader = (PSPXHeader)(pecb->fragmentDescriptor[0].address);
    // Check if this is the end of message or end-of-connection message
    if (IS_END_OF_MESSAGE(pspxHeader) || IS_END_OF_CONNECTION(pspxHeader))
    {
        sprintf(szMessageBuffer, "End of Connection or Message");
    }
    else
```

```
    {
          Source     = &(pspxHeader->source);
          sprintf( szMessageBuffer,
                   "Network Number: %02X%02X%02X%02X"\
                   "Node Number: %02X%02X%02X%02X%02X%02X\n"\
                   "sent a message:\n%s",
                   Source->network[0], Source->network[1],
                   Source->network[2], Source->network[3],
                   Source->node[0], Source->node[1], Source->node[2],
                   Source->node[3], Source->node[4], Source->node[5],
                   pecb->fragmentDescriptor[1].address
                 );
          // Resubmit the ECB to listen to incoming packets
          SPXListenForSequencedPacket (SPXTaskID, pecb);
     }
    // Display the message (this will only display for small messages)
    InvalidateRect (hWnd, NULL, TRUE);
    // Clear out data that has arrived
    lpszData = (LPSTR)pecb->fragmentDescriptor[1].address;
    *lpszData = '\0';

    return (iRc);
}
////////////////////////////////////////////////////////////////////////
// FUNCTION: SendSPXPacket(DWORD, WORD )
//
// PURPOSE:  Send an SPX packet
//
// COMMENTS: We send one SPX packet. Caller must have created a local
//           socket and an SPX connection already
//
////////////////////////////////////////////////////////////////////////
int SendSPXPacket ( DWORD SPXTaskID, WORD socket)
{
    int           i, iRc;
    PECB     pecb = NULL;
    PSPXHeader  pspxHead;

    // Find a free send ECB
    for (i = MAX_RCV_ECBS; i <= MAX_ECBS; i++)
    {
        if (apmyECB [ i ]->ECB.inUseFlag == 0)
        {
              pecb = (PECB) &apmyECB [i]->ECB;
              // Message sent by ESR when send completes
              apmyECB [i]->wMsg = WM_MSG_SENT;

              // Set a fixed string as data for SPX
              pecb->fragmentDescriptor[1].size = strlen(SPX_DEMO_STRING);
              strcpy ( aDataBuffer [i], SPX_DEMO_STRING);
              break;
        }
    }
    if (pecb == NULL)
    {
        DISPLAY_ERROR_MSG ("No Free Send ECBs", 0);
```

```
        return 1;
    }

    // The following are initialized just to be safe
    pspxHead = (PSPXHeader)(pecb->fragmentDescriptor[0].address);
    pspxHead->connectionControl     = 0;
    pspxHead->dataStreamType        = 0;

    // Submit a send ECB
    SPXSendSequencedPacket (SPXTaskID, SPXConnID, pecb);

    return ESUCCESS;
}
////////////////////////////////////////////////////////////////////////
//
// FUNCTION: TerminateSPXConnection(DWORD)
//
// PURPOSE:  Terminate a Connection
//
// COMMENTS: A connection can be broken gracefully by calling
//           SPXTerminateConnection() API. This API can be called
//           on successful completion of SPXEstablishConnection() or
//           SPXListenForConnection() APIs whether the connection
//           with remote end has occurred or not.
////////////////////////////////////////////////////////////////////////
int TerminateSPXConnection ( DWORD SPXTaskID)
{
    int iRc, i;
    PECB            pecb = NULL;
    PSPXHeader      pspxHead = NULL;
    // Find a free send ECB
    for (i = MAX_RCV_ECBS; i <= MAX_ECBS; i++)
    {
        if (apmyECB [ i ]->ECB.inUseFlag == 0)
        {
              pecb = (PECB) &apmyECB [ i ]->ECB;
              // Message sent by ESR when connection terminates
              apmyECB [ i ]->wMsg = WM_CONN_TERMINATED;
              break;
         }
    }
    if (pecb == NULL)
    {
          DISPLAY_ERROR_MSG ("No Free Send ECBs", 0);
          return 1;
    }
    // We must set the fragment count to 1 and first fragment
    // descriptor is already pointing to a buffer of size
    // large enough to hold SPXHeader
    pecb->fragmentCount = 1;
    pspxHead = (PSPXHeader)pecb->fragmentDescriptor[0].address;

    // The following are initialized just to be safe
    pspxHead->connectionControl     = 0;
    pspxHead->dataStreamType        = 0; // SPX will set this to FEh
```

```
    // Post the termination request
    SPXTerminateConnection (SPXTaskID, SPXConnID, pecb);

    return ESUCCESS;
}
//////////////////////////////////////////////////////////////////////////
//
// FUNCTION: SetupSPXConnection(DWORD, WORD, BOOL )
//
// PURPOSE:  Set up a Connection
//
// COMMENTS: Calls SPXListenForConnection() or SPXEstablishConnection() API
//
//////////////////////////////////////////////////////////////////////////
int SetupSPXConnection (DWORD SPXTaskID, WORD socket, BOOL fListen)
{
    int iRc, i;
    PECB            pecb = NULL;
    PSPXHeader      pspxHead = NULL;
    PIPXAddress     Dest;
    // Find a send ECB
    for (i = MAX_RCV_ECBS; i <= MAX_ECBS; i++)
    {
        if (apmyECB [ i ]->ECB.inUseFlag == 0)
        {
                pecb = (PECB) &apmyECB [i]->ECB;
                // Message sent by ESR when connection terminates
                apmyECB [i]->wMsg = WM_CONN_SETUP;
                break;
        }
    }
    if (pecb == NULL)
    {
        DISPLAY_ERROR_MSG ("No Free Send ECBs", 0);
        return 1;
    }
    // Set the fragment count to 1; first fragment descriptor is already
    // pointing to a buffer of size large enough to hold SPXHeader
    pecb->fragmentCount = 1;
    pspxHead = (PSPXHeader)pecb->fragmentDescriptor[0].address;
    // The following are initialized just to be safe
    pspxHead->connectionControl     = 0;
    pspxHead->dataStreamType        = 0;

    // add socket number
    pecb->socketNumber = socket;
    // Set receiver's address if this is the 'Client'
    Dest = (PIPXAddress)&(pspxHead->destination);
    memcpy(Dest->node,    Receiver.node, NODE_SIZE);
    memcpy(Dest->network, Receiver.network, NETWORK_SIZE);
    *(WORD *)&(Dest->socket) = IPX_DEMO_SOCKET;

    // Post the request
    if (fListen)
        iRc = SPXListenForConnection ( SPXTaskID,
                                       0, // default retryCount
```

```
                                        0, // no watchdog
                                           pecb);
    else
        iRc = SPXEstablishConnection ( SPXTaskID,
                                           0, // default retryCount
                                           0, // no watchdog
                                           &SPXConnID, // Connection ID
                                           pecb);
    return (iRc);
}
```

9.5 SUMMARY

The SPX/IPX programming interfaces available in Novell NetWare environments can be used to build a client/server communication conduit.

Although Novell has not yet released the final version of the IPX/SPX interface libraries and DLLs for the Windows NT NetWare client, the early releases indicate that the IPX/SPX interfaces have not changed. Of course, there may be substantial changes before the final release—or none at all.

It is also possible that Novell will remove the IPX/SPX APIs and expose them via Windows Sockets. The Microsoft-supplied SPX/IPX protocol (NWLink) already works with Windows Sockets. Thus, such a move (by Novell) will not be surprising, if the final direction is to use SPX/IPX services via the Windows Socket interfaces.

Additionally, it may be that Win32 applications will use the name service provider interfaces to locate remote servers and services irrespective of the provider (Windows NT servers, NetWare servers, and so on). The advantage of this approach is that the applications are not bound to a provider-specific API (for example, SAP APIs).

Appendix A

DETERMINING NETBIOS SYSTEM CHARACTERISTICS

This appendix contains some programs illustrating how to use the

- NCB.ENUM
- NCB.ADAPTER.STATUS
- NCB.SESSION.STATUS

commands, which are helpful in determining system configuration and for debugging NetBIOS-based applications.

In Windows NT, the Netbios() API exposes the NCB.ENUM command, which enumerates the existing LANAs in a workstation. Recall that each protocol/adapter configuration is usually associated with a LANA number. Thus, a system configured with the NetBEUI protocol and two network cards will have two LANAs (0 and 1). The NCB.ENUM command returns a structure enumerating the valid LANAs on a Windows NT workstation. The following sample program illustrates how to use the NCB.ENUM command.

```
// FILE: %BOOK%\netbios\lanaenum.c
#include <windows.h>
#include <stdio.h>
// LANAENUM structure extracted from NB30.h
typedef struct _LANA_ENUM
{ UCHAR     length;     //   Number of valid entries in lana[]
  UCHAR     lana[MAX_LANA+1];
} LANA_ENUM, *PLANA_ENUM;
// Enumerate the valid LANAs on a workstation
BYTE EnumLana ()
{
    BYTE   bRc;
    NCB     ncb;
    LANA_ENUM leEnumBuffer;
    int         i;
    // Clean out the NCB
    memset (&ncb, 0x00, sizeof(NCB));
    // Call NCB.ENUM
    ncb.ncb_command = NCBENUM;
    // Set the ptr to LANA_ENUM structure in ncb_buffer
    ncb.ncb_buffer = (PUCHAR) &leEnumBuffer;
    ncb.ncb_length = sizeof(leEnumBuffer);
```

```
    if ((bRc = Netbios (&ncb)) != NRC_GOODRET)
        printf("NCB Enum Failed: %x\n", bRc);
    else
    {
        printf("Enum Length: %d\n", leEnumBuffer.length );
        for (i= 0; i< leEnumBuffer.length; i++)
             printf ("Index: %d LANA: %d\n", i, leEnumBuffer.lana[i]);
    }
    return(bRc);
}
//   Main: Demonstrates usage of NCB.ENUM command
//   Usage - lanaenum
int _cdecl main (int argc, char ** argv)
{
    EnumLana ();
    return 0;
}
```

Another of the interesting NetBIOS commands is NCB.ADAPTER.STATUS, which shows the physical adapter characteristics for each LANA. When calling this command, the caller supplies the LANA number (via the ncb_lana_num field) and a NetBIOS name (via the ncb_callname field). The name can be a workstation name, a group name, or a unique name. When an asterisk "*" is specified in the ncb_callname field, the command returns information about the local adapter present at the LANA specified.

The next code fragment lists all the structures returned by the NCB.ADAPTER.STATUS command. On successful completion of the command, the buffer is filled with an ADAPTER_STATUS structure, which is immediately followed by NAME_BUFFER structures. The number of NAME_BUFFER structures is defined by the name_count field of the ADAPTER_STATUS structure. Each NAME_BUFFER structure contains details of names registered with the adapter on which the command NCB.ADAPTER.STATUS. The name_flags field sheds further light on each name registered with NetBIOS (for the given LANA). The name_flags field can have a combination of the following flags:

GROUP_NAME	Group name
UNIQUE_NAME	Unique name
REGISTERING	The name is being registered
REGISTERED	The name is fully registered
DEREGISTERED	The name is being deregistered
DUPLICATE	Duplicate name
DUPLICATE_DEREG	Duplicate name being deregistered

```
// Structure returned to the NCB command NCBASTAT is
// ADAPTER_STATUS followed
// by an array of NAME_BUFFER structures.
typedef struct _ADAPTER_STATUS {
    UCHAR       adapter_address[6];
    UCHAR       rev_major;
```

```
    UCHAR       reserved0;
    UCHAR       adapter_type;
    UCHAR       rev_minor;
    WORD        duration;
    WORD        frmr_recv;
    WORD        frmr_xmit;
    WORD        iframe_recv_err;
    WORD        xmit_aborts;
    DWORD       xmit_success;
    DWORD       recv_success;
    WORD        iframe_xmit_err;
    WORD        recv_buff_unavail;
    WORD        t1_timeouts;
    WORD        ti_timeouts;
    DWORD       reserved1;
    WORD        free_ncbs;
    WORD        max_cfg_ncbs;
    WORD        max_ncbs;
    WORD        xmit_buf_unavail;
    WORD        max_dgram_size;
    WORD        pending_sess;
    WORD        max_cfg_sess;
    WORD        max_sess;
    WORD        max_sess_pkt_size;
    WORD        name_count;
} ADAPTER_STATUS, *PADAPTER_STATUS;
typedef struct _NAME_BUFFER {
    UCHAR       name[NCBNAMSZ];
    UCHAR       name_num;
    UCHAR       name_flags;
} NAME_BUFFER, *PNAME_BUFFER;
#define NAME_FLAGS_MASK     0x87

#define GROUP_NAME          0x80
#define UNIQUE_NAME         0x00

#define REGISTERING         0x00
#define REGISTERED          0x04
#define DEREGISTERED        0x05
#define DUPLICATE           0x06
#define DUPLICATE_DEREG     0x07
```

The next program example shows how to use the command NCB.ADAPTER.STATUS. Notice that the PrintAdapterBuffer() command parses the output returned by the NCB.ADAPTER.STATUS command. It illustrates the meaning of the fields on the ADAPTER_STATUS structure.

```
// FILE: %BOOK%\netbios\adapstat.c
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
static  BYTE bLanaNum = 0;
void PrintAdapterBuffer ( UCHAR * pBuffer)
{
    ADAPTER_STATUS   *pas;
    NAME_BUFFER      *pnb;
```

```
WORD   i,   Count,   j;
char   achNameTemplate [ NCBNAMSZ *2 ] = "\tName";

// Print Adapter Status field
printf("Adapter Status Header\n");
pas = (ADAPTER_STATUS *) pBuffer;
printf("Adapter Address:          ");
for (i = 0; i < 6 ;i++ )
    printf("%2.2x", pas->adapter_address[i]);
printf("\n");
printf("Rev major:                    %2.2x\n", pas->rev_major);
printf("Adapter type:                 %2.2x\n", pas->adapter_type);
printf("Rev Minor:                    %2.2x\n", pas->rev_minor);

printf("Duration                      %4.4x\n", pas->duration);
printf("Frames received:              %4.4x\n", pas->frmr_recv);
printf("Frames transmitted:           %4.4x\n", pas->frmr_xmit);
printf("Frames Receive Error:         %4.4x\n", pas->iframe_recv_err);
printf("Transmit Aborts:              %4.4x\n", pas->xmit_aborts);

printf("Tranmit Success:              %8.8x\n", pas->xmit_success);
printf("Receive Success:              %8.8x\n", pas->recv_success);

printf("Frame Transmit Error:         %4.4x\n", pas->iframe_xmit_err);
printf("Receive Buffer Unavail:       %4.4x\n", pas->recv_buff_unavail);
printf("Tl Timeout:                   %4.4x\n", pas->tl_timeouts);
printf("Ti Timeout:                   %4.4x\n", pas->ti_timeouts);

printf("Free NCBs:                    %4.4x\n", pas->free_ncbs);
printf("Max Configured NCBs:          %4.4x\n", pas->max_cfg_ncbs);
printf("Max NCBs:                     %4.4x\n", pas->max_ncbs);
printf("Transmit Buffer Unavail:      %4.4x\n", pas->xmit_buf_unavail);
printf("Max Datagram Size:            %4.4x\n", pas->max_dgram_size)
printf("Max pending sessions:         %4.4x\n", pas->pending_sess);
printf("Max Configured Sessions:      %4.4x\n", pas->max_cfg_sess);
printf("Max Session:                  %4.4x\n", pas->max_sess);
printf("Max Sess Pkt Size:            %4.4x\n", pas->max_sess_pkt_size);

printf("\nName Count: %x\n", Count = pas->name_count);
// Print Name Buffer Information
pnb = (NAME_BUFFER *) ((PCHAR)pBuffer + sizeof(ADAPTER_STATUS));

printf("%s\t\t\tName Number\tName Type\n", achNameTemplate);
for (i = 0; i< Count; i++)
{
    for (j = 0 ; j< NCBNAMSZ ; j++)
    {
          if (isalnum(pnb->name[j])       | |
               (pnb->name[j] == 0x20)     | |
               (pnb->name[j] == 0x2d))
               printf("%c", pnb->name[j]);
          else
               printf("%2.2x", pnb->name[j]);
      }
    printf("\t\t%x\t", pnb->name_num);
    printf("\t%s", (pnb->name_flags & GROUP_NAME)?"Group ":"Unique");
    switch (pnb->name_flags & (BYTE)0x0F)
```

```
        {
        case REGISTERING:       printf(" Registering\n"); break;
        case REGISTERED:        printf(" Registered\n");  break;
        case DEREGISTERED:      printf(" DeRegistered\n");break;
        case DUPLICATE:         printf(" Duplicate\n");   break;
        case DUPLICATE_DEREG:   printf(" Duplicate Deregistered\n"); break;
        }
        pnb++;
    }
}
#define NOS_MAX_NAMES 20
static UCHAR aucNcbBuffer [ sizeof(ADAPTER_STATUS)
                          + NOS_MAX_NAMES * sizeof(NAME_BUFFER)];
BYTE AdapterStatus ( PCHAR pchName )
{
        BYTE     bRc;
        NCB      ncb;
        printf("Adapter Status of Lan Adapter: %d At: %s \n",
            bLanaNum, pchName);
        // Clean the NCB
        memset (&ncb, 0x00, sizeof(NCB));
        // call NCB.ADAPTER.STATUS
        ncb.ncb_command = NCBASTAT;
        // Adapter to use
        ncb.ncb_lana_num = bLanaNum;
        // Local or remote computer name whose
        // adapter status is to be determined
        memset (ncb.ncb_callname, 0x20, NCBNAMSZ);
        memcpy (ncb.ncb_callname, pchName, strlen(pchName));

        // Specify the buffer to collect information.
        ncb.ncb_buffer = aucNcbBuffer;
        ncb.ncb_length = sizeof(aucNcbBuffer);

        // Submit a synchronous command
        bRc = Netbios ( &ncb );
        switch (bRc)
        {
        case NRC_GOODRET: printf("Adapter Status successful\n");
                          PrintAdapterBuffer ( aucNcbBuffer );
                          break;
        case NRC_CMDTMO:
                  printf("Command timed out. Maybe no such name exists\n");
                  break;
        case NRC_BUFLEN:  printf("Illegal buffer length.\n");
                  break;
        case NRC_BRIDGE:  printf("Invalid Lan Adapter Number.\n");
                  break;
        default: printf("Adapter Status Error: %x\n", bRc);
        }
        return(bRc);
}
// Main - Locates presence of a NetBIOS name and prints addressing info
// Usage - adapstat <computer name> <lana>
int _cdecl main (int argc, char **argv)
{
    BYTE   bRc;
    char      achName [ NCBNAMSZ + 1] = "*"; // Default is Local Case
```

```
    argc--;
    argv++;
    if (argc)
    {
        sprintf(achName, "%s", argv[0]);
        argc--;argv++;
        if (argc)
              bLanaNum = (BYTE) atoi(argv[0]);
    }
    if ((Reset() == NRC_GOODRET))
    {
              bRc = AdapterStatus  ( achName );
    }
    return 0;
}
```

Following is the result of the program run on a Windows NT workstation (named ALOKSDOS). Note that the physical address of the adapter on LANA 1 is 0x00DD010F394A, and the workstation has 15 names registered. ALOKS-DOMAIN is the Windows NT domain name.

```
// Output from the adapstat program on ALOKSDOS
Adapter Status of Lan Adapter: 1 At: ALOKSDOS
Adapter Status successful
Adapter Status Header
Adapter Address:                00dd010f394a
Rev major:                      03
Adapter type:                   fe
Rev Minor:                      00
Duration                        0000
Frames received:                0000
Frames transmitted:             0000
Frames Receive Error:           0004
Transmit Aborts:                0000
Transmit Success:               00002674
Receive Success:                0000274b
Frame Transmit Error:           003c
Receive Buffer Unavail:         720d
T1 Timeout:                     015f
Ti Timeout:                     e7a7
Free NCBs:                      ffff
Max Configured NCBs:            ffff
Max NCBs:                       ffff
Transmit Buffer Unavail:        0000
Max Datagram Size:              05ad
Max pending sessions:           0008
Max Configured Sessions:        ffff
Max Session:                    ffff
Max Sess Pkt Size:              05ca

Name Count: f
    Name                            Name Number     Name Type
ALOKSDOS            1f              1               Unique Registered
ALOKSDOS                            2               Unique Registered
ALOKSDOS            00              3               Unique Registered
ALOKS-DOMAIN        00              4               Group  Registered
```

```
ALOKS-DOMAIN          1c          5          Group  Registered
ALOKSDOS              01          6          Unique Registered
ALOKSDOS              03          7          Unique Registered
ALOKS-DOMAIN          1b          8          Unique Registered
ALOKS-DOMAIN          1e          9          Group  Registered
ALOKSDOS              87          a          Unique Registered
ALOKS-DOMAIN          1d          b          Unique Registered
01025f5fMSBROWSE5f5f0201          c          Group  Registered
ALOKSDOS              21          d          Unique Registered
ALOKSDOS              22          e          Unique Registered
ALOKSDOS              23          f          Unique Registered
```

Another interesting NetBIOS command that can be used for debugging is NCB.SESSION.STATUS. It determines the status of each session registered on a workstation. The caller supplies the LANA number (via ncb_lana_num), a NetBIOS name (via ncb_name), and a data buffer (via ncb_buffer and ncb_length). When an asterisk is specified as the name, the command returns the information on all the sessions associated with the process. On the other hand, the caller can specify a name (for example, Server1), and the command returns the session's status for that name. This option is useful in determining the status of active sessions of a NetBIOS-based server. Readers familiar with this command on other platforms will notice that NCB.SESSION.STATUS gives only per-process information, not per-workstation information. In other words, a user cannot determine the status of sessions associated with other processes on the Windows NT workstation. For the latter purpose, you can use the nbtstat program available in the Windows NT Resource Kit.

```
// Structure returned to the NCB command NCBSSTAT is SESSION_HEADER followed
// by array of SESSION_BUFFER structures. If NCB_NAME starts with asterisk,
// then array of these structures is returned with status for all names.
typedef struct _SESSION_HEADER {
    UCHAR   sess_name;
    UCHAR   num_sess;
    UCHAR   rcv_dg_outstanding;
    UCHAR   rcv_any_outstanding;
} SESSION_HEADER, *PSESSION_HEADER;

typedef struct _SESSION_BUFFER {
    UCHAR   lsn;
    UCHAR   state;
    UCHAR   local_name[NCBNAMSZ];
    UCHAR   remote_name[NCBNAMSZ];
    UCHAR   rcvs_outstanding;
    UCHAR   sends_outstanding;
} SESSION_BUFFER, *PSESSION_BUFFER;

//   Values for state

#define LISTEN_OUTSTANDING      0x01
#define CALL_PENDING            0x02
#define SESSION_ESTABLISHED     0x03
#define HANGUP_PENDING          0x04
```

```
#define HANGUP_COMPLETE          0x05
#define SESSION_ABORTED          0x06
```

The following code illustrates the NCB.SESSION.STATUS command.

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>

// Create a blank padded name
void CopyName ( PCHAR pszTo, PCHAR pszFrom)
{
    int len = strlen(pszFrom);
    // First set space in the target name
    memset ( pszTo, 0x20, NCBNAMSZ);
    // now only copy the characters
    memcpy ( pszTo, pszFrom, len);
}
#define MAX_SESSIONS    12
#define MAX_NAMES       12
static  BYTE bLanaNum = 0;
// Reset the adapter
BYTE Reset ()
{
    BYTE    bRc;
    NCB     ncb;
    // Clean out the NCB
    memset (&ncb, 0x00, sizeof(NCB));
    // Call NCB.RESET
    ncb.ncb_command = NCBRESET;
    // Reset LAN Adapter
    ncb.ncb_lana_num   = bLanaNum;

    // Free all resources
    ncb.ncb_lsn = 0;
    // Set Max Sessions and Names
    ncb.ncb_callname[0] = MAX_SESSIONS;
    ncb.ncb_callname[2] = MAX_NAMES;

    if ((bRc = Netbios (&ncb)) != NRC_GOODRET)
        printf("Reset Failed: %x\n", bRc);

    return(bRc);
}
void PrintNetBiosName(UCHAR* pName)
{
    int j;
    for (j = 0 ; j< NCBNAMSZ ; j++)
    {
        if (isalnum(pName[j])   ||
                    (pName[j] == 0x20)  ||
                    (pName[j] == 0x2d))
                    printf("%c", pName[j]);
                else
                    printf("%2.2x", pName[j]);
    }
    printf("\n");
}
```

```
void PrintSessionStatus ( UCHAR * pBuffer)
{
    PSESSION_HEADER  psh;
    PSESSION_BUFFER  psb;
        UCHAR i;
    // Print Session Header
    printf("Session Status Header\n");
    psh = (PSESSION_HEADER)pBuffer;

    printf("session name number         0x%d\n", psh->sess_name);
    printf("number of sessions          0x%d\n", psh->num_sess);
    printf("Recv datagram outstanding   0x%d\n", psh->rcv_dg_outstanding);
    printf("Recv any outstanding        0x%d\n", psh->rcv_dg_outstanding);
    printf("\nSessions \n");

    // Now print the session buffers
    psb = (PSESSION_BUFFER) ((LPSTR)pBuffer + sizeof(SESSION_HEADER ));
    for (i = 0; i <psh->num_sess; i++)
    {
        printf("Session Number: 0x%x\n",    psb->lsn);
        switch ( psb->state)
        {
        case LISTEN_OUTSTANDING:
               printf("State: Listen  Oustanding\n"); break;
        case CALL_PENDING:
               printf("State: Call Pending\n"); break;
        case SESSION_ESTABLISHED:
               printf("State: Session Established\n"); break;
        case HANGUP_PENDING:
               printf("State: Hangup  Pending\n"); break;
        case HANGUP_COMPLETE:
               printf("State: Hangup  Complete\n"); break;
        case SESSION_ABORTED:
               printf("State: Session Aborted\n"); break;
        }
        printf("Local  Name :");
        PrintNetBiosName ( psb->local_name);
        printf("Remote Name :");
        PrintNetBiosName ( psb->remote_name);
        printf("Receive outstanding: %d\n", psb->rcvs_outstanding);
        printf("Send outstanding: %d\n", psb->sends_outstanding);
        printf("\n");
        // Go to next session buffer
        psb++;
    }
}
#define NOS_MAX_NAMES 20
static UCHAR aucNcbBuffer [ sizeof(SESSION_HEADER)
                          + NOS_MAX_NAMES * sizeof(SESSION_BUFFER)];
BYTE SessionStatus ( PCHAR pchName )
{
        BYTE   bRc;
        NCB    ncb;
        printf("Session Status of Lan Adapter: %d Name: %s \n",
            bLanaNum, pchName);
        memset (&ncb, 0x00, sizeof(NCB)); // Clean the NCB
        ncb.ncb_command = NCBSSTAT;       // call NCB.SESSION.STATUS
```

```
        ncb.ncb_lana_num = bLanaNum;       // Adapter to use

        // Name for which session status is to be determined
        memset (ncb.ncb_name, 0x20, NCBNAMSZ);
        memcpy (ncb.ncb_name, pchName, strlen(pchName));

        // Specify the buffer to collect information.
        ncb.ncb_buffer = aucNcbBuffer;
        ncb.ncb_length = sizeof(aucNcbBuffer);

        bRc = Netbios ( &ncb ); // Submit a synchronous command
        switch (bRc)
        {
        case NRC_GOODRET: printf("Session Status sucessful\n");
             PrintSessionStatus ( aucNcbBuffer );
             break;
        case NRC_CMDTMO:
             printf("Command timed out. Maybe no such name exists\n");
             break;
        case NRC_BUFLEN:  printf("Illegal buffer length.\n");
             break;
        case NRC_BRIDGE:  printf("Invalid Lan Adapter Number.\n");
             break;
        default: printf("Adapter Status Error: %x\n", bRc);
        }
    return(bRc);
}
// Prints session status
// Usage: sesstat <name> <lana>
int _cdecl main (int argc, char **argv)
{
    BYTE   bRc;
    char   achName [ NCBNAMSZ + 1] = "*"; // Default is all session
    argc--;
    argv++;
    if (argc)
    {
        CopyName(achName, argv[0]);
        argc--;argv++;
        if (argc)
               bLanaNum = (UCHAR) atoi(argv[0]);
    }
    if ((Reset() == NRC_GOODRET))
    {
        bRc = SessionStatus  ( achName );
    }
    return 0;
}
```

Appendix B

MULTICASTING WITH WINDOWS SOCKETS

In this appendix, we demonstrate the multicasting capability of **Windows Sockets (WinSock)** programming interfaces. Multicasting is useful for applications that involve participation by multiple users. Examples are multiplayer games, video conferencing, live concerts broadcast over the Internet, and so on. In the last couple of years, multicasting has become quite popular on the Internet.

Recall from our earlier discussions that in multicasting, an entire group of active nodes can communicate with each other by means of datagrams. A nonmember can send a message to all the members of the group but cannot receive a message that is directed to the membership. Each member formally joins or leaves a group, so that, at any time, the membership is explicitly known.

Windows Sockets allows applications to join or leave a multicast group and send messages to each other. When a member sends a multicast message, WinSock makes a best-effort attempt to send the message to all group members. However, it does not guarantee data receipt by all group members. Similarly, there is no guarantee that the order of the messages will be maintained. WinSock multicasting is based on the Multicast 1.2 Release [see Deering 89[1]]. Note that the multicasting capability of WinSock completely depends on the multicasting facility of the underlying network drivers. In other words, it does not attempt to provide multicasting on networks that do not support such a feature.

Multicast datagrams are similiar to the broadcast datagrams (see Chapter 8). However, multicast datagrams are only received by hosts that have expressed interest in receiving them. In addition, if a network's routers are configured to route multicast packets, the packets are sent beyond the current subnet. In other words, the multicast message can traverse a WAN, provided the intermediate routers are properly configured.

1. You can obtain the most recent complete version of that document via anonymous ftp to `gregorio.stanford.edu, directory: /vmtp-ip`.

B.1 JOINING AND LEAVING MULTICAST GROUPS

In order to use multicasting, the socket must have opened using the UDP protocol (that is, the SOCK_DGRAM option) and must have selected the AF_INET family. As soon as the socket has been opened and bound, the application uses the setsockopt() API to add itself to the multicast group. This process is shown in the following code fragment:

```
// Prototype of ip_mreq structure
struct ip_mreq {
struct in_addr  imr_multiaddr;  // IP multicast address of group
struct in_addr  imr_interface;  // local IP address of interface
};
// Prototype of setsockopt() API
int PASCAL FAR
setsockopt (SOCKET s, int level, int optname,
           const char FAR * optval, int optlen);
//
// Pseudo Code - Shows joining and leaving muticast group
//
// Joining a multicast group
ip_mreq mpreq;
char chMCastAddress [ 255 ] = "233.1.1.1";
char chLocalAddress [ 255 ] = "11.1.12.128";
mreq.imr_multiaddr.s_addr = inet_addr(chMCastAddress);
mreq.imr_interface.s_addr = inet_addr (chLocalAddress);
if (setsockopt(sock,
               IPPROTO_IP,
               IP_ADD_MEMBERSHIP,
               (char *) &mreq,
               sizeof(struct ip_mreq)) == SOCKET_ERROR)
{
    printf("\nError %d in joining Multicast Group",
           WSAGetLastError());
    exit(1);
}
// Dropping membership from the multicast group
setsockopt(sock, IPPROTO_IP, IP_DROP_MEMBERSHIP,
          (char *) &mreq, sizeof(mreq));
```

As shown, the application fills in the *imr_interface* field of the *ip_mreq* structure with the local IP address and the *imr_multiaddr* field with the multicast IP address. The multicast IP can be any IP address from 224.0.0.0 to 239.255.255.255. The prepared *ip_mreq* structure is submitted to WinSock via the setsockopt() API with *optname* set to IP_ADD_MEMBERSHIP.

Every membership is associated with a single network interface, and it is possible to join the same group on more than one interface. For example, when a workstation has two network cards (each connected to a distinct subnet), it has two network addresses. In such a case, a multicast group in one subnet is not visible to the same-named multicast group in the second

subnet. Thus, the application will need to annouce its membership on each subnet by calling setsockopt() multiple times (once for each IP address). Alternatively, you can set the *imr_interface* to INADDR_ANY, to choose the default multicast interface, or one of the host's local addresses, to choose a particular (multicast-capable) interface.

To drop a membership, use the IP_DROP_MEMBERSHIP option. The memberships associated with a socket are also dropped when the socket is closed or when the process holding the socket is killed. However, on a workstation, more than one socket may claim a membership in a particular group. In this case, the workstation will remain a member of that group until the last claim is dropped. The memberships associated with a socket do not necessarily determine which datagrams are received on that socket. Incoming multicast packets are accepted by the kernel IP layer if any socket has claimed a membership in the destination group of the datagram; however, delivery of a multicast datagram to a particular socket is based on the destination port, just as with unicast datagrams. To receive multicast datagrams sent to a particular port, it is necessary to bind to that local port, leaving the local address unspecified (that is, INADDR_ANY).

More than one process may bind to the same SOCK_DGRAM UDP port if the bind is preceded by call to setsockopt() with SO_REUSEADDR option as shown here:

```
int one = 1;
setsockopt(sock, SOL_SOCKET, SO_REUSEADDR, &one, sizeof(one))
```

In this case, every incoming multicast or broadcast UDP datagram destined to the shared port is delivered to all sockets bound to the port. For backwards-compatibility reasons, this rule does not apply to incoming unicast datagrams—unicast datagrams are never delivered to more than one socket, no matter how many sockets are bound to the datagram's destination port.

B.2 RECEIVING MULTICAST MESSAGES

An application can receive multicast datagrams targeted toward a group after it has associated its UDP socket with the group. The messages can be received by calling the recvfrom() API, whose prototype is shown here:

```
int PASCAL FAR
recvfrom (SOCKET s, char FAR * buf, int len, int flags,
          struct sockaddr FAR *from, int FAR * fromlen);
```

On successful completion of the recvfrom() API, the caller receives the multicast message (via the *buf* parameter) as well as the address of the sender (via the *from* parameter). When the application leaves the multicast group, it can no longer receive the multicast messages directed to the group.

B.3 SENDING MULTICAST MESSAGES

To send a multicast datagram, the caller specifies an IP multicast address in the range from 224.0.0.0 to 239.255.255.255 as the destination address in a sendto() call. (A prototype is shown in the next example.) The caller does not have to be a member of the multicast group.

```
int PASCAL FAR
sendto (SOCKET s, const char FAR * buf, int len, int flags,
        const struct sockaddr FAR *to, int tolen);
```

By default, IP multicast datagrams are sent with a time-to-live (TTL) of 1, which prevents them from being forwarded beyond a single subnetwork. The IP_MULTCAST_TTL socket option allows the TTL for subsequent multicast datagrams to be set to any value from 0 to 255, in order to control the scope of the multicasts, as shown here:

```
u_char ttl;
setsockopt(sock, IPPROTO_IP, IP_MULTICAST_TTL, &ttl, sizeof(ttl))
```

Multicast datagrams with a TTL of 0 will not be transmitted on any subnet, but they can be delivered locally if the sending workstation belongs to the destination group [MS-WinSock 94]. Multicast datagrams with TTL greater than one may be delivered to more than one subnet if one or more multicast routers are attached to the first-hop subnet. To provide meaningful scope control, the multicast routers support the notion of TTL "thresholds," which prevent datagrams with less than a certain TTL from traversing certain subnets. The thresholds enforce a convention shown in Table B-1.

Table B-1. *Effect of TTL on Routing of Multicast Datagrams*

Initial TTL	*Restricted to*
0	Same workstation
1	Same subnet
32	Same site*
64	Same region*
128	same continent
255	unlimited

* "Sites" and "regions" are not strictly defined, and sites may be further subdivided into smaller administrative units, as a local matter.

An application may choose an initial TTL other than the ones listed in Table B-1. For example, an application might perform an "expanding-ring search" for a network resource by sending a multicast query, first with a TTL of 0, and then with larger and larger TTLs, until a reply is received, perhaps using the TTL sequence 0, 1, 2, 4, 8, 16, 32 [MS-WinSock 94].

Note: Many multicast routers refuse to forward any multicast datagram with a destination address between 224.0.0.0 and 224.0.0.255, inclusive, regardless of its TTL. This range of addresses is reserved for the use of routing protocols and other low-level topology discovery or maintenance protocols, such as gateway discovery and group membership reporting. The current specification for IP multicasting requires this behavior only for addresses 224.0.0.0 and 224.0.0.1; the next revision of the specification is expected to contain this more general restriction.

Each multicast message is sent from a single network interface, even if the workstation has more than one multicast-capable interface. In this case, the system usually selects one of the interfaces to be the default multicast interface. How is the default interface selected? Selection is controlled via the kernel (unicast) routing table. If there is no multicast route in the table, all multicasts will, by default, be sent on the interface associated with the default gateway. If that interface does not support multicast, attempts to send will receive a WSAENETUNREACH error. A route may be added for a particular multicast address or for all multicast addresses, to direct them to a different default interface. For example, to specify that multicast datagrams addressed to 233.1.1.1 should, by default, be sent on the interface with local address 11.1.12.128, use the following command:

```
route add 233.1.1.1  11.1.12.128
```

To set the default for all multicast addresses other than those with individual routes to be the interface with local address 11.1.12.128, use:

```
route add 224.0.0.0 11.1.12.128
```

When you point a multicast route at an interface that does not support multicasting, any attempt to multicast via that route will generate an WSAENETUNREACH error.

An application can choose to override the default interface by using the IP_MULTICAST_IF socket option. In this case, the caller supplies an interface address to which all subsequent multicasts are sent. (You can use an address of INADDR_ANY to revert to the default interface.) This situation is illustrated by the following code fragment, in which a user-supplied address is set as the multicast interface.

```
struct  in_addr MCastAddr;
char    chLocalAddress[16];

printf ("Enter Local IP Address: ");
scanf ("%s",chLocalAddress);
printf("\nAddress to be used as Multicast Interface: %s\n",
chLocalAddress);
MCastAddr.s_addr = inet_addr(chLocalAddress);
if (setsockopt(sock, IPPROTO_IP,
               IP_MULTICAST_IF,
               (char *) &MCastAddr,
               sizeof(struct in_addr))==SOCKET_ERROR)
```

```
{
    printf("\nWarning: Could not set Default MCast Interface "\
                "%d", WSAGetLastError());
}
```

B.4 SAMPLE PROGRAM

We now illustrate the usage of multicasting with an example program. The program can be invoked as the server (using the -s option) or as a client (using the -c option). The server joins the multicast group using the IP address of (233.1.1.1). It then waits on the recvfrom() API to receive multicast messages. On the other hand, the client picks a multicast interface (by using the IP_MULTICAST_IF option) and then sends messages to the multicast group using the sendto() API.

```
//////////////////////////////////////////////////////////////////////////
//
// File: See %BOOK%\winsock\multicst\multicst.c
// Program : MultiCst.c
//
//    IP Multicast demo program showing usage of IP Multicast
//    on Microsoft TCP/IP.
//
//
// History: Based on a sample program made available by Microsoft Corp.
//
//////////////////////////////////////////////////////////////////////////
#include <windows.h>
#include <stdio.h>
#include <winsock.h>
#include <stdlib.h>
#include <string.h>

#define  MULTICAST_PORT    9999
#define  MULTICAST_ADDRESS "233.1.1.1"
#define  MAX_BUFFER_SIZE   300
#define  MAX_LINE_SIZE     250

VOID usage()
{
    printf("\nUsage: multicst -s | -c ");
    printf("\n-s for Server and -c for Client ");
}
int main(int argc, char *argv[])
{
    int      iRet = 0;
    WSADATA  wsaData;
    SOCKET   sock;
    struct   sockaddr_in sock_addr, dst_sock_addr, new_sock_addr;
    int      new_sock_addr_len;
    long     bytes;
    char     chBuffer [ MAX_BUFFER_SIZE ];
    char     chLine   [ MAX_LINE_SIZE   ];
    struct   ip_mreq mreq;
```

```
struct  in_addr MCastAddr;
char    chMCastAddress[16];
char    chLocalAddress[16];
u_short usPort;
char    szUserName  [ MAX_LINE_SIZE ];

if (argc<2)
{
    usage();
    exit(1);
}
iRet = WSAStartup(0x0101, &wsaData);
if (iRet != 0)
{
    printf ("\nError in WSAStartup initialization %d", iRet);
    return (1);
}
// Creating Udp Socket
if ((sock = socket(AF_INET, SOCK_DGRAM, 0)) == INVALID_SOCKET)
{
    printf ("\nError creating socket - %d", WSAGetLastError());
    return (1);
}
printf ("\nIP Multicast Address [Enter 0 for default]: ");
scanf ("%s",chMCastAddress);

if (*chMCastAddress == '0')
    strcpy (chMCastAddress, MULTICAST_ADDRESS);

printf ("IP Multicast Address: %s", chMCastAddress);
printf ("\nPort [Enter 0 for default]: ");
scanf("%d", &usPort);
if (!usPort)
    usPort = MULTICAST_PORT;

printf ("Port: %d", usPort);

printf("\nLocal IP interface: ");
scanf ("%s",chLocalAddress);
printf ("Local IP Interface: %s", chLocalAddress);
switch (argv[1][1])
{
case ('C'):
case ('c'):
    printf("\nIP Multicast Client Demo");
    printf("\nEnter your name: ");
    scanf ("%s", szUserName);
    printf("\nEnter your message (no longer than 255 characters)");
    printf("\nType !EXIT to quit client and !KILL to quit"\
                "server\n");
    sock_addr.sin_family = AF_INET;
    sock_addr.sin_addr.s_addr = inet_addr (chLocalAddress);
    sock_addr.sin_port = 0;

    // Binding the socket to the address
    if (bind(sock, (LPSOCKADDR) &sock_addr,
```

```
                sizeof(sock_addr))==SOCKET_ERROR)
        {
            printf ("\nError in binding the socket - %d",
                    WSAGetLastError());
            WSACleanup();
            return(1);
        }
        // Setting default IP interface for multicast
        MCastAddr.s_addr = inet_addr(chLocalAddress);
        if (setsockopt(sock, IPPROTO_IP, IP_MULTICAST_IF,
                       (char *) &MCastAddr,
                       sizeof(struct in_addr))==SOCKET_ERROR)
            printf("\nWarning: Could not set Default MCast Interface "\
                          "%d", WSAGetLastError());

        dst_sock_addr.sin_family = AF_INET;
        dst_sock_addr.sin_addr.s_addr = inet_addr(chMCastAddress);
        dst_sock_addr.sin_port = htons(usPort);
        while (TRUE)
        {
            printf("\nLine:>");
            gets(chLine);
            if (!strcmp(chLine, "!EXIT"))
                break;
            if (strlen(chLine)+strlen(szUserName)> MAX_BUFFER_SIZE)
            {
                printf("\nPlease enter a smaller message");
                continue;
            }
            if (!strcmp(chLine, "!KILL"))
                strcpy(chBuffer, chLine);
            else
                sprintf(chBuffer,"%s: %s", szUserName, chLine);

            if (sendto(sock, chBuffer, strlen(chBuffer), MSG_DONTROUTE,
                       (LPSOCKADDR) &dst_sock_addr,
                       sizeof(dst_sock_addr))==SOCKET_ERROR)
            {
                printf ("\nError in Sending data on the socket - %d",
                         WSAGetLastError());
                WSACleanup();
                return(1);
            }
        }
        printf("\nExiting...");
        break;
    case ('S'):
    case ('s'):
        printf("\nIP Multicast Server Demo");
        sock_addr.sin_family = AF_INET;
        sock_addr.sin_addr.s_addr = inet_addr(chLocalAddress);
        sock_addr.sin_port = htons(usPort);
```

```
        // Binding the socket to the address
        if (bind(sock, (LPSOCKADDR) &sock_addr,
                 sizeof(sock_addr))==SOCKET_ERROR)
        {
            printf ("\nError in binding the socket - %d",
                     WSAGetLastError());
            return(1);
        }
        // Joining a multicast group
        mreq.imr_multiaddr.s_addr = inet_addr(chMCastAddress);
        mreq.imr_interface.s_addr = inet_addr (chLocalAddress);
        if (setsockopt(sock,
                       IPPROTO_IP,
                       IP_ADD_MEMBERSHIP,
                       (char *) &mreq,
                       sizeof(struct ip_mreq)) == SOCKET_ERROR)
        {
            printf("\nError %d in joining Multicast Group",
                    WSAGetLastError());
            exit(1);
        }
        while(TRUE)
        {
            chBuffer[0]='\0';
            new_sock_addr_len = sizeof(new_sock_addr);
            if (((bytes=recvfrom(sock, chBuffer, MAX_BUFFER_SIZE, 0,
                    (LPSOCKADDR) &new_sock_addr,
                    (int *) &new_sock_addr_len)) == SOCKET_ERROR))
            {
                printf ("\nError in receiving data - %d",
                          WSAGetLastError());
                return(1);
            }
            if (!strcmp(chBuffer, "!KILL"))
            {
                printf ("\nClient wishes to kill server. Exiting..");
                // Dropping membership from the Mcast group
                setsockopt(sock, IPPROTO_IP, IP_DROP_MEMBERSHIP,
                                 (char *) &mreq, sizeof(mreq));
                break;
             }
             printf ("%s\n", chBuffer);
        }
        break;
    default:
        usage();
        WSACleanup();
        exit(1);
    }
    closesocket(sock);
    WSACleanup();
}
```

Allard 93. Allard, J. et al. "Plug into Serious Network Programming with the Windows Sockets API." *Microsoft Systems Journal* 8, no. 7 (July 1993).

Allard 94. Allard, J. *Advanced Internetworking with TCP/IP on Windows NT.* Microsoft Developer Network CD, Redmond, WA: Microsoft Corporation, spring 1994.

Baker 93. Baker, Steven. "An Overview of Network Programming Interfaces for Windows and Windows NT." *Microsoft Systems Journal* 8, no. 11 (1993).

Comer 91a. Comer, Douglas E. *Internetworking with TCP/IP.* Vol. I. Englewood Cliffs, NJ: Prentice-Hall, Inc., 1991.

Comer 91b. Comer, Douglas E., and David L. Stevens, *Internetworking with TCP/IP.* Vol II. Englewood Cliffs, NJ: Prentice-Hall, Inc., 1991.

Custer 93. Custer, Helen. *Inside Windows NT.* Redmond, WA: Microsoft Press, 1993.

Davis 93. Davis, Ralph. *Windows Network Programming.* Reading, MA: Addison-Wesley, 1993.

Deering 89. Deering, Steve. "IP Multicast Extensions for 4.3 BSD UNIX and related systems (MULTICAST 1.2 Release)," June 24, 1989, via anonymous ftp to `gregorio.stanford.edu, directory: / vmtp-ip.`

Digital 82. Digital Equipment Corporation. *Programming in VAX-11.* Maynard, MA: Digital Equipment Corporation, 1982.

Finnegan 94. Finnegan, James. "Building Windows NT-Based Client/Server Applications Using Remote Procedure Calls." *Microsoft Systems Journal* 9, no. 10 (1994).

Geary 90. Geary, Michael. "An Introduction to Microsoft Windows Version 3.0: A Developer's Viewpoint." *Microsoft Systems Journal* 5, no. 4 (July 1990).

Hall 91. Hall, William S. "Adapt Your Program for Worldwide Use with Windows Internationalization Support." *Microsoft Systems Journal* (November/December 1991): 29–58.

Hall 92. Martin et al. "Windows Sockets: An Open Interface for Network Programming under Microsoft Windows." winsock@microdyne.com Version 1.1 (January 1993).

IBM 84a. "IBM Local Area Network Technical Reference." IBM part no. SC30–3383–03.

IBM 84b. "NetBIOS Application Development Guide." IBM part no. S68X–2270.

Johnson 94. Johnson, Margaret et al. "Working Together: NetWare and Windows Sockets." *NetWare Technical Journal* (January–February 1994).

King 94. King, Adrian. "Examining the Peer-to-Peer Connectivity and Multiple Network Support of Chicago." *Microsoft Systems Journal* (November 1994).

Liu 94. Liu, Yusheng, and Doan Hoang. "OSI RPC Model and Protocol." *Computer Communications* (Butterworth-Heinemann Ltd.) vol. 17, no. 1 (January 1994).

Manson 89. Manson, Carl, and Ken Thurber. "Remote Control," *Byte* (July 1989): 235–239.

MS 90. Microsoft Corporation, Intel. "Server Message Block (SMB) Specifications." Redmond, WA: Microsoft Corporation.

MS 92. Microsoft Corporation. "Windows 3.1 Device Driver Kit." Redmond, WA: Microsoft Corporation.

MS 93a. Microsoft Corporation. *Microsoft Win32 API Programmer's Reference*. Redmond, WA: Microsoft Press, July 1993a.

MS 93b. Microsoft Corporation. *Microsoft Windows NT Device Development Kit*. Redmond, WA: Microsoft Press, July 1993b.

MS 93c. Microsoft Corporation. *Network Drivers Interface Specification*. Redmond, WA: Microsoft Press, July 1993c.

MS 93d. Microsoft Corporation. *Microsoft Windows for WorkGroup Software Development Kit*. Redmond, WA: Microsoft Press, 1992–1993d.

MS-WFW 93. Microsoft Corporation. *Microsoft Windows for WorkGroups Version 3.1: Architecture Highlights*. Redmond, WA: Microsoft Product Services, 1993.

MS-WinSock 94. Microsoft, "Multicast Extensions to Windows Sockets for Win32," `ftp.microsoft.com, directory: /bussys/WinSock/ms-ext`

Novell 93. Novell, Inc. *NetWare Client SDK for DOS, OS/2 and Windows*. Provo, UT: Novell, Inc., 1993.

Novell 95. Novell, Inc. "Novell Software Development Kits." Volume 1–5, Part no. 884–000120–003.

Quarterman 93. Quarterman, John S., and Susan Wilhelm. *UNIX, POSIX, and Open Systems: The Open Standards Puzzle*. Reading, MA: Addison-Wesley, 1993.

Rose 90. Rose, M.T. *The Open Book: A Practical Perspective on OSI*. Englewood Cliffs, NJ: Prentice-Hall, Inc., 1990.

Rosenbary 93. Rosenbary, Ward, and Jim Teague. *Distributed Applications Across DCE and Windows NT*. Sebastopol, CA: O'Reilly & Associates, Inc., 1993.

Schwaderer 88. Schwaderer, W. David. *C Programmer's Guide to NetBIOS*. Indianapolis, IN: Howard W. Sams & Company, 1988.

Sheldon 91. Sheldon, Kenneth M. "ASCII Goes Global." *Byte* (July 1991): 108–16.

Sinha 92a. Sinha, Alok K. "Client Server Computing: Current Technology Review." *Communications of the ACM* 35, no. 7 (July 1992a).

Sinha 92b. Sinha, Alok K. *NetBIOS Programming*. Microsoft Windows 3.1 Developer's Workshop. Redmond, WA: Microsoft Press, 1992b.

Stevens 90. Stevens, W.R. *UNIX Network Programming*. Englewood Cliffs, NJ: Prentice-Hall, Inc., 1990.

Tanenbaum 88. Tanenbaum, Andrew S. *Computer Networks*. Englewood Cliffs, NJ: Prentice-Hall, Inc., 1988.

Unicode 92. The Unicode Consortium. *The Unicode Standard: World-Wide Character Encoding*, Version 1.0, 2 vols. Reading, MA: 1992.

Yao 92. Yao, Paul. "Windows 3.0 Memory Management: Supporting Disparate 80x86 Architectures." *Microsoft Systems Journal* 5, no. 4 (1992).

Index

A

accept() function, in Windows Sockets, 49–52
access control entries (ACEs), 38–40
access control lists (ACLs), 38–40
access controls, 188–91
addresses
  associating with bindings or sockets, 202, 206–19
  mapping files into address spaces, 73–75
  Windows Socket address resolution in TCP/IP environments, 229–32
anonymous pipes, 366–73
ANSI/IEEE 802–1985 Standard, 14–15
application environments, in NetBIOS programming, 437–40
application programming interfaces (APIs)
  DOS APIs, 7
  I/O (input/output) APIs, 7
  IPX programming APIs, 518–20
  MNet APIs, 112–13
  Name Service Provider APIs for Windows Sockets, 280–82
  Novell communication APIs, 277, 278
  OS/2 APIs, 7
  POSIX APIs, 8
  remote procedure call management APIs, 182–84
  SPX programming APIs, 556–57
  Win16 APIs, 6
  Win32 APIs, 6–7, 82
  Windows and, 4
  Windows NT APIs, 8, 23
  WinNet APIs, 19, 31–33, 109–13
  WSAAsyncSelect() API, 267–77
  See also functions
application-managed handles, in remote procedure calls, 173–78
applications
  debugging in NetBIOS programming, 583–93
  performance of, 2–3
  thread usage and application overhead, 51
  See also Echo Servers; sample programs
architecture
  of Mailslots, 395–97
  of Windows NT, 21–23
arrays, in remote procedure calls, 127–29, 137
asynchronous database functions, in Windows Sockets, 262–67
asynchronous I/O, 65–69
  completion routines, 69–72
  with OVERLAPPED structure, 65–69

asynchronous NetBIOS commands, 434–35, 462–80, 481
asynchronous pipes, 356–57, 358
asynchronous processing
  of IPX event control blocks (ECBs), 524–28
  of SPX event control blocks (ECBs), 557–59
asynchronous Windows Socket operations, 247–54
  asynchronous select() API, 267–77
  blocking sockets, 247, 260–62
  using nonblocking sockets, 247–49
  using nonblocking sockets with connect(), 249
  using select() with accept(), 249–52
  using select() with recv() or recvfrom(), 252–53
  using select() with send() or sendto(), 253–54
auditing, in Windows NT, 40
autobinding handles, in remote procedure calls, 170–73

B

backup domain controllers (BDCs), 36, 161–62
backup systems, in Windows NT, 10
base data types, in remote procedure calls, 126–27
Berkeley sockets, 199, 202–3
binding handles, 170–78
bindings, associating addresses with, 202, 206–19
brackets ([ ]), in remote procedure calls, 123
broadcasts, 394–95

C

C structures, 131–32
callback functions, 178–82
carrier sense multiple access with collision detection (CSMA/CD), 15
Cell Directory Service (CDS), 163
child processes, 53–55
cleanup functions, in Windows Sockets, 56–58
Client Services for NetWare (CSNW), 30, 512–13, 514
client/server systems (CSSs), 8–11
  client characteristics, 8
  creating Mailslot clients, 401–4
  creating Mailslot servers, 397–401
  defined, 8
  memory-mapped file I/O in, 73
  overview of, 1–2
  server characteristics, 8–9
  simple Named Pipes example, 305–6
  thread usage guidelines, 50–51
  transaction-oriented applications using Named Pipes, 333–43
  Windows NT as a server platform, 9–11
  See also servers
commands in NetBIOS programming, 431–35, 462–81, 583–93
  asynchronous commands, 434–35, 462–81
  NCB.ADAPTER.STATUS command, 584–89
  NCB.ENUM command, 583–84
  NCB.SESSION.STATUS command, 589–93
  overview of, 431–35
communication methods, 11–19
  connectionless services, 13, 427–28
  connection-oriented services, 11–13, 427
  data encapsulation, 11, 12
  datagram-based communications Mailslots and, 394–95

in NetBIOS programming, 427–28, 487–96
overview of, 13, 16–17
sending IPX datagram packets, 536–37
defined, 3
local area networks (LANs) and, 394–95
memory-mapped file I/O mechanism, 19, 72–79
overview of, 17–19
peer-to-peer communication, 11, 12
protocol stacks and, 11
terms and concepts, 11–15
virtual circuits, 11–12
wide area networks (WANs) and, 15–16, 394–95
in Windows and Windows NT, 17–19
WinNet APIs, 19, 31–33, 109–13
See also IPX/SPX; Mailslots; Named Pipes; NetBIOS; Open Systems Interconnection; remote procedure calls; Windows Sockets
connect() function, 249
connectionless communication services, 13, 427–28
connection-oriented communication services, 11–13, 427
context handles, in remote procedure calls, 175–78
Control-C handlers, 79–81
critical-section objects, 61–64
CSMA/CD (carrier sense multiple access with collision detection), 15

D

data encapsulation, 11, 12
database functions, in Windows Sockets, 262–67
datagram-based communications
datagram sockets, 238–45
Mailslots and, 394–95
in NetBIOS programming, 427–28, 487–96
overview of, 13, 16–17
sending IPX datagram packets, 536–37
See also IPX/SPX programming
DCE (Distributed Computing Environment), 115, 118, 163
debugging
in NetBIOS programming, 583–93
thread usage and debugging overhead, 51
device drivers
NetWare open datalink interface (ODI) drivers, 108
Windows operating modes and, 100, 101, 103
WinNet drivers in Windows NT, 19, 31–33
Distributed Computing Environment (DCE), 115, 118, 163
DLINK functions, in remote procedure calls, 140–41
domain controllers (DCs), 35–37, 161–62
domain name system (DNS), 230–31, 232
domains, in remote procedure calls, 161–62
DOS
DOS APIs, 7
DOS Protected Mode Interface (DPMI), 100
Virtual DOS Machine (VDM) in Windows NT, 25–26, 42
versus Windows, 4
Windows real mode and, 100
dynamic link libraries (DLLs), 19, 24–25, 31–33

E
Echo Servers
  defined, 21
  Mailslots example, 408–23
  Named Pipes example, 373–85
  NetBIOS example, 496–506
  remote procedure call (RPC) example, 191–95
  Windows NT example, 81–97
  Windows Socket example, 282–88
  See also sample programs; servers
endpoints, in remote procedure calls, 152–55
enhanced mode
  in Windows, 103–5
  in Windows for Workgroups, 100, 106–8
entry point vectors (EPVs), 158–60
error handling
  in remote procedure calls, 184–88
  in Windows NT, 43–44
  in Windows Sockets, 259–60
  event control blocks (ECBs)
  in IPX programming, 520–28
  in SPX programming, 557–59
events, 51–52, 59–61, 64, 347–56
examples. *See* Echo Servers; sample programs
exception handling
  in remote procedure calls, 184–88
  in Windows NT, 44–46
Executive, in Windows NT, 22, 23–24
extension functions for Windows Sockets, 255–77
external data representation (XDR), 116–17

F
FDDI (fiber distributed data interface), 15
FIFO (first in, first out) communication method, 18
file mapping, 19, 72–79
first_is() attribute, in remote procedure calls, 129
full pointers, in remote procedure calls, 134
functions
  for asynchronous Windows Socket operations, 249–54
  callback functions in remote procedure calls, 178–82
  for datagram sockets, 241
  extension functions for Windows Sockets, 255–77
  IPX functions, 518–20
  in NetBIOS programming, 429–31
  nonblocking sockets with connect(), 249
  process-related functions in Windows NT, 48–49
  SPX functions, 556–57
  thread-related functions in Windows NT, 48–49
  wait functions for synchronization objects, 52
  See also application programming interfaces

G
graphical user interfaces (GUIs), 4, 5–6

H
hardware abstraction layer (HAL), 22, 23

I
I/O (input/output) APIs, 7
  See also overlapped I/O
I/O Manager, 24
Intel processors, 100–105

interprocess communication (IPC), 1, 40–41, 105
interprocess and intraprocess synchronization, 51–64
IPX/SPX programming, 277–80, 511–82
  IPX programming, 517–54
    event control blocks (ECBs) and asynchronous processing, 520–28
    initializing and deinitializing IPX applications, 528–29
    IPX programming APIs, 518–20
    life cycle of IPX sockets, 517–18
    locating servers receiving IPX packets, 532–35
    NetWare and, 511–14, 517–18
    opening and closing IPX sockets, 529–30
    overview of, 517–18
    receiving IPX packets, 531–32
    sample program, 538–54
    sending datagram packets, 536–37
    Windows Sockets and, 511
  NetWare and, 511–14, 517–18
  NWLink software, 512–14
  overview of, 18, 19, 582
  SPX programming, 554–82
    breaking connections, 564–65
    establishing connections, 563–64
    event control blocks (ECBs) and asynchronous processing, 557–59
    initializing SPX applications, 559–60
    life cycle of SPX sockets, 554–55
    listening for connection requests, 560–63
    opening and closing SPX sockets, 560
    overview of, 554–57
    sample program, 567–82
    sending and receiving data, 565–67
    SPX programming APIs, 556–57
    Windows Sockets and, 511, 555
  in Windows NT, 40–41
  Windows Sockets IPX/SPX-based application interoperability, 277–80
  Windows for Workgroups and, 105, 107

K

kernel, in Windows NT, 23–24

L

last_is() attribute, in remote procedure calls, 129
length_is() attribute, in remote procedure calls, 128
LMHOSTS file, 231
local area networks (LANs), 1, 394–95
local procedure calls (LPCs), 24
logical (binding) handles, 170–78
logical link controls (LLCs), 14–15

M

Mailslots, 393–424
  architecture of, 395–97
  creating Mailslot clients, 401–4
  creating Mailslot servers, 397–401
  Echo Server example, 408–23
  name formats, 396–97
  overview of, 18, 19, 41, 393–95, 424
  variable-length messages and, 405–7
  in Windows for Workgroups, 105, 423–24
manager entry point vectors (EPVs), 158–60

MANs (metropolitan area networks), 15
mapping files, 19, 72–79
max_is() attribute, in remote procedure calls, 129
media access control (MAC) layer, 15
memory
  memory protection in Windows NT, 40
  memory-mapped file I/O, 19, 72–79
  Windows enhanced mode and, 103
  Windows real mode and, 101
message-driven applications, 267–77, 405–7
metropolitan area networks (MANs), 15
microprocessors, 4, 5, 100–105
Microsoft Windows, 99–114
  communication methods, 17–19
  defined, 3
  DOS Protected Mode Interface (DPMI), 100
  dynamic link libraries (DLLs), 24–25
  enhanced mode, 103–5
  history of, 4
  limitations of, 4
  NetBIOS programming in, 507–10
  overview of, 1–2, 99–100, 114
  protected mode, 100–101, 103
  real mode, 100–101, 103
  remote procedure call programming in, 196–98
  standard mode, 102–3
  system virtual machine (SVM), 103–4
  virtual device drivers (VxDs), 104–5
  versus Windows NT, 5, 24–25
  Windows Sockets in, 298–99
  See also Microsoft Windows for Workgroups
Microsoft Windows NT, 21–97
  access control lists (ACLs) and access control entries (ACEs), 38–40
  accessing network resources, 30–34
  architecture features, 21–23
  asynchronous I/O completion routines, 69–72
  asynchronous I/O with OVERLAPPED structure, 65–69
  auditing feature, 40
  backup domain controllers (BDCs), 36, 161–62
  Client Services for NetWare (CSNW), 30, 512–13, 514
  communication methods, 17–19
  Control C handlers, 79–81
  critical-section objects, 61–64
  defined, 3
  developers and, 6–8
  domain controllers (DCs), 35–37, 161–62
  dynamic link libraries (DLLs), 19, 24–25, 31–33
  Echo Server example, 81–97
  error handling, 43–44
  events, 51-52, 59–61, 64
  hardware abstraction layer (HAL), 22, 23
  history of, 5
  I/O Manager, 24
  interprocess communication in, 40–41
  interprocess and intraprocess synchronization, 51–64
  kernel, 23–24
  local procedure calls (LPCs), 24
  logon facility, 37–38
  memory protection security feature, 40
  memory-mapped file I/O, 19, 72–79
  multiple UNC providers (MUPs), 34, 35

multiple VDM (MVDM), 25
multitasking in, 26–27
mutex objects, 51–52, 55–59, 64
NetBIOS support, 40–41, 425, 435–37
network binding in, 29
Network Driver Interface Specification (NDIS), 28
networking components, 27–30
NT File System (NTFS), 10
Object Manager, 23
Open Systems Interconnection (OSI) model and, 40–41
overlapped I/O feature, 64–72
overview of, 1–2, 5, 9–11, 21–23, 97
primary domain controllers (PDCs), 36, 161–62
Process Manager, 23–24
process-related functions, 48–49
protected systems, 24–27
redirector and server, 28–29, 30–31, 33–34, 42
remote access service (RAS), 28
scalability of, 10
security features, 5, 23, 37–40
Security Reference Manager, 23
semaphore objects, 51–55, 64
server message block (SMB) protocol, 28–29, 30–31, 304–5
as a server platform, 9–11
structured exception handling, 44–46
threads, 46–51
transport driver interface (TDI), 28–29, 30
uniform naming convention (UNC), 33–34
user interface and, 5–6
user logons, 37–38
virtual device drivers (VxDs), 42
Virtual DOS Machine (VDM), 25–26, 42
Virtual Memory Manager, 24
Win32 APIs, 6–7, 82
Win32 subsystem, 25
versus Windows, 5, 24–25
Windows NT APIs, 8, 23
Windows NT Executive, 22, 23–24
Windows on Win32 (WOW) subsystem, 26–27, 42
WinNet APIs, drivers, and DLLs, 19, 31–33
Microsoft Windows NT Advanced Server (NTAS)
domain controllers (DCs), 35–37, 161–62
overview of, 34–37
versus Windows NT Workstation, 3
Windows for Workgroups and, 107, 109
Microsoft Windows for Workgroups, 105–13
accessing network resources, 109–13
enhanced mode, 100, 106–8
interprocess communication interfaces and, 105
Mailslots in, 105, 423–24
MNet APIs, 112–13
multiprotocol support, 107
NetWare client support, 107–8
overview of, 105–6
peer-to-peer server in, 108
redirectors, 105, 106, 107
standard mode, 100, 109
WinNet APIs, 109–13
See also Microsoft Windows
min_is() attribute, in remote procedure calls, 129
MNet APIs, 112–13
multicasting
overview of, 394–95
with Windows Sockets, 595–603
multiple UNC providers (MUPs), 34, 35

multiple VDM (MVDM), 25
multitasking, 4, 5, 26–27, 104
mutex objects, 51–52, 55–59, 64

N
name formats, for Mailslots, 396–97
name management services
  Name Service Provider APIs for Windows Sockets, 280–82
  in NetBIOS programming, 428–29, 432–34, 440–49
  in remote procedure calls, 145–52, 160–70
    clients not using Name Service, 147–48
    locating servers with Name Service, 145–46, 160–70
    locating servers without Name Service, 145–52
    Name Service APIs, 164–65
    servers exporting details into Name Service, 165–67
    servers not using Name Service, 148–52
Named Pipes, 301–91
  accepting incoming open requests from clients, 309–17
  anonymous pipes, 366–73
    creating, 367
    overview of, 366–67
    reading or writing, 368–73
  asynchronous pipes, 356–57, 358
  changing Named Pipe modes at run time, 344–46
  creating, 306–9
  Echo Server example, 373–85
  getting information available on, 346
  multiple instance management, 357–64
    creating asynchronous non-blocking pipes, 358
    creating Named Pipes in overlapped I/O mode, 358–64
    creating one thread per Named Pipe instance, 358
    overview of, 357–58
  opening pipes, 303, 317–22
  overlapped I/O and, 347–56, 358–64
  overview of, 18, 19, 41, 301–3, 390–91
  "peeking" into, 328–32
  platforms supporting, 303–5
  reading or writing, 322–28
  security considerations, 364–66
  server message block (SMB) protocol and, 304–5
  simple client/server system example, 305–6
  transaction-oriented client/server applications, 333–43
  versus UNIX pipes, 389–90, 391
  Win32 events and, 347–56
  in Windows, 385–89
  Windows for Workgroups and, 105
NCA (Network Computing Architecture) remote procedure call, 117–18
NCB.ADAPTER.STATUS command, 584–89
NCB.ENUM command, 583–84
NCB (NetBIOS Control Block) structures, 430–35
NCB.SESSION.STATUS command, 589–93
NDIS (Network Driver Interface Specification), 28
NDR (Network Data Representation), 119–20
nested arrays, in remote procedure calls, 137
NetBEUI protocol, 9, 41, 426, 427
NetBIOS programming, 425–510, 583–93

commands, 431–35, 462–81, 583–93
asynchronous commands, 434–35, 462–81
NCB.ADAPTER.STATUS command, 584–89
NCB.ENUM command, 583–84
NCB.SESSION.STATUS command, 589–93
overview of, 431–35
connectionless service, 427–28
connection-oriented services, 427
datagram services, 427–28, 487–96
debugging applications, 583–93
Echo Server example, 496–506
functions, 429–31
history of, 425–26
name management services, 428–29, 432–34, 440–49
NetBIOS Control Block (NCB) data structures, 430–35
on-the-wire data format, 427
overview of, 18, 19, 40–41, 425–31, 510
post routines, 465–76, 509–10
session services
handling multiple clients and large data transfers, 480–87
overview of, 427, 449–62
setting NetBIOS application environments, 437–40
virtual circuits in, 427–28, 433–34
in Windows, 507–10
Windows NT support for, 40–41, 425, 435–37
Windows for Workgroups and, 105, 107
NetWare, 511–18
Client Services for NetWare (CSNW) in Windows NT, 30, 512–13, 514
client support in Windows for Workgroups, 107–8
IPX/SPX programming and, 511–14, 517–18
NetWare Client for Windows NT, 511
NetWare services, 515–16
Novell communication APIs, 277, 278
NWLink software, 512–14
open datalink interface (ODI) drivers, 108
overview of, 512–14
service advertising protocol (SAP), 517–18, 533–35
Windows Socket/IPX interoperability, 277–80
See also IPX/SPX programming; Novell
network binding, 29
Network Computing Architecture (NCA) remote procedure call, 117–18
Network Data Representation (NDR), 119–20
Network Driver Interface Specification (NDIS), 28
network resources
accessing in Windows NT, 30–34
accessing in Windows for Workgroups, 109–13
nonblocking sockets, 247–49
Novell. See IPX/SPX programming; NetWare
NT File System (NTFS), 10
NWLink software, 512–14

O

Object Manager, 23
on-the-wire data format, in NetBIOS programming, 427
open datalink interface (ODI) drivers, in NetWare, 108

Open Network Computing (ONC) remote procedure call, 116–17, 118
Open Systems Interconnection (OSI) model, 11, 13–15, 27–30, 40–41, 118
OS/2 operating system, 5, 7
out-of-band (OOB) data processing, with Windows Sockets, 254–55
overlapped I/O
  asynchronous I/O completion routines, 69–72
  asynchronous I/O with OVERLAPPED structure in Windows NT, 65–69
  Named Pipes and, 347–56, 358–64

P

peer-to-peer communications, 11, 12
peer-to-peer server, in Windows for Workgroups, 108
pipes. *See* Named Pipes
pointers in remote procedure calls, 132–37
POSIX interface, 5, 8
post routines, in NetBIOS programming, 465–76, 509–10
primary domain controllers (PDCs), 36, 161–62
primitive binding handles, in remote procedure calls, 173–75
process synchronization in Windows NT, 51–64
processors, 4, 5, 100–105
process-related functions, in Windows NT, 48–49
protected mode, in Windows, 100–101, 103
protected systems, in Windows NT, 24–27
protocol stacks, 11
protocols
  multiprotocol support in Windows for Workgroups, 107
  NetBEUI, 9, 41, 426, 427
  remote procedure calls and, 155–57
  server message block (SMB) protocol, 28–29, 30–31, 304–5
  service advertising protocol (SAP), 517–18, 533–35
  SNMP (Simple Network Management Protocol), 10
  TCP/IP, 9, 13, 229–32
  Windows Sockets and, 200–201
  See also IPX/SPX programming; NetBIOS programming

R

RAM. *See* memory
RAS (Remote Access Service), 28, 200
real mode, in Windows, 100–101, 103
recv() or recvfrom() functions, in Windows Sockets, 252–53
redirectors
  in Windows NT, 28–29, 30–31, 33–34, 42
  in Windows for Workgroups, 105, 106, 107
reference pointers, in remote procedure calls, 135–36
Remote Access Service (RAS), 28, 200
remote procedure calls (RPCs), 2, 17–19, 41, 115–98
  Apollo Computer Network Computing Architecture (NCA) RPC, 117–18
  callback functions, 178–82
  client/server examples, 121–41
    array transmission, 127–29, 137
    base data types, 126–27

C structures, 131–32
DLINK functions, 140–41
full pointers, 134
nested arrays, 137
overview of, 124–25
pointer defaults, 136–37
pointers, 132–37
reference pointers, 135–36
simple example, 121–24
string transmission, 129–31
transmit_as attribute, 139–41
unions, 137–39
unique pointers, 134–35
connecting RPC clients and servers, 141–78
application-managed handles, 173–78
autobinding handles, 170–73
binding handles, 170–78
clients not using Name Service, 147–48
context handles, 175–78
domains and domain controllers (DCs), 161–62
locating servers with Name Service, 145–46, 160–70
locating servers without Name Service, 145–52
manager entry point vectors (EPVs) and, 158–60
Name Service APIs, 164–65
overview of, 141–45
primitive binding handles, 173–75
servers exporting details into Name Service, 165–67
servers not using Name Service, 148–52
servers supporting multiple objects, 157–60
servers supporting multiple protocols, 155–57
user-defined handles, 175
well-known endpoints versus dynamic endpoints, 152–55
Distributed Computing Environment (DCE) and, 115, 118, 163
Echo Server example, 191–95
error and exception handling, 184–88
external data representation (XDR) and, 116–17
history of, 116–18
Microsoft RPC overview, 115–16, 118–20
Name Service, 145–52, 160–70
Network Data Representation (NDR) and, 119–20
OSI session layer and, 118
overview of, 2, 17–19, 41, 115–16, 198
protocols supported by, 156
RPC management APIs, 182–84
security and access controls, 188–91
square brackets ([ ]) in, 123
Sun Microsystems Open Network Computing (ONC) RPC, 116–17, 118
in Windows, 196–98
Windows for Workgroups and, 105
RISC processors, 5
ROM-BIOS, 25

S

sample programs
downloading, 2
IPX example, 538–54
NCB.ADAPTER.STATUS command example, 584–89
NCB.ENUM command example, 583–84
NCB.SESSION.STATUS command example, 589–93

sample programs,
NCB.SESSION.STATUS,
continued
SPX example, 567–82
Windows Sockets multicasting
example, 600–603
See also Echo Servers
SAP (service advertising protocol),
517–18, 533–35
security
memory protection feature in
Windows NT, 40
Named Pipes and, 364–66
in remote procedure calls, 188–91
user logons in Windows NT, 37–38
in Windows NT, 5, 23, 37–40
select() function in Windows
Sockets, 249–54, 267–77
semaphore objects, 51–55, 64
send() or sendto() function, in
Windows Sockets, 253–54
server message block (SMB) proto-
col, 28–29, 30–31, 304–5
servers
accepting calls from clients with
stream sockets, 219–24
characteristics of, 8–9
creating Mailslot servers, 397–401
locating servers receiving IPX
packets, 532–35
Windows NT as a server platform,
9–11
See also client/server systems; Echo
Servers
service advertising protocol (SAP),
517–18, 533–35
session services, in NetBIOS
programming, 427, 449–62,
480–87
size_is() attribute, in remote
procedure calls, 128
SMP (symmetric multiprocessing),
10
SNMP (Simple Network
Management Protocol), 10
sockets. *See* Windows Sockets
SPX programming. *See* IPX/SPX
programming
square brackets ([ ]), in remote
procedure calls, 123
standard mode
in Windows, 102–3
in Windows for Workgroups, 100,
109
startup functions, in Windows
Sockets, 256–58
stream sockets
clients connecting to servers, 224–29
sending and receiving data on,
232–38, 239
servers accepting calls from clients,
219–24
See also Windows Sockets
strings, in remote procedure calls,
129–31
symmetric multiprocessing (SMP), 10
synchronization in Windows NT,
51–64
system virtual machine (SVM),
103–4

T

TCP/IP protocol, 9, 13, 229–32
threads
creating one thread per Named
Pipe instance, 358
mutex objects and, 51–52, 55–59
usage guidelines for client/server
systems, 50–51
in Windows NT, 46–51
in Windows NT Echo Server
example, 92–95
transaction-oriented client/server
applications using Named
Pipes, 333–43

transmit_as attribute, in remote procedure calls, 139–41
transport driver interface (TDI), 28–29, 30
TSRs (terminate-and-stay-resident programs), 100, 101

U

uniform naming convention (UNC), 33–34
uninterruptible power supplies (UPSs), 10
unions, in remote procedure calls, 137–39
unique pointers, in remote procedure calls, 134–35
UNIX
  POSIX interface, 5, 8
  UNIX pipes, 389–90, 391
  See also POSIX interface
unNamed Pipes. *See* anonymous pipes
user interfaces, 4, 5–6
user logons, in Windows NT, 37–38
user-defined handles, in remote procedure calls, 175

V

variable-length messages, Mailslots and, 405–7
Virtual 80386 (V86) mode, 103
virtual circuits, 11–12, 427–28, 433–34
virtual device drivers (VxDs)
  in Windows, 104–5
  in Windows NT, 42
Virtual DOS Machine (VDM), 25–26, 42
Virtual Memory Manager, 24

W

wait functions, for synchronization objects, 52
well-known endpoints, in remote procedure calls, 152–55
wide area networks (WANs), 15–16, 394–95
Win16 APIs, 6
Win32 APIs, 6–7, 82
Win32 events, 51–52, 59–61, 64, 347–56
Win32 Service. *See* Echo Servers
Win32 subsystem, in Windows NT, 25
Windows. *See* Microsoft Windows; Microsoft Windows NT; Microsoft Windows for Workgroups
Windows Internetwork Name Service (WINS), 230, 231
Windows NT APIs, 8, 23
Windows Sockets, 199–299, 595–603
  address resolution in TCP/IP environments, 229–32
    with domain name system (DNS), 230–31, 232
    with LMHOSTS file, 231
    with Windows Internetwork Name Service (WINS), 230, 231
  asynchronous operations, 247–54
    blocking sockets, 247, 260–62
    using nonblocking sockets, 247–49
    using nonblocking sockets with connect(), 249
    using select() with accept(), 249–52
    using select() with recv() or recvfrom(), 252–53
    using select() with send() or sendto(), 253–54
  datagram sockets, 238–45
  Echo Server example, 282–88

Windows Sockets, *continued*
extension functions, 255–77
asynchronous database functions, 262–67
error-handling functions, 259–60
for handling blocked sockets, 260–62
overview of, 255–56, 257
for socket-state-driven and message-driven applications (WSAAsyncSelect() API), 267–77
startup and cleanup functions, 256–58
history of, 199–200
IPX/SPX programming and, 511, 555
IPX/SPX-based application interoperability, 277–80
multicasting with, 595–603
joining and leaving multicasting groups, 596–97
overview of, 595
receiving multicast messages, 597
sample program, 600–603
sending multicast messages, 598–600
Name Service Provider APIs, 280–82
out–of-band (OOB) data processing, 254–55
overview of, 18, 19, 41, 199–203, 299
protocols and, 200–201
Remote Access Service (RAS) and, 200
sending and receiving data
on datagram sockets, 238–45
on stream sockets, 232–38, 239
sockets
associating addresses with bindings or sockets, 202, 206–19
Berkeley sockets, 199, 202–3
blocking sockets, 247, 260–62
closing, 246–47
creating, 203–6
datagram sockets, 238–45
defined, 201
types of, 204–5
stream sockets
clients connecting to servers, 224–29
sending and receiving data on, 232–38, 239
servers accepting calls from clients, 219–24
terminating connections, 246–47
in Windows, 298–99
Windows for Workgroups and, 105
Windows on Win32 (WOW) subsystem, 26–27, 42
WinNet APIs
in Windows NT, 19, 31–33
in Windows for Workgroups, 109–13
WSAAsyncSelect() API, 267–77

X

XDR (external data representation), 116–17

Unix and Open System Series

Programming under Mach	Joseph Boykin David Kirschen Alan Langerman Susan LoVerso
Practical Internetworking with TCP/IP and UNIX	Smoot Carl-Mitchell John S. Quarterman
Frontiers of Electronic Commerce	Ravi Kalakota Andrew Whinston
Network Management: A Practical Perspective, 2nd Edition	Allan Leinwand Karen Fang Conroy
The Internet Connection: System Connectivity and Configuration	John S. Quarterman Smoot Carl-Mitchell
UNIX, POSIX, and Open Systems: The Open Standards Puzzle	John S. Quarterman Susanne Wilhelm
A Quarter Century of UNIX	Peter H. Salus
Casting the Net: From ARPANET to Internet and Beyond	Peter H. Salus
Network Programming in Windows NT™	Alok Sinha

Series Editors
Marshall Kirk McKusick
John S. Quarterman

Other Related Titles

UNIX for the Impatient, Second Edition	Paul W. Abrahams Bruce R. Larson
Distributed Systems: Concepts and Design, Second Edition	George Coulouris Jean Dollimore Tim Kindberg
Operating System Concepts, Fourth Edition	Abraham Silberschatz Peter Galvin
A Practical Guide to the UNIX System, Third Edition	Mark G. Sobell

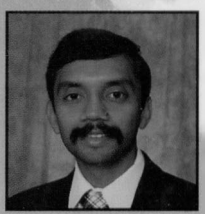
Network Programming in Windows NT™

Alok K. Sinha
Microsoft

Powerful IPC programming paradigms are the building blocks of today's client/server applications for Windows and Windows NT. This book explores the challenges that network developers face when developing network-aware or client/server programs. ***Network Programming in Windows NT*** focuses on the different networking/communication methods, such as RPC, Sockets, Named Pipes, MailSlots, and NetBIOS, and presents each method with beginners in mind. Sinha discusses the advantages and disadvantages of each communication method so that developers can decide which method works best for new or modified applications. Code fragments throughout the book illustrate the basic design concepts.

Highlights

- Provides an overview of Windows and Windows NT architecture and shows how Windows NT IPC mechanisms can be used to build client/server applications
- Illustrates harnessing the power of RPC and interaction with DCE
- Shows how to use Window Sockets
- Covers SPX/IPX programming, which allows applications to access applications in a NetWare environment
- Encompasses NetBIOS programming for building applications
- Includes the MailSlots programming interface, which accords a simple application level interface



Alok K. Sinha, a software design engineer at Microsoft, specializes in distributed computing and has worked on directory service technology for Microsoft Windows NT. The author is currently a member of the Broadband Media Applications team at Microsoft.

Code is available free via anonymous ftp `ftp aw.com` in the directory `cseng/authors/sinha/windowsnt`. Access the latest information about Addison-Wesley books from our Internet gopher site or our World Wide Web page:

```
gopher aw.com
http://www.aw.com/cseng/
```

9 780201 590562 90000

ISBN 0-201-59056-5

Addison-Wesley Publishing Company